Machine Learning Handbook

Math · data · algorithms · deep learning · ML system design · MLOps · case studies — with runnable Python

20 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — What Machine Learning Is**
   - 1.1 The AI family tree
   - 1.2 Learning from data vs writing rules
   - 1.3 How a model learns: training, inference and feedback loops
   - 1.4 Types of learning by supervision
   - 1.5 Classification vs regression
   - 1.6 Key terms
   - 1.7 Batch vs online learning
   - 1.8 Instance-based vs model-based learning
   - 1.9 The end-to-end ML workflow
   - 1.10 The main challenges of machine learning
   - 1.11 A first end-to-end project with scikit-learn
   - Chapter TL;DR
2. **Chapter 2 — The Math You Need for Machine Learning**
   - 2.1 Vectors
   - 2.2 Matrices and matrix multiplication
   - 2.3 Eigenvalues, eigenvectors and SVD (intuition)
   - 2.4 Derivatives, partial derivatives and gradients
   - 2.5 Gradient descent
   - 2.6 Random variables and distributions
   - 2.7 Conditional probability and Bayes' theorem
   - 2.8 Descriptive statistics and standardisation
   - 2.9 Correlation vs causation
   - 2.10 Sampling and the central limit theorem
   - 2.11 Hypothesis tests, p-values and confidence intervals
   - 2.12 Information theory: entropy, cross-entropy and KL divergence
   - Chapter TL;DR
3. **Chapter 3 — The Python ML Stack**
   - 3.1 Setting up
   - 3.2 NumPy: fast arrays
   - 3.3 pandas: tables with labels
   - 3.4 matplotlib: seeing the data
   - 3.5 scikit-learn: the estimator API
   - 3.6 Jupyter workflow
   - 3.7 Deep-learning frameworks: when scikit-learn is not enough
   - Chapter TL;DR
4. **Chapter 4 — Data, EDA and Feature Engineering**
   - 4.1 Where training data comes from
   - 4.2 Exploratory data analysis (EDA)
   - 4.3 Data cleaning
   - 4.4 Encoding categorical variables
   - 4.5 Feature scaling
   - 4.6 Feature creation
   - 4.7 Train, validation and test splits
   - 4.8 Data leakage
   - 4.9 Class imbalance
   - 4.10 End-to-end: `Pipeline` + `ColumnTransformer`
   - Chapter TL;DR
5. **Chapter 5 — Linear and Logistic Regression**
   - 5.1 The supervised-learning workflow
   - 5.2 Linear regression: model and loss
   - 5.3 Fitting it: the normal equation vs gradient descent
   - 5.4 Gradient descent in practice
   - 5.5 Assumptions of linear regression
   - 5.6 Polynomial regression
   - 5.7 Regularization: Ridge, Lasso, ElasticNet
   - 5.8 Logistic regression
   - 5.9 Multiclass classification: one-vs-rest and softmax
   - 5.10 Interpreting coefficients
   - Chapter TL;DR
6. **Chapter 6 — Evaluation and Tuning**
   - 6.1 Baselines, and offline vs business metrics
   - 6.2 Regression metrics
   - 6.3 Classification metrics
   - 6.4 Ranking metrics (an introduction)
   - 6.5 Calibration
   - 6.6 Cross-validation
   - 6.7 The bias–variance trade-off
   - 6.8 Learning curves and validation curves
   - 6.9 Hyperparameter tuning
   - 6.10 Early stopping
   - 6.11 Putting it together: from ML metric to business decision
   - Chapter TL;DR
7. **Chapter 7 — Decision Trees and Ensembles**
   - 7.1 Decision trees
   - 7.2 Bias–variance of trees
   - 7.3 Bagging and Random Forests
   - 7.4 Boosting
   - 7.5 Voting and stacking
   - 7.6 When trees beat neural networks
   - 7.7 Hyperparameter cheat sheet
   - Chapter TL;DR
8. **Chapter 8 — k-Nearest Neighbours, Support Vector Machines, Naive Bayes**
   - 8.1 k-Nearest Neighbours (k-NN)
   - 8.2 Support Vector Machines (SVM)
   - 8.3 Naive Bayes
   - 8.4 Choosing a classic supervised algorithm
   - Chapter TL;DR
9. **Chapter 9 — Unsupervised Learning**
   - 9.1 k-means clustering
   - 9.2 Hierarchical (agglomerative) clustering
   - 9.3 DBSCAN — density-based clustering
   - 9.4 Gaussian mixture models (GMM) and EM
   - 9.5 Dimensionality reduction: PCA
   - 9.6 t-SNE and UMAP — non-linear visualisation
   - 9.7 Anomaly detection
   - 9.8 Association rules (Apriori)
   - 9.9 Evaluating unsupervised models
   - Chapter TL;DR
10. **Chapter 10 — Neural Networks and Deep Learning**
    - 10.1 From the perceptron to the multi-layer perceptron
    - 10.2 Activation functions
    - 10.3 The forward pass and loss functions
    - 10.4 Backpropagation
    - 10.5 Optimizers and learning-rate schedules
    - 10.6 Initialization and vanishing/exploding gradients
    - 10.7 Regularization: fighting overfitting
    - 10.8 Practical training habits
    - 10.9 PyTorch basics
    - Chapter TL;DR
11. **Chapter 11 — CNNs and Computer Vision**
    - 11.1 Convolution
    - 11.2 Pooling and the CNN architecture
    - 11.3 A small CNN on 8×8 digits (PyTorch, CPU)
    - 11.4 Famous architectures — a timeline

- 11.5 Transfer learning and fine-tuning
- 11.6 Data augmentation for images
- 11.7 Vision tasks: classification, detection, segmentation
- 11.8 Putting it together — trade-offs, mistakes, interview
- Chapter TL;DR

12. **Chapter 12 — Sequences: RNNs, LSTMs and Attention**
    - 12.1 Sequence data
    - 12.2 Recurrent neural networks (RNNs)
    - 12.3 Vanishing and exploding gradients through time
    - 12.4 LSTM and GRU
    - 12.5 Sequence-to-sequence (encoder-decoder) models
    - 12.6 The attention mechanism
    - 12.7 Self-attention and the transformer (high level)
    - 12.8 Text preprocessing and tokenization
    - 12.9 Word embeddings
    - 12.10 Time-series forecasting — an introduction
    - Chapter TL;DR
13. **Chapter 13 — Recommender Systems and Embeddings**
    - 13.1 The recommendation problem
    - 13.2 Explicit vs implicit feedback
    - 13.3 Similarity measures
    - 13.4 Content-based vs collaborative filtering
    - 13.5 Matrix factorisation
    - 13.6 Embeddings
    - 13.7 Two-tower retrieval
    - 13.8 The funnel: candidate generation → ranking → re-ranking
    - 13.9 Approximate nearest neighbour (ANN) search
    - 13.10 Cold start
    - 13.11 Evaluation
    - 13.12 Popularity bias and feedback loops
    - 13.13 Real-world systems
    - Chapter TL;DR
14. **Chapter 14 — Reinforcement Learning**
    - 14.1 What reinforcement learning is
    - 14.2 Markov decision processes, returns and discounting
    - 14.3 Policies, value functions and the Bellman equations
    - 14.4 Exploration vs exploitation — multi-armed bandits
    - 14.5 Dynamic programming — planning when the model is known
    - 14.6 Learning from experience: Monte Carlo vs temporal difference
    - 14.7 Q-learning and SARSA — tabular control
    - 14.8 Deep reinforcement learning — DQN
    - 14.9 Policy gradients, REINFORCE and actor-critic
    - 14.10 RLHF — the bridge to large language models
    - 14.11 Real-world RL
    - 14.12 Pitfalls
    - Chapter TL;DR
15. **Chapter 15 — ML System Design: The Interview Framework**
    - 15.1 ML design vs classic system design
    - 15.2 The three question families
    - 15.3 The seven-step framework
    - 15.4 Capstone — scoping a vague AI question
    - 15.5 Common mistakes (the whole interview)
    - Chapter TL;DR
16. **Chapter 16 — ML in Production: Pipelines, Serving, Evaluation, Monitoring and MLOps**
    - 16.1 The five pieces and the offline/online divide
    - 16.2 Training vs serving
    - 16.3 Where training data comes from
    - 16.4 Features and feature stores
    - 16.5 Batch vs online vs streaming inference
    - 16.6 Model serving basics
    - 16.7 Offline evaluation: choosing the metric
    - 16.8 Monitoring, drift and retraining
    - 16.9 Online experiments and A/B testing
    - 16.10 MLOps: the glue that makes it repeatable
    - 16.11 Capstone — an ML platform for ETA prediction
    - Chapter TL;DR
17. **Chapter 17 — ML Case Studies I: Recommendations, Feed Ranking, Search**
    - 17.0 The case-study template
    - 17.1 Case study: video recommendations
    - 17.2 Case study: social feed ranking
    - 17.3 Case study: e-commerce search ranking
    - 17.4 Case study: text-to-video search
    - Chapter comparison (17.1–17.4)
    - Chapter TL;DR
18. **Chapter 18 — ML Case Studies II: Ads, Fraud, Moderation, PYMK, Music, Visual Search, Blurring, Events, Similar Listings**
    - 18.1 Case study: ad click prediction
    - 18.2 Case study: payment fraud detection
    - 18.3 Case study: harmful content detection (and running it at platform scale)
    - 18.4 Case study: People You May Know (PYMK)
    - 18.5 Case study: music discovery (Spotify-style)
    - 18.6 Case study: visual search for a second-hand marketplace
    - 18.7 Case study: privacy blurring in street imagery
    - 18.8 Case study: event recommendation
    - 18.9 Case study: similar listings from session embeddings
    - Comparison across all thirteen case studies (Chapters 17–18)
    - Chapter TL;DR
19. **Chapter 19 — Responsible ML: Fairness, Explainability, Privacy and Security**
    - 19.1 Where bias comes from
    - 19.2 Fairness definitions and the impossibility trade-off
    - 19.3 Measuring fairness in code
    - 19.4 Mitigating bias: before, during and after training
    - 19.5 Explainability
    - 19.6 Privacy
    - 19.7 Security: attacks on ML models
    - 19.8 Documentation: model cards and datasheets
    - 19.9 Regulation overview
    - 19.10 Real-world incidents
    - 19.11 Responsible-ML checklist
    - 19.12 Interview Q&A
    - Chapter TL;DR
20. **Chapter 20 — Cheat Sheets and Glossary**
    - 20.1 The ML workflow (§1.9, §4.10, §6.11, §15.3, §16.1)
    - 20.2 Math essentials (Ch 2)
    - 20.3 NumPy, pandas and scikit-learn API (Ch 3, §4.10)
    - 20.4 Data preparation and feature engineering (Ch 4)
    - 20.5 Every algorithm in one table (Ch 5, 7–14)
    - 20.6 Loss functions (Ch 2, 5, 8, 10, 13, 14)
    - 20.7 Metrics: formula and when to use (Ch 6, 9, 11, 13, 16)
    - 20.8 Validation and tuning (Ch 4, 6)
    - 20.9 Deep learning: layers, activations, optimizers, regularisation, debugging (Ch 10)
    - 20.10 CNN, RNN and attention: shapes and formulas (Ch 11, 12)
    - 20.11 Recommenders (Ch 13, 17.1)
    - 20.12 Reinforcement-learning update rules (Ch 14)
    - 20.13 ML system design: the 7-step framework and napkin math (Ch 15, 17, 18)
    - 20.14 Production and MLOps checklist (Ch 16)
    - 20.15 Responsible-ML checklist (Ch 19)
    - 20.16 Decision tables: which algorithm, which metric
    - 20.17 Top 50 interview questions
    - 20.18 Glossary (A–Z)
    - Chapter TL;DR

Chapter 1 — What Machine Learning Is

Machine learning (ML) is the craft of getting a computer to improve at a task by studying examples instead of following rules a person wrote by hand. This chapter builds the map you will use for the rest of the book: where ML sits inside "AI", the different ways a machine can learn, the vocabulary everyone uses, the steps of a real project, and the problems that sink most projects. It ends with a complete, runnable scikit-learn project.

What you'll learn

- How AI relates to its branches: machine learning, deep learning, and the generative and agentic systems built on them
- Why "learning from data" beats "writing rules" for some problems (and when it does not)
- The five learning setups: supervised, unsupervised, semi-supervised, self-supervised, reinforcement
- Classification vs regression, batch vs online learning, instance-based vs model-based learning
- How a model learns: training, inference and feedback loops
- The end-to-end workflow: framing → data → features → model → evaluation → deployment → monitoring
- The core vocabulary: features, labels, parameters, hyperparameters, loss, train/validation/test
- The main ways projects fail: bad data, overfitting, underfitting, leakage, non-representative data
- A first end-to-end scikit-learn project you can run and modify

All code is in `code/`. Every script runs in a few seconds on a laptop: `python code/<file>.py`.

1.1 The AI family tree

What it is

"AI" is used loosely in everyday speech, but it is really an umbrella with several branches nested inside it.

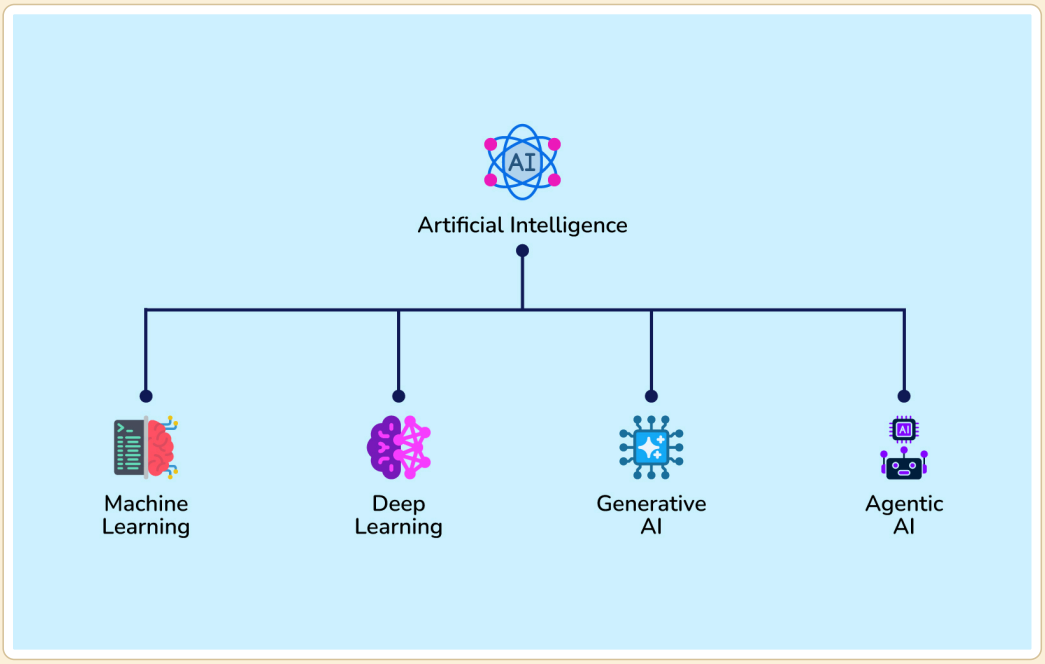
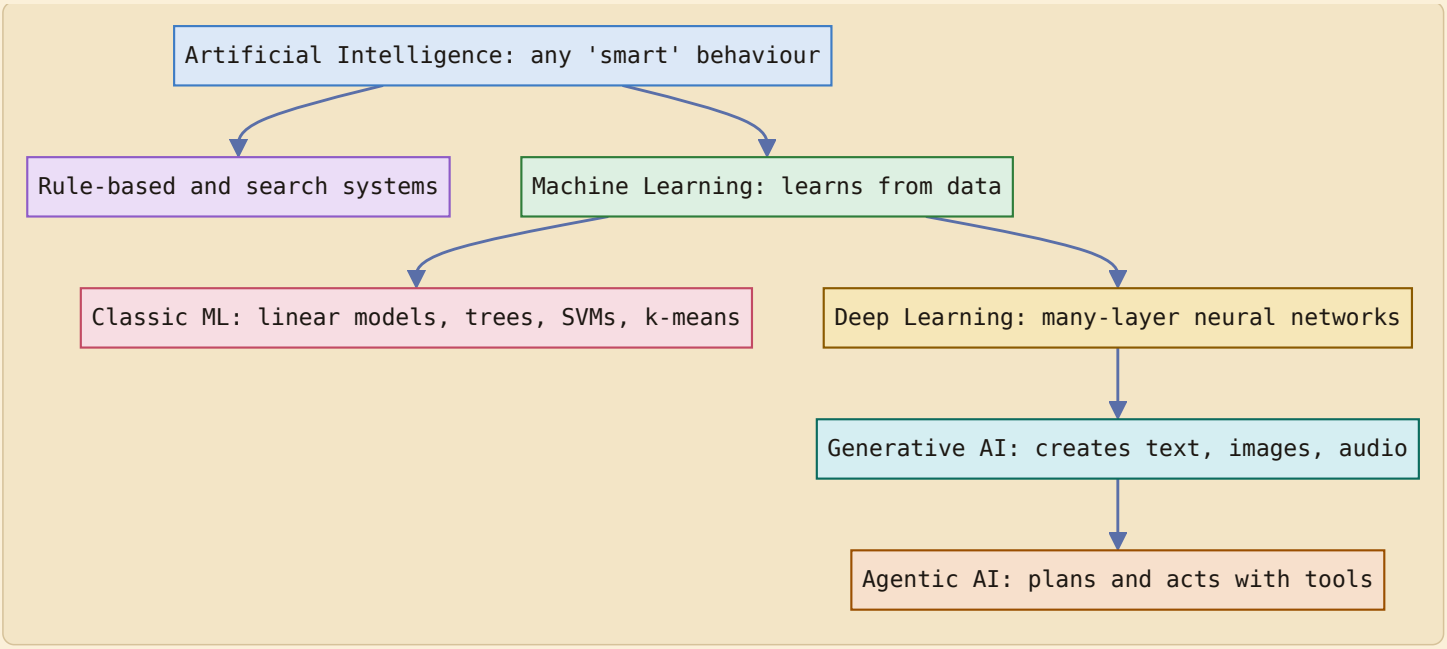


Figure — AI and four branches people talk about most (course diagram).

Term	Plain definition	Relationship	Typical example
Artificial intelligence (AI)	Any technique that makes a machine do something we would call "intelligent": reasoning, perceiving, planning, deciding.	The whole field. Includes hand-written rules and search, not only learning.	A chess engine searching moves; a rules-based tax calculator; a route planner
Machine learning (ML)	AI systems that *learn their behaviour from data* rather than having every rule spelled out.	A subset of AI.	Spam filter, fraud detector, "customers also bought"
Deep learning (DL)	ML using *neural networks* with many layers, where each layer transforms the output of the previous one.	A subset of ML.	Face unlock, speech recognition, machine translation
Generative AI (GenAI)	Models that *produce new content* (text, images, audio, video, code) that resembles their training data.	Mostly built with deep learning today.	Chat assistants, image generators (DALL·E, Midjourney), music tools
Agentic AI	Systems that *plan and take multi-step actions* toward a goal, using tools and adapting as they go.	Usually a generative model wrapped in a loop of plan → act → observe.	A coding agent that edits files and runs tests; a warehouse robot choosing routes



A useful way to read the tree: every deep learning model is an ML model, and every ML model is an AI system, but not the other way round. Generative and agentic AI are not strictly "inside" deep learning by definition, but nearly every practical system today is built on deep neural networks, so the diagram shows the common case.

Why it matters

People (and job descriptions) mix these words up constantly. Knowing the boundaries lets you pick the right tool: a small tabular dataset of 10,000 loan applications calls for classic ML, not a large language model; generating product descriptions calls for GenAI; a simple VAT calculation needs neither.

How each branch works, briefly

1. **Machine learning** — show the computer many examples (e.g. photos marked "cat" or "dog"). An algorithm finds patterns (shapes, textures, colours) that separate the groups, and uses them to label photos it has never seen. The "rules" are discovered, not typed in.
2. **Deep learning** — the same idea, but the model is a stack of layers. In an image network, early layers respond to simple things such as edges; middle layers combine edges into textures and parts; late layers respond to whole objects or faces. Because the layers learn their own features, DL shines on raw, unstructured data (pixels, audio, text). The 2012 AlexNet result on the ImageNet competition is usually cited as the moment DL overtook older computer-vision methods.
3. **Generative AI** — instead of outputting a label, the model learns the structure of its training material well enough to *sample* new material in the same style. A text model such as GPT learns to predict the next word (token); repeating that prediction produces paragraphs. Image generators learn to turn random noise into an image that matches a text prompt.
4. **Agentic AI** — a model is given a goal and a set of tools (web search, a calendar API, a code runner). It decides on a step, calls a tool, reads the result, and decides the next step, repeating until done. The difference from a plain model is autonomy over *multiple* steps in a changing environment.

Trade-offs

	Classic ML	Deep learning	Generative AI	Agentic AI
Data needed	Hundreds to millions of rows	Usually large (or a pre-trained model)	Enormous for pre-training; small for adapting	Builds on a pre-trained model
Best input type	Tables (spreadsheets, databases)	Images, audio, text	Text, images, audio	Tasks with tools and steps
Interpretability	Often good	Low	Low	Low, and actions compound
Compute cost	Low (CPU)	High (GPU)	Very high to train; high to run	High (many model calls)
Main risk	Bad data	Needs lots of data and compute	Confident but wrong output ("hallucination"), copyright and authenticity questions	Mistakes chain across steps; safety of actions

When not to use it

- If the rules are known, stable and few (tax brackets, unit conversion, access control), write the rules. ML adds cost and uncertainty for no benefit.
- If a mistake is unacceptable and you cannot measure the error rate, do not hand the decision to a learned model without human review.
- Do not use generative AI where a classic ML model or a database query answers the question exactly.

Common mistakes

- Saying "AI" when you mean a specific technique; say "a gradient-boosted classifier" or "a fine-tuned language model".

- Assuming newer = better. On tabular business data, tree ensembles such as random forests and gradient boosting (Chapter 7) frequently beat deep networks.
- Believing a model "understands" or "thinks" like a person. It maps inputs to outputs using statistical patterns found in data.

Real-world examples

- **Classic ML:** card-fraud scoring at banks and payment companies; Gmail's spam filtering; credit scoring; demand forecasting in retail.
- **Recommendations:** Netflix and YouTube rank what to show you next from your viewing history and that of similar users.
- **Deep learning:** Apple's Face ID, Google Translate, speech recognition in voice assistants, detection of abnormalities in medical scans.
- **Generative AI:** chat assistants that draft emails and articles, image generators such as DALL·E and Midjourney, tools that compose music or edit video.
- **Agentic AI:** warehouse robots that plan routes around obstacles, assistants that book appointments or troubleshoot a laptop, coding agents; automated trading systems act autonomously too, although many are rule-based rather than learned.

In the interview

> **30-second answer:** "AI is the broad goal of making machines behave intelligently. Machine learning is the part of AI where behaviour is learned from data instead of hand-coded. Deep learning is ML with many-layer neural networks, which learn their own features from raw data like pixels and text. Generative AI uses such models to create new content, and agentic AI wraps them in a loop so they can plan and take actions with tools."

- *Is a rules engine AI?* — Yes, classical AI includes rule-based expert systems; it is just not machine learning.
- *Why did deep learning take off around 2012?* — Three things arrived together: large labelled datasets (ImageNet), cheap parallel compute (GPUs), and training tricks (ReLU activations, dropout, better initialisation).
- *When would you choose classic ML over deep learning?* — Small or medium tabular data, a need for interpretability, tight latency or budget.

Self-check

1. Is every ML model a deep learning model? → No. DL is a subset of ML; linear regression and decision trees are ML but not DL.
2. What makes a system "agentic"? → It pursues a goal over multiple steps, choosing actions and tools and adapting to what it observes.
3. Give one reason not to use ML. → The rules are simple and known, so hand-written code is cheaper, exact and easier to audit.
4. What does a generative model output that a classifier does not? → New content (text, image, audio), rather than a label or number.

TL;DR

AI ⊃ ML ⊃ DL. Generative AI creates content, agentic AI takes multi-step actions; both are usually built on deep learning. Pick the simplest branch that solves the problem.

1.2 Learning from data vs writing rules

What it is

In traditional programming a person writes the rules: *input + rules → output*. In machine learning a person supplies examples of inputs with the desired outputs, and an algorithm produces the rules: *inputs + outputs → rules (the model)*. The model is then used like any program: *new input + model → output*.

Tom Mitchell's classic definition (1997) makes this precise: a program *learns* from experience **E** with respect to a task **T** and a performance measure **P** if its performance on T, measured by P, improves with E. For a spam filter: T = flag spam, E = a pile of emails labelled spam/not-spam, P = the fraction of emails classified correctly.

The problem it solves

Some rules are too many, too fuzzy, or change too often to write by hand:

- **Too complex:** nobody can write the rules for "is there a cat in this photo?" in terms of pixel values.
- **Constantly changing:** spammers adapt; a hand-written list of spam words is always out of date. A learning system can be retrained on new examples.
- **Too many cases:** recognising speech across accents, microphones and background noise.
- **Discovery:** ML can reveal patterns people had not noticed (e.g. which combinations of features predict churn). Inspecting what a model learned is sometimes called *data mining*.

How it works

1. Collect examples of the task, with the right answers if you have them.
2. Turn each example into numbers (features).
3. Choose a model family (e.g. "a weighted sum of word counts").
4. A training algorithm adjusts the model's internal numbers until its answers match the examples as closely as possible.
5. Use the trained model on new inputs.

Code: a hand-written rule vs a learned model

`code/rules_vs_learning.py` trains a Naive Bayes spam filter (a simple probabilistic classifier, Chapter 8) on ten labelled messages and compares it with a rule "spam if it contains *free* or *prize*".

```python
def rule_based(text: str) -> int:
    return int("free" in text or "prize" in text)

vec = CountVectorizer()                 # text -> word-count features
X = vec.fit_transform(texts)
model = MultinomialNB().fit(X, labels)   # learns how often each word appears in spam vs ham
p = model.predict_proba(vec.transform([t]))[0, 1]
```

Output:

```
message                          rule  learned  P(spam)
limited offer click to claim        0        1    0.95
free coffee in the kitchen          1        1    0.61
slides for the review meeting       0        0    0.06
```

The rule misses spam that avoids its two words and cannot be taught anything new without a programmer. The learned model catches the first message from words it saw in spam ("limited", "offer", "click", "claim"). It also gets "free coffee" wrong: with only ten examples, "free" looks strongly spammy. That is the other half of the lesson: **a model is only as good as its data**. With thousands of examples the office-email use of "free" would be learned too.

Trade-offs

	Hand-written rules	Machine learning
Needs	Expert knowledge	Representative, labelled data
Transparency	Fully readable	Varies; often opaque
Adapting to change	Rewrite code	Retrain on new data
Handles fuzzy patterns (images, speech)	Poorly	Well
Guarantees	Exact behaviour	Statistical: right *most* of the time

When not to use it

When the logic is known and must be exact (payroll, permissions), when there is no data, or when nobody can tolerate a small error rate.

Common mistakes

- Starting with ML before trying a simple rule or heuristic as a baseline.
- Forgetting that a learned model inherits every bias and gap in its examples.

Self-check

1. In Mitchell's definition, what are T, E and P for a house-price model? → T: predict price; E: past sales with prices; P: e.g. average error in dollars.
2. Why can't you easily hand-code a cat detector? → The rules would have to be written in terms of raw pixels; the patterns are too numerous and varied.
3. What did the spam example show about small data? → The model over-trusted a single word ("free") because it saw too few examples.

TL;DR

Rules come from people in programming and from data in ML. Use ML when the rules are too complex or change too often, and only when you have good examples.

1.3 How a model learns: training, inference and feedback loops

What it is

A model's life has three phases.

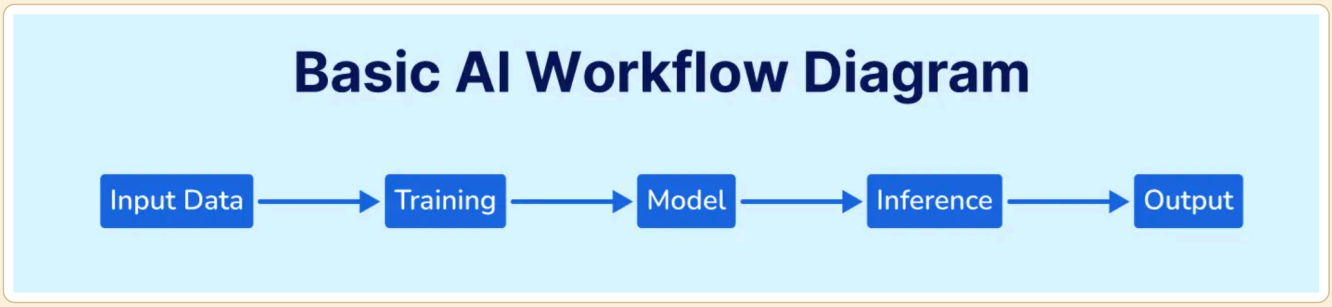


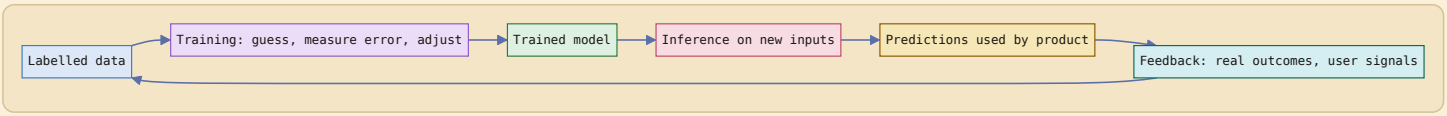
Figure — Data feeds training, which produces a model used for inference (course diagram).

1. **Training** — the model sees examples whose answers are known, makes a guess for each, measures how wrong it was, and nudges its internal numbers to be less wrong next time. Think of a student working through practice problems with an answer key. After many rounds, it

predicts well on examples it has *not* seen.

2. **Inference** — the trained model is used on new data to produce predictions. No learning happens here; it is just applying what was learned. A voice assistant turning your spoken command into text in real time is inference.
3. **Feedback loop** — in production you compare predictions to what actually happened, collect the new outcomes as fresh training data, and retrain. When you give a show a thumbs-down, a recommender can treat that as a new label and adjust future suggestions.

An analogy that covers all three: teaching a child what a cat is. You show many pictures and correct mistakes (training). Later the child spots a cat on the street without help (inference). When the child later points at a dog and says "cat", you step in again (feedback).



How it works (the core loop)

For almost every model in this book, training is the same loop:

1. Start with some values for the model's **parameters** (its adjustable numbers, e.g. the weights in $\hat{y} = w \cdot x + b$).
2. Predict for a batch of training examples.
3. Compute the **loss**: a single number measuring how wrong the predictions are (e.g. mean squared error).
4. Change the parameters in the direction that lowers the loss (Chapter 2 shows how, with gradient descent).
5. Repeat until the loss stops improving.

Data is king

A model can never be better than the examples it learns from.

- **Garbage in, garbage out.** Mislabelled, outdated or one-sided data yields a model with the same flaws, often in ways that are hard to spot.
- **Diversity matters.** Just as reading widely broadens a person's judgment, a model exposed to varied examples copes with more situations. The *Gender Shades* study (Buolamwini and Gebru, 2018) found that commercial face-analysis systems made far more errors on darker-skinned women than on lighter-skinned men, a gap traced largely to unbalanced training and benchmark data.
- **High stakes.** A diagnostic model trained on patients from one population may misdiagnose patients from another.

The goal of training is **generalisation**: performing well on new, real-world data, not just on the examples it studied. Good, diverse data is the single biggest lever for generalisation.

Self-check

1. Does a model learn during inference? → Not by default; inference only applies learned parameters. Learning resumes when you retrain (or with online learning, §1.7).
2. What is a feedback loop in production? → Capturing real outcomes of predictions and feeding them back as training data to improve the model.
3. What is generalisation? → Performing well on data the model has not seen during training.

TL;DR

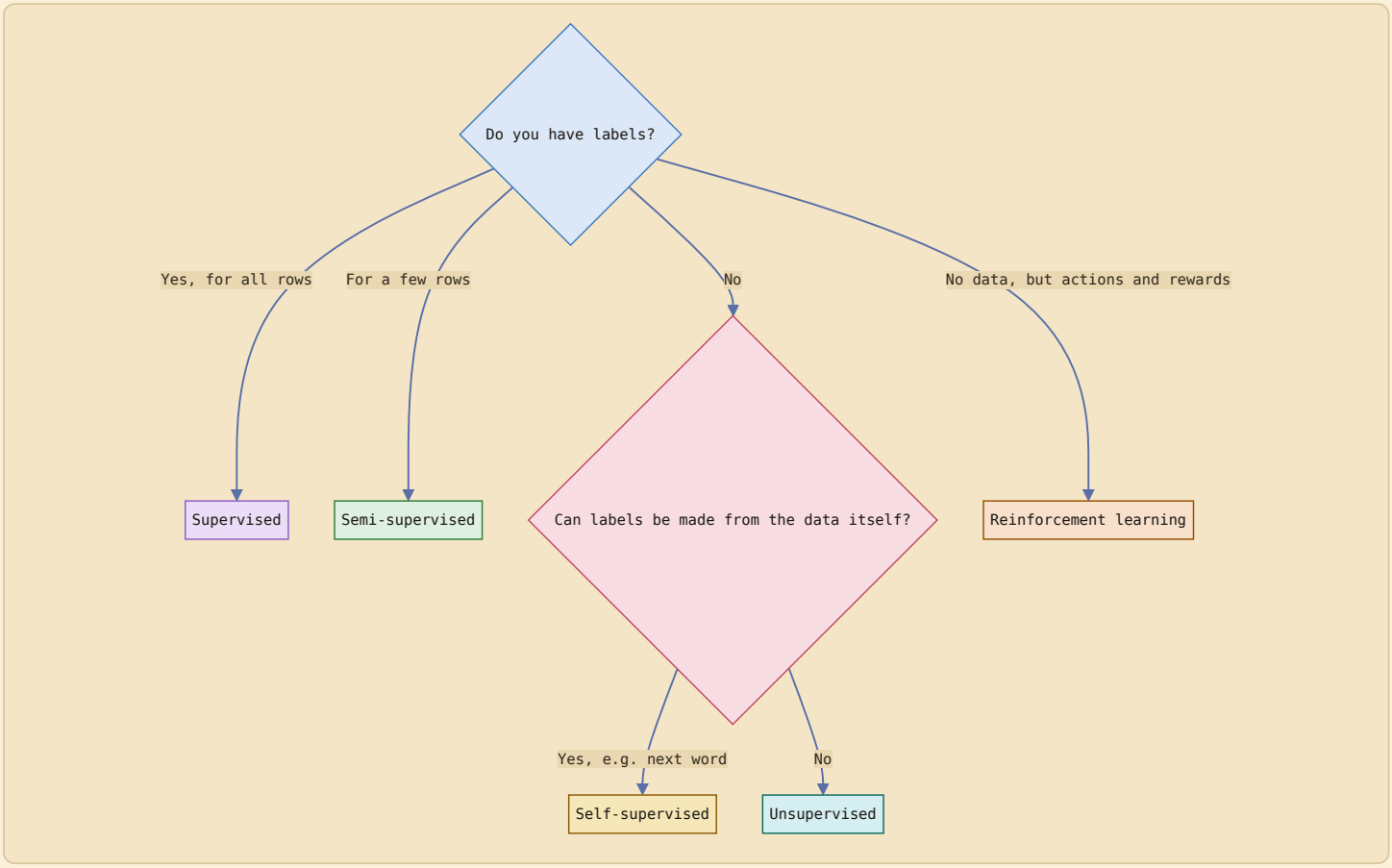
Training adjusts parameters to reduce loss on known examples; inference applies the model to new data; feedback loops turn real outcomes into new training data. Data quality and diversity decide how well a model generalises.

1.4 Types of learning by supervision

What it is

ML systems are grouped by *how much and what kind of answer* they get during training.

Type	What the training data contains	Goal	Typical algorithms	Example
Supervised	Inputs **with** labels (the right answer)	Predict the label for new inputs	Linear/logistic regression, trees, random forests, gradient boosting, SVMs, neural nets	Emails marked spam / not spam → classify new emails
Unsupervised	Inputs **without** labels	Find structure: groups, low-dimensional summaries, anomalies	k-means, DBSCAN, PCA, isolation forest	Group customers by behaviour; flag odd transactions
Semi-supervised	A few labelled rows + many unlabelled rows	Use the unlabelled data to do better than the labelled part alone	Self-training, label propagation	Photo app clusters faces, you name one photo per person, it names the rest
Self-supervised	Unlabelled data from which labels are *generated automatically*	Learn useful representations by predicting a hidden part of the data	Next-word prediction (GPT), masked-word prediction (BERT), contrastive learning	Language models trained to predict the next token on web text
Reinforcement learning (RL)	No fixed dataset; an *agent* acts in an *environment* and receives *rewards*	Learn a *policy* (what to do in each situation) that maximises total reward	Q-learning, policy gradients, PPO	DeepMind's AlphaGo; robots learning to walk; fine-tuning chatbots from human preferences (RLHF)



Why it matters

Labels are usually the most expensive part of a project (doctors labelling scans, lawyers labelling contracts). The supervision type tells you how much labelling you must pay for, which algorithms apply, and how you can evaluate the result.

How it works (one demo each)

`code/learning_types.py` runs a tiny example of every type:

```
# supervised: iris species (classification) and diabetes progression (regression)
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
reg = LinearRegression().fit(Xtr, ytr)
# unsupervised: k-means never sees the true groups
labels = KMeans(n_clusters=3, n_init=10, random_state=SEED).fit_predict(X)
# semi-supervised: hide 95% of labels (-1 = unlabelled), then self-train
self_train = SelfTrainingClassifier(LogisticRegression(max_iter=2000), threshold=0.9).fit(Xtr, y_partial)
# self-supervised: the "label" is the next value of the series
X = np.stack([series[i:i + 3] for i in range(len(series) - 3)]); y = series[3:]
# reinforcement: epsilon-greedy 3-armed bandit learns which arm pays best
arm = rng.integers(3) if rng.random() < 0.1 else int(np.argmax(estimates))
```

Output:

```
[classification] iris accuracy = 1.000
[regression]     diabetes MAE = 45.1 (target std = 77.0)
[unsupervised]   k-means agreement with hidden groups (ARI) = 0.767
[semi-supervised] 69 labels only: 0.867 | + self-training on unlabelled rows: 0.882
[self-supervised] next-value MAE = 0.068
[reinforcement]  estimated payouts [0.19, 0.46, 0.8], pulls [81, 72, 1847], avg reward 0.766
```

How to read it:

- *Supervised:* near-perfect species prediction; regression error (45) well below the natural spread of the target (77).
- *Unsupervised:* k-means recovered most of the hidden grouping (ARI = 1 would be perfect agreement, 0 = random) without ever seeing it.
- *Semi-supervised:* with only 69 labelled digits, letting the model label confident unlabelled rows and retrain added about 1.5 points of accuracy. The gain varies with data; it is not guaranteed.
- *Self-supervised:* no human wrote a label; the future of the series served as the target.
- *Reinforcement:* the agent explored all three arms, discovered arm 3 pays about 80% of the time, and spent most pulls on it. The 10% of random pulls is the *exploration* cost.

Trade-offs

	Labelling cost	Evaluation	Main risk
Supervised	High	Easy: compare to labels	Label errors and bias
Unsupervised	None	Hard: no ground truth	Finds patterns that mean nothing
Semi-supervised	Low	Needs a labelled test set	Wrong pseudo-labels reinforce themselves
Self-supervised	None, but huge data and compute	Via downstream tasks	Cost; learns biases of web-scale data
Reinforcement	Need a reward and an environment	Total reward	Reward hacking; slow, unstable training

Common mistakes

- Using unsupervised clustering and then treating clusters as if they were true categories without checking.
- Designing an RL reward that can be "gamed" (the agent finds a loophole that earns reward without doing the task).
- Calling LLM pre-training "unsupervised". It is better described as self-supervised: there *is* a target (the next token), it is just generated from the data.

In the interview

30-second answer: "Supervised learning has labelled examples and predicts labels. Unsupervised has no labels and finds structure like clusters. Semi-supervised mixes a few labels with lots of unlabelled data. Self-supervised creates labels from the data itself, like predicting the next word, which is how large language models are pre-trained. Reinforcement learning learns actions from rewards by trial and error."

- *Is anomaly detection supervised?* — Usually unsupervised or semi-supervised (you train mostly on normal data), because labelled anomalies are rare.
- *What is the exploration–exploitation trade-off?* — In RL, choosing between trying actions to learn more (explore) and using the best-known action (exploit).
- *How is ChatGPT-style training split?* — Self-supervised pre-training on text, then supervised fine-tuning on example conversations, then reinforcement learning from human (or AI) feedback.

Self-check

1. Fraud labels exist for only 1% of transactions; others are unlabelled. Which setup fits? → Semi-supervised (or supervised on the labelled part plus anomaly detection).
2. What generates the labels in self-supervised learning? → The data itself, e.g. a masked word or the next token.
3. What does an RL agent learn? → A policy mapping situations to actions that maximises cumulative reward.
4. Why is unsupervised learning hard to evaluate? → There is no correct answer to compare against.

TL;DR

Supervision type = what answers you have. Labels → supervised; none → unsupervised; a few → semi-supervised; made from the data → self-supervised; rewards from actions → reinforcement.

1.5 Classification vs regression

What it is

The two main supervised tasks, split by the *type of the label*:

- **Classification** predicts a **category** (a discrete class): spam/ham, digit 0–9, species. With two classes it is *binary*; with more it is *multiclass*; if each example can have several labels at once (a photo tagged "beach" and "dog") it is *multilabel*.

- **Regression** predicts a **number** on a continuous scale: price, temperature, days until churn.

	Classification	Regression
Output	Class (often with a probability)	Real number
Example losses	Log loss (cross-entropy), hinge loss	Mean squared error (MSE), mean absolute error (MAE)
Example metrics	Accuracy, precision, recall, F1, ROC-AUC	MAE, RMSE, R^2
Example algorithms	Logistic regression, decision tree classifier, k-NN classifier	Linear regression, decision tree regressor, k-NN regressor

Watch the name: **logistic regression is a classifier**. It predicts the probability of a class, then thresholds it (usually at 0.5).

Common mistakes

- Encoding categories as numbers (red = 1, green = 2, blue = 3) and treating it as regression, which invents an order that does not exist.
- Turning a regression problem into classes too early (e.g. price "high/low"), throwing away information.
- Judging a classifier by accuracy on imbalanced data: if 99% of transactions are legitimate, "always legitimate" is 99% accurate and useless. Chapter 6 covers better metrics.

Self-check

1. Predicting a star rating 1–5: classification or regression? → Either can work; regression respects the order, classification treats ratings as separate categories. Many teams try both.
2. Is logistic regression used for regression? → No, for classification.
3. Which metric suits regression? → MAE or RMSE (both in the target's units), or R^2.

TL;DR

Category → classification. Number → regression. The label type decides the loss, the metrics and the algorithms.

1.6 Key terms

Term	Meaning	Example (house prices)
Example / instance / sample / row	One item in the dataset	One house sale
Feature (attribute, input, predictor, X)	A measurable property used as input	Size in m^2, number of rooms, postcode
Label (target, output, y)	The answer we want to predict	Sale price
Feature vector	All features of one example as a list of numbers	[120, 3, 1995]
Model	A function with adjustable parts that maps features to a prediction	price = $w_1 \cdot$size + $w_2 \cdot$rooms + b
Parameters	Numbers *learned* from data during training	w_1, w_2, b
Hyperparameters	Settings *chosen by you before* training; they control the learning	Polynomial degree, regularisation strength C, learning rate, number of trees
Loss (cost, objective) function	A number measuring how wrong predictions are; training minimises it	Mean squared error
Training set	Data the model learns parameters from	60–80% of sales
Validation set	Data used to compare models and tune hyperparameters	10–20%
Test set	Data used once at the end to estimate real-world performance	10–20%, never touched before
Cross-validation	Rotating the validation role through k chunks of the training data and averaging	5-fold CV
Prediction (ŷ, "y-hat")	The model's output for an example	$412,000
Generalisation	Doing well on new data	Accurate price for next month's listings
Inference	Using a trained model to predict	Pricing a newly listed house

Parameters vs hyperparameters

Parameters are what training finds; hyperparameters are knobs you set that shape how training searches. In $\hat{y} = w \cdot x + b$, **w** and **b** are parameters. If you fit a polynomial, its **degree** is a hyperparameter. You tune hyperparameters by trying several values and comparing them on the *validation* set (or with cross-validation), never on the test set.

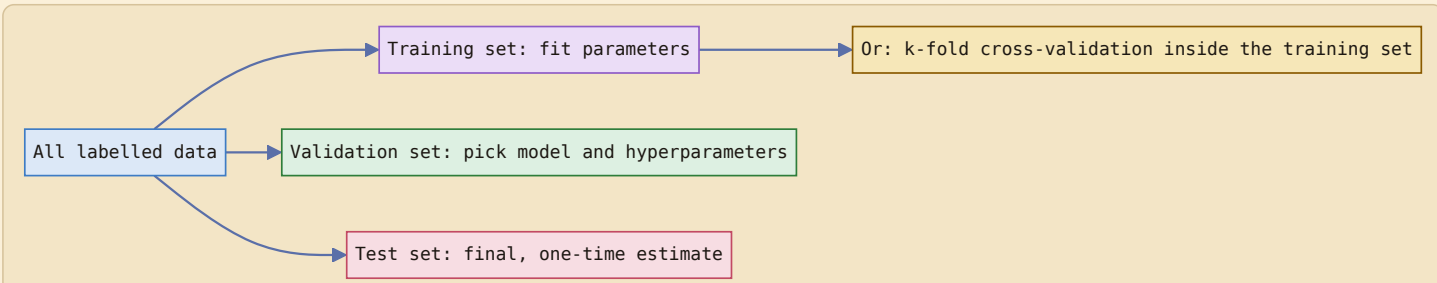
Loss, with worked numbers

Three houses have true prices 300, 250, 400 (in $1000s) and the model predicts 310, 240, 380.

- Errors: −10, +10, +20 (true − predicted)
- **MSE** = (10² + 10² + 20²) / 3 = (100 + 100 + 400) / 3 = **200** (units: $1000², hard to read)
- **RMSE** = √200 ≈ **14.1** → "typically off by about $14k"
- **MAE** = (10 + 10 + 20) / 3 ≈ **13.3**

MSE punishes the big error (20) much more than the small ones, because errors are squared.

Training, validation and test sets



Why three sets? If you tune hyperparameters by watching test performance, you slowly fit the test set too, and your reported score becomes optimistic. The validation set absorbs that tuning; the test set stays honest.

`code/train_val_test.py` makes a 60/20/20 split, picks the regularisation strength C by validation log loss, reports test accuracy once, and then simulates *drift* in production:

```
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=SEED)
for c in [0.001, 0.01, 1.0]:
    m = LogisticRegression(C=c).fit(X_train, y_train)
    loss = log_loss(y_val, m.predict_proba(X_val))
```

Output:

```
sizes train/val/test: 1800 600 600
C=0.001  validation log-loss = 0.529
C=0.01   validation log-loss = 0.287
C=1.0    validation log-loss = 0.063
chosen C=1.0; test accuracy = 0.998
after drift, live accuracy = 0.746  -> alert, retrain
after retraining on fresh labelled data, accuracy = 0.986
```

(0.25 of the remaining 80% is 20% of the total, hence 60/20/20.) The last two lines show why monitoring matters: the world changed, accuracy fell from 99.8% to 74.6%, and retraining on fresh data recovered it.

Self-check

1. Is the learning rate a parameter or a hyperparameter? → Hyperparameter; you choose it.
2. Why not tune on the test set? → The test score would then be biased upward and no longer estimate performance on new data.
3. What does RMSE ≈ 14.1 mean for prices in $1000s? → Predictions are typically about $14,000 off.

TL;DR

Features in, label out. Training learns parameters by minimising a loss; you choose hyperparameters with a validation set or cross-validation; the test set is used once.

1.7 Batch vs online learning

What it is

- **Batch (offline) learning:** train on the full dataset at once, deploy the frozen model, and retrain from scratch (say nightly or weekly) to include new data.
- **Online (incremental) learning:** feed data in small chunks (mini-batches) or one example at a time; the model updates as data arrives, and old data can be thrown away.

A related idea, **out-of-core learning**, uses the online approach to train on datasets too large for memory by streaming chunks from disk.

Why it matters

Batch learning is simple and reproducible but slow to react and expensive to retrain on huge data. Online learning adapts fast (stock prices, news trends, sensor streams) and runs on limited memory, but it can drift in bad directions if bad data arrives.

How it works

1. Initialise the model.

2. When a mini-batch arrives, compute the loss on it and nudge the parameters (one step of gradient descent).
3. Discard the batch (or keep it for audit), wait for the next.

The **learning rate** controls how strongly each new batch moves the model. High → adapts quickly but forgets old patterns and reacts to noise. Low → stable but slow to adapt.

`code/batch_vs_online.py` trains `SGDClassifier` (a linear classifier trained by stochastic gradient descent) both ways:

```
batch = SGDClassifier(average=True, random_state=SEED).fit(X[:16_000], y[:16_000])

online = SGDClassifier(average=True, random_state=SEED)
for start in range(0, 16_000, 2000):
    online.partial_fit(X[start:start + 2000], y[start:start + 2000], classes=[0, 1])
```

Output:

```
batch model (16,000 rows at once): test acc = 0.842
online after  2,000 rows: test acc = 0.833
online after  8,000 rows: test acc = 0.838
online after 16,000 rows: test acc = 0.839
```

The online model reaches almost the batch result while only ever holding 2,000 rows in memory. (`average=True` averages the weights over updates; without it, plain SGD's accuracy jumped around between 0.66 and 0.83 on this data, a real-world reminder that online learners are sensitive to their learning-rate settings.) In scikit-learn, estimators that support incremental learning expose `partial_fit`, e.g. `SGDClassifier`, `SGDRegressor`, `MultinomialNB`, `MiniBatchKMeans`.

Trade-offs

	Batch	Online
Adapts to change	At the next retrain	Continuously
Memory / compute	Whole dataset per retrain	One chunk at a time
Reproducibility	High	Lower: depends on data order
Risk	Stale model	Bad data corrupts the live model quickly
Monitoring needs	Standard	Must watch inputs and performance closely, with a rollback plan

When not to use online learning

When data arrives slowly, when changes are rare (retraining weekly is enough), or when you cannot monitor inputs, since poisoned or buggy data flows straight into the live model. Microsoft's Tay chatbot (2016) was taken offline within a day after users deliberately fed it offensive content it then repeated, a well-known cautionary tale about learning from unfiltered live input.

Self-check

1. What scikit-learn method signals online learning support? → `partial_fit`.
2. What does a high learning rate do in online learning? → The model adapts quickly but also forgets older patterns and chases noise.
3. What is out-of-core learning? → Training on data larger than memory by streaming it in chunks with an online algorithm.

TL;DR

Batch: train on everything, redeploy periodically. Online: update continuously from small chunks. Online adapts faster but needs careful monitoring.

1.8 Instance-based vs model-based learning

What it is

Two ways a system can *generalise* from examples:

- **Instance-based:** memorise the training examples; to predict, find the most **similar** stored examples and use their answers. Example: **k-nearest neighbours (k-NN)**, which averages the labels of the k closest points.
- **Model-based:** use the examples to fit a **compact model** (a formula with parameters) and throw the examples away. Example: **linear regression** $\hat{y} = w \cdot x + b$.

How it works (worked example)

Data: $y \approx 3x + 2$ with noise, x between 0 and 10. `code/batch_vs_online.py`:

```
lin = LinearRegression().fit(X, y)             # stores 2 numbers
knn = KNeighborsRegressor(n_neighbors=5).fit(X, y)  # stores all 5000 rows
x_new = np.array([[4.0], [15.0]])              # 15 lies outside 0..10
```

Output:

```
linear model: w=3.00, b=2.04 -> predicts [14. 47.]
k-NN (k=5): stores 5000 rows -> predicts [14.4 32. ]
prediction time for 10,000 queries: linear 0.22 ms vs k-NN 3.1 ms (exact times vary by machine)
```

- At x = 4 both agree (true value 3·4 + 2 = 14).
- At x = 15 the linear model extrapolates correctly (3·15 + 2 = 47). k-NN can only average its nearest stored neighbours (all near x = 10), giving ≈ 32.
- The linear model is a compressed summary (2 numbers) and predicts much faster; k-NN keeps all data and searches it at prediction time.

Trade-offs

	Instance-based (k-NN)	Model-based (linear model, tree, network)
Training	Almost instant (store data)	Takes time to fit
Prediction	Slower, needs all data in memory	Fast, small memory
Flexibility	Adapts to any shape if data is dense	Limited by the chosen model form
Extrapolation	Poor	Possible if the model form is right
Needs	A good similarity measure; scaled features	A good model family

Self-check

1. Why does k-NN fail at x = 15? → It can only reuse answers of stored neighbours; none exist near 15.
2. Which approach suits a system where new examples must be usable instantly without retraining? → Instance-based: just add the example to the store.
3. What does a model-based learner keep after training? → Only its parameters.

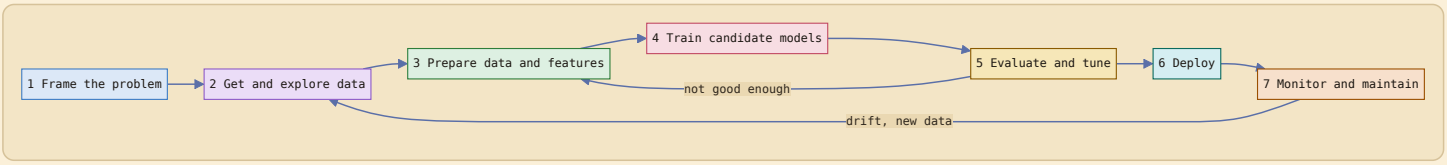
TL;DR

Instance-based learners compare new cases to stored ones; model-based learners fit parameters and discard the data. Most algorithms in this book are model-based.

1.9 The end-to-end ML workflow

What it is

A real ML project is a cycle of steps, and the modelling step is usually the smallest part of the work.



How it works

1. **Frame the problem.** What decision will the prediction drive, and what is it worth? Supervised or not? Classification or regression? Batch or online? What metric reflects business value (e.g. recall on fraud, dollars of error on prices)? What does the current solution (a rule, a human) achieve? That is your **baseline**. Is ML needed at all?
2. **Get and explore the data.** Collect it, check it is legal and ethical to use, look at distributions, missing values, outliers and correlations. **Set aside the test set now**, before exploring deeply, so your choices are not influenced by it.
3. **Prepare data and engineer features.** Clean errors, handle missing values, encode categories, scale numbers, create informative features (e.g. "price per m²"). Put these steps in a pipeline so they are applied identically in training and production.
4. **Train candidate models.** Start simple (a linear model), then try a few families (trees, ensembles). Compare with cross-validation.
5. **Evaluate and tune.** Tune hyperparameters on validation data, analyse errors, check fairness across groups, then measure the final choice *once* on the test set.
6. **Deploy.** Package the whole pipeline (preprocessing + model), serve it through a batch job or a web API, and roll out gradually (shadow mode, A/B test).
7. **Monitor and maintain.** Track input distributions, prediction distributions and real performance as labels arrive. Data changes over time (**data drift**: inputs change; **concept drift**: the input→label relationship changes). Retrain on fresh data, version models and data, and keep a rollback path.

Real-world examples

- **Spam filtering** is a textbook feedback loop: every "Report spam" click becomes a new label.
- **Google Flu Trends** estimated flu levels from search queries. It overestimated the 2012–13 peak substantially, a widely studied example of a model going stale as search behaviour changed, i.e. drift without enough monitoring and recalibration.
- **Zillow Offers** used price predictions to buy homes; in 2021 Zillow shut the business down, citing difficulty forecasting prices accurately in a fast-moving market.

Common mistakes

- Jumping to step 4 without a clear metric or baseline.
- Preprocessing in a notebook and re-implementing it differently in production (**training–serving skew**).
- No monitoring: the model silently decays.

In the interview

> **30-second answer:** "I start by framing the problem: the decision, the metric, and a baseline. Then I collect and explore data, holding out a test set early. I build a preprocessing-plus-model pipeline, compare a few model families with cross-validation, tune on validation data and test once. Then I deploy the pipeline as a unit, and monitor drift and performance, retraining when needed."

- *What is training–serving skew?* — When features are computed differently in production than in training, so the model sees inputs unlike those it learned from.
- *How do you know it's time to retrain?* — Monitored performance drops as labels arrive, or input distributions shift noticeably from training; or on a regular schedule.

Self-check

1. Why hold out the test set before exploring the data? → To avoid unconsciously tailoring choices to it (data snooping).
2. What is concept drift? → The relationship between inputs and the label changes over time.
3. Why ship preprocessing and model together? → So production applies exactly the same transformations as training.

TL;DR

Frame → data → features → model → evaluate → deploy → monitor, and loop. Baseline first, test set once, pipeline everything, monitor always.

1.10 The main challenges of machine learning

The two things that can go wrong are **bad data** and **a bad model**. `code/challenges.py` demonstrates each problem below.

1. Not enough or poor-quality data

What it is. Too few examples, wrong labels, noise, missing values, outliers. **Demo** — flipping a share of training labels on the breast-cancer dataset:

```
  0% wrong training labels -> test accuracy 0.958
 20% wrong training labels -> test accuracy 0.951
 40% wrong training labels -> test accuracy 0.636
```

Random label noise is survivable in moderation (the errors partly cancel out), but it quickly becomes fatal. Systematic errors (e.g. one annotator always mislabels a class) are worse because they do not cancel. **Fixes:** clean and audit labels, get more data, remove or fix outliers, handle missing values deliberately.

2. Non-representative training data (sampling bias)

What it is. The training data does not look like the data the model will face. Causes: collecting only convenient data, self-selection (only happy customers answer surveys), or historical bias. **Demo** — trained only on houses under 120 m², then used on bigger ones:

```
trained on houses < 120 m^2: mean error small houses 14,086 | big houses 43,250
```

The relationship bends outside the range the model saw, and error triples. **Real world:** the face-analysis gaps in §1.3; a famous historical example is the 1936 *Literary Digest* poll, which predicted a Landon win over Roosevelt from about 2.4 million returned ballots. Its mailing lists leaned on sources such as telephone directories and car registrations, and only about a quarter of those mailed replied; both the skewed list and the self-selected responses pushed the sample away from the electorate. **Fixes:** sample to match the deployment population, check performance per subgroup, collect data where the model will be used.

3. Irrelevant features

Garbage features add noise and invite spurious patterns. **Feature engineering** (selecting useful features, combining features, creating new ones from domain knowledge) is often where most gains come from.

4. Overfitting

What it is. The model learns the training data *too* well, including its noise and quirks, and fails on new data. Like a student who memorises past exam answers without understanding. **Signal:** training error low, validation error much higher. **Causes:** model too complex for

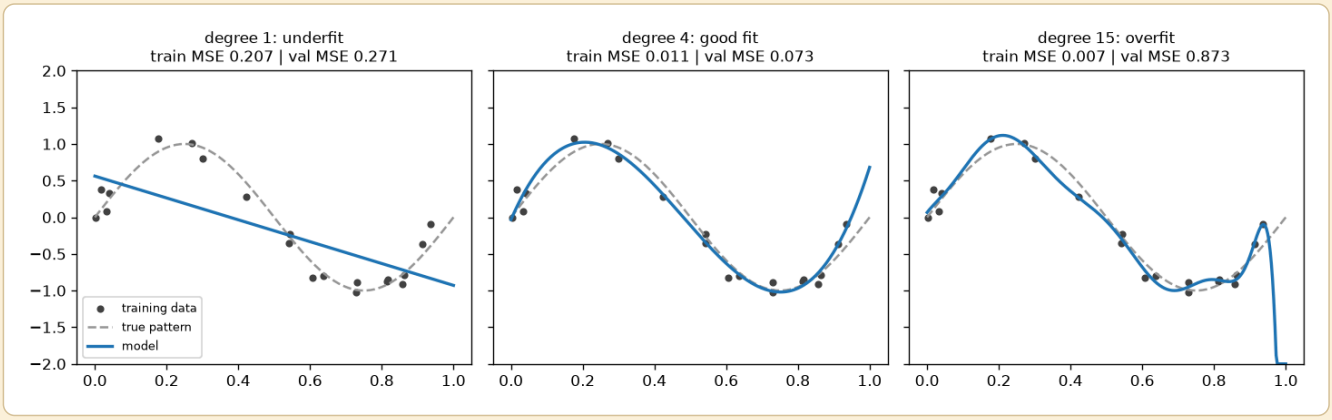
the amount of data, too many features, too little data, noisy data. **Fixes:** simpler model, fewer features, **regularisation** (a penalty on large parameters), more data, less noise, early stopping, cross-validation to choose complexity.

5. Underfitting

What it is. The model is too simple to capture the real pattern. **Signal:** training *and* validation error both high. **Fixes:** a more powerful model, better features, less regularisation, train longer.

Demo — fitting polynomials of degree 1, 4 and 15 to 20 noisy points from a sine curve:

```
degree  train MSE  val MSE
     1      0.207    0.271
     4      0.011    0.073
    15      0.007    0.873
```



Degree 1 is too rigid (both errors high). Degree 15 has the *lowest* training error but a validation error 12× worse than degree 4: it bends to pass through every noisy point and swings wildly between and beyond them. Degree 4 balances the two. This is the **bias–variance trade-off**: simple models have high *bias* (systematic error), complex ones high *variance* (sensitivity to the particular training sample). Chapter 6 treats it in depth.

Symptom	Train error	Validation error	Diagnosis	First fixes
Both high	High	High	Underfitting	More model capacity or better features
Big gap	Low	High	Overfitting	Regularise, simplify, more data
Both low, close	Low	Low	Good fit	Test once, ship

6. Data leakage

What it is. Information that would not be available at prediction time sneaks into training or evaluation, making scores look far better than real performance. Two common forms:

- **Target leakage:** a feature that is really a consequence of the label (e.g. "was prescribed antibiotics" when predicting "has infection"; "account closed date" when predicting churn).
- **Train–test contamination:** a preprocessing step (scaling, feature selection, imputation) is fit on the *whole* dataset before splitting, so the model has glimpsed the test data. Also: random splits of time-series data, or the same patient appearing in both train and test.

Demo — 100 rows of pure random noise with random labels (the honest answer is 50%):

```
# WRONG: select features using all rows, then cross-validate
X_leaky = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(max_iter=1000), X_leaky, y, cv=5).mean()
# RIGHT: selection inside the pipeline, refit on each training fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression(max_iter=1000))
right = cross_val_score(pipe, X, y, cv=5).mean()
```

```
leaky CV accuracy on pure noise: 0.85  |  honest CV accuracy: 0.48
```

85% accuracy on noise! Choosing the 20 features that best matched the labels, *using the test folds*, baked the answers in. **Real world:** a 2018 study (Zech et al.) found chest X-ray pneumonia models could recognise *which hospital* an image came from, and hospital correlated with disease rates, so part of the apparent skill did not transfer to new hospitals. **Fixes:** split first; put every data-dependent step in a `Pipeline`; split time series by time; group splits by patient/user; ask of every feature "would I know this at prediction time?"

When a result is too good to be true

Suspect leakage when a simple model scores near-perfectly, when one feature dominates importance, or when production performance falls far below test performance.

In the interview

> **30-second answer:** "The main risks are data problems — too little, noisy, non-representative or leaky data — and model problems — overfitting, where the model memorises noise and generalises poorly, and underfitting, where it's too simple. I diagnose by comparing training and validation error, guard against leakage with pipelines and proper splits, and check performance across subgroups."

- *How do you detect overfitting?* — Low training error with much higher validation error; learning curves.
- *Give a leakage example.* — Scaling or selecting features on the full dataset before cross-validation; or a feature recorded after the outcome.
- *More data or a better model?* — If you are overfitting, more data helps; if underfitting, more data of the same kind barely helps and you need a better model or features.

Self-check

1. Train MSE 0.01, validation MSE 0.9. Diagnosis? → Overfitting.
2. Why is fitting a scaler on the full dataset leakage? → The scaler's mean and std include test-set information.
3. A churn model's top feature is "cancellation reason". Problem? → Target leakage: the reason exists only after the customer churns.
4. How can a huge sample still be biased? → If it is drawn from an unrepresentative population (Literary Digest).

TL;DR

Bad data (too little, noisy, non-representative, leaky) and bad models (overfit, underfit) cause most failures. Compare train vs validation error, use pipelines, split correctly, and check each subgroup.

1.11 A first end-to-end project with scikit-learn

What it is

The whole workflow in one script, `code/first_end_to_end.py`, on the **Breast Cancer Wisconsin** dataset bundled with scikit-learn: 569 tumour samples, 30 numeric features computed from cell images (radius, texture, smoothness, ...), label malignant (0) or benign (1).

Framing: binary classification, supervised, batch. Missing a malignant tumour is worse than a false alarm, so beyond accuracy we care about **recall on the malignant class**.

How it works

```
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target

# 1. hold out a stratified test set first
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)

# 2. baseline: always predict the most common class
base = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)

# 3. pipeline = scaling + logistic regression; cross-validate on training data
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
cv = cross_val_score(pipe, X_train, y_train, cv=5)

# 4. tune the hyperparameter C with cross-validation
grid = GridSearchCV(pipe, {"logisticregression__C": [0.01, 0.1, 1, 10]}, cv=5)
grid.fit(X_train, y_train)

# 5. evaluate once on the test set
y_pred = grid.predict(X_test)

# 6. save the whole pipeline and reload it as a serving process would
joblib.dump(grid.best_estimator_, path)
served = joblib.load(path)
```

What each part does:

- `stratify=y` keeps the malignant/benign ratio equal in train and test.
- `DummyClassifier` gives the score to beat. Any real model must clearly exceed it.
- `StandardScaler` rescales each feature to mean 0, std 1; logistic regression trains better on scaled inputs. Being inside the pipeline, it is re-fit on each training fold, so no leakage.
- `GridSearchCV` tries each C with 5-fold CV and refits the best on all training data. The `step__param` naming (`logisticregression__C`) addresses a parameter inside a pipeline step.
- `joblib` saves the fitted pipeline (scaler + model) as one file; the Python docs and scikit-learn warn to load such files only from trusted sources, since unpickling can run code.

Output:

```
shape: (569, 30) | class counts: [212, 357]
train: 455 test: 114
baseline accuracy (test): 0.632
5-fold CV accuracy: 0.980 +/- 0.013
best C: 0.1 | best CV accuracy: 0.980
test accuracy: 0.974
confusion matrix [rows=true, cols=pred]:
 [[40  2]
 [ 1 71]]
              precision    recall  f1-score   support

   malignant      0.976     0.952     0.964        42
      benign      0.973     0.986     0.979        72

    accuracy                          0.974       114
   macro avg      0.974     0.969     0.972       114
weighted avg      0.974     0.974     0.974       114

prediction for a new sample: malignant | P(benign) = 0.0
learned parameters: coef_ shape (1, 30) intercept [0.504]
```

Reading it:

- Baseline 63.2% (always "benign") → model 97.4%. A real improvement.
- CV score (98.0%) and test score (97.4%) are close, so there is no sign of overfitting or leakage.
- The confusion matrix shows **2 malignant tumours predicted benign** (false negatives) — the costly error. Malignant recall is 40/42 = 0.952. In a real medical setting you might lower the decision threshold to trade some false alarms for fewer misses (Chapter 6).
- `coef_` holds 30 learned weights (parameters); C = 0.1 is the chosen hyperparameter.
- `P(benign) = 0.0` is rounded: the model is extremely confident about this particular sample.

Common mistakes

- Reporting the CV score of a tuned model as the final estimate (it is slightly optimistic; the test set is the honest number).
- Forgetting `stratify` on small or imbalanced datasets.
- Saving only the model and not the scaler, then feeding unscaled data in production.

Self-check

1. Why include `StandardScaler` inside the pipeline? → So it is fitted only on training folds (no leakage) and saved together with the model.
2. What is the baseline for, if it's useless? → It sets the minimum bar; a model that barely beats it is not worth deploying.
3. Which error matters most here and how many did we make? → False negatives (malignant predicted benign): 2 of 42.

TL;DR

Split (stratified) → baseline → pipeline → cross-validate → tune → test once → save the whole pipeline. That skeleton carries through every chapter.

Chapter TL;DR

- **AI ⊃ ML ⊃ DL.** GenAI creates content; agentic AI plans and acts. Choose the simplest branch that works.
- **ML learns rules from examples**; use it when rules are too complex or change too often, and you have good data.
- **Training** fits parameters by minimising a **loss**; **inference** applies the model; **feedback loops** feed real outcomes back into training.
- **Supervision types:** supervised (labels), unsupervised (none), semi-supervised (few), self-supervised (labels made from data), reinforcement (rewards).
- **Classification** predicts categories; **regression** predicts numbers.
- **Batch** learns all at once and retrains periodically; **online** updates continuously with `partial_fit`.
- **Instance-based** (k-NN) compares to stored examples; **model-based** fits parameters.
- **Workflow:** frame → data → features → model → evaluate → deploy → monitor, in a loop.
- **Parameters** are learned; **hyperparameters** are chosen on validation data; the **test set** is used once.
- **Main failures:** bad or non-representative data, overfitting (train ≪ validation error), underfitting (both high), and data leakage (too-good-to-be-true scores). Pipelines and correct splits prevent most of them.

Chapter 2 — The Math You Need for Machine Learning

You do not need to be a mathematician to do machine learning, but four areas of math appear in every model, every library call and every interview: **linear algebra** (how data and models are stored and combined), **calculus** (how models learn), **probability** (how models express uncertainty) and **statistics** (how we judge results). A fifth, small area, **information theory**, explains the most common loss function in classification.

This chapter teaches each idea with a plain definition, a small hand calculation, and NumPy code you can run. It keeps only what ML practitioners actually use.

What you'll learn

- Vectors, matrices, dot products and matrix multiplication, and why a model's prediction is "just" a matrix product
- Transpose, inverse, determinant and norms; the intuition behind eigenvectors and SVD
- Derivatives, partial derivatives, gradients and the chain rule
- Gradient descent: by hand, in code, with plots, and the role of the learning rate
- Random variables; Bernoulli, binomial and normal distributions; expectation and variance
- Conditional probability and Bayes' theorem with a worked medical-test example
- Mean, median, standard deviation, standardisation, correlation vs causation, sampling
- Hypothesis tests, p-values and confidence intervals, read correctly
- Entropy, cross-entropy and KL divergence, and why classifiers minimise "log loss"

All code is in `code/`. Notation: bold or plain lowercase letters (x, w) are vectors, uppercase (X, A) are matrices, ŷ is a prediction, Σ means "sum of", and · is a dot product.

2.1 Vectors

What it is

A **vector** is an ordered list of numbers, such as [2, 5, −3]. In ML, **one data point is one vector**: a house with 120 m², 3 bedrooms and 2 bathrooms is x = [120, 3, 2]. Each position is a **feature**, and the number of positions is the vector's **dimension**. Geometrically, a vector is an arrow from the origin to a point in space; a 3-feature house is a point in 3-D space, and a 784-pixel image is a point in 784-D space.

Why it matters

Once data is a vector, "similar houses" becomes "nearby points", "a model" becomes "a set of weights to combine the numbers", and the computer can process millions of rows with fast array operations.

How it works

Operation	Rule	Example with a = [1, 2, 3], b = [4, 5, 6]
Addition	Add position by position	a + b = [5, 7, 9]
Scalar multiplication	Multiply every entry	2a = [2, 4, 6]
Dot product a · b	Multiply position by position, then sum	1·4 + 2·5 + 3·6 = **32**
L2 norm $\|a\|_2$ (length)	√(sum of squares)	√(1 + 4 + 9) = √14 ≈ 3.74
L1 norm $\|a\|_1$	Sum of absolute values	1 + 2 + 3 = 6
Max norm $\|a\|_\infty$	Largest absolute value	3
Unit vector	a / $\|a\|_2$ (length 1, same direction)	[0.27, 0.53, 0.80]
Cosine similarity	(a · b) / (‖a‖ ‖b‖)	32 / (3.74 · 8.77) ≈ 0.975

The dot product is the most important operation in ML. A linear model's prediction is a dot product plus a constant:

$\hat{y} = w \cdot x + b = w_1x_1 + w_2x_2 + w_3x_3 + b$

With weights w = [2, 10, 5] and the house x = [120, 3, 2]: w · x = 2·120 + 10·3 + 5·2 = 240 + 30 + 10 = **280**.

Geometrically, a · b = ‖a‖ ‖b‖ cos θ, where θ is the angle between the arrows. So the dot product is large when vectors point the same way, zero when they are perpendicular, and negative when they point in opposite directions. **Cosine similarity** keeps only the angle; it is how search engines and recommendation systems compare text or item **embeddings** (learned vectors that represent meaning).

Norms measure size. The L2 norm is ordinary distance; the distance between two points is $\|a - b\|_2$. L1 and L2 norms of the *weights* appear as regularisation penalties (Lasso uses L1, Ridge uses L2).

Code

`code/linear_algebra.py`:

```
house = np.array([120.0, 3.0, 2.0])
w = np.array([2.0, 10.0, 5.0])
print("dot w.x =", w @ house)                    # @ is the dot / matrix product
a, b = np.array([1.0, 0.0]), np.array([1.0, 1.0])
cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
v = np.array([3.0, -4.0])
print(np.linalg.norm(v), np.linalg.norm(v, 1), np.linalg.norm(v, np.inf))
```

Output:

```
x = [120.   3.   2.] | w = [ 2. 10.  5.]
x + w = [122.  13.   7.] | 2 * x = [240.   6.   4.]
dot w.x = 280.0 (= 2*120 + 10*3 + 5*2)
cosine similarity of [1,0] and [1,1] = 0.707 (angle 45 deg)
L2 norm of [3,-4] = 5.0 | L1 norm = 7.0 | max norm = 4.0
unit vector = [ 0.6 -0.8]
```

Common mistakes

- Using `*` when you meant a dot product: `a * b` in NumPy is element-wise and returns a vector; `a @ b` (or `np.dot`) returns the sum.
- Comparing distances between vectors whose features have wildly different scales (m² in hundreds, bedrooms in units). The large feature dominates. Standardise first (§2.8).

Self-check

1. Dot product of [1, −1, 2] and [3, 4, 0.5]? → 3 − 4 + 1 = 0: the vectors are perpendicular.
2. L2 norm of [6, 8]? → 10.
3. Why is cosine similarity preferred over dot product for comparing documents? → It ignores length (document size) and compares only direction (content).

TL;DR

A data point is a vector. The dot product (multiply and sum) is how a linear model combines features; norms measure size and distance; cosine similarity measures direction.

2.2 Matrices and matrix multiplication

What it is

A **matrix** is a 2-D grid of numbers with m rows and n columns (shape m × n). In ML, **a dataset is a matrix**: each row is one example (a vector), each column is one feature. Four houses with three features form a 4 × 3 matrix X. Model weights are also stored as matrices: a neural-network layer that turns 3 inputs into 2 outputs has a 2 × 3 weight matrix.

How matrix multiplication works

To multiply A (m × n) by B (n × p):

1. The inner sizes must match (n = n). The result has shape m × p.
2. Entry (i, j) of the result = dot product of **row i of A** with **column j of B**.

Worked example:

```
A = | 1 2 |     B = | 5 6 |
    | 3 4 |         | 7 8 |

(A B)[0,0] = 1·5 + 2·7 = 19      (A B)[0,1] = 1·6 + 2·8 = 22
(A B)[1,0] = 3·5 + 4·7 = 43      (A B)[1,1] = 3·6 + 4·8 = 50
```

Order matters: in general **AB ≠ BA** (here BA = [[23, 34], [31, 46]]).

Why it matters: batch prediction is one matrix product. For all four houses at once, ŷ = Xw + b. Each row of X dotted with w gives one prediction. Libraries run this on optimised CPU/GPU code, which is why ML code avoids Python loops. A neural-network layer computes h = activation(Wx + b) in exactly the same way; deep networks are mostly long chains of matrix multiplications, which is why GPUs, built for fast matrix maths, power deep learning.

Transpose, identity, inverse, determinant

Concept	Definition	Example / use
Transpose A^T	Swap rows and columns (shape n × m)	$[[1, 2], [3, 4]]^T = [[1, 3], [2, 4]]$; X^T appears in gradients and covariance
Identity I	1s on the diagonal, 0s elsewhere; AI = A	The "1" of matrices
Inverse A^{-1}	The matrix with $A\,A^{-1} = I$ (only for square matrices with det ≠ 0)	"Undoes" A; used to solve linear equations
Determinant det(A)	One number measuring how A scales area/volume; 0 means A squashes space flat and has no inverse	For 2 × 2: det = ad − bc = 1·4 − 2·3 = **−2**

The 2 × 2 inverse by formula: $A^{-1} = (1/\det) \cdot [[d, -b], [-c, a]] = (1/-2) \cdot [[4, -2], [-3, 1]] = [[-2, 1], [1.5, -0.5]]$.

Solving equations. The system $x_1 + 2x_2 = 5$, $3x_1 + 4x_2 = 11$ is $Ax = c$. Then $x = A^{-1}c = [1, 2]$. In code, use `np.linalg.solve(A, c)` rather than computing the inverse: it is faster and numerically more accurate. The closed-form linear-regression solution (the "normal equation") $w = (X^TX)^{-1}X^Ty$ is solved the same way; scikit-learn's `LinearRegression` uses a least-squares solver based on matrix decompositions rather than an explicit inverse.

Code

```
A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
A @ B            # matrix product
A * B            # element-wise product (different!)
A.T              # transpose
np.linalg.det(A); np.linalg.inv(A)
np.linalg.solve(A, np.array([5.0, 11.0]))
X @ w + b        # predictions for all houses at once
```

Output:

```
X shape: (4, 3) | predictions X @ w + b = [330. 235. 505. 270.]
A @ B =
 [[19 22]
 [43 50]]
B @ A =
 [[23 34]
 [31 46]]
(A @ B != B @ A: order matters)
A * B (element-wise) =
 [[ 5 12]
 [21 32]]
A.T =
 [[1 3]
 [2 4]]
det(A) = -2.0
inv(A) =
 [[-2.   1. ]
 [ 1.5 -0.5]]
A @ inv(A) =
 [[1. 0.]
 [0. 1.]]
solve A x = [5, 11] -> [1. 2.]
layer output W @ x = [2.2 0.3]
```

First house check: 2·120 + 10·3 + 5·2 + 50 = 330. ✓

Common mistakes

- **Shape errors.** Before multiplying, write the shapes: (4 × 3)(3,) → (4,). If inner sizes differ, you need a transpose or you have the wrong matrix.
- Inverting nearly singular matrices (det ≈ 0), e.g. when two features are almost copies of each other (*multicollinearity*). Results blow up; regularisation or `solve`/`lstsq` helps.
- Confusing `*` (element-wise) with `@` (matrix product).

In the interview

30-second answer: "A dataset is a matrix with examples as rows and features as columns, and a linear model's predictions for the whole dataset are one matrix-vector product, Xw + b. Neural network layers are matrix multiplications followed by nonlinearities, which is why GPUs, which are very fast at matrix math, dominate deep learning."

- *Is matrix multiplication commutative?* — No; AB ≠ BA in general, and the shapes may not even allow BA.
- *When does a matrix have no inverse?* — When it is not square or its determinant is 0 (its columns are linearly dependent).

Self-check

1. Shape of (100 × 20) @ (20 × 5)? → 100 × 5.

2. det([[2, 1], [4, 2]])? → 2·2 − 1·4 = 0, so no inverse (the rows are multiples).
3. Why prefer `np.linalg.solve` to `inv`? → Faster and more numerically stable.

> **TL;DR**

A dataset is a matrix; predictions are Xw + b. Matrix multiplication = rows dotted with columns, order matters. Transpose swaps axes; the inverse undoes a matrix and exists only when det ≠ 0.

2.3 Eigenvalues, eigenvectors and SVD (intuition)

> **What it is**

Multiplying a vector by a matrix usually rotates and stretches it. An **eigenvector** of a square matrix A is a special direction that A only stretches (or flips), never rotates: A v = λ v. The stretch factor λ is the **eigenvalue**.

Worked example: A = [[2, 1], [1, 2]] and v = [1, 1]. A v = [2+1, 1+2] = [3, 3] = 3 · [1, 1]. So v is an eigenvector with eigenvalue 3. The other eigenvector is [1, −1] with eigenvalue 1 (A[1, −1] = [1, −1]).

Singular value decomposition (SVD) extends the idea to any matrix, square or not: every m × n matrix M can be written as $M = U \Sigma V^T$, meaning "rotate (V^T), stretch along axes (Σ), rotate again (U)". The **singular values** on Σ's diagonal, sorted largest first, say how much each direction matters.

Why it matters

- **PCA (principal component analysis)**, the standard dimensionality-reduction method, finds the eigenvectors of the data's covariance matrix (equivalently, the SVD of the centred data). The top eigenvectors are the directions along which data varies most; the eigenvalues say how much variance each captures.
- **Compression and denoising:** keeping only the top k singular values gives the best rank-k approximation of a matrix (Eckart–Young theorem).
- **Recommender systems:** matrix-factorisation methods, made famous during the Netflix Prize (2006–2009), approximate the huge user × movie ratings matrix as a product of two thin matrices, an idea closely related to SVD.
- **PageRank**, Google's original ranking algorithm, is the leading eigenvector of a matrix built from the web's link graph.

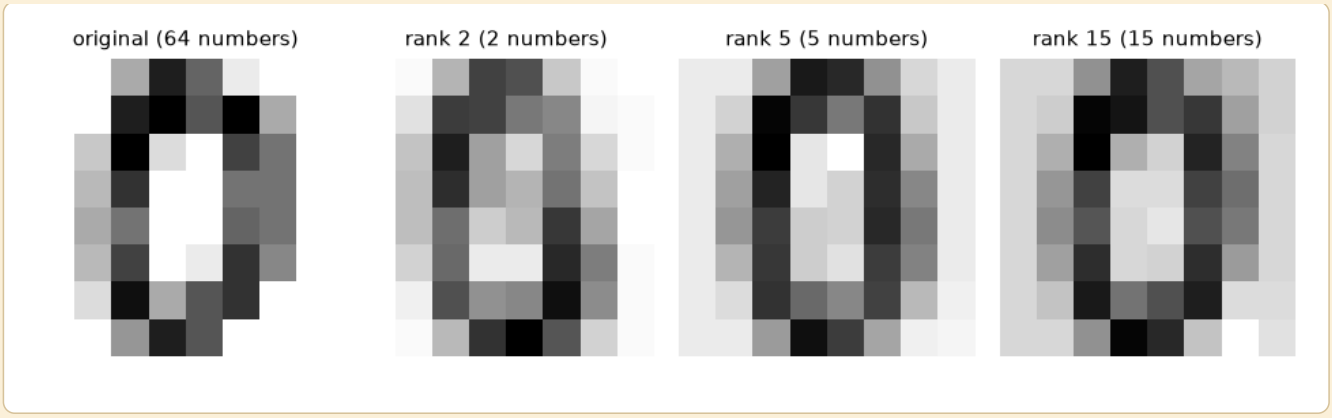
Code

```
vals, vecs = np.linalg.eigh(np.array([[2.0, 1.0], [1.0, 2.0]]))  # eigh: symmetric matrices
data = rng.multivariate_normal([0, 0], [[3, 2], [2, 2]], size=2000)
vals, vecs = np.linalg.eigh(np.cov(data, rowvar=False))           # PCA in two lines
U, s, Vt = np.linalg.svd(pixels - pixels.mean(0), full_matrices=False)
approx = (x @ Vt[:k].T) @ Vt[:k] + pixels.mean(0)                 # keep top-k directions
```

Output:

```
eigenvalues: [1. 3.]
eigenvectors (columns):
 [[-0.707  0.707]
 [ 0.707  0.707]]
A @ v = [2.121 2.121] = 3 * v = [2.121 2.121]
covariance eigenvalues: [0.433 4.584] -> share of variance on main axis: 0.914
singular values: [3.464 3.162]
rebuild U @ diag(s) @ Vt =
 [[ 3.  1.  1.]
 [-1.  3.  1.]]
top  5 of 64 singular values keep 54.5% of the variance
top 10 of 64 singular values keep 73.8% of the variance
top 20 of 64 singular values keep 89.4% of the variance
```

- The eigenvectors come out normalised to length 1, so [1, 1] appears as [0.707, 0.707].
- For the 2-D cloud, one direction holds 91% of the variance: you could describe each point with one number instead of two and lose little.
- On 8 × 8 digit images (64 pixels), 20 directions keep 89% of the variance:



Common mistakes

- Running PCA without centring (and usually standardising) the data.
- Using `np.linalg.eig` on symmetric matrices like covariance; `eigh` is faster, more stable and returns real, sorted eigenvalues.

Self-check

1. If A v = 5 v, what is v, and what is 5? → v is an eigenvector of A, 5 its eigenvalue.
2. What does a large singular value mean? → That direction carries a lot of the matrix's "energy" (variance).
3. Which ML technique is "eigenvectors of the covariance matrix"? → PCA.

TL;DR

Eigenvectors are directions a matrix only stretches; eigenvalues are the stretch amounts. SVD factors any matrix into rotate–stretch–rotate. Together they power PCA, compression and recommenders.

2.4 Derivatives, partial derivatives and gradients

What it is

- A **derivative** f'(x) is the slope of a function at a point: how much the output changes per tiny change in the input. For $f(x) = x^2$, $f'(x) = 2x$, so at $x = 3$ the slope is 6: nudge x up by 0.001 and f rises by about 0.006.
- A **partial derivative** $\partial f/\partial x$ is the slope with respect to one input while holding the others fixed.
- The **gradient** ∇f is the vector of all partial derivatives. It points in the direction of steepest *increase*; its negative points downhill.

Why it matters

A model's **loss** is a function of its parameters. The gradient of the loss tells each parameter which way to move, and by how much, to reduce the error. That is all "learning" is for most models, from linear regression to GPT-scale networks.

How it works: rules you actually need

Rule	Formula	Example
Constant	$(c)' = 0$	$(7)' = 0$
Power	$(x^n)' = n\,x^{n-1}$	$(x^3)' = 3x^2$
Sum	$(f + g)' = f' + g'$	$(x^2 + 3x)' = 2x + 3$
Constant multiple	$(c\,f)' = c\,f'$	$(5x^2)' = 10x$
Exponential / log	$(e^x)' = e^x$, $(\ln x)' = 1/x$	
Chain rule	$(f(g(x)))' = f'(g(x)) \cdot g'(x)$	$((3x + 1)^2)' = 2(3x + 1) \cdot 3$
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$	$\sigma(0) = 0.5$, $\sigma'(0) = 0.25$

Partial derivatives, worked. $f(x, y) = x^2y + 3y$ at (2, 5):

- $\partial f/\partial x$: treat y as a constant → $2xy = 2 \cdot 2 \cdot 5 = \mathbf{20}$
- $\partial f/\partial y$: treat x as a constant → $x^2 + 3 = 4 + 3 = \mathbf{7}$
- Gradient $\nabla f(2, 5) = [20, 7]$.

The chain rule (the engine of backpropagation)

When functions are nested, multiply the local slopes along the chain. A one-weight "neuron" with squared error:

$z = w \cdot x \to \hat{y} = \sigma(z) \to L = (\hat{y} - y)^2$

With $x = 2$, $y = 1$, $w = 0.5$:

1. Forward: $z = 1.0$, $\hat{y} = \sigma(1) = 0.7311$, $L = (0.7311 - 1)^2 = 0.0723$

2. Local slopes: $dL/d\hat{y} = 2(\hat{y} - y) = -0.5379$; $d\hat{y}/dz = \hat{y}(1 - \hat{y}) = 0.1966$; $dz/dw = x = 2$
3. Chain: $dL/dw = -0.5379 \times 0.1966 \times 2 = \mathbf{-0.2115}$

The negative sign says "increase w to reduce the loss" (makes sense: ŷ is below the target 1). **Backpropagation** is exactly this, applied efficiently to millions of weights, reusing shared intermediate products from the output layer backwards. Frameworks such as PyTorch compute it automatically (*autograd*).

Code: check every derivative numerically

`code/calculus.py` compares hand-derived formulas with the **central difference** approximation f'(x) ≈ (f(x + h) − f(x − h)) / 2h, a standard "gradient check":

```
def numerical_derivative(f, x, h=1e-5):
    return (f(x + h) - f(x - h)) / (2 * h)

# gradient of MSE for linear regression, vectorised: (2/n) X^T (Xw - y)
grad = 2 / len(yv) * X.T @ (X @ wv - yv)
```

Output:

```
d/dx x^2 at x=3: numeric 6.0000, exact 6
gradient of x^2*y + 3y at (2,5): numeric [20.  7.] exact [20. 7.]
d/dx (3x+1)^2 at x=2: numeric 42.0000, exact 42
z=1.0, y_hat=0.7311, dL/dyhat=-0.5379, dyhat/dz=0.1966, dz/dw=2.0
dL/dw via chain rule -0.21151 | numeric -0.21151
MSE gradient: formula [-4.         -9.33333333] | numeric [-4.     -9.3333]
```

The MSE gradient formula, which you will see everywhere: for $L(w) = (1/n)\ \Sigma\ (x_i \cdot w - y_i)^2$, $\nabla L = (2/n)\ X^T(Xw - y)$. In words: multiply each example's error by its features and average. Here X has a column of 1s (so w[0] acts as the bias), y = [2, 4, 6], w = [0, 1]; the errors are [−1, −2, −3], giving gradient [2/3·(−6), 2/3·(−14)] = [−4, −9.33].

Common mistakes

- Forgetting a factor from the inner function in the chain rule (the "· 3" in ((3x + 1)²)').
- Thinking the gradient points downhill; it points uphill, so we step along **minus** the gradient.

Self-check

1. Derivative of $4x^3 - 2x$? → $12x^2 - 2$.
2. $f(x, y) = 3x + xy^2$. $\partial f/\partial y$ at (1, 2)? → $2xy = 4$.
3. Why does σ'(z) get tiny for very large |z|? → σ flattens near 0 and 1, so σ(1 − σ) → 0; this is the "vanishing gradient" problem in deep nets.

TL;DR

Derivative = slope; partial derivative = slope along one input; gradient = vector of partials pointing uphill. The chain rule multiplies slopes through nested functions and is how backpropagation computes gradients.

2.5 Gradient descent

What it is

Gradient descent is an algorithm for finding the parameters that minimise a function: start somewhere, compute the gradient, take a small step in the opposite direction, repeat. Picture walking down a foggy hill by always stepping in the steepest downhill direction you can feel under your feet.

Update rule: **$w \leftarrow w - \eta \cdot \nabla L(w)$**, where η (eta) is the **learning rate**, a hyperparameter that sets the step size.

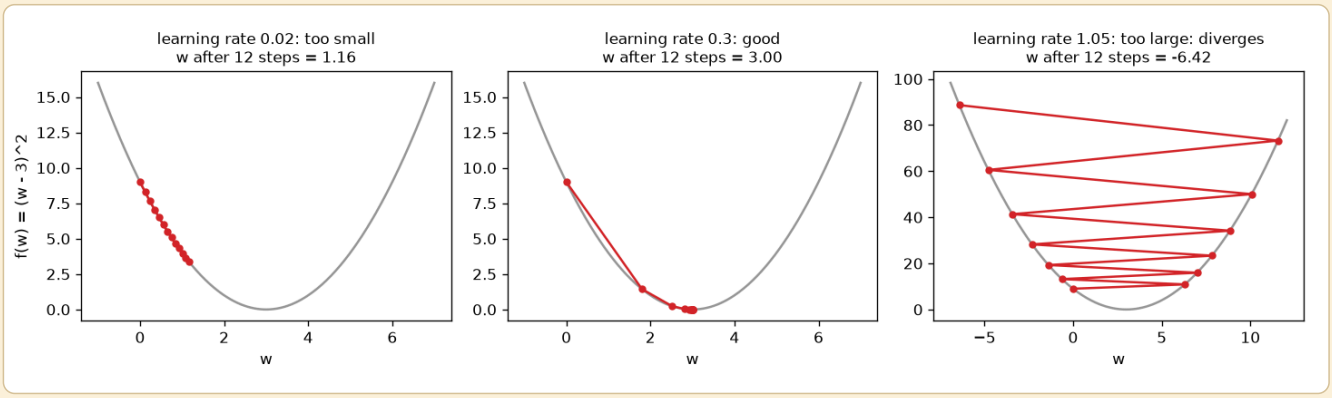
Worked by hand

Minimise $f(w) = (w - 3)^2$, whose minimum is at w = 3. Gradient: f'(w) = 2(w − 3). Start at w = 0 with η = 0.1.

Step	w	f(w)	gradient 2(w − 3)	new w = w − 0.1 · gradient
0	0.0000	9.0000	−6.0000	0 + 0.6 = 0.6000
1	0.6000	5.7600	−4.8000	0.6 + 0.48 = 1.0800
2	1.0800	3.6864	−3.8400	1.4640
3	1.4640	2.3593	−3.0720	1.7712
4	1.7712	1.5099	−2.4576	2.0170

Each step closes 20% of the remaining distance to 3 (because w − 3 is multiplied by 1 − 2η = 0.8), and steps shrink as the slope flattens. The script prints the same table.

The learning rate



- **Too small (0.02):** safe but slow; after 12 steps w is only 1.16.
- **Good (0.3):** reaches 3.00 in 12 steps.
- **Too large (1.05):** each step overshoots further than the last; w bounces across the valley and diverges (−6.42 and growing). For this function, any η > 1 diverges because |1 − 2η| > 1.

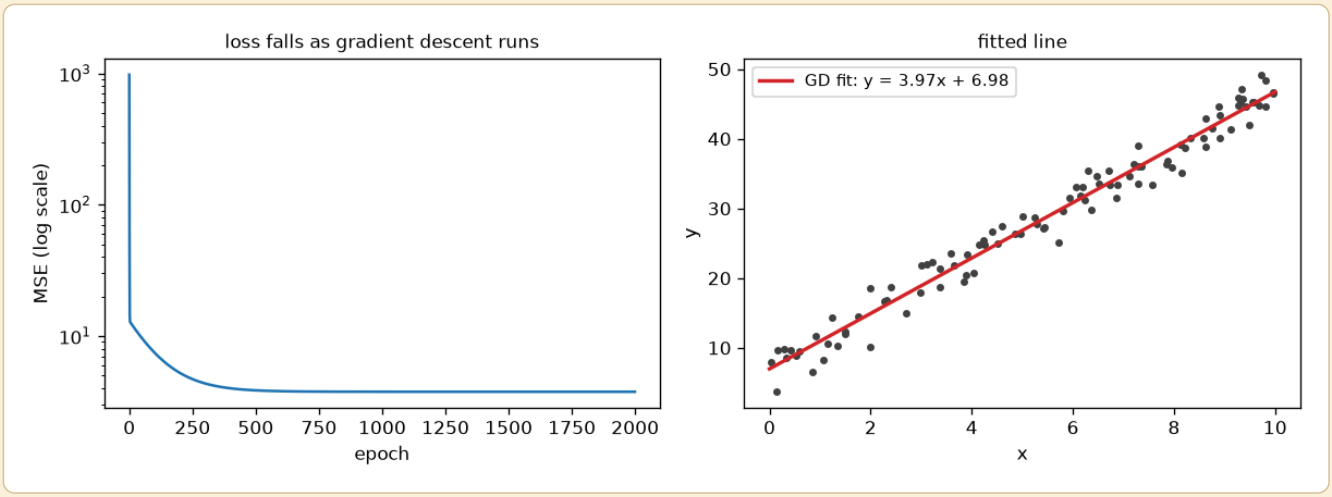
Gradient descent for linear regression

`code/gradient_descent.py` fits y = w·x + b to 100 noisy points generated from y = 4x + 7:

```
w, b, lr = 0.0, 0.0, 0.01
for epoch in range(2000):
    err = (w * x + b) - y
    dw = 2 * np.mean(err * x)      # dMSE/dw
    db = 2 * np.mean(err)          # dMSE/db
    w -= lr * dw
    b -= lr * db
```

Output:

```
epoch    0: w=0.000 b=0.000 MSE=976.742
epoch    1: w=3.885 b=0.575 MSE=51.369
epoch   10: w=4.820 b=0.966 MSE=12.240
epoch  100: w=4.533 b=3.007 MSE=7.464
epoch 1999: w=3.974 b=6.984 MSE=3.760
scikit-learn closed form: w=3.974 b=6.985
```



Gradient descent lands on the same answer as scikit-learn's exact solver. Notice w shoots past 4 early on and b creeps up slowly: the loss surface is a long, narrow valley because x (0–10) and the constant 1 have very different scales. **Standardising features** makes the valley rounder and gradient descent much faster, one reason feature scaling matters.

Variants

Variant	Gradient computed on	Pros	Cons
Batch GD	All n examples per step	Smooth, exact gradient	Slow per step on big data
Stochastic GD (SGD)	One random example per step	Very cheap steps; noise can escape shallow local minima	Noisy; needs a decaying learning rate
Mini-batch GD	A small batch (e.g. 32–512)	Best of both; uses GPU parallelism	One more hyperparameter (batch size)

One pass over the whole training set is an **epoch**. Deep-learning optimisers such as Momentum, RMSProp and Adam are mini-batch gradient descent with smarter, adaptive step sizes.

Convex vs non-convex. A convex loss (bowl-shaped, like MSE for linear regression or log loss for logistic regression) has no separate local minima: every minimum is a global one, so gradient descent with a suitable learning rate reaches the best possible loss. Neural-network losses are non-convex, with many local minima and saddle points; in practice, mini-batch methods still find good solutions.

When not to use it

If a closed-form or specialised solver exists and the data fits in memory (small linear regression), use it. Gradient descent shines when data is large, the model is complex, or no closed form exists.

Common mistakes

- Unscaled features → slow, zig-zagging convergence.
- Learning rate too large → loss grows or becomes NaN. First fix: divide η by 10.
- Not shuffling data between epochs for SGD.
- Stopping too early; plot the loss curve.

In the interview

> **30-second answer:** "Gradient descent minimises a loss by repeatedly moving parameters a small step opposite the gradient: $w \leftarrow w - \eta \nabla L$. The learning rate η controls step size; too small is slow, too large diverges. In practice we use mini-batch stochastic gradient descent, often with an adaptive optimiser like Adam, and we scale features so it converges quickly."

- *Why mini-batches?* — Cheaper than full-batch, less noisy than single examples, and maps well to GPU parallelism.
- *Does gradient descent always find the global minimum?* — Only guaranteed for convex problems with a suitable learning rate.
- *What is an epoch?* — One full pass through the training data.

Self-check

1. Take one step on $f(w) = w^2$ from $w = 4$ with $\eta = 0.25$. → gradient 8, $w = 4 - 2 = 2$.
2. Loss goes up every epoch. First suspect? → Learning rate too high.
3. Why does scaling features help gradient descent? → It makes the loss surface more evenly curved, so one learning rate suits all directions.

TL;DR

$w \leftarrow w - \eta \nabla L$. Pick η carefully, scale features, use mini-batches, watch the loss curve.

2.6 Random variables and distributions

What it is

A **random variable** is a quantity whose value comes from a random process: the outcome of a die roll, whether an email is spam, tomorrow's temperature. **Discrete** random variables take countable values (0, 1, 2, …); **continuous** ones take any value in a range. A **probability distribution** lists how likely each value is: a *probability mass function (PMF)* for discrete, a *probability density function (PDF)* for continuous (probabilities are areas under the PDF).

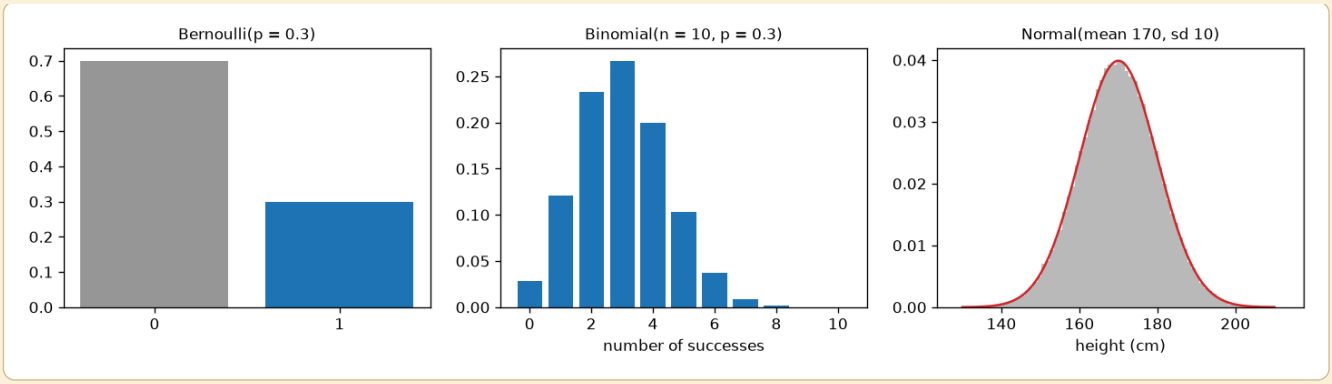
Probability itself is a number between 0 (impossible) and 1 (certain) measuring how likely an event is, for example P(spam) = 0.2.

The three distributions you'll meet most

Distribution	Models	Parameters	Mean	Variance	ML use
Bernoulli	One yes/no trial	$p = P(1)$	p	$p(1 - p)$	A binary label; logistic regression outputs p
Binomial	Number of successes in n independent trials	n, p	np	$np(1 - p)$	Count of clicks out of n impressions; A/B tests
Normal (Gaussian)	Continuous values clustered symmetrically around a mean	μ (mean), σ (std)	μ	σ^2	Measurement noise, heights, errors of linear regression; weight initialisation

Worked binomial: P(exactly 3 successes in 10 trials, $p = 0.3$) = $C(10, 3) \cdot 0.3^3 \cdot 0.7^7 = 120 \cdot 0.027 \cdot 0.0824 \approx$ **0.2668**. $C(10, 3) = 120$ is the number of ways to choose which 3 trials succeed.

The **68–95–99.7 rule** for normal data: about 68% of values lie within 1σ of the mean, 95% within 2σ, 99.7% within 3σ. For heights ~ Normal(170, 10): P(height > 190) is the chance of being more than $z = (190 - 170)/10 = 2$ standard deviations above the mean ≈ **2.3%**.



Expectation and variance

- **Expectation (mean)** E[X] = Σ x · P(x): the long-run average. Fair die: (1 + 2 + … + 6)/6 = **3.5**.
- **Variance** Var[X] = E[(X − E[X])²]: the average squared distance from the mean. Die: [(−2.5)² + (−1.5)² + (−0.5)² + 0.5² + 1.5² + 2.5²]/6 = 17.5/6 ≈ **2.917**.
- **Standard deviation** σ = √Var, in the same units as X (die: ≈ 1.71).

Useful rules: E[aX + b] = aE[X] + b; Var[aX + b] = a²Var[X]; for independent X and Y, Var[X + Y] = Var[X] + Var[Y].

Code

`code/probability.py` simulates each distribution and compares to theory:

```
flips = rng.random(100_000) < 0.3                # Bernoulli(0.3)
draws = rng.binomial(10, 0.3, 100_000)           # Binomial(10, 0.3)
heights = rng.normal(170, 10, 100_000)           # Normal(170, 10)
stats.binom.pmf(3, 10, 0.3); 1 - stats.norm.cdf(190, 170, 10)
```

Output:

```
Bernoulli(0.3): sample mean 0.299 (theory 0.3), variance 0.210 (theory p(1-p) = 0.21)
Binomial(10, 0.3): P(X=3) = C(10,3) 0.3^3 0.7^7 = 0.2668 | scipy 0.2668
  mean 2.994 (theory np = 3.0), variance 2.093 (theory np(1-p) = 2.1)
Normal(170, 10): within 1 sd 0.683, within 2 sd 0.954 (theory 0.683, 0.954)
  P(height > 190) = 0.0228  (z = (190-170)/10 = 2)
die: E[X] = 3.500, Var[X] = 2.917 | simulated mean 3.510, var 2.925
```

Common mistakes

- Reading a PDF value as a probability. For continuous variables, P(X = exactly 170) = 0; only ranges have probability.
- Assuming everything is normal. Incomes, session lengths and prices are usually right-skewed; a log transform often helps.

Self-check

1. Mean and variance of Binomial(100, 0.1)? → 10 and 9.
2. Values within 2σ of the mean for normal data? → About 95%.
3. Var[3X] if Var[X] = 2? → 9 · 2 = 18.

TL;DR

Random variables have distributions. Bernoulli = one yes/no, binomial = count of yeses, normal = symmetric bell. Expectation is the long-run average, variance the average squared spread.

2.7 Conditional probability and Bayes' theorem

What it is

Conditional probability P(A | B) is the probability of A *given that* B happened: P(A | B) = P(A and B) / P(B). Knowing B narrows the world to the cases where B is true.

Worked example with counts: of 1,000 emails, 200 are spam; 120 spam emails and 40 normal emails contain "free". Given an email contains "free", P(spam | free) = 120 / (120 + 40) = **0.75**, much higher than the base rate P(spam) = 0.2.

Two events are **independent** if P(A | B) = P(A): knowing B tells you nothing about A.

Bayes' theorem flips a conditional around:

P(A | B) = P(B | A) · P(A) / P(B)

In words: **posterior = likelihood × prior / evidence**.

- **Prior** P(A): what you believed before seeing data.
- **Likelihood** P(B | A): how probable the data is if A is true.

- **Evidence** P(B): the overall probability of the data, = P(B | A)P(A) + P(B | not A)P(not A).
- **Posterior** P(A | B): the updated belief.

Worked example: a medical test

A disease affects 1% of people. A test catches 90% of sick people (sensitivity, P(+ | sick) = 0.90) and wrongly flags 5% of healthy people (false-positive rate 0.05, so specificity 0.95). You test positive. How likely are you to be sick?

1. Prior: P(sick) = 0.01
2. Evidence: P(+) = 0.90 · 0.01 + 0.05 · 0.99 = 0.009 + 0.0495 = **0.0585**
3. Posterior: P(sick | +) = 0.009 / 0.0585 = **0.154**

Only about 15%! Intuition with 10,000 people: 100 are sick and 90 test positive; 9,900 are healthy and 495 test positive. Of 585 positives, only 90 are sick. When a condition is rare, false positives from the large healthy group swamp true positives. This is the **base-rate fallacy**, and it matters for every rare-event classifier (fraud, rare diseases, intrusion detection).

Bayes also chains: after a *second* independent positive test, the posterior from step 3 becomes the new prior: 0.9 · 0.154 / (0.9 · 0.154 + 0.05 · 0.846) ≈ **0.766**.

```
evidence = sens * prior + fpr * (1 - prior)
posterior = sens * prior / evidence
```

Output (including a 1,000,000-person simulation):

```
P(free) = 0.16, P(spam | free) = 120/160 = 0.75
P(positive) = 0.9*0.01 + 0.05*0.99 = 0.0585
P(disease | positive) = 0.009 / 0.0585 = 0.154
simulation: P(disease | positive) = 0.153
after a second independent positive test: 0.766
```

Real-world examples

- **Naive Bayes spam filters** apply Bayes' theorem with the simplifying ("naive") assumption that words are independent given the class. Paul Graham's 2002 essay "A Plan for Spam" popularised Bayesian spam filtering.
- **Probability calibration:** a fraud model's 0.9 score should mean 90% of such cases are fraud; Bayes explains why scores trained on balanced samples must be adjusted when real fraud is rare.

Common mistakes

- Confusing P(A | B) with P(B | A): P(positive | sick) = 0.9 is not P(sick | positive) = 0.15.
- Ignoring the prior (base rate).

In the interview

30-second answer: "Bayes' theorem updates a prior belief with evidence: P(A|B) = P(B|A)P(A)/P(B). A classic example is a test with 90% sensitivity and 5% false positives for a disease with 1% prevalence: a positive result gives only about a 15% chance of disease, because the healthy population is so much larger. It underlies Naive Bayes and explains why precision is low for rare-event classifiers."

- *What's "naive" in Naive Bayes?* — It assumes features are conditionally independent given the class.
- *How does this relate to precision?* — P(sick | positive) is exactly the precision of the test.

Self-check

1. P(A and B) = 0.1, P(B) = 0.4. P(A | B)? → 0.25.
2. Why is the posterior so low in the medical example? → The prior is tiny, so false positives from healthy people outnumber true positives.
3. Name the four parts of Bayes' theorem. → Prior, likelihood, evidence, posterior.

TL;DR

P(A | B) = P(A and B) / P(B). Bayes: posterior ∝ likelihood × prior. Rare events + imperfect tests = many false alarms.

2.8 Descriptive statistics and standardisation

What it is

Statistics summarises data and quantifies how much we can trust conclusions drawn from it.

Measure	Definition	Robust to outliers?
Mean	Sum ÷ count; the "central tendency"	No
Median	Middle value when sorted	Yes
Mode	Most frequent value	Yes
Variance	Average squared distance from the mean; high = spread out	No
Standard deviation	√variance, in original units	No
Percentiles / IQR	Values below which a % of data falls; IQR = 75th − 25th percentile	Yes

Worked example. Salaries (in $1000s): 40, 42, 45, 47, 50, 52, 55, 300 (one CEO).

- Mean = 631 / 8 = **78.9**; median = (47 + 50)/2 = **48.5**. The single outlier drags the mean far above what a typical person earns; the median barely notices.
- **Population vs sample std:** dividing by n (NumPy's default `ddof=0`) describes this exact group; dividing by n − 1 (`ddof=1`, pandas' default) gives an unbiased variance estimate when the data is a sample from a bigger population. For large n the difference is negligible.

Standardisation (z-scores). Subtract the mean and divide by the standard deviation: z = (x − μ)/σ. The result has mean 0 and standard deviation 1, and z says "how many standard deviations from the mean". For x = [2, 4, 4, 4, 5, 5, 7, 9]: μ = 5, σ = 2, so 9 → (9 − 5)/2 = **2.0** and 2 → **−1.5**.

Standardising puts features on a common scale so that no feature dominates just because of its units. It speeds up gradient descent and is essential for distance-based methods (k-NN, k-means, SVMs) and regularised models. scikit-learn's `StandardScaler` does exactly this, learning μ and σ from the training data only.

Output of `code/statistics_basics.py`:

```
mean 78.9, median 48.5
population std (ddof=0) 83.7, sample std (ddof=1) 89.5
without the outlier: mean 47.3, median 47.0
x mean 5.0, std 2.0 -> standardised z = [-1.5 -0.5 -0.5 -0.5  0.   0.   1.   2. ]
z mean 0.0, z std 1.0
```

Self-check

1. House prices are right-skewed. Mean or median for a "typical" price? → Median.
2. z-score of 130 when μ = 100, σ = 15? → 2.
3. Why fit the scaler on training data only? → Using test data to compute μ and σ leaks test information.

TL;DR

Mean and std summarise centre and spread but are pulled by outliers; median and IQR are robust. Standardise features to mean 0, std 1 before distance-based or gradient-based models.

2.9 Correlation vs causation

What it is

Correlation (Pearson's r, from −1 to +1) measures how strongly two variables move together *in a straight line*. r = +1: perfect increasing line; −1: perfect decreasing line; 0: no *linear* relationship. **Causation** means changing one variable actually changes the other.

Correlation does not prove causation. Common reasons for correlation without causation:

1. **Confounder:** a third variable drives both. Hot weather increases both ice-cream sales and swimming (hence drownings).
2. **Reverse causation:** the arrow runs the other way.
3. **Coincidence:** with enough variables, some correlate by chance.
4. **Selection bias:** the way data was collected creates the pattern.

```
temp = rng.uniform(10, 35, 365)
ice_cream = 20 * temp + noise;  drownings = 0.3 * temp + noise
np.corrcoef(ice_cream, drownings)          # strong correlation
# remove temperature's effect from both, then correlate what is left
```

Output:

```
corr(ice cream, drownings) = 0.77
corr after controlling for temperature = -0.06
corr(x, x^2) = -0.00  (perfect dependence, zero LINEAR correlation)
```

Once temperature is accounted for, the relationship vanishes. The last line shows the opposite trap: y = x² is completely determined by x, yet r = 0 because the relationship is not a line.

Why it matters for ML

Most ML models find correlations, not causes. That is fine for *prediction* (umbrella sales predict rain), but dangerous for *decisions* ("sell more umbrellas to cause rain"). To learn causal effects you need **randomised experiments (A/B tests)** or specialised causal-inference methods.

Self-check

1. Countries with more hospitals have more deaths. Causal? → Probably a confounder: population size.
2. r = 0. Are the variables independent? → Not necessarily; there may be a non-linear relationship.

TL;DR

Correlation measures linear co-movement; it can come from confounders, reverse causation or chance. Use experiments to establish cause.

2.10 Sampling and the central limit theorem

What it is

A **population** is everything you care about (all customers); a **sample** is the part you measure. A **statistic** computed from a sample (e.g. the sample mean) estimates a **parameter** of the population. A good sample is **random** and **representative**; biased samples give biased estimates no matter how large (Chapter 1's *Literary Digest* poll).

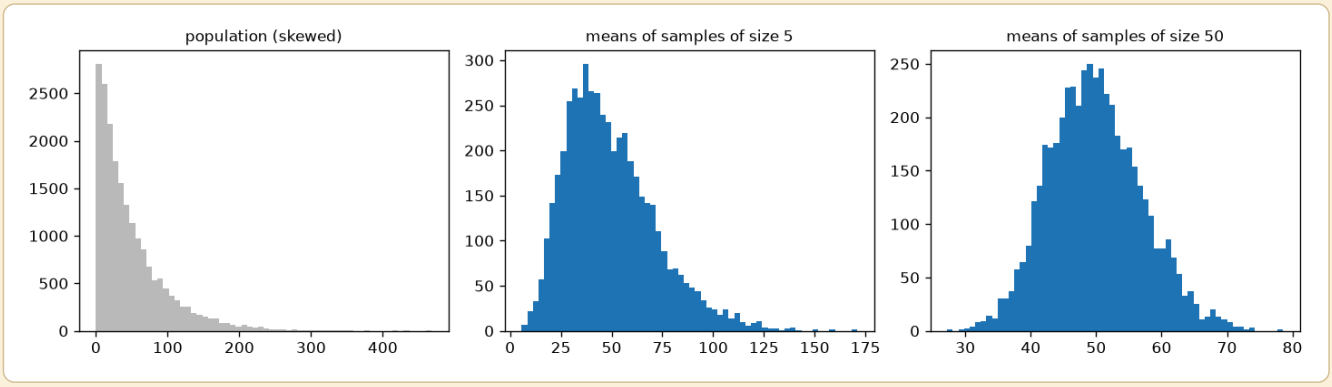
The **central limit theorem (CLT)**: if you take many random samples of size n and compute each sample's mean, those means are approximately normally distributed around the true mean, even if the population itself is skewed, provided n is reasonably large. Their spread, the **standard error**, is **σ/√n**. Quadrupling the sample size halves the uncertainty.

Code

```
population = rng.exponential(scale=50, size=1_000_000)       # skewed
means = rng.choice(population, size=(5000, n)).mean(axis=1)  # 5000 samples of size n
```

Output:

```
population mean 49.99, std 50.07
n= 5: mean of sample means 49.81, std of sample means 22.48 (theory sigma/sqrt(n) = 22.39)
n=50: mean of sample means 49.89, std of sample means 7.11 (theory sigma/sqrt(n) = 7.08)
```



With n = 50 the sample means form a bell curve even though the population is heavily skewed. The CLT is why normal-based confidence intervals and t-tests work for averages of most real data, and why larger test sets give more reliable accuracy estimates.

Self-check

1. Standard error of a mean with σ = 20, n = 100? → 2.
2. How much larger must a sample be to halve the standard error? → 4×.

TL;DR

Sample means cluster normally around the truth with spread σ/√n. Random, representative samples matter more than big ones.

2.11 Hypothesis tests, p-values and confidence intervals

What it is

A **hypothesis test** asks whether an observed effect could plausibly be random noise.

1. State the **null hypothesis H_0** (no effect: "the new model does not change load time") and the **alternative H_1** (there is an effect).
2. Choose a **significance level α** before looking, usually 0.05.
3. Compute a **test statistic** from the data (e.g. a t-statistic: the difference in means divided by its standard error).

4. Compute the **p-value**: *if H_0 were true*, the probability of seeing a result at least this extreme.
5. If p < α, **reject H_0** ("statistically significant"); otherwise you **fail to reject** it (not the same as proving H_0).

Error	Meaning	Probability
Type I (false positive)	Reject H_0 when it is true	α
Type II (false negative)	Miss a real effect	β; **power** = 1 − β

A **confidence interval (CI)** gives a range of plausible values for a parameter: estimate ± (critical value × standard error). A 95% CI is built by a method that, over many repeated experiments, captures the true value 95% of the time.

Worked example: an A/B test

Page load times (ms), 60 users each: old model mean 202.7, new model mean 187.3, difference −15.4 ms. Welch's t-test (which does not assume equal variances):

```
t, p = stats.ttest_ind(new, old, equal_var=False)
m, se = new.mean(), new.std(ddof=1) / np.sqrt(len(new))
tcrit = stats.t.ppf(0.975, df=len(new) - 1)
```

Output:

```
old mean 202.7, new mean 187.3, diff -15.4 ms
Welch t = -2.72, p-value = 0.0075 -> reject H0 at 0.05
95% CI for new mean: 187.3 +/- 2.001*4.02 = [179.2, 195.3]
accuracy 0.850, bootstrap 95% CI [0.800, 0.900]
coverage of 2000 simulated 95% CIs: 0.949
```

- p = 0.0075: if the new model truly made no difference, a gap this large would appear less than 1% of the time. We reject H_0.
- The CI for the new mean is 187.3 ± 2.001 × 4.02 (2.001 is the t critical value for 59 degrees of freedom; it approaches 1.96 for large samples).
- **Bootstrap CI** for a model's test accuracy: resample the 200 test predictions with replacement 5,000 times and take the 2.5th and 97.5th percentiles. An accuracy of 85% on 200 examples is really "somewhere between about 80% and 90%": worth remembering before celebrating a 1-point improvement on a small test set.
- The last line verifies the meaning of "95%": in 2,000 simulated experiments, 94.9% of the intervals contained the true mean.

Common mistakes

- **"p = 0.03 means a 3% chance H_0 is true."** No. The p-value assumes H_0 is true and measures how surprising the data is.
- **Significant ≠ important.** With huge samples, trivial differences become significant; always report the effect size and its CI.
- **Multiple comparisons / p-hacking:** test 20 useless features at α = 0.05 and expect one "significant" result by chance. Correct for it (e.g. Bonferroni: use α/20) or pre-register the test.
- Stopping an A/B test the moment p dips below 0.05 ("peeking") inflates false positives.

In the interview

30-second answer: "A p-value is the probability of seeing data at least as extreme as ours if the null hypothesis were true. If it's below a pre-chosen α, like 0.05, we reject the null. It's not the probability the null is true, and significance isn't practical importance, so I report effect sizes with confidence intervals and correct for multiple comparisons."

- *What's statistical power?* — The probability of detecting a real effect; it rises with sample size and effect size.
- *How would you compare two models' accuracy?* — Evaluate on the same test set and use a paired test (e.g. McNemar's test) or bootstrap the difference.

Self-check

1. p = 0.2, α = 0.05. Conclusion? → Fail to reject H_0; the data is consistent with no effect (it doesn't prove no effect).
2. What does a 95% CI of [0.80, 0.90] for accuracy tell you? → The true accuracy is plausibly anywhere in that range; small differences inside it may be noise.
3. Type I error? → A false positive: rejecting a true null.

TL;DR

Set α, compute p; p < α → reject H_0. p is not P(H_0 true). Report effect sizes with confidence intervals, beware multiple testing.

2.12 Information theory: entropy, cross-entropy and KL divergence

What it is

- **Entropy** H(p) = −Σ p(x) $\log_2$ p(x) measures the average **uncertainty** (or surprise) in a distribution, in bits. A fair coin has 1 bit; a coin that lands heads 90% of the time has 0.469 bits; a certain outcome has 0.

- **Cross-entropy** $H(p, q) = -\Sigma\, p(x) \log q(x)$ measures the average surprise when reality follows p but you *predict* with q. It is always $\geq H(p)$, and equals it only when $q = p$.
- **KL divergence** $KL(p \parallel q) = \Sigma\, p(x) \log (p(x)/q(x)) = H(p, q) - H(p)$: the *extra* surprise from using q instead of the truth. It is ≥ 0, is 0 only when $q = p$, and is **not symmetric**.

Why it matters

- **Classification loss.** For a labelled example, the true distribution p is one-hot (probability 1 on the right class). Cross-entropy then simplifies to $-\log q$(correct class): the model is penalised by how little probability it gave the right answer. Averaged over the data, this is **log loss**, the loss minimised by logistic regression and nearly every neural-network classifier, including language models predicting the next token.
- **Decision trees** choose splits that most reduce entropy (information gain).
- **KL divergence** appears in variational autoencoders, knowledge distillation, and in RLHF, where it keeps a fine-tuned language model close to its original.

Worked numbers

True $p = [0.7, 0.2, 0.1]$, model $q = [0.5, 0.3, 0.2]$ (base 2):

- $H(p) = -(0.7 \log_2 0.7 + 0.2 \log_2 0.2 + 0.1 \log_2 0.1) = 0.360 + 0.464 + 0.332 =$ **1.157 bits**
- $H(p, q) = -(0.7 \log_2 0.5 + 0.2 \log_2 0.3 + 0.1 \log_2 0.2) = 0.700 + 0.347 + 0.232 =$ **1.280 bits**
- $KL(p \parallel q) = 1.280 - 1.157 =$ **0.123 bits**

Log loss for labels [1, 0, 1, 1] and predicted P(class 1) = [0.9, 0.2, 0.6, 0.99] (natural log): $-\frac{1}{4}$ [ln 0.9 + ln 0.8 + ln 0.6 + ln 0.99] = $\frac{1}{4}$ [0.105 + 0.223 + 0.511 + 0.010] = **0.2123**.

```
manual = -np.mean(y_true * np.log(p_pred) + (1 - y_true) * np.log(1 - p_pred))
log_loss(y_true, p_pred)   # scikit-learn gives the same number
```

Output of `code/information_theory.py`:

```
entropy of fair coin            = 1.000 bits
entropy of biased coin 0.9/0.1  = 0.469 bits
entropy of certain              = 0.000 bits
entropy of fair 8-sided die     = 3.000 bits

H(p) = 1.157, H(p, q) = 1.280, KL(p||q) = 0.123, H(p) + KL = 1.280
KL(q||p) = 0.133  (KL is not symmetric)

log loss by hand = 0.2123 | sklearn = 0.2123
  true class predicted with p=0.99: loss = 0.010
  true class predicted with p=0.6 : loss = 0.511
  true class predicted with p=0.1 : loss = 2.303
  true class predicted with p=0.01: loss = 4.605
```

The last four lines show why log loss punishes **confident mistakes** heavily: giving the true class 1% probability costs 460 times more than giving it 99%.

Common mistakes

- Mixing log bases: information theory often uses $\log_2$ (bits); ML libraries use the natural log (nats). Only the scale changes.
- Treating KL as a distance; it is asymmetric and does not satisfy the triangle inequality.
- Predicting exact 0 or 1 probabilities; log(0) is $-\infty$. Libraries clip probabilities for this reason.

Self-check

1. Entropy of a fair 4-sided die? → $\log_2 4 = 2$ bits.
2. Cross-entropy loss when the true class gets probability 0.5? → $-\ln 0.5 \approx 0.693$.
3. When is $KL(p \parallel q) = 0$? → When $q = p$.

TL;DR

Entropy = uncertainty; cross-entropy = surprise when predicting with q; KL = the gap between them. Classification models minimise cross-entropy (log loss), which punishes confident errors hardest.

Chapter TL;DR

Area	Key idea	Where it shows up
Vectors	A data point is a vector; dot product = multiply and sum	Linear models, embeddings, similarity search
Matrices	A dataset is a matrix; predictions = Xw + b	Every model; neural-net layers; GPUs
Transpose / inverse / det	Swap axes; undo a matrix; det = 0 means no inverse	Normal equation, gradients, covariance
Eigen / SVD	Main directions and their strengths	PCA, compression, recommenders
Derivatives / gradients	Slope of the loss for each parameter	How every model learns
Chain rule	Multiply slopes through nested functions	Backpropagation
Gradient descent	$w \leftarrow w - \eta \nabla L$; tune η, scale features	Training almost everything
Distributions	Bernoulli, binomial, normal; mean and variance	Labels, noise, A/B tests
Bayes	Posterior $\propto$ likelihood $\times$ prior; mind the base rate	Naive Bayes, rare-event precision
Statistics	Median vs mean, standardisation, CLT, $\sigma/\sqrt{n}$	Preprocessing, trusting results
Correlation ≠ causation	Confounders; use experiments for causes	Interpreting models
Tests and CIs	p-value, α, CI, bootstrap; significance ≠ importance	Model comparison, A/B tests
Information theory	Entropy, cross-entropy, KL	Log loss, decision trees, LLM training

A little of each goes a long way: you now have everything needed to read the math in the rest of this book.

Chapter 3 — The Python ML Stack

Almost all practical machine learning today happens in Python, not because Python is fast (it isn't), but because a small set of libraries wrap fast C, C++ and Fortran code behind a friendly interface. This chapter teaches the five tools you will use in every later chapter: **NumPy** for arrays, **pandas** for tables, **matplotlib** for plots, **scikit-learn** for classic ML, and **Jupyter** for interactive work. It closes with a short tour of the deep-learning frameworks (PyTorch, TensorFlow/Keras) and when to reach for them.

It assumes basic Python (variables, lists, dicts, functions, loops, classes). If any of that is new, work through the **Python Handbook** first; this chapter only covers what is specific to data work.

What you'll learn

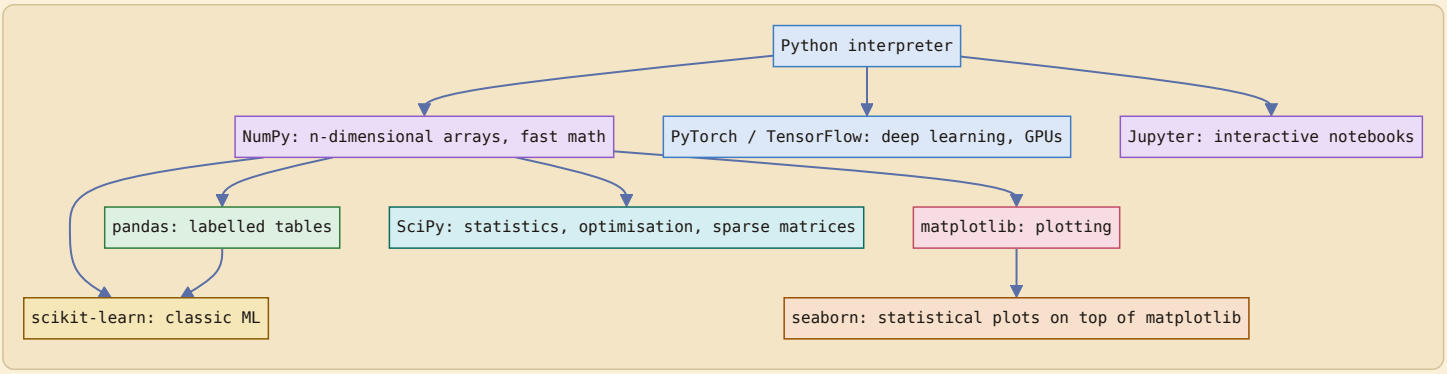
- How to set up Python and the core libraries in a virtual environment
- NumPy: arrays, dtypes, shapes, indexing, views vs copies, axis operations, broadcasting, vectorisation, random numbers
- pandas: Series and DataFrame, loading a CSV, inspecting, cleaning missing values and types, `loc`/`iloc`, filtering, `groupby`, `merge`, apply vs vectorised
- matplotlib: line, scatter, histogram, bar, box plot, heatmap and subplots, saved to files
- scikit-learn: the estimator API (`fit`, `predict`, `transform`), `Pipeline`, `ColumnTransformer`, cross-validation and grid search
- A Jupyter workflow that stays reproducible
- scikit-learn vs PyTorch vs TensorFlow/Keras: which to choose

All code is in `code/`.

3.1 Setting up

What it is

The **stack** is a set of open-source libraries that build on each other:



How it works

1. **Install Python 3** from python.org (or your OS package manager). Check it in a terminal: `python --version` (on macOS and many Linux systems the command is `python3`). This book uses Python 3.12 or newer.
2. **Create a virtual environment** per project, so each project has its own library versions:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

3. **Install the libraries** with pip:

```
pip install numpy pandas matplotlib scikit-learn scipy jupyter
pip install seaborn                # optional, nicer statistical plots
pip install torch                  # only when you need deep learning
```

4. **Record versions** so others can reproduce your work: `pip freeze > requirements.txt`, later `pip install -r requirements.txt`. Alternatives such as conda, uv or Poetry do the same job (the Python Handbook's packaging chapter covers them).
5. **Import with the standard aliases** everyone uses: `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt`.

The code in this chapter was run with NumPy 2.5, pandas 3.0, scikit-learn 1.9 and matplotlib 3.11.

Common mistakes

- Installing into the system Python with `sudo pip`. Use a virtual environment.
- Installing with one Python and running with another ("but I installed it!"). `python -m pip install ...` guarantees pip matches the interpreter.

TL;DR

One virtual environment per project; `pip install numpy pandas matplotlib scikit-learn`; freeze versions; import as `np`, `pd`, `plt`.

3.2 NumPy: fast arrays

What it is

NumPy provides the `ndarray`: an n-dimensional grid of numbers, all of the same type, stored in one contiguous block of memory. A 1-D array is a vector, a 2-D array is a matrix (Chapter 2), and higher dimensions hold things like colour images (height × width × 3) or batches of images.

Why it matters

A Python list stores pointers to separate Python objects, and a Python loop runs one interpreted instruction at a time. A NumPy array stores raw numbers and runs whole operations in compiled code. The result is often 10–1000× faster and far more compact. pandas, scikit-learn, SciPy and matplotlib all speak NumPy arrays, and PyTorch tensors convert to and from them easily.

How it works

Arrays, shapes and dtypes

Attribute / function	Meaning	Example
`np.array([1, 2, 3])`	Create from a list	
`a.dtype`	Element type: `int64`, `float64`, `float32`, `bool`, ...	Mixing ints and floats upcasts to float
`a.shape`	Size of each dimension	`(3, 4)` = 3 rows, 4 columns
`a.ndim`, `a.size`, `a.nbytes`	Dimensions, element count, memory	
`np.zeros`, `np.ones`, `np.eye`, `np.arange`, `np.linspace`	Common constructors	`np.linspace(0, 1, 5)` → 5 evenly spaced values
`a.reshape(3, 4)`, `a.reshape(-1)`, `a.T`	New shape (−1 = "work it out"), flatten, transpose	
`a[:, None]` or `a.reshape(-1, 1)`	Turn shape (n,) into a column (n, 1)	scikit-learn wants 2-D X

dtype matters. `float64` uses 8 bytes per number, `float32` uses 4: 1 million values are 8 MB vs 4 MB. Deep learning usually uses `float32` (or smaller) to save memory and speed up GPUs. Small integer types silently **wrap around** on overflow: `int8` 127 + 1 = −128. And floating point is approximate: `0.1 + 0.2 == 0.3` is `False`; compare floats with `np.isclose`.

Indexing

```
M = np.arange(12).reshape(3, 4)
M[1, 2]          # one element (row 1, column 2) -> 6
M[0]             # first row
M[:, 1]          # second column
M[:2, 1:3]       # rows 0-1, columns 1-2 (slices are start:stop, stop excluded)
M[M % 2 == 0]    # boolean mask: all even values
M[[2, 0]]        # "fancy" indexing with a list: rows 2 and 0
np.where(M > 5, 1, 0)   # vectorised if/else
```

Views vs copies. Basic slicing (`M[0]`, `M[:2, 1:3]`) returns a **view**: a window onto the same memory. Changing the view changes the original. Boolean and fancy indexing return **copies**. When you need independent data, call `.copy()`.

Axis operations

For a 2-D array, `axis=0` runs **down the rows** (one result per column) and `axis=1` runs **across the columns** (one result per row). A way to remember: the axis you name is the one that disappears.

```
scores = np.array([[80, 90, 70],     # rows = students, columns = exams
                   [60, 75, 85]])
scores.sum(axis=0)      # per exam:    [140 165 155]
scores.mean(axis=1)     # per student: [80. 73.33]
scores.argmax(axis=1)   # best exam per student: [1 2]
```

`keepdims=True` keeps the reduced axis with size 1 (shape (2, 1) instead of (2,)), which makes the result broadcast back against the original.

Broadcasting

Broadcasting lets NumPy combine arrays of different shapes without copying data. Rules, applied from the *last* dimension backwards:

1. Two dimensions are compatible if they are equal or one of them is 1.
2. Missing leading dimensions are treated as 1.
3. A size-1 dimension is stretched to match the other.

Shapes	Result	Why
(3, 2) and (2,)	(3, 2)	(2,) acts as (1, 2) and is repeated for each row
(3, 1) and (1, 4)	(3, 4)	Both stretch: an outer "table"
(3, 2) and (3,)	**error**	Last dimensions 2 vs 3 clash; reshape to (3, 1) if you meant "per row"

The most common use in ML: **standardising every column at once**.

```
X = np.array([[1.0, 200.0], [2.0, 300.0], [3.0, 400.0]])   # (3, 2)
(X - X.mean(axis=0)) / X.std(axis=0)                        # (3, 2) - (2,) / (2,)
```

Output:

```
mean [  2. 300.] std [ 0.816 81.65 ]
standardised (X - mu) / sd =
 [[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
```

A more advanced example computes the squared distance from every point to every cluster centre (the core step of k-means) with no loops, by inserting axes so shapes (4, 1, 2) and (1, 2, 2) broadcast to (4, 2, 2):

```
d2 = ((pts[:, None, :] - centres[None, :, :]) ** 2).sum(axis=2)   # (4 points, 2 centres)
d2.argmin(axis=1)                                                 # nearest centre per point
```

```
squared distances (points x centres):
 [[ 0. 50.]
 [ 2. 32.]
 [50.  0.]
 [61.  1.]]
nearest centre: [0 0 1 1]
```

Vectorisation vs loops

Vectorisation means expressing a computation as whole-array operations instead of a Python loop.

```
def loop():
    total = 0.0
    for v in x:            # 1,000,000 Python-level iterations
        total += v * v
    return total

def vec():
    return float(x @ x)    # one call into optimised compiled code
```

Output from `code/numpy_basics.py`:

```
sum of squares of 1M floats: loop 58 ms, vectorised 0.06 ms -> ~930x faster (varies by machine)
same answer: True
```

Rule of thumb: if you are writing a `for` loop over the rows of an array, there is probably a NumPy function or a broadcasting expression that does it faster.

Random numbers

Use the modern **Generator** API with an explicit seed so results are reproducible:

```
rng = np.random.default_rng(42)
rng.random(3)                # uniform [0, 1)
rng.normal(0, 1, 3)          # normal, mean 0, std 1
rng.integers(0, 10, 5)       # integers 0..9
rng.choice(["a", "b", "c"], size=4)
rng.permutation(10)          # shuffled indices, e.g. for a manual train/test split
```

```
uniform [0.774 0.439 0.859]
normal [ 0.941 -1.951 -1.302]
integers [7 7 7 7 5]
choice ['a' 'c' 'b' 'b']
shuffled indices for a train/test split: [7 0 4 2 3 5 8 1 9 6] -> train [7 0 4 2 3 5 8 1] test [9 6]
same seed, same numbers: [0.774 0.439 0.859]
```

The older `np.random.seed()` / `np.random.rand()` functions still work but share one global state; NumPy's documentation recommends `default_rng` for new code.

Common mistakes

- Confusing shape (n,) with (n, 1). scikit-learn expects 2-D `X`, so a single feature must be `x.reshape(-1, 1)`.
- Modifying a slice and being surprised that the original changed (views).
- Silent broadcasting that "works" but computes the wrong thing, e.g. (n,) − (n, 1) produces an (n, n) matrix. Check shapes.
- Using `*` for matrix multiplication; use `@`.

In the interview

30-second answer: "NumPy arrays are homogeneous, contiguous blocks of memory, so operations run in compiled code instead of the Python interpreter. Vectorising, meaning whole-array operations plus broadcasting, is typically orders of magnitude faster than Python loops. Broadcasting aligns shapes from the trailing dimension and stretches size-1 dimensions."

- *View or copy?* — Basic slices are views; boolean and fancy indexing return copies.
- *What does axis=0 mean?* — Operate down the rows, producing one result per column.

Self-check

1. Shape of `np.zeros((5, 3)).sum(axis=0)`? → (3,).
2. Can (4, 3) and (4,) broadcast? → No, trailing dims 3 and 4 differ; use (4, 1).
3. How do you make random results repeatable? → `rng = np.random.default_rng(seed)` and draw everything from `rng`.

TL;DR

ndarrays are typed, fixed-size and fast. Master shape, dtype, slicing (views!), `axis`, broadcasting and vectorisation, and seed a `default_rng`.

3.3 pandas: tables with labels

What it is

pandas adds labels to NumPy arrays. A **Series** is a 1-D array with an **index** (row labels) and a name. A **DataFrame** is a table: a collection of Series (columns) sharing one index, each column with its own dtype. Think of it as a spreadsheet you control with code.

Why it matters

Real data arrives as CSV files, Excel sheets and database queries, with mixed types, missing values and messy text. pandas is the standard tool for loading, cleaning, exploring and reshaping it before modelling. scikit-learn accepts DataFrames directly.

How it works

`code/pandas_basics.py` walks through a small employee table.

Series and DataFrame

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="points")
s["b"]        # 20  (by label)
df = pd.DataFrame({"city": ["Paris", "Lima"], "pop_m": [2.1, 10.0]})
```

Load and inspect

```
df = pd.read_csv("employees.csv", parse_dates=["start_date"])
df.head(3)            # first rows
df.shape              # (rows, columns)
df.dtypes             # type of each column
df.isnull().sum()     # missing values per column
df.describe()         # count, mean, std, min, quartiles, max for numeric columns
df.info()             # dtypes, non-null counts, memory
```

Output:

```
   emp_id  name   dept    salary age start_date
0       1  Asha    Eng  120000.0  34 2019-03-01
1       2   Ben    Eng       NaN  29 2021-07-15
2       3  Chen  Sales   70000.0  41 2015-01-10

shape: (7, 6)
emp_id                int64
name                    str
dept                    str
salary              float64
age                     str
start_date    datetime64[us]

missing values per column:
 emp_id        0
name           0
dept           0
salary         2
age            0
start_date     0
```

Two things to notice. `salary` became `float64` because an integer column cannot hold the missing-value marker **`NaN`** ("not a number"). And `age` was read as text (`str`, pandas 3's default string dtype) because one row contains `"unknown"`, so `isnull()` does not count it as missing. **Always check dtypes after loading.** `read_csv` also reads from URLs and compressed files, and pandas has `read_excel`, `read_parquet`, `read_json` and `read_sql` for other sources.

Clean: types and missing values

```
df["age"] = pd.to_numeric(df["age"], errors="coerce")     # "unknown" -> NaN
df["salary_missing"] = df["salary"].isna()                # remember what was missing
df["salary"] = df["salary"].fillna(df.groupby("dept")["salary"].transform("median"))
df["age"] = df["age"].fillna(df["age"].median())
```

Output:

```
age dtype after to_numeric: float64 | missing ages: 1
dropna() would keep 4 of 7 rows
   name   dept    salary  salary_missing   age
0  Asha    Eng  120000.0           False  34.0
1   Ben    Eng  127500.0            True  29.0
...
6   Gus     HR   60000.0            True  27.0
```

Choices for missing values:

Strategy	Code	When
Drop rows	`df.dropna()`	Few missing rows, missing at random. Here it would throw away 3 of 7 rows.
Drop a column	`df.drop(columns="x")`	Column mostly empty or useless
Fill with a constant	`df["x"].fillna(0)`	0 has a real meaning (e.g. "no purchases")
Fill with mean / median	`df["x"].fillna(df["x"].median())`	Numeric; median is robust to outliers
Fill per group	`groupby(...).transform("median")`	Groups differ (Ben's missing salary gets the Eng median, 127,500)
Add a "was missing" flag	`df["x_missing"] = df["x"].isna()`	Missingness itself may be informative

In a modelling project, do the filling inside a scikit-learn `SimpleImputer` in a pipeline (§3.5), so the fill values are learned from training data only.

Select: `loc` vs `iloc`

	`df.loc[...]`	`df.iloc[...]`
Selects by	**Labels** (index values, column names) and boolean masks	**Integer positions**
Slice end	**Included** (`loc[0:2]` gives 3 rows)	Excluded (`iloc[0:2]` gives 2 rows)
Example	`df.loc[df["dept"] == "Eng", ["name", "salary"]]`	`df.iloc[:2, :3]`

`df["col"]` selects one column (a Series); `df[["a", "b"]]` selects several (a DataFrame).

Filter

```
df[(df["salary"] > 100_000) & (df["age"] < 40)]   # combine with & | ~ and parentheses
df.query("dept == 'Sales' and salary >= 65000")   # same idea, SQL-like string
df[df["dept"].isin(["HR", "Sales"])]
df.sort_values("years", ascending=False)
```

```
salary > 100k AND age < 40: ['Asha', 'Ben']
query(): ['Chen', 'Dana']
isin(): ['Chen', 'Dana', 'Eli', 'Gus']
```

Use `&`, `|`, `~` (not `and`, `or`, `not`), and wrap each condition in parentheses, because `&` binds tighter than `>`.

groupby: split, apply, combine

`groupby` splits rows into groups, applies a function to each, and combines the results, like SQL's `GROUP BY`.

```
df.groupby("dept").agg(headcount=("emp_id", "count"),
                       avg_salary=("salary", "mean"),
                       max_age=("age", "max"))
df["dept"].value_counts()
df.pivot_table(index="dept", values="salary", aggfunc="median")
```

```
       headcount  avg_salary  max_age
dept
Eng            3    127500.0     45.0
HR             2     60000.0     38.0
Sales          2     67500.0     41.0
```

`agg` reduces each group to one row; `transform` returns a result the same length as the original (used above to fill each row with its group's median).

merge / join

`merge` combines two tables on a key column, like SQL joins.

```
depts = pd.DataFrame({"dept": ["Eng", "Sales", "Legal"], "floor": [3, 1, 5]})
df.merge(depts, on="dept", how="inner")   # only matching keys
df.merge(depts, on="dept", how="left")    # all rows of df, NaN where no match
```

```
inner join rows: 5 (HR has no match) | left join rows: 7 (HR kept with floor NaN: True)
```

how=	Keeps
`"inner"`	Keys in both tables
`"left"`	All rows of the left table
`"right"`	All rows of the right table
`"outer"`	All keys from both

Check row counts after every merge: duplicate keys in the right table multiply rows (a "many-to-many" explosion). `validate="many_to_one"` makes pandas raise an error if that happens. `pd.concat([df1, df2])` stacks tables vertically instead.

apply vs vectorised

`df.apply(func, axis=1)` calls a Python function once per row: flexible but slow. Column arithmetic, `np.where`, `pd.cut` and the `.str` / `.dt` accessors are vectorised.

```
big.apply(lambda r: r["price"] * r["qty"], axis=1)   # row by row
big["price"] * big["qty"]                            # whole columns at once
big["band"] = np.where(big["price"] > 50, "high", "low")
big["bucket"] = pd.cut(big["price"], bins=[0, 25, 75, 100], labels=["cheap", "mid", "premium"])
```

```
row-wise apply: 430 ms | vectorised: 0.21 ms -> ~2068x faster (varies by machine)
```

Use `apply` only when no vectorised alternative exists (e.g. calling an external parser on each value).

A pandas 3 pitfall: chained assignment

Since pandas 3.0, **copy-on-write** is always on: every indexing result behaves as a copy. So the once-common pattern of calling an in-place method on a selected column no longer modifies the DataFrame:

```
df["A"].fillna(0, inplace=True)    # does NOT change df (pandas warns: ChainedAssignmentError)
df["A"] = df["A"].fillna(0)        # correct: assign the result back
```

```
after df['A'].fillna(0, inplace=True): [1.0, nan, 3.0]
after df['A'] = df['A'].fillna(0):    [1.0, 0.0, 3.0]
```

Prefer assignment over `inplace=True` in general; it is clearer and works in every pandas version.

Common mistakes

- Not checking dtypes: numbers stored as text, dates as strings.
- `and` / `or` in filters instead of `&` / `|`, or missing parentheses.
- Confusing `loc` (labels, end inclusive) with `iloc` (positions, end exclusive).
- Row-wise `apply` on large data.
- Dropping rows with missing values without checking how many you lose, or whether missingness is related to the target.

In the interview

30-second answer: "pandas gives labelled, column-typed tables on top of NumPy. My first steps on new data are `head`, `dtypes`, `info`, `describe` and `isnull().sum()`, then fixing types with `to_numeric` or `to_datetime`, handling missing values, and exploring with `groupby` and plots. I use `loc` for labels and masks, `iloc` for positions, and vectorised operations rather than `apply`."

- *`agg` vs `transform`?* — `agg` returns one row per group; `transform` returns a same-length result aligned to the original rows.
- *How do you avoid a merge blowing up the row count?* — Check key uniqueness and pass `validate="one_to_one"` or `"many_to_one"`.

Self-check

1. `df.loc[0:2]` vs `df.iloc[0:2]` with a default index: how many rows each? → 3 and 2.
2. A numeric column shows dtype `str`. First step? → `pd.to_numeric(col, errors="coerce")` and inspect the new NaNs.
3. Why fill missing values inside a scikit-learn pipeline rather than in pandas before splitting? → To learn fill values from training data only (no leakage).

TL;DR

Load → check `dtypes`, `isnull().sum()`, `describe()` → fix types → handle missing values deliberately → select with `loc` / `iloc`, filter with masks, summarise with `groupby`, combine with `merge`. Vectorise; assign instead of `inplace`.

3.4 matplotlib: seeing the data

What it is

matplotlib is Python's foundational plotting library. **seaborn** is a higher-level library on top of it that makes statistical plots (distributions, pair plots, heatmaps) in one line; pandas' `.plot()` and `.hist()` also call matplotlib. Learning matplotlib's core lets you customise anything the others produce.

Why it matters

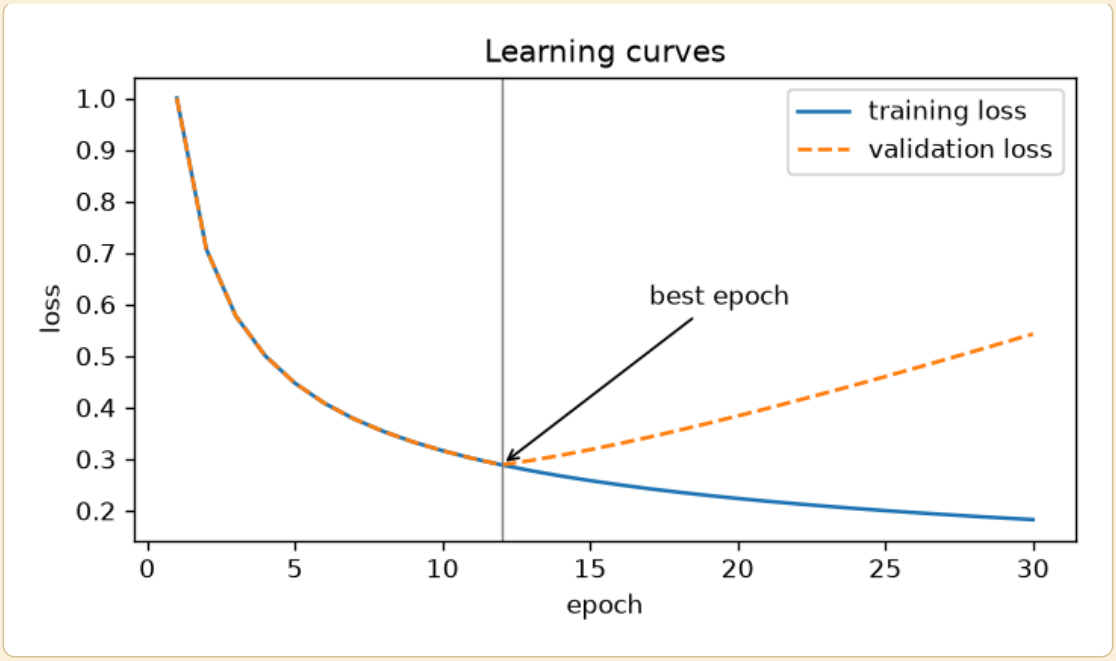
Plots reveal what summary numbers hide: skewed distributions, outliers, class overlap, non-linear relationships, data-entry errors, and overfitting in learning curves. Plot before you model.

How it works

Use the **object-oriented API**: create a figure (`fig`, the whole image) and one or more axes (`ax`, individual plots), then call methods on `ax`.

```
import matplotlib
matplotlib.use("Agg")                       # draw to files (scripts, servers); omit in Jupyter
import matplotlib.pyplot as plt

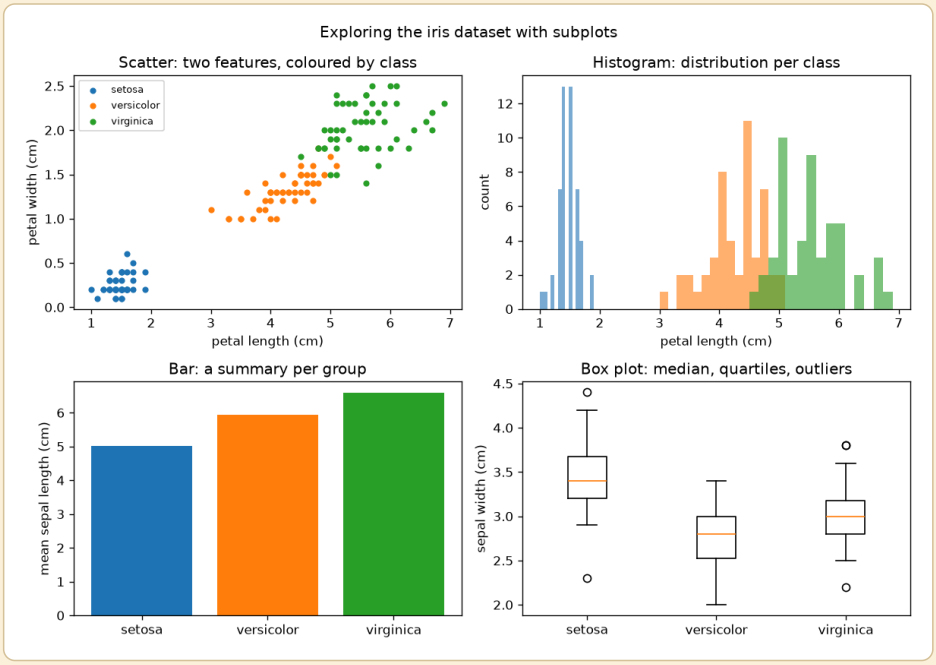
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(epochs, train, label="training loss")
ax.plot(epochs, val, "--", label="validation loss")
ax.set(xlabel="epoch", ylabel="loss", title="Learning curves")
ax.legend()
fig.tight_layout()
fig.savefig("img/line_learning_curves.png", dpi=120)
plt.close(fig)                              # free memory when making many figures
```



A **line plot** suits ordered x-values (time, epochs). This one shows the classic overfitting signature: validation loss turns upward while training loss keeps falling.

Subplots put several plots in one figure: `fig, axes = plt.subplots(2, 2)` returns a 2 × 2 array of axes.

```
ax = axes[0, 0]
for sp, g in df.groupby("species"):
    ax.scatter(g["petal_length"], g["petal_width"], s=15, label=sp)
axes[0, 1].hist(g["petal_length"], bins=15, alpha=0.6)
axes[1, 0].bar(means.index, means.values)
axes[1, 1].boxplot([...], tick_labels=[...])
```

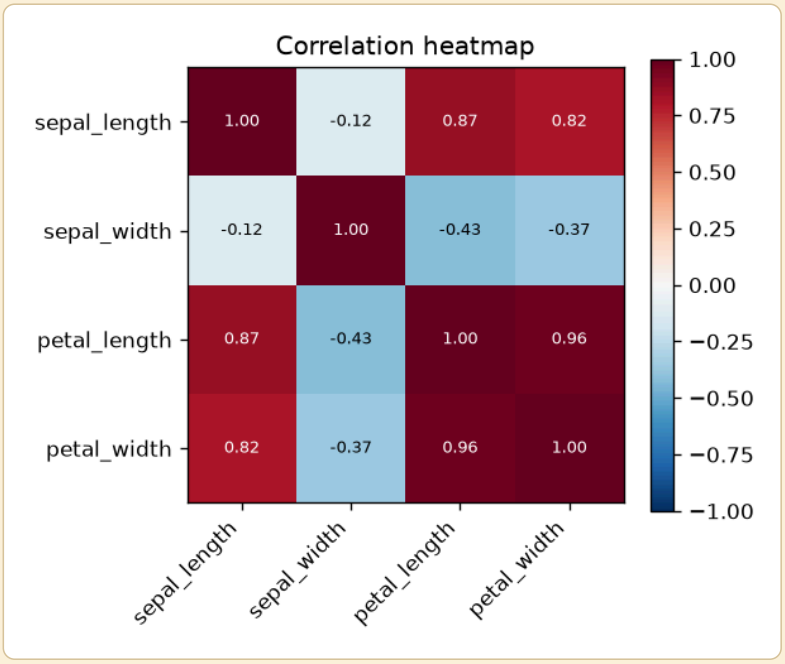


Plot	Shows	ML use
Scatter	Relationship between two numeric variables	Are classes separable? Is a relationship linear?
Histogram	Distribution of one variable	Skew, outliers, per-class overlap
Bar	A value per category	Class balance, group means
Box plot	Median, quartiles (box), whiskers, outliers (dots)	Compare distributions across groups, spot outliers
Line	Trend over an ordered axis	Learning curves, time series
Heatmap	A matrix as colours	Correlation matrices, confusion matrices

From these four panels alone you can see that petal measurements separate *setosa* perfectly and the other two species mostly, which predicts that even a simple classifier will do well on iris.

Correlation heatmap (seaborn's `sns.heatmap(corr, annot=True)` in one line; here in plain matplotlib):

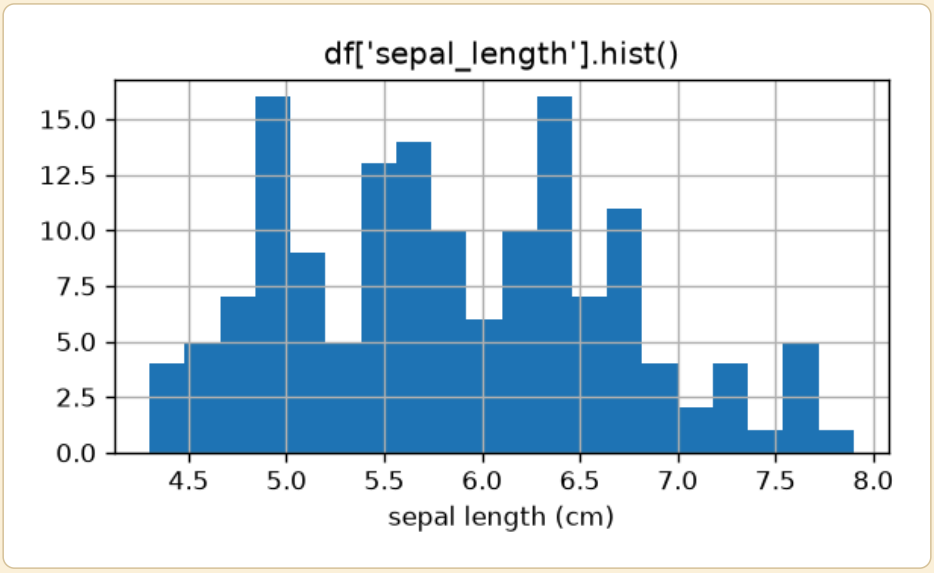
```
corr = df.drop(columns=["species", "target"]).corr()
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
fig.colorbar(im, ax=ax)
```



Petal length and width correlate at 0.96: they carry nearly the same information, which matters for linear models (Chapter 2's multicollinearity) and feature selection.

pandas shortcut, handy for a first look:

```
ax = df["sepal_length"].hist(bins=20)
```



Common mistakes

- Mixing the state-based `plt.plot(...)` style with multiple subplots and losing track of which axes is active. Use `fig, ax = plt.subplots()`.
- No axis labels or units.
- Default 10 bins hiding structure; try several `bins` values.
- Forgetting `plt.close(fig)` in loops, which leaks memory.
- Calling `plt.show()` in a script on a server with no display; save with `savefig` instead.

Self-check

1. Which plot to check class balance? → A bar chart of `value_counts()`.
2. Which plot to spot outliers per group? → A box plot.
3. What does `fig, axes = plt.subplots(1, 3)` return? → A figure and an array of 3 axes.

TL;DR

`fig, ax = plt.subplots()`, call `ax.plot/scatter/hist/bar/boxplot/imshow`, label everything, `savefig`. Plot distributions, relationships and class balance before modelling.

3.5 scikit-learn: the estimator API

What it is

scikit-learn is the standard Python library for classic ML: linear models, trees and forests, SVMs, k-NN, clustering, dimensionality reduction, preprocessing, model selection and metrics. Its power comes from **one consistent interface** that every algorithm follows, described in its API design paper (Buitinck et al., 2013).

How it works

Object type	Key methods	Learns in `fit`	Examples
Estimator (all objects)	`fit(X, y)`; hyperparameters set in the constructor; `get_params()`, `set_params()`		Everything below
Transformer	`transform(X)`, `fit_transform(X)`	Statistics needed to transform (mean, categories, ...)	`StandardScaler`, `OneHotEncoder`, `SimpleImputer`, `PCA`
Predictor	`predict(X)`, often `predict_proba(X)`, `score(X, y)`	Model parameters	`LogisticRegression`, `RandomForestClassifier`, `KMeans`

Conventions:

1. **Hyperparameters** go in the constructor: `LogisticRegression(C=0.1)`.
2. **`fit` learns from data and returns `self`**, so you can chain: `LogisticRegression().fit(X, y).predict(X_new)`.
3. **Learned attributes end with an underscore**: `scaler.mean_`, `model.coef_`, `model.classes_`. They exist only after fitting.
4. **`X` is 2-D** (n_samples × n_features: an array or DataFrame); **`y` is 1-D**.
5. `score` returns a default metric: accuracy for classifiers, R^2 for regressors.

```
scaler = StandardScaler()
scaler.fit(X_train)                  # learn mean_ and scale_ from TRAINING data
scaler.transform(X_train)
scaler.transform(X_new)              # uses the training mean and std

clf = LogisticRegression().fit(X, y)
clf.predict([[1.0], [4.5]]); clf.predict_proba([[1.0], [4.5]]); clf.score(X, y)
```

Output from `code/sklearn_api.py`:

```
mean_ [  2. 200.] scale_ [ 0.816 81.65 ]
transform(new): [[2.449 2.449]] (uses the training mean/std)
hyperparameters via get_params(): {'copy': True, 'with_mean': True, 'with_std': True}
predict([[1.0],[4.5]]) = [0 1]
predict_proba = [[0.843 0.157]
 [0.096 0.904]]
score (accuracy) = 1.0 | coef_ [[1.121]] intercept_ [-2.802]
```

`fit_transform` on training data, `transform` only on validation/test/production data. Calling `fit` (or `fit_transform`) on test data re-learns the statistics from it, which is leakage.

Pipeline

A **`Pipeline`** chains transformers and a final estimator into one estimator. Calling `fit` fits each step in order on the training data and passes the transformed data along; `predict` runs the same transformations and then predicts.

Why use one:

- **No leakage:** in cross-validation, every step is refit on each training fold.
- **One object to tune** (`GridSearchCV` can search any step's hyperparameters) and **one object to save and deploy**, so training and serving apply identical preprocessing.

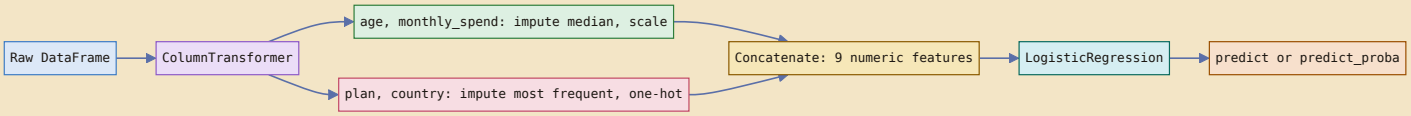
Parameters inside a pipeline are addressed as `step__parameter` (double underscore), nesting as deep as needed: `prep__num__impute__strategy`.

ColumnTransformer

Real tables mix numeric and categorical columns that need different preprocessing. **`ColumnTransformer`** applies a different transformer to each group of columns and concatenates the results.

```
numeric = ["age", "monthly_spend"]
categorical = ["plan", "country"]
preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), categorical),
])
model = Pipeline([("prep", preprocess), ("clf", LogisticRegression(max_iter=1000))])
model.fit(X_train, y_train)

grid = GridSearchCV(model, {
    "prep__num__impute__strategy": ["mean", "median"],
    "clf": [LogisticRegression(max_iter=1000), RandomForestClassifier(n_estimators=100, random_state=0)],
}, cv=5)
grid.fit(X_train, y_train)
```



Output on a synthetic 600-customer churn dataset with missing values:

```
    age  monthly_spend   plan country
0  62.0          75.10  basic      DE
1   NaN          62.60  basic      IN
2  44.0          89.69    pro      DE
missing: {'age': 43, 'monthly_spend': 0, 'plan': 32, 'country': 0} | churn rate: 0.372
output features: ['num__age', 'num__monthly_spend', 'cat__plan_basic', 'cat__plan_family', 'cat__plan_pro', 'cat__country_BR',
'cat__country_DE', 'cat__country_IN', 'cat__country_US']
test accuracy: 0.827 | baseline (always 'no churn'): 0.627
CV accuracy (5 folds): 0.773
P(churn) for a customer with unseen categories: 0.933
best: {'clf': 'LogisticRegression', 'prep__num__impute__strategy': 'mean'} | CV 0.773 | test 0.827
```

What each part shows:

- `get_feature_names_out()` lists the 9 columns the model actually sees: 2 scaled numerics plus one 0/1 column per category (**one-hot encoding**).
- `handle_unknown="ignore"` lets the model score a customer with a plan (`"enterprise"`) and country (`"JP"`) never seen in training, by setting all of that feature's one-hot columns to 0, instead of crashing in production.
- `GridSearchCV` treated the whole pipeline as one estimator and even swapped the final model; logistic regression won, which makes sense because the data was generated from a logistic formula.
- The test score (0.827) is higher than the CV mean (0.773); with only 150 test rows, a difference of this size is within normal sampling noise (Chapter 2's bootstrap CI).

Other scikit-learn tools you will use constantly

Module	Tools
`sklearn.model_selection`	`train_test_split`, `cross_val_score`, `cross_validate`, `KFold`, `StratifiedKFold`, `GridSearchCV`, `RandomizedSearchCV`
`sklearn.preprocessing`	`StandardScaler`, `MinMaxScaler`, `OneHotEncoder`, `OrdinalEncoder`, `PolynomialFeatures`, `FunctionTransformer`
`sklearn.impute`	`SimpleImputer`, `KNNImputer`
`sklearn.metrics`	`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `mean_squared_error`, `confusion_matrix`, `classification_report`
`sklearn.datasets`	Bundled toy datasets (`load_iris`, `load_breast_cancer`, `load_digits`, `load_diabetes`) and generators (`make_classification`, `make_blobs`)
`sklearn.pipeline`, `sklearn.compose`	`Pipeline`, `make_pipeline`, `ColumnTransformer`, `make_column_transformer`

`set_output(transform="pandas")` on a transformer or pipeline makes it return DataFrames with named columns, which helps debugging. Save fitted pipelines with `joblib.dump` (Chapter 1) and only load files you trust.

Common mistakes

- Calling `fit` or `fit_transform` on the test set.
- Preprocessing with pandas on the full dataset, then cross-validating (leakage). Put it in the pipeline.
- Passing a 1-D array as `X`; reshape to (n, 1).
- Forgetting `handle_unknown="ignore"` and crashing on a new category in production.
- Using `LabelEncoder` on input features; it is meant for targets. Use `OrdinalEncoder` or `OneHotEncoder` for features.

In the interview

> **30-second answer:** "Every scikit-learn object is an estimator with `fit`; transformers add `transform`, predictors add `predict`. Hyperparameters go in the constructor and learned attributes end in an underscore. I wrap preprocessing and the model in a Pipeline, with a ColumnTransformer for mixed column types, so cross-validation refits everything per fold without leakage, and I can tune and deploy one object."

- *`fit_transform` vs `transform`?* — `fit_transform` learns statistics and applies them (training data only); `transform` applies already-learned statistics.
- *How do you tune a hyperparameter inside a pipeline?* — `GridSearchCV` with keys like `step__param`.

Self-check

1. What does the trailing underscore in `coef_` tell you? → It is learned from data during `fit`.
2. Why does `OneHotEncoder` need `handle_unknown="ignore"` in production? → New categories otherwise raise an error.
3. Name the key for the imputer's strategy in the pipeline above. → `prep__num__impute__strategy`.

TL;DR

Constructor = hyperparameters; `fit` learns (attributes_); `transform`/`predict` apply. Pipeline + ColumnTransformer = leak-free, tunable, deployable preprocessing plus model.

3.6 Jupyter workflow

What it is

Jupyter notebooks (`.ipynb`) mix code cells, their outputs (tables, plots) and Markdown notes in one document, running against a live Python process called the **kernel**. You can use them in JupyterLab (`jupyter lab`), VS Code, or hosted services such as Google Colab.

Why it matters

Data work is exploratory: load, look, tweak, look again. Notebooks keep the data in memory between steps and show results inline, which makes exploration and explanation fast.

How it works: a reproducible routine

1. **Start in the project's virtual environment** so the kernel uses its libraries: `source .venv/bin/activate && jupyter lab`. If you install from inside a notebook, use `%pip install package` (installs into the kernel's environment) rather than `!pip`.
2. **First cell: imports, settings and seeds** (`import numpy as np`, `rng = np.random.default_rng(42)`, display options).
3. **Explore top to bottom**: load → inspect → clean → plot → model, with a Markdown heading per stage.
4. **Useful magics**: `%timeit expr` (benchmark), `%time` (one run), `%load_ext autoreload` + `%autoreload 2` (reload your own modules when edited), `df.head()` as the last line to display a table. Plots appear inline by default in current Jupyter.
5. **Restart & Run All before trusting or sharing results.** Cells can be run in any order, so hidden state (a variable defined in a deleted cell) is the most common source of "works on my notebook" bugs.
6. **Move stable code into `.py` modules** and import it; keep notebooks for exploration and reports. `jupyter nbconvert --to script notebook.ipynb` exports the code; `jupyter nbconvert --to html --execute` produces a fresh, fully executed report.
7. **Version control:** clear large outputs before committing, or pair notebooks with scripts (e.g. the jupytext tool).

Trade-offs

	Notebook	`.py` script / module
Exploration, plots, teaching	Excellent	Clumsy
Reproducibility	Risky (out-of-order execution)	Good
Testing, code review, diffs	Poor	Good
Production pipelines	Avoid	Yes

Common mistakes

- Running cells out of order and trusting the result.
- Huge notebooks that do everything; split into modules.
- Committing notebooks with megabytes of output or secrets printed in cells.

TL;DR

Notebooks for exploration, scripts for anything that must run again. Seed first, run top to bottom, Restart & Run All before sharing.

3.7 Deep-learning frameworks: when scikit-learn is not enough

What it is

Framework	Origin	Focus	Style
scikit-learn	Community project, started 2007	Classic ML on tabular data: regression, classification, clustering, preprocessing, evaluation	High level: `fit` / `predict`; CPU
PyTorch	Meta (Facebook) AI Research, 2016; now governed by the PyTorch Foundation	Deep learning; dominant in research and widely used in industry	Define-by-run ("dynamic graph"): the model is ordinary Python code executed each step; automatic differentiation; GPUs
TensorFlow + Keras	Google, 2015	Deep learning with strong deployment tooling (TF Serving, TF Lite / LiteRT for mobile)	Keras is a high-level API (`model.compile`, `model.fit`); since Keras 3 it can run on TensorFlow, JAX or PyTorch backends

How the styles differ

`code/frameworks_compared.py` trains the same logistic-regression classifier on the breast-cancer data in both scikit-learn and PyTorch:

```
# scikit-learn: the library owns the training loop
sk = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)

# PyTorch: you define the model, loss and loop
model = torch.nn.Linear(X_tr.shape[1], 1)               # w.x + b
loss_fn = torch.nn.BCEWithLogitsLoss()                  # sigmoid + cross-entropy
opt = torch.optim.SGD(model.parameters(), lr=0.1, weight_decay=1e-3)
for epoch in range(300):
    opt.zero_grad()
    loss = loss_fn(model(Xt), yt)
    loss.backward()        # autograd: the chain rule from Chapter 2, automatically
    opt.step()             # gradient-descent update from Chapter 2
```

Output:

```
scikit-learn accuracy: 0.982
  epoch   0 loss 0.6904
  epoch 299 loss 0.0738
PyTorch accuracy:      0.974
device available: mps
```

Both reach about the same accuracy (they optimise slightly different objectives and stop at different points). The PyTorch version is longer, but every piece is replaceable: swap `nn.Linear` for a 50-layer network, the loss for any differentiable function, and move tensors to a GPU (`"cuda"` on NVIDIA, `"mps"` on Apple silicon, as detected here) without changing the loop.

Which one to choose

- **Start with scikit-learn** for tabular data, small-to-medium datasets, baselines, and anything where a linear model or tree ensemble might suffice. (Gradient-boosting libraries such as XGBoost, LightGBM and CatBoost follow the scikit-learn API and are often the strongest choice for tables.)
- **Use PyTorch or TensorFlow/Keras** for images, audio, text and other unstructured data, for very large datasets, for GPU training, and for custom neural architectures. PyTorch is the more common choice in current research and many companies; TensorFlow/Keras remains widespread in production and on mobile.
- They combine well: scikit-learn for splitting, preprocessing and metrics around a PyTorch model is a very common setup.

Real-world examples

- scikit-learn is used for classic ML in many companies; its website's testimonials page includes quotes from organisations such as Spotify, J.P. Morgan and Booking.com.
- PyTorch underlies most open research code and projects such as Hugging Face Transformers' PyTorch models, and Tesla's then AI director Andrej Karpathy described training its Autopilot vision networks with it in a 2019 PyTorch Developer Conference talk.
- TensorFlow powers many Google products, and TensorFlow Lite (renamed LiteRT in 2024) runs models on mobile and embedded devices.

Self-check

1. Which library would you pick for a 20,000-row customer table? → scikit-learn (or a gradient-boosting library).
2. What does `loss.backward()` do? → Computes gradients of the loss with respect to every parameter via automatic differentiation.
3. What is Keras? → A high-level deep-learning API; in Keras 3 it runs on TensorFlow, JAX or PyTorch.

TL;DR

scikit-learn for classic ML on tables; PyTorch or TensorFlow/Keras for neural networks, unstructured data and GPUs. They complement each other.

Chapter TL;DR

- **Setup:** one virtual environment per project; `pip install numpy pandas matplotlib scikit-learn`; `import numpy as np, pandas as pd, matplotlib.pyplot as plt`.
- **NumPy:** typed, contiguous arrays. Know shape and dtype, slicing returns views, `axis=0` = down rows, broadcasting aligns trailing dimensions, vectorise instead of looping, seed `default_rng`.
- **pandas:** `read_csv` → `head`, `dtypes`, `isnull().sum()`, `describe` → `to_numeric(errors="coerce")`, handle missing values → `loc` (labels) / `iloc` (positions) → masks with `&` / `|` → `groupby().agg()` → `merge(how=...)`. Vectorise; assign rather than `inplace` (copy-on-write in pandas 3).
- **matplotlib:** `fig, ax = plt.subplots()`; line, scatter, histogram, bar, box, heatmap; label axes; `savefig`. Plot before modelling.
- **scikit-learn:** constructor = hyperparameters, `fit` learns attributes_, `transform`/`predict` apply. `Pipeline` + `ColumnTransformer` = leak-free, tunable, deployable.
- **Jupyter:** great for exploration; seed first, Restart & Run All, move stable code to modules.
- **Frameworks:** scikit-learn for classic ML; PyTorch or TensorFlow/Keras for deep learning.

Chapter 4 — Data, EDA and Feature Engineering

Models are only as good as the rows you feed them. In practice most of an ML project's time goes into the work in this chapter: finding labels, looking at the data, cleaning it, turning raw columns into numbers a model can use, splitting it so your scores are honest, and wrapping all of that into one reproducible pipeline. A mediocre model on well-prepared data usually beats a clever model on messy data.

What you'll learn

1. Where labels come from (outcomes, implicit feedback, humans, weak supervision, LLMs) and how to measure their quality.
2. Exploratory data analysis (EDA) with plots.
3. Cleaning: duplicates, impossible values, outliers, missing values.
4. Encoding categories: one-hot, ordinal, target encoding (and its leakage trap), hashing.
5. Scaling — and which models need it.
6. Creating features: interactions, binning, logs, dates, TF-IDF, feature groups.
7. Splitting: stratified, time-based and group splits.
8. Data leakage, with three runnable examples.
9. Class imbalance: weights, resampling, SMOTE, thresholds, recalibration.
10. An end-to-end scikit-learn `Pipeline` + `ColumnTransformer`.

All code is in `code/` and runs with the shared environment (Python 3.14, scikit-learn 1.9, pandas 3.0) with fixed seeds; outputs are real (long ones trimmed with "..."). Most scripts use a synthetic "subscription churn" table from `code/make_data.py`: 3,000 customers plus 25 duplicate rows, a skewed `income`, a 60-level `city`, dates, short reviews, missing values and typos (ages of 999, incomes of 9,999,999). **Churn** means a customer cancelling; about 12% churn.

4.1 Where training data comes from

What it is

Supervised learning needs **labels**: the correct answer (the **target**, y) attached to each example's **features** (the inputs, X). A **label pipeline** is the process that produces those (features, label) pairs. Labels do not appear by magic; someone or something has to generate them, and each source has its own cost, delay and bias.

Analogy: a student learning from past exam papers. The papers are useless without the answer key — and if the answer key is late, expensive, or written by a sloppy marker, the student learns the wrong lessons.

Why it matters

The first design question for any predictive system is "where does y come from, and when does it arrive?" Label quality caps model quality: a model can't be more accurate than the labels it's graded against.

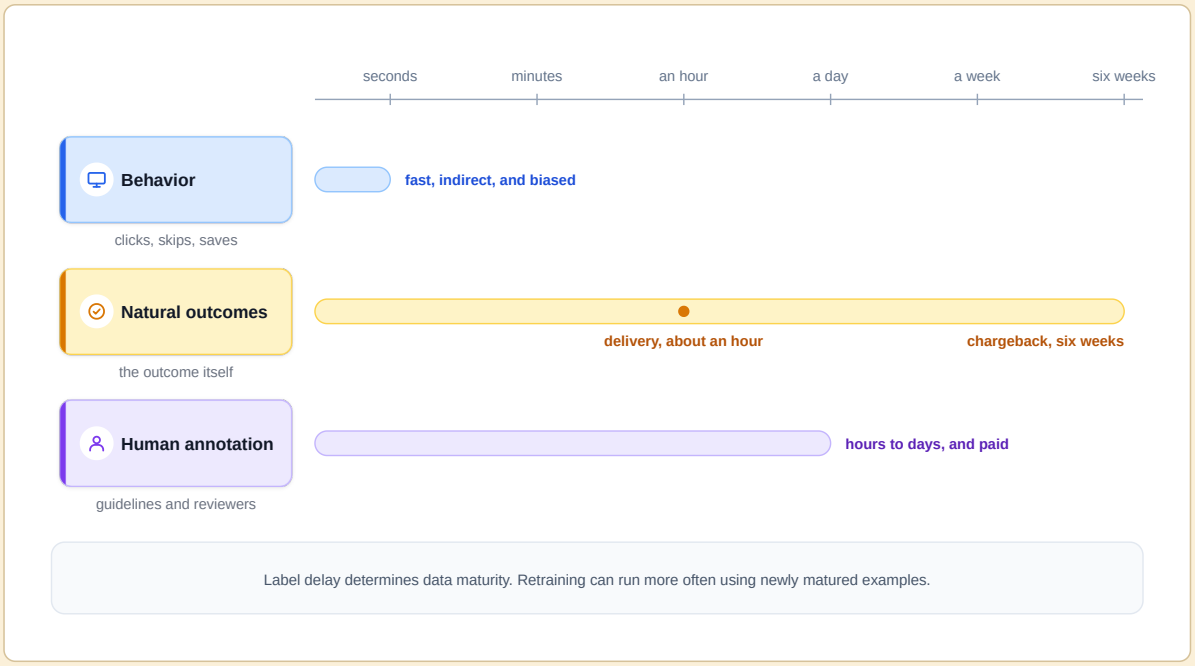


Figure — label sources differ mainly in delay and cost (course diagram)

How it works — the label sources

Source	Example	Delay	Main weakness
Natural (outcome) labels — the world records the answer	Delivery took 34 min; loan defaulted; a card payment was reversed (a **chargeback**)	Minutes to weeks	Late; outcome ≠ truth (a chargeback is evidence of fraud, not proof)
Behavioural labels / implicit feedback — user actions stand in for judgments	Clicks, skips, saves, purchases, watch time	Seconds	Noisy (a click is not satisfaction) and biased toward what the system showed
Human annotation — trained reviewers follow a written guideline	Is this post harmful? Is this document relevant?	Hours to days	Costly (cents to dollars per judgment); reviewer quality needs managing
Weak supervision — many cheap, noisy rules vote	"Contains 'wire transfer' and sender is new → spam"	Instant once written	Rules are noisy and correlated; a model must combine them
LLM-assisted labeling — an LLM applies the guideline	Tag support tickets by topic	Seconds	Systematic errors (the same blind spot repeated everywhere)

Explicit vs implicit feedback. *Explicit* feedback is the user telling you (a star rating); *implicit* is inferred from behaviour (they watched 90% of the video). Implicit feedback is plentiful but biased: a non-click may mean "never shown" (**exposure bias**), and top positions get clicked just for being on top (**position bias**). Chapter 13, section 13.2 covers modelling it.

Label delay. Outcome labels arrive late — chargebacks can take ~6 weeks to settle — so you train only on examples old enough to have a "mature" label and use faster **proxy labels** (rule hits, analyst decisions) for recent data. How delayed labels are joined to features and monitored in a live system is covered in Chapter 16, section 16.3.

Weak supervision in one paragraph

Instead of hand-labeling 100,000 items, experts write dozens of **labeling functions** — small heuristics that output a label or "abstain". A **label model** estimates each function's accuracy from where they agree and disagree, and combines their votes into a probabilistic label per item; a normal model trained on those labels usually generalises beyond the rules. This is the idea behind Stanford's **Snorkel** project.

Human labeling, and measuring agreement

Treat annotation as a system: **versioned guidelines** (a label means what guideline v3 says, stored with the data), **gold questions** (items with known answers mixed into each queue to track reviewer accuracy), **active learning** (spend labels on items the model is least sure about) and reuse of decisions the business already makes (fraud analysts' verdicts). Chapter 16 (16.3) covers running this in production; the tool to master here is the agreement statistic.

Inter-annotator agreement (IAA) — have several reviewers label the same items and measure how often they agree. Raw agreement flatters: two reviewers who guess at random still agree sometimes. **Cohen's kappa** corrects for that:

$$\kappa = (p_o - p_e) / (1 - p_e)$$

where p_o = observed agreement (share of items with the same label) and p_e = chance agreement = Σ over labels of (A's share of that label × B's share). $\kappa = 1$ is perfect, 0 is chance level, negative is worse than chance.

Worked example: two reviewers label 100 emails and agree on 80 ($p_o = 0.8$). Each says "spam" half the time, so $p_e = 0.5 \cdot 0.5$ (both say spam) + $0.5 \cdot 0.5$ (both say not spam) = 0.5. $\kappa = (0.8 - 0.5)/(1 - 0.5) =$ **0.6** — "moderate" on the common Landis–Koch scale (0.41–0.60 moderate, 0.61–0.80 substantial). Low agreement usually means the guideline is unclear or the items are genuinely ambiguous; high agreement does not prove the labels are *correct* (reviewers can share a mistake).

LLMs in the labeling loop

An LLM can label at scale from the written guideline, but inside a hierarchy: humans review a random sample plus low-confidence and disputed cases, and accuracy is measured against human-labeled slices **per important subgroup** (Chapter 16, section 16.3).

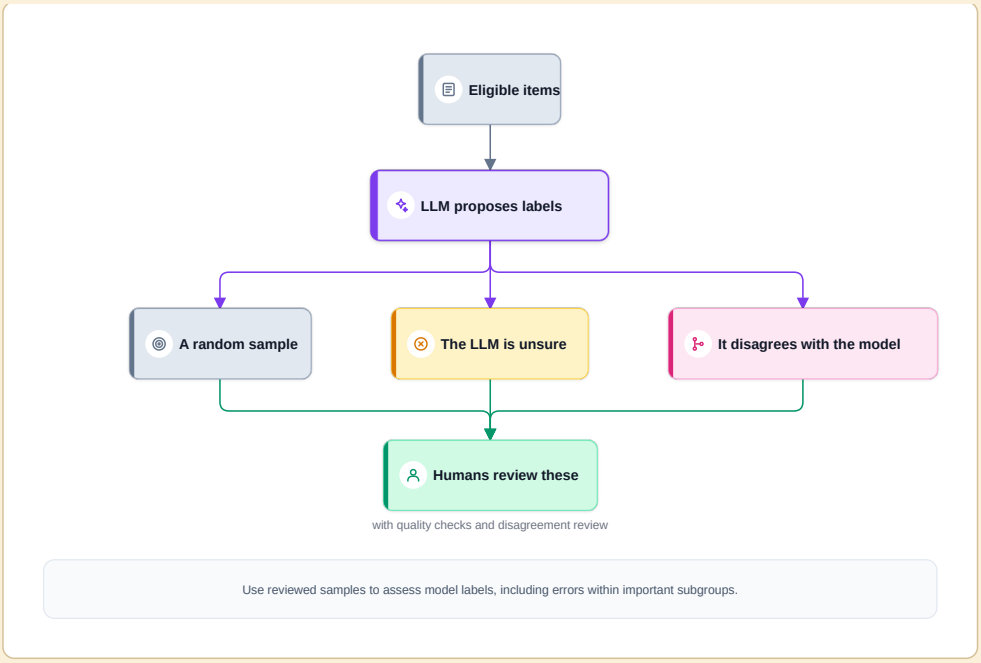


Figure — LLM-assisted labeling with human audit (course diagram)

Negatives for retrieval, and privacy

- **Manufactured negatives.** Logs record clicks, never the millions of items a user would *not* have clicked, so retrieval models create **negative** examples themselves — random, **in-batch** (other users' positives in the same batch) and **hard** (shown but not clicked; some are **false negatives** the user would have liked). See Chapter 13, section 13.7.
- **Privacy.** Training data is user data: collect only what the task needs, restrict access, set retention; a hashed ID is still linkable across tables. More in Chapter 16, section 16.3.

Common mistakes

- Assuming labels "exist" in the warehouse without asking who creates them, when, and how reliably.
- Treating clicks as ground-truth relevance (they're biased toward what was shown and where).
- Changing the labeling guideline without versioning it.
- Reporting raw percent agreement instead of a chance-corrected κ.
- Letting an LLM label everything with no human-audited slices.

Real-world examples

- **YouTube**'s 2016 recommendation paper (Covington et al.) trains its ranking model on implicit feedback — impressions that were clicked or not — and weights each clicked impression by its watch time, so a long watch counts more than a quick click.
- Content-moderation teams at large platforms run the "guidelines + reviewers + gold questions + agreement" loop above.
- Card networks and payment companies use chargebacks (slow) and analyst decisions (fast) as fraud labels.

In the interview

30-second answer — "How will you get labels for this model?": *"I'd list the candidate sources — observed outcomes, user behaviour, human review, rules, LLMs — with their cost, delay and bias. I'd check label quality (agreement with κ, gold questions), train only on matured labels, and join them to features as they were at prediction time."*

- *"Two annotators agree 90% of the time — is that good?"* — Not necessarily. If each annotator says "not spam" 90% of the time, chance agreement is already $p_e = 0.9^2 + 0.1^2 = 0.82$, so $\kappa = (0.90 - 0.82)/(1 - 0.82) \approx 0.44$ — only moderate. Always report a chance-corrected κ.
- *"When is weak supervision worth it?"* — When experts can state rules faster than they can label thousands of items, and labels are needed quickly or at scale; audit the result on a small hand-labeled set.
- *"Why is a click a weak label?"* — It measures the old system's exposure and position as much as the user's preference; a non-click is not a dislike.

Self-check

1. Why is a click a biased label? → Users can only click what was shown, and position affects clicks — so it partly measures the old system.
2. κ when $p_o = 0.9$ and $p_e = 0.5$? → $(0.9 - 0.5)/0.5 = 0.8$.
3. What is a labeling function? → A cheap heuristic that outputs a label or abstains; many are combined by a label model in weak supervision.

TL;DR

Labels come from outcomes (late), behaviour (fast but biased), people (costly), rules (noisy) or LLMs (scalable but correlated errors). Measure label quality with chance-corrected agreement (κ), version the guideline, and audit machine labels against human ones.

4.2 Exploratory data analysis (EDA)

What it is

EDA is looking at your data — with summary tables and plots — before modeling, to learn its shape, spot problems and form hypotheses. Think of it as a doctor's first examination before any treatment.

Why it matters

EDA finds problems that silently wreck models: impossible values, sentinel codes (like 9,999,999 meaning "unknown"), duplicates, skewed columns, leaky columns, and imbalance. It also tells you which transformations and models make sense.

How it works — a checklist

1. **Shape and types** — numeric / categorical / dates / text columns.
2. **Missingness** — share missing per column; is it related to the target?
3. **Distributions** — histograms (skew, outliers, spikes at round numbers); category counts (**cardinality** = number of distinct values; rare levels).
4. **Target** — class balance, or the distribution for regression.
5. **Relationships** — each feature vs the target (rate per bucket, box plots) and vs each other (correlation matrix).
6. **Sanity checks** — duplicates, impossible ranges, IDs that predict the target (possible leakage).

Correlation, briefly. **Pearson** correlation measures *linear* association (−1 to 1). **Spearman** correlation is Pearson on the *ranks*, so it captures any **monotonic** (always-increasing or always-decreasing) relationship and is robust to outliers and skew — a good EDA default. Neither captures a U shape (correlation ≈ 0), and correlation is not causation.

```python
# code/eda.py (key parts)
df = make_churn()
print(df.isna().mean().loc[lambda s: s > 0].round(3))          # missing share
print(df[num_cols].describe().T[["mean", "50%", "std", "min", "max"]])
print(df[["plan", "device", "city"]].nunique())                   # cardinality
print(df.duplicated().sum())                                      # exact duplicates
corr = num.corr(method="spearman")                                # rank correlation
```

Output (trimmed):

```
--- missing values (share) ---
age       0.03
income    0.08
device    0.02

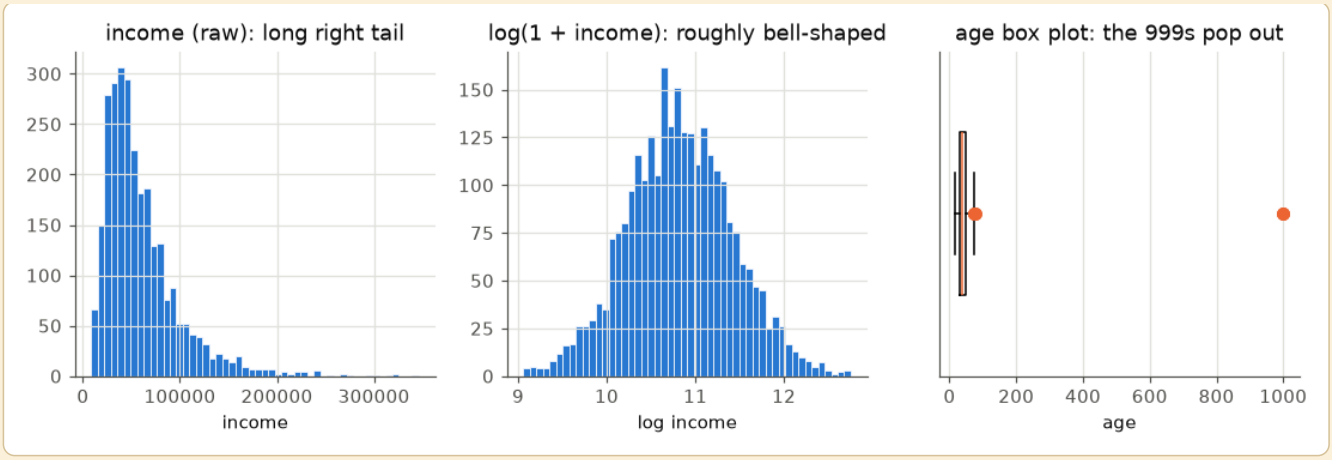
--- numeric summary ---
                    mean      50%       std     min        max
age                 41.1     39.0      37.2    18.0      999.0
income           70763.8  49100.0  328637.8  8600.0  9999999.0
tenure_months       23.3     23.0      13.5     1.0       47.0
support_tickets      1.2      1.0       1.1     0.0        6.0
usage_hours         12.0     10.1       8.4     0.1       58.9

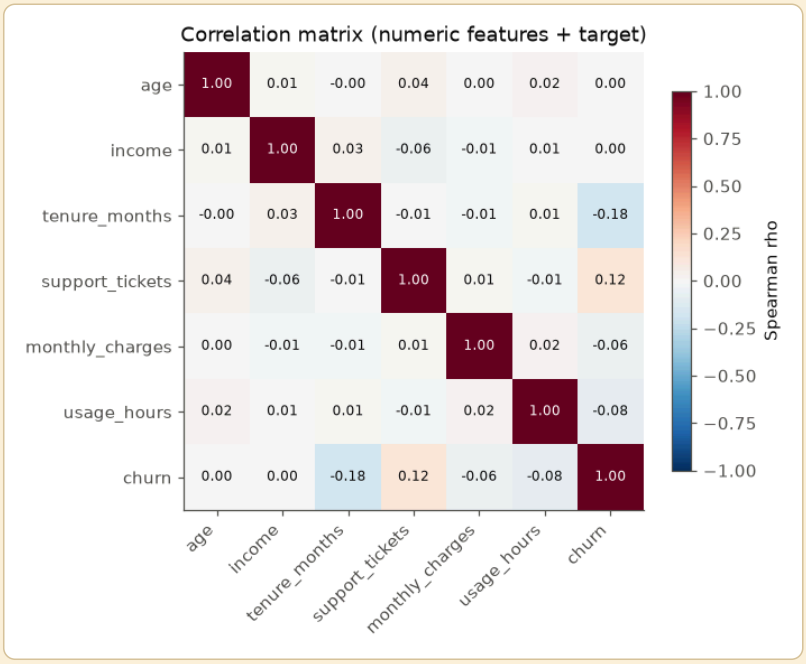
--- categorical cardinality ---
plan 3 | device 3 | city 60

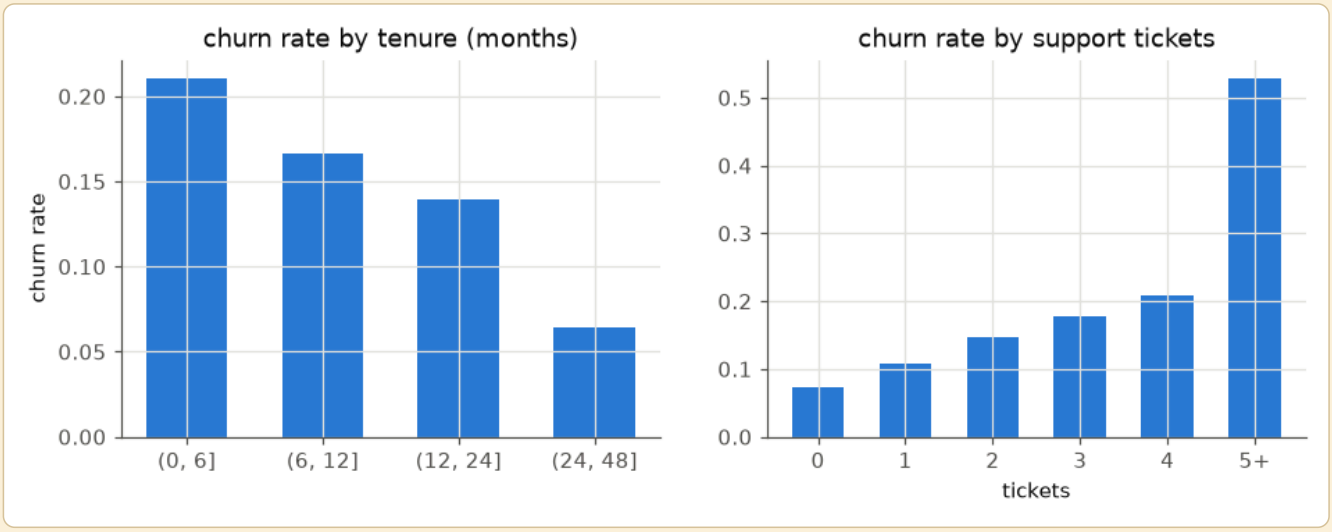
churn rate: 0.117
exact duplicate rows: 25

--- Spearman correlation with churn ---
tenure_months     -0.18
usage_hours       -0.08
...
support_tickets    0.12
```

How to read it: `age` has max 999 and a std of 37 years — typos. `income` has mean ≫ median (70.8k vs 49.1k): a long right tail plus a 9,999,999 sentinel. Churn is 11.7% (mildly imbalanced). Tenure (−) and support tickets (+) relate to churn, as we'd expect.







The bar charts show *shape*, which a single correlation number can't: churn drops steeply in the first year, then flattens.

Common mistakes

- Doing EDA on the full dataset including the test set and then designing features around test-set quirks — do detailed EDA on the training split.
- Trusting `mean` on skewed columns — always compare with the median.
- Reading a near-zero correlation as "no relationship" (it only rules out a *monotonic* one).

Real-world examples

`pandas` (`describe`, `value_counts`, `corr`), `seaborn` pair plots and `ydata-profiling` automate this checklist.

Self-check

1. Mean income ≫ median income tells you what? → Right skew / large outliers.
2. When prefer Spearman over Pearson? → Skewed data, outliers, or monotonic-but-nonlinear relationships.
3. A customer-ID column strongly predicts the target. What do you suspect? → Leakage (IDs assigned in a way that encodes the outcome or time).

TL;DR

Look before you model: types, missingness, distributions, target balance, feature–target and feature–feature relationships, duplicates and impossible values. Plots reveal shapes that single numbers hide.

4.3 Data cleaning

What it is

Fixing or removing data problems so the model learns from reality, not from errors: **duplicates**, **impossible values**, **outliers**, and **missing values**.

Why it matters

Duplicates leak between train and test and overweight some examples; typos distort means and scales; many estimators (e.g. `LinearRegression`, `SVC`) reject NaNs outright.

How it works

1. Duplicates. Remove exact duplicate rows (`df.drop_duplicates()`), then check the entity key (`customer_id`) is unique. Near-duplicates (same text with different whitespace, a resized image) need fuzzy matching.

2. Impossible values are errors, not outliers. An age of 999 or an income of 9,999,999 is a data-entry error or a sentinel code. Convert them to missing (NaN) and handle them with the missing-value strategy.

3. Outliers are legitimate-but-extreme values. Detect them, then decide: keep, cap, transform, or remove.

Method	Rule	Good for	Caveat
IQR fences	Flag x < Q1 − 1.5·IQR or x > Q3 + 1.5·IQR (IQR = Q3 − Q1)	Any distribution, robust	Flags many points on skewed data
z-score	Flag |x − mean| / std > 3	Roughly normal data	Mean and std are themselves pulled by outliers
Transform first	Apply log, then z-score	Right-skewed positive data	—
Winsorize / clip	Cap values at e.g. the 1st and 99th percentiles	Keeping rows while limiting influence	Choose caps on training data only
Model-based	`IsolationForest`, `LocalOutlierFactor`	Multivariate outliers	More to tune

4. Missing values. First ask *why* they're missing:

- **MCAR** (missing completely at random) — e.g. a sensor randomly drops readings. Any sensible imputation is fine.
- **MAR** (missing at random given other columns) — e.g. older customers skip the income field. Other columns can predict the missing value.
- **MNAR** (missing not at random) — the value itself causes the missingness, e.g. high earners hide income. Missingness is itself a signal: add a **missing-indicator** column.

Strategies:

Strategy	What it does	Use when
Drop rows / column	Remove incomplete rows, or a mostly-empty feature	Very few missing rows (MCAR) / low-value column
Mean / median / mode (`SimpleImputer`)	Fill with a column statistic	Default baseline; median for skewed data; mode (`most_frequent`) for categories
Constant / "missing" category	Fill with a sentinel like `"missing"`	Categoricals — keeps the information that it was absent
Missing indicator (`add_indicator=True`)	Extra 0/1 column "was missing"	Missingness may be informative (MNAR)
KNN (`KNNImputer`)	Average of the k most similar rows	Other features predict this one; scale features first
Iterative / MICE (`IterativeImputer`)	Models each column from the others, round-robin	MAR data with correlated columns (still experimental: first `from sklearn.experimental import enable_iterative_imputer`)
Leave it to the model	`HistGradientBoosting*`, XGBoost, LightGBM handle NaN natively	Tree-boosting models

```python
# code/cleaning.py (key parts)
df = df.drop_duplicates()
df.loc[df["age"] > 110, "age"] = np.nan                # impossible -> missing
df.loc[df["income"] >= 9_999_999, "income"] = np.nan    # sentinel  -> missing

q1, q3 = inc.quantile([0.25, 0.75]); iqr = q3 - q1       # IQR fences (z-scores similar)

# Evaluate imputers honestly: hide 300 KNOWN incomes, impute them, measure the error
X_masked[hide, 1] = np.nan
for name, imp in {"mean": SimpleImputer(strategy="mean"), "median": SimpleImputer(strategy="median"),
                  "KNN (k=5)": KNNImputer(n_neighbors=5),
                  "iterative (MICE-like)": IterativeImputer(random_state=0)}.items():
    filled = imp.fit_transform(X_masked)[hide, 1]
    print(name, np.abs(filled - truth).mean())
```

Output:

```
duplicates removed: 25  -> rows now 3000
duplicate customer_ids left: 0
impossible ages -> NaN: 4, sentinel incomes -> NaN: 3

IQR fences: [-29,300, 137,100]  -> 141 flagged
|z| > 3                     -> 50 flagged
|z| > 3 on log(income)      -> 4 flagged  (skew made raw z-score over-flag)
winsorize (clip) to 1st/99th pct: [12,800, 207,332]

missing per column: {'age': 94, 'income': 243, 'device': 60}
churn rate | income missing: 0.152  | present: 0.114

imputer                MAE on hidden incomes
drop rows (listwise)   n/a  (would keep only 79% of rows)
mean                   32,096
median                 29,903
KNN (k=5)              35,635
iterative (MICE-like)  32,096

device with 'missing' level : {'android': 1314, 'ios': 1036, 'web': 590, 'missing': 60}
...
```

What this shows:

- On a skewed column, the IQR rule flags 141 perfectly normal high earners and the raw z-score flags 50; after a log transform only 4 are extreme. **Skew is not the same as outliers.**
- Dropping every row with any missing value would throw away 21% of the data.
- **Fancy imputers are not automatically better.** Here income was generated independently of the other columns, so KNN and MICE have nothing to learn from (MICE collapses to the mean) and the median wins. The lesson is the *method*: hide known values and measure.
- Churn is 15.2% when income is missing vs 11.4% when present — but missingness was random by construction, so the gap is chance (≈1.8 standard errors with 243 rows). Test "informative missingness" rather than assume it; an indicator column lets the model use it if it's real.

Trade-offs

Simple imputation is fast, stable and easy to reproduce at serving time; model-based imputation can be more accurate but is slower and one more thing to monitor. Any imputation shrinks variance (every filled value is "typical").

Common mistakes

- Computing imputation statistics (mean, median, clip percentiles) on the whole dataset before splitting — leakage (4.8); fit on training data only.
- Deleting outliers that are real and important (fraud *is* the outlier).
- Filling missing categoricals with the mode and losing the "was missing" signal.

In the interview

30 seconds: *"Dedupe on the entity key, turn impossible values into missing, and treat outliers by whether they're errors or real extremes — often a log transform or clipping at training-set percentiles. For missing data, start with median plus a missing indicator, compare KNN or iterative imputation by masking known values, and fit it all inside a pipeline."*

- *"Mean or median?"* — Median for skewed columns; it isn't dragged by the tail.
- *"How do you know imputation helped?"* — Mask known values and measure error, and compare downstream model scores with cross-validation.

Self-check

1. Why convert 999 ages to NaN rather than clip them to 85? → They're errors; the true value is unknown, and clipping invents a specific wrong value.
2. What does `add_indicator=True` add? → A 0/1 column per imputed feature marking which rows were missing.
3. Why might raw z-scores over-flag outliers? → On skewed data the tail is long, and the mean/std are inflated by it.

TL;DR

Dedupe, turn impossible values into NaN, handle outliers deliberately (log/clip, don't blindly delete), impute with training-set statistics (median + indicator is a strong default), and measure imputation quality.

4.4 Encoding categorical variables

What it is

Most models only accept numbers. **Encoding** turns categories like `"ios"` or `"city_17"` into numeric columns. Which encoding you choose tells the model something about how categories relate.

How it works — the four main encoders

Encoder	Output	Example	Use for	Risk
One-hot (`OneHotEncoder`, `pd.get_dummies`)	One 0/1 column per category	`device=ios` → `[0,1,0,0]`	Nominal (unordered) categories with low–moderate cardinality	Width explodes with many levels
Ordinal (`OrdinalEncoder`)	One integer column	`basic/standard/premium` → 0/1/2	Categories with a real order; also fine for tree models	For linear models it imposes a false order and equal spacing on nominal data
Target encoding (`TargetEncoder`)	Each category → (smoothed) mean of y for that category	`city_17` → 0.19 churn rate	High-cardinality categories (hundreds or thousands of levels)	**Target leakage** if computed on the same rows the model trains on
Hashing (`FeatureHasher`)	Category → hash(value) mod m buckets	`city_17` → column 13	Huge or unbounded ID spaces, streaming data	**Collisions**: unrelated values share a bucket; not reversible

Notes:

- **Label encoding** is the name often used for "category → integer code". In scikit-learn, `LabelEncoder` is meant for the *target* y; use `OrdinalEncoder` for features.
- **Dummy-variable trap**: with an intercept, the k one-hot columns always sum to 1, so they're perfectly collinear with the intercept and the coefficients aren't unique. Classical OLS (e.g. statsmodels) then needs `drop="first"`; scikit-learn's `LinearRegression` still fits (section 5.1), and regularised models and trees don't care.
- **Embeddings**: in deep learning each ID gets a learned dense vector (e.g. 32 numbers) — the standard for user and item IDs (Chapter 13).
- Unseen categories at prediction time: `OneHotEncoder(handle_unknown="ignore")` outputs all zeros; `TargetEncoder` uses the global mean.

Target encoding and its leakage trap

Naive target encoding replaces each category with its mean target on the training rows. For a category with 2–3 rows, that mean is basically *those rows' own labels* — the feature copies the answer, so the model shines on training data and fails on new data.

The fix (what scikit-learn's `TargetEncoder` does):

1. **Cross-fitting** — in `fit_transform`, split the training data into k folds; encode each fold using target means learned on the *other* folds, so no row sees its own label.
2. **Smoothing / shrinkage** — blend each category's mean with the global mean, weighting by the category's size, so rare categories stay close to the global rate.
3. At prediction time (`transform`), use encodings learned on the full training set.

CatBoost solves the same problem differently with "ordered target statistics" (each row is encoded using only rows before it in a random permutation).

```
# code/encoding.py (key parts)
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False).fit(df[["device"]].fillna("missing"))
ord_enc = OrdinalEncoder(categories=[["basic", "standard", "premium"]])      # explicit order
fh = FeatureHasher(n_features=16, input_type="string")                        # 16 buckets

df["noise_id"] = rng.integers(0, 1000, len(df)).astype(str)                   # pure noise, 1000 levels
means = pd.Series(y[tr]).groupby(df["noise_id"].iloc[tr].to_numpy()).mean()  # naive: leaky
naive_tr = df["noise_id"].iloc[tr].map(means)
te_enc = TargetEncoder(target_type="binary", cv=KFold(5, shuffle=True, random_state=0))
cf_tr = te_enc.fit_transform(df[["noise_id"]].iloc[tr], y[tr])               # cross-fitted
```

Output:

```
one-hot columns: ['device_android', 'device_ios', 'device_missing', 'device_web']
encode ['ios'], ['tablet'(unseen)]:
 [[0. 1. 0. 0.]
 [0. 0. 0. 0.]]

ordinal plan: [2. 0. 1.]

hashed city (16 buckets), row 0 non-zeros at: [13] [-1.]

Feature = target-encoded 'noise_id' (contains NO real signal):
naive target mean (leaky)              train AUC 0.941 | test AUC 0.519
TargetEncoder (cross-fitted, smoothed) train AUC 0.533 | test AUC 0.518

Feature = target-encoded 'city' (60 levels, real signal):
TargetEncoder(city)                    train AUC 0.632 | test AUC 0.555
OneHot(city)                           train AUC 0.722 | test AUC 0.554
```

Reading it: **AUC** (area under the ROC curve, Chapter 6) is 0.5 for random guessing and 1.0 for perfect ranking. Naive encoding of a pure-noise column scores **0.94 on training data** but 0.52 (chance) on test — a mirage that cross-fitting removes (train ≈ test ≈ 0.52). On the real `city` signal, target encoding matches one-hot on test with **one column instead of 60**. The hashed value is −1 because `FeatureHasher` uses a second hash to pick a sign, so collisions cancel out on average instead of piling up.

When not to use them

- Don't one-hot a 100,000-level ID for a linear model — use hashing, target encoding, or embeddings.
- Don't ordinal-encode nominal categories for linear models or k-NN.
- Don't target-encode without cross-fitting, and never with statistics computed on validation/test rows.

Real-world examples

- Feature hashing ("the hashing trick") was popularised for large-scale spam filtering and ad click prediction, and is built into Vowpal Wabbit and scikit-learn's `FeatureHasher` / `HashingVectorizer`.
- CatBoost (Yandex) exists largely to handle categorical features safely with ordered target statistics.

In the interview

30 seconds: *"Low-cardinality nominal → one-hot. Ordered → ordinal. High-cardinality → target encoding (cross-fitted and smoothed), hashing if the ID space is unbounded, or embeddings in a neural model. Target encoding's risk is a row's own label leaking into its feature, so I encode out-of-fold."*

- *"Hashing vs target encoding?"* — Hashing needs no labels, no state and handles new values, but collides and isn't interpretable; target encoding is compact and informative but needs leakage protection and can go stale.
- *"Why is ordinal encoding OK for trees?"* — Trees split on thresholds and can isolate any level with several splits; the false order just costs a few extra splits.

Self-check

1. What does `handle_unknown="ignore"` do for an unseen category? → Encodes it as all zeros.
2. Why did naive target encoding of noise get train AUC 0.94? → Each small category's mean was computed from the row's own label.
3. Two ways `TargetEncoder` prevents that? → Cross-fitting and shrinkage toward the global mean.

TL;DR

One-hot for small unordered sets, ordinal for ordered ones, target encoding (out-of-fold, smoothed) or hashing for huge ones, embeddings in deep models. Target encoding is the most powerful and the easiest to leak.

4.5 Feature scaling

What it is

Scaling puts numeric features on comparable ranges. Without it, a feature measured in thousands (income, proline) dominates one measured in fractions.

Scaler	Formula	Result	Sensitive to outliers?
Standardization (`StandardScaler`)	z = (x − mean) / std	mean 0, std 1	Yes (mean and std move)
Min-max (`MinMaxScaler`)	(x − min) / (max − min)	range [0, 1]	Very (one outlier squashes everyone else)
Robust (`RobustScaler`)	(x − median) / IQR	median 0, IQR 1	No (median and IQR ignore extremes)
Log / power (`np.log1p`, `PowerTransformer`)	reshapes the distribution	less skew	Reduces their influence

Scaling changes the *units*, not the *shape* — a skewed feature stays skewed after `StandardScaler`. To change the shape, use a log or power transform.

Why it matters — and which models care

- **Distance-based models** (k-NN, k-means, RBF-kernel SVM): the widest feature dominates the distance.
- **Gradient-descent models** (linear/logistic regression, neural nets) converge much faster on similar scales (section 5.4 shows why).
- **Regularised models** (Ridge, Lasso, logistic regression's default L2) penalise coefficient size, which depends on units.
- **PCA** finds directions of largest variance — unscaled, that's just the column with the biggest units.
- **Trees don't care**: a split "income ≤ 50,000" works the same in dollars or thousands.

```
# code/scaling.py (key parts)
x = np.array([[10.0], [12], [11], [13], [12], [11], [100]])
for s in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
    print(type(s).__name__, s.fit_transform(x).ravel().round(2))

X, y = load_wine(return_X_y=True)     # 13 features on very different scales
for name, m in models.items():
    accs = [cross_val_score(m, X, y, cv=5).mean()]
    for sc in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
        accs.append(cross_val_score(make_pipeline(sc, m), X, y, cv=5).mean())
```

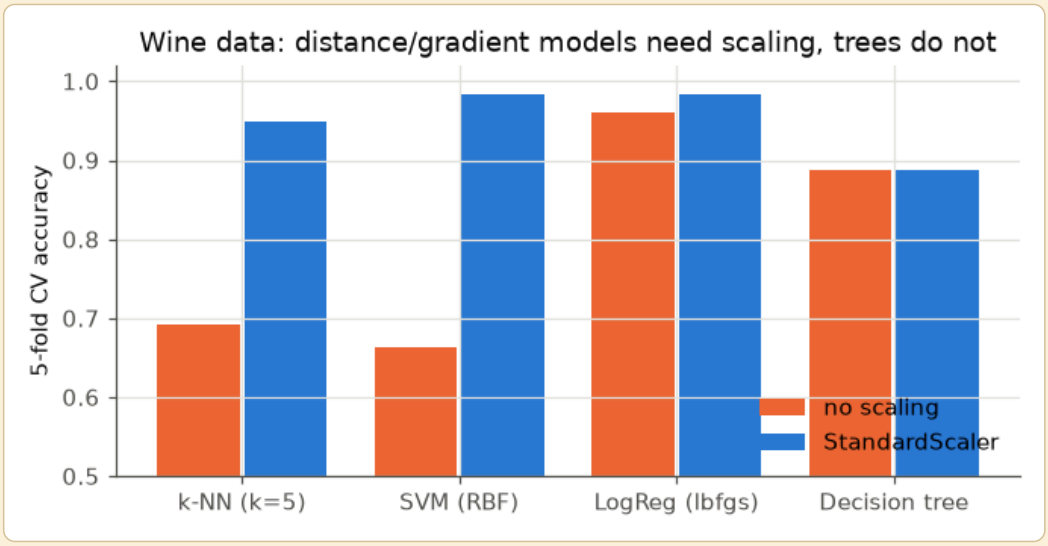
Output:

```
raw       : [ 10.  12.  11.  13.  12.  11. 100.]
StandardScaler : [-0.46 -0.39 -0.42 -0.36 -0.39 -0.42  2.45]
MinMaxScaler   : [0.   0.02 0.01 0.03 0.02 0.01 1.  ]
RobustScaler   : [-1.33  0.   -0.67  0.67  0.   -0.67 58.67]

feature ranges: min std = 0.124, max std = 314.0

model             no scaling  standard  min-max  robust
k-NN (k=5)           0.691     0.949     0.950     0.938
SVM (RBF)            0.663     0.983     0.978     0.983
LogReg (lbfgs)       0.961     0.983     0.978     0.989
Decision tree        0.888     0.888     0.888     0.888
```

Worked check for the robust scaler: median = 12, Q1 = 11, Q3 = 12.5, so IQR = 1.5 and 100 → (100 − 12)/1.5 = 58.67. The six normal values keep a useful spread (−1.33 to 0.67); with min-max they are all squeezed into [0, 0.03] by the one outlier.



Trade-offs / when to use which

StandardScaler is the default (linear models, SVMs, neural nets, PCA); **MinMaxScaler** when a bounded range is needed (image pixels to [0, 1]) and there are no big outliers; **RobustScaler** when there are outliers you want to keep; **nothing** for tree ensembles.

Common mistakes

- Fitting the scaler on all data before splitting (leakage, 4.8) — fit on train, `transform` the rest.
- Scaling one-hot columns is harmless but usually unnecessary; scaling the *target* in regression requires inverting predictions (`TransformedTargetRegressor`).
- Expecting scaling to fix skew.

In the interview

30 seconds: *"Scale for anything that uses distances, gradients, or penalties — k-NN, SVM, linear models, neural nets, PCA, k-means. Standardize by default, robust-scale if there are outliers, skip it for trees. Always fit the scaler on the training fold only."*

- *"Does logistic regression need scaling?"* — For correctness, not strictly; for convergence speed and fair regularisation, yes.

Self-check

1. Why is min-max bad with outliers? → The outlier becomes 1 and squashes all other values near 0.
2. Does `StandardScaler` make data normal? → No, it only shifts and rescales.
3. Why didn't the tree's accuracy change? → Splits are thresholds; they're invariant to monotonic rescaling.

TL;DR

Standardize for distance/gradient/penalty-based models, robust-scale with outliers, nothing for trees, log-transform for skew — and fit the scaler on training data only.

4.6 Feature creation

What it is

Feature engineering is creating new inputs that make the pattern easier for a model to learn. A linear model can only draw straight lines in the features you give it; a good feature bends the problem so a straight line works.

How it works — the core techniques

1. Interactions and polynomials. If the target depends on a *product* (revenue = price × quantity), add that product. `PolynomialFeatures(degree=2)` adds all squares and pairwise products.

2. Binning (discretization). Cut a numeric feature into intervals (equal-width, quantile or domain-chosen) and one-hot them (`KBinsDiscretizer`), so a linear model can fit non-monotonic shapes such as risk that's high for both young and old customers.

3. Log and power transforms for heavy tails. A **heavy tail** means most values are small but a few are enormous (income, counts, prices). `log1p(x) = log(1 + x)` compresses the tail and handles zeros.

4. Date and time features. Extract parts (month, day of week, hour, is_weekend, is_holiday) and durations ("days since signup"), and use **cyclical encoding** for periodic values: month m → (sin(2πm/12), cos(2πm/12)), so December and January are neighbours instead of 11 apart.

5. Text: bag-of-words and TF-IDF. Bag-of-words counts each word per document. **TF-IDF** (term frequency × inverse document frequency) down-weights words that appear in many documents. In scikit-learn's default: tf = raw count in the document, idf(t) = ln((1 + n) / (1 + df(t))) + 1 where n = number of documents and df(t) = documents containing t, then each row is scaled to unit length (L2 norm).

6. Aggregates and ratios. "Tickets per month of tenure", "spend in the last 7 days", "user's average order value" — often the strongest features in business data.

```
# code/features.py (key parts)
revenue = price * qty + rng.normal(0, 1, n)
lin   = cross_val_score(LinearRegression(), X, revenue, cv=5, scoring="r2").mean()
inter = cross_val_score(make_pipeline(PolynomialFeatures(2, interaction_only=True, include_bias=False),
                                      LinearRegression()), X, revenue, cv=5, scoring="r2").mean()

p = 1 / (1 + np.exp(-(0.004 * (age - 50) ** 2 - 2)))       # U-shaped risk
binned = make_pipeline(KBinsDiscretizer(n_bins=6, encode="onehot", strategy="quantile",
                                        quantile_method="averaged_inverted_cdf"), LogisticRegression())

"month_sin": np.sin(2 * np.pi * ts.dt.month / 12), "month_cos": np.cos(2 * np.pi * ts.dt.month / 12)

docs = ["app is slow and expensive", "love the app, fast and easy",
        "slow slow slow, will cancel", "easy to use and helpful support"]
M = TfidfVectorizer().fit_transform(docs)
```

Output:

```
revenue ~ price, qty          R^2 = 0.893
revenue ~ price, qty, price*qty R^2 = 0.997
PolynomialFeatures(2) names: ['price' 'qty' 'price^2' 'price qty' 'qty^2']

U-shaped age effect: raw age AUC = 0.531 | 6 quantile bins AUC = 0.747

 signup_date  year  month  dayofweek  is_weekend  days_since  month_sin  month_cos
 2025-10-09  2025     10          3           0          84     -0.866        0.5
 2024-12-11  2024     12          2           0         386      0.000        1.0
 ...

vocabulary: ['and', 'app', 'cancel', 'easy', 'expensive', 'fast', 'helpful', 'is', 'love', 'slow', 'support', 'the', 'to', 'use', 'will']
doc 2 non-zero weights: {'cancel': 0.363, 'slow': 0.858, 'will': 0.363}
idf('slow') = 1.511, idf('and') = 1.223
churn ~ TF-IDF(review) alone: AUC = 0.736
```

TF-IDF worked by hand for doc 2 ("slow slow slow, will cancel"), n = 4 documents:

- idf(slow): in 2 docs → ln(5/3) + 1 = 1.511. idf(will), idf(cancel): 1 doc each → ln(5/2) + 1 = 1.916. ("and" appears in 3 docs → 1.223: common words get low weight.)
- raw tf·idf: slow 3 × 1.511 = 4.533; will 1.916; cancel 1.916.

- L2 norm = √(4.533² + 1.916² + 1.916²) = √27.89 = 5.281 → slow 0.858, will 0.363, cancel 0.363. ✓ matches the output.

The interaction feature takes R^2 from 0.893 to 0.997 because the true relationship is a product. Binning takes AUC from 0.53 (a single slope can't draw a U) to 0.75.

Feature groups in production systems

When brainstorming features for a real product, think in four groups so you don't miss a whole family. For predicting a food-delivery ETA at checkout:

- **Entity** — one participant on its own, changing rarely (the restaurant's typical kitchen time, the courier's vehicle).
- **Behavioural aggregate** — a count or average over a recent window (orders received in the past 30 minutes); worthless unless fresh.
- **Context** — the moment of the request (hour, weekday, rain, trip distance).
- **Cross** — a pair of participants together (this courier's past speed at this restaurant).

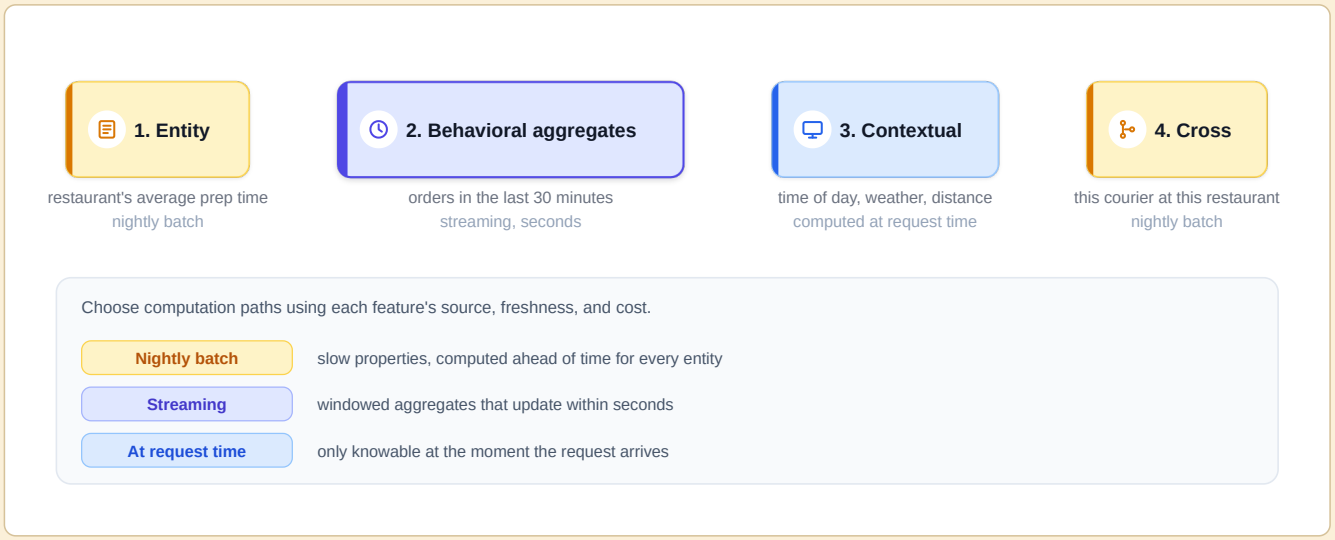


Figure — feature groups and their compute paths (course diagram)

One rule applies from day one: **point-in-time availability** — every feature must be computable *at prediction time* from data that existed then (section 4.8 shows what happens otherwise). How each group is computed and served (nightly batch vs streaming, the offline/online **feature store**, latency budgets and fallbacks for request-time features) is the subject of Chapter 16, section 16.4.

Trade-offs

More features can capture more signal but add overfitting risk, compute, latency and maintenance (each feature is a pipeline that can break); degree-2 polynomials on 100 features give 5,150 columns. Trees and neural nets learn interactions themselves, so explicit engineering matters most for linear models and for domain knowledge (ratios, time windows) no model can invent.

Common mistakes

- Features that use future information (a 30-day average that includes days *after* the prediction).
- Treating month or hour as a plain number (December far from January).
- Fitting `TfidfVectorizer` or bin edges on all data instead of training data.

Real-world examples

- **Uber** and **DoorDash** have publicly described ETA models that combine entity, real-time aggregate and context features.
- Search engines long ranked with TF-IDF-style scores (BM25 is its refined descendant), still used as a first-stage retriever.
- Google's **Wide & Deep** recommender (2016) used hand-crafted cross features in the "wide" linear part.

In the interview

30 seconds: *"I'd brainstorm features by group — entity, behavioural aggregates, request context, crosses — check each is available at prediction time, then transform them: log for heavy tails, one-hot or embeddings for categories, cyclical encoding for time, TF-IDF or embeddings for text, products or bins where a linear model needs curvature."*

- *"Why cyclical encoding?"* — So the model sees hour 23 and hour 0 as neighbours.

Self-check

1. Which transform helps a linear model fit a U-shaped effect? → Binning (or a squared term).
2. Encode hour = 6 cyclically. → (sin(2π·6/24), cos(2π·6/24)) = (1, 0).
3. Why does TF-IDF give "and" a low weight? → It appears in many documents (low idf).
4. Why is "orders in the last 30 minutes" risky to compute nightly? → It's only useful if fresh; a stale value is noise (see 16.4).

TL;DR

Give the model the shape of the problem: products for interactions, bins or polynomials for curves, logs for tails, sin/cos for cycles, TF-IDF for text, aggregates for behaviour. In production, every feature must exist at prediction time.

4.7 Train, validation and test splits

What it is

You split data so that the score you report reflects performance on **unseen** data:

- **Training set** — the model learns its parameters here.
- **Validation set** — you compare models and tune **hyperparameters** (settings chosen before training, like tree depth) here.
- **Test set** — touched **once**, at the end, for the final unbiased estimate.

Analogy: homework (train), mock exams (validation), the real exam (test). If you practise on the real exam paper, your score means nothing.

How it works

1. **Random split** — shuffle, then e.g. 60/20/20, or 80/20 with cross-validation on the 80% replacing a fixed validation set (Chapter 6).
2. **Stratified split** — keep the class proportions the same in every split (`stratify=y`). Essential for imbalanced or small datasets.
3. **Time-based split** — train on the past, validate/test on the future (`TimeSeriesSplit`, or cut by date). Use it whenever the model will predict the future, which is almost every production system.
4. **Group split** — keep all rows of one entity (a customer, a patient, a device) in the same split (`GroupKFold`, `GroupShuffleSplit`). Use it when the model must work for *new* entities.

```
# code/splits_and_leakage.py (key parts)
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.25, stratify=y_tmp,
                                                  random_state=0)      # 0.25 of 80% = 20%
for tr, va in TimeSeriesSplit(n_splits=4).split(np.arange(20)): ...
GroupKFold(n_splits=5).split(X, groups=customer_ids)
```

Output:

```
sizes train/val/test: 600 200 200
positive rate  all/train/val/test: 0.045 0.045 0.045 0.045
WITHOUT stratify, test positive rate over 200 random splits: min 0.005  max 0.080
fold 0: train t=0..3  validate t=4..7
...
fold 3: train t=0..15  validate t=16..19
GroupKFold val customers: [4]  train customers: [0, 1, 2, 3]
...
```

With 4.5% positives and a 200-row test set, an unstratified split can leave you with as few as 1 positive (0.5%) or as many as 16 (8%) — your metric would swing wildly. Stratification pins it at 4.5%.

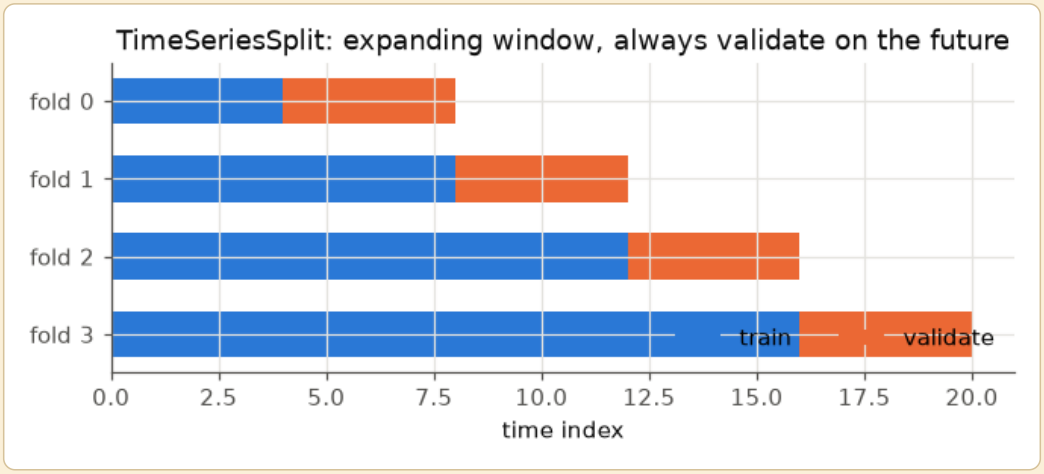




Figure — temporal vs random split (course diagram)

If the goal is "future behaviour of *known* users", the same user may appear in train and test (at different times); if it is "generalise to *new* users", split by user.

Trade-offs

Split	Pros	Cons
Random	Simple, uses data efficiently	Optimistic when data has time order or repeated entities
Stratified	Stable class ratios	Only for classification labels (or binned targets)
Time-based	Matches deployment; exposes drift	Less training data for early folds; must respect label delay
Group	Honest for new entities	Fewer effective samples; groups vary in size

Sizes: with millions of rows, 98/1/1 is fine (1% is still tens of thousands); with a few thousand, use cross-validation instead of a single validation set.

Common mistakes

- Tuning on the test set ("peeking") — the test score is then biased upward.
- Random splits on time-ordered data — the model trains on the future.
- Splitting *after* oversampling or deduplication mistakes, so copies of the same row land in train and test.

Self-check

1. Why stratify? → Keep class proportions stable across splits, especially for rare classes.
2. When do you need a group split? → When rows from the same entity are correlated and you must generalise to new entities.
3. What's the validation set for? → Model selection and tuning; the test set is for the final estimate only.

TL;DR

Train to learn, validate to choose, test once to report. Stratify for classification, split by time when predicting the future, split by group when predicting for new entities.

4.8 Data leakage

What it is

Data leakage is when information that won't be available at prediction time sneaks into training or evaluation. The model looks brilliant offline and disappoints (or fails) in production. It's the single most common reason for "too good to be true" results.

The main kinds — and how to prevent each

Kind	Example	Prevention
Target leakage — a feature is a consequence or proxy of the label	`refund_issued` or `case_resolution_code` is filled in only *after* churn/fraud happened	For every feature ask: *would I know this value at the moment of prediction?*
Future data in aggregates (not point-in-time correct)	A per-merchant fraud rate whose window already contains the transaction being scored	**Point-in-time joins**: take each feature's value as of the prediction time
Train–test contamination — preprocessing fitted on all data	Scaler, imputer, feature selection or target encoder fitted before the split	Split **first**; put every learned step inside a `Pipeline` so it's re-fitted per fold
Duplicates across splits	The same document, image or patient in train and test	Deduplicate (including near-duplicates) *across* splits; group splits
Temporal leakage — random split on time-ordered data	Training on next month to predict this month	Split by time
Leaky identifiers	IDs assigned in an order that correlates with the outcome	Drop IDs; be suspicious of any feature with huge importance

A famous published case: in the KDD Cup 2008 breast-cancer challenge, patient ID numbers turned out to be predictive of the label because of how the data had been assembled — a leak, not medicine.

Three runnable demonstrations

```
# (a) Feature selection BEFORE cross-validation on pure noise (100 rows, 5,000 random features)
leaky_X = SelectKBest(f_classif, k=20).fit_transform(Xn, yn)         # looked at ALL labels
leaky  = cross_val_score(LogisticRegression(max_iter=1000), leaky_X, yn, cv=5).mean()
honest = cross_val_score(make_pipeline(SelectKBest(f_classif, k=20),
                                       LogisticRegression(max_iter=1000)), Xn, yn, cv=5).mean()

# (b) 'refund_issued' is set for 90% of churners, 2% of others - but only AFTER churn
# (c) Drifting data: random split vs train-on-past / test-on-future
```

Output:

```
(a) pure-noise data: select-then-CV accuracy = 0.90 | select-inside-CV = 0.57
(b) with refund_issued: offline AUC 0.975 | without: 0.751 | with it at serving time (field is empty): 0.750
    recall at threshold 0.5: offline 0.89 -> in production 0.00
(c) drifting data: random-split AUC 0.751 | train-on-past, test-on-future AUC 0.551
```

- **(a)** On pure noise honest accuracy is ~0.5 (0.57 is chance on 100 rows). Selecting the 20 "best" features with *all* labels, then cross-validating, reports **90%** — selection already saw the validation labels. Selection inside the pipeline fixes it.
- **(b)** The leaky feature lifts offline AUC from 0.75 to 0.975. In production the field is always empty at prediction time; the model relied on it, so at the offline-tuned threshold it catches **0%** of churners.
- **(c)** When the world changes over time, a random split mixes future rows into training and reports 0.75; the honest forward-in-time estimate is 0.55.

A general rule on top of the table: be suspicious of any score far above published baselines — investigate before celebrating.

In the interview

30 seconds: *"Leakage is training on information you won't have at prediction time. I prevent it with point-in-time-correct features, time or entity splits that match deployment, all preprocessing inside the CV pipeline, deduplication across splits, and treating suspiciously strong features as bugs until proven otherwise."*

- *"Your offline AUC is 0.99 on fraud. What do you check first?"* — Feature timestamps vs the label time (post-outcome fields, aggregates including the current event), split method, duplicates, and top feature importances.
- *"Is fitting a StandardScaler on all data really a problem?"* — Usually a small bias, but it's the same mistake that becomes huge with target encoding or feature selection, so never do it.

Self-check

1. Name a label-adjacent feature for fraud. → Chargeback flag, refund flag, case-resolution code.
2. Why does the pipeline fix demo (a)? → Selection is re-fitted on each training fold without seeing that fold's validation labels.
3. What is a point-in-time join? → Joining each training example to feature values as they were at that example's timestamp.

TL;DR

If the model couldn't know it at prediction time, it mustn't see it in training. Split first, pipeline everything, split by time/group, dedupe across splits, and distrust scores that look too good.

4.9 Class imbalance

What it is

Class imbalance means one class is much rarer than the other — fraud (e.g. 0.2% of transactions, about 1 in 500), disease, churn, ad clicks. The rare class is usually the one you care about.

Why it matters

Accuracy lies: if 99.8% of transactions are legitimate, always saying "legitimate" is 99.8% accurate and catches no fraud. **Learning is harder**: the loss is dominated by negatives, so models tend to predict low probabilities for everything. And millions of easy negatives **waste compute**.

How it works — the toolbox

Technique	What it does	Effect on probabilities
Better metrics	Precision, recall, F1, PR-AUC instead of accuracy (Chapter 6)	—
Threshold moving	Keep the model; lower the decision threshold from 0.5	None (probabilities unchanged)
Class weights (`class_weight="balanced"`)	Each positive counts more in the loss (weight ∝ 1/class frequency)	Inflated — need recalibration
Random oversampling	Duplicate minority rows	Inflated; risk of overfitting repeated rows
Random undersampling / negative downsampling	Keep all positives, a fraction w of negatives	Inflated; but the training set shrinks to about a fraction w, so training is ~1/w times cheaper
SMOTE	Create *synthetic* minority points between a minority point and one of its k nearest minority neighbours	Inflated; can blur class boundaries
Recalibration	Map scores back to real-world probabilities	Fixes the above

SMOTE (Synthetic Minority Over-sampling Technique, Chawla et al., 2002): for a minority point a, pick one of its k nearest minority neighbours b, and create a + u·(b − a) with u random in [0, 1]. It's in the `imbalanced-learn` library (`imblearn.over_sampling.SMOTE`); here it's written from scratch.

Recalibrating after negative downsampling. If you keep a fraction w of negatives, every negative now counts w times as often, so the model's **odds** p/(1 − p) are inflated by 1/w. Multiply the odds back by w and convert to a probability:

odds_true = w · p/(1 − p) → p_true = odds_true / (1 + odds_true) = p·w / (p·w + (1 − p))

Worked example: w = 0.1, model outputs p = 0.5 on the downsampled data → p_true = 0.05 / (0.05 + 0.5) = **0.091**. (Keeping 1 in 10 negatives on a 0.2% base rate raises the positive share to about 2%.) General recalibration methods (Platt scaling, isotonic regression) are in Chapter 6.

Resample only the training data, and only *after* splitting — never the validation or test sets, which must reflect the real population.

```
# code/imbalance.py (key parts) — 20,000 rows, ~2.2% positives, gradient-boosted trees
def smote(X_min, n_new, k=5, seed=0):
    rng = np.random.default_rng(seed)
    _, idx = NearestNeighbors(n_neighbors=k + 1).fit(X_min).kneighbors(X_min)
    base = rng.integers(0, len(X_min), n_new)               # random minority points a
    neigh = idx[base, rng.integers(1, k + 1, n_new)]       # one of their k neighbours b
    return X_min[base] + rng.random((n_new, 1)) * (X_min[neigh] - X_min[base])   # a + u(b - a)

report("GBM class_weight=bal.", Model(class_weight="balanced").fit(X_tr, y_tr), X_te, y_te)
keep = np.r_[pos, rng.choice(neg, int(len(neg) * 0.10), replace=False)]   # downsample negatives
p_corr = p_s * w / (p_s * w + (1 - p_s))                                   # prior correction
```

Output:

```
positive rate: train 0.0221, test 0.0222  (about 1 in 45)

method                        acc   prec    rec     F1 ROC-AUC  PR-AUC   mean p
majority-class baseline     0.978  0.000  0.000  0.000   0.500   0.022   0.0000
GBM, no fix                 0.980  0.604  0.218  0.320   0.905   0.402   0.0151
GBM class_weight=bal.       0.969  0.376  0.579  0.456   0.906   0.402   0.0668
random oversampling         0.967  0.353  0.571  0.437   0.910   0.412   0.0659
SMOTE (from scratch)        0.936  0.209  0.684  0.320   0.901   0.384   0.1196
downsample negatives 1:10   0.958  0.283  0.571  0.378   0.916   0.368   0.0729

after downsampling, mean predicted p = 0.0729 vs true rate 0.0222
after prior correction p' = p*w / (p*w + 1 - p): mean p' = 0.0208 (ranking unchanged: ROC-AUC 0.916)
plain model, threshold  0.5: precision 0.604  recall 0.218
plain model, threshold  0.2: precision 0.490  recall 0.376
plain model, threshold  0.1: precision 0.426  recall 0.496
```

What to take from this:

- The do-nothing baseline has **97.8% accuracy** and zero recall — accuracy is the wrong metric.
- **Ranking metrics barely change** (ROC-AUC ≈ 0.90–0.92, PR-AUC ≈ 0.37–0.41). Resampling and weighting mostly *move the operating point* at threshold 0.5 — which threshold moving on the plain model does for free.
- Every rebalancing method inflates the mean predicted probability (0.07–0.12 vs a true 0.022); the prior correction brings the downsampled model back to 0.021 with ranking untouched.
- SMOTE isn't magic: the most recall, the worst precision, slightly lower PR-AUC.

Trade-offs

Downsampling is the fastest (a real win at billions of rows) but discards data; class weights keep all data at more compute; oversampling/SMOTE enlarge the training set and risk overfitting copies or inventing unrealistic points (SMOTE works poorly with categorical or high-dimensional features); threshold moving is free and keeps calibrated probabilities but doesn't help the model *learn* the minority better. All but threshold moving need recalibration if scores are used as probabilities.

When not to use it

If the model is only used for ranking (top-k alerts) or you'll choose the threshold on a precision–recall curve anyway, rebalancing is often unnecessary. Strong models like gradient boosting on moderately imbalanced data (say 5–20% positives) usually do fine without it — just use the right metric.

Common mistakes

- Oversampling **before** splitting — copies of the same positive end up in train and test.
- Resampling the test set.
- Reading a downsampled model's scores as real probabilities (e.g. in an ad auction or an expected-cost calculation).

Real-world examples

- Facebook's 2014 ad click-prediction paper (He et al.) used negative downsampling and re-calibrated with q = p / (p + (1 − p)/w) — the same formula as above, rearranged. The production version of this workflow, with Platt/isotonic calibration, is in Chapter 16, section 16.3.

In the interview

30 seconds: *"First, change the metric — precision, recall and PR-AUC, not accuracy. Then reweight the loss or downsample negatives for speed, and recalibrate scores to the true base rate if they're used as probabilities. Pick the threshold from the business cost of each error. SMOTE is an option, but I'd compare it with class weights on a stratified validation set."*

- *"Downsample or reweight?"* — Downsample when data is huge and compute matters; reweight when data is modest. Both need recalibration.
- *"Does class_weight improve ROC-AUC?"* — Often barely; it mainly shifts where a 0.5 threshold lands. Use it if it helps the chosen metric in validation.

Self-check

1. Why is 99% accuracy meaningless at a 1% positive rate? → The all-negative model gets it.
2. Recalibrate p = 0.2 with w = 0.1. → 0.02/(0.02 + 0.8) = 0.024.
3. How does SMOTE create a sample? → A random point on the line segment between a minority point and one of its minority nearest neighbours.

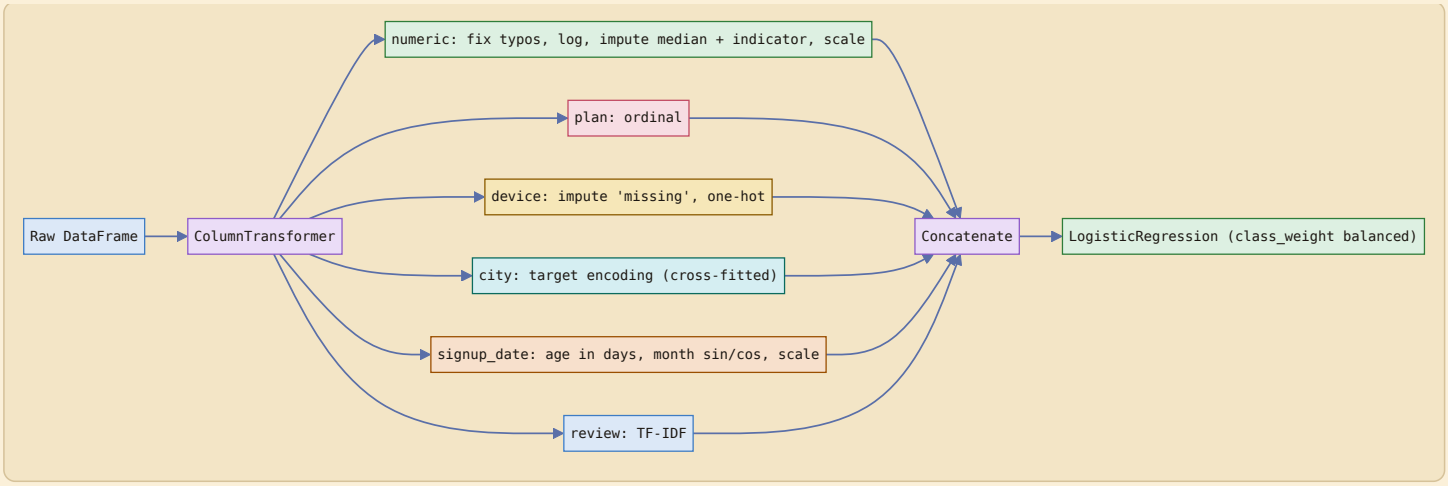
TL;DR

Imbalance breaks accuracy, not learning per se. Use precision/recall/PR-AUC, then weight, downsample or oversample the training data, move the threshold to fit business costs, and recalibrate probabilities to the real base rate.

4.10 End-to-end: `Pipeline` + `ColumnTransformer`

What it is

A scikit-learn `Pipeline` chains steps (transformers, then a final model) into one estimator with `fit` and `predict`. A `ColumnTransformer` applies different preprocessing to different columns and concatenates the results. Together they turn "a notebook of manual steps" into **one object** that you fit, tune, save and serve.



Why it matters

1. **No leakage** — in cross-validation every learned step (imputer medians, scaler means, TF-IDF vocabulary, target encodings) is re-fitted on each training fold only.
2. **No training-serving skew** (production features differing from training features) — the same code transforms data at training and prediction time.
3. **Tuning across steps** — `GridSearchCV` tunes preprocessing and model together via `step__param` names.
4. **One artifact to ship** — `joblib.dump(pipe, "model.joblib")`.

```
# code/pipeline_end_to_end.py (key parts)
def fix_impossible(X):                      # stateless rule: no fitting, safe anywhere
    X = X.copy()
    X["age"] = X["age"].where(X["age"] <= 110)
    X["income"] = np.log1p(X["income"].where(X["income"] < 9_999_999))
    return X

numeric_pipe = make_pipeline(FunctionTransformer(fix_impossible, feature_names_out="one-to-one"),
                             SimpleImputer(strategy="median", add_indicator=True), StandardScaler())
pre = ColumnTransformer([
    ("num", numeric_pipe, numeric),
    ("plan", OrdinalEncoder(categories=[["basic", "standard", "premium"]]), ["plan"]),
    ("device", make_pipeline(SimpleImputer(strategy="constant", fill_value="missing"),
                             OneHotEncoder(handle_unknown="ignore")), ["device"]),
    ("city", TargetEncoder(target_type="binary", cv=KFold(5, shuffle=True, random_state=0)), ["city"]),
    ("date", make_pipeline(FunctionTransformer(date_parts, feature_names_out=date_names),
                           StandardScaler()), ["signup_date"]),
    ("text", TfidfVectorizer(min_df=2), "review"),     # string, not list: TF-IDF wants 1-D text
])                                                     # remainder="drop" drops customer_id
pipe = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=2000, class_weight="balanced"))])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1.0, 10.0, 100.0]},
                      cv=StratifiedKFold(5, shuffle=True, random_state=0), scoring="average_precision")
search.fit(X_tr, y_tr)
p = search.best_estimator_.predict_proba(X_te)[:, 1]      # test set used once
```

Output:

```
best C: 10.0 | CV PR-AUC: 0.414
features after ColumnTransformer: 27
most negative coefs:
 text__fast    -1.12
...
most positive coefs:
 ...
text__cancel    0.76
city__city      4.94
scaled numeric coefs: {'num__age': 0.0, 'num__income': 0.05, 'num__tenure_months': -0.65, 'num__support_tickets': 0.41,
'num__monthly_charges': 0.43, 'num__usage_hours': -0.49}

TEST  model   : ROC-AUC 0.836 | PR-AUC 0.413
TEST  baseline: ROC-AUC 0.500 | PR-AUC 0.117  (PR-AUC of a no-skill model = positive rate 0.117)

raw new row with unseen city + missing device -> p(churn) = 0.368
```

How to read it:

- The model recovers the real drivers: tenure and usage lower churn; tickets and negative review words raise it; age and income (pure noise here) get ≈ 0. The `city` coefficient looks huge only because a target-encoded value is a probability (0 to ~0.3), not a

standardized number — compare coefficients only on the same scale.

- PR-AUC 0.41 vs 0.117 for a no-skill model: a large lift. Always report a baseline next to the model.
- The best `C` (inverse regularisation strength) is 10, with CV scores nearly flat for C ≥ 10 (0.414 at both 10 and 100). Had the winner sat at the grid's edge with scores still rising, the right move would be to extend the grid (Chapter 6).
- A raw row with a never-seen city and a missing device still gets a sensible prediction.

Useful API details

- `remainder="drop"` is the default (`"passthrough"` keeps untouched columns). Pass a list `["plan"]` for 2-D transformers, a single string `"review"` for text vectorizers that expect 1-D input.
- `pipe.named_steps["pre"].get_feature_names_out()` names every output column (custom `FunctionTransformer`s need `feature_names_out`); `set_output(transform="pandas")` returns DataFrames (not with sparse TF-IDF output).
- Custom functions in a pipeline must be importable (module-level) for `joblib` to save/load it.

Common mistakes

- Preprocessing with pandas *before* the split, then using a pipeline only for the model.
- Forgetting `handle_unknown="ignore"` and crashing on a new category in production.
- Putting data-dependent logic ("clip at the 99th percentile") in a `FunctionTransformer` — it recomputes on each batch; use a fitted transformer instead.

In the interview

30 seconds: *"One sklearn Pipeline with a ColumnTransformer per column type; tune it with cross-validation so every fold re-fits the preprocessing; evaluate once on a held-out test set against a baseline; ship the whole pipeline as one artifact so training and serving share code."*

- *"How does this prevent leakage?"* — `fit` is only ever called on training folds; validation folds only see `transform`.
- *"Where does this break at scale?"* — Streaming data and real-time aggregates need a feature pipeline and feature store (Chapter 16); the idea stays the same.

Self-check

1. What does `model__C` refer to? → Parameter `C` of the step named `model`.
2. Why is `"review"` passed as a string? → `TfidfVectorizer` expects a 1-D sequence of documents.
3. What happens to `customer_id`? → Dropped by the default `remainder="drop"`.

TL;DR

One `Pipeline` (with a `ColumnTransformer` for per-column preprocessing) = no leakage in CV, no training–serving skew, joint tuning, one artifact to deploy.

Chapter TL;DR

- **Labels**: outcomes (late), implicit feedback (biased by exposure and position), humans (costly; κ agreement, versioned guidelines), weak supervision, LLMs (audit with human slices).
- **EDA and cleaning**: look first (on the training split); impossible values → NaN; outliers → log/clip, not blind deletion; median + missing indicator, measured by masking known values.
- **Encoding**: one-hot (small nominal), ordinal (ordered), target encoding (cross-fit + smooth), hashing (unbounded IDs), embeddings (deep models).
- **Scaling**: for distance/gradient/penalty models, not trees; fit on train only.
- **Features**: interactions, bins, logs, sin/cos dates, TF-IDF, aggregates — available at prediction time.
- **Splits and leakage**: stratify; split by time or group to match deployment; split first and pipeline everything.
- **Imbalance**: not accuracy; weight/resample the training data or move the threshold; recalibrate $p \cdot w / (p \cdot w + 1 - p)$.
- **Pipeline + ColumnTransformer**: one leak-proof, tunable, deployable object.

Chapter 5 — Linear and Logistic Regression

Linear regression and logistic regression are the "hello world" of machine learning — and they are still everywhere in production: as baselines, as the final layer of neural networks, as click-through-rate models, credit scores and A/B-test analyses. They are also the best place to learn the machinery every other model uses: a **model**, a **loss function**, and an **optimizer** (gradient descent) that lowers the loss.

What you'll learn

1. The supervised-learning workflow end to end, on a small real dataset.
2. Linear regression: the model, the MSE loss, and two ways to fit it — the normal equation and gradient descent — both written in NumPy and checked against scikit-learn.
3. Gradient descent in practice: learning rate, batch vs mini-batch vs stochastic, and why feature scaling changes everything.
4. The assumptions of linear regression and how to check them with residual plots.
5. Polynomial regression and how model complexity leads to under- and over-fitting.
6. Regularization: Ridge (L2), Lasso (L1), ElasticNet — with coefficient-path plots.
7. Logistic regression: sigmoid, log-loss, decision boundary, gradient descent from scratch vs scikit-learn.
8. Multiclass: one-vs-rest and softmax.
9. How to interpret coefficients (and when not to trust them).

All code is in `code/` and runs with the shared environment (Python 3.14, scikit-learn 1.9, NumPy 2.5). Outputs are real; timings will differ on your machine.

5.1 The supervised-learning workflow

What it is

Supervised learning means learning from **labeled data** — examples where the correct answer is attached (an email marked "spam", a house with its sale price). The model studies these examples, finds the pattern linking inputs to answers, and then predicts answers for new, **unseen** examples. It's called "supervised" because the answer key guides training, like a student checking homework against a teacher's marked solutions.

- **Regression** = the answer is a number (price, temperature, petal length).
- **Classification** = the answer is a category (spam / not spam, species).

How it works — the steps

1. **Get data.** Download a public dataset (Kaggle, UCI, `sklearn.datasets`) or collect your own; it may arrive as CSV, Excel, database tables or text.
2. **Inspect and clean.** Look for missing values (`NaN`, blanks). Drop rows or columns that are mostly empty and unimportant, or **impute** (fill in) with the mean, median, mode or a default. Some models refuse missing values entirely; others just get worse.
3. **Encode categories.** Models need numbers. **Label (integer) encoding** maps Red → 0, Blue → 1, Green → 2 — fine for trees, but it tells a linear model that Green > Blue > Red, which is meaningless. **One-hot encoding** creates `is_Red`, `is_Blue`, `is_Green` 0/1 columns and avoids the fake order. (Chapter 4 covers all encoders.)
4. **Split** into training and test sets — commonly 80/20 or 70/30 — so you measure performance on data the model has never seen.
5. **Choose a model**, fit it on the training set.
6. **Evaluate** on the test set with a suitable metric, and compare with a trivial baseline.
7. **Iterate** — features, model, hyperparameters (Chapter 6).

Choosing between linear regression and a decision tree

	Linear regression	Decision tree
Task	Regression (logistic regression for classification)	Regression and classification
How it works	Fits a line / plane / **hyperplane** (a flat surface in many dimensions) that minimises squared error	Splits the data with if-then questions into groups with similar outcomes
Strengths	Simple, fast, interpretable coefficients, extrapolates a trend	Captures non-linear patterns and interactions, handles mixed feature types, easy to visualise
Weaknesses	Misses non-linear patterns unless you engineer features	Overfits if grown too deep; can't extrapolate beyond training range
Pick it when	The relationship looks roughly linear (e.g. weight vs height)	The data has clear thresholds or if-then structure

(Chapter 7 covers trees in depth.)

The workflow in code

Predict **petal length** from the other iris measurements plus the species (one-hot encoded):

```python
# code/supervised_walkthrough.py
iris = load_iris(as_frame=True)                          # bundled with sklearn, no download
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = iris.target_names[df.pop("target")]
print(df.head(3).to_string(), "\n")
print("missing values (total):", df.isna().sum().sum())

X = pd.get_dummies(df.drop(columns="petal_length"), columns=["species"], dtype=int)
y = df["petal_length"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)   # 80/20

lin = LinearRegression().fit(X_tr, y_tr)
print("coefficients:", {c: round(float(w), 3) for c, w in zip(X.columns, lin.coef_)})
print("intercept:", round(float(lin.intercept_), 3))
for name, m in [("linear regression", lin),
                ("tree, no depth limit", DecisionTreeRegressor(random_state=0)),
                ("tree, max_depth=3", DecisionTreeRegressor(max_depth=3, random_state=0))]:
    m.fit(X_tr, y_tr)
    print(name, mean_squared_error(y_tr, m.predict(X_tr)), mean_squared_error(y_te, m.predict(X_te)))
```

Output:

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa

missing values (total): 0
model inputs: ['sepal_length', 'sepal_width', 'petal_width', 'species_setosa', 'species_versicolor', 'species_virginica']

coefficients: {'sepal_length': 0.585, 'sepal_width': -0.145, 'petal_width': 0.64, 'species_setosa': -1.144, 'species_versicolor': 0.315,
'species_virginica': 0.829}
intercept: 0.019

y range: 1.0 to 6.9 cm, std 1.77
model                         train MSE  test MSE
baseline (predict mean)                      3.302
linear regression                 0.061     0.088
tree, no depth limit              0.001     0.142
tree, max_depth=3                 0.078     0.121
```

Interpreting the result. **MSE** (mean squared error) is in squared units (cm²), and "lower is better" only means something against the target's scale and a baseline. Petal length spans 1–6.9 cm; predicting the mean gives MSE 3.3, so 0.088 (an average error of roughly √0.088 ≈ 0.3 cm) is good, while an MSE of 5 would be worse than guessing the mean. The unlimited tree scores almost 0 on training data but worse on test: it **overfit** (memorised the training set). Limiting `max_depth` (or raising `min_samples_split`) trades a bit of training accuracy for better generalisation.

One subtlety: the three species columns always sum to 1, which duplicates the intercept (the **dummy-variable trap**). `LinearRegression` still works — it returns the minimum-norm solution, which is why the three species coefficients sum to 0 — but the individual species coefficients are then only meaningful relative to each other. Use `drop="first"` in `OneHotEncoder` (or `drop_first=True` in `get_dummies`) if you want each one read against a reference species.

Self-check

1. Test MSE 0.088 — good or bad? → Only meaningful against the target's scale and a baseline: the mean-predictor scores 3.3, so it's good.
2. Train MSE 0.001, test MSE 0.142 — what happened? → Overfitting: the model memorised the training set.
3. Why one-hot rather than integer-encode `species` for linear regression? → Integers impose a fake order and equal spacing.

TL;DR

Get data → clean → encode → split → fit → evaluate against a baseline → iterate. Linear models for roughly linear trends; trees for thresholds and interactions, with depth limits against overfitting.

5.2 Linear regression: model and loss

What it is

Linear regression predicts a number as a weighted sum of the features plus a constant:

$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_d x_d + b = \mathbf{w \cdot x + b}$

- $\hat{y}$ ("y-hat") — the prediction.
- $x = (x_1 \ldots x_d)$ — the **feature vector** for one example.
- w — the **weights** (coefficients): how much $\hat{y}$ changes when that feature increases by 1, holding the others fixed.
- b — the **bias** or **intercept**: the prediction when all features are 0.

With one feature it's a line; with two, a plane; with more, a hyperplane. A trick used in all the maths: add a constant feature $x_0 = 1$ to every example, so b becomes weight w_0 and the model is just $\hat{y} = \theta^T x$ (θ = all parameters).

The loss: mean squared error

Training means choosing θ to make predictions close to the targets. For m training examples, the **mean squared error** is:

$$MSE(\theta) = (1/m) \Sigma_i (\hat{y}_i - y_i)^2$$

Worked example: targets y = (3, 5, 7), predictions $\hat{y}$ = (2.5, 5, 8). Errors = (−0.5, 0, 1). Squared = (0.25, 0, 1). MSE = 1.25/3 = **0.417**. RMSE = $\sqrt{0.417}$ = 0.65 (back in the target's units).

Why squares? They make every error positive, punish big errors more than small ones, and give a smooth bowl-shaped (convex) function with a single minimum that calculus can find. Minimising MSE is also what you get from **maximum likelihood** if the noise is Gaussian.

Self-check

1. Errors (1, −1, 2): MSE? → (1 + 1 + 4)/3 = 2.
2. What does the intercept b mean? → The prediction when every feature is 0.
3. Why is the MSE "bowl" easy to minimise? → It's convex: one minimum, no local traps.

TL;DR

$\hat{y} = w \cdot x + b$; fit by minimising the average squared error, a convex bowl with one minimum.

5.3 Fitting it: the normal equation vs gradient descent

What it is

Two ways to find the bottom of the MSE bowl:

1. **Normal equation** — solve for θ directly in one step with linear algebra.
2. **Gradient descent (GD)** — start somewhere, repeatedly step downhill until you reach the bottom.

Analogy: the normal equation is reading the lowest point off a map; gradient descent is walking downhill in fog, feeling the slope under your feet.

How it works

Normal equation. At the bottom of the bowl the slope is zero. The gradient of MSE is $(2/m)\, X^T(X\theta - y)$ (derived just below); setting it to zero gives $X^TX\theta = X^Ty$ — the "normal equations" — and, if X^TX is invertible:

$$\theta = (X^TX)^{-1} X^Ty$$

where X is the m × (d+1) matrix of examples (with the column of 1s) and y the vector of targets. In code, never compute the inverse explicitly; solve the linear system $X^TX\,\theta = X^Ty$ (`np.linalg.solve`) or use a least-squares routine (`np.linalg.lstsq`, which uses the SVD and also works when X^TX is not invertible). scikit-learn's `LinearRegression` calls SciPy's least-squares solver.

Gradient descent. The gradient (vector of slopes) of MSE is:

$$\nabla MSE(\theta) = (2/m)\, X^T(X\theta - y)$$

Why: $MSE = (1/m) \Sigma_i (\theta^T x_i - y_i)^2$. By the chain rule, the derivative of one squared term with respect to weight θ_j is $2(\theta^T x_i - y_i) \cdot x_{ij}$ — "twice the error times that feature". Averaging over the m examples and stacking all j gives $(2/m)\, X^T$(error vector).

Then repeat: $\theta \leftarrow \theta - \eta\, \nabla MSE(\theta)$, where η (eta) is the **learning rate** (step size). Each full pass over the training data is an **epoch**.

Worked single step on one feature, data x = (1, 2), y = (2, 4), starting at w = 0, b = 0, η = 0.1:

- predictions (0, 0); errors (−2, −4)
- $\partial/\partial b = (2/2)(-2 - 4) = -6$; $\partial/\partial w = (2/2)(1 \cdot -2 + 2 \cdot -4) = -10$
- new b = 0 − 0.1·(−6) = 0.6; new w = 0 − 0.1·(−10) = 1.0 → already heading toward the true line y = 2x.

```python
# code/linreg_from_scratch.py (key parts)
def add_bias(X):
    return np.c_[np.ones(len(X)), X]                    # column of ones -> intercept is w0

def normal_equation(X, y):
    Xb = add_bias(X)
    return np.linalg.solve(Xb.T @ Xb, Xb.T @ y)         # solves (X^T X) theta = X^T y

def batch_gd(X, y, lr=0.1, epochs=1000):
    Xb = add_bias(X)
    theta = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        err = Xb @ theta - y
        theta -= lr * (2 / len(y)) * Xb.T @ err         # gradient step on MSE
    return theta

x = 2 * rng.random((100, 1)); y = 4 + 3 * x[:, 0] + rng.normal(0, 1, 100)    # truth: b=4, w=3
# ... then the 10-feature diabetes dataset, standardized, compared with sklearn
```

Output:

```
normal equation  [b, w]: [3.949 3.038]
batch GD         [b, w]: [3.949 3.038] (MSE 51.39 -> 0.959)
sklearn          [b, w]: [3.949 3.038]

method            test MSE   max |diff vs sklearn|      time
normal eq.          3424.3               1.46e-12    0.07ms
batch GD            3424.6               2.68e-01   35.06ms
sklearn             3424.3               0.00e+00
baseline (predict train mean) test MSE: 5134.8
GD after 5000 epochs, coef for s1, s2 (correlated pair): [-31.   15.1] exact: [-31.2  15.3]

largest standardized effects (+1 std in feature -> change in disease score):
    s5:   +35.8
    s1:   -31.2
   bmi:   +27.5
    s2:   +15.3
intercept = mean training target: 151.6 = 151.6
```

All three methods agree on the toy problem (3.949, 3.038 — close to the true 4 and 3; the gap is noise). On the diabetes data the normal equation matches scikit-learn to 12 decimal places. Gradient descent is close but still slightly off after 5,000 epochs on the coefficients of `s1` and `s2` — two strongly correlated blood-serum measurements that create a long, narrow valley in the loss surface where GD crawls (section 5.4 shows why). Because the features are standardized (mean 0), the intercept equals the mean training target.

Trade-offs

	Normal equation / least squares	Gradient descent
Result	Exact, one shot	Approximate, iterative
Hyperparameters	None	Learning rate, epochs (and batch size)
Cost	About $O(m \cdot d^2)$ to form X^TX plus $O(d^3)$ to solve — fine for d up to a few thousand	$O(m \cdot d)$ per epoch; scales to huge m and d
Memory	Needs X^TX (d × d)	Can stream mini-batches from disk
Feature scaling	Not needed	Needed for fast convergence
Works for other losses	Only squared error (and Ridge)	Any differentiable loss — logistic regression, neural networks

When not to use each

- Don't use the normal equation when d is very large (tens of thousands of features) or data doesn't fit in memory.
- Don't hand-roll GD when a closed form or a well-tuned library solver exists — use it for learning and for losses without closed forms.

Common mistakes

- Computing `np.linalg.inv(X.T @ X)` explicitly — slower and numerically less stable than `solve` / `lstsq`.
- Forgetting the column of ones (no intercept).
- Running GD on unscaled features and concluding "GD doesn't work".

Real-world examples

- `LinearRegression` in scikit-learn (least squares via SciPy); `statsmodels` OLS for statistical inference (p-values, confidence intervals).
- `SGDRegressor` / `SGDClassifier` and Vowpal Wabbit fit linear models one small batch at a time with SGD, so the dataset never has to fit in RAM.
- Every neural network is trained by variants of gradient descent (SGD with momentum, Adam).

In the interview

30 seconds: *"Linear regression minimises MSE. You can solve it exactly with the normal equation, $\theta = (X^TX)^{-1}X^Ty$ — in practice with a least-squares solver — which is cheap when there are few features. Gradient descent iterates $\theta \leftarrow \theta - \eta \cdot (2/m) X^T(X\theta - y)$; it scales to huge data, works for any differentiable loss, but needs a learning rate and scaled features."*

- *"When would the normal equation fail?"* — X^TX singular (duplicate or perfectly collinear features, or more features than rows); use the pseudo-inverse (SVD) or Ridge, which makes $X^TX + \alpha I$ invertible.
- *"Is the MSE loss convex?"* — Yes, for linear regression; GD with a small enough learning rate reaches the global minimum.

Self-check

1. Write the GD update for linear regression. → $\theta \leftarrow \theta - \eta\ (2/m)\ X^T(X\theta - y)$.
2. Why add a column of ones? → So the intercept is learned as an ordinary weight.
3. Which scales better to 10 million features: normal equation or GD? → GD.
4. Why did GD lag on `s1` / `s2`? → Correlated features make a narrow valley; GD needs many steps along it.

TL;DR

Normal equation: exact, no tuning, O(d³) — great for small d. Gradient descent: iterative, scalable, general — needs a learning rate and scaled features.

5.4 Gradient descent in practice

What it is

Gradient descent comes in three flavours, differing only in **how many examples are used to compute each gradient**:

Variant	Examples per update	Updates per epoch (m = 100)	Behaviour
Batch GD	All m	1	Smooth, exact gradient; slow per step on big data
Mini-batch GD	A batch, e.g. 16–512	m / batch size (100/16 → 7)	Noisy but efficient; uses vectorised hardware (GPUs); the default in deep learning
Stochastic GD (SGD)	1	m (100)	Very noisy, cheap steps; noise can escape shallow local minima; never settles unless the learning rate decays

How it works

1. Shuffle the training data at the start of each epoch.
2. For each batch, compute the gradient on that batch only and take a step.
3. Optionally decay the learning rate over time (a **learning-rate schedule**), e.g. $\eta_t = \eta_0 / (1 + k \cdot t)$, so the noisy steps shrink and the parameters settle.

```
# code/gd_variants.py (key parts)
def run(Xb, y, lr, epochs, batch_size, seed=0, lr_decay=False):
    rng = np.random.default_rng(seed)
    theta, step = np.array([-1.0, -1.0]), 0
    for epoch in range(epochs):
        idx = rng.permutation(len(y))                    # reshuffle every epoch
        for start in range(0, len(y), batch_size):
            b = idx[start:start + batch_size]
            eta = lr / (1 + 0.05 * step) if lr_decay else lr
            theta = theta - eta * mse_grad(Xb[b], y[b], theta)
            step += 1
    return theta

run(Xb, y, lr=0.1, epochs=50, batch_size=100)            # batch
run(Xb, y, lr=0.1, epochs=8,  batch_size=16)             # mini-batch
run(Xb, y, lr=0.1, epochs=1,  batch_size=1, lr_decay=True)  # stochastic
```

Output:

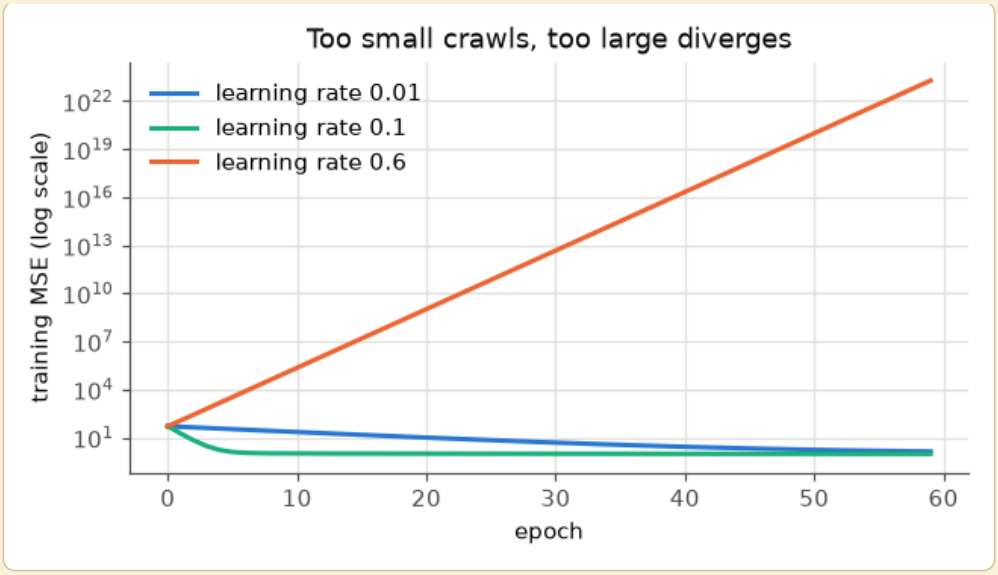
```
lr=0.01  MSE after 60 epochs: 1.37
lr=0.1   MSE after 60 epochs: 0.963
lr=0.6   MSE after 60 epochs: 1.92e+23

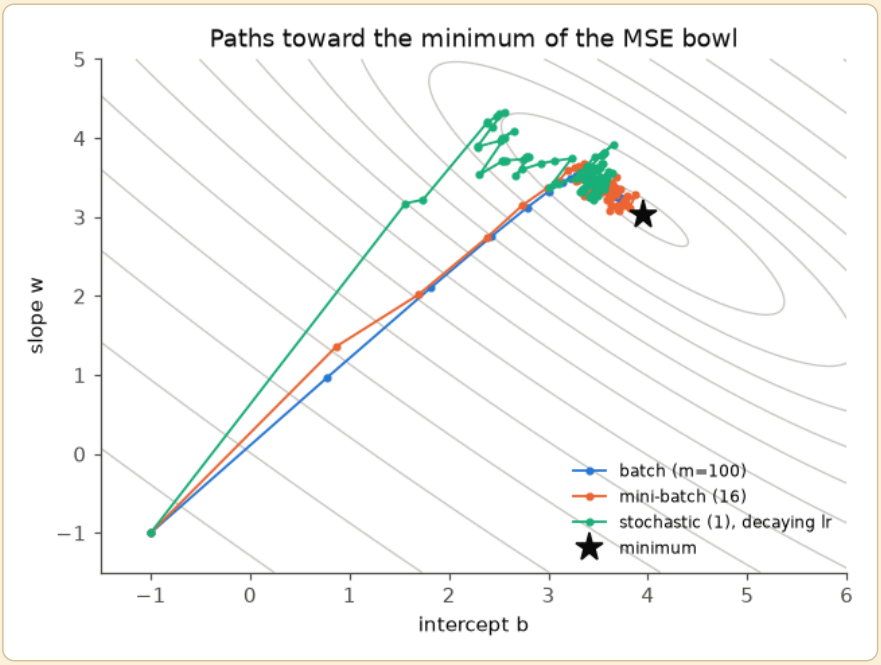
exact (normal eq.): [3.949 3.038]
batch (m=100)                updates=  50  final=[3.762 3.203]
mini-batch (16)              updates=  56  final=[3.814 3.109]
stochastic (1), decaying lr  updates= 100  final=[3.574 3.434]
unscaled      condition number        36  lr=1.36e-02  iterations to converge: 485
standardized  condition number         1  lr=4.86e-01  iterations to converge: 5
```

The learning rate

- **Too small** (0.01): steady but slow — still at MSE 1.37 after 60 epochs.
- **Good** (0.1): reaches the minimum (≈ 0.96, the noise level).
- **Too large** (0.6): every step overshoots further; the loss explodes to 10^{23} — **divergence**.

For MSE the rule is concrete: GD converges if $\eta < 2/\lambda_{max}$, where λ_{max} is the largest eigenvalue of the loss's curvature matrix (the **Hessian**, here $(2/m)X^TX$). In practice you try values on a log scale (0.001, 0.01, 0.1, …) and watch the loss curve.

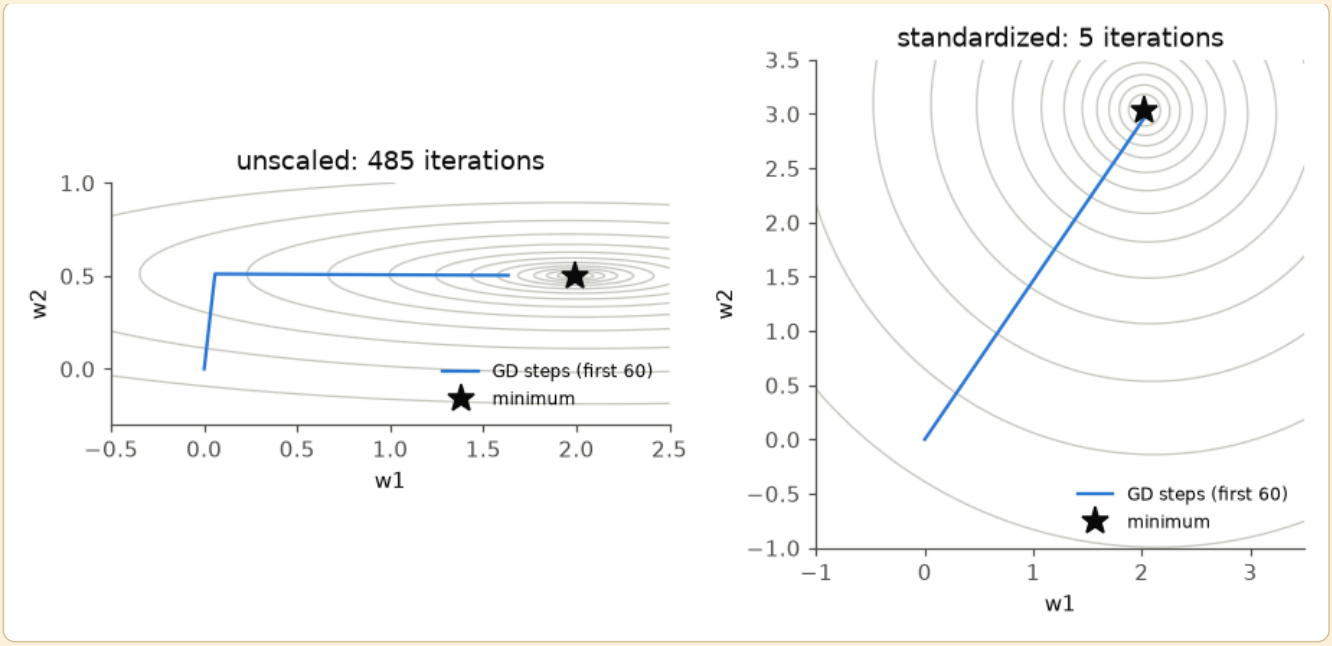




Batch GD moves in a smooth line; mini-batch wiggles; SGD bounces around the minimum and only settles because its learning rate decays. With equal numbers of updates, batch GD saw 50× more data than SGD — in terms of *compute*, SGD and mini-batch are far cheaper per unit of progress on large datasets, which is why they dominate.

Why feature scaling matters for GD

If one feature has a much larger spread than another, the MSE bowl becomes a long, narrow valley. The learning rate must be small enough for the steep direction (or it diverges), which makes progress along the flat direction painfully slow. The **condition number** (λ_max / λ_min) measures how stretched the bowl is: GD's number of steps grows roughly in proportion to it.



Here a feature with 6× the spread of the other gives a condition number of 36 and needs **485** iterations; after standardizing both features the bowl is round (condition number ≈ 1) and GD converges in **5**. With 100× spread the unscaled version needs well over 100,000 iterations. Correlated features (like `s1` / `s2` in the diabetes data) stretch the bowl too — scaling doesn't fully fix that; regularization helps.

Trade-offs / common mistakes

- Batch size: larger batches give smoother gradients and better hardware use, but fewer updates per epoch; very large batches may need a larger learning rate or warm-up.
- Forgetting to shuffle — batches become unrepresentative (e.g. sorted by class).
- Judging SGD by its final iterate without a decaying learning rate — it keeps bouncing.
- Stopping by a fixed number of epochs instead of monitoring validation loss (**early stopping**, Chapter 6).

In the interview

30 seconds: *"Batch GD uses all data per step — smooth but expensive; SGD uses one example — cheap and noisy, needs a decaying learning rate; mini-batch is the practical middle ground and fits GPUs. The learning rate is the key hyperparameter: too small is slow, too large diverges. Scale features first — otherwise the loss surface is a narrow valley and GD zig-zags or crawls."*

- *"Why does SGD sometimes generalise better?"* — Its noise acts like a regularizer and can escape sharp minima; this is an empirical observation in deep learning, not a guarantee.
- *"How do you pick a learning rate?"* — Log-scale search, look at the training-loss curve; or a learning-rate range test; then add a schedule.

Self-check

1. With m = 1,000 and batch size 50, how many updates per epoch? → 20.
2. What happens with η above 2/λ_max on MSE? → Divergence.
3. Why does scaling speed up GD? → It makes the loss bowl rounder (lower condition number), so one learning rate suits all directions.

TL;DR

Batch = exact and slow, SGD = noisy and cheap, mini-batch = the default. Tune the learning rate on a log scale, decay it for SGD, and always scale features.

5.5 Assumptions of linear regression

What it is

Linear regression will always return *some* line. Whether that line's **predictions** are good and its **coefficients and p-values** are trustworthy depends on assumptions:

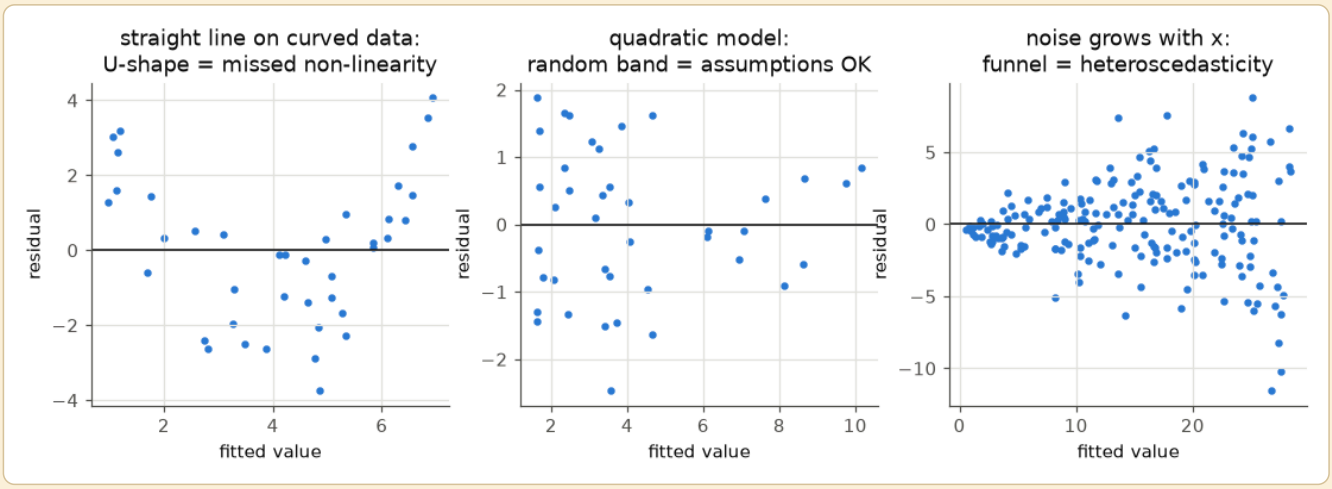
Assumption	Meaning	If violated
Linearity	y is a linear function of the (possibly transformed) features	Systematic errors; add polynomial/log terms or use a non-linear model
Independence	Errors of different rows are unrelated	Typical with time series or repeated measures; standard errors too small
Homoscedasticity	Error spread is constant across predictions	"Funnel" residuals; coefficients still unbiased but standard errors wrong; try log(y) or weighted least squares
Normal errors	Residuals roughly normal	Matters mainly for small-sample confidence intervals and p-values
No (perfect) multicollinearity	Features aren't (near-)copies of each other	Coefficients unstable and hard to interpret (predictions can still be fine)

For pure prediction, linearity and a stable data distribution matter most; the others matter most when you interpret coefficients or report statistical significance.

How to check: residual plots

A **residual** is $y_i - \hat{y}_i$. Plot residuals against fitted values: a good model leaves a structureless horizontal band around 0.

```
# code/polynomial_and_assumptions.py (residual part)
lin = LinearRegression().fit(x[:, None], y)            # straight line on quadratic data
resid = y - lin.predict(x[:, None])
ax.scatter(lin.predict(x[:, None]), resid); ax.axhline(0)
```



- **U-shape** (left): the straight line misses curvature — add x^2, or transform.
- **Random band** (middle): the quadratic model captures the pattern.
- **Funnel** (right): noise grows with x — heteroscedasticity. Log-transform y, use weighted least squares, or report robust standard errors.

Common mistakes

- Checking only R^2 and never plotting residuals.
- Interpreting coefficients causally ("raising x by 1 *causes* y to rise by w"). Regression finds associations in observational data.

In the interview

30 seconds: *"Linear regression assumes a linear relationship, independent errors with constant variance, roughly normal errors, and no strong multicollinearity. For prediction, linearity and a stable distribution matter most; the rest matter when you interpret coefficients or p-values. I check with a residuals-vs-fitted plot."*

- *"What does a funnel-shaped residual plot mean?"* — Heteroscedasticity: error variance grows with the prediction; log-transform y, use weighted least squares or robust standard errors.

Self-check

1. Residuals form a U shape — what's wrong? → The model misses curvature; add x^2 or transform.
2. Does heteroscedasticity bias the coefficients? → No, it makes their standard errors wrong.
3. Can multicollinearity hurt predictions? → Usually not much; it makes individual coefficients unstable.

TL;DR

Plot residuals vs fitted values: a U means missing non-linearity, a funnel means non-constant variance, and correlated features make coefficients unreliable.

5.6 Polynomial regression

What it is

Polynomial regression is linear regression on polynomial features. For one feature x and degree 2: $\hat{y} = w_2x^2 + w_1x + b$. It is still "linear" — linear in the *weights* — so the same solvers work; only the inputs are expanded (`PolynomialFeatures`).

Why it matters

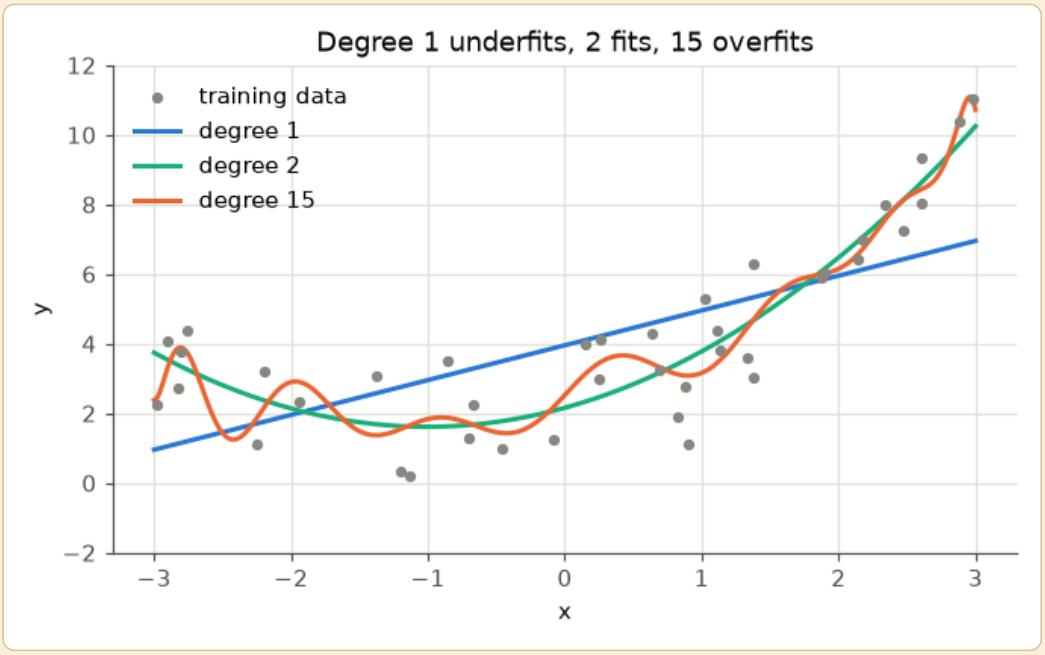
It is the simplest way to fit curves, and the clearest demonstration of **underfitting vs overfitting**:

- **Underfitting** — the model is too simple to capture the pattern: high error on training *and* test data (high **bias**).
- **Overfitting** — the model is so flexible that it fits the noise: low training error, higher test error (high **variance**).

```
# code/polynomial_and_assumptions.py (key parts) — truth: y = 0.5x² + x + 2 + noise, 40 training points
for deg in [1, 2, 15]:
    model = make_pipeline(PolynomialFeatures(deg, include_bias=False), StandardScaler(), LinearRegression())
    model.fit(x[:, None], y)
```

Output:

```
degree  train MSE  test MSE
     1       3.73      2.83
     2       1.14      1.04
    15       0.90      1.35
degree-2 learned: y = 0.54 x^2 + 1.08 x + 2.16   (truth 0.5, 1, 2)
```



Degree 2 recovers the true coefficients (0.54, 1.08, 2.16 vs 0.5, 1, 2). Degree 15 has the *lowest training error* but a higher test error, and its curve wiggles between points and shoots off at the edges. (Here the test points lie inside the training range; outside it, a degree-15 polynomial is wildly wrong.) The noise level is 1.0 (variance of the added noise), so a test MSE ≈ 1.0 is the best any model can do.

Trade-offs / when not to use it

- Features explode with degree and number of inputs: d features at degree k gives C(d+k, k) − 1 columns (10 features at degree 2 → 65; at degree 3 → 285).
- High-degree polynomials are unstable at the edges and extrapolate badly. For smooth curves prefer low degrees, **splines** (`SplineTransformer`) or tree/boosting models.
- Always scale after expanding: x^{15} dwarfs x.

In the interview

30 seconds: *"Polynomial regression expands the inputs with powers and products and fits an ordinary linear model — linear in the weights. The degree is a complexity knob: too low underfits, too high overfits and extrapolates wildly. I'd choose the degree, or add Ridge, using validation data."*

Self-check

1. Is polynomial regression a linear model? → Yes, linear in its parameters.
2. Degree-15 has the lowest training MSE — why not use it? → It fits noise; test error is higher.
3. How many degree-2 features from 3 inputs (no bias)? → 3 + 3 squares + 3 pairwise = 9.

TL;DR

Expand features with powers and products, then fit a linear model. Too low a degree underfits; too high overfits — pick the degree (or regularize) using validation data.

5.7 Regularization: Ridge, Lasso, ElasticNet

What it is

Regularization adds a penalty on the size of the weights to the loss, so the model prefers simpler explanations. It's like a budget: the model can still use features, but each unit of weight costs something, so it only spends where the data strongly justifies it.

Model	Penalty	scikit-learn objective	Effect
Ridge (L2)	$\alpha \sum w_j^2$	$\|y - Xw\|^2 + \alpha\|w\|_2^2$	Shrinks all weights smoothly toward 0; never exactly 0; splits weight among correlated features
Lasso (L1)	$\alpha \sum \|w_j\|$	$(1/2m)\|y - Xw\|^2 + \alpha\|w\|_1$	Sets some weights **exactly 0** → automatic feature selection; picks one of a correlated group
ElasticNet	mix of both	$(1/2m)\|y - Xw\|^2 + \alpha \cdot r\|w\|_1 + \tfrac{1}{2}\alpha(1 - r)\|w\|_2^2$	Sparse *and* stable with correlated features; r = `l1_ratio`

Notes:

- **α** (alpha) is the regularization strength: α = 0 is plain least squares; large α pushes weights toward 0. Note that Ridge's and Lasso's α are on different scales in scikit-learn (Ridge's squared-error term is a sum, Lasso's is divided by 2m), so the same α means different amounts of shrinkage.
- The **intercept is not penalized**.
- **Scale features first.** The penalty treats all weights equally, so it only makes sense if features are on the same scale.
- In logistic regression and SVMs, scikit-learn uses **C = 1/regularization strength** instead: *smaller* C means *stronger* regularization.

Why it matters

- Reduces overfitting when there are many features relative to rows, or correlated features.
- Makes $X^TX + \alpha I$ always invertible, so Ridge has a closed form: $\theta = (X^TX + \alpha I)^{-1}X^Ty$ (with the intercept left unpenalized).
- Lasso gives compact, interpretable models.

Why does L1 produce exact zeros and L2 doesn't? Geometrically, the L1 "budget" region is a diamond with corners on the axes; the loss contours usually first touch it at a corner, where some weights are 0. The L2 region is a circle with no corners. Computationally, the L1 penalty's slope is a constant ±α however small the weight, so it keeps pushing a weight all the way to 0; the L2 penalty's slope 2αw fades as w shrinks.

```
# code/regularization.py (key parts) — diabetes data, standardized
ridge_path = np.array([Ridge(alpha=a).fit(X, y).coef_ for a in np.logspace(-2, 4, 60)])
lasso_path = np.array([Lasso(alpha=a, max_iter=50_000).fit(X, y).coef_ for a in np.logspace(-2, 2, 60)])

X_dup = np.c_[X[:, [2]], X[:, [2]] + rng.normal(0, 0.01, (len(X), 1))]    # bmi twice (near-copy)

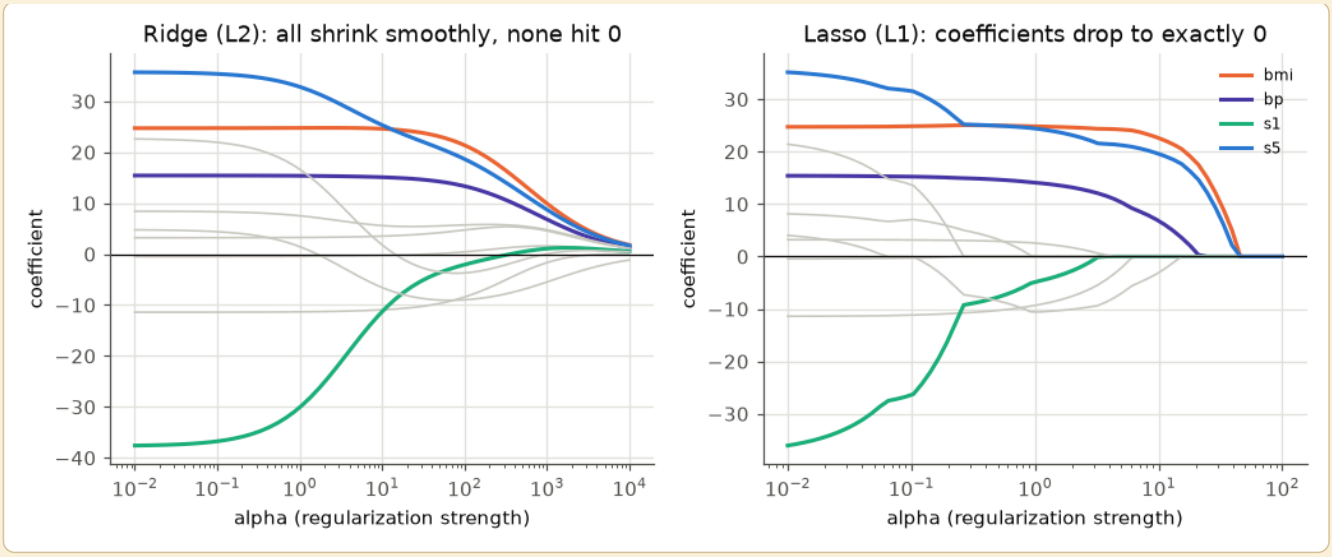
poly = PolynomialFeatures(2, include_bias=False)                          # 65 features, ~330 rows
models = {"OLS": make_pipeline(poly, StandardScaler(), LinearRegression()),
          "RidgeCV": make_pipeline(poly, StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 50))),
          "LassoCV": make_pipeline(poly, StandardScaler(), LassoCV(cv=5, random_state=0)),
          "ElasticNetCV": make_pipeline(poly, StandardScaler(),
                                        ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=5, random_state=0))}
```

Output:

```
Lasso alpha=0.1 :  9 of 10 coefficients non-zero
Lasso alpha=1   :  7 of 10 coefficients non-zero
Lasso alpha=5   :  5 of 10 coefficients non-zero
Lasso alpha=20  :  3 of 10 coefficients non-zero

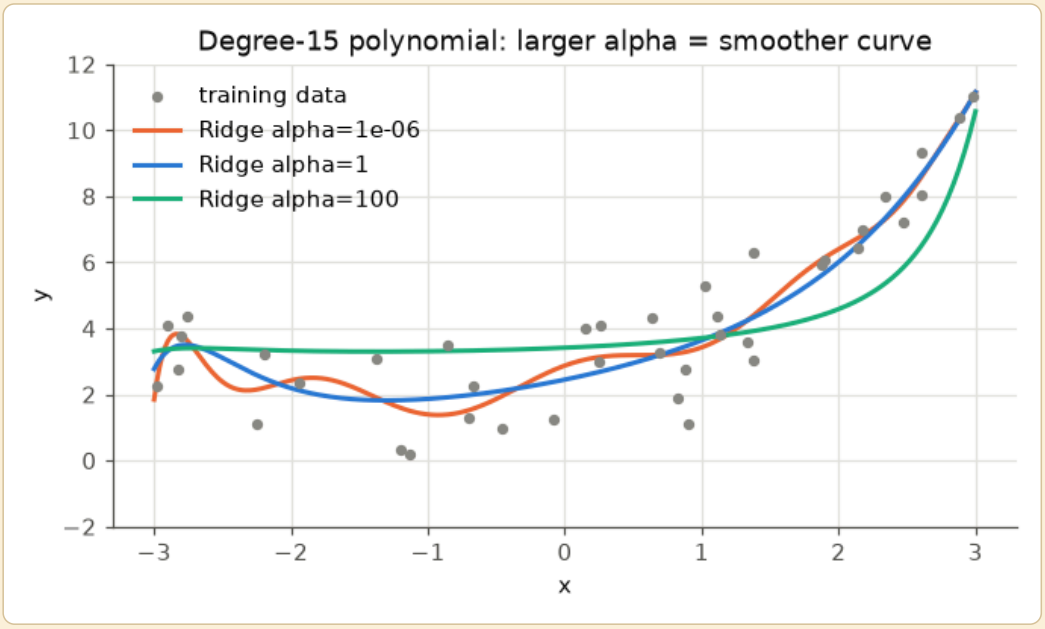
bmi duplicated -> OLS: [ 434.3 -389.1]  Ridge(1): [31.5 13.5]  Lasso(1): [44.2  0. ]

model          test MSE    alpha l1_ratio non-zero
OLS                4112      nan      nan       65
RidgeCV            3291   59.636      nan       65
LassoCV            3113    4.256      nan       11
ElasticNetCV       3178    0.754      0.8       41
```



What the numbers show:

- **Sparsity:** as Lasso's α grows, coefficients drop out: 9 → 7 → 5 → 3 non-zero. The last survivors (`bmi`, `s5`) are the strongest predictors.
- **Correlated features:** with `bmi` duplicated, plain least squares produces absurd, opposite-signed weights (+434, −389) that nearly cancel. Ridge shares the weight (31.5 + 13.5 ≈ 45); Lasso keeps one (44.2) and zeroes the other.
- **Generalisation:** with 65 polynomial features and ~330 training rows, unregularized OLS overfits (test MSE 4112). Ridge, Lasso and ElasticNet with α chosen by cross-validation all do much better (≈ 3100–3300); Lasso keeps only 11 of the 65 features. (Honest footnote: on this split plain OLS on the *original* 10 features scores about 2900 — regularization rescues an over-expanded model, but it doesn't make needless features useful.)



Trade-offs

	Ridge	Lasso	ElasticNet
Feature selection	No	Yes	Yes (less aggressive)
Correlated features	Shares weight — stable	Picks one arbitrarily — unstable	Groups them — stable
Closed form	Yes	No (coordinate descent)	No
When p > n	Works	Selects at most n features	Works
Good default when	Many small effects	Few features truly matter	Correlated groups + want sparsity

When not to use it

Regularization matters little when you have plenty of rows per feature (diabetes with its raw 10 features: OLS, Ridge, Lasso and ElasticNet all score within about 2% of each other, test MSE ≈ 2900–2950). Don't use Lasso's selected set as "the true causes" — with correlated features its choice can flip between resamples.

Common mistakes

- Regularizing unscaled features.
- Choosing α on the test set — use `RidgeCV` / `LassoCV` / `GridSearchCV`.
- Forgetting that C in `LogisticRegression` is the *inverse* of the strength.

Real-world examples

- Ridge is the classic fix for multicollinearity in econometrics; L2 "weight decay" is standard when training neural networks.
- L1-regularized logistic regression is used for sparse, high-dimensional text and click models (e.g. Google's FTRL-Proximal ad click model from 2013 used L1 to keep models sparse).
- In genomics (thousands of genes, few patients), Lasso and ElasticNet are standard tools for selecting predictive genes.

In the interview

30 seconds: *"Regularization adds a weight penalty to fight overfitting. Ridge (L2) shrinks all weights smoothly and handles correlated features; Lasso (L1) zeroes some weights, doing feature selection; ElasticNet mixes both. Scale features first and tune α with cross-validation."*

- *"Why does Lasso give zeros?"* — The L1 penalty's constant slope pushes small weights all the way to 0; geometrically, the diamond-shaped constraint has corners on the axes.
- *"Ridge vs Lasso with two identical features?"* — Ridge splits the weight equally; Lasso picks one.
- *"Effect of regularization on bias and variance?"* — More regularization → more bias, less variance.

Self-check

1. Large α in Ridge does what? → Shrinks all weights toward 0 (more bias, less variance).
2. Which penalty can do feature selection? → L1 (Lasso, ElasticNet).
3. In `LogisticRegression`, does C = 0.01 mean strong or weak regularization? → Strong.
4. Why scale before regularizing? → The penalty is on raw weights, whose size depends on feature units.

TL;DR

Penalize big weights: L2 shrinks, L1 selects, ElasticNet does both. Scale first, tune α (or C) by cross-validation.

5.8 Logistic regression

What it is

Logistic regression is a linear model for **binary classification**. It computes the same weighted sum z = w·x + b (called the **logit**), then squashes it into a probability with the **sigmoid** (logistic) function:

p = P(y = 1 | x) = σ(z) = 1 / (1 + e^−z)

Despite the name, it's a classifier. The name comes from the logistic function it uses.

How it works

1. Sigmoid. σ maps any real number to (0, 1): σ(−2) = 0.119, σ(0) = 0.5, σ(2) = 0.881. Large positive z → close to 1; large negative → close to 0.

2. Log-odds interpretation. Inverting the sigmoid: z = ln(p / (1 − p)). The quantity p/(1 − p) is the **odds** (p = 0.8 → odds 4, "4 to 1"). So logistic regression is linear in the **log-odds**. Each weight w_j says: increasing x_j by 1 adds w_j to the log-odds, i.e. **multiplies the odds by e^w_j** (the **odds ratio**).

3. Decision rule and boundary. Predict class 1 when p ≥ threshold (default 0.5). Since p = 0.5 exactly when z = 0, the **decision boundary** is the set of points where w·x + b = 0 — a straight line (in 2-D) or a hyperplane. Changing the threshold shifts the boundary parallel to itself (Chapter 6 covers choosing thresholds).

4. Loss: log-loss (binary cross-entropy). Squared error would make the optimisation non-convex, so logistic regression uses:

Loss = −(1/m) Σ [y_i ln p_i + (1 − y_i) ln(1 − p_i)]

Only one term is active per example: −ln p if y = 1, −ln(1 − p) if y = 0.

True y	Predicted p	Loss	Reading
1	0.9	−ln 0.9 = 0.105	confident and right: tiny loss
1	0.5	−ln 0.5 = 0.693	unsure
1	0.1	−ln 0.1 = 2.303	confident and wrong: big loss
0	0.1	−ln 0.9 = 0.105	confident and right

It is the negative log-likelihood of the data under a Bernoulli model, so minimising it is maximum likelihood. It's convex in the weights, so gradient descent finds the global minimum. There's no closed-form solution, so it must be solved iteratively.

5. Gradient. Remarkably, the gradient has the same form as linear regression's:

∇Loss = (1/m) X^T(σ(Xθ) − y) — "features × (prediction − truth)".

Why, in three steps for one example ($p = \sigma(z)$, $z = \theta^T x$):

1. $dLoss/dp = -y/p + (1 - y)/(1 - p)$.
2. The sigmoid's derivative is $dp/dz = p(1 - p)$.
3. Multiply: $dLoss/dz = -y(1 - p) + (1 - y)p = \mathbf{p - y}$, and $dz/d\theta_j = x_j$, so $dLoss/d\theta_j = (p - y)x_j$. Averaging over examples gives the formula. (No factor 2 here, because there is no square.)

```python
# code/logreg_from_scratch.py (key parts)
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def fit_logreg_gd(X, y, lr=0.5, epochs=5000):
    Xb = np.c_[np.ones(len(X)), X]
    theta = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        p = sigmoid(Xb @ theta)
        theta -= lr * Xb.T @ (p - y) / len(y)            # gradient of mean log-loss
    return theta

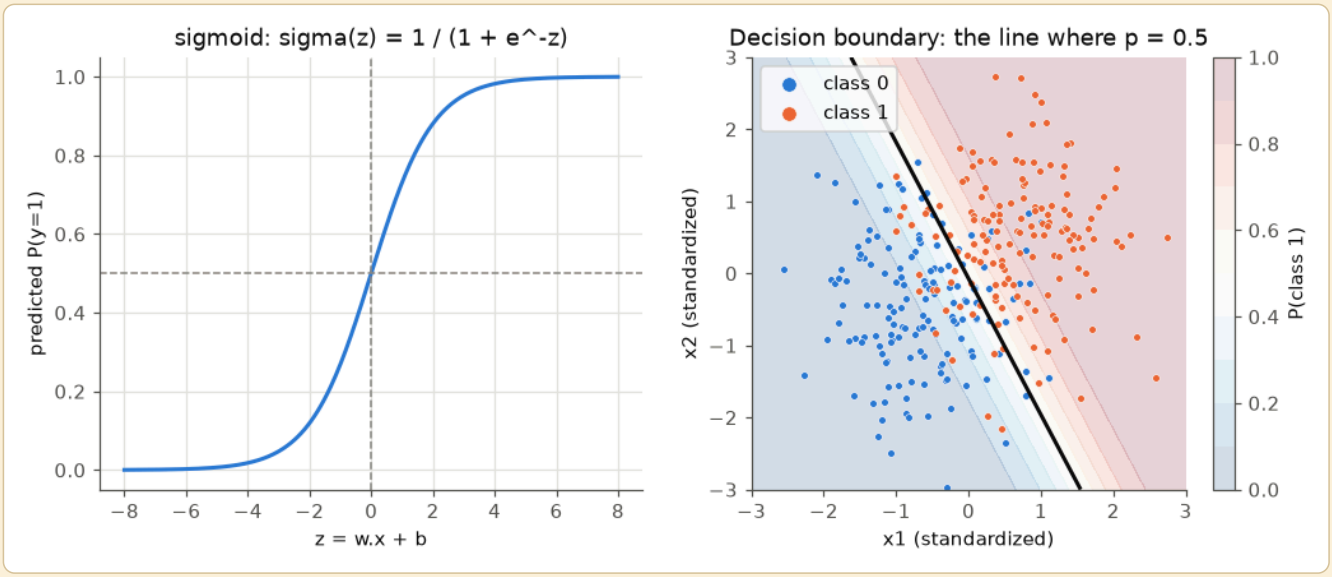
X, y = make_blobs(n_samples=400, centers=[[-1, -0.5], [1, 0.8]], cluster_std=1.1, random_state=0)
# ... stratified split, StandardScaler fitted on train
theta = fit_logreg_gd(X_tr, y_tr)
sk = LogisticRegression(C=np.inf).fit(X_tr, y_tr)        # C=inf -> no penalty (sklearn >= 1.8)
```

Output:

```
sigmoid(-2) = 0.119
sigmoid(+0) = 0.500
sigmoid(+2) = 0.881
y=1, p=0.9: loss = 0.105
y=1, p=0.5: loss = 0.693
y=1, p=0.1: loss = 2.303
y=0, p=0.1: loss = 0.105

from scratch [b, w1, w2]: [0.092 2.454 1.297]
sklearn      [b, w1, w2]: [0.092 2.454 1.298]
test log-loss scratch 0.4048 | sklearn 0.4048
test accuracy scratch 0.850 | sklearn 0.850
w1 = 2.454: +1 std in x1 multiplies the odds of class 1 by e^w1 = 11.64
```

Our 10-line gradient descent matches scikit-learn's L-BFGS solver to about three decimals.



The probability changes smoothly across the boundary; points near the line get probabilities near 0.5. That's the model being honestly uncertain about overlapping classes.

scikit-learn details (version 1.8+). `LogisticRegression` is L2-regularized by default with `C=1.0`. The old `penalty=` argument is deprecated: use `C=np.inf` for no penalty, `l1_ratio=1` (with `solver="saga"` or `"liblinear"`) for L1, and a value between 0 and 1 for ElasticNet. The default solver is `lbfgs`.

Trade-offs

Strengths	Weaknesses
Fast to train and serve, tiny model	Linear boundary only (needs feature engineering or kernels for curves)
Outputs probabilities that are often reasonably calibrated	Sensitive to correlated features and outliers in X
Coefficients interpretable as odds ratios	On perfectly separable data, unregularized weights grow without bound
Convex: one global optimum	Usually beaten by gradient boosting on complex tabular data

When not to use it

Strongly non-linear boundaries without good features (use trees/boosting or neural nets); very high-cardinality interactions you can't enumerate.

Common mistakes

- Using `LinearRegression` for a 0/1 target — predictions fall outside [0, 1] and the fit is dominated by easy points.
- Forgetting that the default is regularized (C = 1) — compare coefficients across models only with the same C and scaling.
- Assuming 0.5 is the right threshold for imbalanced data.

Real-world examples

- **Credit scoring** scorecards are traditionally logistic regression, because regulators and loan officers need to read the coefficients.
- **Ad click-through-rate** prediction at Google and Facebook used large-scale logistic regression (Facebook's 2014 paper fed boosted-tree features into a logistic regression).
- Medical risk scores and the final layer of every neural-network binary classifier (a sigmoid on a linear layer).

In the interview

30 seconds: *"Logistic regression computes a linear score $z = w \cdot x + b$, turns it into a probability with the sigmoid, and is trained by minimising log-loss — convex, so gradient descent finds the global optimum. The decision boundary is the hyperplane $w \cdot x + b = 0$. Each coefficient is a change in log-odds, so e^{w} is an odds ratio."*

- *"Why not MSE for classification?"* — With a sigmoid, MSE is non-convex and gives weak gradients for confidently wrong predictions; log-loss is convex and punishes them hard.
- *"What happens on perfectly separable data?"* — The unregularized loss keeps decreasing as weights grow, so they diverge; regularization keeps them finite.
- *"Is logistic regression's output calibrated?"* — Often roughly, because log-loss is a proper scoring rule, but check with a reliability curve (Chapter 6).

Self-check

1. $\sigma(0)$? → 0.5.
2. Loss for y = 1, p = 0.25? → $-\ln 0.25 = 1.386$.
3. Coefficient w = 0.7: effect on odds? → Multiplied by $e^{0.7} \approx 2.01$ per unit increase.
4. What does the decision boundary look like in 2-D? → A straight line.

TL;DR

Linear score → sigmoid → probability; train with log-loss (convex, gradient = $X^T(p - y)/m$); boundary is $w \cdot x + b = 0$; coefficients are log-odds.

5.9 Multiclass classification: one-vs-rest and softmax

What it is

Two ways to extend logistic regression to K > 2 classes:

1. **One-vs-rest (OvR)** — train K separate binary classifiers ("class k vs everything else"); predict the class whose classifier is most confident.
2. **Softmax (multinomial) regression** — one model with a weight vector per class, producing K scores $z_k = w_k \cdot x + b_k$, turned into probabilities that sum to 1 by the **softmax** function:

$$P(y = k \mid x) = e^{z_k} / \Sigma_j e^{z_j}$$

Worked example: scores (2, 1, 0.1) → $e^2 = 7.39$, $e^1 = 2.72$, $e^{0.1} = 1.11$, total 11.21 → probabilities **(0.659, 0.242, 0.099)**. The largest score wins, and the probabilities sum to 1.

The loss is **cross-entropy**: $-(1/m)\ \Sigma_i \ln P(y_i \mid x_i)$ — the negative log-probability of the correct class. With K = 2 it reduces exactly to binary logistic regression. Its gradient again has the form $X^T(P - Y)/m$, with Y the one-hot labels.

```python
# code/multiclass.py (key parts)
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)                # subtract row max: avoids overflow
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)

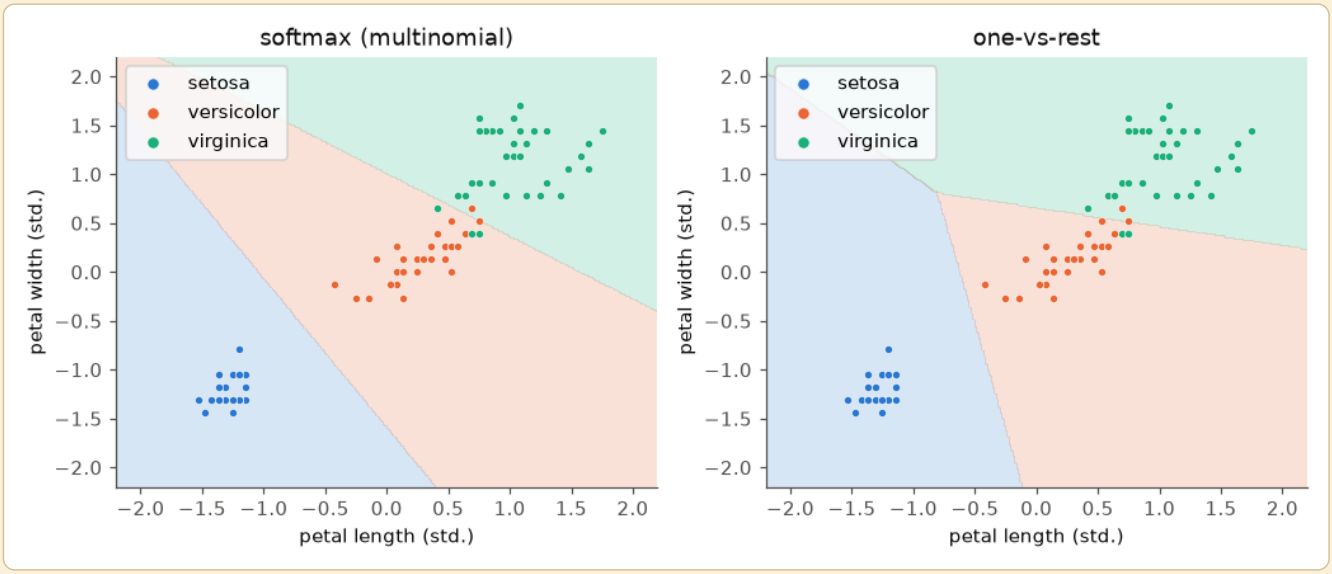
def fit_softmax_gd(X, y, k, lr=0.5, epochs=3000, l2=0.0):
    Xb = np.c_[np.ones(len(X)), X]
    W = np.zeros((Xb.shape[1], k)); Y = np.eye(k)[y]      # one-hot targets
    for _ in range(epochs):
        grad = Xb.T @ (softmax(Xb @ W) - Y) / len(y)
        grad[1:] += l2 * W[1:]
        W -= lr * grad
    return W

softmax_sk = LogisticRegression(C=10).fit(X_tr, y_tr)                  # multinomial by default
ovr = OneVsRestClassifier(LogisticRegression(C=10)).fit(X_tr, y_tr)     # 3 binary models
```

Output:

```
softmax([2, 1, 0.1]) = [[0.659 0.242 0.099]]  sum = 1.0

test accuracy  softmax(sklearn) 0.978 | OvR 0.978 | softmax(scratch) 0.978
one flower, P(setosa, versicolor, virginica):
  softmax sklearn : [[0.    0.001 0.999]]
  softmax scratch : [0.    0.001 0.999]
  OvR raw scores  : [0.    0.175 0.998] (sum 1.174) -> normalized [[0.    0.149 0.851]]
sklearn softmax coef_ shape: (3, 2) (one weight vector per class)
```



Both reach 97.8% accuracy here, but their probabilities differ: the OvR classifiers are trained independently, so their raw scores don't sum to 1 (1.174) and scikit-learn simply rescales them, giving 0.851 for virginica where softmax says 0.999. The regions differ too — OvR's boundaries come from three separate problems.

Trade-offs

	One-vs-rest	Softmax
Models	K binary models	One joint model
Probabilities	Not naturally normalised	Sum to 1, jointly trained — better calibrated
Parallel training / per-class tuning	Easy	No
Use with	Any binary classifier (e.g. SVMs); multi-label problems	Mutually exclusive classes; neural-network output layers

Also: **one-vs-one** trains K(K−1)/2 pairwise classifiers (used internally by `SVC`). For **multi-label** problems (an item can have several labels at once, e.g. photo tags), use independent sigmoids (OvR), not softmax.

Current scikit-learn (1.9 here) has no `multi_class` argument any more: `LogisticRegression` fits the multinomial (softmax) model for K > 2, and you wrap it in `OneVsRestClassifier` if you want OvR.

In the interview

30 seconds: *"For K classes you can train K one-vs-rest binary models, or one softmax model that outputs K scores and normalises them with e^z / Σe^z, trained with cross-entropy. Softmax gives coherent probabilities for mutually exclusive classes and is the output layer of*

neural classifiers; OvR works with any binary model and suits multi-label problems."

- *"Why subtract the max in softmax?"* — Numerical stability: e^1000 overflows, and subtracting a constant from all scores doesn't change the result.
- *"Softmax with 2 classes?"* — Equivalent to logistic regression on the score difference.

Self-check

1. softmax(0, 0, 0)? → (1/3, 1/3, 1/3).
2. Multi-label tagging: softmax or sigmoids? → Independent sigmoids.
3. How many classifiers does OvR train for 10 classes? → 10 (one-vs-one: 45).

TL;DR

OvR = K binary models; softmax = one model with normalised probabilities and cross-entropy loss. Use softmax for exclusive classes, sigmoids for multi-label.

5.10 Interpreting coefficients

What it is

A linear model's coefficients are its explanation. Reading them correctly depends on units, scaling and correlations.

How it works

- **Raw units:** w_j = change in $\hat{y}$ per 1-unit increase in x_j, **holding all other features fixed**. Change the unit (m² → km²) and the number changes by a factor of a million — the model doesn't.
- **Standardized features:** w_j = change in $\hat{y}$ per 1 standard deviation of x_j. Now magnitudes are comparable across features — a rough importance ranking.
- **Logistic regression:** w_j is a change in log-odds; e^{w_j} is the **odds ratio**. $e^w > 1$ raises the odds, < 1 lowers them.
- **Intercept:** the prediction when all features are 0 (with standardized features: at the average feature values).

```
# code/interpret_coefficients.py (key parts)
price = 50 + 3.0 * size - 1.5 * age + rng.normal(0, 20, n)     # $1000s, size in m², age in years
LinearRegression().fit(np.c_[size, age], price)
LinearRegression().fit(np.c_[size / 1e6, age], price)            # size in km²
make_pipeline(StandardScaler(), LinearRegression()).fit(np.c_[size, age], price)

clf = make_pipeline(StandardScaler(), LogisticRegression(C=0.1, max_iter=5000)).fit(cancer.data, cancer.target)

x2 = x1 + rng.normal(0, 0.05, 300)                               # near-duplicate feature
# fit on 3 bootstrap resamples
```

Output:

```
raw units      : price = 52.6 + 2.98*size_m2 -1.49*age_years
size in km^2   : coef for size = 2984148  (same model, different units -> different number)
standardized   : +1 std of size -> +137.6, +1 std of age -> -20.8  (comparable importance)

breast cancer (target 1 = benign), standardized features, C=0.1:
  worst texture            coef -0.60  odds ratio e^coef = 0.55
  worst radius             coef -0.54  odds ratio e^coef = 0.58
  worst concave points     coef -0.53  odds ratio e^coef = 0.59
  fractal dimension error  coef +0.27  odds ratio e^coef = 1.31
  compactness error        coef +0.27  odds ratio e^coef = 1.31

bootstrap coefficients for two near-duplicate features (true: 1.0 and 1.0):
  resample 0: w1 = +1.07, w2 = +0.95, sum = 2.02
  resample 1: w1 = +2.91, w2 = -0.92, sum = 2.00
  resample 2: w1 = -1.03, w2 = +3.04, sum = 2.01
```

Reading it:

- Each extra m² adds about $2,980 and each year of age subtracts about $1,490 (the price unit is $1000s). In raw units, age's coefficient looks half as big as size's; standardized, size matters about 6.6× more (137.6 vs 20.8), because sizes vary much more than ages.
- In the cancer model, one standard deviation more "worst texture" multiplies the odds of *benign* by 0.55 (about halves them).
- With near-duplicate features, individual coefficients swing wildly between resamples (even flipping sign), while their **sum** stays stable at ≈ 2. Predictions are fine; the individual numbers are meaningless. Diagnose with correlations or the **variance inflation factor (VIF)**; fix by dropping/combining features or using Ridge.

Common mistakes

- Comparing raw coefficients across features with different units.
- Reading coefficients causally from observational data.
- Interpreting one coefficient of a correlated group in isolation.

- Forgetting that regularization shrinks coefficients (they are biased toward 0).

In the interview

30 seconds: *"A coefficient is the change in prediction per unit of that feature, holding the others constant. Standardize features to compare magnitudes; in logistic regression exponentiate to get odds ratios. Coefficients are unstable under multicollinearity and are associations, not causal effects."*

Self-check

1. Logistic coefficient −0.69: effect on odds? → ×e^−0.69 ≈ 0.5, halves the odds.
2. Why are coefficients of correlated features unstable? → Many weight combinations give nearly the same predictions; small data changes flip between them.
3. How to make coefficient magnitudes comparable? → Standardize the features.

TL;DR

Coefficients = effect per unit (or per std if standardized), holding others fixed; exponentiate logistic coefficients for odds ratios; don't trust individual coefficients of correlated features or read them causally.

Chapter TL;DR

- **Workflow:** data → clean → encode (one-hot for nominal) → split (80/20) → fit → evaluate against a baseline → iterate. MSE only means something relative to the target's scale and a baseline.
- **Linear regression:** $\hat{y} = w \cdot x + b$, minimise MSE (convex). **Normal equation** $\theta = (X^T X)^{-1} X^T y$ (use a solver, not an explicit inverse) — exact, $O(d^3)$, no scaling needed. **Gradient descent** $\theta \leftarrow \theta - \eta(2/m)X^T(X\theta - y)$ — scalable and general, needs a learning rate and scaled features.
- **GD variants:** batch (smooth, costly), stochastic (noisy, needs a decaying learning rate), mini-batch (default). Too-large η diverges (for MSE, $\eta < 2/\lambda_{max}$). Unscaled features → elongated loss bowl → many more iterations.
- **Assumptions:** linearity, independent errors, constant variance, roughly normal errors, no strong multicollinearity. Check residual plots (U = missing curvature, funnel = heteroscedasticity).
- **Polynomial regression:** linear model on expanded features; degree controls under/overfitting.
- **Regularization:** Ridge (L2, shrink, stable with correlated features), Lasso (L1, exact zeros, feature selection), ElasticNet (both). Scale first; tune α (or $C = 1/\text{strength}$) by CV.
- **Logistic regression:** $p = \sigma(w \cdot x + b)$, log-loss (convex, gradient $X^T(p - y)/m$), boundary $w \cdot x + b = 0$, coefficients are log-odds (e^w = odds ratio). In sklearn ≥ 1.8, `C=np.inf` means no penalty.
- **Multiclass:** OvR (K binary models) vs softmax ($e^{z_k}/\Sigma e^{z_j}$ + cross-entropy; coherent probabilities).
- **Coefficients:** per-unit effects holding others fixed; standardize to compare; unstable under multicollinearity; associations, not causes.

Chapter 6 — Evaluation and Tuning

Training a model is the easy part. The hard questions come after: *Is it any good? Good at what? Compared to what? Will it still be good on next month's data? Could different settings make it better?* This chapter gives you the tools to answer them honestly — the right metric for the job, reliable ways to estimate performance, a clear picture of over- and underfitting, and systematic hyperparameter tuning.

What you'll learn

1. Why every number needs a baseline, and how offline metrics relate to business metrics.
2. Regression metrics: MAE, MSE, RMSE, R^2, percentile and per-cohort errors.
3. Classification metrics: confusion matrix, accuracy, precision, recall, F1, ROC-AUC, PR-AUC — with a worked example — and how to choose a threshold.
4. Ranking metrics: precision@k, recall@k, MAP, MRR, NDCG.
5. Calibration: when a probability of 0.8 should really mean 80%.
6. Cross-validation: k-fold, stratified, time-series, group, and nested.
7. The bias–variance trade-off, learning curves and validation curves, and what to do about over- and underfitting.
8. Hyperparameter tuning: grid, random, successive halving, Bayesian optimisation (Optuna), and early stopping.

All code is in `code/` and runs with the shared environment (scikit-learn 1.9). Outputs shown are real.

6.1 Baselines, and offline vs business metrics

What it is

An **evaluation metric** is a number that summarises how well predictions match reality. A **baseline** is the score of a trivially simple predictor. An **offline metric** is computed on held-out historical data; an **online** or **business metric** is what the product actually cares about (revenue, retention, fraud losses, delivery promises kept), measured on live traffic.

Why it matters

A metric on its own is meaningless. "MAE of 6 minutes" or "PR-AUC 0.41" says nothing until you know what a naive approach scores and what the current production system scores. If predicting "same as last week" already gives an MAE of 8 minutes, a model at 6 is a real gain; if the naive approach gives 6.2, the model is an expensive way to save 12 seconds.

How it works — baselines to always compute

Task	Naive baseline	scikit-learn
Classification	Majority class; or random guessing at the class rates	`DummyClassifier(strategy="most_frequent" / "prior" / "stratified")`
Regression	Mean or median of the training target	`DummyRegressor(strategy="mean" / "median")`
Time series	Last value; same hour last week (seasonal naive)	a one-liner in pandas (`shift`)
Ranking / recommendation	Most popular items	a `value_counts()`
Any	The model currently in production; a simple rule or a linear model	—

Offline metric vs business metric

Offline metrics are **proxies**. They're cheap, fast and repeatable, so you use them to compare candidates, debug, and decide what deserves a live test. But they can disagree with the business outcome:

- The label is a proxy (clicks ≠ satisfaction; a chargeback ≠ all fraud).
- The test set no longer resembles live traffic (drift, new users, new items).
- Serving differs from training (feature delays, timeouts, fallbacks).
- The metric ignores costs the business cares about (latency, review-team workload, fairness across groups).

So the pattern in practice is two gates (the production view of metric choice is in Chapter 16, section 16.7): **offline evaluation** to shortlist, then an **online experiment** (A/B test, covered in Chapter 16) to confirm business impact. Choose the offline metric so that improving it is *likely* to improve the business metric, and report it together with the dataset, base rate and threshold used.

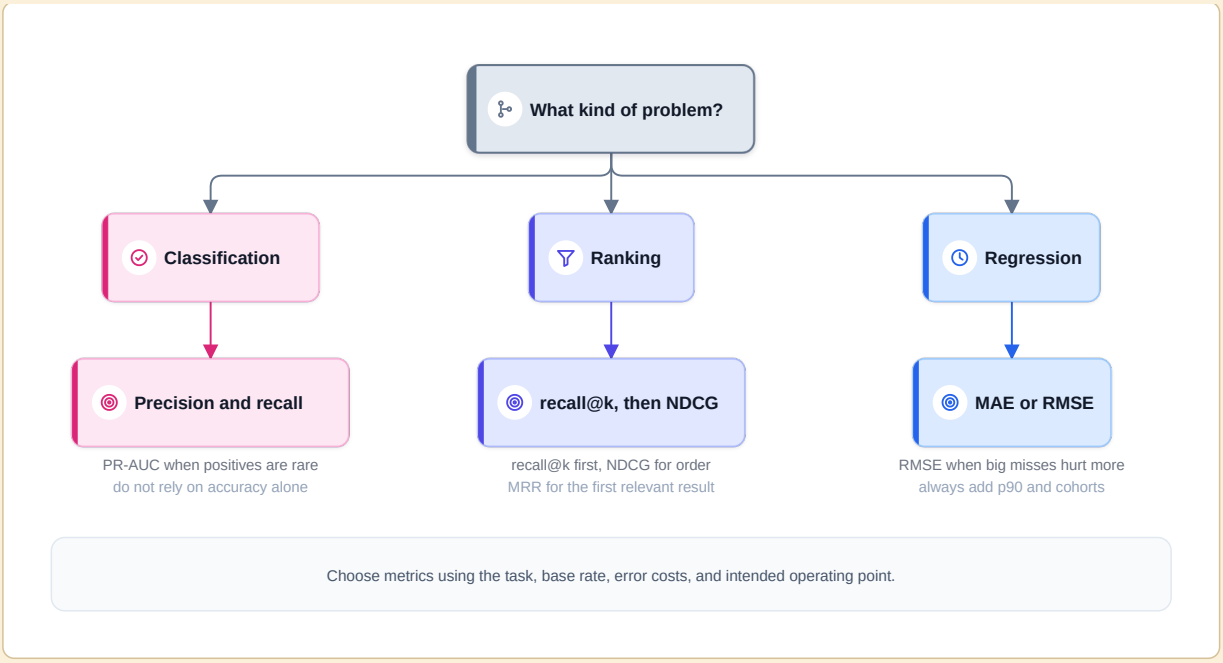


Figure — mapping problem types to metric families (course diagram)

In the interview

30 seconds: *"I'd pick the offline metric from the task and the cost of each kind of error — precision/recall and PR-AUC for rare-event classification, recall@k and NDCG for ranking, MAE or RMSE plus tail and per-segment errors for regression — check calibration separately if scores are used as probabilities, and always compare against a naive baseline and the production model before going to an A/B test."*

- *"Offline metric improved, A/B test flat — why?"* — The offline metric is a proxy that drifted from the goal: proxy labels, a stale test set, or training/serving differences. Also check the experiment's statistical power and logging before blaming the metric.

Self-check

1. Your churn model has PR-AUC 0.41. Is that good? → Unknown until you compare: a no-skill model scores the churn rate (e.g. 0.12), and the production model scores something too.
2. Name two reasons an offline gain may not show up online. → Proxy labels, drift between test set and live traffic, training/serving differences, ignored costs (latency, workload).

TL;DR

No number means anything without a baseline and the production model next to it. Offline metrics shortlist; online experiments decide.

6.2 Regression metrics

What it is

Regression metrics summarise the **errors** $e_i = \hat{y}_i - y_i$ between predictions and true values.

Metric	Formula	Units	Reads as
MAE (mean absolute error)	$(1/n)\ \Sigma\ \lvert e_i \rvert$	same as y	typical size of an error
MSE (mean squared error)	$(1/n)\ \Sigma\ e_i^2$	y^2	average squared error; the usual training loss
RMSE (root MSE)	$\sqrt{\text{MSE}}$	same as y	like MAE but big errors weigh more
R^2 (coefficient of determination)	$1 - \Sigma e_i^2 \ /\ \Sigma(y_i - \bar{y})^2$	none	share of the target's variance explained; 1 = perfect, 0 = no better than predicting the mean, < 0 = worse than the mean
MAPE (mean absolute % error)	$(100/n)\ \Sigma\ \lvert e_i / y_i \rvert$	%	relative error; breaks when y is near 0
Percentile error (e.g. p90)	90th percentile of $\lvert e_i \rvert$	same as y	90% of predictions are at least this good

How it works — a worked example

Delivery times (minutes): $y = (30, 25, 40, 35, 50)$, predictions $\hat{y} = (28, 27, 35, 35, 60)$.

1. Errors $e = (-2, 2, -5, 0, 10)$.
2. MAE $= (2 + 2 + 5 + 0 + 10)/5 =$ **3.8** min.
3. MSE $= (4 + 4 + 25 + 0 + 100)/5 =$ **26.6** min²; RMSE $= \sqrt{26.6} =$ **5.16** min.

4. R^2: $\bar{y} = 36$, $\Sigma(y - \bar{y})^2 = 36 + 121 + 16 + 1 + 196 = 370$, $\Sigma e^2 = 133 \rightarrow R^2 = 1 - 133/370 = \mathbf{0.641}$.

```
# code/regression_metrics.py (key parts)
from sklearn.metrics import mean_absolute_error, mean_squared_error, root_mean_squared_error, r2_score
y, yhat = np.array([30, 25, 40, 35, 50]), np.array([28, 27, 35, 35, 60])
print(mean_absolute_error(y, yhat), mean_squared_error(y, yhat), root_mean_squared_error(y, yhat), r2_score(y, yhat))

base = np.full(10, 10.0); one_big = base.copy(); one_big[0] = 40      # one miss grows from 10 to 40 min
# ... then diabetes data: Ridge vs mean/median DummyRegressor, p50/p90 error, error per cohort
```

Output:

```
errors: [-2  2 -5  0 10]
MAE  = mean(|e|)      = 3.80   sklearn 3.80
MSE  = mean(e^2)      = 26.60  sklearn 26.60
RMSE = sqrt(MSE)      = 5.16   sklearn 5.16
R^2  = 1 - 133/370 = 0.641  sklearn 0.641
ten 10-min errors        MAE  10.0   RMSE  10.0
one of them is 40 min    MAE  13.0   RMSE  15.8

model                     MAE   RMSE    R^2
mean baseline            59.5   71.4 -0.000
median baseline          59.1   72.5 -0.029
Ridge                    44.3   55.2  0.402
p50 abs error 40.6 | p90 abs error 90.6
cohort A: n= 74  MAE 47.8
cohort B: n= 59  MAE 39.9
```

MAE vs RMSE — how much should a big miss count? Because errors are squared, a 40-minute miss weighs 16× a 10-minute miss inside MSE (1600 vs 100), whereas inside MAE it weighs just 4× (40 vs 10). In the output, turning one of ten 10-minute errors into a 40-minute error lifts MAE by 30% (10 → 13) but RMSE by 58% (10 → 15.8). (The final RMSE is not 16× larger — the square root and averaging dampen it.) Choose:

- **RMSE** when large misses are disproportionately bad (a very late delivery loses the customer).
- **MAE** when cost grows linearly with error size, or when outliers in the labels shouldn't dominate.
- Also note: the constant that minimises MSE is the **mean**; the one that minimises MAE is the **median**.

Averages hide things — add two views.

1. **Tail percentiles.** The median error here is 41 but the p90 is 91: one prediction in ten is off by more than 90 points. Products often promise on the tail (e.g. "delivered within the promised window" for 90%+ of orders).
2. **Cohorts.** Report error per segment the business cares about (city, device, customer type, new vs mature market). Here cohort A's MAE is 20% worse than cohort B's; a model can look fine on average while failing a new region entirely.

The mean baseline has $R^2 \approx 0$ by construction (slightly negative on a test set, because the training mean isn't exactly the test mean). The Ridge model explains 40% of the variance and cuts MAE by 26% — a real but modest improvement, which is the honest way to report it.

Common mistakes

- Reporting R^2 alone — it depends on the target's variance in the test set, so it isn't comparable across datasets.
- Using MAPE with targets near zero (division blows up) or asymmetrically (it penalises over-prediction more than under-prediction for positive targets).
- Comparing RMSE across targets in different units.

In the interview

30 seconds: *"MAE is the typical error in the target's units; RMSE squares errors first, so big misses count more — use it when large misses are disproportionately costly. R^2 compares against predicting the mean. I'd add a tail percentile like p90 and per-segment errors, because averages hide the failures that matter."*

- *"Why might RMSE be much larger than MAE?"* — A few large errors: RMSE is inflated by the tail, MAE isn't. The gap itself is a signal of heavy-tailed errors.

Self-check

1. MAE of errors (1, −1, 4)? → 2. RMSE? → √(18/3) = 2.45.
2. Which constant minimises MAE? → The median.
3. What does $R^2 = -0.2$ mean? → Worse than always predicting the mean.

TL;DR

MAE = typical error; RMSE = punishes big misses; R^2 = variance explained vs the mean. Add p90 error and per-cohort errors, and compare with a baseline.

6.3 Classification metrics

What it is

For a binary classifier, every prediction lands in one of four cells of the **confusion matrix**:

	Predicted positive	**Predicted negative**
Actually positive	**TP** — true positive (caught)	**FN** — false negative (missed)
Actually negative	**FP** — false positive (false alarm)	**TN** — true negative

All the common metrics are ratios of these four counts.

Metric	**Formula**	**Question it answers**
Accuracy	(TP + TN) / all	What fraction of predictions are right?
Precision	TP / (TP + FP)	Of the items I flagged, how many were really positive?
Recall (sensitivity, true-positive rate)	TP / (TP + FN)	Of the real positives, how many did I catch?
F1	2·P·R / (P + R)	The **harmonic mean** of precision and recall — high only if both are high
Specificity (true-negative rate)	TN / (TN + FP)	Of the real negatives, how many did I leave alone?
FPR (false-positive rate)	FP / (FP + TN) = 1 − specificity	What share of negatives did I wrongly flag?
Fβ	(1 + β²)·P·R / (β²·P + R)	F1 with recall weighted β times as much as precision (F2 favours recall, F0.5 precision)

How it works — a worked example

1,000 card transactions, 100 of them fraud. The model flags 80; 60 of those are fraud.

- TP = 60, FP = 80 − 60 = 20, FN = 100 − 60 = 40, TN = 900 − 20 = 880.
- Accuracy = (60 + 880)/1000 = **0.94**
- Precision = 60/80 = **0.75** — three in four alerts are real.
- Recall = 60/100 = **0.60** — 40% of fraud slips through.
- F1 = 2·0.75·0.6/(0.75 + 0.6) = 0.9/1.35 = **0.667**
- FPR = 20/900 = **0.022**

And the trap: a model that flags *nothing* scores accuracy 900/1000 = **0.90** with zero recall. At a 0.2% fraud rate, "never fraud" is 99.8% accurate and catches nothing. **Accuracy is misleading whenever classes are imbalanced.**

A smaller illustration, as in many tutorials: y_true = [0, 1, 0, 1], y_pred = [0, 1, 1, 1] → one false positive, no misses: accuracy 0.75, precision 2/3 = 0.667, recall 1.0, F1 0.8.

```
# code/classification_metrics.py (key parts)
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
                             roc_auc_score, average_precision_score, roc_curve, precision_recall_curve)
y    = np.r_[np.ones(100), np.zeros(900)].astype(int)
pred = np.r_[np.ones(60), np.zeros(40), np.ones(20), np.zeros(880)].astype(int)
tn, fp, fn, tp = confusion_matrix(y, pred).ravel()        # sklearn's layout: [[TN FP] [FN TP]]
```

Output:

```
tiny: acc 0.75  precision 0.667  recall 1.00  F1 0.80

confusion matrix [[TN FP] [FN TP]] =
[[880  20]
 [ 40  60]]
TP=60 FP=20 FN=40 TN=880
accuracy  (TP+TN)/N      = 0.940
precision TP/(TP+FP)     = 0.750
recall    TP/(TP+FN)     = 0.600
F1        2PR/(P+R)      = 0.667
FPR       FP/(FP+TN)     = 0.022   specificity = 0.978
always-'legit' baseline accuracy = 0.900
```

Thresholds: the model gives scores, you choose the operating point

Most classifiers output a **score** or probability; a **threshold** turns it into yes/no. Raising the threshold flags fewer items: recall goes down, and precision *usually* goes up (not necessarily at every step on a finite dataset). One model therefore supports many **operating points**, and you should quote precision and recall *at a stated threshold* ("90% precision at 70% recall"), or summarise across all thresholds with an area under a curve:

- **ROC curve** — recall (TPR) against FPR as the threshold sweeps. **ROC-AUC** = the probability that a randomly chosen positive gets a higher score than a randomly chosen negative (ties count half). 0.5 = random, 1.0 = perfect ranking.

- **Precision–recall (PR) curve** — precision against recall. **PR-AUC**, usually computed as **average precision (AP)**, summarises it. A no-skill model's PR-AUC equals the positive rate.

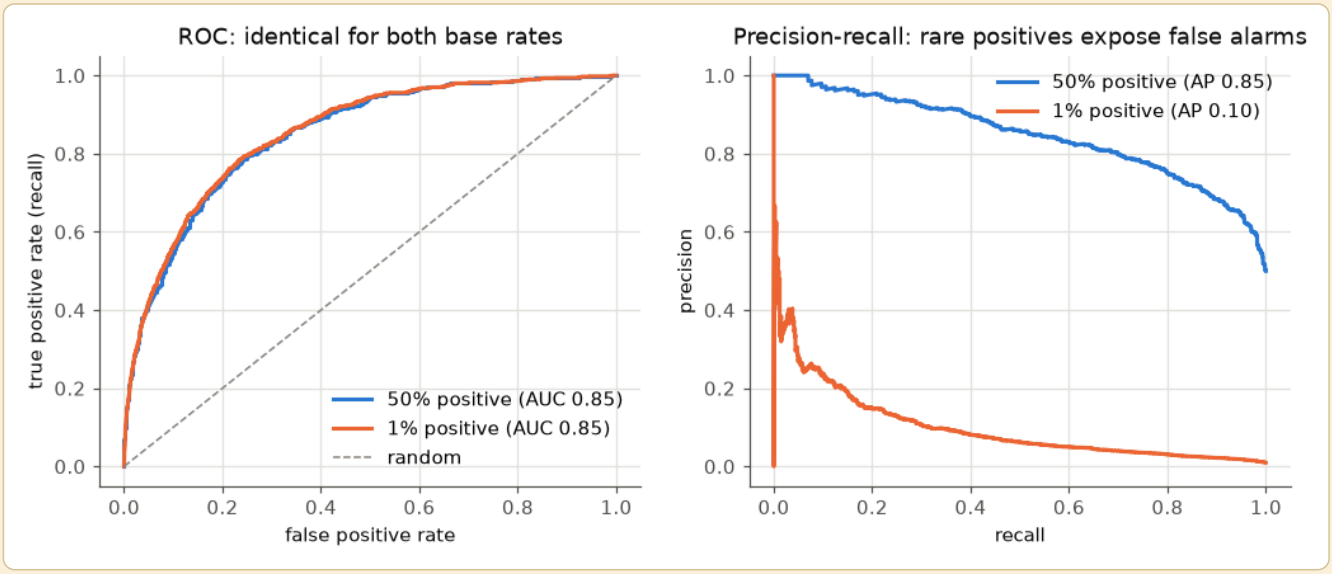
ROC-AUC by hand: positives score (0.9, 0.7, 0.4), negatives (0.8, 0.3, 0.2). Of the 9 positive–negative pairs, the positive wins 7 (0.9 beats all three; 0.7 beats 0.3 and 0.2; 0.4 beats 0.3 and 0.2) → AUC = 7/9 = **0.778**.

Why PR-AUC for rare positives? FPR divides false positives by *all* negatives. With 10 million legitimate transactions, even thousands of extra false alarms barely move FPR, so the ROC curve stays flattering even as analysts are buried under bogus alerts. Precision has false positives in its denominator directly, so the PR curve shows that pain.

```
pos = rng.normal(1.5, 1, 1000)                        # the SAME score model for positives
for n_neg in [1000, 99_000]:                          # 50% vs 1% positives
    neg = rng.normal(0, 1, n_neg)
    roc_auc_score(yy, ss), average_precision_score(yy, ss)
```

Output:

```
ROC-AUC by pair counting: 7/9 = 0.778  sklearn 0.778
positives  50.0%: ROC-AUC 0.850 | PR-AUC (avg precision) 0.846 | no-skill PR-AUC 0.500
positives   1.0%: ROC-AUC 0.853 | PR-AUC (avg precision) 0.099 | no-skill PR-AUC 0.010
```



Identical model, identical ROC-AUC (0.85) — but at a 1% base rate the average precision drops from 0.85 to 0.10: at 50% recall, precision is below 0.1 — more than nine false alarms for every real positive. Report the base rate with every precision number: the same model yields a different precision whenever the mix of positives in the traffic shifts.

Choosing the threshold by cost

The threshold is a business decision: it prices a missed positive against a false alarm, so someone who owns those costs (product, risk, operations) should own it. With **calibrated** probabilities (section 6.5) the cost-minimising rule is simple: flag when p · cost(FN) > (1 − p) · cost(FP), i.e. when

p > cost(FP) / (cost(FP) + cost(FN))

Example: a missed fraud costs $100, reviewing a false alarm costs $5 → flag when p > 5/105 = **0.048**.

```
# 20,000 transactions, 5% fraud, calibrated probabilities p
for t in [0.01, 0.025, 0.048, 0.1, 0.2, 0.5]:
    cost = 100 * missed + 5 * false_alarms
```

Output:

```
threshold  precision  recall  flagged   cost($)
     0.01      0.075   0.970    13006    63,180
    0.025      0.107   0.894     8361    47,935
    0.048      0.151   0.784     5195    43,655
      0.1      0.219   0.573     2611    52,890
      0.2      0.335   0.367     1095    66,940
      0.5      0.595   0.103      173    90,050
cheapest threshold: 0.048  (theory for calibrated p: flag if p > 5/(5+100) = 0.048)
```

The default 0.5 would cost twice as much as the best threshold. Mature fraud systems go one step further and use **two thresholds**: below the low one → approve automatically; above the high one → block; in between → send to a human reviewer. The review band's width is set by the team's capacity.

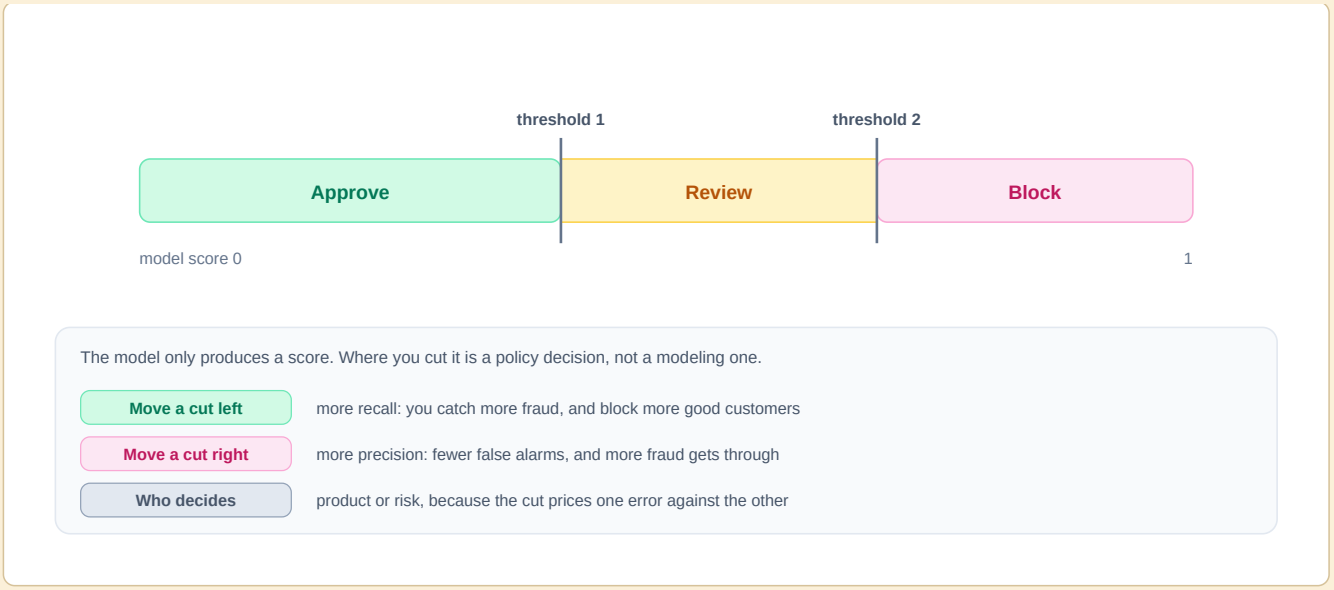


Figure — two thresholds on one score (course diagram)

Which metric when?

Situation	Use	Why
Balanced classes, equal error costs	Accuracy (plus the confusion matrix)	Simple and meaningful when neither class dominates
False positives are expensive (spam filter hiding real mail, blocking good customers)	Precision (at a recall floor)	Every alert must be trustworthy
Missing a positive is expensive (cancer screening, fraud, safety)	Recall (at a precision floor)	Catch as many as possible
Need one number balancing both, imbalanced data	F1 or Fβ	Harmonic mean punishes a weak side
Compare rankers across thresholds, moderate imbalance	ROC-AUC	Threshold-free, base-rate-insensitive ranking quality
Rare positives (fraud, ads, disease)	PR-AUC / average precision, plus precision & recall at the chosen threshold	Shows the false-alarm burden
Probabilities feed decisions (bids, expected cost)	Log-loss, Brier score, calibration plot	Rewards honest probabilities
Multiclass	Per-class P/R/F1 with **macro** (unweighted mean over classes) or **weighted** averages; `classification_report`	Macro exposes weak minority classes

Common mistakes

- Reporting accuracy on imbalanced data.
- Quoting precision without the threshold and base rate.
- Tuning the threshold on the test set (tune it on validation data).
- Comparing F1 across datasets with different base rates.

Real-world examples

- Spam filters (Gmail and others) favour very high precision — losing a real email is worse than seeing some spam.
- Medical screening tests favour recall (sensitivity) and follow up with a more precise second test.
- Card-fraud systems run on precision–recall trade-offs with approve/review/block bands, tuned to review-team capacity and fraud losses.

In the interview

30 seconds: *"Start from the confusion matrix. Precision is 'how many alerts are real', recall 'how many positives we caught'; the threshold trades one for the other, and the right point comes from the cost of each error. For rare positives I'd report PR-AUC plus precision and recall at the chosen threshold, not accuracy or only ROC-AUC, because the flood of easy negatives hides extra false alarms inside ROC's FPR denominator."*

- *"PR-AUC or ROC-AUC at a 0.2% base rate?"* — PR-AUC: it keeps precision — what reviewers and customers actually experience — on the axis.
- *"What threshold would you ship?"* — Not a pure modeling decision: present the precision–recall curve as a menu and let the cost owner pick; derive it from expected cost if probabilities are calibrated; for fraud, two thresholds with a review band.
- *"Why harmonic mean for F1?"* — It stays low if either precision or recall is low (P = 1, R = 0.01 → F1 ≈ 0.02, whereas the arithmetic mean would be 0.5).

Self-check

1. TP = 30, FP = 10, FN = 20: precision, recall, F1? → 0.75, 0.6, 0.667.
2. What does ROC-AUC = 0.8 mean? → A random positive outranks a random negative 80% of the time.
3. PR-AUC of a random model at 2% positives? → About 0.02.
4. FN costs $50, FP costs $10; threshold for calibrated p? → 10/60 = 0.167.

TL;DR

Confusion matrix → precision, recall, F1. Threshold sets the trade-off; choose it from error costs. ROC-AUC measures ranking; PR-AUC exposes false alarms when positives are rare. Accuracy only for balanced problems.

6.4 Ranking metrics (an introduction)

What it is

Search, recommendation and feeds return an **ordered list**. Ranking metrics reward putting relevant items **near the top**, usually looking only at the first **k** results. Relevance labels come from **judged sets** (trained raters score each query and result against a rubric) or **behavioural signals** (clicks, purchases), which are cheaper but biased by what was shown and where.

Metric	Definition	Use when
Precision@k	Relevant items in the top k ÷ k	The user sees exactly k slots
Recall@k	Relevant items in the top k ÷ all relevant items	Candidate generation: did the good items survive?
AP (average precision)	Mean of precision@i at each rank i holding a relevant item (divided by all relevant items)	Binary relevance, whole list
MAP	Mean of AP over queries	Standard IR benchmark metric
MRR (mean reciprocal rank)	Mean of 1/rank of the first relevant item (0 if none)	Finding one good answer fast (Q&A, navigation search)
NDCG@k	DCG@k ÷ ideal DCG@k, with DCG = Σ rel_i / $\log_2(i + 1)$	Graded relevance (perfect/good/fair/bad), position matters

How it works — worked example

Query A returns 6 results with graded relevance [3, 0, 2, 1, 0, 0] (3 = perfect … 0 = bad). The collection holds 4 relevant documents; one (grade 2) wasn't returned.

- Precision@3 = 2/3 = 0.667; recall@3 = 2/4 = 0.5.
- AP: relevant items sit at ranks 1, 3, 4 with precision 1/1, 2/3, 3/4 → (1 + 0.667 + 0.75)/4 = **0.604** (the missing one contributes 0).
- Reciprocal rank = 1/1 = 1.
- DCG@5 = $3/\log_2 2 + 0 + 2/\log_2 4 + 1/\log_2 5 + 0$ = 3 + 1 + 0.431 = **4.431**. The ideal order of all relevant documents [3, 2, 2, 1] gives 3 + 1.262 + 1 + 0.431 = 5.693, so **NDCG@5 = 0.778**.

```python
# code/ranking_metrics.py (key parts)
def average_precision(rels, n_relevant):
    hits, total = 0, 0.0
    for i, r in enumerate(rels, start=1):
        if r > 0:
            hits += 1
            total += hits / i                # precision at this relevant item's rank
    return total / n_relevant

def dcg(rels, k):
    rels = np.asarray(rels[:k], dtype=float)
    return np.sum(rels / np.log2(np.arange(2, len(rels) + 2)))
```

Output:

```
query A graded list: [3, 0, 2, 1, 0, 0] | relevant in collection: 4
  precision@3 = 0.667   recall@3 = 0.500
  precision@5 = 0.600   recall@5 = 0.750
  AP  = (1/1 + 2/3 + 3/4) / 4 = 0.604
  RR  = 1/rank of first relevant = 1.000
  DCG@5 = 4.431, ideal DCG@5 = 5.693, NDCG@5 = 0.778

query B list: [0, 0, 1, 1, 0, 0] AP = 0.417  RR = 0.333
MAP over A and B = 0.510   MRR = 0.667

sklearn ndcg_score@5 on the returned list only: 0.930  (ours, same ideal): 0.930
note: sklearn builds the ideal ordering from the returned items only; with the unreturned grade-2
document included in the ideal list, NDCG@5 drops to 0.778
```

Watch the definition: `sklearn.metrics.ndcg_score` computes the ideal ordering from the items you pass it, so a relevant document the system never retrieved doesn't lower the score (0.930 vs 0.778). Some libraries also use the exponential gain (2^rel − 1) instead of rel; always state which NDCG you report.

The funnel assigns the metric

Large recommenders narrow millions of items to a page in stages, and each stage gets its own metric: **candidate generation** (retrieval) is judged by **recall@k** — did the items the user would have chosen survive the cut from millions to, say, 500? — while the **ranking** stage is judged by ordering metrics like **NDCG** or MRR. If a later stage re-scores every candidate, the retrieval stage's internal order matters little; if later stages truncate, it matters.

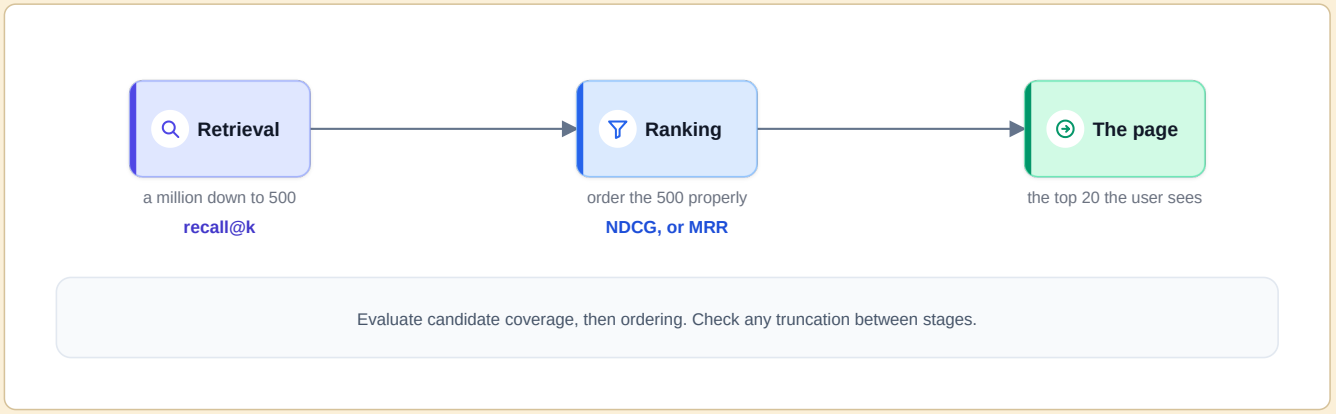


Figure — metrics per recommendation-funnel stage (course diagram)

Full treatment with case studies: Chapter 13 (recommenders) and Chapter 17 (search and feed ranking).

Common mistakes

- Comparing NDCG numbers computed with different gains (rel vs 2^rel − 1) or different "ideal" lists.
- Evaluating a ranking only on items the old system showed (exposure bias — see Chapter 13, sections 13.11–13.12).

In the interview

30 seconds: *"For a ranked list I'd use recall@k for the retrieval stage — did good items survive the cut — and NDCG@k or MRR for the ranking stage, because position matters. NDCG handles graded relevance with a log discount; MRR fits tasks with one right answer. Labels come from judged sets or clicks, and clicks carry position bias."*

- *"MAP or NDCG?"* — MAP for binary relevant/not-relevant labels; NDCG when relevance is graded (perfect/good/fair).

Self-check

1. First relevant result at rank 4: reciprocal rank? → 0.25.
2. Why is recall@k the retrieval-stage metric? → Retrieval's job is not to lose good candidates; ordering is fixed later.
3. What does the $\log_2(i + 1)$ do in DCG? → Discounts lower positions: rank 1 counts fully, rank 3 half, and so on.

TL;DR

Precision@k/recall@k for set quality, MAP for binary relevance over the list, MRR for "first good answer", NDCG for graded relevance with position discounts. Retrieval → recall@k; ranking → NDCG.

6.5 Calibration

What it is

A model is **calibrated** if its probabilities match reality: among all cases it scores 0.8, about 80% are positive. **Ranking well and being calibrated are different properties.** ROC-AUC only depends on the order of scores — you could square every score and AUC wouldn't change — so a model with ROC-AUC 0.95 can still say "0.9" for events that happen 50% of the time.

Why it matters

You need calibration whenever downstream logic does arithmetic with the score instead of merely sorting by it: ad auctions (bid = value × predicted click probability), expected-cost thresholds (section 6.3), risk scores shown to people, and combining models. After class re-weighting or negative downsampling, scores are systematically inflated and must be corrected (section 4.9 gives the closed-form fix for downsampling; Chapter 16, section 16.3 compares it with Platt and isotonic on a production-sized example).

How it works

1. **Measure** with a **reliability diagram**: bin predictions (e.g. 10 bins), plot the mean predicted probability against the observed fraction of positives per bin. Perfect calibration is the diagonal. Summaries: **Brier score** = mean((p − y)²) (lower is better) and **log-loss**; both are *proper scoring rules* — they are minimised by the true probabilities.
2. **Fix** by learning a mapping from raw score to probability on held-out data:

- **Platt scaling** (`method="sigmoid"`) — fit a logistic regression on the score: p = σ(a·s + b). Two parameters; works with little data; assumes an S-shaped distortion.
- **Isotonic regression** (`method="isotonic"`) — fit any non-decreasing step function. More flexible; needs more data (roughly ≥ 1,000 calibration examples) or it overfits.
- `CalibratedClassifierCV` does this with cross-validation so the calibrator never sees the base model's own training rows.

3. **Re-check** when the serving population changes — calibration drifts with the base rate.

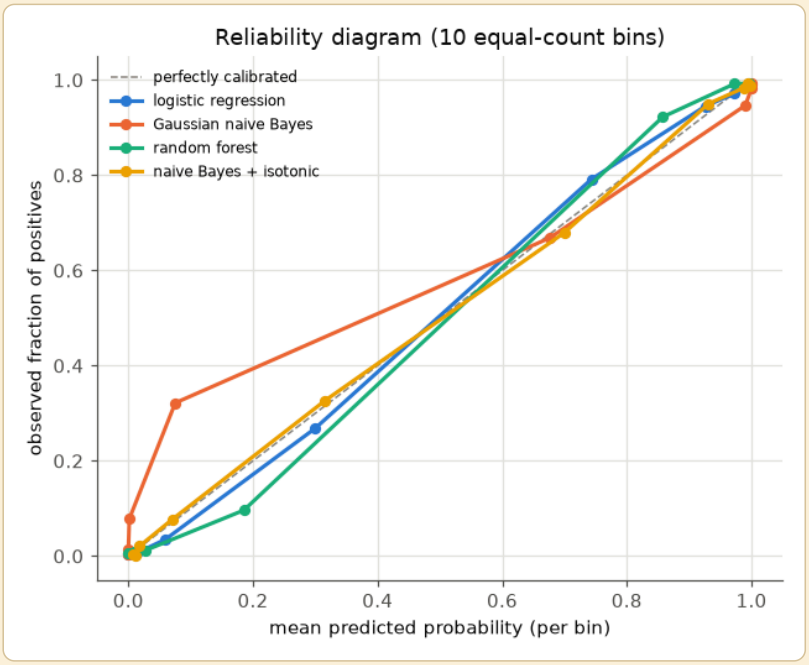
Brier by hand: y = (1, 0, 1, 1), p = (0.9, 0.2, 0.6, 0.3) → ((0.1)² + (0.2)² + (0.4)² + (0.7)²)/4 = (0.01 + 0.04 + 0.16 + 0.49)/4 = **0.175**.

```
# code/calibration.py (key parts)
models = {"logistic regression": LogisticRegression(max_iter=1000), "Gaussian naive Bayes": GaussianNB(),
          "random forest": RandomForestClassifier(n_estimators=100, random_state=0),
          "naive Bayes + isotonic": CalibratedClassifierCV(GaussianNB(), method="isotonic", cv=5),
          "naive Bayes + Platt (sigmoid)": CalibratedClassifierCV(GaussianNB(), method="sigmoid", cv=5)}
frac_pos, mean_pred = calibration_curve(y_te, p, n_bins=10, strategy="quantile")
```

Output:

```
model                          ROC-AUC   Brier log-loss
logistic regression              0.980   0.048    0.176
Gaussian naive Bayes             0.975   0.069    0.335
random forest                    0.992   0.020    0.173
naive Bayes + isotonic           0.975   0.060    0.201
naive Bayes + Platt (sigmoid)    0.975   0.068    0.246

Brier by hand: mean((p - y)^2) = 0.175  sklearn 0.175
```



Naive Bayes ranks nearly as well as logistic regression (ROC-AUC 0.975 vs 0.980) but its probabilities are badly off: its "naive" independence assumption double-counts the many redundant features in this data, pushing scores toward 0 and 1 (log-loss 0.335 vs 0.176). Isotonic calibration leaves ROC-AUC essentially unchanged (it is a non-decreasing map) and cuts log-loss to 0.201. Logistic regression, trained on log-loss, is close to calibrated out of the box. Random forests often have the opposite distortion (averaging many trees pulls probabilities away from 0 and 1); boosted trees and neural nets can drift either way.

Common mistakes

- Calibrating on the training data (the calibrator learns the model's overconfidence on data it memorised).
- Using isotonic regression with a few hundred examples.
- Assuming high AUC means trustworthy probabilities.

Real-world examples

- Ad click prediction (Google, Meta) — predicted click-through rates enter the auction directly; calibration errors translate into money.
- Weather forecasting — "70% chance of rain" is the textbook calibrated forecast; forecasters are scored with the Brier score (which was invented for weather forecasts).

In the interview

30 seconds: *"Calibration means predicted probabilities match observed frequencies — separate from ranking quality. I'd check it with a reliability diagram and Brier or log-loss on held-out data, and fix it with Platt scaling or isotonic regression fitted on a separate calibration set, especially after resampling or class weighting, and recheck it when the population shifts."*

Self-check

1. Does calibration change ROC-AUC? → Not with a strictly increasing map (Platt); isotonic can create ties, which may change it slightly.
2. Platt or isotonic with 300 calibration examples? → Platt.
3. Brier score for p = 0.5 on every example? → 0.25.

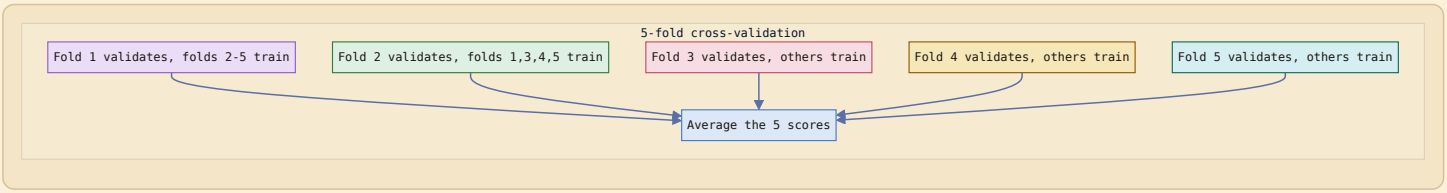
TL;DR

Calibration = probabilities you can take literally. Measure with reliability diagrams and Brier/log-loss; fix with Platt (small data) or isotonic (more data) on held-out data.

6.6 Cross-validation

What it is

Cross-validation (CV) estimates performance by training and validating several times on different splits and averaging. In **k-fold CV**, you cut the training data into k equal **folds**; each fold takes one turn as the validation set while the other k − 1 train the model.



Why it matters

- **One split is noisy.** Here, 20 different 80/20 splits of the same data give accuracies from 0.947 to 0.991 — the choice of split alone moves the result by 4 points.
- CV gives **several estimates**: their mean is more reliable and their spread tells you how much to trust differences between models.
- Every row is used for both training and validation, which matters for small datasets.

How it works — the variants

Splitter	How folds are formed	Use for
`KFold(k, shuffle=True)`	Random equal folds	Regression, i.i.d. data
`StratifiedKFold`	Each fold keeps the class proportions	Classification — the default in `cross_val_score` for classifiers
`GroupKFold` / `StratifiedGroupKFold`	All rows of a group (user, patient) in one fold	Correlated rows; generalising to new entities
`TimeSeriesSplit(gap=…)`	Train on the past, validate on the next block; optional gap	Anything time-ordered; the gap mimics label delay
`RepeatedStratifiedKFold`	Repeat k-fold with different shuffles	Small data, more stable estimates
`LeaveOneOut`	k = n	Tiny datasets only (expensive, high variance)

Typical k is 5 or 10. Larger k → more training data per fold (less pessimistic) but more compute and more correlated estimates.

```
# code/cross_validation.py (key parts)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))   # preprocessing inside!
scores = cross_val_score(model, X, y, cv=5)
KFold(5)                                              # on label-sorted data: badly unbalanced folds
StratifiedKFold(5, shuffle=True, random_state=0)
TimeSeriesSplit(n_splits=3, gap=2, test_size=3)
cross_validate(model, X, y, cv=..., scoring=["accuracy", "roc_auc", "f1"], return_train_score=True)
inner = GridSearchCV(model, {"logisticregression__C": [0.01, 0.1, 1, 10, 100]}, cv=...)
outer = cross_val_score(inner, X, y, cv=...)              # nested CV
```

Output:

```
20 different 80/20 splits: accuracy from 0.947 to 0.991
5-fold CV scores: [0.982 0.982 0.974 0.974 0.991] mean 0.981 +/- 0.007
KFold (no shuffle), sorted y   positive rate per fold [0.   0.14 1.   1.   1.  ]  accuracy 0.968
StratifiedKFold                positive rate per fold [0.62 0.62 0.63 0.63 0.63]  accuracy 0.974
time split: train 0..3  (gap)  validate 6..8
time split: train 0..6  (gap)  validate 9..11
time split: train 0..9  (gap)  validate 12..14
accuracy  train 0.988 | validation 0.979
roc_auc   train 0.998 | validation 0.995
f1        train 0.991 | validation 0.983
non-nested best CV score 0.979 (optimistic) | nested CV 0.975
```

Notes on the output:

- Report CV as **mean ± standard deviation** (0.981 ± 0.007). Two models whose means differ by less than about one standard deviation are not clearly different.
- Unshuffled `KFold` on data sorted by label produces folds that are 0% or 100% positive — nonsense evaluation. Stratification fixes the class mix (~63% in every fold).
- `cross_validate` with `return_train_score=True` shows the train–validation gap (here small: little overfitting).
- **Nested CV.** If you tune hyperparameters with CV and report the *best* CV score, that score is optimistically biased — you picked the maximum of several noisy estimates. Nested CV wraps the whole tuning procedure in an outer CV loop to estimate how well "tune then train" will do on new data (0.975 vs 0.979 here; the gap grows with the size of the search). In practice a held-out test set used once serves the same purpose.

Common mistakes

- Preprocessing (scaling, imputation, feature selection, target encoding) outside the CV loop → leakage (Chapter 4). Put it in a `Pipeline`.
- Shuffled k-fold on time series.
- Using CV for model *selection* and then reporting the same CV score as the final performance.

In the interview

30 seconds: *"k-fold CV trains k models, each validated on a different fold, and averages the scores — a lower-variance estimate than one split, with an error bar. I use stratified folds for classification, group folds when rows share an entity, and forward-in-time splits with a gap for temporal data. All preprocessing goes inside the pipeline, and a final untouched test set — or nested CV — gives the unbiased number."*

- *"Why not always use leave-one-out?"* — n model fits, and its estimate has high variance; 5–10 folds is the usual sweet spot.
- *"CV on a 100-million-row dataset?"* — Usually unnecessary: one large, well-designed (e.g. time-based) validation set already has low variance.

Self-check

1. With 5-fold CV, what fraction of data trains each model? → 80%.
2. Why stratify? → So every fold has the same class mix, especially for rare classes.
3. What does nested CV estimate? → The performance of the whole tuning-plus-training procedure.

TL;DR

k-fold CV = k train/validate rounds, averaged, with a spread. Stratify for classes, group for entities, go forward in time for temporal data, keep preprocessing inside the folds.

6.7 The bias–variance trade-off

What it is

A model's expected error on new data splits into three parts:

Expected test error = Bias² + Variance + Irreducible noise

- **Bias** — error from wrong assumptions: how far the *average* model (averaged over many possible training sets) is from the truth. High bias = **underfitting** (a straight line through a curve).
- **Variance** — how much the model changes when trained on a different sample. High variance = **overfitting** (the model chases noise that won't be there next time).
- **Irreducible noise** — randomness in the labels themselves; no model can remove it.

Analogy: darts. High bias = all darts land close together but off-centre. High variance = darts scattered all over, centred on the bullseye on average.

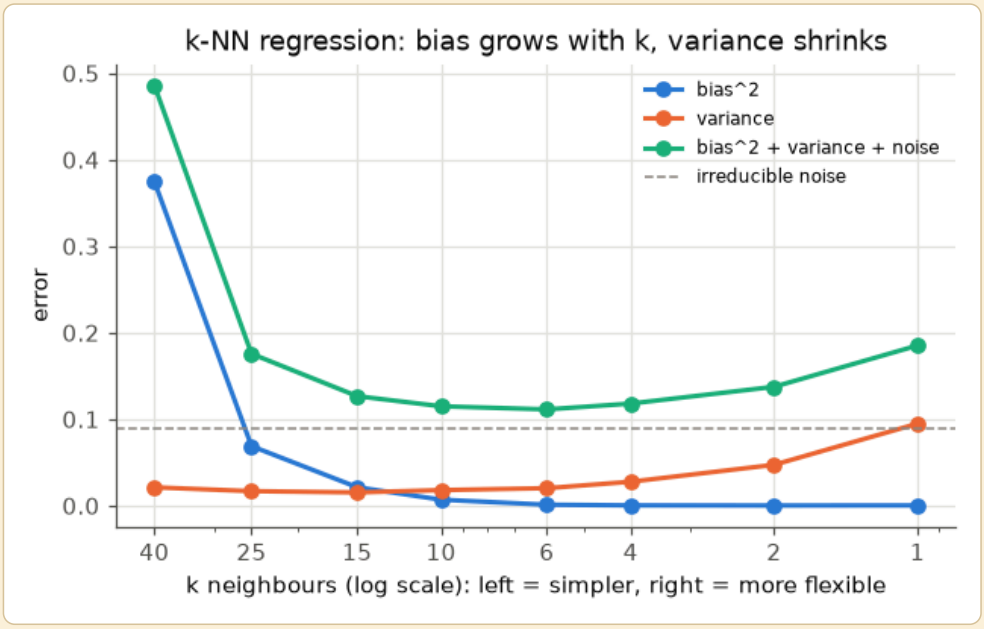
How it works — a simulation you can run

Train **k-nearest-neighbours regression** (predict the average y of the k closest training points) on 300 independent training sets of 50 noisy points from y = sin(2πx) + noise (noise σ = 0.3, so σ² = 0.09), then measure bias² and variance of the predictions:

```
# code/bias_variance.py (key parts)
for k in [40, 25, 15, 10, 6, 4, 2, 1]:
    preds = []
    for _ in range(300):
        x = rng.uniform(0, 1, 50); y = true_f(x) + rng.normal(0, sigma, 50)
        preds.append(KNeighborsRegressor(n_neighbors=k).fit(x[:, None], y).predict(x_test[:, None]))
    preds = np.array(preds)
    bias2 = ((preds.mean(0) - true_f(x_test)) ** 2).mean()   # average model vs truth
    var = preds.var(0).mean()                                 # spread across training sets
```

Output:

```
  k   bias^2  variance  expected test MSE
 40    0.375     0.021              0.486
 25    0.069     0.017              0.176
 15    0.021     0.015              0.127
 10    0.007     0.018              0.115
  6    0.001     0.020              0.111
  4    0.000     0.028              0.118
  2    0.000     0.047              0.137
  1    0.000     0.095              0.185
```



With k = 40 (averaging 40 of 50 points) the prediction is almost flat: huge bias. With k = 1 it copies the nearest noisy point: zero bias, highest variance. The total is lowest around k = 6–10. Every model family has a "complexity knob" like this: tree depth, polynomial degree, regularization strength (inverse), number of boosting rounds, network size.

Over- and underfitting: symptoms and remedies

	Underfitting (high bias)	Overfitting (high variance)
Symptom	Training **and** validation scores both poor, and close to each other	Training score much better than validation score
More data helps?	No — the model can't use it	Yes
Remedies	More flexible model (deeper trees, more features, polynomial/interaction terms, a neural net); **less** regularization; train longer; better features	More data; **more** regularization (L1/L2, dropout, weight decay); simpler model (shallower trees, fewer features); early stopping; bagging/ensembles; data augmentation; fix leakage-free feature selection

Common mistakes

- Adding data to a high-bias model and expecting improvement.
- Judging overfitting from the training score alone — always compare with validation.
- Thinking "more complex model = better": only until variance takes over.

In the interview

30 seconds: *"Test error = bias² + variance + noise. Simple models have high bias and underfit — both scores bad; flexible models have high variance and overfit — training much better than validation. I diagnose with learning curves: if the curves have converged at a low score, add capacity or features; if there's a big gap, add data or regularization."*

- *"Does bagging reduce bias or variance?"* — Mainly variance (averaging unstable models). Boosting mainly reduces bias.
- *"What happens to bias and variance as you increase regularization?"* — Bias up, variance down.

Self-check

1. Training accuracy 99%, validation 80%: which problem? → Overfitting (high variance).
2. Training 70%, validation 69%, target 90%: remedy? → More capacity / better features / less regularization.
3. Can any model beat the irreducible noise? → No.

TL;DR

Bias = systematically wrong; variance = unstable across samples. Total error is U-shaped in model complexity. Big train–validation gap → regularize or add data; both scores low → add capacity.

6.8 Learning curves and validation curves

What it is

Two diagnostic plots:

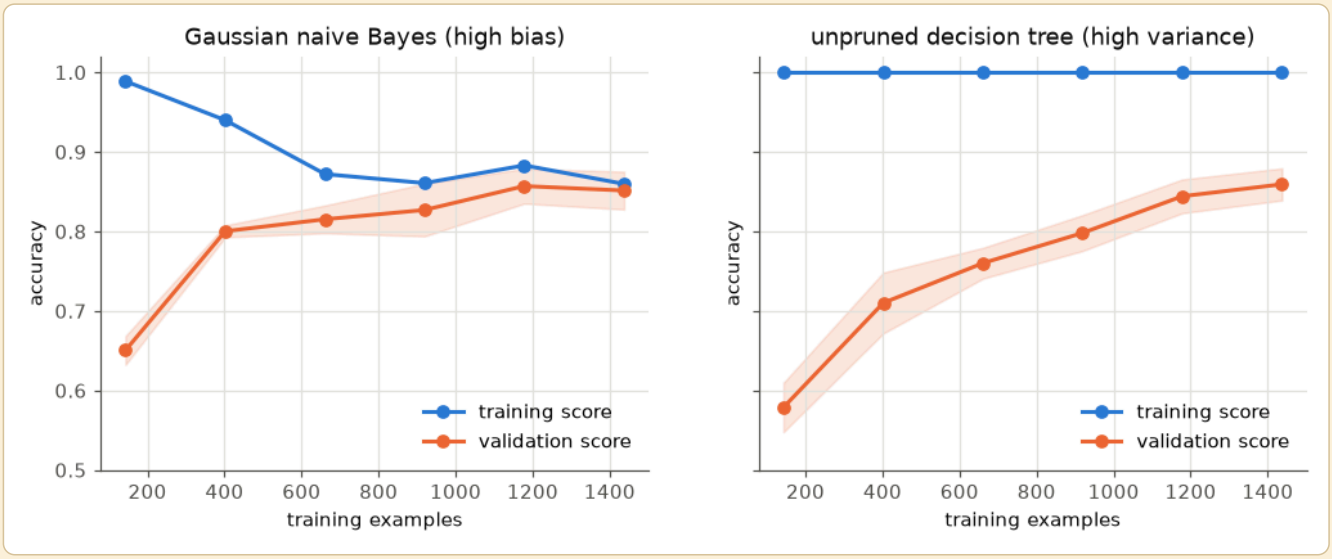
- A **learning curve** plots training and validation scores against **training-set size**. It answers: *would more data help?*
- A **validation curve** plots training and validation scores against **one hyperparameter**. It answers: *where is the sweet spot between under- and overfitting?*

```
# code/bias_variance.py (curves)
sizes, tr, va = learning_curve(model, X, y, cv=cv, train_sizes=np.linspace(0.1, 1.0, 6))
tr, va = validation_curve(make_pipeline(StandardScaler(), SVC()), X, y,
                          param_name="svc__gamma", param_range=np.logspace(-6, -1, 11), cv=cv)
```

Output:

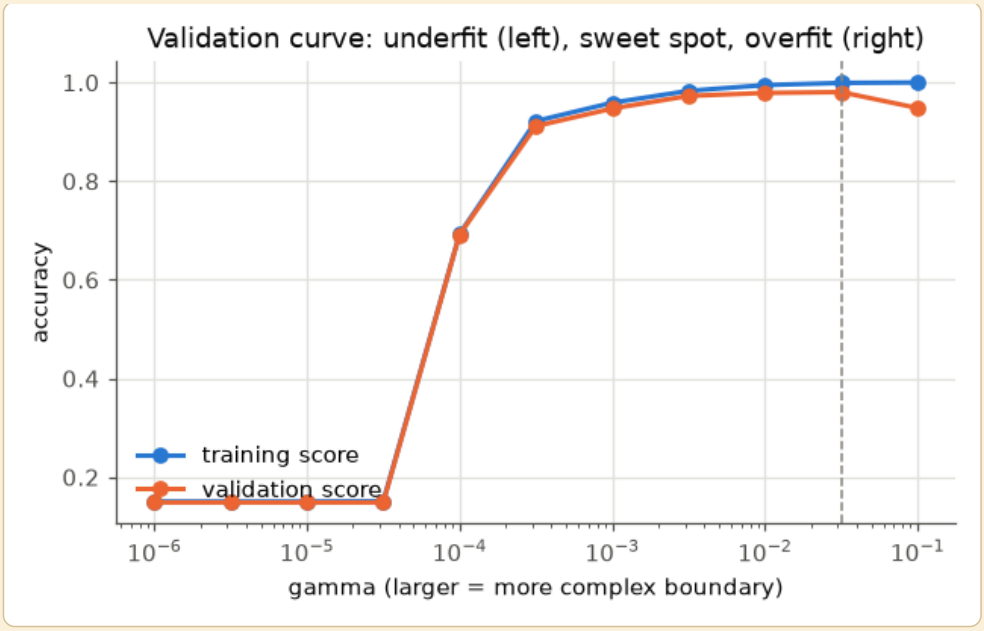
```
Gaussian naive Bayes (high bias)         n=1437: train 0.860  validation 0.851
unpruned decision tree (high variance)   n=1437: train 1.000  validation 0.859

validation curve: best gamma = 3e-02, validation acc 0.981
  gamma 1e-06: train 0.151  validation 0.149
  gamma 1e-05: train 0.151  validation 0.149
  gamma 1e-04: train 0.693  validation 0.692
  gamma 1e-03: train 0.959  validation 0.947
  gamma 1e-02: train 0.995  validation 0.979
  gamma 1e-01: train 1.000  validation 0.948
```



How to read learning curves:

- **Naive Bayes (left):** training and validation scores meet at about 0.85 and flatten. The model is too simple for the data (high bias); more data won't help — change the model or features.
- **Unpruned tree (right):** training accuracy is always 1.0 (memorisation) and validation climbs slowly — a big gap = high variance. More data would help; so would pruning (`max_depth`, `min_samples_leaf`) or a random forest.



How to read the validation curve: with tiny **gamma** (the RBF kernel's reach — larger gamma = each training point influences only a tiny neighbourhood = wigglier boundary) both scores are low (underfit: 0.15, close to the 0.10 chance level for 10 digits). As gamma grows both rise; past about 0.03 the training score stays at 1.0 while validation falls (overfit). Pick the peak of the validation curve.

Common mistakes

- Plotting only the training curve.
- Reading a learning curve's small final gap as "no overfitting" when both curves are still rising — you may just need more data.
- Drawing conclusions from one noisy point — show the CV spread (the shaded band).

In the interview

30 seconds: *"A learning curve plots train and validation score against training-set size: if both have converged low, the model is biased and more data won't help; if there's a big gap that narrows with more data, it's variance and more data or regularization will. A validation curve sweeps one hyperparameter to find the peak between under- and overfitting."*

Self-check

1. Learning curves converge at a low score: more data or more capacity? → More capacity (or better features).
2. Big persistent gap: remedy? → More data, regularization, or a simpler model.
3. What's on the x-axis of a validation curve? → A hyperparameter's values.

TL;DR

Learning curve (score vs data size) tells you whether more data will help; validation curve (score vs hyperparameter) finds the under/overfitting sweet spot.

6.9 Hyperparameter tuning

What it is

Parameters are learned from data (weights, split thresholds). **Hyperparameters** are settings you choose **before** training: tree `max_depth`, k in k-NN, regularization strength C or α, learning rate, number of trees. **Tuning** searches for the hyperparameters that give the best *validation* performance — always with cross-validation or a validation set, never the test set.

How it works — the main strategies

1. Grid search (`GridSearchCV`) — list values for each hyperparameter; try **every combination** with CV; keep the best.

- *Pros:* systematic; guaranteed to find the best point *on the grid*; easy to parallelise.
- *Cons:* cost multiplies: 3 values × 3 values × 5 folds = 45 fits; 5 hyperparameters × 5 values = 3,125 combinations. And the grid may miss the good region entirely.

2. Random search (`RandomizedSearchCV`) — sample a fixed number of combinations from **distributions** (e.g. `randint`, `loguniform`).

- *Pros:* you control the budget (`n_iter`); explores more distinct values of each hyperparameter; usually as good as grid search in far fewer trials when only a few hyperparameters really matter (Bergstra & Bengio, 2012).
- *Cons:* results vary with the random seed; no guarantee of hitting the exact optimum.

3. Successive halving / Hyperband (`HalvingGridSearchCV`, `HalvingRandomSearchCV` — still experimental in scikit-learn 1.9, so first run `from sklearn.experimental import enable_halving_search_cv`) — evaluate many candidates cheaply (few samples or few iterations), keep the best fraction, give survivors more resources, repeat. With `factor=2`, each round keeps half the candidates and doubles the samples per candidate.

4. Bayesian optimisation — build a **surrogate model** of "hyperparameters → score" from the trials so far (a Gaussian process, or TPE — Tree-structured Parzen Estimator), then choose the next trial with an **acquisition function** such as **expected improvement**, which balances *exploiting* promising regions and *exploring* uncertain ones. It shines when each trial is expensive (minutes to hours). Tools: **Optuna** (TPE by default, plus pruning of bad trials), Hyperopt, scikit-optimize, Ray Tune.

Scale matters: search learning rates, C, α and gamma on a **log scale** (`loguniform(1e-4, 1e1)`), not linearly.

```
# code/tuning.py (key parts)
grid = GridSearchCV(DecisionTreeClassifier(random_state=0),
                    {"max_depth": [3, 5, 7], "min_samples_split": [2, 5, 10]}, cv=3, scoring="f1")
rnd = RandomizedSearchCV(DecisionTreeClassifier(random_state=0),
                         {"max_depth": randint(3, 10), "min_samples_split": randint(2, 20),
                          "min_samples_leaf": randint(1, 10)}, n_iter=10, cv=3, scoring="f1", random_state=0)

svm = make_pipeline(StandardScaler(), SVC())
GridSearchCV(svm, {"svc__C": [0.1, 1, 10], "svc__gamma": [1e-4, 1e-3, 1e-2]}, cv=cv)
RandomizedSearchCV(svm, {"svc__C": loguniform(1e-2, 1e3), "svc__gamma": loguniform(1e-5, 1e-1)}, n_iter=9, cv=cv)
HalvingRandomSearchCV(svm, {...same distributions...}, n_candidates=32, factor=2, cv=cv)

# Tiny Bayesian optimisation over log10(gamma): GP surrogate + expected improvement
gp = GaussianProcessRegressor(Matern(nu=2.5) + WhiteKernel(1e-4), normalize_y=True).fit(xs, ys)
mu, sd = gp.predict(grid_pts, return_std=True)
z = (mu - max(ys)) / sd
ei = (mu - max(ys)) * norm.cdf(z) + sd * norm.pdf(z)       # expected improvement
next_x = grid_pts[ei.argmax()]
```

Output:

```
grid   best params: {'max_depth': 5, 'min_samples_split': 2} CV F1 0.942 | test F1 0.920
random best params: {'max_depth': 6, 'min_samples_leaf': 6, 'min_samples_split': 17} CV F1 0.937 | test F1 0.939

SVM, 9 evaluations each: grid best 0.9866 {'svc__C': 10, 'svc__gamma': 0.01}
                         random best 0.9844 {'svc__C': 1.22, 'svc__gamma': 0.00761}
halving: [32, 16, 8, 4, 2] candidates per round, samples per round [60, 120, 240, 480, 960] -> best 0.9698

Bayesian optimisation of gamma (9 evaluations):
  eval 1: gamma = 1e-2.45  CV accuracy 0.9861  (random start)
  eval 2: gamma = 1e-3.92  CV accuracy 0.9510  (random start)
  eval 3: gamma = 1e-4.84  CV accuracy 0.7284  (random start)
  eval 4: gamma = 1e-2.37  CV accuracy 0.9878
  eval 5: gamma = 1e-3.31  CV accuracy 0.9777
  eval 6: gamma = 1e-1.18  CV accuracy 0.9688
  eval 7: gamma = 1e-1.78  CV accuracy 0.9850
  eval 8: gamma = 1e-2.09  CV accuracy 0.9861
  eval 9: gamma = 1e-3.57  CV accuracy 0.9672
```

Reading it:

- **Grid vs random on trees:** the best CV scores differ by 0.005 — within noise — and the random search's pick happens to score better on the test set. Small differences between tuned configurations are usually not meaningful; don't over-tune.
- **SVM:** with 9 trials each, a well-placed grid and random search land within 0.2% of each other. Random search's advantage grows with more hyperparameters, especially unimportant ones.
- **Halving** tried 32 candidates, but most only on 60–240 rows; only 2 finalists were trained on 960. (Its `best_score_` is measured on the last round's subsample, so it isn't directly comparable with the others.)
- **Bayesian optimisation:** after 3 random starts, the first guided trial immediately improves on them (0.9878), then the expected-improvement rule alternates between refining near the best and exploring uncertain regions (evals 6, 7, 9).

Optuna, the popular Bayesian-optimisation library *(Optuna isn't installed in this book's environment — snippet not run)*:

```
import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 8, 128, log=True),
        "l2_regularization": trial.suggest_float("l2_regularization", 1e-6, 10, log=True),
    }
    model = HistGradientBoostingClassifier(**params, random_state=0)
    return cross_val_score(model, X_train, y_train, cv=5, scoring="roc_auc").mean()

study = optuna.create_study(direction="maximize")      # TPE sampler by default
study.optimize(objective, n_trials=50)
print(study.best_params, study.best_value)
```

Trade-offs

Method	Budget control	Scales to many hyperparameters	Uses past results	Best for
Grid	No (product of grid sizes)	Poorly	No	1–2 hyperparameters, cheap models
Random	Yes (`n_iter`)	Well	No	Default for 3+ hyperparameters
Successive halving	Yes	Well	Yes (early elimination)	Many candidates, resource can be scaled (rows, epochs)
Bayesian (Optuna, GP)	Yes	Moderately (tens of dims)	Yes (surrogate model)	Expensive trials: boosting on large data, deep nets

Practical recipe

1. Start from library defaults and a baseline.
2. Tune the few hyperparameters that matter most (learning rate + tree size for boosting; C for linear models/SVMs; depth/leaf size for trees).
3. Random search (or Optuna) on log scales, 20–100 trials, with CV.
4. If the best value sits at the **edge** of a range, extend the range.
5. Refit on all training data with the chosen values (`refit=True` does this), then evaluate **once** on the test set.

Common mistakes

- Tuning on the test set, or reporting the best CV score as the final performance (optimistic — use a test set or nested CV).
- Linear ranges for scale-type hyperparameters.
- Grids so fine that you're fitting noise in the CV estimate.
- Tuning before fixing data problems and features — usually a much bigger win.

In the interview

30 seconds: *"Hyperparameters are set before training, so I tune them with cross-validation: grid search for one or two parameters, random search for more — it's budget-controlled and explores each dimension better — and Bayesian optimisation like Optuna when each trial is expensive; successive halving to discard bad configurations early. Log-scale ranges, tune on validation folds only, and evaluate the winner once on the test set."*

- *"Why is random search often better than grid search?"* — With the same budget, grid search tests only a few distinct values of each hyperparameter; random search tests a new value in every trial, so it explores the important dimensions more finely.
- *"How does Bayesian optimisation choose the next trial?"* — It fits a surrogate of score vs hyperparameters and maximises an acquisition function (e.g. expected improvement) that trades off predicted score against uncertainty.

Self-check

1. Grid of 4 × 5 values with 5-fold CV: how many fits (before refit)? → 100.
2. Which distribution for a learning rate between 1e-4 and 1? → Log-uniform.
3. Why is the best CV score from a search optimistic? → It's the maximum of many noisy estimates.

TL;DR

Grid for tiny spaces, random for most, halving to prune early, Bayesian (Optuna) for expensive trials. Log scales, CV only, test set once.

6.10 Early stopping

What it is

For models trained iteratively (gradient boosting, neural networks, SGD), **early stopping** monitors a validation score during training and stops when it hasn't improved for a set number of iterations (the **patience**). The number of iterations becomes a hyperparameter tuned automatically and almost for free.

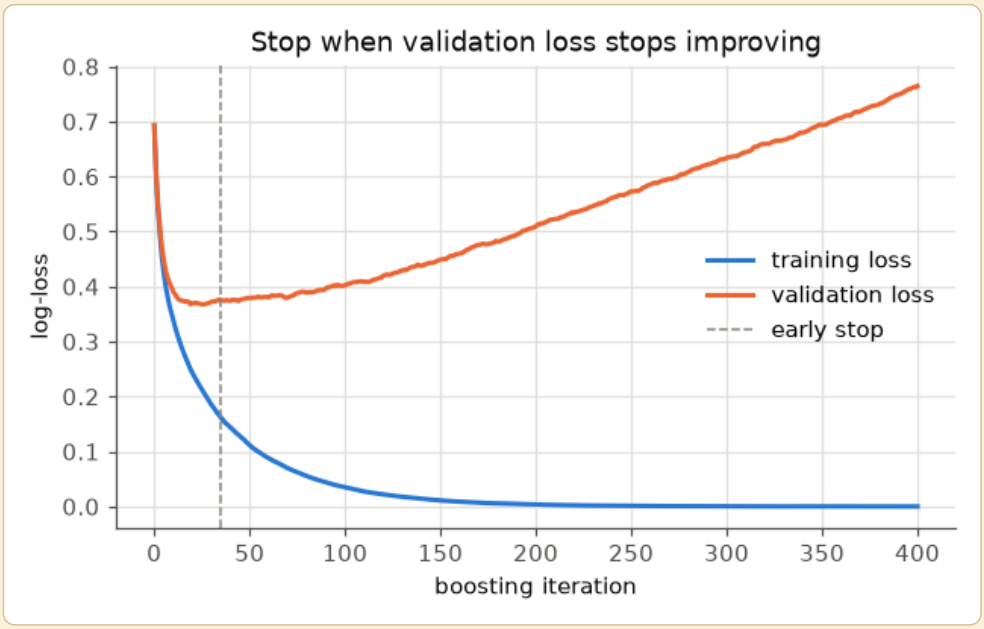
Why it matters

Training loss keeps falling forever; validation loss falls, bottoms out, then rises as the model starts fitting noise. Stopping near the bottom is a cheap, effective regularizer — and saves compute.

```
# code/tuning.py (early stopping)
full = HistGradientBoostingClassifier(max_iter=400, learning_rate=0.2, early_stopping=True,
                                      n_iter_no_change=400, validation_fraction=0.2, random_state=0).fit(Xc, yc)
stop = HistGradientBoostingClassifier(max_iter=400, learning_rate=0.2, early_stopping=True,
                                      n_iter_no_change=10, validation_fraction=0.2, random_state=0).fit(Xc, yc)
```

Output:

```
full run: best validation loss at iteration 25 of 400
early stopping (patience 10) stopped after 35 iterations
```



The validation loss is lowest at iteration 25; with patience 10, training stops at 35 instead of running all 400 iterations, whose later rounds only overfit (validation loss doubles by the end). Note: scikit-learn's `HistGradientBoosting*` keeps all iterations up to the stop, while Keras's `EarlyStopping(restore_best_weights=True)` and XGBoost/LightGBM's best-iteration tracking can roll back to the best one.

How to use it

- scikit-learn: `HistGradientBoosting*(early_stopping=True, validation_fraction=0.1, n_iter_no_change=10)` (on by default when n_samples > 10,000), `GradientBoosting*(n_iter_no_change=…)`, `SGDClassifier(early_stopping=True)`, `MLPClassifier(early_stopping=True)`.
- XGBoost / LightGBM: pass an evaluation set and early-stopping rounds; Keras: the `EarlyStopping` callback.
- Set a generous maximum (iterations/epochs) and a small learning rate; let early stopping choose the count.

Common mistakes

- Early-stopping on the **test** set — it then becomes part of training.
- Patience too small on noisy validation curves (stops on a random bump).
- Using an internal validation split for a time-series problem that is randomly sampled.

In the interview

30 seconds: *"Early stopping holds out a validation set, tracks its loss each iteration or epoch, and stops once it hasn't improved for `patience` rounds — tuning the number of iterations for free and acting as a regularizer. Set a generous maximum, keep or restore the best iteration, and never early-stop on the test set."*

Self-check

1. Validation loss is lowest at iteration 25, patience = 10: when does training stop? → Around iteration 35 (if nothing improves in between).
2. Why is early stopping a form of regularization? → It limits how far the model can fit the training data, like limiting complexity.

TL;DR

Watch validation loss while training; stop after `patience` rounds without improvement. Cheap regularization and compute savings — on a validation set, never the test set.

6.11 Putting it together: from ML metric to business decision

A complete evaluation plan, in order:

1. **Define the business goal and error costs** (e.g. fraud losses vs customer friction; late deliveries vs over-cautious promises).
2. **Pick the offline metric** that best tracks it (section 6.3 table; PR-AUC plus precision/recall at the operating point for rare events; recall@k/NDCG for ranking; MAE/RMSE + p90 + cohorts for regression), and a **calibration** check if scores are used as probabilities.
3. **Split correctly** (stratified / group / time-based) and use **cross-validation** on the training part.
4. **Baselines**: naive + current production model.
5. **Tune** with random search or Bayesian optimisation, early stopping for iterative models.
6. **Diagnose** with learning and validation curves; fix under/overfitting.
7. **Evaluate once on the test set**; report the metric with its base rate, threshold, cohorts and uncertainty.
8. **Choose the threshold** with the cost owner.

9. **Confirm online** with an A/B test on the business metric (Chapter 16), then monitor.

In the interview

- *"Which metric would you use?"* — Name the task's metric family, the base rate, and the cost asymmetry; give an operating-point metric (precision at recall X) plus a threshold-free one (PR-AUC or NDCG); mention calibration if probabilities are consumed; compare with a naive baseline and production; confirm online.

Chapter TL;DR

- **Baselines first**: majority class / mean / last value / most popular, and the production model. Offline metrics are proxies; confirm with online experiments.
- **Regression**: MAE (typical error, median-optimal), RMSE (punishes big misses, mean-optimal), R^2 (variance explained vs the mean). Add p90 error and per-cohort breakdowns.
- **Classification**: confusion matrix → precision TP/(TP+FP), recall TP/(TP+FN), F1 (harmonic mean). Accuracy only for balanced data. The threshold sets the operating point; pick it from error costs (calibrated p > C_FP/(C_FP + C_FN)); fraud uses approve/review/block bands.
- **ROC-AUC vs PR-AUC**: ROC-AUC = probability a positive outranks a negative, insensitive to base rate; PR-AUC exposes false alarms and is the choice for rare positives (no-skill value = base rate).
- **Ranking**: precision@k, recall@k (retrieval), MAP, MRR (first good answer), NDCG (graded relevance with position discount) — check which "ideal" your library uses.
- **Calibration**: probabilities matching frequencies, separate from ranking. Reliability diagram, Brier, log-loss; fix with Platt (small data) or isotonic (more data) on held-out data.
- **Cross-validation**: k-fold with mean ± std; stratified for classes, group for entities, time-series with gap for temporal data; preprocessing inside the pipeline; nested CV or a test set for unbiased final numbers.
- **Bias–variance**: error = bias² + variance + noise. Underfit → more capacity/features, less regularization. Overfit → more data, more regularization, simpler model, early stopping, ensembles.
- **Curves**: learning curves (score vs data size) say whether more data helps; validation curves (score vs hyperparameter) locate the sweet spot.
- **Tuning**: grid (small spaces), random (default, budgeted), successive halving (prune early), Bayesian/Optuna (expensive trials). Log scales; never tune on the test set.
- **Early stopping**: stop when validation loss stops improving for `patience` rounds.

Chapter 7 — Decision Trees and Ensembles

Decision trees are the most "human" ML model: a chain of yes/no questions you can print and read. On their own they overfit easily, but when you combine many of them — by **averaging** (Random Forests) or by **correcting each other's mistakes** (gradient boosting) — you get the models that win most real-world problems on **tabular data** (rows and columns, like a spreadsheet or SQL table).

What you'll learn

1. How a decision tree picks each question (Gini, entropy, information gain — with a worked example).
2. Depth, pruning, and regression trees.
3. Why a single tree has high variance, and how bagging and Random Forests fix it (bootstrap, feature subsampling, out-of-bag score).
4. Feature importance — and why the default one can lie (permutation importance).
5. Boosting: AdaBoost intuition and gradient boosting step by step on residuals, with learning rate.
6. XGBoost vs LightGBM vs CatBoost.
7. Voting and stacking.
8. Why trees still beat neural nets on tabular data, and a hyperparameter cheat sheet.

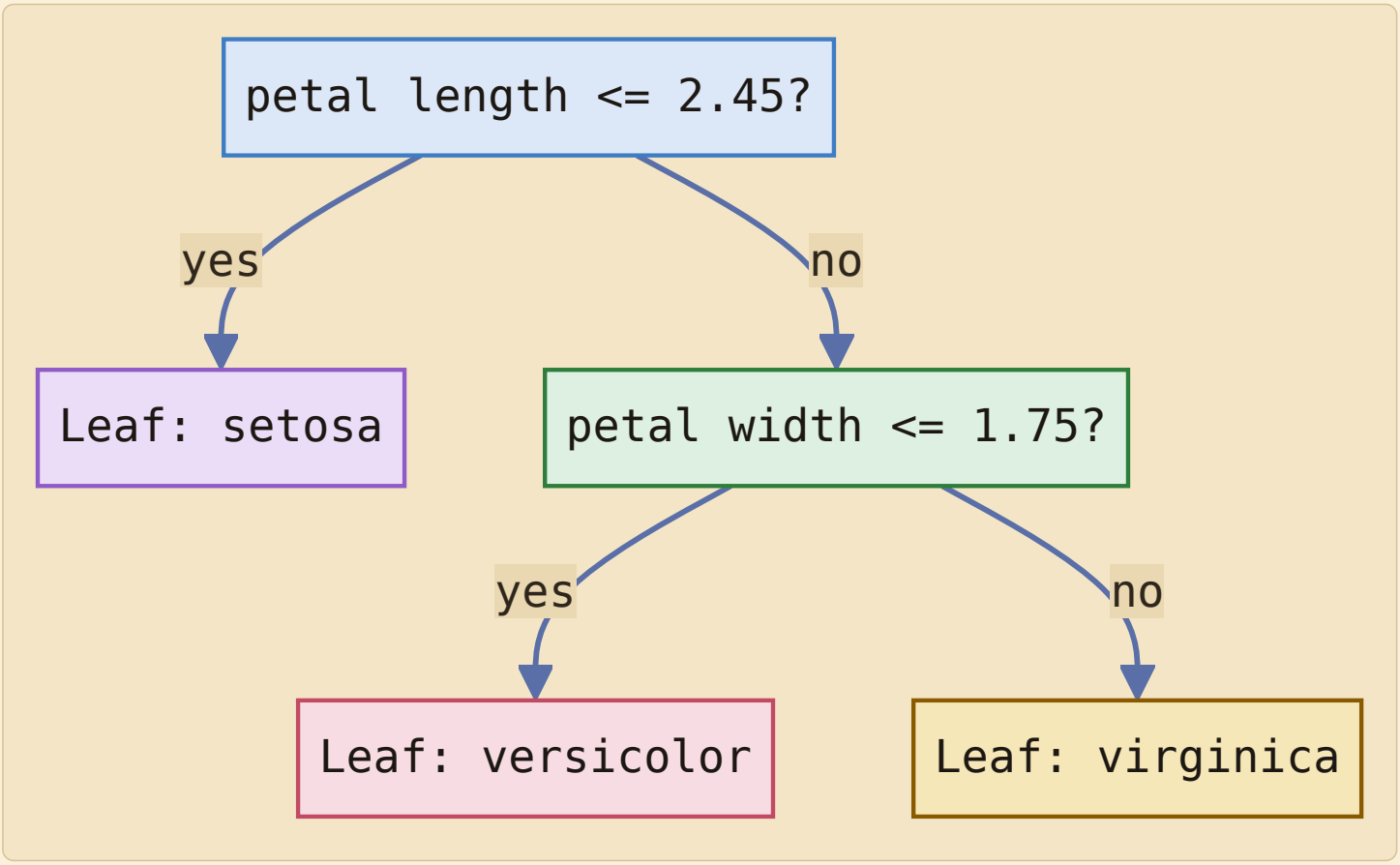
All code lives in `code/` and runs with the shared environment (scikit-learn 1.9). Outputs below are real.

7.1 Decision trees

What it is

A **decision tree** is a model that predicts by asking a sequence of questions about the features, like "Is petal length ≤ 2.45 cm?". Each question is a **node**; each answer sends you down a **branch**; when there are no more questions you reach a **leaf**, which holds the prediction (a class for classification, a number for regression).

Analogy: the game "20 Questions". A good player asks the question that rules out the most possibilities first. A tree-learning algorithm does exactly that, greedily, one question at a time.



Why it matters

- **Interpretable**: a shallow tree is a flowchart a doctor or loan officer can check.
- **No scaling needed**: a split "x ≤ 3.2" does not care whether x is in metres or millimetres.
- **Handles mixed data and non-linear effects and interactions** automatically ("high income AND young" can be its own branch).
- It is the **building block** of Random Forests, XGBoost, LightGBM and CatBoost.

How it works — growing a tree (CART)

scikit-learn implements **CART** (Classification And Regression Trees), which always makes **binary** splits.

1. Start with all training rows at the root.
2. For every feature and every candidate threshold (midpoints between sorted unique values), split the rows into left (≤ threshold) and right (> threshold).
3. Score each split by how much it reduces **impurity** (how mixed the labels are). Pick the best one.
4. Recurse on each child.
5. Stop when a stopping rule fires: node is pure, `max_depth` reached, too few samples (`min_samples_split`, `min_samples_leaf`), or no split improves impurity enough.
6. Each leaf predicts the majority class (classification; it also stores class proportions, which become `predict_proba`) or the mean target value (regression).

This is **greedy**: the best split now, never revisited. Finding the globally optimal tree is NP-hard, so greedy is the practical choice.

Impurity measures

For a node where class k has proportion p_k:

Measure	Formula	Pure node	50/50 binary node
Gini impurity	$G = 1 - \Sigma\, p_k^2$	0	0.5
Entropy	$H = -\Sigma\, p_k \log_2 p_k$	0	1.0 bit
Misclassification error	$1 - \max_k p_k$	0	0.5 (rarely used for growing — not sensitive enough)

- **Gini** = the probability you mislabel a random sample if you label it randomly according to the node's class mix.
- **Entropy** = average number of bits (yes/no answers) needed to identify the class. From information theory.
- **Information gain** = entropy(parent) − weighted average entropy(children). The weights are the fraction of rows that go to each child. The "Gini version" is simply called Gini decrease.

Worked example: Gini vs entropy

Ten loan applicants, label 1 = defaulted. Candidate split: *income < 40k?*

Node	Labels	Gini	Entropy
Parent	4 defaults, 6 repaid	$1 - 0.4^2 - 0.6^2 = \mathbf{0.480}$	$-0.4 \cdot \log_2 0.4 - 0.6 \cdot \log_2 0.6 = \mathbf{0.971}$
Left (income < 40k)	3 defaults, 1 repaid	$1 - 0.75^2 - 0.25^2 = 0.375$	0.811
Right (income ≥ 40k)	1 default, 5 repaid	$1 - (1/6)^2 - (5/6)^2 = 0.278$	0.650

Weighted children: Gini = 0.4·0.375 + 0.6·0.278 = 0.317 → **Gini decrease 0.163**. Entropy = 0.4·0.811 + 0.6·0.650 = 0.715 → **information gain 0.256**.

The tree does not only test 40k — it tests every midpoint. `code/split_math.py`:

```
def gini(labels):
    n = len(labels)
    return 1 - sum((c / n) ** 2 for c in Counter(labels).values())

def entropy(labels):
    n = len(labels)
    return -sum((c / n) * math.log2(c / n) for c in Counter(labels).values())

def weighted(children, fn):          # impurity of a split
    n = sum(len(c) for c in children)
    return sum(len(c) / n * fn(c) for c in children)
# ... loop over every midpoint threshold and print both scores
```

Output:

```
parent [1, 1, 1, 0, 0, 0, 0, 0, 1, 0]  -> gini=0.480  entropy=0.971
left   [1, 1, 1, 0]         -> gini=0.375  entropy=0.811
right  [0, 0, 0, 0, 1, 0]     -> gini=0.278  entropy=0.650
weighted child gini    = 0.317  -> gini decrease   = 0.163
weighted child entropy = 0.715  -> information gain = 0.256

threshold  gini_decrease  info_gain
     27.5          0.080      0.144
     32.5          0.180      0.322
     36.5          0.309      0.557
     41.5          0.163      0.256
     ...
     85.0          0.036      0.079
```

The best threshold is **36.5**, by both measures: the left side (25, 30, 35) is pure. This is typical — Gini and entropy almost always rank splits the same way. Gini is the scikit-learn default because it avoids a logarithm and is slightly faster. Do not spend tuning time on this choice.

Reading a fitted tree

`code/decision_tree_demo.py` fits a depth-2 tree on the Iris flowers and prints its rules with `export_text`:

```
tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(iris.data, iris.target)
print(export_text(tree, feature_names=list(iris.feature_names)))
```

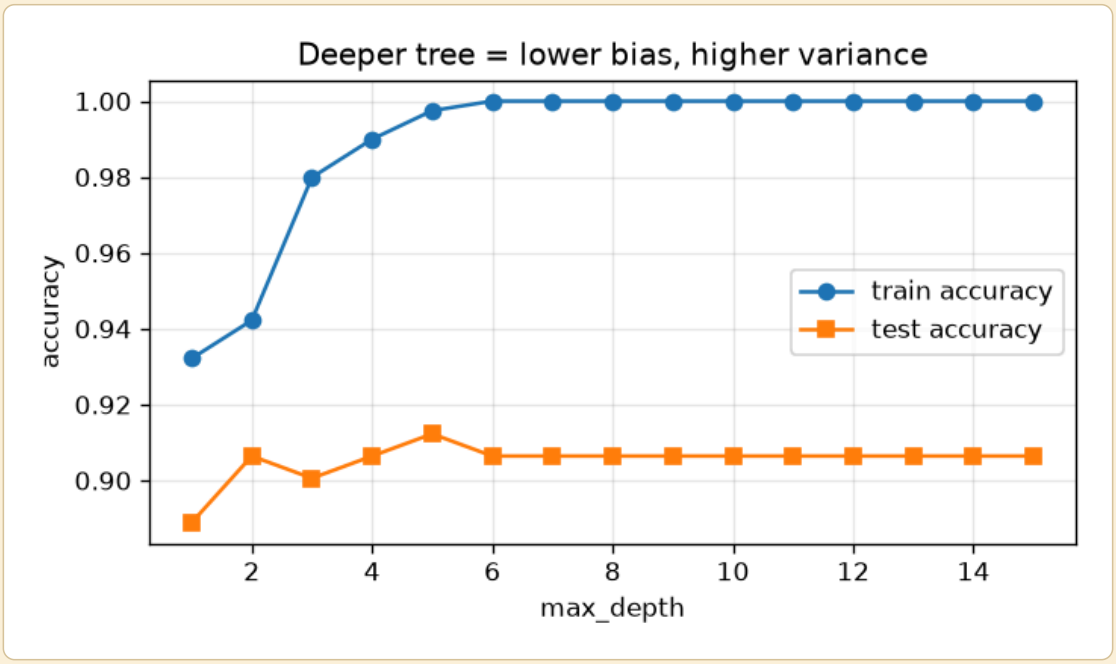
```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2
```

Two questions classify ~96% of the flowers correctly. That is the interpretability selling point.

Depth and overfitting

A tree with no limits keeps splitting until each leaf is pure — it can memorise the training set. Same script, breast-cancer dataset (30 features, 569 patients):

```
depth  train  test
    1  0.932  0.889
    2  0.942  0.906
    3  0.980  0.901
    4  0.990  0.906
    6  1.000  0.906
   10  1.000  0.906
   15  1.000  0.906
```



Training accuracy hits 100% at depth 6, but test accuracy is stuck around 0.90: the extra splits fit noise, not signal.

Pruning

Two ways to keep a tree small:

Kind	How	scikit-learn knobs
Pre-pruning (early stopping)	Stop growing early	`max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, `min_impurity_decrease`
Post-pruning (cost-complexity pruning)	Grow a full tree, then cut back branches that don't pay for themselves	`ccp_alpha`

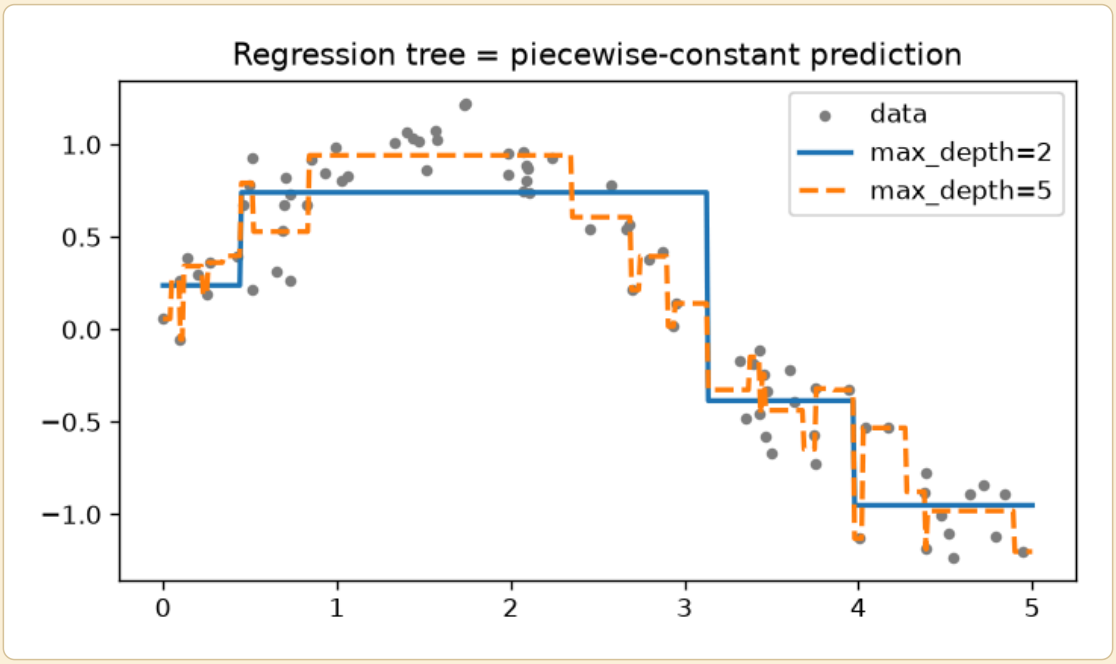
Cost-complexity pruning minimises $R_\alpha(T) = R(T) + \alpha \cdot |\text{leaves}(T)|$, where $R(T)$ is total training impurity (error) and α is a price per leaf. $\alpha = 0$ keeps the full tree; larger α removes the weakest branches first. `cost_complexity_pruning_path` lists the α values at which a branch disappears; you choose α by cross-validation.

```
full tree: 16 leaves, test acc 0.906
ccp_alpha=0.0    leaves= 16  test acc=0.906
ccp_alpha=0.005  leaves=  7  test acc=0.918
ccp_alpha=0.01   leaves=  5  test acc=0.918
ccp_alpha=0.02   leaves=  4  test acc=0.901
ccp_alpha=0.05   leaves=  2  test acc=0.889
```

A 5-leaf tree beats the 16-leaf tree: simpler *and* better. Pruning too hard (2 leaves) underfits.

Regression trees

Same algorithm, different impurity: a regression tree picks the split that most reduces the **squared error** (equivalently, the variance) of the target in the children, and each leaf predicts the **mean** of its training targets. (`criterion="absolute_error"` uses the median instead and is more robust to outliers.) The result is a **step function**:




```
|--- x <= 3.13
|   |--- x <= 0.44
|   |   |--- value: [0.24]
|   |--- x >  0.44
|   |   |--- value: [0.74]
|--- x >  3.13
|   |--- x <= 3.98
|   |   |--- value: [-0.39]
|   |--- x >  3.98
|   |   |--- value: [-0.95]
```

Depth 2 = 4 flat steps (underfit); depth 5 chases individual noisy points (overfit). A key consequence: **a tree cannot extrapolate** — outside the training range it predicts the value of the last leaf, forever flat.

Trade-offs

Strengths	Weaknesses
Readable rules, easy to explain	High variance: small data change → very different tree
No feature scaling; handles non-linearity and interactions	Overfits without depth limits or pruning
Fast prediction: O(depth)	Axis-aligned splits: a diagonal boundary needs a staircase of many splits
Works with numeric + categorical data (scikit-learn ≥ 1.3 trees also handle missing values natively)	Cannot extrapolate beyond the training range
	Greedy — may miss the best overall tree

When not to use it

- When accuracy matters more than a readable model — use a forest or boosting instead.
- For smooth or linear relationships (a linear model needs 2 parameters; a tree needs a staircase).
- For forecasting values outside the range seen in training (e.g. prices that keep rising).

Common mistakes

- Leaving `max_depth=None` and reporting training accuracy (it is ~100% and meaningless).
- Scaling features "for the tree" — harmless but pointless.
- Over-reading a single tree's structure: retrain on a slightly different sample and the top split may change.
- One-hot encoding a categorical with thousands of levels into a single tree — each split can only isolate one level; prefer target/ordinal encoding or a library with native categorical support.

Real-world examples

- Credit scoring and medical triage rules, where regulators or clinicians must read the logic.
- As the weak learner inside every gradient-boosting library.

In the interview

> **30-second answer:** "A decision tree recursively splits the data on the feature/threshold that most reduces impurity — Gini or entropy for classification, variance for regression. Leaves predict the majority class or the mean. Trees are interpretable and need no scaling, but they have high variance and overfit unless you limit depth or prune with cost-complexity pruning, which is why we usually use them inside ensembles."

- *Gini or entropy?* They agree on almost all splits; Gini is default because it's cheaper (no log). Not worth tuning.
- *Why is tree learning greedy?* Finding the optimal tree is NP-hard; greedy top-down splitting is fast and works well in practice.
- *How does a tree handle a feature that's 1000× larger in scale?* It doesn't care — splits depend only on the order of values.

Self-check

1. A node has 8 cats and 2 dogs. Gini? → $1 - 0.8^2 - 0.2^2 = \mathbf{0.32}$.
2. What does a regression-tree leaf predict? → The mean target of the training rows in that leaf.
3. What does increasing `ccp_alpha` do? → Prunes more branches (smaller tree, higher bias, lower variance).
4. Why can't a tree predict a house price higher than any it has seen? → Leaves are averages of training targets; no leaf can exceed the maximum.

TL;DR

Trees ask the best yes/no question at each step (max impurity decrease), are readable and scale-free, but overfit — control them with depth limits or cost-complexity pruning.

7.2 Bias–variance of trees

What it is

Recall: **bias** = error from wrong assumptions (model too simple); **variance** = error from sensitivity to the particular training sample (model too flexible). Test error ≈ bias² + variance + irreducible noise.

- A **shallow** tree (stump) = high bias, low variance.
- A **deep** tree = low bias, **very high variance** — retrain on a bootstrap sample and you get a noticeably different tree.

Why it matters

Deep trees have exactly the property that averaging can fix: they are accurate *on average* but noisy. If you average B models, each with variance σ^2 and pairwise correlation ρ, the variance of the average is:

$$\mathbf{Var(average) = \rho\sigma^2 + (1 - \rho)\sigma^2/B}$$

As B grows, the second term vanishes — but the first term, $\rho\sigma^2$, stays. So **averaging helps only as much as the models disagree**. Two ideas follow:

- **Bagging** attacks variance by averaging many deep trees (lowers the second term).
- **Random Forests** additionally **de-correlate** the trees (lowers ρ, so the first term shrinks too).
- **Boosting** does the opposite: it starts from high-bias shallow trees and adds them sequentially to **reduce bias**.

Method	Base trees	Mainly reduces	Trees built
Bagging / Random Forest	Deep (low bias, high variance)	Variance	Independently, in parallel
Boosting	Shallow (high bias, low variance)	Bias (and some variance via shrinkage)	Sequentially

TL;DR

Deep trees = low bias, high variance → average them (bagging/RF). Shallow trees = high bias → chain them (boosting).

7.3 Bagging and Random Forests

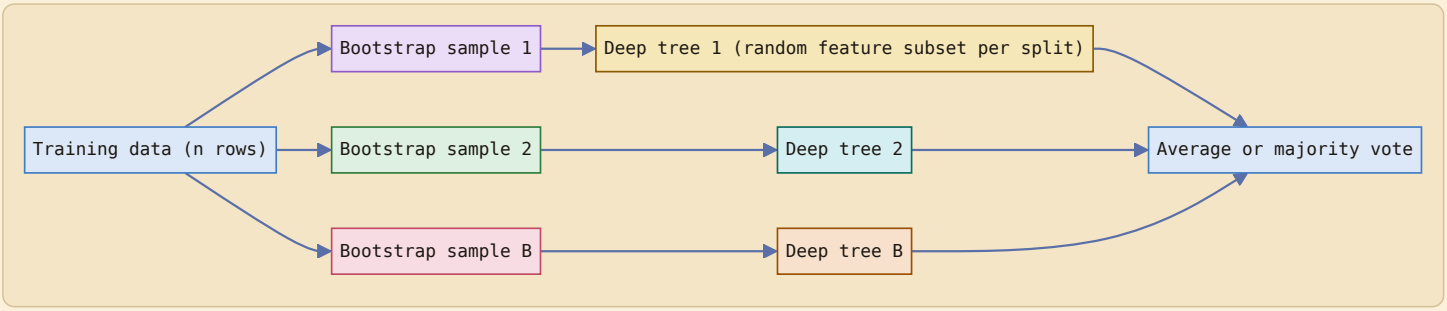
What it is

Bagging (Bootstrap AGGregatING, Breiman 1996) trains many copies of the same model on different **bootstrap samples** and averages them (majority vote or averaged probabilities for classification, mean for regression).

A **bootstrap sample** is n rows drawn **with replacement** from the n training rows: some rows appear twice or more, others not at all.

A **Random Forest** (Breiman 2001) = bagged decision trees **plus** a twist: at every split, the tree may only choose from a **random subset of the features** (`max_features`).

Analogy: asking 500 doctors for a diagnosis. If they all trained at the same school and read the same chart (correlated), polling them adds little. If each saw a slightly different set of tests (bootstrap) and focuses on different symptoms (feature subsampling), their majority vote is far more reliable.



How it works

1. For b = 1..B (`n_estimators`, e.g. 100–500):
    1. Draw a bootstrap sample of n rows.
    2. Grow a deep tree on it (usually unpruned). At each node, pick `max_features` features at random and search for the best split **only among those**.
2. Predict: average the trees' predicted probabilities (scikit-learn does soft voting) or values.

Why feature subsampling helps: if one feature is very strong, each bagged tree chooses it for the first split, so the trees end up nearly identical (high ρ). Hiding it from some splits forces other trees to find other useful patterns, making trees less correlated.

Defaults (scikit-learn): `max_features="sqrt"` for classification (√p features per split) and `1.0` (all features, i.e. plain bagging) for regression. The classic textbook suggestion for regression is p/3; try a few values.

Out-of-bag (OOB) evaluation

The chance a given row is **never** picked in n draws with replacement is $(1 - 1/n)^n \approx 1/e \approx$ **36.8%**. So each tree never sees about a third of the rows — its **out-of-bag** rows. Predict each row using only the trees that didn't train on it, and you get a free validation score without a hold-out set (`oob_score=True`).

Code — single tree vs bagging vs Random Forest

`code/forest_demo.py`:

```
models = {
    "single tree": DecisionTreeClassifier(random_state=0),
    "bagged trees (100)": BaggingClassifier(DecisionTreeClassifier(), n_estimators=100, random_state=0),
    "random forest (100)": RandomForestClassifier(n_estimators=100, random_state=0),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=5)

rf = RandomForestClassifier(n_estimators=300, oob_score=True, random_state=0, n_jobs=-1)
rf.fit(X_tr, y_tr)
print(rf.oob_score_, rf.score(X_te, y_te))
```

Output:

```
5-fold CV accuracy (mean +/- std)
  single tree          0.917 +/- 0.016
  bagged trees (100)   0.958 +/- 0.029
  random forest (100)  0.963 +/- 0.022

OOB accuracy  = 0.967  (free validation estimate, no hold-out used)
test accuracy = 0.953
expected out-of-bag fraction per tree = 0.367  (1/e = 0.368)
```

Averaging lifts accuracy by ~4 points; feature subsampling adds a little more. The OOB estimate is close to the true test score.

Feature importance and its pitfalls

Impurity-based importance (Mean Decrease in Impurity, **MDI**; `feature_importances_`): for each feature, sum the impurity decrease of every split that uses it, weighted by the fraction of samples reaching that node, averaged over trees, normalised to sum to 1.

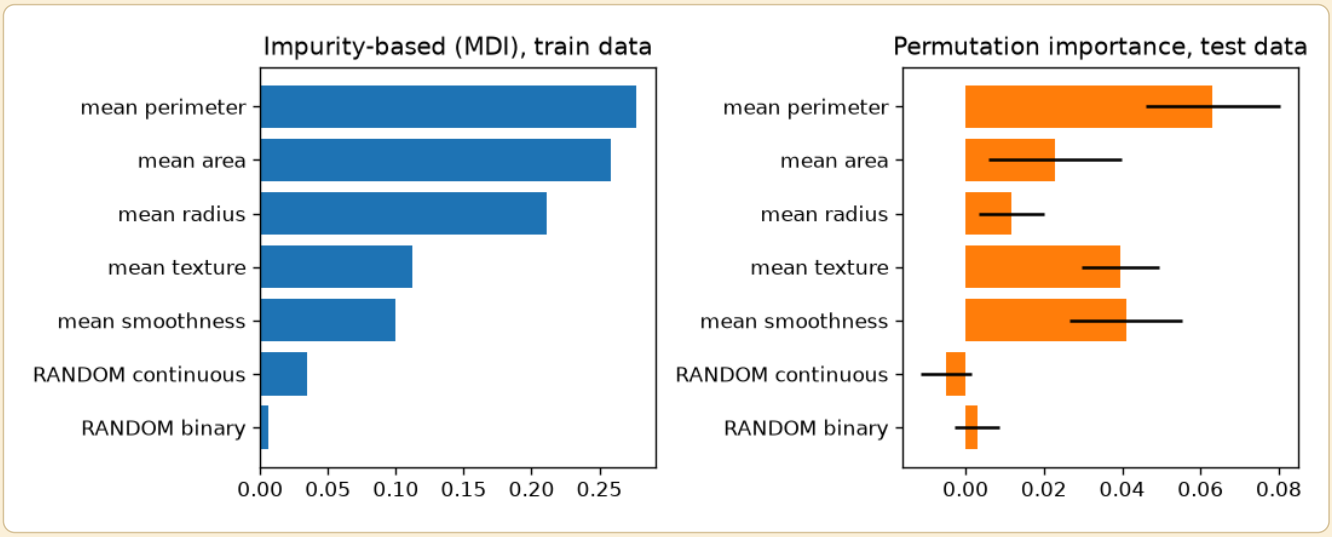
Pitfalls:

1. **Biased toward high-cardinality features** — a continuous feature (or an ID column) offers many thresholds, so it can find chance splits. A pure-noise continuous column gets non-zero importance.
2. **Computed on training data** — it rewards features the model used to *overfit*, not features that help on new data.
3. **Correlated features share credit** — if radius, perimeter and area carry the same information, importance is split among them unpredictably, so each looks less important than the concept really is.

Permutation importance fixes the first two: on a **held-out set**, shuffle one column (breaking its link to the target) and measure how much the score drops. Repeat several times. A feature the model truly relies on causes a big drop; a noise feature causes ~0. It is model-agnostic (works for any model). Caveat: with correlated features, shuffling one leaves its twin intact, so **both** can look unimportant — group or drop correlated features first.

```
X = np.hstack([X, noise_cont, noise_bin])   # add a random continuous and a random binary column
rf = RandomForestClassifier(n_estimators=300, random_state=0, n_jobs=-1).fit(X_tr, y_tr)
perm = permutation_importance(rf, X_te, y_te, n_repeats=20, random_state=0, n_jobs=-1)
```

```
feature              impurity(MDI)  permutation(test)
  mean perimeter             0.278     0.063 +/- 0.017
  mean area                  0.258     0.023 +/- 0.017
  mean radius                0.212     0.012 +/- 0.008
  mean texture               0.112     0.039 +/- 0.010
  mean smoothness            0.100     0.041 +/- 0.014
  RANDOM continuous          0.034    -0.005 +/- 0.006
  RANDOM binary              0.006     0.003 +/- 0.006
```



Two lessons in one table: (1) MDI gives the random continuous column ~6× more importance than the random binary one — pure cardinality bias; permutation importance correctly puts both at ≈ 0. (2) Radius, perimeter and area are nearly the same measurement; MDI ranks them top-3, but permutation importance ranks texture and smoothness above radius and area because shuffling one of the three size features barely hurts while its twins remain.

Better still: SHAP values (Shapley-value-based per-prediction attributions; `shap` library, with a fast exact algorithm for trees) explain individual predictions and aggregate to global importance.

Trade-offs

Strengths	Weaknesses
Strong accuracy with almost no tuning	Larger, slower model than one tree; not readable
Hard to overfit by adding trees (more trees never hurts accuracy, only time)	Usually a bit less accurate than well-tuned boosting
Trees train in parallel (`n_jobs=-1`)	Cannot extrapolate (average of step functions)
OOB score for free, robust to outliers and noisy features	Memory heavy with deep trees on big data

When not to use it

- Very high-dimensional sparse data (text) — linear models are faster and often better.
- Latency- or memory-critical deployment with hundreds of deep trees.
- When you need the last percent of accuracy — try gradient boosting.

Common mistakes

- Treating `feature_importances_` as causal or as "true" importance.
- Tuning `n_estimators` as if more trees overfits — it doesn't; stop when the score plateaus.
- Forgetting `n_jobs=-1` and waiting on one core.
- Using OOB score with `bootstrap=False` (there are no OOB rows then; scikit-learn raises an error).

Real-world examples

- Microsoft's Kinect body-part recognition used randomized decision forests on depth-image features (Shotton et al., 2011).
- A standard, strong baseline in credit risk, fraud, churn and bioinformatics (gene-expression) pipelines.

In the interview

> **30-second answer:** "A Random Forest trains many deep trees on bootstrap samples and, at each split, considers only a random subset of features. Averaging reduces variance; feature subsampling de-correlates the trees so averaging helps more. About 37% of rows are out-of-bag for each tree, which gives a free validation score. It works well out of the box and rarely overfits by adding trees."

- *Bagging vs Random Forest?* RF = bagging + random feature subset at each split, to reduce correlation between trees.
- *Why can MDI importance mislead?* It's computed on training data and favours high-cardinality features; use permutation importance on held-out data, or SHAP.
- *Does adding trees overfit?* No — the test error converges as trees are added; depth and leaf size control overfitting.

Self-check

1. What fraction of rows does a bootstrap sample leave out, on average? → ≈ 36.8% (1/e).
2. Default `max_features` for `RandomForestClassifier`? → `"sqrt"`.
3. Which term of $\rho\sigma^2 + (1-\rho)\sigma^2/B$ does feature subsampling target? → $\rho\sigma^2$ (it lowers ρ).
4. Two features are near-duplicates. What happens to their permutation importances? → Both can look small, because the model can still use the unshuffled twin.

TL;DR

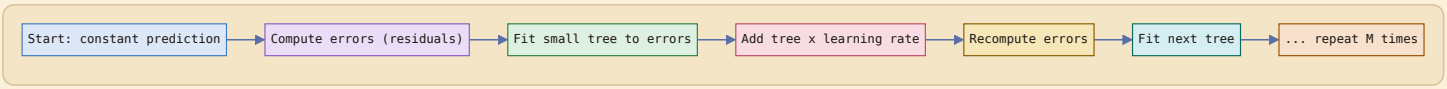
Random Forest = many de-correlated deep trees, averaged. Great default, free OOB score. Use permutation importance or SHAP, not raw MDI, to explain it.

7.4 Boosting

What it is

Boosting builds an ensemble **sequentially**: each new weak model focuses on what the ensemble so far gets wrong. A **weak learner** is a model only slightly better than guessing — typically a shallow tree (a depth-1 tree is a **stump**).

Analogy: a student doing practice exams. After each exam they study only the questions they got wrong. Many small corrections add up to mastery.



AdaBoost (intuition)

AdaBoost (Adaptive Boosting, Freund & Schapire 1997) re-weights *samples*:

1. Give every training sample weight 1/n.
2. Fit a stump using those weights. Compute its weighted error ε.
3. Give the stump a vote $\alpha = \tfrac{1}{2}\cdot\ln((1-\varepsilon)/\varepsilon)$ — accurate stumps get bigger votes; $\varepsilon = 0.5$ (coin flip) gets $\alpha = 0$.
4. Multiply the weight of each **misclassified** sample by e^{α} and each correct one by $e^{-\alpha}$; renormalise so weights sum to 1.
5. Repeat. Final prediction = sign of $\Sigma\ \alpha_m \cdot h_m(x)$ (a weighted vote).

Worked numbers: $\varepsilon = 0.2 \rightarrow \alpha = \tfrac{1}{2}\cdot\ln 4 \approx 0.693$, so wrong samples ×2 and right samples ×0.5. Total weight on the wrong ones becomes $0.2\cdot 2 = 0.4$, on the right ones $0.8\cdot 0.5 = 0.4$ → after renormalising, the next stump sees the previously misclassified 20% as **half** of the problem.

(scikit-learn's `AdaBoostClassifier` uses the multi-class **SAMME** variant: its vote is $\alpha = \ln((1-\varepsilon)/\varepsilon) + \ln(K-1)$ for K classes (so exactly twice the α above when K = 2), scaled by a `learning_rate`; the idea is identical.) AdaBoost can be shown to be gradient boosting with an **exponential loss**, which also makes it sensitive to outliers and label noise: a mislabelled point keeps getting heavier.

Gradient boosting, step by step on residuals

Gradient boosting (Friedman 2001) re-fits *targets* rather than re-weighting samples. For squared error it is wonderfully simple:

1. $F_0(x) = \text{mean}(y)$ — the best constant.
2. For m = 1..M:
    1. Residuals $r_i = y_i - F_{m-1}(x_i)$.
    2. Fit a small regression tree h_m to predict r from x.
    3. Update $F_m(x) = F_{m-1}(x) + \eta\cdot h_m(x)$, where η is the **learning rate** (shrinkage), e.g. 0.1.

3. Predict with F_M.

Why "gradient"? For loss $L = \frac{1}{2}(y - F)^2$, the negative gradient with respect to the prediction F is exactly $y - F$, the residual. For other losses you fit the tree to the **negative gradient** ("pseudo-residuals") instead — e.g. for log-loss in classification, it is $y - p$ (label minus predicted probability). So gradient boosting is gradient descent in *function space*: each tree is one step downhill.

Worked example (4 houses, size in 100 m² → price in k$, η = 0.5, stumps), from `code/boosting_demo.py`:

```
round 0: predict mean = 295.0 for every house
round 1: residuals [-95. -35.   5. 125.] -> new preds [274.2 274.2 274.2 357.5]  MSE=2568.8
round 2: residuals [-74.2 -14.2  25.8  62.5] -> new preds [252.1 252.1 296.2 379.6]  MSE=1105.7
round 3: residuals [-52.1   7.9   3.8  40.4] -> new preds [226.  260.8 304.9 388.3]  MSE=427.6
```

Round 1: the stump splits size ≤ 2.5; the left group's mean residual is (−95 − 35 + 5)/3 = −41.7, the right's is 125. Half of each (η = 0.5) is added: 295 − 20.8 = 274.2, 295 + 62.5 = 357.5. Each round the residuals shrink and the MSE falls.

The whole algorithm in ~15 lines:

```python
class TinyGradientBoosting:
    def fit(self, X, y):
        self.f0 = y.mean()                      # step 0: constant prediction
        pred = np.full(len(y), self.f0)
        self.trees = []
        for _ in range(self.n):
            residual = y - pred                 # = negative gradient of 1/2 (y - pred)^2
            t = DecisionTreeRegressor(max_depth=self.depth, random_state=0).fit(X, residual)
            pred += self.lr * t.predict(X)      # take a small step toward the residuals
            self.trees.append(t)
        return self

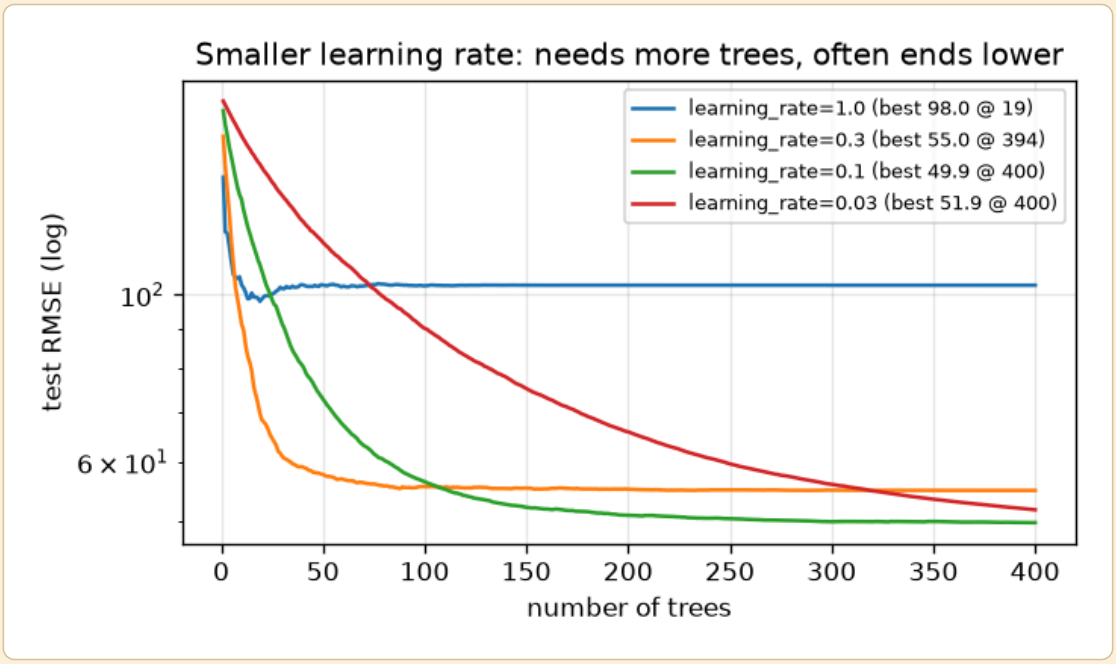
    def predict(self, X):
        return self.f0 + self.lr * sum(t.predict(X) for t in self.trees)
```

```
from-scratch GB test RMSE = 50.86
sklearn GB      test RMSE = 51.01
```

Our toy matches scikit-learn's `GradientBoostingRegressor` with the same settings.

Learning rate and number of trees

The learning rate η and the number of trees M trade off: smaller η means each tree corrects less, so you need more trees, but the final model usually generalises better (shrinkage is a form of regularisation).



- η = 1.0: fast drop, then overfits and stalls at a poor RMSE (98).
- η = 0.3: good quickly, plateaus at 55.
- η = 0.1: best result here (49.9) by 400 trees.
- η = 0.03: still improving at 400 trees — would need more.

Practical recipe: fix η around 0.05–0.1, set a large M, and use **early stopping** on a validation set (stop when the validation score hasn't improved for, say, 50 rounds). scikit-learn: `n_iter_no_change` + `validation_fraction` (`GradientBoostingClassifier`) or `early_stopping=True` (`HistGradientBoosting*`; it's on automatically above 10,000 samples); XGBoost/LightGBM: `early_stopping_rounds` / an early-stopping

callback. Other regularisers: shallow trees (depth 3–8 or `num_leaves` ≤ 31–255), **row subsampling** (`subsample` < 1 = *stochastic* gradient boosting), column subsampling, min samples per leaf, L1/L2 penalties on leaf values.

Comparison on a real dataset

```
models = {
    "AdaBoost (stumps)": AdaBoostClassifier(n_estimators=200, random_state=0),
    "RandomForest": RandomForestClassifier(n_estimators=200, random_state=0, n_jobs=-1),
    "HistGradientBoosting": HistGradientBoostingClassifier(random_state=0),
}
```

```
5-fold CV accuracy
  AdaBoost (stumps)      0.977 +/- 0.009
  RandomForest           0.960 +/- 0.019
  HistGradientBoosting   0.965 +/- 0.018
```

On a small, clean dataset all three are within ~2 points and the ranking can change with the seed — the difference between ensemble families shows more clearly on larger, messier data.

XGBoost vs LightGBM vs CatBoost (and scikit-learn)

All four are gradient-boosted decision trees (**GBDT**). They differ in engineering and defaults:

	XGBoost (2014)	**LightGBM (Microsoft, 2017)**	**CatBoost (Yandex, 2017)**	**scikit-learn `HistGradientBoosting`**
Split finding	Histogram (`tree_method="hist"`, default since 2.0); exact also available	Histogram	Histogram	Histogram (≤ 255 bins), inspired by LightGBM
Tree growth	Level-wise (depth-wise) by default; `grow_policy="lossguide"` for leaf-wise	**Leaf-wise** (split the leaf with the biggest gain) — often faster and more accurate, can overfit small data	**Symmetric (oblivious)** trees: same split for a whole level → very fast prediction, strong regularisation	Leaf-wise (`max_leaf_nodes=31`)
Regularisation	Built into objective: γ per leaf, L1/L2 on leaf weights; uses 1st + 2nd derivatives (Newton step)	L1/L2, min gain, min data in leaf; GOSS (keep large-gradient rows, sample small ones) and EFB (bundle exclusive sparse features) for speed	**Ordered boosting** to reduce target leakage	L2, min samples per leaf
Categorical features	Native support (`enable_categorical=True`)	Native support	**Best-in-class**: ordered target statistics, feature combinations	Native (`categorical_features="from_dtype"` default)
Missing values	Native (learns default direction)	Native	Native	Native
GPU	Yes	Yes	Yes	No
Best when	Solid all-rounder, huge ecosystem	Large datasets, speed	Many categorical features, good defaults with little tuning	Mid-size data, no extra dependency

XGBoost's regularised objective, for the curious: minimise Σ loss$(y_i, \hat{y}_i)$ + Σ_trees $(\gamma \cdot T + \tfrac{1}{2}\lambda \cdot \|w\|^2)$, where T = number of leaves and w = leaf values. With gradients G and second derivatives H summed over a leaf, the optimal leaf value is $w^* = -G/(H + \lambda)$.

The APIs are all scikit-learn-compatible (not run here — packages not installed in the book environment):

```
# pip install xgboost lightgbm catboost   (not run)
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

xgb = XGBClassifier(n_estimators=2000, learning_rate=0.05, max_depth=6,
                    subsample=0.8, colsample_bytree=0.8, early_stopping_rounds=100)
xgb.fit(X_tr, y_tr, eval_set=[(X_val, y_val)])

lgbm = LGBMClassifier(n_estimators=2000, learning_rate=0.05, num_leaves=31)
cat = CatBoostClassifier(iterations=2000, learning_rate=0.05, cat_features=["city", "device"])
```

Trade-offs: bagging vs boosting

	Random Forest	Gradient boosting
Accuracy on tabular data	Very good	Usually best
Tuning needed	Little	Moderate (η, trees, depth/leaves, subsampling)
Overfits by adding trees?	No	**Yes** — use early stopping
Training	Parallel across trees	Sequential across trees (parallel within a tree)
Sensitivity to noise/outliers	Low	Higher (esp. AdaBoost)

When not to use it

- Unstructured data (images, audio, raw text) — deep learning wins.
- Tiny datasets with strong domain priors — a regularised linear model may be as good and more explainable.
- When you need predictions outside the training range (trees cannot extrapolate — combine with a linear model or model the trend separately).

Common mistakes

- Tuning `n_estimators` without early stopping → overfitting.
- Learning rate 1.0 "because it's faster".
- Leaking the validation set: using the same set for early stopping *and* final reporting (keep a separate test set).
- Using AdaBoost on noisy labels.
- Deep trees in boosting (depth 15) — boosting wants weak learners.

Real-world examples

- GBDT dominates Kaggle tabular competitions; XGBoost's 2016 paper noted it was used in 17 of 29 winning Kaggle solutions published in 2015.
- Web search ranking: LambdaMART (gradient-boosted trees for learning-to-rank, from Microsoft Research) was used in Bing's ranker, and an ensemble of LambdaMART models won the 2010 Yahoo! Learning to Rank Challenge.
- Fraud detection, credit scoring, click-through-rate and demand forecasting across the industry; Yandex uses CatBoost in search and recommendation.

In the interview

30-second answer: "Boosting builds trees sequentially, each one correcting the current ensemble. In gradient boosting, you start from a constant, and each new shallow tree is fitted to the negative gradient of the loss — for squared error that's just the residuals — and added with a small learning rate. It mainly reduces bias. It's usually the most accurate model on tabular data but it can overfit as you add trees, so you use a small learning rate with early stopping."

- *AdaBoost vs gradient boosting?* AdaBoost re-weights misclassified samples (exponential loss); gradient boosting fits trees to the negative gradient of any differentiable loss. AdaBoost is a special case.
- *Why a small learning rate?* Shrinkage: smaller steps mean each tree fixes only part of the error, which regularises and usually generalises better, at the cost of more trees.
- *LightGBM vs XGBoost?* LightGBM grows trees leaf-wise and uses GOSS/EFB for speed; XGBoost is depth-wise by default with a regularised second-order objective. Both are histogram-based today; results are often similar after tuning.

Self-check

1. In gradient boosting with squared error, what does tree m fit? → The residuals y − F_{m−1}(x).
2. What's the negative gradient for log-loss? → y − p.
3. AdaBoost stump with ε = 0.5 gets what vote? → α = 0.
4. Learning rate 0.1 → 0.01: what else must change? → Roughly 10× more trees (use early stopping).
5. Which library uses symmetric (oblivious) trees? → CatBoost.

TL;DR

Boosting = shallow trees in sequence, each fixing the last one's errors (residuals/gradients), scaled by a learning rate. Most accurate on tabular data; always use early stopping.

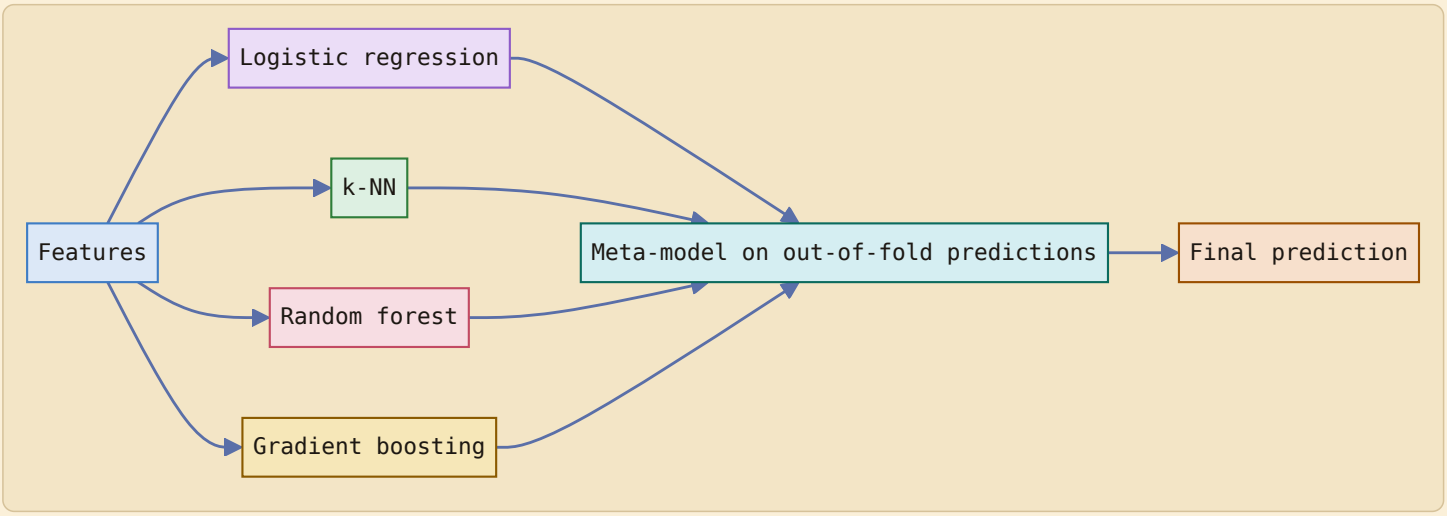
7.5 Voting and stacking

What it is

Ensembles of **different** model families:

- **Hard voting**: each model votes a class; majority wins.
- **Soft voting**: average the predicted probabilities; pick the highest. Usually better when models are well calibrated.
- **Stacking** (stacked generalisation, Wolpert 1992): train a **meta-model** (often logistic/linear regression) on the base models' predictions. To avoid leakage, the meta-model is trained on **out-of-fold** predictions — each base prediction comes from a model that did

not see that row (scikit-learn does this with the `cv` argument).



Why it matters

Different families make **different errors** (low correlation → averaging helps, same math as §7.2). Stacking learns *how much to trust* each model.

Code

`code/voting_stacking.py`:

```
base = [
    ("lr", make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))),
    ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7))),
    ("rf", RandomForestClassifier(n_estimators=200, random_state=0, n_jobs=-1)),
    ("hgb", HistGradientBoostingClassifier(random_state=0)),
]
models["soft voting"] = VotingClassifier(base, voting="soft")
models["stacking (meta = LR)"] = StackingClassifier(
    base, final_estimator=LogisticRegression(max_iter=1000), cv=5)
```

```
10-fold CV accuracy
  lr                    0.981 +/- 0.015
  knn                   0.967 +/- 0.020
  rf                    0.965 +/- 0.028
  hgb                   0.968 +/- 0.022
  hard voting           0.977 +/- 0.021
  soft voting           0.975 +/- 0.016
  stacking (meta = LR)  0.979 +/- 0.019
```

An honest result: the ensembles beat three of the four base models but **not** the best single one (scaled logistic regression, since this dataset is nearly linearly separable). Ensembles reduce risk — you don't need to know in advance which model is best — but they are not guaranteed to win.

Trade-offs / when not to use it

- More models = more training time, latency, and maintenance. Netflix famously did not deploy the full Prize-winning blend of 100+ models because the extra accuracy did not justify the engineering cost.
- Gains are small when base models are highly correlated (e.g. five boosting libraries on the same features).

Common mistakes

- Training the meta-model on in-sample base predictions (leakage — the meta-model learns to trust overfit models).
- Soft voting with poorly calibrated models (e.g. SVM scores) without calibration.

In the interview

30-second answer: "Voting averages diverse models' predictions — hard voting by majority, soft voting by averaged probability. Stacking trains a meta-model on out-of-fold predictions of the base models so it learns how to weight them. It helps when base models make different errors."

- *Why out-of-fold?* In-sample predictions of an overfit model look perfect, so the meta-model would over-trust it.

Self-check

1. Soft vs hard voting? → Averaged probabilities vs majority of labels.
2. What prevents leakage in stacking? → Out-of-fold (cross-validated) base predictions.

TL;DR

Combine diverse models; use soft voting for simplicity, stacking with out-of-fold predictions for the last bit of accuracy.

7.6 When trees beat neural networks

On **tabular data**, tree ensembles (especially GBDT) are still the default winner over deep learning for typical dataset sizes. Grinsztajn, Oyallon and Varoquaux (NeurIPS 2022, *"Why do tree-based models still outperform deep learning on typical tabular data?"*) benchmarked 45 medium-sized datasets (~10K samples) and identified reasons:

Property of tabular data	Why trees handle it well	Why neural nets struggle
Target functions are **irregular** (sharp jumps, thresholds)	Splits create sharp steps naturally	Networks are biased toward smooth functions
Many **uninformative features**	Splits simply ignore useless features	MLPs are hurt by them
Features have individual meaning (not rotation-invariant)	Axis-aligned splits respect each column	MLPs are rotation-invariant — they mix columns, which wastes that structure
Mixed types, skewed distributions, missing values	No scaling, native categoricals/missing	Need careful normalisation and encoding

Practical factors add up too: GBDTs train in minutes on a CPU, need far less tuning, and are easier to explain (SHAP). Neural nets win when the data is **unstructured** (images, text, audio), when there is a huge amount of data, or when you need end-to-end learning with embeddings (e.g. recommender systems with millions of IDs, or combining tabular + text + images). Newer tabular foundation models (e.g. TabPFN for small datasets) are narrowing the gap, but a tuned GBDT remains the baseline to beat.

Rule of thumb: tabular problem → start with a GBDT (LightGBM/XGBoost/CatBoost/`HistGradientBoosting`) and a Random Forest baseline.

7.7 Hyperparameter cheat sheet

Goal	Decision tree (sklearn)	Random Forest (sklearn)	GBDT (sklearn `HistGradientBoosting`)	XGBoost	LightGBM
Number of trees	–	`n_estimators` (100–500; more never hurts accuracy)	`max_iter` (use early stopping)	`n_estimators`	`n_estimators` / `num_iterations`
Step size	–	–	`learning_rate` (0.1)	`learning_rate` / `eta` (0.3)	`learning_rate` (0.1)
Tree size	`max_depth`, `max_leaf_nodes`	`max_depth` (None)	`max_leaf_nodes` (31), `max_depth`	`max_depth` (6)	`num_leaves` (31), `max_depth` (−1 = no limit)
Min data per leaf	`min_samples_leaf`	`min_samples_leaf`	`min_samples_leaf` (20)	`min_child_weight` (1)	`min_child_samples` (20)
Row sampling	–	`bootstrap`, `max_samples`	–	`subsample` (1.0)	`bagging_fraction` / `subsample` (1.0)
Feature sampling	`max_features`	`max_features` ("sqrt")	`max_features`	`colsample_bytree`	`feature_fraction` / `colsample_bytree`
Penalties	`ccp_alpha`, `min_impurity_decrease`	`ccp_alpha`	`l2_regularization`	`reg_alpha`, `reg_lambda`, `gamma`	`reg_alpha`, `reg_lambda`, `min_split_gain`
Class imbalance	`class_weight`	`class_weight`	`class_weight`	`scale_pos_weight`	`is_unbalance` / `scale_pos_weight`

(Defaults in parentheses, as of scikit-learn 1.9, XGBoost 2.x and LightGBM 4.x.)

Tuning order for GBDT: (1) set learning rate 0.05–0.1 and early stopping; (2) tune tree size (depth/leaves) and min samples per leaf; (3) tune subsampling; (4) add L1/L2 if still overfitting; (5) finally lower the learning rate and let early stopping add trees.

Chapter TL;DR

- **Decision tree**: greedy yes/no splits that maximise impurity decrease (Gini ≈ entropy); leaves predict majority/mean. Readable, scale-free, but high variance — limit depth or prune with `ccp_alpha`. Cannot extrapolate.
- **Bagging/Random Forest**: average many deep trees on bootstrap samples; RF also samples features per split to de-correlate trees. ~37% OOB rows give a free validation score. Robust default.

- **Feature importance**: MDI is biased (training data, high cardinality, correlated features) — prefer permutation importance on held-out data or SHAP.
- **Boosting**: add shallow trees sequentially, each fitting the residuals/negative gradient, scaled by a learning rate. Best tabular accuracy; use early stopping. AdaBoost = re-weighting, exponential loss.
- **XGBoost/LightGBM/CatBoost**: all histogram GBDTs; differ in tree growth (depth-wise / leaf-wise / symmetric) and categorical handling (CatBoost best).
- **Voting/stacking**: combine diverse models; stacking needs out-of-fold predictions.
- **Tabular data → start with gradient-boosted trees.**

Chapter 8 — k-Nearest Neighbours, Support Vector Machines, Naive Bayes

Three classic algorithms, three completely different ideas of what "learning" means:

- **k-NN** does not build a model at all — it remembers the training data and answers "what did similar examples look like?"
- **SVM** draws the boundary that stays as far as possible from both classes, and with the **kernel trick** can draw curved boundaries without ever computing curved features.
- **Naive Bayes** counts: it applies Bayes' theorem with a bold simplifying assumption and ends up surprisingly good at text.

The chapter ends with a table comparing **every classic supervised algorithm** in this book, so you know which one to reach for.

What you'll learn

1. k-NN: distance metrics, why scaling is mandatory, choosing k, the curse of dimensionality.
2. SVM: margin, support vectors, soft margin and C, kernels (RBF, polynomial) and the kernel trick, SVR for regression.
3. Naive Bayes: Bayes' theorem, Gaussian / Multinomial / Bernoulli variants, a spam filter by hand, the independence assumption.
4. When to use which classic algorithm.

Code lives in `code/`; every output below is real (scikit-learn 1.9).

8.1 k-Nearest Neighbours (k-NN)

What it is

To classify a new point, **k-NN** finds the k training points closest to it and takes a **majority vote** of their labels (classification) or the **average** of their targets (regression).

Analogy: to guess the price of a flat, you look up the 5 most similar flats that sold recently and average their prices. That is k-NN with k = 5.

k-NN is a **lazy** (instance-based) learner: "training" just stores the data; all the work happens at prediction time. It is also **non-parametric**: it has no fixed set of weights — the model *is* the dataset.

Why it matters

- Zero training time, one hyperparameter, no assumptions about the shape of the decision boundary — it can be arbitrarily curved.
- It is the idea behind **similarity search**: "customers like you", "similar products", and the **vector search** used in modern retrieval (e.g. finding the documents closest to a query embedding in RAG systems).
- A classic theory result (Cover & Hart, 1967): with infinite data, the 1-NN error rate is at most twice the best possible (Bayes) error rate. Simple does not mean weak.

How it works

1. Choose k and a distance metric.
2. Store the training set (optionally in a KD-tree or ball tree for faster search).
3. For a query point x: compute its distance to every training point, keep the k smallest.
4. Classification: majority vote (ties broken by the classifier's rules; odd k avoids ties in binary problems). Regression: mean of the k targets.
5. Optional `weights="distance"`: closer neighbours count more (weight = 1/distance).

Distance metrics

For a = (1, 2, 3), b = (4, 6, 3) — `code/knn_demo.py`:

Metric	Formula	Value	Use it for
Euclidean (L2)	$\sqrt{\Sigma(a_i - b_i)^2}$	$\sqrt{(9 + 16 + 0)} =$ **5.000**	Default, continuous features
Manhattan (L1)	$\Sigma\lvert a_i - b_i\rvert$	3 + 4 + 0 = **7.000**	Grid-like movement, more robust to one large difference
Chebyshev (L∞)	$\max_i \lvert a_i - b_i\rvert$	**4.000**	"Worst single feature" matters
Minkowski (Lp)	$(\Sigma\lvert a_i - b_i\rvert^p)^{(1/p)}$	p=1 → L1, p=2 → L2	scikit-learn default: `metric="minkowski", p=2`
Cosine distance	$1 - (a\cdot b)/(\lVert a\rVert\lVert b\rVert)$	1 − 0.855 = **0.145**	Text/embeddings, where direction matters more than length
Hamming	fraction of positions that differ	—	Binary or categorical vectors

Scaling is mandatory

Distances add up raw feature differences, so a feature measured in large units **drowns out** the others. In the Wine dataset one feature (proline) has a standard deviation of 314 while another has 0.12. Output:

```
wine feature ranges: min std 0.12, max std 314.0 (proline)
k-NN (k=5) CV accuracy  raw features: 0.691   standardized: 0.949
```

Standardising (each feature → mean 0, std 1) lifts accuracy from 69% to 95%. Always put the scaler **inside a pipeline** so it is fitted only on training folds:

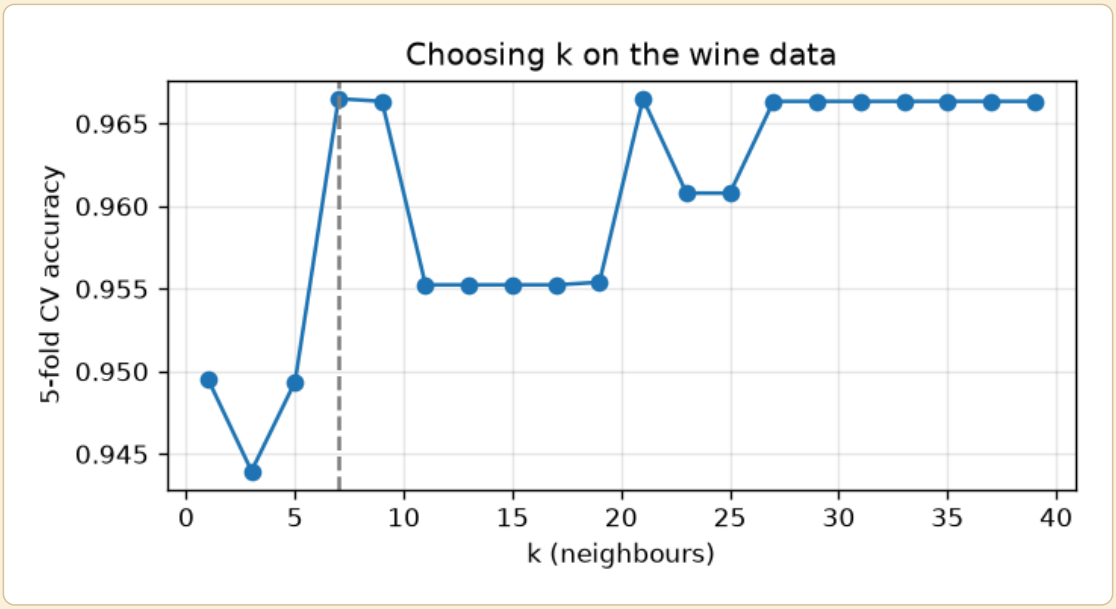
```
make_pipeline(StandardScaler(), KNeighborsClassifier(5))
```

Choosing k

- **Small k** (k = 1): very flexible, jagged boundary, sensitive to noise → low bias, high variance.
- **Large k**: smooth boundary, can blur real structure → high bias, low variance. With k = n you always predict the majority class.
- Choose k by **cross-validation**; odd k for binary tasks. A common starting guess is k ≈ √n, but always validate.

```
ks = list(range(1, 41, 2))
scores = [cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassifier(k)), X, y, cv=5).mean()
          for k in ks]
```

```
best k by 5-fold CV = 7 (accuracy 0.967);  k=1: 0.950, k=39: 0.966
```



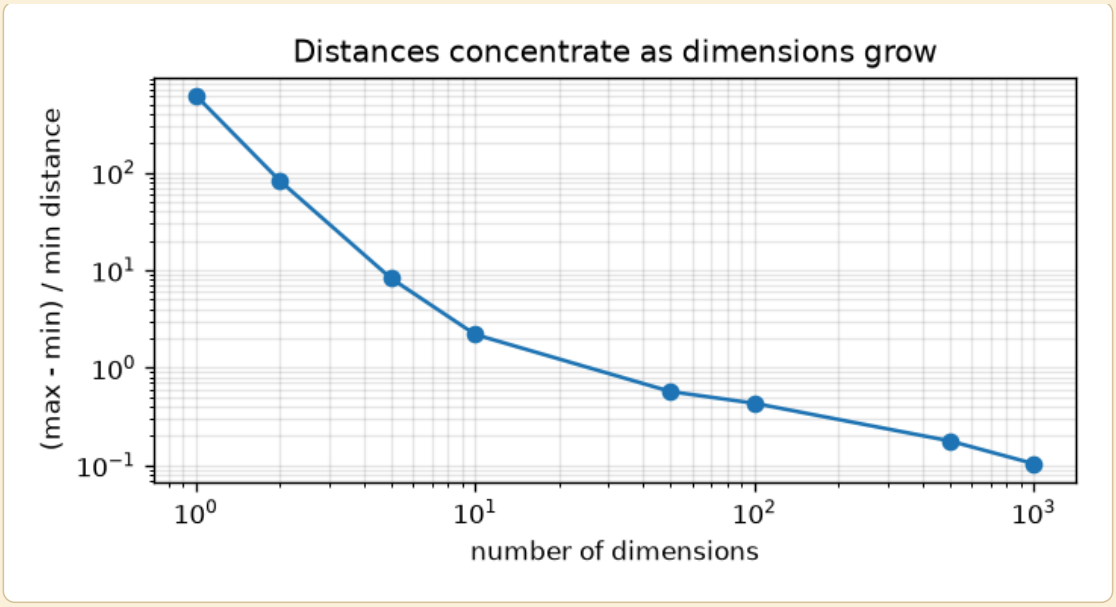
The curse of dimensionality

As the number of features d grows, three things go wrong for any distance-based method:

1. **Space empties out.** To capture 10% of uniformly spread data in a unit hypercube, a sub-cube needs edge length 0.1^(1/d): 0.1 in 1-D, but **0.79** in 10-D. Your "local neighbourhood" covers almost the full range of every feature — it is not local any more.
2. **Distances concentrate.** The nearest and farthest points end up almost the same distance away, so "nearest" loses meaning. Measured on 500 random points (`curse_of_dimensionality()`):

```
dim  (farthest - nearest) / nearest distance, 500 uniform points
    1   615.806
    2    82.019
   10     2.190
  100     0.432
 1000     0.104
```

In 1000 dimensions the farthest point is only ~10% farther than the nearest one.



3. **Data needed grows exponentially** to keep the same density.

Remedies: feature selection, dimensionality reduction (PCA — Chapter 9), learned embeddings where the real data lies on a lower-dimensional surface, or a model that does not rely on distances.

Trade-offs

Strengths	Weaknesses
No training; add new data instantly	Slow, memory-hungry prediction: brute force is O(n·d) per query
Any boundary shape; naturally multi-class	Needs scaling and a good metric
One intuitive hyperparameter	Suffers badly in high dimensions and with irrelevant features
Works for classification and regression	Sensitive to class imbalance (majority floods the vote)

KD-trees and ball trees speed up exact search in low dimensions; for millions of high-dimensional vectors, production systems use **approximate nearest neighbour** (ANN) indexes such as HNSW graphs or FAISS.

When not to use it

Large datasets needing fast predictions, many features (> ~20–50 raw features without reduction), lots of irrelevant features, or when you need a compact model or explanations beyond "these examples were similar".

Common mistakes

- Forgetting to scale (or scaling on the full dataset before the train/test split — leakage).
- Using even k in binary problems (ties).
- Including ID-like or irrelevant columns — they add noise to every distance.
- Using Euclidean distance on one-hot categorical data or text counts where Hamming or cosine fit better.

Real-world examples

- "Similar items" and "users like you" recommendations (item-based collaborative filtering is a nearest-neighbour method).
- Vector databases and semantic search (approximate k-NN over embeddings) — the retrieval step in RAG.
- Imputation: `KNNImputer` fills a missing value with the average of the nearest rows.

In the interview

> **30-second answer:** "k-NN stores the training set and predicts by majority vote or average of the k closest points under a distance metric. It has no training cost but prediction is O(n·d) without an index. You must scale features, pick k by cross-validation — small k overfits, large k underfits — and it degrades in high dimensions because distances concentrate."

- *Why is k-NN called lazy?* It defers all computation to query time; there is no fitted model.
- *How to speed it up?* KD-tree/ball tree in low dimensions; approximate NN (HNSW, FAISS) at scale; reduce dimensions first.
- *k-NN regression?* Average (or distance-weighted average) of the k neighbours' targets.

Self-check

1. Euclidean distance between (0, 0) and (3, 4)? → 5.
2. Why must you scale for k-NN but not for trees? → k-NN adds raw feature differences; trees only compare values within one feature.
3. What happens with k = 1 on noisy data? → It memorises the noise (training accuracy 100%, high variance).
4. What does the curse of dimensionality do to nearest neighbours? → Nearest and farthest distances become nearly equal, so neighbours are not meaningfully "near".

TL;DR

k-NN predicts from the k most similar training examples. Scale features, tune k by CV, and avoid it in high dimensions or when predictions must be fast.

8.2 Support Vector Machines (SVM)

What it is

A **Support Vector Machine** is a classifier that finds the separating boundary (a **hyperplane**: a line in 2-D, a plane in 3-D, …) with the **largest margin** — the widest "street" between the two classes. The points that sit on the edge of the street are the **support vectors**; they alone define the boundary.

Analogy: building a road between two villages. You don't just want a road that separates them — you want the widest possible road, so small shifts in where a house stands don't put it on the wrong side.

Why it matters

- A large margin gives good generalisation, especially with many features and few samples.
- With **kernels**, the same algorithm learns curved boundaries.
- Before deep learning (roughly 1995–2012), SVMs were a top choice for text classification, image recognition and bioinformatics, and they remain a strong baseline for small-to-medium datasets.

How it works — the margin

A linear classifier predicts with the sign of $f(x) = w \cdot x + b$. The boundary is where $f(x) = 0$.

1. Scale w and b so the closest points of each class satisfy $f(x) = +1$ or -1. These are the support vectors.
2. The distance between the two lines $f = +1$ and $f = -1$ is $2/\|w\|$, where $\|w\|$ is the length of the weight vector.
3. **Hard-margin SVM**: maximise $2/\|w\|$ (equivalently, minimise $\tfrac{1}{2}\|w\|^2$) subject to every point being on the correct side: $y_i(w \cdot x_i + b) \geq 1$ for labels $y_i \in \{-1, +1\}$.
4. This is a **convex** quadratic optimisation problem — it has one global optimum, no local minima.

Soft margin and C

Real data overlap, and a single outlier can force a terrible narrow margin. The **soft-margin** SVM allows violations with slack variables $\xi_i \geq 0$:

minimise $\tfrac{1}{2}\|w\|^2 + C \cdot \Sigma\, \xi_i$ subject to $y_i(w \cdot x_i + b) \geq 1 - \xi_i$

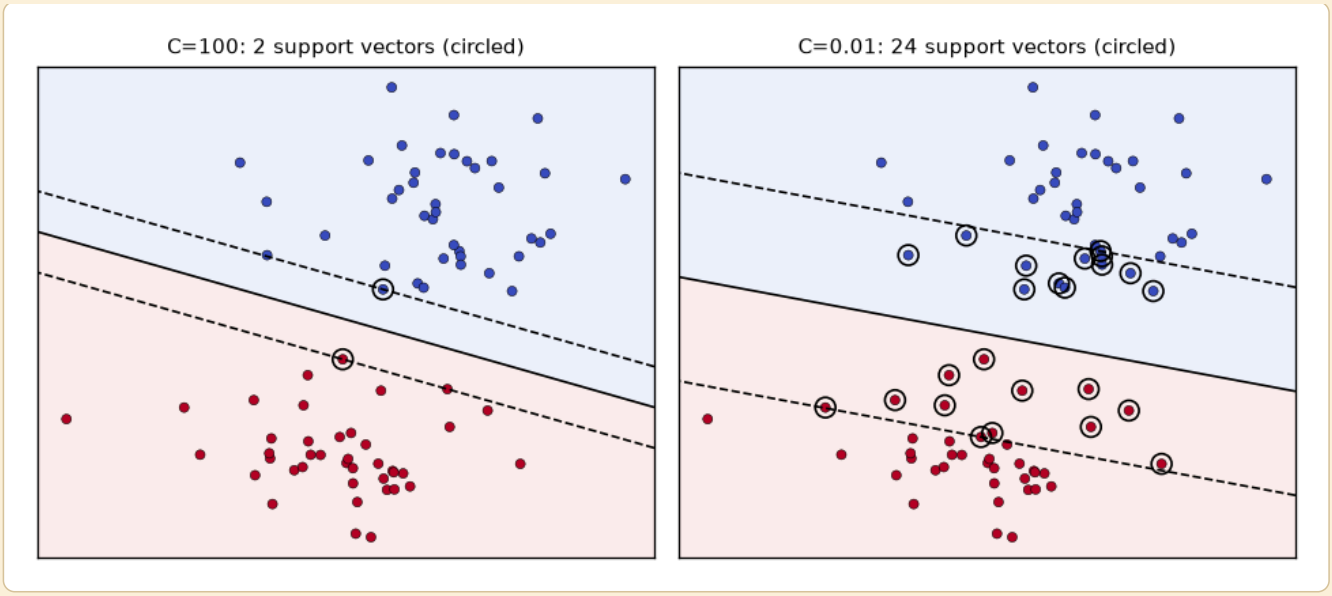
Equivalently, it minimises the **hinge loss** $\max(0, 1 - y_i \cdot f(x_i))$ plus an L2 penalty on w. Points beyond the margin on the correct side cost nothing; points inside the margin or misclassified cost linearly.

- **Large C** → violations are expensive → narrow margin, fits training data tightly (low bias, higher variance).
- **Small C** → violations are cheap → wide margin, more support vectors, smoother (higher bias, lower variance).

C behaves like the inverse of a regularisation strength (like `C` in logistic regression). From `code/svm_demo.py`:

```
for C in (100, 0.01):
    m = SVC(kernel="linear", C=C).fit(X, y)
    w = m.coef_[0]
    print(len(m.support_), 2 / np.linalg.norm(w), m.score(X, y))
```

```
linear SVM on 2 blobs
  C=100    support vectors=  2  margin width 2/||w|| = 1.85  train acc=1.000
  C=0.01   support vectors= 24  margin width 2/||w|| = 4.94  train acc=1.000
```



With C = 100 only 2 points define the boundary; with C = 0.01 the street is 2.7× wider and 24 points lean on it.

Kernels and the kernel trick

Some data can't be split by any straight line (e.g. one class inside a ring of the other). Idea: map each point to a higher-dimensional feature space $\varphi(x)$ where a flat hyperplane *does* separate them. The ring becomes separable once you add the feature $x_1^2 + x_2^2$.

Why only dot products appear. Solving the soft-margin problem with Lagrange multipliers (the so-called **dual form**) gives one weight α_i per training point, with $0 \le \alpha_i \le C$. The solution is $w = \Sigma\ \alpha_i\ y_i\ x_i$, so the prediction becomes

$f(x) = \Sigma_i\ \alpha_i\ y_i\ (x_i \cdot x) + b$

Most α_i come out exactly 0; the points with $\alpha_i > 0$ are the support vectors, which is why only they matter. Notice that the data enter this formula (and the training problem) only through dot products $x_i \cdot x$.

The kernel trick. Because the SVM's optimisation and prediction only ever use **dot products** between points, you can swap each $x_i \cdot x_j$ for a kernel value: $f(x) = \Sigma_i\ \alpha_i\ y_i\ K(x_i, x) + b$. A **kernel** $K(x, z)$ is a function that returns $\varphi(x) \cdot \varphi(z)$ **without computing** φ. So you get the power of a huge (even infinite) feature space at the cost of computing K.

Worked check (`kernel_trick_check()`): for 2-D points, the polynomial kernel $K(x, z) = (x \cdot z)^2$ equals the dot product in the 3-D space $\varphi(v) = (v_1^2, \sqrt{2} \cdot v_1 v_2, v_2^2)$:

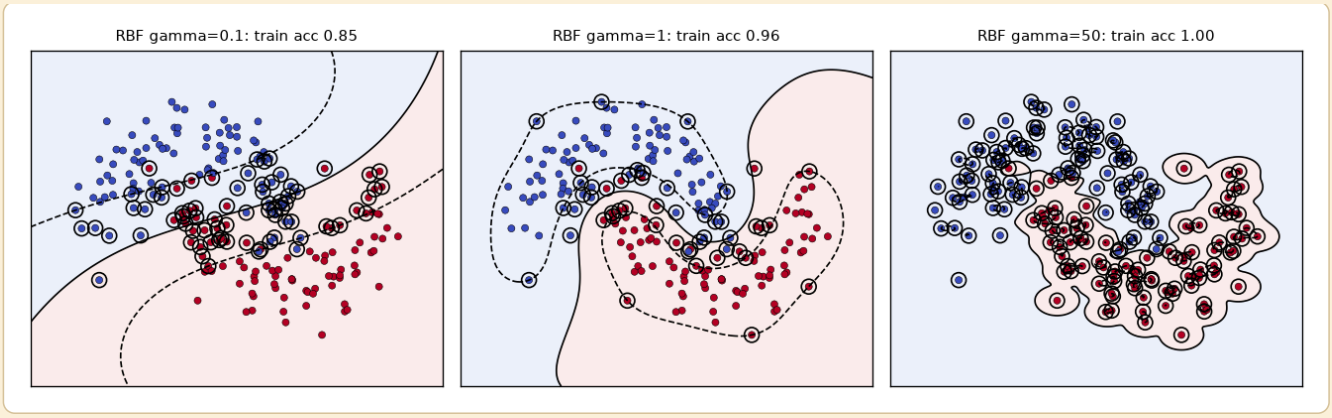
```
(x.z)^2 = 121.0   phi(x).phi(z) = 121.0
```

With $x = (1, 2)$, $z = (3, 4)$: $x \cdot z = 11$, $11^2 = 121$. Same answer, but the kernel never built the 3-D vectors.

Kernel	Formula	Hyperparameters	Use
Linear	$x \cdot z$	C	Many features (text), linearly separable data
Polynomial	$(\gamma \cdot x \cdot z + r)^d$	C, degree d, γ, r (`coef0`)	Feature interactions of known order
RBF (Gaussian)	$\exp(-\gamma\|x - z\|^2)$	C, γ	**Default** for non-linear problems
Sigmoid	$\tanh(\gamma \cdot x \cdot z + r)$	C, γ, r	Rarely used

RBF intuition: K is 1 when x = z and decays towards 0 with distance. The prediction is a weighted sum of "bumps" centred on the support vectors. **γ** sets bump width: small γ = wide bumps, smooth boundary; large γ = narrow bumps around each point, wiggly boundary that can memorise the data.

```
5-fold CV accuracy      moons   circles
  linear              0.845   0.555
  poly (degree 3)     0.940   0.995
  RBF                 0.940   1.000
```



γ = 0.1 underfits (nearly linear), γ = 1 follows the moons, γ = 50 draws islands around individual points (100% train accuracy — overfit). Tune **C and γ together** with a grid on a log scale:

```
grid = GridSearchCV(make_pipeline(StandardScaler(), SVC()),
                    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]}, cv=5)
```

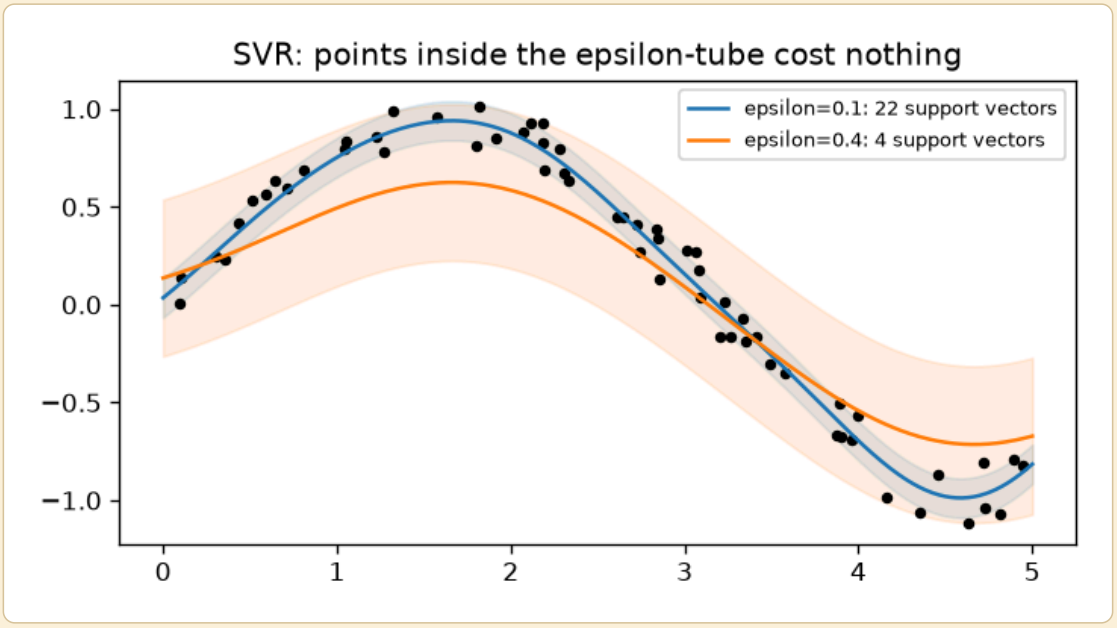
```
grid search best params {'svc__C': 0.1, 'svc__gamma': 10}  CV acc 0.960
```

(Here a large γ is balanced by a small C — the two knobs interact, which is why you search them jointly.)

SVR — Support Vector Regression

SVR flips the idea: fit a function with a tube of half-width **ε** around it. Points **inside the tube cost nothing** (ε-insensitive loss); only points outside are penalised (weighted by C) and become support vectors.

```
SVR epsilon=0.1: 22 of 60 points are support vectors
SVR epsilon=0.4: 4 of 60 points are support vectors
```



A wider tube ignores more noise and uses fewer support vectors, but here ε = 0.4 is too loose and flattens the curve.

Practical notes

- **Always scale** features: RBF and polynomial kernels use distances/dot products.
- Training `SVC` scales at least quadratically with the number of samples (scikit-learn docs note it becomes impractical beyond tens of thousands of samples). For large, linear problems use `LinearSVC` or `SGDClassifier(loss="hinge")`; for large non-linear ones consider kernel approximation (`Nystroem`, `RBFSampler`) + a linear model.
- SVMs output **scores** (signed distances from the boundary), not probabilities. To get probabilities, wrap the model in `CalibratedClassifierCV(SVC(), ensemble=False)`, which fits Platt scaling (`method="sigmoid"`, the default) on cross-validated scores. The older `SVC(probability=True)` did the same internally but is deprecated since scikit-learn 1.9 (removal planned for 1.11). From `svm_probabilities()` in `code/svm_demo.py`:

  ```
  svc = make_pipeline(StandardScaler(), SVC())
  cal = CalibratedClassifierCV(svc, method="sigmoid", ensemble=False).fit(X_tr, y_tr)
  cal.predict_proba(X_te[:3])[:, 1]
  ```

Output:

```
scores  [-1.33  1.12  1.48]
P(y=1)  [0.04 0.92 0.96]
calibrated SVC test accuracy 0.900
```

Negative scores map to low probabilities and positive ones to high; calibration is covered in §6.5.

- Multi-class: `SVC` uses one-vs-one internally; `LinearSVC` uses one-vs-rest.
- Imbalanced classes: `class_weight="balanced"`.

Trade-offs

Strengths	Weaknesses
Maximum margin → good generalisation, strong with many features / few samples	Slow on large n (≥ ~100k)
Convex: unique optimum, no random restarts	Needs scaling and careful C/γ tuning
Kernels give flexible boundaries	No native probabilities; hard to interpret with kernels
Only support vectors matter → compact	Struggles with heavy noise and overlapping classes

When not to use it

Very large datasets, when calibrated probabilities or explanations are required, raw images/audio/text sequences (deep learning), and messy tabular data with mixed types (gradient boosting).

Common mistakes

- Not scaling → one feature dominates the kernel.
- Tuning C and γ separately instead of jointly.
- Using RBF on high-dimensional sparse text, where a linear kernel is faster and usually as good.
- Treating `decision_function` values as probabilities.

Real-world examples

- Text categorisation: linear SVMs were a leading method for document and spam classification (Joachims, 1998).
- Bioinformatics: protein and gene-expression classification with specialised kernels (string kernels).
- Face detection and handwriting recognition pipelines in the 2000s (often on HOG or pixel features).

In the interview

30-second answer: "An SVM finds the hyperplane with the largest margin between classes; only the support vectors — points on or inside the margin — determine it. The soft-margin version uses C to trade margin width against violations: large C fits tightly, small C regularises. Kernels like RBF implicitly map data into a higher-dimensional space through the kernel trick, since the algorithm only needs dot products. Scale features and tune C and γ together."

- *What is the kernel trick?* Replacing dot products φ(x)·φ(z) with K(x, z), computed directly in the original space, so you never build the high-dimensional features.
- *What does γ do in RBF?* Controls the reach of each support vector: high γ → very local, wiggly boundary (overfit); low γ → smooth (underfit).
- *SVM vs logistic regression?* Both linear classifiers with L2 penalties; SVM uses hinge loss (ignores well-classified points, max margin), LR uses log-loss (every point matters, gives probabilities).

Self-check

1. Margin width formula? → 2/‖w‖.
2. Effect of increasing C? → Fewer margin violations allowed → narrower margin, risk of overfitting.
3. Why only support vectors matter? → Other points have zero hinge loss and don't appear in the solution's weights.
4. In SVR, what happens to a point inside the ε-tube? → Zero loss; it's not a support vector.

TL;DR

SVM = widest-street classifier; C trades width for errors; kernels (RBF by default) bend the street via the kernel trick. Scale, tune C and γ jointly, avoid for huge datasets.

8.3 Naive Bayes

What it is

Naive Bayes is a family of probabilistic classifiers that apply **Bayes' theorem** with the "naive" assumption that **features are independent of each other given the class**.

Bayes' theorem

P(class | data) = P(data | class) · P(class) / P(data)

Term	Name	Meaning
P(class)	Prior	How common the class is before seeing the data
P(data | class)	Likelihood	How typical the data is for that class
P(data)	Evidence	Same for every class — just normalises
P(class | data)	Posterior	What we want

Worked example (`bayes_theorem()`): a disease affects 1% of people; the test catches 90% of cases and gives a false positive 5% of the time. You test positive — are you sick?

P(positive) = 0.90·0.01 + 0.05·0.99 = 0.0585, so P(sick | positive) = 0.009 / 0.0585 = **0.154**.

> P(disease | positive) = 0.0090 / 0.0585 = 0.154

Only 15%! The low prior dominates. This "base-rate" effect matters in fraud and medical ML too.

The naive assumption

For features $x_1 \ldots x_d$, the exact likelihood $P(x_1, \ldots, x_d \mid \text{class})$ needs a probability for every combination of feature values — impossible to estimate. Assuming independence given the class:

$P(x_1, \ldots, x_d \mid c) \approx P(x_1 \mid c) \cdot P(x_2 \mid c) \cdot \ldots \cdot P(x_d \mid c)$

So the classifier is: **$\hat{y} = \text{argmax}_c\ P(c) \cdot \Pi\ P(x_i \mid c)$**

In practice you **sum logs** instead of multiplying: $\log P(c) + \Sigma \log P(x_i \mid c)$. Multiplying hundreds of small probabilities underflows to 0 in floating point.

Gaussian NB, worked. For a continuous feature, `GaussianNB` models $P(x_i \mid c)$ with the normal density $N(x; \mu_c, \sigma_c^2) = \exp(-(x-\mu_c)^2/(2\sigma_c^2)) / \sqrt{(2\pi\sigma_c^2)}$, using the class's own mean and variance. Suppose petal length has $\mu = 1.5$, $\sigma = 0.2$ for class A and $\mu = 4.5$, $\sigma = 0.5$ for class B, with equal priors. For a flower with length 2.0:

- class A: $z = (2.0 - 1.5)/0.2 = 2.5 \rightarrow$ density = e^(−3.125)/(0.2·√(2π)) ≈ 0.0439/0.501 ≈ 0.088
- class B: $z = (2.0 - 4.5)/0.5 = -5 \rightarrow$ density = e^(−12.5)/(0.5·√(2π)) ≈ $3.7\times10^{-6}/1.253 \approx 3.0\times10^{-6}$ Class A wins overwhelmingly. With several features, you multiply (add the logs of) one such density per feature.

The assumption is almost always false ("free" and "cash" co-occur in spam), yet NB classifies well because it only needs the *ranking* of classes to be right, not the probabilities. The side effect: its probabilities are often **over-confident** — calibrate them if you need real probabilities.

The three main variants

Variant	Feature type	$P(x_i \mid c)$ modelled as	Typical use
`GaussianNB`	Continuous	Normal distribution with per-class mean and variance	Sensor readings, quick baseline on numeric data
`MultinomialNB`	Counts (word counts, TF-IDF also works in practice)	Word frequency within the class	Text classification, topic labelling
`BernoulliNB`	Binary (present / absent)	Probability the feature is present	Short texts, binary flags; it also penalises *absent* words
(`ComplementNB`)	Counts	Statistics from all *other* classes	Imbalanced text classes — often beats Multinomial
(`CategoricalNB`)	Categories	Per-category frequency	Categorical tabular data

Worked example — a spam filter by hand

Training data (6 messages):

Spam	Ham
win cash now	meeting at noon
win free prize now	lunch meeting tomorrow
free cash offer	project update at noon

Vocabulary: 13 distinct words. Each class has 10 words in total. New message: **"free cash for the meeting"**.

1. **Priors**: P(spam) = P(ham) = 3/6 = 0.5.
2. **Likelihoods with Laplace (add-one) smoothing**: P(word | c) = (count of word in c + 1) / (total words in c + vocabulary size). Without smoothing, one unseen word gives probability 0 and wipes out the whole product.
    - Spam: free (2+1)/23, cash (2+1)/23, meeting (0+1)/23
    - Ham: free (0+1)/23, cash (0+1)/23, meeting (2+1)/23

    - "for" and "the" never appeared in training → ignored (same as scikit-learn's vectoriser).
3. **Scores**: spam ∝ 0.5 · 3·3·1 / 23³, ham ∝ 0.5 · 1·1·3 / 23³ → ratio 9 : 3.
4. **Normalise**: P(spam | message) = 9 / 12 = **0.75**.

`code/naive_bayes_demo.py` computes it by hand and with scikit-learn:

```
vec = CountVectorizer()
X = vec.fit_transform(texts)
nb = MultinomialNB(alpha=1.0).fit(X, labels)       # alpha = Laplace smoothing
proba = nb.predict_proba(vec.transform([message]))[0]
```

```
vocabulary size = 13
by hand : P(spam|'free cash for the meeting') = 0.750, P(ham) = 0.250
sklearn : P(spam|'free cash for the meeting') = 0.750, P(ham) = 0.250
```

And on numeric data:

```
GaussianNB on iris, 5-fold CV accuracy = 0.953
BernoulliNB on binarised iris          = 0.753
```

Picking the variant that matches the feature type matters: throwing away detail by binarising costs 20 points.

Trade-offs

Strengths	Weaknesses
Extremely fast: one pass of counting; trains on millions of documents	Independence assumption → poor probability estimates
Works with very little data and very many features	Can't learn feature interactions ("not good")
Naturally multi-class, easy to update incrementally (`partial_fit`)	Usually beaten by logistic regression / linear SVM with enough data
Robust to irrelevant features	Gaussian variant assumes bell-shaped features

When not to use it

When features are strongly dependent and you have enough data for a discriminative model, when you need calibrated probabilities, or when interactions matter.

Common mistakes

- Forgetting smoothing (`alpha=0` → zero probabilities).
- Multiplying raw probabilities instead of adding logs (underflow).
- Feeding negative values to `MultinomialNB` (counts must be ≥ 0; standardised features break it).
- Trusting `predict_proba` values as real probabilities.

Real-world examples

- Email spam filtering: Paul Graham's 2002 essay *A Plan for Spam* popularised Bayesian filtering, and SpamAssassin includes a Bayesian classifier component.
- Quick baselines for sentiment analysis, news topic tagging, support-ticket routing, and language detection.

In the interview

30-second answer: "Naive Bayes applies Bayes' theorem — posterior ∝ prior × likelihood — and assumes features are independent given the class, so the likelihood is a product of per-feature probabilities. Multinomial NB for word counts, Bernoulli for binary features, Gaussian for continuous. Use Laplace smoothing and log probabilities. It's fast and strong on text with little data, but its probabilities are over-confident."

- *Why does it work if the assumption is wrong?* Classification only needs the argmax to be correct; errors in the probabilities often cancel or don't change the ranking.
- *What is Laplace smoothing?* Add α (usually 1) to every count so unseen words don't produce zero probability.
- *Generative vs discriminative?* NB is generative — it models P(x | y) and P(y). Logistic regression is discriminative — it models P(y | x) directly. NB learns faster from little data; LR usually wins with more.

Self-check

1. Prior 1%, sensitivity 90%, false-positive rate 5%: P(disease | +)? → ≈ 0.15.
2. Which NB for word counts? → `MultinomialNB`.
3. Why add logs? → Products of many small numbers underflow to zero.
4. A word appears 0 times in spam; α = 1, spam has 10 words, vocabulary 13. P(word | spam)? → 1/23.

TL;DR

Naive Bayes = Bayes' theorem + "features independent given class". Counting-fast, great text baseline; smooth, use logs, don't trust its probabilities.

8.4 Choosing a classic supervised algorithm

We ran every classic classifier in the book on four datasets with default settings (`code/compare_classifiers.py`, 5-fold CV accuracy):

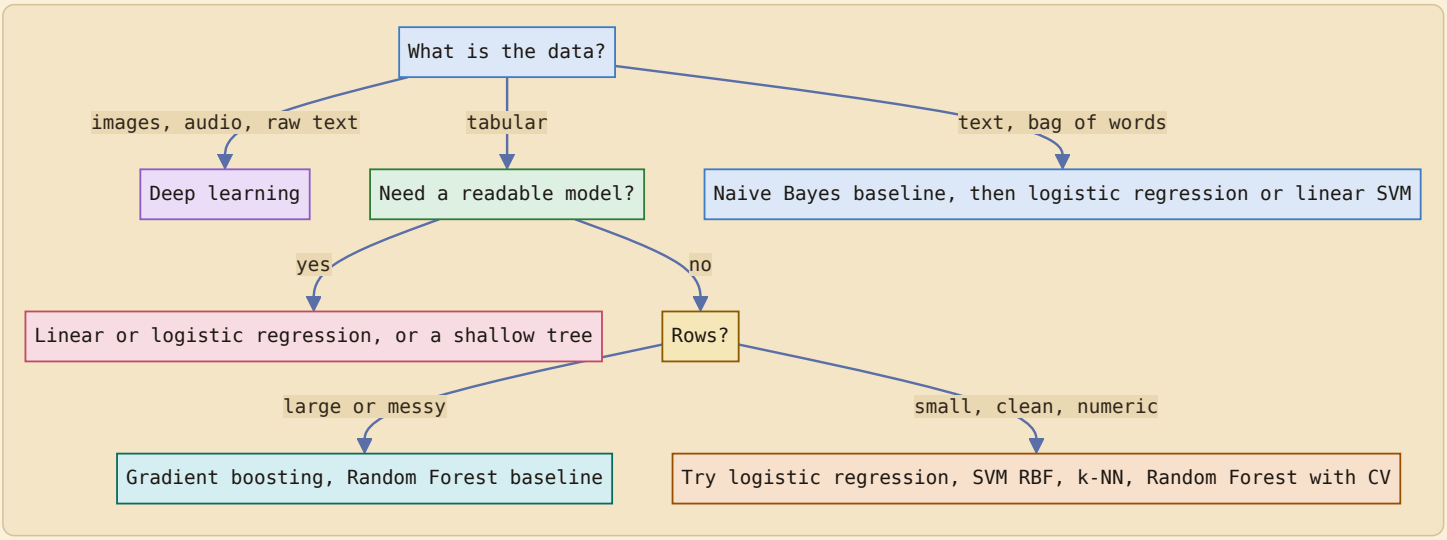
```
model                  breast_cancer    wine   digits  moons (nonlinear)
LogisticRegression             0.981   0.983    0.920              0.844
k-NN (k=5)                     0.965   0.949    0.944              0.886
SVM (RBF)                      0.974   0.983    0.946              0.898
GaussianNB                     0.939   0.966    0.807              0.840
DecisionTree                   0.917   0.888    0.786              0.850
RandomForest                   0.963   0.983    0.938              0.876
HistGradBoosting               0.965   0.955    0.935              0.872
```

Observations: no single winner (the "no free lunch" idea in practice). Logistic regression wins on the nearly linear breast-cancer data; the RBF SVM wins on the curved moons and on digits; a single decision tree is last almost everywhere; ensembles are consistently near the top. On small clean datasets like these, the differences are a few points — on large, messy tabular data, gradient boosting usually pulls ahead.

The comparison table

Algorithm	Type	Scale features?	Interpretable?	Training speed	Prediction speed	Non-linear?	Best for	Avoid when
Linear regression	Regression	Yes (if regularised)	High (coefficients)	Very fast	Very fast	No (unless you add features)	Baselines, linear relationships, explaining effects	Strong non-linearity, outliers (use robust loss)
Logistic regression	Classification	Yes	High	Very fast	Very fast	No	Baseline classifier, calibrated probabilities, high-dim sparse text	Complex interactions without feature engineering
k-NN	Both	**Yes**	Medium (show neighbours)	None	**Slow** ($O(n \cdot d)$)	Yes	Small low-dimensional data, similarity search, recommendations	Many features, big data, real-time latency
SVM (RBF)	Both (SVC/SVR)	**Yes**	Low	Slow for large n	Medium ($\propto$ support vectors)	Yes (kernels)	Small/medium data, many features, clear margins	> ~100k rows, need probabilities
Linear SVM	Classification	Yes	High	Very fast	Very fast	No	Text classification at scale	Non-linear problems
Naive Bayes	Classification	No	Medium	**Fastest**	Very fast	No (per-feature only)	Text, tiny data, streaming, baseline	Dependent features, need probabilities
Decision tree	Both	No	**High** (rules)	Fast	Very fast	Yes	Explainable rules, quick look at data	Accuracy matters (high variance)
Random Forest	Both	No	Low–medium (importances)	Medium (parallel)	Medium	Yes	Strong default, little tuning, robust	Tight latency/memory, extrapolation
Gradient boosting	Both	No	Low–medium (SHAP)	Medium (sequential)	Fast	Yes	**Best accuracy on tabular data**	Unstructured data, tiny data, no time to tune
Neural network (MLP)	Both	**Yes**	Low	Slow	Fast	Yes	Huge data, images/text/audio (with CNNs, Transformers)	Small tabular data

A decision flow



Practical rule: always start with a simple baseline (logistic/linear regression or Naive Bayes for text), then try a Random Forest and a gradient-boosting model, and compare with cross-validation.

Chapter TL;DR

- **k-NN**: predict from the k closest examples. Scale features, choose k by CV, prefer cosine for text/embeddings. Lazy: free training, slow prediction. Breaks down in high dimensions (distances concentrate).
- **SVM**: maximise the margin $2/\|w\|$; support vectors define it. C = strictness (large C → tight fit). Kernel trick = dot products in a hidden feature space; RBF default, γ = reach of each point. Scale; tune C and γ together; SVR uses an ε-tube. Slow beyond tens of thousands of rows.
- **Naive Bayes**: posterior ∝ prior × product of per-feature likelihoods. Multinomial (counts), Bernoulli (binary), Gaussian (continuous). Laplace smoothing + log probabilities. Fast and good for text; probabilities over-confident.
- **Choosing**: no universal winner — baseline first, compare with CV; gradient boosting for tabular, deep learning for unstructured data.

Chapter 9 — Unsupervised Learning

In supervised learning every example came with an answer (a label). **Unsupervised learning** works with **no labels**: the goal is to find structure in the data itself. The four big jobs:

Job	Question it answers	Main algorithms
Clustering	Which examples belong together?	k-means, hierarchical, DBSCAN, Gaussian mixtures
Dimensionality reduction	Can I describe the data with fewer numbers, or see it in 2-D?	PCA, t-SNE, UMAP
Anomaly detection	Which examples don't fit?	z-score, Isolation Forest, One-Class SVM, LOF
Association rules	Which items occur together?	Apriori, FP-Growth

The hard part is always **evaluation**: without labels, what does "good" mean? We finish with that.

What you'll learn

1. k-means step by step, k-means++, choosing k with the elbow and silhouette, and where k-means fails.
2. Hierarchical (agglomerative) clustering and dendrograms; DBSCAN; Gaussian mixture models and the EM idea.
3. PCA with the maths made concrete; t-SNE and UMAP and when to use which.
4. Anomaly detection from simple statistics to Isolation Forest, One-Class SVM and LOF.
5. Association rules with Apriori.
6. How to evaluate unsupervised models.

All code is in `code/`, all figures in `img/`; outputs are real (scikit-learn 1.9, SciPy 1.18).

9.1 k-means clustering

What it is

k-means splits the data into k groups (**clusters**) so that each point belongs to the cluster with the nearest **centroid** (the cluster's mean point). You choose k.

Analogy: placing k pizza shops in a town so every household walks to its nearest shop and total walking is as small as possible. Then you move each shop to the middle of its customers, reassign customers, and repeat.

Why it matters

It is simple, fast (roughly O(n·k·d) per iteration), scales to millions of points, and is the default first clustering method: customer segmentation, grouping documents, compressing colours in an image (colour quantisation), building "visual words" or codebooks, and initialising other models.

How it works (Lloyd's algorithm)

k-means minimises the **inertia** (within-cluster sum of squares, WCSS): the sum of squared distances from each point to its centroid.

1. **Initialise** k centroids (randomly chosen points, or k-means++ — below).
2. **Assign** each point to its nearest centroid.
3. **Update** each centroid to the mean of its assigned points.
4. **Repeat** 2–3 until assignments stop changing.

Each step can only lower (or keep) the inertia, so it always converges — but to a **local** minimum that depends on the starting centroids.

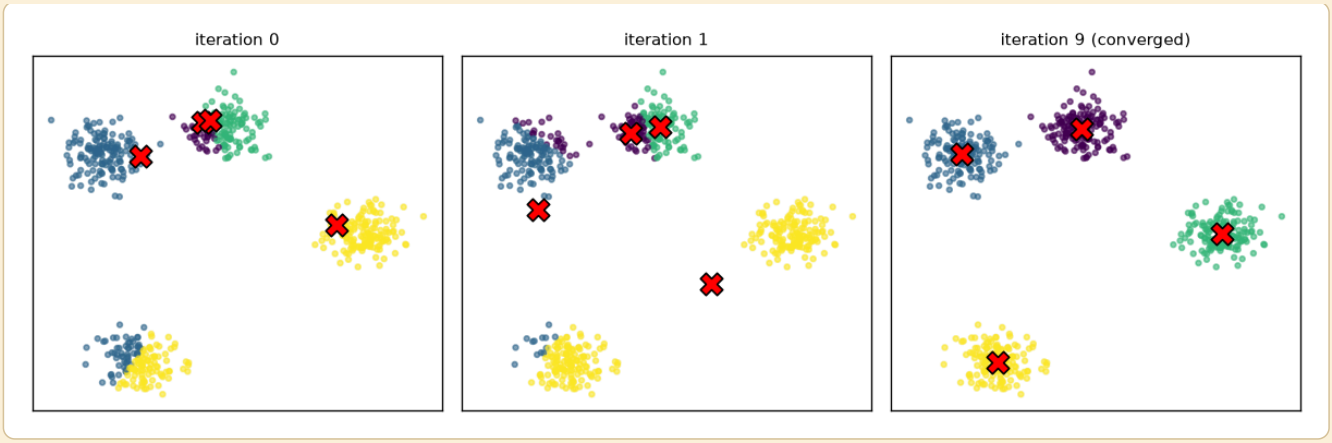
Worked example (1-D). Points 1, 2, 3, 10, 11, 12; k = 2; start with centroids 1 and 2.

- Assign: {1} → c_1; {2, 3, 10, 11, 12} → c_2. Update: $c_1 = 1$, $c_2 = (2+3+10+11+12)/5 = 7.6$.
- Assign: 3 is 2 from c_1 and 4.6 from c_2 → {1, 2, 3} and {10, 11, 12}. Update: $c_1 = 2$, $c_2 = 11$.
- Assign again: nothing changes → converged. Inertia = $(1^2 + 0 + 1^2) + (1^2 + 0 + 1^2) = 4$.

From scratch in NumPy (`code/kmeans_demo.py`):

```
def kmeans_scratch(X, k, rng, max_iter=100):
    centroids = X[rng.choice(len(X), k, replace=False)]          # 1. random init
    for it in range(max_iter):
        d = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
        labels = d.argmin(axis=1)                                # 2. assign to nearest centroid
        new = np.array([X[labels == j].mean(axis=0) for j in range(k)])  # 3. move centroids to means
        if np.allclose(new, centroids):                          # 4. stop when nothing moves
            break
        centroids = new
    inertia = ((X - centroids[labels]) ** 2).sum()
    return labels, centroids, inertia
```

```
from scratch: converged in 9 iterations, inertia = 1160
sklearn     : inertia = 1160
```



In this run, two starting centroids fell into the same blob and the bottom blob had none; the algorithm still recovered in 9 iterations. It is not always that lucky.

k-means++ initialisation

Bad starts give bad local minima. **k-means++** (Arthur & Vassilvitskii, 2007) spreads the initial centroids out:

1. Pick the first centroid uniformly at random from the data.
2. For each point, compute D(x) = distance to the nearest centroid chosen so far.
3. Pick the next centroid at random with probability **proportional to D(x)²** — far points are much more likely.
4. Repeat until k centroids; then run normal k-means.

It comes with a guarantee: the expected result is within O(log k) of the optimal inertia. On 8 well-separated blobs, with a single run per seed:

```
init          inertia over 20 single-run seeds, 8 true clusters (min / mean / max)
random            1673 /     9002 /    32908
k-means++          704 /      704 /      704
```

Random init never found the right answer in 20 tries; k-means++ found it every time. scikit-learn uses `init="k-means++"` by default, and `n_init` restarts keep the best run.

Choosing k: elbow and silhouette

Elbow method: plot inertia against k. Inertia always falls as k grows (k = n gives 0), so look for the "elbow" where adding a cluster stops helping much.

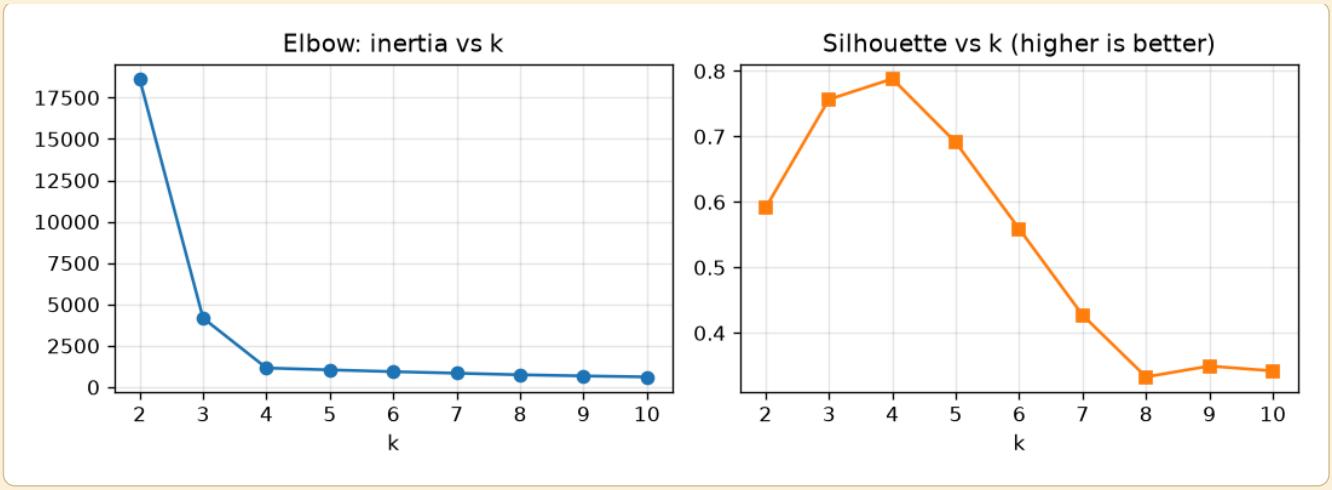
Silhouette score: for each point,

- a = mean distance to the other points in its own cluster (cohesion),
- b = mean distance to the points of the **nearest other** cluster (separation),
- s = (b − a) / max(a, b), between −1 and 1.

s ≈ 1: well inside its cluster; s ≈ 0: on a border; s < 0: probably in the wrong cluster. The **silhouette score** is the average over all points. In the 1-D example, point 3 has a = (2 + 1)/2 = 1.5, b = (7 + 8 + 9)/3 = 8, so s = (8 − 1.5)/8 = 0.81.

On 4 true blobs:

```
 k   inertia   silhouette
 2     18638   0.591
 3      4179   0.757
 4      1160   0.788
 5      1046   0.691
 6       938   0.559
```



Both point to k = 4: the inertia curve bends sharply there and the silhouette peaks. On real data the elbow is often vague — then prefer silhouette, domain knowledge (e.g. "marketing can handle 5 segments"), and stability across runs.

Limits of k-means

k-means implicitly assumes clusters are **round (spherical), similar in size and spread, and separated by straight-line boundaries** (each point goes to the nearest mean). It fails on:

- **Non-convex shapes** (moons, rings) — see the comparison in §9.4.
- **Elongated or differently-spread clusters** — a big diffuse cluster gets cut up.
- **Outliers** — they drag the means.
- **Unscaled features** — distance-based, so always standardise.
- **Categorical data** — means are meaningless (use k-modes or k-prototypes).
- You must choose k.

Common mistakes

- Not scaling features.
- Trusting one run with random init.
- Using the elbow on data that has no clusters — k-means will always return k groups, even from uniform noise.

In the interview

> **30-second answer:** "k-means picks k centroids, assigns each point to the nearest, moves each centroid to the mean of its points, and repeats until nothing changes. It minimises within-cluster squared distance and converges to a local minimum, so we use k-means++ initialisation and multiple restarts. We choose k with the elbow or silhouette score. It assumes round, similar-sized clusters and needs scaled features."

- *Why k-means++?* It spreads initial centroids (probability ∝ squared distance), giving faster convergence and better, more consistent results.
- *Complexity?* O(n·k·d) per iteration; Mini-Batch k-means for very large data.
- *Does k-means always converge?* Yes (inertia never increases and there are finitely many assignments), but only to a local optimum.

Self-check

1. What does k-means minimise? → Sum of squared distances from points to their cluster centroid (inertia).
2. Why does inertia always drop as k grows? → More centroids can only sit closer to points; k = n gives 0.
3. Silhouette with a = 2, b = 6? → (6 − 2)/6 = 0.67.
4. Name two shapes k-means handles badly. → Moons/rings (non-convex) and elongated clusters.

TL;DR

k-means = assign to nearest mean, move means, repeat. Scale data, use k-means++ with restarts, pick k with silhouette/elbow, and remember it assumes round clusters.

9.2 Hierarchical (agglomerative) clustering

What it is

Agglomerative clustering builds a hierarchy bottom-up: start with every point as its own cluster, then repeatedly **merge the two closest clusters** until one remains. The full merge history is drawn as a **dendrogram** — a tree whose vertical axis shows the distance at which clusters merged. Cut the tree at a height to get clusters.

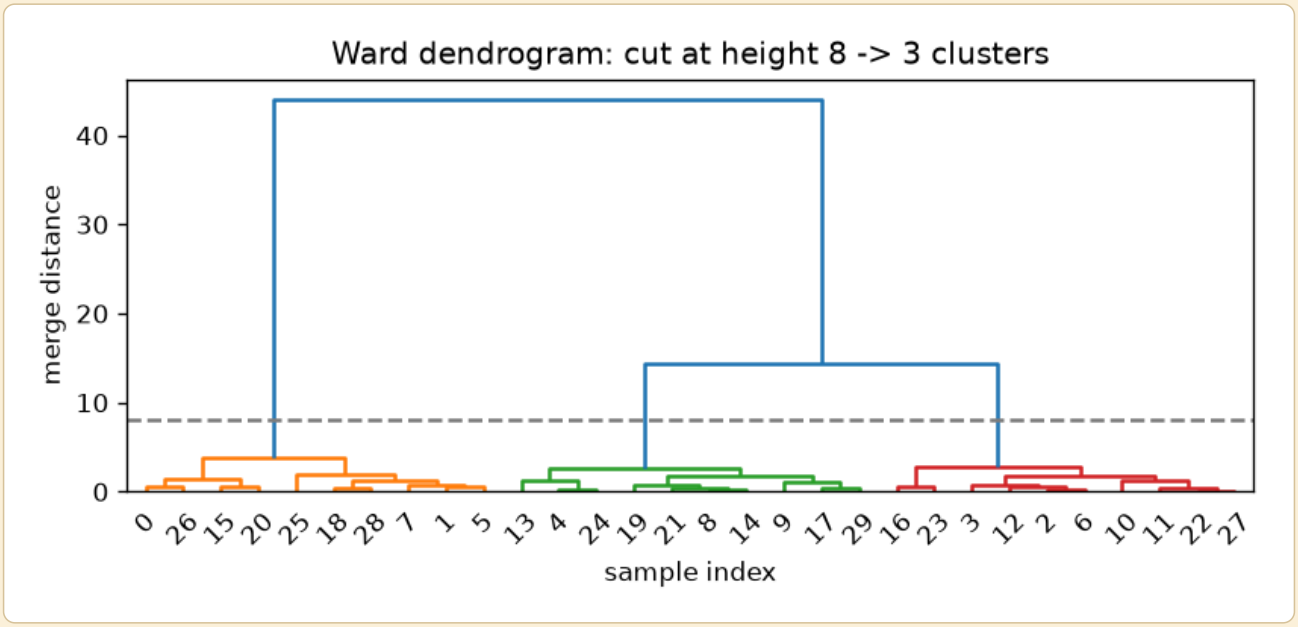
How it works

1. Compute pairwise distances; each point is a cluster.
2. Merge the two clusters with the smallest **linkage** distance.
3. Update distances between the new cluster and the rest; repeat until one cluster.
4. Choose clusters by cutting the dendrogram at a height (or asking for n clusters).

Linkage	Distance between clusters A and B	Behaviour
Single	Closest pair of points	Follows chains — finds snakes and moons, but a thin bridge of noise merges clusters ("chaining")
Complete	Farthest pair of points	Compact clusters of similar diameter
Average	Mean of all pairwise distances	Compromise
Ward (default in scikit-learn)	Increase in total within-cluster variance caused by the merge	Similar to k-means: round, balanced clusters

```
Z = linkage(X, method="ward")          # SciPy: the full merge history
dendrogram(Z, color_threshold=8)
```

```
last 3 merge heights: [ 3.74 14.37 44.1 ]  (big jump before the final merges = natural cut)
```



Merges below height ~4 are within groups; the jump to 14.37 and 44.1 shows those merges join genuinely different groups. Cutting in the big gap (e.g. at 8) gives 3 clusters.

Trade-offs

Strengths	Weaknesses
No need to fix k in advance — one tree, any number of clusters	O(n²) memory for the distance matrix; slow beyond ~10–20k points
Dendrogram is an interpretable picture of nested structure	Merges are greedy and never undone
Any distance metric; linkage choice adapts to shapes	Sensitive to noise (single) or biased to round shapes (Ward)

Real-world examples

Phylogenetic trees in biology, gene-expression heatmaps (clustered rows/columns), organising documents or products into taxonomies.

In the interview

30-second answer: "Agglomerative clustering starts with each point alone and repeatedly merges the closest pair of clusters, where 'closest' is defined by the linkage — single, complete, average or Ward. The dendrogram records the merges; cutting it at a height gives the clusters. It's great for small data and nested structure, but it's O(n²) in memory."

- *Which linkage for moon-shaped clusters?* Single linkage (it chains), as long as there's little noise between them.

Self-check

1. What does the height of a dendrogram merge mean? → The linkage distance at which the two clusters were joined.
2. Which linkage behaves most like k-means? → Ward.

TL;DR

Merge closest clusters bottom-up; read the dendrogram; cut at a big gap. Linkage decides the cluster shapes. Small data only.

9.3 DBSCAN — density-based clustering

What it is

DBSCAN (Density-Based Spatial Clustering of Applications with Noise, Ester et al. 1996) defines clusters as **dense regions separated by sparse regions**. It finds clusters of any shape, figures out how many there are, and labels isolated points as **noise** (label −1).

Analogy: people at a festival. Crowds are clusters — however oddly shaped — and the lone wanderers between them are noise.

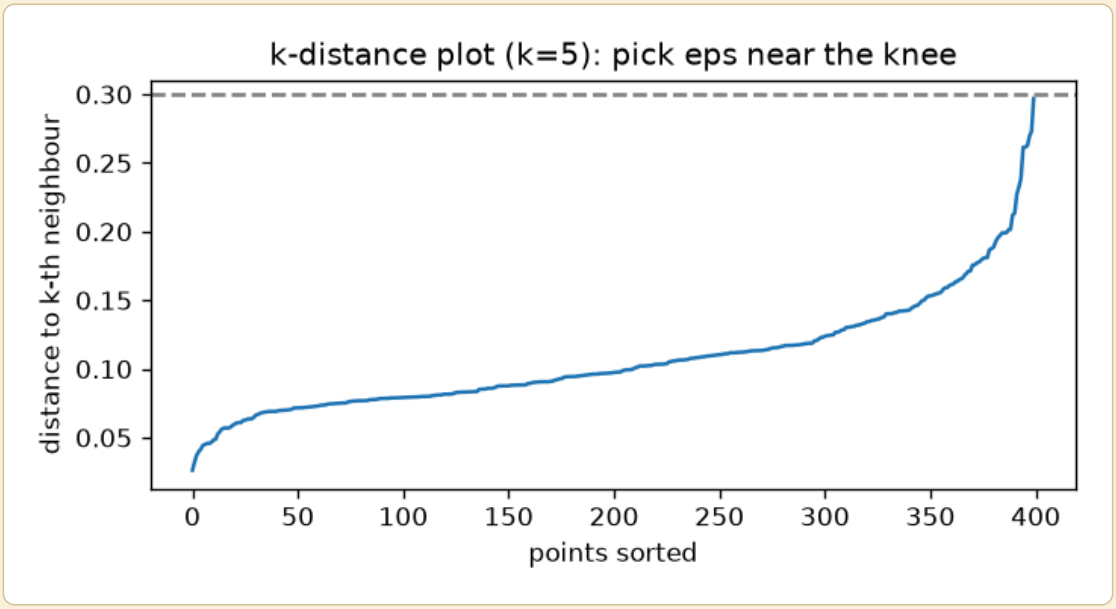
How it works

Two parameters: **eps** (neighbourhood radius) and **min_samples** (points needed to be "dense"; in scikit-learn this count includes the point itself).

1. A **core point** has at least `min_samples` points within distance eps.
2. A **border point** is within eps of a core point but is not core itself.
3. A **noise point** is neither.
4. Start at an unvisited core point; its cluster is everything **density-reachable** from it (chains of core points each within eps of the next, plus their border points). Repeat for unvisited core points.

Choosing eps: the k-distance plot

Compute each point's distance to its k-th nearest neighbour (k = min_samples), sort, plot. The "knee" where the curve shoots up separates points inside dense regions from outliers; pick eps near it.




```
DBSCAN on moons (min_samples=5)
  eps=0.1  clusters=28  noise points=84
  eps=0.2  clusters=2  noise points=1
  eps=0.3  clusters=2  noise points=0
  eps=0.5  clusters=2  noise points=0
  eps=1.0  clusters=1  noise points=0
```

Too small an eps shatters the data into 28 fragments plus noise; too large merges everything into one cluster. A rule of thumb for min_samples is about 2 × number of features (at least 3–5), larger for noisy data.

Trade-offs

Strengths	Weaknesses
Any cluster shape; no k needed	One global density: fails when clusters have **different densities**
Built-in noise/outlier detection	Sensitive to eps; eps hard to choose in high dimensions
Deterministic for core points	Distance-based: needs scaling, suffers from curse of dimensionality

HDBSCAN (available as `sklearn.cluster.HDBSCAN` since 1.3) extends the idea to varying densities and removes eps, needing mainly `min_cluster_size` — often the better modern choice. **OPTICS** is a related alternative.

Real-world examples

Clustering GPS points into places of interest, grouping geospatial events (crime, taxi pickups), spotting outliers in sensor data.

In the interview

> **30-second answer:** "DBSCAN groups points in dense regions: a core point has at least min_samples neighbours within eps; clusters are chains of core points plus their border points; everything else is noise. It finds arbitrary shapes and the number of clusters automatically, but struggles with clusters of different densities — HDBSCAN addresses that."

Self-check

1. What label does DBSCAN give noise? → −1.
2. Increasing eps a lot does what? → Merges clusters, eventually into one.
3. Why might DBSCAN fail with a tight cluster next to a diffuse one? → A single eps can't fit both densities.

TL;DR

DBSCAN = clusters are dense regions; noise is labelled. Choose eps from a k-distance plot; use HDBSCAN for mixed densities.

9.4 Gaussian mixture models (GMM) and EM

What it is

A **Gaussian mixture model** assumes the data was generated by k Gaussian (bell-shaped) distributions, each with its own **mean** (centre), **covariance** (shape and orientation — can be stretched and tilted ellipses) and **weight** (share of the data). Clustering = figuring out which Gaussian probably produced each point.

Unlike k-means' hard assignment, GMM gives **soft assignments**: "this point is 43% cluster A, 35% cluster C, 21% cluster B".

How it works — Expectation-Maximisation (EM), intuitively

We don't know the Gaussians' parameters, and we don't know which Gaussian each point came from. If we knew one, the other would be easy. EM alternates:

1. **Initialise** means, covariances and weights (scikit-learn uses k-means by default).
2. **E-step (Expectation):** for every point, compute its **responsibility** under each Gaussian — the probability it came from that component, given current parameters (Bayes' rule: weight × density, normalised).
3. **M-step (Maximisation):** re-estimate each Gaussian using all points **weighted by responsibility**: weighted mean, weighted covariance, and weight = average responsibility.
4. Repeat until the log-likelihood stops improving.

EM never decreases the likelihood, but like k-means it finds a local optimum (use `n_init` > 1). In fact, k-means is a limiting case of GMM with hard assignments and identical round (spherical) covariances.

Choosing k: BIC

GMMs are probabilistic models, so you can use the **Bayesian Information Criterion** (BIC) — likelihood penalised by the number of parameters; **lower is better**. (`gmm.bic(X)`, also `aic`.)

```
GMM model selection by BIC (lower is better)
  k=1  BIC=  4797.3
  k=2  BIC=  4514.9
  k=3  BIC=  4513.3
  k=4  BIC=  4550.7
soft assignments: 103 of 500 points have max membership < 0.9, e.g. [0.43 0.21 0.35]
EM converged in 4 iterations; mixing weights [0.33 0.4  0.27]
```

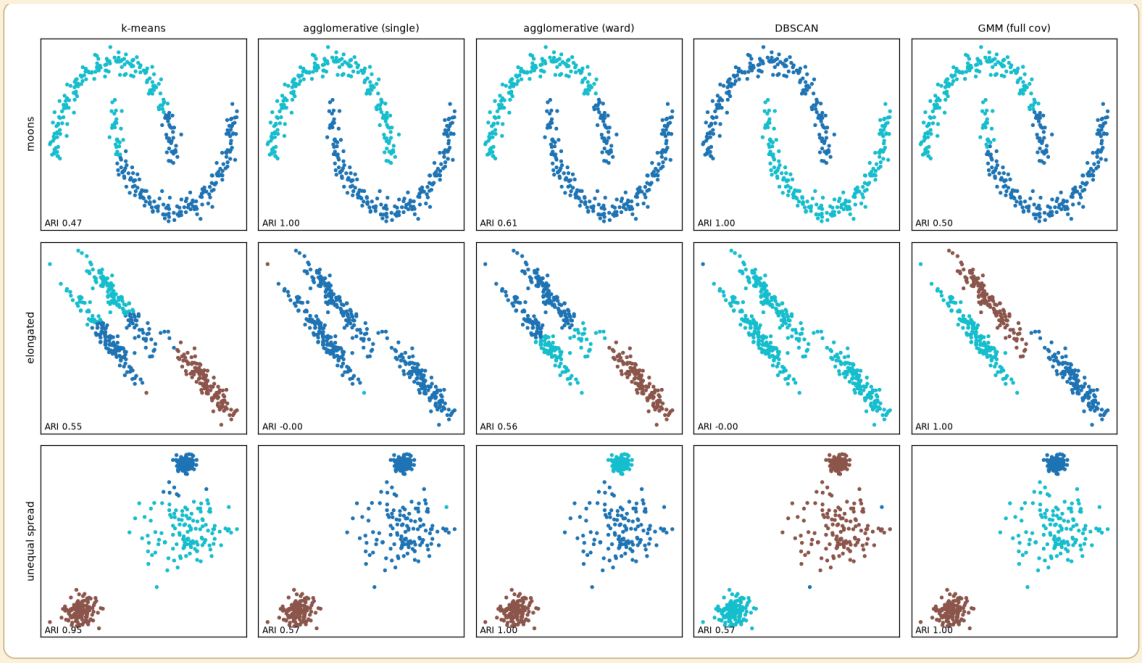
BIC picks k = 3 (the truth), but only narrowly over k = 2 because two of the three generated blobs overlap — and ~20% of points are genuinely ambiguous, which the soft assignments reveal honestly.

`covariance_type` controls flexibility: `"full"` (any ellipse per component), `"tied"` (all share one), `"diag"` (axis-aligned ellipses), `"spherical"` (circles, like k-means).

Comparing all four clusterers

`code/clustering_shapes.py` runs each method on three awkward datasets and scores against the true grouping with the **Adjusted Rand Index** (ARI: 1 = perfect agreement, ≈ 0 = random):

```
Adjusted Rand Index               moons       elongated  unequal spread
k-means                            0.47            0.55            0.95
agglomerative (single)             1.00           -0.00            0.57
agglomerative (ward)               0.61            0.56            1.00
DBSCAN                             1.00           -0.00            0.57
GMM (full cov)                     0.50            1.00            1.00
```



- **Moons** (non-convex): only density/chaining methods (DBSCAN, single linkage) succeed.
- **Elongated** clusters close together: only GMM with full covariance models the stretched ellipses; DBSCAN and single linkage merge everything through the dense bridge.
- **Unequal spread**: GMM and Ward are perfect; DBSCAN's one eps can't fit both the tight and diffuse clusters.

No algorithm wins everywhere — pick based on the shapes you expect, and look at the result.

When to use which clustering algorithm

Algorithm	Needs k?	Cluster shapes	Noise handling	Scales to	Pick it when
k-means	Yes	Round, similar size	None (outliers distort)	Millions (Mini-Batch)	Fast default, compact clusters
Agglomerative	Yes or cut height	Depends on linkage	Poor	~10k	Small data, want a hierarchy
DBSCAN / HDBSCAN	No	Any	Labels noise	~100k+ (with index)	Odd shapes, outliers, unknown k
GMM	Yes (choose by BIC)	Ellipses, overlapping	None	~100k	Need soft probabilities, elongated clusters

In the interview

30-second answer: "A GMM models the data as a weighted mix of Gaussians, each with its own mean and covariance, and fits them with EM: the E-step computes each point's probability of belonging to each component, the M-step re-estimates the parameters with those weights. It gives soft assignments and elliptical clusters; you choose k with BIC. k-means is a special case with hard assignments and spherical clusters."

Self-check

1. What does the E-step compute? → Responsibilities: P(component | point) for every point and component.
2. Lower or higher BIC is better? → Lower.
3. When would you choose GMM over k-means? → Elliptical or overlapping clusters, or when you need membership probabilities.

TL;DR

GMM = mixture of Gaussians fitted with EM; soft, elliptical clusters; choose k with BIC.

9.5 Dimensionality reduction: PCA

What it is

Principal Component Analysis finds new axes — **principal components** — that are straight-line (linear) combinations of the original features, ordered so the first captures the **most variance** (spread) in the data, the second the most of what's left while being perpendicular to the first, and so on. Keep the top few and you compress the data with minimal loss.

Analogy: photographing a teapot. From some angles the photo shows almost everything about its shape; from others (straight down the spout) you lose a lot. PCA finds the most informative "camera angle".

Why it matters

- **Compression / speed**: fewer features → faster training, less memory.
- **Noise reduction**: the small-variance directions are often noise.
- **Visualisation**: project to 2-D or 3-D.
- **Removing correlation** (multicollinearity): components are uncorrelated.
- Fights the **curse of dimensionality** before distance-based methods (k-NN, clustering).

How it works — the maths made concrete

1. **Centre** each feature (subtract its mean). **Standardise** too if features have different units.
2. Compute the **covariance matrix** C (d × d): how each pair of features varies together.
3. Find C's **eigenvectors** (directions) and **eigenvalues** (variance along each direction). An eigenvector v satisfies C·v = λ·v — C only stretches it.
4. Sort by eigenvalue; the **explained variance ratio** of component i is λ_i / Σλ.
5. **Project**: new coordinates Z = X_centred · V_k (the top-k eigenvectors as columns).

(In practice libraries compute the same thing with the **singular value decomposition** (SVD) of the centred data, which is more numerically stable; scikit-learn does this.)

Worked example — hours studied vs exam score for 5 students (`code/dim_reduction.py`):

```
X = np.array([[2, 50], [4, 60], [6, 70], [8, 85], [10, 95]], dtype=float)
Xs = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)   # centre and scale
C = np.cov(Xs, rowvar=False)                          # covariance matrix
vals, vecs = np.linalg.eigh(C)                        # eigen-decomposition
z = Xs @ vecs[:, 0]                                   # project onto PC1
```

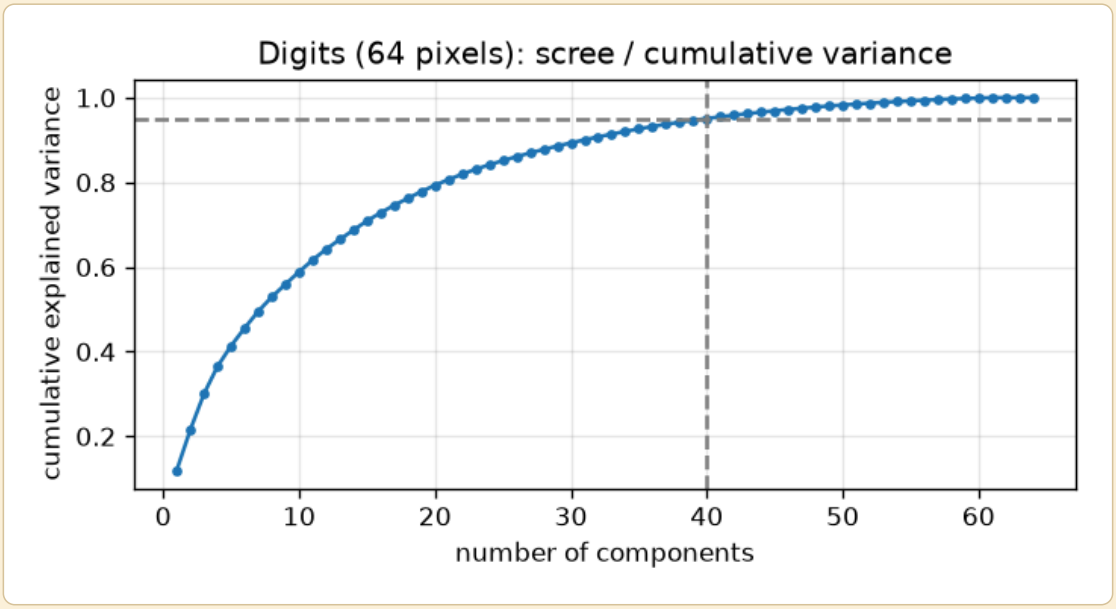
```
covariance (standardised):
 [[1.    0.997]
 [0.997 1.   ]]
eigenvalues: [1.997 0.003]  explained variance ratio: [0.999 0.001]
PC1 direction: [0.707 0.707]
students on PC1: [-1.75 -0.91 -0.08  0.95  1.79]
sklearn explained_variance_ratio_: [0.999 0.001]
```

For two standardised features with correlation r, the eigenvalues are always 1 + r and 1 − r, with directions (1, 1)/√2 and (1, −1)/√2. Here r = 0.997, so PC1 ("overall diligence") holds 99.9% of the variance: two columns compress to one number per student with almost no loss. (The sign of an eigenvector is arbitrary; we flipped it so bigger = more.)

How many components? Explained variance

On the 8×8 digit images (64 pixel features):

```
components for 50% variance:  8 of 64
components for 80% variance: 21 of 64
components for 90% variance: 31 of 64
components for 95% variance: 40 of 64
components for 99% variance: 54 of 64
PCA(n_components=0.95) keeps 40 components, reconstruction MSE = 0.047
```



Common choices: keep enough components for 90–99% of variance (`PCA(n_components=0.95)` does it for you), look for an elbow in the **scree plot** (eigenvalues vs component number), or tune the number by cross-validating the downstream model. `inverse_transform` maps back to the

original space; the **reconstruction error** tells you what was lost.

Variants

- **Incremental PCA**: data that doesn't fit in memory (mini-batches).
- **Randomized PCA** (`svd_solver="randomized"`): fast approximate top components for large matrices.
- **Kernel PCA**: non-linear PCA via the kernel trick (Chapter 8).
- **TruncatedSVD**: PCA-like for sparse matrices (text), without centring.

Trade-offs / when not to use it

- Only captures **linear** structure — curved manifolds (a rolled-up "Swiss roll") need non-linear methods.
- Components are mixtures of all features → harder to interpret.
- Sensitive to scaling and outliers.
- Max variance ≠ most useful for prediction: a low-variance direction may be the one that separates classes (for supervised reduction, see LDA).

Common mistakes

- Forgetting to standardise features with different units (the large-unit feature becomes PC1).
- Fitting PCA on the full dataset before splitting (leakage) — put it in the pipeline.
- Reading t-SNE-like meaning into a 2-component PCA plot that explains only 20% of the variance.

Real-world examples

Eigenfaces for face recognition (Turk & Pentland, 1991); compressing embeddings before vector search; population-structure plots in genetics; removing correlated features before regression; image compression.

In the interview

> **30-second answer:** "PCA centres (and usually scales) the data, computes the covariance matrix, and takes its eigenvectors ordered by eigenvalue. Each eigenvalue is the variance along that direction, so the explained-variance ratio tells you how much information each component keeps. You project onto the top k — chosen for, say, 95% variance — to compress, denoise or visualise. It's linear, unsupervised, and sensitive to scaling."

- *PCA vs SVD?* PCA is usually computed via SVD of the centred data matrix; the right singular vectors are the principal directions and squared singular values/(n−1) are the eigenvalues.
- *Does PCA help every model?* No — trees don't need it, and it can discard low-variance but predictive directions.

Self-check

1. What do eigenvalues represent in PCA? → Variance along each principal component.
2. Two standardised features with correlation 0.8 — PC1's explained variance ratio? → (1 + 0.8)/2 = 0.9.
3. Why scale before PCA? → Otherwise the feature with the largest units dominates the variance.

TL;DR

PCA rotates the data onto uncorrelated axes of decreasing variance; keep the top ones. Scale first; choose k by explained variance; linear only.

9.6 t-SNE and UMAP — non-linear visualisation

What they are

t-SNE (t-distributed Stochastic Neighbor Embedding, van der Maaten & Hinton, 2008) and **UMAP** (Uniform Manifold Approximation and Projection, McInnes et al., 2018) are **non-linear** methods designed mainly for **visualising** high-dimensional data in 2-D or 3-D. They try to keep **neighbours as neighbours**: points close in the original space should land close in the map.

How t-SNE works (intuition)

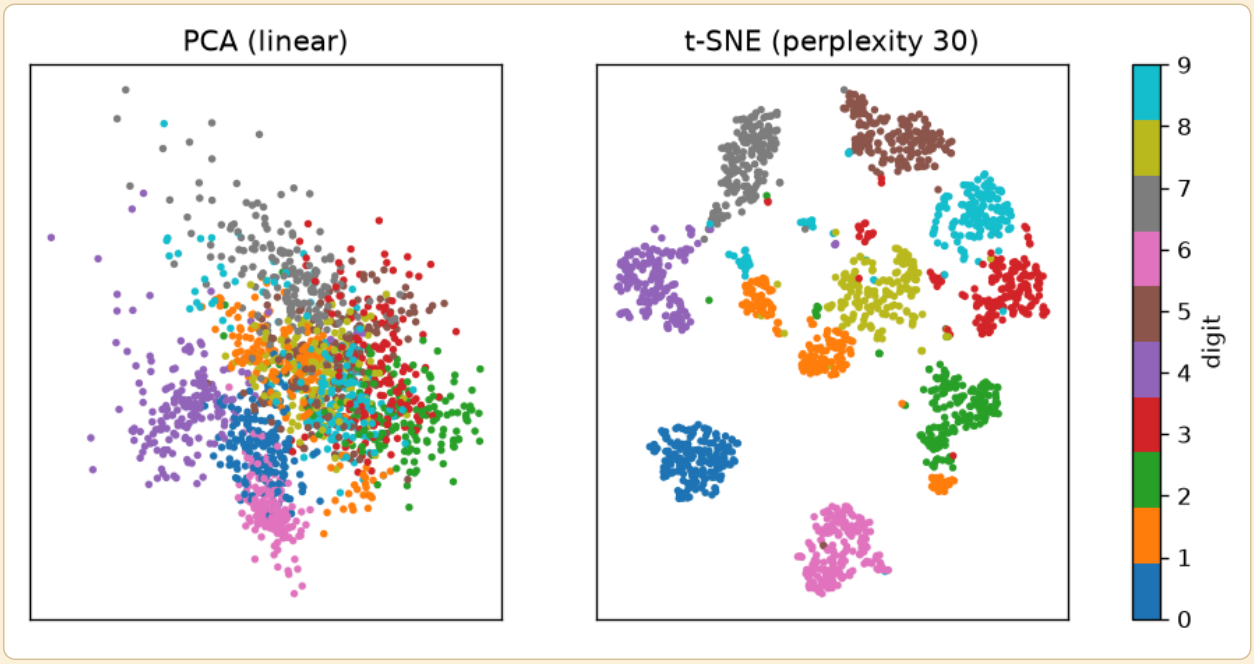
1. In the original space, turn distances into probabilities: for each point, nearby points get high "I'd pick you as a neighbour" probability (Gaussian kernel). **Perplexity** (typically 5–50) sets roughly how many neighbours each point considers.
2. In 2-D, define similar probabilities using a heavy-tailed **Student-t** distribution — the heavy tails let moderately distant points be pushed far apart, avoiding everything crowding into the middle.
3. Move the 2-D points by gradient descent to make the two sets of probabilities match (minimise KL divergence).

UMAP builds a weighted k-nearest-neighbour graph of the data (`n_neighbors`), then optimises a low-dimensional layout whose graph matches it (`min_dist` controls how tightly points pack). It is typically faster than t-SNE on large datasets, tends to keep more of the global arrangement, and can `transform` new points.

Code — PCA vs t-SNE on digits

```
Z_pca = PCA(n_components=2, random_state=0).fit_transform(Xs)
Z_tsne = TSNE(n_components=2, perplexity=30, init="pca", random_state=0).fit_transform(
    PCA(n_components=30, random_state=0).fit_transform(Xs))   # common trick: PCA to ~30-50 dims first
print(trustworthiness(Xs, Z, n_neighbors=5))
```

```
trustworthiness (k=5) PCA  : 0.818
trustworthiness (k=5) t-SNE: 0.992
```



PCA's best linear 2-D view mixes most digits; t-SNE separates all ten into distinct islands — without ever seeing the labels (colour is added only for the plot). **Trustworthiness** (0–1) measures how many of each point's neighbours in the 2-D map were also true neighbours in the original space.

UMAP code (the `umap-learn` package is not installed in the book environment — not run):

```
# pip install umap-learn   (not run)
import umap
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0)
Z = reducer.fit_transform(Xs)
Z_new = reducer.transform(X_new_scaled)   # UMAP can embed new points; sklearn's TSNE cannot
```

Reading t-SNE/UMAP plots safely

- **Cluster sizes mean nothing** — t-SNE expands dense clusters and contracts sparse ones.
- **Distances between clusters mean little** — two islands far apart are not necessarily "more different".
- **Results change with perplexity/n_neighbors and the random seed** — try several.
- **Don't cluster on t-SNE output** or use it as model features — it's for looking, not measuring. (UMAP is more often used as a pre-processing step for clustering, with care.)

PCA vs t-SNE vs UMAP

	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Preserves	Global variance	Local neighbourhoods	Local, and more global structure than t-SNE
Speed	Very fast	Slow on big data (Barnes-Hut O(n log n))	Fast
New points	`transform`	No (in scikit-learn)	`transform`
Deterministic	Yes	No (seed)	No (seed)
Use for	Compression, pre-processing, denoising, quick look	2-D visualisation of clusters	Visualisation, sometimes pre-processing for clustering
Key knobs	n_components	perplexity	n_neighbors, min_dist

Rule of thumb: PCA first, always (fast, interpretable, and a good pre-step to reduce to ~50 dimensions). Use t-SNE or UMAP to **look** at the structure; prefer UMAP for large datasets or when you need to embed new points.

In the interview

> **30-second answer:** "PCA is a linear projection that keeps global variance — good for compression and pre-processing. t-SNE and UMAP are non-linear and keep local neighbourhoods, so they're great for 2-D visualisation of clusters, but cluster sizes and inter-cluster distances in the plot aren't meaningful and the output depends on hyperparameters. UMAP is faster and can transform new data."

Self-check

1. What does perplexity control? → Roughly the effective number of neighbours each point considers.
2. Can you trust the gap between two t-SNE clusters as a distance? → No.
3. Which of the three can embed new points out of the box in scikit-learn/umap-learn? → PCA and UMAP.

TL;DR

PCA to compress; t-SNE/UMAP to see. Only local neighbourhoods in t-SNE/UMAP plots are trustworthy.

9.7 Anomaly detection

What it is

Anomaly (outlier) detection finds data points that differ markedly from the majority: fraudulent transactions, failing machines, network intrusions, data-entry errors. Usually anomalies are **rare and unlabelled**, so this is mostly unsupervised.

Two settings (scikit-learn terms): **outlier detection** — the training data itself contains outliers to find; **novelty detection** — train on clean data, flag new points that look different.

Statistical baseline: z-score and robust z-score

z-score: z = (x − mean) / std; flag |z| > 3. Simple — but the outlier itself inflates the mean and std (**masking**). A more robust version uses the median and the **MAD** (median absolute deviation): modified z = 0.6745·(x − median)/MAD, flag |modified z| > 3.5 (Iglewicz & Hoaglin). The box-plot **IQR rule** (flag outside Q1 − 1.5·IQR, Q3 + 1.5·IQR) is another robust option.

```
amount   z-score   robust z
    42     -0.32      -0.54
    ...
    60     -0.25       1.89
   950      3.32     121.95  <- anomaly
mean=121.7 (dragged up by the outlier), median=46.0
```

The $950 payment only just passes z = 3. In fact, with population standard deviation, no point in a sample of n can ever have |z| above √(n − 1) — here √11 = 3.32 — so with small samples the plain z-score **cannot** flag extreme values reliably. The robust z-score says 122. Statistical rules work for one feature at a time; for many features, use the methods below.

Isolation Forest

Idea (Liu, Ting & Zhou, 2008): anomalies are **few and different, so they are easy to isolate**. Build many random trees: at each node pick a random feature and a random split value between its min and max. A normal point in a dense region needs many splits before it's alone in a leaf; an anomaly gets isolated in a few.

1. Build ~100–200 **isolation trees**, each on a small random subsample (default 256 rows).
2. For each point, measure the average **path length** from root to its leaf.
3. Convert to an anomaly score in (0, 1): close to 1 = anomaly, around 0.5 or below = normal.

```
anomaly score of (6,6): 0.82   typical point (median): 0.43
```

(scikit-learn's `score_samples` returns the *negative* of this score: lower = more abnormal.) Fast (linear time), scales to large, high-dimensional data, no distance computations, no scaling needed. Weakness: axis-parallel splits can miss anomalies that are only unusual in a *combination* of features.

One-Class SVM

Learns a boundary around the normal data (Schölkopf et al., 2001): with an RBF kernel it finds a smooth region containing most points; points outside are anomalies. **ν** (`nu`) is an upper bound on the fraction of training points treated as outliers (and a lower bound on the fraction of support vectors). Good for **novelty detection** with clean training data; sensitive to γ and ν; needs scaling; doesn't scale well past tens of thousands of points (`SGDOneClassSVM` is a linear, scalable alternative).

Local Outlier Factor (LOF)

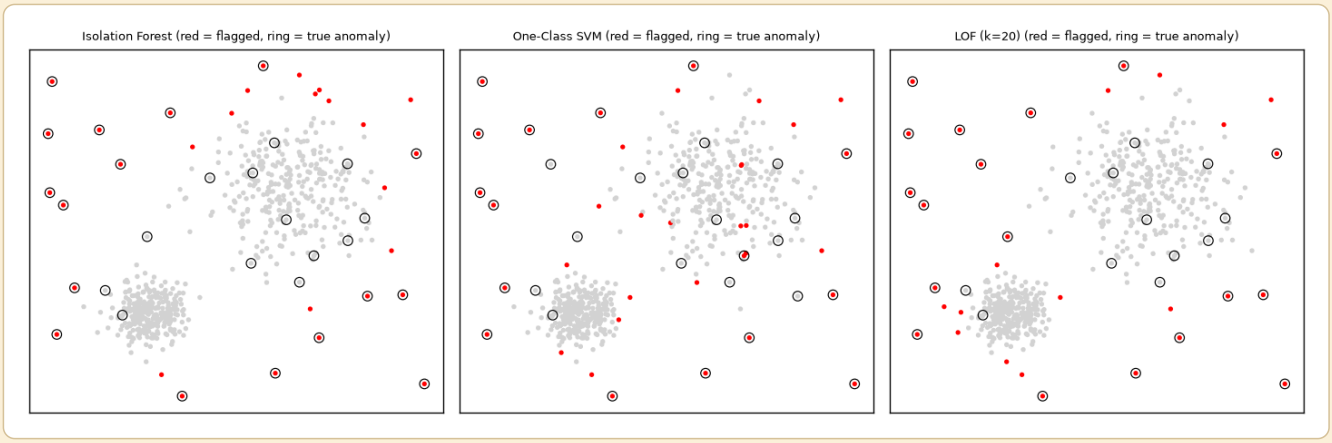
Compares a point's **local density** with the density of its k neighbours (Breunig et al., 2000). LOF ≈ 1: as dense as its neighbours (normal); LOF ≫ 1: much sparser than its neighbours (outlier). Because it's **local**, it catches a point slightly outside a *tight* cluster even if that point would look normal next to a *diffuse* cluster. By default it only scores the training data; use `novelty=True` to score new points.

Comparison

Synthetic data: a tight cluster, a diffuse cluster and 30 uniformly scattered anomalies (`code/anomaly_detection.py`):

```
detectors = {
    "Isolation Forest": IsolationForest(n_estimators=200, contamination=30 / 630, random_state=0),
    "One-Class SVM": OneClassSVM(kernel="rbf", gamma=0.3, nu=0.05),
    "LOF (k=20)": LocalOutlierFactor(n_neighbors=20, contamination=30 / 630),
}
pred = det.fit_predict(X)           # +1 inlier, -1 outlier
```

```
detector            ROC-AUC   avg precision   flagged   true anomalies caught
Isolation Forest      0.920           0.628        30   17 / 30
One-Class SVM         0.813           0.522        36   16 / 30
LOF (k=20)            0.886           0.617        30   18 / 30
```



Roughly a third of the "anomalies" were generated uniformly and happened to land **inside** the normal clusters — no method can (or should) find those; that's why recall tops out near 18/30. The `contamination` parameter sets the expected fraction of outliers, which fixes the score threshold; in real projects you rarely know it, so rank by score and review the top cases. When a few labelled anomalies exist, evaluate with **precision–recall** metrics (average precision), because anomalies are rare and ROC-AUC looks flattering under heavy imbalance.

Method	Idea	Strengths	Weaknesses
z-score / MAD / IQR	Distance from centre in std/MAD units	Trivial, explainable	One feature at a time; assumes one "normal" centre
Isolation Forest	Anomalies isolate in few random splits	Fast, scales, high-dim, no scaling	Misses anomalies only visible in feature combinations
One-Class SVM	Boundary around normal data	Flexible shapes, good for novelty	Slow on big n; tuning γ, ν; needs scaling
LOF	Density vs neighbours' density	Handles clusters of different density	O(n²)-ish neighbour search; choice of k; needs scaling

Other tools worth knowing: **autoencoders** (flag high reconstruction error; covered with neural networks later in the book), Elliptic Envelope (robust Gaussian fit), and time-series methods for seasonal data.

Real-world examples

Card-fraud scoring, detecting failing disks or turbines from sensor logs, intrusion detection on network traffic, flagging data-quality problems before training a model.

In the interview

30-second answer: "For one feature, a robust z-score or IQR rule is enough. For multivariate data, Isolation Forest is my default: anomalies get isolated by random splits in fewer steps, so short average path length means anomaly — it's fast and scales. LOF compares local density to neighbours and handles clusters of different density. One-Class SVM learns a boundary around clean data for novelty detection. Evaluate with precision–recall on whatever labels exist, and set the threshold from business cost."

- *Why not plain z-scores?* The outlier inflates mean and std (masking), and it's univariate.
- *Isolation Forest intuition?* Few-and-different points are separated by random cuts quickly.

Self-check

1. Isolation Forest: short or long path means anomaly? → Short.
2. LOF of 1.0 means? → Same density as its neighbours — normal.
3. Why is ROC-AUC misleading for rare anomalies? → With huge numbers of negatives, a small false-positive *rate* can still mean many false alarms; precision–recall shows it.

TL;DR

Robust statistics for one feature; Isolation Forest as the multivariate default; LOF for varying densities; One-Class SVM for novelty on clean data. Rank by score and judge with precision–recall.

9.8 Association rules (Apriori)

What it is

Association rule mining finds "if-then" patterns in transactions: *{diapers}* → *{beer}* means baskets with diapers often contain beer. Classic use: **market-basket analysis**.

Measure	Formula	Meaning
Support(A)	fraction of baskets containing A	How common
Confidence(A → B)	support(A ∪ B) / support(A)	How often B appears when A does
Lift(A → B)	confidence(A → B) / support(B)	> 1: A makes B more likely than usual; = 1: independent; < 1: less likely

How Apriori works

Key property (anti-monotone / "Apriori principle"): every subset of a frequent itemset must also be frequent. So if {beer} is rare, no set containing beer need be counted.

1. Count single items; keep those with support ≥ min_support (L1).
2. **Join**: combine frequent (k−1)-itemsets into k-item candidates. **Prune**: drop any candidate with an infrequent (k−1)-subset.
3. Count candidates in one pass over the data; keep frequent ones (Lk). Repeat until none remain.
4. From each frequent itemset, generate rules A → B and keep those with confidence ≥ min_confidence. Rank by lift.

`code/apriori.py` on five baskets (min_support 0.6, min_confidence 0.7):

```
L1: {beer}=0.6, {bread}=0.8, {diapers}=0.8, {milk}=0.8
L2: {beer,diapers}=0.6, {bread,diapers}=0.6, {bread,milk}=0.6, {diapers,milk}=0.6

rule                        support  confidence  lift
           beer -> diapers       0.6        1.00  1.25
         diapers -> beer         0.6        0.75  1.25
           bread -> diapers      0.6        0.75  0.94
           bread -> milk         0.6        0.75  0.94
           ...
```

Every beer basket has diapers (confidence 1.0) and lift 1.25 > 1: a real association. But bread → milk, despite 75% confidence, has lift 0.94: milk is in 80% of all baskets anyway, so bread makes it slightly *less* likely. **High confidence alone can mislead — check lift.** (The only 3-item candidate, {bread, diapers, milk}, has support 0.4 and is dropped.)

FP-Growth finds the same itemsets without generating candidates (compresses the data into a prefix tree) and is usually faster; libraries such as `mlxtend` provide both.

Real-world examples and limits

Product placement and "frequently bought together" bundles, cross-selling, co-occurring symptoms or web-page visits. (The famous beer-and-diapers supermarket story is often retold but its details are partly legend.) Limits: many rules, most trivial; thresholds are arbitrary; association is not causation; modern recommenders mostly use collaborative filtering and learned embeddings instead.

In the interview

30-second answer: "Association rules find co-occurrence patterns like A → B. Support is how common an itemset is, confidence is P(B | A), and lift is confidence divided by B's base rate — lift above 1 means a genuine positive association. Apriori finds frequent itemsets level by level and prunes candidates using the fact that all subsets of a frequent set must be frequent."

Self-check

1. Support(AUB) = 0.2, support(A) = 0.4, support(B) = 0.5: confidence and lift? → 0.5 and 1.0 (independent).
2. What does the Apriori principle let you skip? → Any candidate containing an infrequent subset.

TL;DR

Support, confidence, lift; Apriori prunes by "subsets of frequent sets are frequent". Always check lift.

9.9 Evaluating unsupervised models

The problem

There is no ground-truth answer to compare against — and clustering algorithms **always return clusters**, even from pure noise. So evaluate from several angles.

Metrics

Kind	Metric	Range / better	Notes
Internal (no labels)	Silhouette	−1 to 1, higher	Cohesion vs separation; favours round, well-separated clusters
	Davies–Bouldin	≥ 0, **lower**	Average similarity of each cluster to its most similar one
	Calinski–Harabasz	≥ 0, higher	Between-cluster vs within-cluster dispersion
	Inertia / BIC	lower	Only to compare k for the same algorithm
External (some labels exist)	Adjusted Rand Index (ARI)	≈ 0 random, 1 perfect	Pair-counting agreement, corrected for chance
	Normalized Mutual Information (NMI)	0 to 1, higher	Shared information between labelings
Dimensionality reduction	Explained variance, reconstruction error	higher / lower	PCA-type methods
	Trustworthiness	0 to 1, higher	Are map neighbours true neighbours?
Anomaly detection	Precision@k, average precision (on a small labelled set)	higher	Rank-based; matches how analysts review alerts

Internal metrics can disagree with reality

`code/evaluate_clustering.py`:

```
iris (true k = 3 species)
  k-means k=2          silhouette= 0.582  DB= 0.59  CH=  251.3  ARI= 0.57  NMI= 0.73
  k-means k=3          silhouette= 0.460  DB= 0.83  CH=  241.9  ARI= 0.62  NMI= 0.66
  k-means k=4          silhouette= 0.387  DB= 0.87  CH=  207.3  ARI= 0.47  NMI= 0.60

moons (true k = 2, non-convex)
  k-means k=2          silhouette= 0.497  DB= 0.81  CH=  556.6  ARI= 0.47  NMI= 0.38
  DBSCAN eps=0.3       silhouette= 0.385  DB= 1.03  CH=  342.5  ARI= 1.00  NMI= 1.00
```

- On iris, all three internal metrics prefer **k = 2** — two of the species overlap heavily in measurement space, so "2 blobs" is the geometrically cleaner answer. Biology says 3.
- On moons, internal metrics prefer k-means' wrong split over DBSCAN's perfect one, because silhouette, DB and CH all reward **convex** clusters.

A practical evaluation checklist

1. **Look at it**: plot clusters on a PCA/UMAP projection; read samples from each cluster.
2. **Internal metrics** to compare settings of the *same* algorithm (k, eps), knowing their shape bias.
3. **Stability**: re-run with different seeds or bootstrap samples — do the clusters persist (measure with ARI between runs)?
4. **External check**: a small labelled sample, or known categories, via ARI/NMI.
5. **Downstream value** — the real test: do the segments lead to better campaigns, do PCA features improve the model, do flagged anomalies turn out to be fraud when investigated?

In the interview

30-second answer: "Without labels I use internal metrics like silhouette, Davies–Bouldin and Calinski–Harabasz to compare settings — but they favour convex clusters. If some labels exist, ARI or NMI. I check stability across seeds and, most importantly, whether the result is useful for the downstream task. For dimensionality reduction: explained variance, reconstruction error, trustworthiness; for anomalies: precision on a reviewed sample."

Self-check

1. Davies–Bouldin: higher or lower is better? → Lower.
2. Why can silhouette prefer the wrong answer on moons? → It rewards compact, convex, well-separated clusters.
3. ARI of 0 means? → Agreement no better than random labelling.

TL;DR

Unsupervised evaluation = internal metrics (with their biases) + external labels when available + stability + downstream usefulness. Always look at the results.

Chapter TL;DR

- **k-means**: assign to nearest centroid, recompute means, repeat; k-means++ init + restarts; choose k by silhouette/elbow; assumes round, similar clusters; scale features.
- **Hierarchical**: merge closest clusters bottom-up; dendrogram shows structure; linkage (single/complete/average/Ward) sets the shape; small data.
- **DBSCAN**: dense regions = clusters, sparse points = noise; any shape, no k; eps from a k-distance plot; HDBSCAN for varying density.
- **GMM + EM**: soft, elliptical clusters; E-step responsibilities, M-step weighted updates; choose k with BIC.
- **PCA**: eigenvectors of the covariance matrix = directions of max variance; keep enough for ~95% variance; linear; scale first.
- **t-SNE / UMAP**: non-linear 2-D maps preserving neighbourhoods; for visualisation — don't read cluster sizes or gaps. UMAP is faster and can transform new data.
- **Anomalies**: robust z/IQR for one feature; Isolation Forest default; LOF for local density; One-Class SVM for novelty. Evaluate with precision–recall.
- **Apriori**: support, confidence, lift; prune with "subsets of frequent sets are frequent"; trust lift over confidence.
- **Evaluation**: internal metrics are shape-biased — combine with external labels, stability, and downstream usefulness.

Chapter 10 — Neural Networks and Deep Learning

A **neural network** is a function built from many tiny, identical pieces — "neurons" — each of which does something almost trivially simple: multiply its inputs by some numbers, add them up, and bend the result. Stack thousands of these pieces in layers, tune their numbers on data with calculus, and you get systems that recognise faces, transcribe speech and write text. **Deep learning** is simply the practice of training neural networks with many layers.

This chapter builds the idea from zero: one neuron, a network, how it learns (backpropagation, every number worked out), the engineering that makes deep networks train (optimizers, schedules, initialization, normalization, regularization), and the practical habits that save days of debugging — first in NumPy (no magic), then in PyTorch.

What you'll learn

1. Perceptron → multi-layer perceptron (MLP); why depth matters.
2. Activation functions and when to use each.
3. The forward pass and loss functions.
4. Backpropagation, worked with numbers, and a from-scratch NumPy network.
5. Optimizers (SGD, momentum, RMSProp, Adam/AdamW) and learning-rate schedules.
6. Initialization and vanishing/exploding gradients.
7. Regularization: dropout, weight decay, BatchNorm, early stopping, augmentation.
8. Practical training and debugging: normalization, mini-batches, class imbalance, loss curves, overfitting one batch, the LR finder.
9. PyTorch basics: tensors, autograd, `nn.Module`, `DataLoader`, the training loop.

All code is in `code/`, runs on CPU in seconds (PyTorch 2.x, scikit-learn), and every output shown is real.

10.1 From the perceptron to the multi-layer perceptron

What it is

An **artificial neuron** turns several numbers into one, in two steps:

1. **Weighted sum.** Each input x_i is multiplied by a **weight** w_i (how much that input matters, and in which direction), and a **bias** b (a baseline that shifts the result) is added: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w \cdot x + b$. The value z is called the **pre-activation** or **logit**.
2. **Activation.** A fixed function f bends z into the output $a = f(z)$ — for example ReLU, which keeps positive values and turns negatives into 0.

The output a feeds the next layer or is the final prediction.

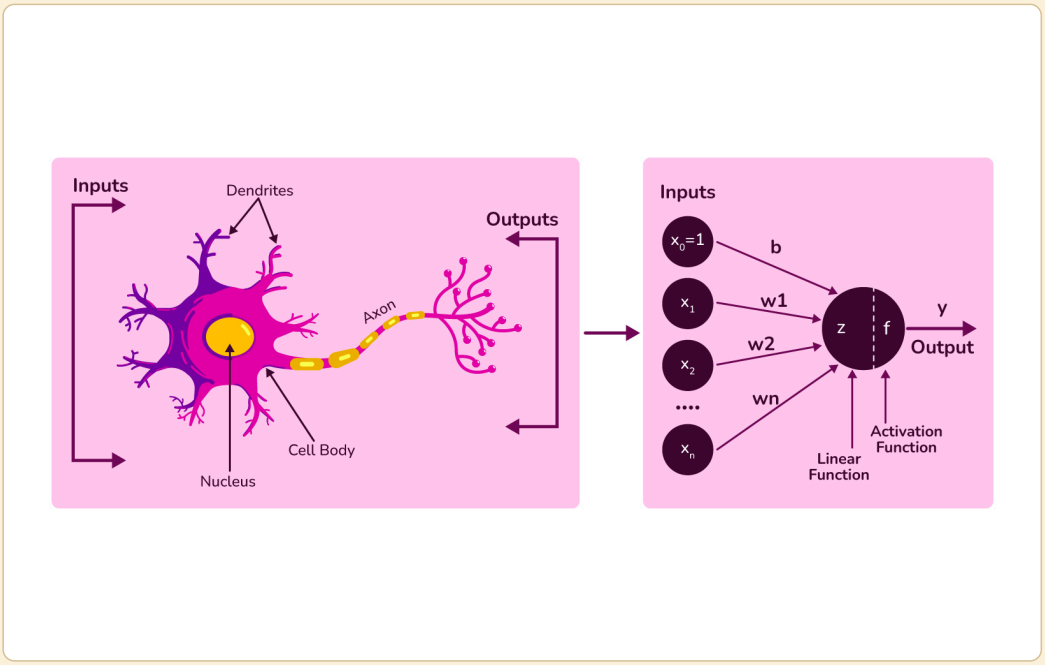


Figure — Left: a biological neuron (dendrites collect signals, the axon sends one out). Right: the artificial version — inputs $x_1 \dots x_n$ times weights, plus bias b (drawn as a constant input $x_0 = 1$), summed into z, then passed through activation f (course diagram).

Analogy: a **committee vote**. Members (inputs) speak; some get more say (weights); the chair has a standing lean (bias); a simple rule turns the tally into the announcement (activation). The biology is only inspiration — artificial neurons are just differentiable arithmetic.

Glossary for this chapter

Term	Meaning
Parameters	The numbers the network learns: all weights and biases
Layer	Neurons reading the same inputs; a **dense** (fully connected, linear) layer links every input to every neuron
Input / hidden / output layer	The raw features / any layer in between (its values never appear in the data) / the layer producing the prediction
Hyperparameters	Choices you make, not learned: depth, width, learning rate, batch size…
Epoch	One full pass over the training set
Batch / mini-batch	The subset of examples used for one weight update

The perceptron (1958) and its limit

Frank Rosenblatt's **perceptron** is one neuron with a **step** activation: output 1 if $w \cdot x + b > 0$, else 0. Learning rule: when a prediction is wrong, nudge the weights towards the right answer, $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$, where η is the learning rate.

A single neuron draws a **straight line** (in higher dimensions a flat **hyperplane**) through feature space: fine for AND, impossible for **XOR** ("exactly one input is 1"), whose classes no single line separates. Minsky and Papert's 1969 book *Perceptrons* made this limit famous.

```
# code/perceptron.py (core)
for epoch in range(epochs):                        # w = zeros, b = 0, lr = 1.0
    for xi, yi in zip(X, y):
        pred = 1 if xi @ w + b > 0 else 0          # step activation
        update = lr * (yi - pred)                  # 0 if correct, +-1 if wrong
        w += update * xi; b += update
    # ... stop ("converged") after an epoch with no errors
```

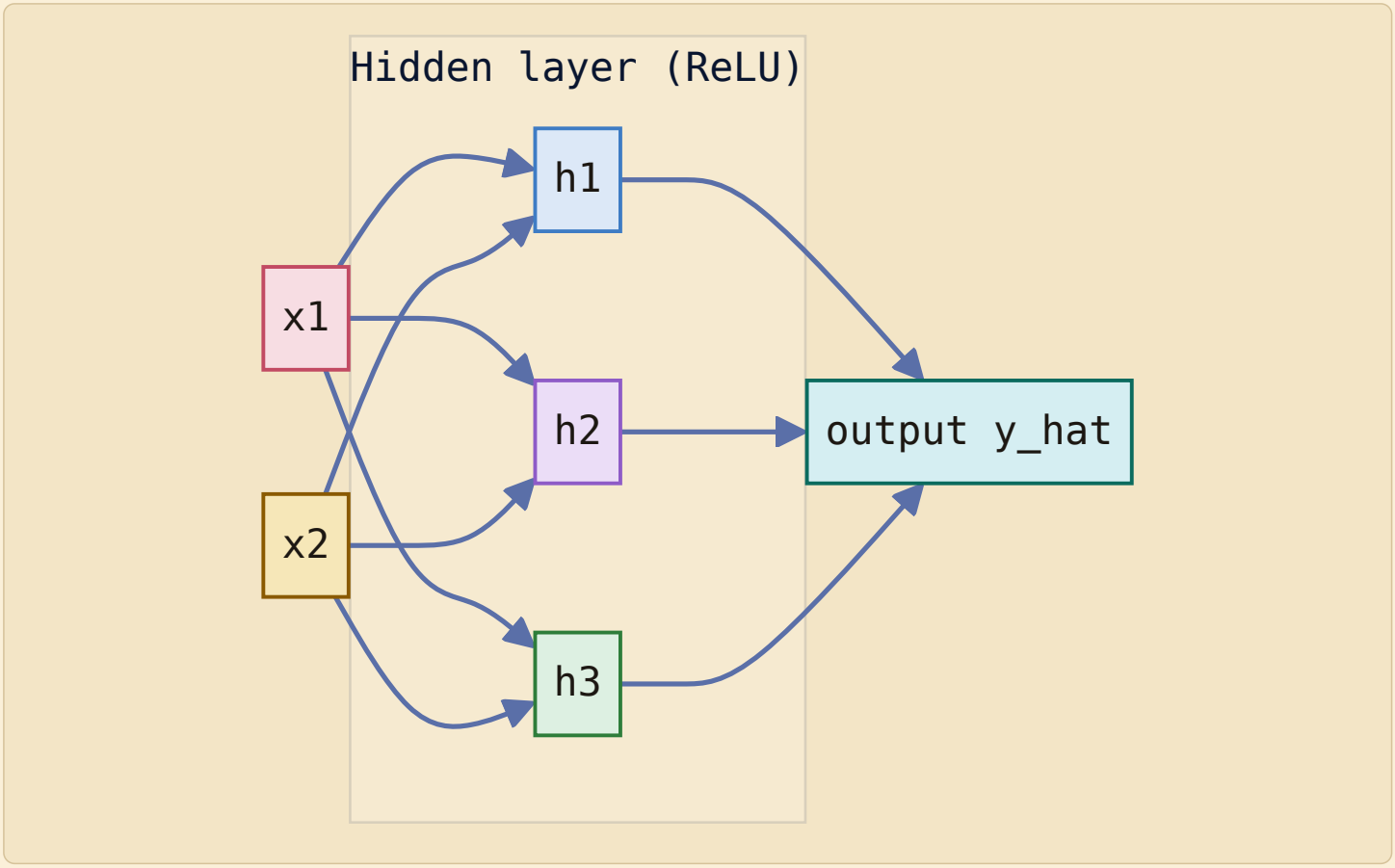
Output:

```
AND: converged=True after 6 epochs, w=[2. 1.], b=-2.0, preds=[0 0 0 1], target=[0 0 0 1]
XOR: converged=False after 20 epochs, w=[-1.  0.], b=1.0, preds=[1 1 0 0], target=[0 1 1 0]
```

The multi-layer perceptron (MLP)

The fix is to **stack** neurons: a **hidden layer** of several neurons transforms the inputs into new features, and the output neuron works on those. With one hidden layer of two neurons, XOR becomes easy: one hidden neuron learns "x_1 OR x_2", another "x_1 AND x_2", and the output computes "OR but not AND".

A network where every layer feeds the next with no loops is a **feed-forward network**; when all layers are dense it is called a **multi-layer perceptron (MLP)** — the name stuck even though modern MLPs use ReLU-like activations, not the step function.



In matrix form, a whole layer is one line: **$h = f(W x + b)$**, where W has one row per neuron. For a batch of examples stored as rows of X: $H = f(X W^T + b)$. Parameter count of a dense layer = (inputs × outputs) + outputs. A 2 → 3 → 1 network has (2·3 + 3) + (3·1 + 1) = 13 parameters.

Why nonlinearity is essential

Without an activation function, stacking layers is pointless: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$ — still one linear function. The bend introduced by f is what lets each extra layer add expressive power. The **universal approximation theorem** (Cybenko 1989, Hornik 1991) says an MLP with one hidden layer and enough neurons can approximate any continuous function on a bounded region as closely as you like — but it says nothing about how many neurons you need or whether training will find the weights.

Why depth matters (hidden layers and "deep" networks)

Hidden layers turn raw inputs into progressively more **abstract representations**: in an image network, early layers respond to edges and colour blobs, middle layers to textures and parts (an eye, a wheel), late layers to whole objects. This **hierarchy of features** reuses lower-level pieces, so a deep network usually needs far fewer neurons than a wide shallow one for the same task.

"Deep" just means many hidden layers. Depth costs computation, data and harder optimization (gradients must travel through every layer — §10.6). ReLU, careful initialization, normalization and residual connections (Chapter 11), together with GPUs and large datasets such as ImageNet, are what made deep networks trainable after about 2010.

Trade-offs

	Neural networks (MLP and beyond)	Gradient-boosted trees / linear models
Best data type	Images, audio, text, sequences — raw, high-dimensional, unstructured	Tabular rows and columns
Feature engineering	Learned automatically	Mostly manual
Data needed	Usually a lot	Works on small data
Compute	High (GPUs for real sizes)	Low
Interpretability	Low	Medium–high
Tuning effort	Many hyperparameters	Fewer

When not to use it

- Small tabular datasets: gradient-boosted trees (Chapter 7) usually win and train in seconds.
- When each decision must be explained to a regulator and a simple model is accurate enough, or there is no compute budget.

Common mistakes

- Forgetting the activation between layers (the network collapses to linear).
- Believing "more layers is always better" — deeper nets overfit and are harder to train without the tricks below.

Real-world examples

Google Photos search, Face ID and self-driving perception (deep CNNs and transformers, Chapters 11–12); speech recognition in voice assistants; recommenders at YouTube and Meta, which run MLPs on learned embeddings (e.g. Meta's open-source DLRM).

In the interview

> **30-second answer:** "A neuron computes a weighted sum plus bias and applies a nonlinear activation. An MLP stacks layers; hidden layers learn intermediate features and depth composes simple features into complex ones. Without nonlinearities the stack collapses to one linear map. A single perceptron draws one hyperplane, so it can't learn XOR; one hidden layer can."

- *Q: How many parameters in a dense layer from 256 to 128 units?* A: 256·128 + 128 = 32,896.
- *Q: What does the universal approximation theorem guarantee?* A: That a one-hidden-layer network approximating a continuous function exists — not that it is small or that training finds it.

Self-check

1. Why can't a perceptron learn XOR? → XOR's classes are not linearly separable; one neuron draws one hyperplane.
2. Remove all activations from a 5-layer MLP? → It becomes equivalent to a single linear layer.
3. What does a bias do? → Shifts the threshold, so the neuron can fire when all inputs are 0 (or stay quiet when they are large).

TL;DR

Neuron = weighted sum + bias + nonlinearity; MLP = layers of them. Hidden layers learn features, depth builds a hierarchy, nonlinearity makes stacking useful.

10.2 Activation functions

What it is

The **activation function** f is the nonlinearity applied to each pre-activation z. It governs how well gradients flow and, at the output layer, what values the network can produce.

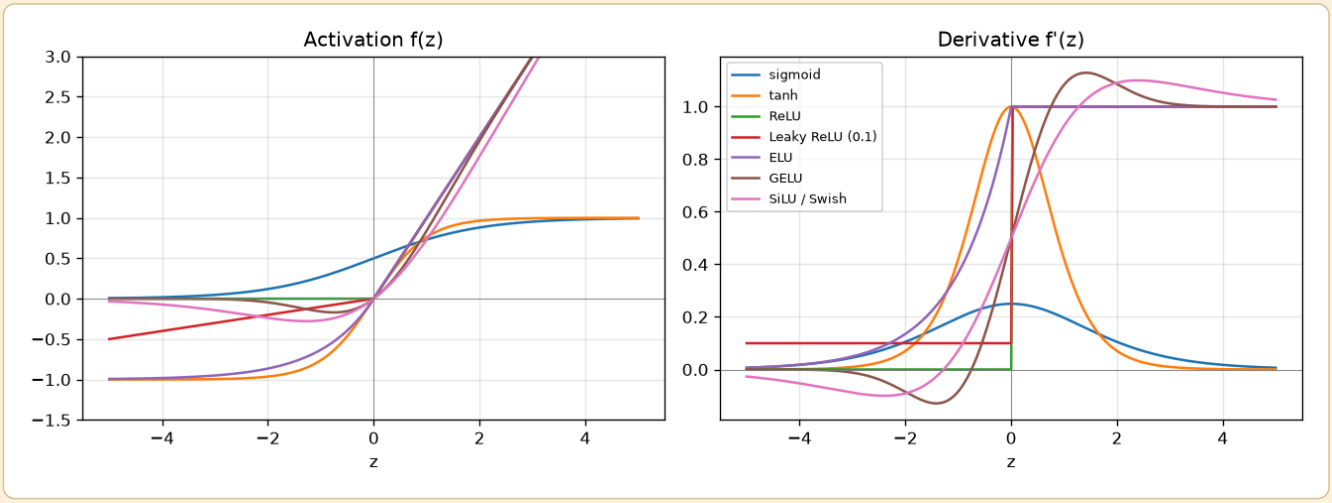


Figure — Common activations (left) and their derivatives (right), computed with PyTorch autograd by `code/activations.py`. Note the sigmoid's derivative never exceeds 0.25 and both sigmoid and tanh flatten to 0 for large $|z|$ ("saturation").

The table

Name	Formula	Output range	Derivative	Where used	Pros	Cons
Sigmoid σ	$1 / (1 + e^{-z})$	$(0, 1)$	$\sigma(z)(1 - \sigma(z))$, max 0.25	Binary / multi-label outputs; LSTM gates	Reads as a probability	Saturates; not zero-centred
Tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	$(-1, 1)$	$1 - \tanh^2(z)$, max 1	RNN/LSTM hidden states	Zero-centred	Still saturates for large $\|z\|$
ReLU	$\max(0, z)$	$[0, \infty)$	1 if $z > 0$, else 0	Default hidden layer	Cheap, no saturation for $z > 0$	"Dying ReLU": stuck at $z < 0$, zero gradient forever
Leaky ReLU (PReLU if α is learned)	z if $z > 0$ else αz ($\alpha \approx 0.01$)	$(-\infty, \infty)$	1 or α	Hidden layers when ReLUs die	Never exactly zero gradient	Extra hyperparameter α
ELU	z if $z > 0$ else $\alpha(e^{z} - 1)$	$(-\alpha, \infty)$	1 or αe^{z}	Hidden layers	Smooth, mean closer to 0	exp is slower
GELU	$z \cdot \Phi(z)$ (Φ = standard normal CDF)	$\approx(-0.17, \infty)$	smooth	Transformers (BERT, GPT)	Smooth ReLU-like	More compute
SiLU / Swish	$z \cdot \sigma(z)$	$\approx(-0.28, \infty)$	smooth	EfficientNet; LLM feed-forward (SwiGLU)	Smooth	More compute
Softmax	$\text{softmax}_i(z) = e^{z_i} / \Sigma_j e^{z_j}$	each $(0,1)$, sum = 1	—	Single-label multi-class output	Probability distribution	Output layer only
Identity (linear)	z	$(-\infty, \infty)$	1	Output for regression	Unbounded	—

Numbers worth remembering (from `code/activations.py`):

```
sigmoid  f(-2,0,2)=[0.119 0.5   0.881]  f'=[0.105 0.25  0.105]
tanh     f(-2,0,2)=[-0.964  0.     0.964]  f'=[0.071 1.    0.071]
ReLU     f(-2,0,2)=[0. 0. 2.]  f'=[0. 0. 1.]
softmax([2,1,0.1]) = [0.659 0.242 0.099]
```

Softmax worked example. $z = (2, 1, 0.1)$: $e^{2} = 7.389$, $e^{1} = 2.718$, $e^{0.1} = 1.105$, sum $11.212 \rightarrow (0.659, 0.242, 0.099)$. Adding a constant to every score changes nothing, so libraries subtract $\max(z)$ first to avoid overflow.

How to choose

1. **Hidden layers: ReLU** (Leaky ReLU if many units die); **GELU/SiLU** in transformers; tanh/sigmoid only inside recurrent gates (Chapter 12).
2. **Output layer:** match the task —

Task	Output activation	Loss
Regression	none (identity)	MSE / MAE / Huber
Binary classification	sigmoid (1 unit)	binary cross-entropy
Multi-class, one label	softmax (K units)	categorical cross-entropy
Multi-label (several can be true)	sigmoid per label	binary cross-entropy per label

In PyTorch, the output layer usually emits **raw logits** and the loss applies the sigmoid/softmax internally (`BCEWithLogitsLoss`, `CrossEntropyLoss`) — this is more numerically stable.

Common mistakes

- Putting `softmax` in the model and then using `nn.CrossEntropyLoss` (which already applies log-softmax) — the network trains poorly.
- Using sigmoid in every hidden layer of a deep network → vanishing gradients.
- Using ReLU on the output of a regression model that must predict negative values.

Real-world examples

AlexNet (2012) popularised ReLU, reporting about 6× faster training than tanh on CIFAR-10; BERT and GPT-2/3 use GELU; EfficientNet uses SiLU; LLaMA uses SwiGLU (a gated SiLU) in its feed-forward blocks.

In the interview

> **30-second answer:** "Activations add nonlinearity. ReLU is the hidden-layer default — cheap, no saturation for positive inputs — with dead units as its weakness (Leaky ReLU helps). Sigmoid and tanh saturate, so they live at outputs and in RNN gates. GELU/SiLU are the smooth transformer defaults. Output: identity for regression, sigmoid for binary/multi-label, softmax for multi-class."

- *Q: Why is ReLU's gradient at 0 not a problem?* A: z is exactly 0 with essentially zero probability; frameworks just define the derivative there (PyTorch uses 0).
- *Q: Softmax vs sigmoid for 3 classes?* A: Softmax if exactly one class is true (probabilities compete); independent sigmoids if several can be true.

Self-check

1. What is a "dead" ReLU? → A unit whose pre-activation is negative for all inputs, so its output and gradient are always 0 and it stops learning.
2. tanh's advantage over sigmoid? → Zero-centred output and a larger maximum gradient (1 vs 0.25).
3. softmax(z + 5) vs softmax(z)? → Identical.

TL;DR

ReLU (or GELU in transformers) for hidden layers; identity / sigmoid / softmax at the output depending on the task; sigmoid and tanh saturate and cause vanishing gradients in deep stacks.

10.3 The forward pass and loss functions

What it is

The **forward pass** runs the network: a batch of inputs flows through the layers, each computing h = f(W h_prev + b), until the last layer produces a **prediction** ŷ (or raw logits), e.g. "probability this photo is a cat: 0.92". The **loss function** compares ŷ with the truth y and returns one number — how wrong the network is — averaged over the batch. Training searches for parameters that make the average training loss small.

Loss functions

Loss	Formula (one example)	Task	Notes
MSE (mean squared error)	$(\hat{y} - y)^2$	Regression	Punishes big errors heavily; sensitive to outliers. `nn.MSELoss`
MAE / L1	$\lvert\hat{y} - y\rvert$	Regression	Robust to outliers; gradient constant size. `nn.L1Loss`
Huber / Smooth L1	$\frac{1}{2}(\hat{y}-y)^2$ if $\lvert\hat{y}-y\rvert \le \delta$ else $\delta(\lvert\hat{y}-y\rvert - \frac{1}{2}\delta)$	Regression	Quadratic near 0, linear far away. `nn.HuberLoss`
Binary cross-entropy (BCE)	$-[y \log p + (1-y) \log(1-p)]$	Binary / multi-label	p = σ(logit). `nn.BCEWithLogitsLoss` takes logits
Categorical cross-entropy	$-\log p_{true}$	Multi-class	p = softmax(logits). `nn.CrossEntropyLoss` takes logits + integer labels
Hinge	$\max(0, 1 - y \cdot s)$, $y \in \{-1, +1\}$	Max-margin classification	SVM loss; rare in deep learning

Worked cross-entropy. A 3-class model outputs probabilities (0.7, 0.2, 0.1) and the true class is 0: loss = −ln 0.7 = 0.357. If the true class were 2: −ln 0.1 = 2.303. Confident wrong answers are punished hardest. A model that guesses uniformly over K classes has loss ln K (ln 10 = 2.303 for digits) — a useful sanity check for the first training step.

Why cross-entropy rather than MSE for classification? With sigmoid/softmax outputs its gradient at the logits is simply **p − y** (§10.4), large when the model is confidently wrong; MSE through a sigmoid is multiplied by σ', tiny when the sigmoid saturates — learning stalls exactly when most needed. Cross-entropy is also the maximum-likelihood loss for a Bernoulli/categorical model.

Common mistakes

- Passing probabilities to `CrossEntropyLoss` or `BCEWithLogitsLoss` (they expect logits).
- Mixing target formats: `CrossEntropyLoss` takes integer class indices (`long`, shape N) *or* class probabilities (`float`, shape N×K) — not a mix.
- Using accuracy as a training loss — it has zero gradient almost everywhere.

Self-check

1. Loss for "predict house price"? → MSE (or Huber if there are outliers).
2. Probabilities (0.7, 0.2, 0.1), true class 1 — cross-entropy? → −ln 0.2 = 1.609.
3. First-step loss on a 4-class problem with a sensible init? → About ln 4 = 1.386.

TL;DR

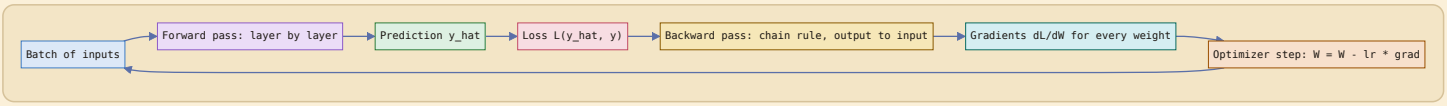
Forward pass = compute predictions layer by layer. Pick the loss by task: MSE/MAE/Huber for regression, BCE for binary or multi-label, cross-entropy for multi-class. Feed logits to PyTorch's combined losses.

10.4 Backpropagation

What it is

Backpropagation ("backprop") is the algorithm that computes the **gradient** of the loss with respect to every weight — that is, for each parameter, "if I increase this a tiny bit, how much does the loss change?" It does so efficiently by applying the **chain rule** of calculus from the output backwards, reusing intermediate results, so the cost is only about 2–3× a forward pass regardless of how many parameters there are. It was popularised for neural networks by Rumelhart, Hinton and Williams in 1986.

Backprop only computes gradients. **Gradient descent** then uses them: each parameter moves a small step in the direction that reduces the loss, $\theta \leftarrow \theta - \eta \cdot \partial L/\partial \theta$, where η (eta) is the **learning rate**.



This forward → loss → backward → update loop, repeated over many mini-batches and epochs, is **how every neural network learns**.

The chain rule in one line

If L depends on a through b (L = g(b), b = h(a)), then $\partial L/\partial a = \partial L/\partial b \cdot \partial b/\partial a$. A network is a long chain of such steps; backprop multiplies the local derivatives from the loss back to each weight. Each layer only needs to know (1) the gradient arriving from above and (2) its own local derivative.

Worked example with numbers

Network: 2 inputs → 2 hidden units (ReLU) → 1 output (sigmoid), binary cross-entropy loss.

- Input x = (1, 2), true label y = 1.
- W_1 = [[0.5, −0.2], [0.3, 0.4]] (row j = weights into hidden unit j), b_1 = (0.1, −0.5).
- w_2 = (0.7, −0.6), b_2 = 0.2. Learning rate η = 0.1.

Forward pass

Step	Computation	Value
z_1 (hidden unit 1)	0.5·1 + (−0.2)·2 + 0.1	0.2
z_1 (hidden unit 2)	0.3·1 + 0.4·2 − 0.5	0.6
h = ReLU(z_1)	both positive, unchanged	(0.2, 0.6)
z_2	0.7·0.2 + (−0.6)·0.6 + 0.2	−0.02
ŷ = σ(z_2)	1/(1 + e^0.02)	0.4950
L = −ln ŷ	(since y = 1)	0.7032

Backward pass (each line = previous gradient × local derivative)

Gradient	Rule	Value
$\partial L/\partial z_2$	for sigmoid + BCE this simplifies to $\hat{y} - y$	$0.4950 - 1 = -0.5050$
$\partial L/\partial w_2$	$\partial L/\partial z_2 \cdot h$ (because $z_2 = w_2 \cdot h + b_2$)	$(-0.101, -0.303)$
$\partial L/\partial b_2$	$\partial L/\partial z_2 \cdot 1$	-0.5050
$\partial L/\partial h$	$\partial L/\partial z_2 \cdot w_2$	$(-0.3535, 0.303)$
$\partial L/\partial z_1$	$\partial L/\partial h \cdot \text{ReLU}'(z_1)$, and ReLU' = 1 because both $z_1 > 0$	$(-0.3535, 0.303)$
$\partial L/\partial W_1$	$\partial L/\partial z_1$ (column) $\times$ x^T (row) — outer product	$[[-0.3535, -0.707], [0.303, 0.606]]$
$\partial L/\partial b_1$	$\partial L/\partial z_1$	$(-0.3535, 0.303)$

Why $\hat{y} - y$? For BCE, $\partial L/\partial \hat{y} = -y/\hat{y} + (1-y)/(1-\hat{y})$, and $\sigma'(z) = \hat{y}(1 - \hat{y})$. Multiply them and everything cancels to $\hat{y} - y$. (The same simplification holds for softmax + cross-entropy: $\partial L/\partial \text{logits} = p - \text{one_hot}(y)$.)

Reading the gradients. $\partial L/\partial w_2$ is negative, so gradient descent *increases* w_2, pushing $\hat{y}$ towards the label 1. Hidden unit 2 has a negative outgoing weight (-0.6), so the network wants its activation *smaller*: its incoming weights get positive gradients and shrink.

Update (one step of gradient descent, $\eta = 0.1$): for example $w_2 \leftarrow (0.7, -0.6) - 0.1 \cdot (-0.101, -0.303) = (0.7101, -0.5697)$. After updating all parameters, the same input gives $\hat{y} = 0.5756$ and loss 0.5523 — down from 0.7032.

The script checks the hand-derived gradients against PyTorch's automatic differentiation:

```
# code/backprop_by_hand.py (backward part)
dz2 = y_hat - y                  # dL/dz2 for sigmoid + BCE
dw2 = dz2 * h;  db2 = dz2
dh  = dz2 * w2
dz1 = dh * (z1 > 0)              # ReLU'(z1)
dW1 = np.outer(dz1, x);  db1 = dz1
# ... then the same network in torch, loss.backward(), and np.allclose on every gradient
```

Output (last lines; the rest repeats the tables above):

```
autograd agrees with hand-derived gradients: True
after one SGD step (lr=0.1): y_hat 0.4950 -> 0.5756, loss 0.7032 -> 0.5523
```

A 2-layer network from scratch in NumPy

`code/numpy_mlp.py` implements the same ideas for a whole batch (rows = examples), with tanh hidden units and a sigmoid output, and trains it by full-batch gradient descent.

```
class TwoLayerNet:
    """x -> tanh(x W1 + b1) -> sigmoid(h W2 + b2). Rows of X are examples."""
    # __init__: W1, W2 ~ N(0, 1/fan_in) (LeCun-style scale), biases 0
    def forward(self, X):
        self.X = X
        self.H = np.tanh(X @ self.W1 + self.b1)
        self.P = 1 / (1 + np.exp(-(self.H @ self.W2 + self.b2)))
        return self.P

    def backward(self, y):
        n = len(y)
        dZ2 = (self.P - y[:, None]) / n                  # BCE + sigmoid, averaged over batch
        dW2 = self.H.T @ dZ2;  db2 = dZ2.sum(0)
        dH = dZ2 @ self.W2.T
        dZ1 = dH * (1 - self.H ** 2)                      # tanh'(z) = 1 - tanh(z)^2
        dW1 = self.X.T @ dZ1;  db1 = dZ1.sum(0)
        return dW1, db1, dW2, db2

    def step(self, grads, lr):
        for p, g in zip((self.W1, self.b1, self.W2, self.b2), grads):
            p -= lr * g
```

`forward` stores X, H and P because backward needs them (why training uses more memory than inference). `backward` walks the chain rule in reverse; a weight matrix's gradient is always (layer input)^T @ (gradient at the layer's output). The script also runs a **numerical gradient check** — nudge one weight by $\pm\varepsilon$, measure the loss change — the standard test for a hand-written backward pass.

Output:

```
XOR  loss 0.687 -> 0.0012, probs = [0.    0.999 0.999 0.002]
Moons loss 0.682 -> 0.0363, train acc 0.986, test acc 0.956
gradient check  numerical: [-0.02769  -0.015654 -0.024803]  analytical: [-0.02769  -0.015654 -0.024803]
```

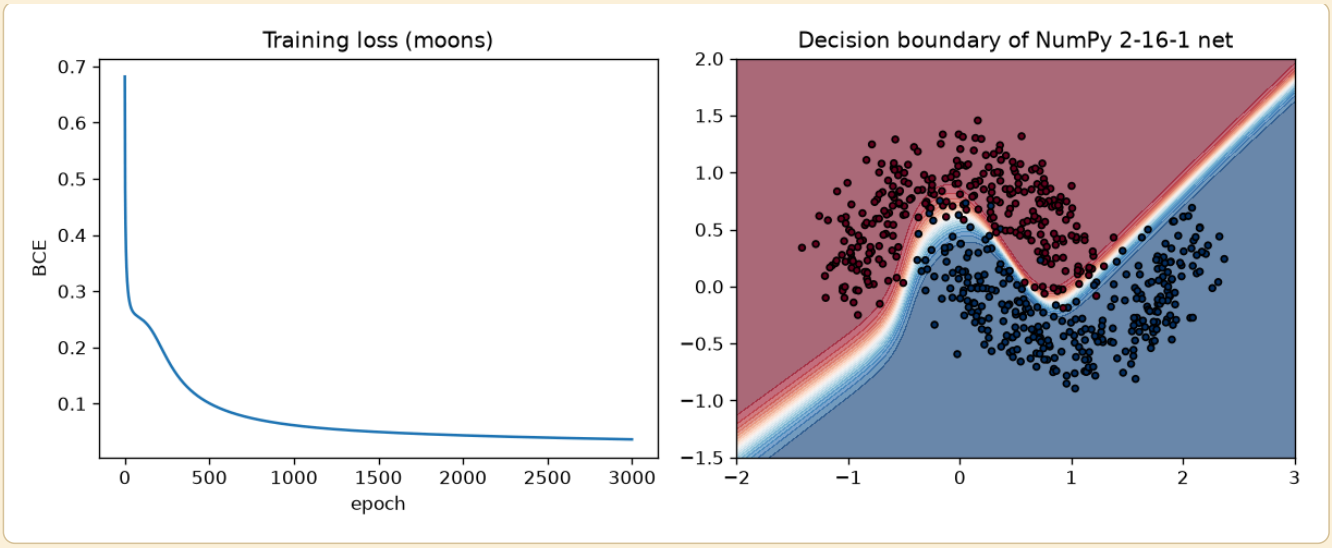


Figure — Left: training loss falls smoothly. Right: the 16-hidden-unit network has learned a curved boundary no linear model could draw.

Computational graphs and autograd

Frameworks record every tensor operation in a **computational graph**; `loss.backward()` walks it in reverse and accumulates gradients into each parameter's `.grad`. This is **reverse-mode automatic differentiation** — exact, and cheap when one scalar output (the loss) depends on many inputs (the parameters). Karpathy's *micrograd* builds such an engine in about 100 lines.

Common mistakes

- Forgetting to zero gradients between steps in PyTorch — they **accumulate** by default.
- Summing instead of averaging the batch loss — the effective learning rate then changes with batch size.
- Writing your own backward pass without a gradient check.

In the interview

30-second answer: "Backprop applies the chain rule from the loss backwards through the computational graph, getting $\partial L/\partial \theta$ for every parameter for a small constant multiple of a forward pass. Each layer multiplies the incoming gradient by its local derivative. An optimizer then steps against the gradient. For sigmoid+BCE or softmax+cross-entropy, the gradient at the logits is prediction minus target."

- *Q: Why reverse mode, not forward mode?* A: One scalar output, millions of inputs — one backward sweep gets every partial derivative; forward mode needs one sweep per input.
- *Q: What must the forward pass store?* A: The activations the local derivatives need — so training memory grows with batch size and depth (gradient checkpointing trades recompute for memory).

Self-check

1. For sigmoid output + BCE, what is $\partial L/\partial z$ at the output? → $\hat{y} - y$.
2. Gradient of a dense layer's weights? → (layer input)$^\top$ @ (gradient at the layer's pre-activation output).
3. Why must activations be stored? → The backward pass needs them to compute local derivatives (for example h in $\partial L/\partial w_2 = \partial L/\partial z_2 \cdot h$).

TL;DR

Forward pass computes the loss; backward pass applies the chain rule layer by layer to get every gradient; gradient descent subtracts learning-rate × gradient. Autograd automates the backward pass.

10.5 Optimizers and learning-rate schedules

What it is

An **optimizer** decides how to turn gradients into parameter updates. The **learning rate** (step size) is the single most important hyperparameter in deep learning.

Analogy: tuning a radio dial. Huge jerks keep jumping past the station (oscillation or divergence); millimetre turns take forever. A good tuner turns fast at first and slowly near the station — what schedules and adaptive optimizers do.

How the main optimizers work

Notation: g = gradient of the mini-batch loss, η = learning rate, θ = a parameter.

Optimizer	Update rule (per parameter)	Idea	PyTorch defaults
SGD (stochastic gradient descent)	$\theta \leftarrow \theta - \eta\, g$	Step downhill using a mini-batch estimate of the gradient	`SGD(lr=…)` — always set lr explicitly
SGD + momentum	$v \leftarrow \mu\, v + g$; $\theta \leftarrow \theta - \eta\, v$	Keep a running "velocity": consistent directions accelerate, zig-zags cancel	momentum μ typically 0.9
Nesterov momentum	gradient evaluated at the look-ahead point	Slightly better-behaved momentum	`nesterov=True`
AdaGrad	$s \leftarrow s + g^2$; $\theta \leftarrow \theta - \eta\, g/(\sqrt{s} + \varepsilon)$	Per-parameter step size, shrinks for frequently-updated params	Good for sparse features; step decays too much for deep nets
RMSProp	$s \leftarrow \rho\, s + (1-\rho)\, g^2$; $\theta \leftarrow \theta - \eta\, g/(\sqrt{s} + \varepsilon)$	Like AdaGrad but with a moving average, so steps don't vanish	lr 0.01, ρ (`alpha`) 0.99
Adam	$m \leftarrow \beta_1 m + (1-\beta_1) g$; $s \leftarrow \beta_2 s + (1-\beta_2) g^2$; bias-correct: $\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{s}} + \varepsilon)$	Momentum + RMSProp + bias correction for the zero start	lr 1e-3, $\beta = (0.9, 0.999)$, ε = 1e-8
AdamW	Adam, plus $\theta \leftarrow \theta - \eta\, \lambda\, \theta$ applied separately from the gradient	"Decoupled" weight decay — the correct way to do L2-style shrinkage with Adam	weight_decay 0.01

Key ideas in plain words:

- **Stochastic:** each mini-batch gives a noisy gradient estimate — cheap, and the noise can help escape sharp minima.
- **Momentum** is a heavy ball rolling downhill: it speeds up along a consistent slope and damps zig-zags in narrow valleys ("ravines").
- **Adaptive methods** (RMSProp, Adam) give each parameter its own effective step size, dividing by the recent gradient magnitude: parameters with steep, noisy gradients take smaller steps; flat directions take bigger ones.
- **Adam's first step, in numbers** ($g = 0.5$, $\beta_1 = 0.9$, $\beta_2 = 0.999$): $m = 0.1 \cdot 0.5 = 0.05$, $s = 0.001 \cdot 0.25 = 0.00025$; bias-corrected $\hat{m} = 0.05/0.1 = 0.5$, $\hat{s} = 0.00025/0.001 = 0.25$; step $= \eta \cdot 0.5/\sqrt{0.25} = \eta$. Without correction the step would be $\eta \cdot 0.05/0.0158 \approx 3.2\eta$ — distorted. Note the step is $\approx\eta$ whatever the gradient's size: Adam's steps are scale-free.

`code/optimizers.py` runs all four with `torch.optim` on (a) a 2-D ravine, $f(x, y) = 0.05x^2 + 2y^2$ — 40× steeper in y than in x — starting at $(-10, 2)$, and (b) an MLP on two moons. Each learning rate (SGD 0.45, momentum 0.05, RMSProp 0.3, Adam 0.5) is near the largest that behaves well:

```
p = torch.tensor([-10.0, 2.0], requires_grad=True)
opt = torch.optim.SGD([p], lr=0.05, momentum=0.9)   # or RMSprop / Adam
for _ in range(60):
    opt.zero_grad(); ravine(p).backward(); opt.step()
```

Output:

```
(a) ravine f = 0.05 x^2 + 2 y^2, start (-10, 2), 60 steps
  SGD                final point [-0.631  0.   ]  loss 1.99e-02
  SGD+momentum(0.9)  final point [ 0.358 -0.073]  loss 1.71e-02
  RMSProp            final point [-0.  0.]  loss 3.77e-14
  Adam               final point [-0.215  0.045]  loss 6.42e-03
(b) MLP 2-32-32-1 on moons, 200 full-batch epochs
  SGD                loss after 20 epochs 0.639, after 200 0.265
  SGD+momentum(0.9)  loss after 20 epochs 0.310, after 200 0.083
  RMSProp            loss after 20 epochs 0.171, after 200 0.080
  Adam               loss after 20 epochs 0.274, after 200 0.075
```

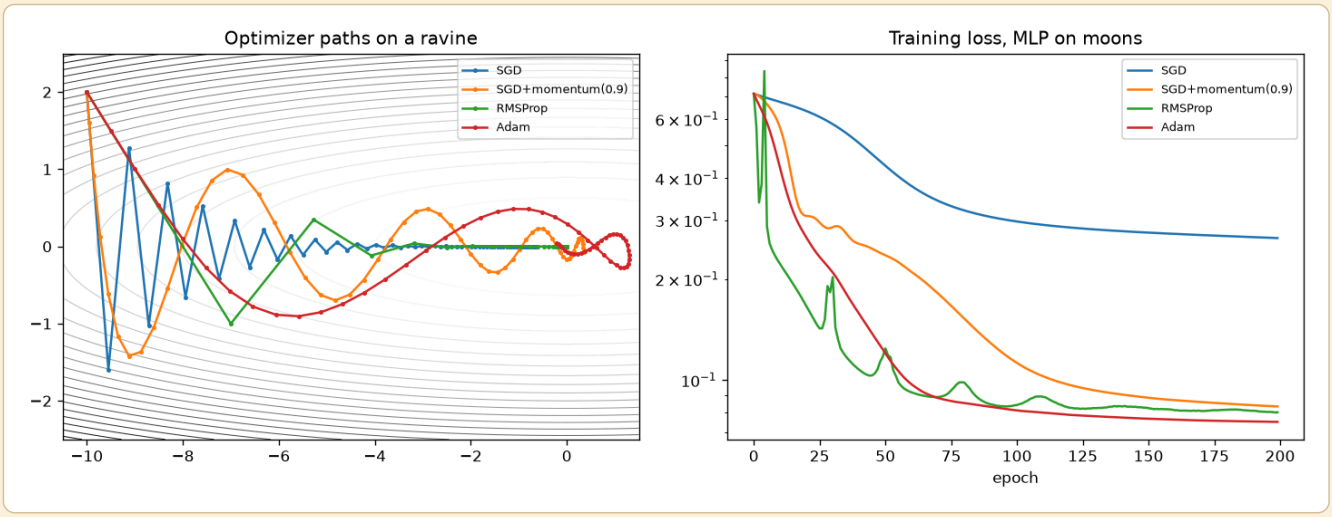


Figure — Left: plain SGD's learning rate is capped by the steep y-direction, so it crawls in x; momentum overshoots but travels fast along the valley; RMSProp and Adam rescale each coordinate and head almost straight for the minimum. Right: on a tiny network, momentum and adaptive methods reach low loss far sooner than plain SGD.

RMSProp looks spectacular here because its per-coordinate scaling exactly undoes this axis-aligned ravine. Don't over-read toys: in practice **Adam/AdamW** is the default for transformers and most new models, while **SGD with momentum** plus a schedule remains competitive (sometimes generalising slightly better) for CNN image classifiers.

Learning-rate schedules

A **schedule** changes η during training: big steps early, small steps late to settle.

Schedule	Shape	PyTorch	Typical use
Constant	flat	—	Baselines, small problems with Adam
Step decay	×γ every k epochs	`StepLR`, `MultiStepLR`	Classic ResNet recipe (÷10 at epochs 30, 60, 90)
Exponential decay	×γ every step/epoch	`ExponentialLR`	Simple smooth decay
Reduce on plateau	drop when validation metric stalls	`ReduceLROnPlateau`	When you don't know the right epoch count
Cosine annealing	half cosine from η_max to ~0	`CosineAnnealingLR`	Modern default for vision and LLMs
Linear warmup + decay	ramp from ~0 over the first steps, then cosine/linear decay	`LinearLR` + `SequentialLR`	Transformers (warmup avoids early instability with Adam)
One-cycle	warm up to max, then anneal far below the start	`OneCycleLR`	Fast training (Leslie Smith's "super-convergence")

```
# code/lr_schedules.py (warmup + cosine) - each step: opt.step() THEN sched.step()
sched = S.SequentialLR(opt, [S.LinearLR(opt, start_factor=0.01, total_iters=10),
                             S.CosineAnnealingLR(opt, T_max=90)], milestones=[10])
```

Output:

```
Constant                lr at steps 0/10/50/99: 0.1000 0.1000 0.1000 0.10000
StepLR (x0.5 every 25)  lr at steps 0/10/50/99: 0.1000 0.1000 0.0250 0.01250
ExponentialLR (0.97)    lr at steps 0/10/50/99: 0.1000 0.0737 0.0218 0.00490
CosineAnnealingLR       lr at steps 0/10/50/99: 0.1000 0.0976 0.0500 0.00002
Warmup 10 + cosine      lr at steps 0/10/50/99: 0.0010 0.1000 0.0587 0.00003
OneCycleLR              lr at steps 0/10/50/99: 0.0040 0.0295 0.0794 0.00000
```

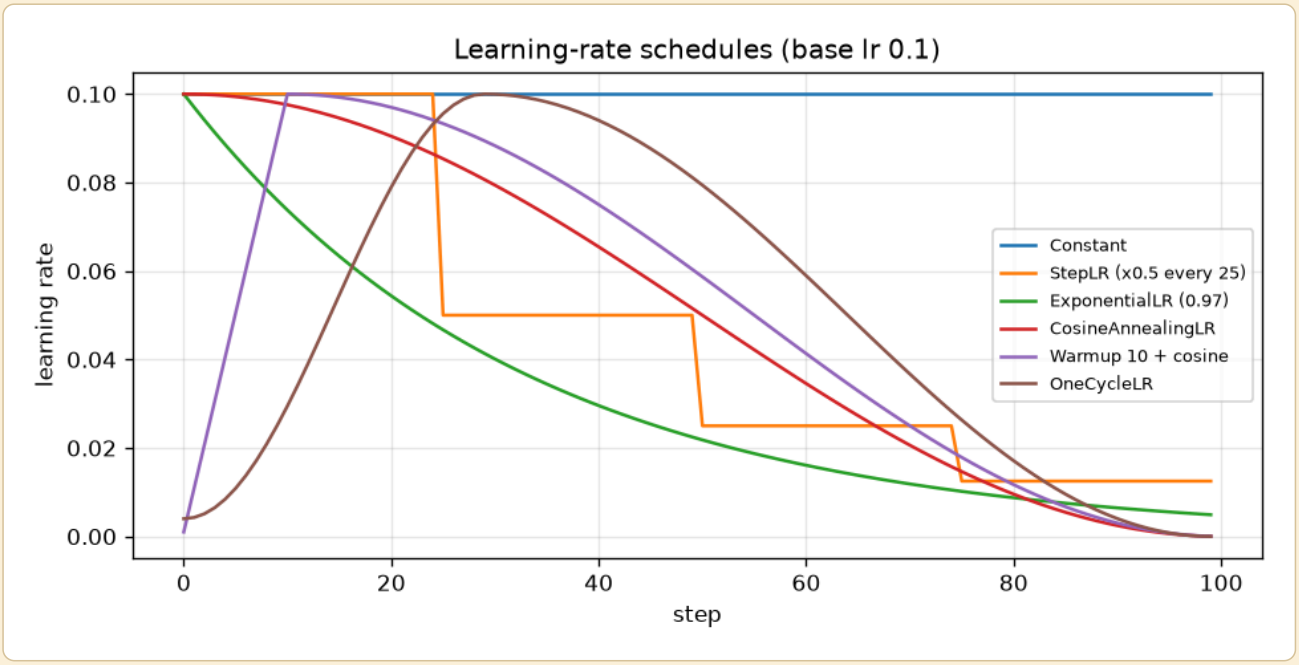


Figure — The six schedules over 100 steps.

Trade-offs

	SGD + momentum	Adam / AdamW
Tuning	LR and schedule matter a lot	Works well near lr=1e-3 out of the box
Speed of convergence	Slower early	Fast early
Generalization	Often excellent on CNNs	Excellent on transformers (with AdamW)
Memory	1 extra buffer per parameter	2 extra buffers per parameter

Common mistakes

- Expecting AdamW behaviour from `Adam(weight_decay=…)` — there the decay is added to the gradient (L2) and distorted by adaptive scaling; use `AdamW`.
- Calling `scheduler.step()` before `optimizer.step()` (PyTorch warns; the first LR value is skipped).
- Changing the batch size without revisiting the LR (heuristic for SGD: scale LR roughly linearly with batch size, plus warmup).

Real-world examples

GPT-3 and LLaMA: Adam/AdamW with linear warmup and cosine decay. The original ResNet: SGD, momentum 0.9, LR ÷10 when error plateaued. fast.ai popularised one-cycle training and the LR finder.

In the interview

30-second answer: "SGD steps against the mini-batch gradient; momentum accumulates a velocity so consistent directions speed up and oscillations cancel; RMSProp divides each step by a running RMS of that parameter's gradients; Adam combines both with bias correction, and AdamW decouples weight decay. A schedule — typically warmup then cosine decay — gives big steps early and fine steps late."

- *Q: Why warmup with Adam?* A: Early second-moment estimates are poor and gradients large; a small initial LR prevents destructive first updates.
- *Q: Why is Adam's bias correction needed?* A: m and s start at 0, so early averages are biased towards 0; dividing by $1 - \beta^t$ fixes this.

Self-check

1. What does momentum 0.9 roughly mean? → The velocity is an exponentially weighted sum of past gradients, covering about 1/(1−0.9) = 10 recent steps.
2. Adam + L2 vs AdamW? → AdamW shrinks weights directly, independent of the adaptive scaling.
3. A schedule used for LLMs? → Linear warmup, then cosine decay.

TL;DR

Learning rate first. Adam/AdamW (lr ≈ 3e-4 to 1e-3) is the safe default; SGD+momentum with a schedule is strong for CNNs; warmup + cosine or one-cycle.

10.6 Initialization and vanishing/exploding gradients

The problem it solves

The gradient reaching layer 1 is a **product** of one factor per layer above it. Factors typically below 1 shrink it exponentially with depth — **vanishing gradients** (early layers barely learn); factors above 1 grow it exponentially — **exploding gradients** (huge updates, NaN loss). Forward activations can likewise fade to 0 or blow up.

Two sources:

1. **Activation derivatives.** The sigmoid derivative is at most 0.25, so ten sigmoid layers scale the gradient by at most $0.25^{10} \approx 9.5 \times 10^{-7}$.
2. **Weight scale.** Each layer multiplies by W; badly scaled weights shrink or grow the signal's variance by a constant factor per layer.

How initialization fixes it

Goal: keep the **variance** of activations (forward) and gradients (backward) roughly constant from layer to layer. Why 1/fan_in? A pre-activation $z = \Sigma\, w_i x_i$ sums fan_in independent terms, so with zero-mean weights and inputs, $\mathrm{Var}(z) = \mathrm{fan_in} \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(x)$. Choosing $\mathrm{Var}(w) = 1/\mathrm{fan_in}$ makes $\mathrm{Var}(z) = \mathrm{Var}(x)$. With fan_in = 256: std = 1/16 = 0.0625. Xavier also balances the backward pass (fan_out); He doubles the variance for ReLU:

Scheme	Weight variance	Designed for	PyTorch
Xavier / Glorot (2010)	2 / (fan_in + fan_out) (normal version)	tanh, sigmoid, linear	`nn.init.xavier_normal_` / `xavier_uniform_`
He / Kaiming (2015)	2 / fan_in	ReLU family (ReLU zeroes half the signal, so double the variance)	`nn.init.kaiming_normal_(w, nonlinearity="relu")`
LeCun	1 / fan_in	SELU, linear	`kaiming_normal_(w, nonlinearity="linear")`

Biases usually start at 0. **Never give all weights the same value** (e.g. zeros): every neuron in a layer would compute the same thing and get the same gradient forever — the **symmetry** is never broken. PyTorch's `nn.Linear` default (Kaiming-uniform with a = √5, i.e. U(−1/√fan_in, 1/√fan_in)) is fine for small nets; explicit He init is common for deep ReLU nets.

`code/init_and_gradients.py` pushes random data through a **20-layer, 256-wide MLP** and measures the spread of activations and the gradient reaching layer 1:

```
tanh  + small N(0, 0.01)  activation std layer1=0.163 layer20=1.22e-16   |grad W1|=7.05e-18
tanh  + large N(0, 1)     activation std layer1=0.975 layer20=0.974    |grad W1|=8.04e+07
tanh  + Xavier            activation std layer1=0.63 layer20=0.16    |grad W1|=0.0255
...
ReLU  + large N(0, 1)     activation std layer1=14.6 layer20=8.73e+20   |grad W1|=inf
ReLU  + Xavier            activation std layer1=0.911 layer20=0.000722   |grad W1|=9.78e-06
ReLU  + He (Kaiming)      activation std layer1=1.29 layer20=0.739   |grad W1|=0.292
```

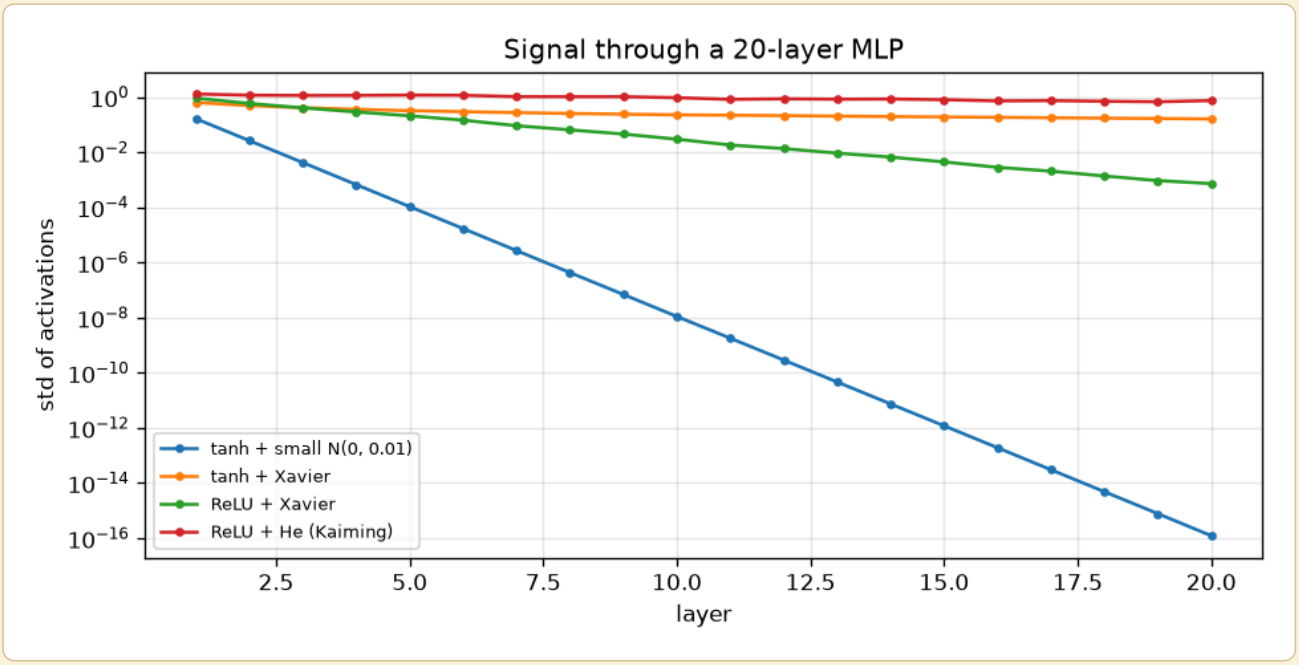


Figure — With too-small weights the signal dies within a few layers; ReLU with Xavier init shrinks by about half per layer; ReLU with He init keeps it steady.

Reading it: too-small weights → signal and gradient vanish (10^{-18}); too-large → tanh saturates and gradients explode through the big weights, ReLU overflows to infinity; He for ReLU keeps everything near 1. (Xavier with tanh still decays slowly because tanh squashes; PyTorch's tanh gain 5/3, `nn.init.calculate_gain("tanh")`, compensates.)

The full toolbox against vanishing/exploding gradients

Fix	How it helps
ReLU-family activations	Derivative is exactly 1 for positive inputs — no shrinking factor
Good initialization (He/Xavier)	Keeps variance ≈ constant across layers at the start
Normalization layers (BatchNorm, LayerNorm — §10.7)	Re-centre and re-scale activations at every layer
Residual (skip) connections (Chapter 11)	Give gradients an identity "highway": y = x + F(x)
Gradient clipping	Caps the gradient norm (`clip_grad_norm_(params, 1.0)`); standard for RNNs and transformers
Gated units (LSTM/GRU — Chapter 12)	Additive memory paths through time

Common mistakes

- All-zero initialization for hidden weights.
- Very deep custom nets with default init and no normalization or residuals.
- Not monitoring per-layer gradient norms — they diagnose most "the loss won't move" bugs.

In the interview

30-second answer: "Gradients are products of per-layer factors, so in deep nets they shrink or grow exponentially — caused by saturating activations (sigmoid' ≤ 0.25) and badly scaled weights. Fixes: ReLU, variance-preserving init (Xavier for tanh, He for ReLU), normalization, residual connections, and clipping for explosions."

- *Q: Why He init has a factor 2?* A: ReLU zeroes about half the inputs, halving the variance; doubling the weight variance compensates.
- *Q: Why not zero init?* A: Symmetry — identical neurons get identical gradients and never differentiate.

Self-check

1. Max derivative of sigmoid? → 0.25.
2. He init variance for fan_in = 256? → 2/256 ≈ 0.0078 (std ≈ 0.088).
3. Two fixes for exploding gradients? → Gradient clipping; a lower LR or better init/normalization.

TL;DR

Many multiplied factors → vanishing/exploding gradients. ReLU + He init (tanh + Xavier), normalization, residuals, clipping.

10.7 Regularization: fighting overfitting

The problem it solves

A big network can **memorise** a small training set, noise included, and fail on new data — **overfitting** (Chapter 1): training loss keeps falling while validation loss rises. **Regularization** is any technique that improves generalisation, usually by limiting how freely the model fits the training data.

`code/regularization.py` deliberately overfits a 3-hidden-layer, 256-unit MLP (~133k parameters) on only **80** noisy two-moons points (validated on 2,000 fresh ones), then tries each remedy.

```
for epoch in range(EPOCHS):
    model.train()                                  # dropout ON, BN uses batch stats
    for idx in torch.randperm(len(Xtr)).split(16):
        xb = Xtr[idx] + noise * torch.randn_like(Xtr[idx])   # augmentation: jitter inputs
        loss = lossf(model(xb).squeeze(1), ytr[idx])
        opt.zero_grad(); loss.backward(); opt.step()
    model.eval()                                   # dropout OFF, BN uses running stats
    # ... record val loss; early stopping keeps the best weights, stops after `patience` bad epochs
```

Output:

```
No regularization            epochs=400  train acc=1.000  val acc=0.831  val loss=2.296
Dropout p=0.5                epochs=400  train acc=0.975  val acc=0.840  val loss=1.290
Weight decay 1.0 (AdamW)     epochs=400  train acc=1.000  val acc=0.826  val loss=0.792
BatchNorm                    epochs=400  train acc=0.938  val acc=0.858  val loss=0.399
Input-noise augmentation     epochs=400  train acc=0.925  val acc=0.858  val loss=0.422
Early stopping (patience 30) epochs= 48  train acc=0.887  val acc=0.872  val loss=0.317
```

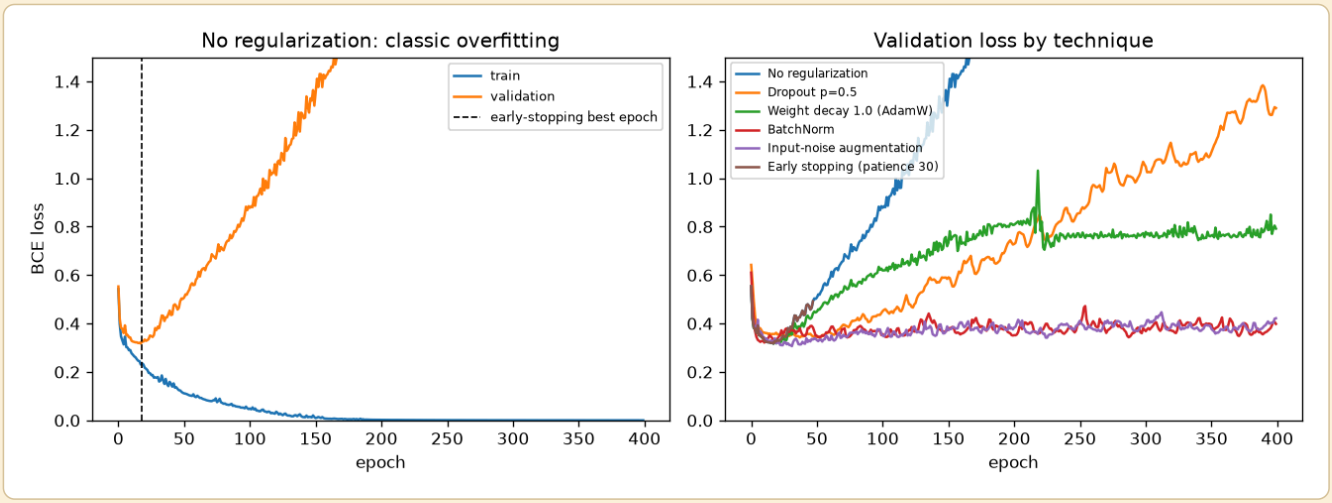


Figure — Left: without regularization, validation loss bottoms out around epoch 18 and then climbs while training loss goes to 0 — textbook overfitting. Right: each technique flattens the validation curve.

Every method improves validation **loss** (less over-confidence), but accuracy gains are modest; early stopping, augmentation and BatchNorm help most here. Real projects combine several.

Dropout

During training, each unit's output is zeroed with probability p (0.5 for large dense layers, 0.1–0.3 elsewhere), independently on every forward pass, so a different random "thinned" network trains at each step. Units can't rely on specific partners, so they can't form fragile **co-adaptations** and learn redundant, robust features — roughly an ensemble of many sub-networks. PyTorch's `nn.Dropout(p)` is **inverted dropout**: survivors are multiplied by 1/(1 − p) during training so the expected value is unchanged, and in `model.eval()` dropout does nothing. Example: p = 0.5 and activations (2, 4, 6, 8) might become (0, 8, 12, 0).

Weight decay (L2 regularization)

Adds a penalty $\lambda \cdot \Sigma w^2$ to the loss — for SGD, equivalent to shrinking every weight by the factor $(1 - 2\eta\lambda)$ each step, since the penalty's gradient is $2\lambda w$. Smaller weights give a smoother function less able to chase noise. With Adam use **AdamW** (decay applied directly, not through the adaptive scaling). Typical λ: 1e-4 to 1e-2 for SGD-style L2, 0.01–0.1 for AdamW; biases and normalization parameters are often excluded.

Batch normalization

What it is. A layer (Ioffe & Szegedy, 2015) that, for each feature, normalizes the activations over the current **mini-batch** to mean 0 and variance 1, then applies a learnable scale γ and shift β:

$\hat{x} = (x - \mu_{batch}) / \sqrt{\sigma^2_{batch} + \varepsilon}$, $y = \gamma \hat{x} + \beta$

Worked numbers: one feature has batch values (2, 4, 6). $\mu = 4$, $\sigma^2 = (4 + 0 + 4)/3 = 2.667$, $\sqrt{\sigma^2 + \varepsilon} \approx 1.633$, so $\hat{x} = (-1.225, 0, 1.225)$. With $\gamma = 2$, $\beta = 0.5$: $y = (-1.95, 0.5, 2.95)$. γ and β let the network undo the normalization if that helps.

At training time it uses batch statistics and updates **running averages** of μ and σ^2; at evaluation it uses the running averages (so single-example inference works).

Benefits. Faster, more stable training, **higher usable learning rates**, less sensitivity to initialization, and a mild **regularizing** noise (each example is normalised with the statistics of a random batch). The original paper credited reducing "internal covariate shift" (layer inputs changing distribution as earlier layers learn); Santurkar et al. (2018) argue the main effect is a smoother loss landscape. The practical benefits are not in doubt.

Limits. Noisy with very small batches and awkward for variable-length sequences. **LayerNorm** (normalize across the features of each single example) is standard in transformers and RNNs; **GroupNorm** suits small-batch vision.

Early stopping

Evaluate validation loss every epoch, keep a copy of the best weights, stop after `patience` epochs without improvement and restore the best copy. Free and simple — the best single method in the demo. Because the validation set now drives a decision, report final numbers on a separate test set.

Data augmentation

Create extra training examples with label-preserving transformations: flips, crops, small rotations and colour jitter for images; time shifts and noise for audio; synonym swaps or back-translation for text. In the demo, small Gaussian noise on the 2-D inputs plays this role; Chapter 11 covers image augmentation.

Trade-offs

Technique	Knob	Cost / watch out
Dropout	p	Slower convergence; needs `eval()`; less used with BatchNorm in CNNs
Weight decay	λ	Free; use AdamW with Adam
BatchNorm	momentum, eps	Small batches; train/eval differences
Early stopping	patience	Needs a validation set; restore the best weights
Data augmentation	strength	CPU time; must not change the label (a rotated "6" becomes "9")
Smaller model / more data	—	Often the most effective of all

Common mistakes

- Evaluating in `train()` mode; randomly augmenting validation/test data; early-stopping on training loss.

Real-world examples

AlexNet (2012): dropout 0.5 in its dense layers plus heavy augmentation. ResNets: BatchNorm after every convolution. Transformers: LayerNorm, dropout, AdamW weight decay.

In the interview

> **30-second answer:** "Dropout randomly zeroes units so the net can't rely on co-adapted features; weight decay penalises large weights; BatchNorm normalises per mini-batch, allowing higher learning rates and adding mild noise; early stopping halts when validation loss stops improving; augmentation adds label-preserving variants. More data is the best regularizer."

- *Q: What does dropout do at test time?* A: Nothing — inverted dropout already scaled during training, so eval uses the full network.
- *Q: BatchNorm at inference?* A: Uses the running mean/variance collected during training, not batch statistics.

Self-check

1. Dropout p = 0.3 in PyTorch: surviving activations during training? → Scaled by 1/0.7.
2. Which normalization do transformers use? → LayerNorm (or its cheaper cousin RMSNorm).
3. Early stopping patience 10? → Stop after 10 epochs without validation improvement; restore the best weights.

TL;DR

Overfitting = train loss ↓, val loss ↑. Combine early stopping, weight decay (AdamW), dropout, normalization and augmentation; more data beats all of them.

10.8 Practical training habits

Normalize the inputs

Features on wildly different scales (pixels 0–255 next to incomes in the hundreds of thousands) stretch the loss surface into a ravine (see the optimizer figure), making training slow and unstable:

Method	Formula	When
Min-max scaling	x' = (x − min)/(max − min) → [0, 1] (or rescale to [−1, 1])	Bounded data such as pixels (divide by 255)
Standardization (z-score)	x' = (x − mean)/std → mean 0, std 1	Most tabular features; per-channel for images (ImageNet mean/std)

Rule: compute the statistics on the **training set only** and apply the same numbers to validation and test data; using all data leaks test information (Chapter 4).

Mini-batches

One example per update is slow and noisy; the whole dataset per update is memory-hungry and gives few updates. **Mini-batches** are the compromise.

Batch size	Gradient estimate	Updates per epoch	Memory	Notes
Small (e.g. 16–64)	Noisy	Many	Low	Noise often helps generalisation
Large (e.g. 256–4096)	Smooth	Few	High	GPU-efficient; needs larger LR + warmup

Start around 32–256. Erratic training → larger batch or smaller LR; out of GPU memory → smaller batch, or **gradient accumulation** (sum gradients over several small batches before one `step()`).

Class imbalance

When one class is rare (fraud, disease, defects), always predicting the majority class scores high accuracy — and is useless.

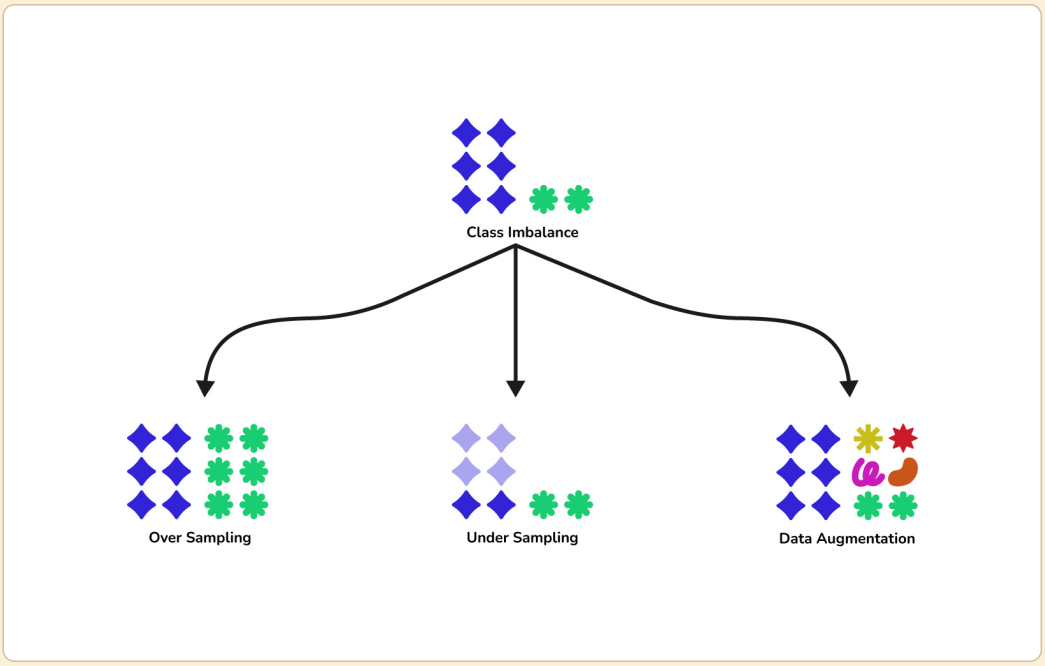


Figure — An imbalanced dataset (top) and three remedies: oversampling the minority class, undersampling the majority class, and augmentation to create new minority examples (course diagram).

Method	How	Risk
Oversampling	Repeat minority examples or synthesise new ones (SMOTE interpolates between minority neighbours; augmentation for images); `WeightedRandomSampler`	Overfitting to repeats
Undersampling	Randomly drop majority examples	Throws away information
Class weights	Minority mistakes cost more: `CrossEntropyLoss(weight=…)`, `BCEWithLogitsLoss(pos_weight=…)`	Probabilities no longer calibrated
Threshold tuning / better metrics	Evaluate with precision, recall, F1, PR-AUC (Chapter 6); tune the decision threshold	—

From `code/practical_tips.py` (95% / 5% synthetic data). The weight for class c is n / (k · n_c) — n examples, k classes, n_c examples of class c: here 2000/(2·112) ≈ 8.9 for the minority and 2000/(2·1888) ≈ 0.53 for the majority:

```
class counts in train: [1888, 112]  (minority = 5.6%)
plain cross-entropy          accuracy=0.957  minority recall=0.369
class-weighted (n/(k*n_c))   accuracy=0.939  minority recall=0.640
(always predicting the majority class would score accuracy 0.945)
```

Accuracy barely moves (and sits near the useless "always majority" baseline), but the weighted loss finds 64% of the rare cases instead of 37%.

Reading loss curves

What you see	Likely cause	What to try
Loss flat from step 0, near ln(K)	LR far too small, or a bug (shuffled labels, missing `optimizer.step()`)	Overfit one batch; check the pipeline
Loss goes up / NaN / wild spikes	LR too high, exploding gradients, log(0)	Lower LR, gradient clipping, use logit-based losses
Training ↓, validation ↓ then ↑	Overfitting	Regularization, early stopping, more data
Training and validation both high and close	Underfitting	Bigger model, train longer, higher LR, better features
Validation better than training	Dropout/augmentation only active in training, or an easier validation set	Usually fine — check the split
Noisy, jagged curve	Small batch or high LR	Larger batch, lower LR, smooth the plot

Overfit a small batch first

Before a long run, train on **one mini-batch** repeatedly; a correct model and pipeline drive its loss almost to zero within a few hundred steps. If not, there's a bug (wrong labels or loss, no gradients flowing) that no tuning will fix. Also check the **first** loss matches chance (ln 10 = 2.303 for 10 classes).

```
1) Overfit one batch of 32 examples
  step   0  loss on the single batch = 2.3176
  step  50  loss on the single batch = 0.0008
  step 200  loss on the single batch = 0.0002
  expected start ~ ln(10) = 2.303 (uniform guess over 10 classes)
```

The learning-rate finder (LR range test)

Leslie Smith's trick, popularised by fast.ai: start with a tiny LR, **multiply it by a constant after every mini-batch** (exponential growth, e.g. 1e-5 → 10), and record the smoothed loss. It is flat while the LR is too small, drops once it is useful, and explodes when it is too large. Pick a value on the steep descent — commonly **one-tenth of the LR at the loss minimum**.

In `code/practical_tips.py` the multiplier is (10 / 1e-5)^(1/149) over 150 batches, the loss is smoothed with an exponential moving average (0.9·old + 0.1·new), and the test stops once the smoothed loss exceeds 4× its best. Output:

```
2) LR range test (SGD+momentum, lr grows 1e-5 -> 10)
  lowest smoothed loss at lr=0.619; loss blew up by lr=2.27; rule of thumb: pick ~1/10 of the minimum -> 0.062
3) Training curves for three learning rates (SGD+momentum, 30 epochs)
  lr=0.0001 final train loss 2.252  val loss 2.250
  lr=0.05   final train loss 0.035  val loss 0.111
  lr=3      final train loss 2.443  val loss 2.444
```

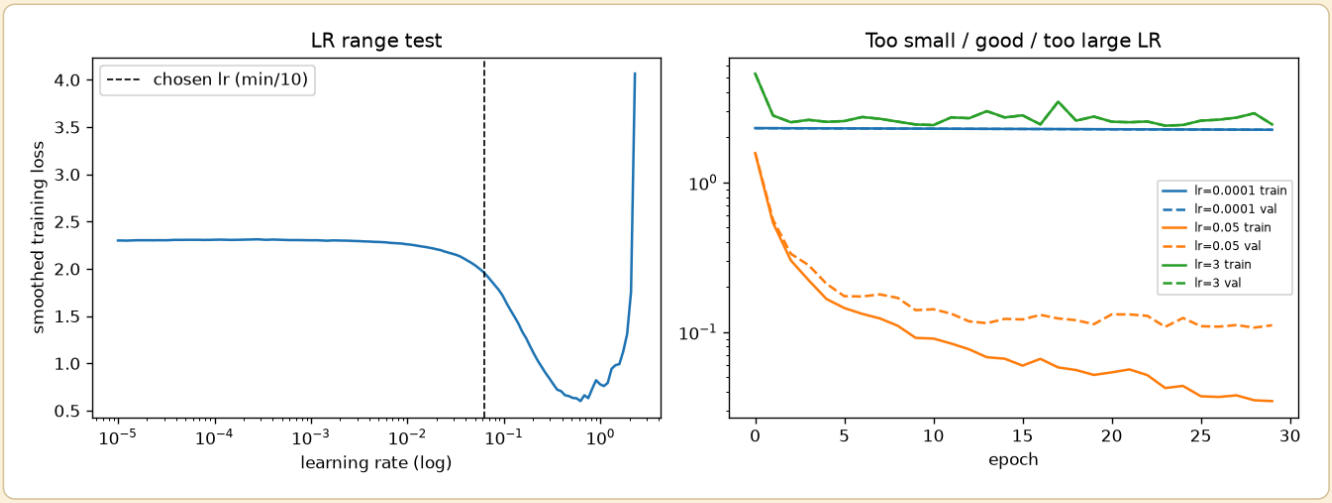


Figure — Left: LR range test on the digits MLP. Right: the LR the test suggests (≈0.05) trains well; 1e-4 barely moves in 30 epochs; 3.0 bounces around chance level.

A debugging checklist

Inspect raw examples and labels after the pipeline → check chance-level first loss → overfit one batch → LR finder (or Adam 1e-3) → train with validation curves → only then regularize and tune; keep a simple baseline (logistic regression, gradient boosting) to know what "good" means. This is the spirit of Karpathy's "A Recipe for Training Neural Networks": know the data, start simple, verify each piece, then scale.

In the interview

30-second answer: "Normalise inputs with training-set statistics, start from a known-good architecture and Adam, check the initial loss equals chance, overfit one batch to rule out bugs, pick the LR with a range test, then read train/validation curves for under- vs overfitting. For imbalance, use class weights or resampling and judge by recall/precision/PR-AUC, not accuracy."

Self-check

1. Initial loss 8.0 on a 10-class problem? → Far above ln 10 ≈ 2.3: bad init/scale, wrong labels or loss — investigate first.
2. How does the LR finder choose a value? → Grow LR exponentially per batch, plot loss vs LR, pick about one-tenth of the LR at the minimum.
3. Three class-imbalance remedies? → Oversampling (or SMOTE), undersampling, class-weighted loss.

TL;DR

Train-set normalisation; batches of 32–256; weights/resampling plus proper metrics for imbalance; chance-level first loss, overfit one batch, LR finder, read the curves.

10.9 PyTorch basics

What it is

PyTorch (created at Meta AI, now governed by the PyTorch Foundation) is the most widely used deep-learning framework in research and common in production. Its core pieces:

Piece	What it does
`torch.Tensor`	N-dimensional array like NumPy's, but can live on a GPU and track gradients
autograd	Records operations on tensors with `requires_grad=True`; `.backward()` fills `.grad`
`nn.Module`	Base class for models/layers; registers parameters and sub-modules; you write `forward()`
`nn.Linear`, `nn.Conv2d`, `nn.ReLU`, `nn.Dropout` …	Ready-made layers (`torch.nn.functional` has stateless versions)
`torch.optim`	Optimizers (SGD, Adam, AdamW…) and `lr_scheduler`
`Dataset` / `DataLoader`	Index into data; batch, shuffle and (optionally) load in parallel worker processes

How it works — the training loop

Five lines per mini-batch: **forward → loss → zero_grad → backward → step.**

```python
# code/pytorch_basics.py (core)
class WineNet(nn.Module):
    def __init__(self, n_in, n_hidden, n_classes):
        super().__init__()                          # registers sub-modules and parameters
        self.net = nn.Sequential(nn.Linear(n_in, n_hidden), nn.ReLU(),
                                 nn.Linear(n_hidden, n_classes))   # logits, no softmax
    def forward(self, x):
        return self.net(x)

mean, std = Xtr.mean(0), Xtr.std(0)                 # standardize with TRAIN statistics only
Xtr, Xte = (Xtr - mean) / std, (Xte - mean) / std
train_ds = TensorDataset(torch.tensor(Xtr, dtype=torch.float32), torch.tensor(ytr, dtype=torch.long))
train_dl = DataLoader(train_ds, batch_size=32, shuffle=True)

model = WineNet(13, 32, 3)
loss_fn = nn.CrossEntropyLoss()                     # = log-softmax + negative log-likelihood
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for epoch in range(1, 31):
    model.train()
    for xb, yb in train_dl:
        logits = model(xb)                          # 1. forward
        loss = loss_fn(logits, yb)                  # 2. loss
        opt.zero_grad()                             # 3. clear old gradients
        loss.backward()                             # 4. backprop
        opt.step()                                  # 5. update weights

    model.eval()
    with torch.no_grad(): ...                       # evaluate: no graph building

torch.save(model.state_dict(), "wine.pt")           # save weights; the script reloads them into a fresh WineNet
```

Output (tensor demos and the model printout trimmed):

```
...
autograd dy/dx at x=2: 7.0
...
trainable parameters: 547
epoch  1  train loss 0.702 acc 0.944 | test loss 0.684 acc 0.907
epoch  5  train loss 0.092 acc 0.992 | test loss 0.083 acc 0.981
epoch 10  train loss 0.022 acc 1.000 | test loss 0.031 acc 0.981
epoch 30  train loss 0.003 acc 1.000 | test loss 0.018 acc 0.981
reloaded model test acc: 0.9814814814814815
...
```

Checks: $y = x^2 + 3x$ at $x = 2$ gives $dy/dx = 2 \cdot 2 + 3 = 7$, exactly what `.backward()` put in `x.grad`; parameters = $13 \cdot 32 + 32 + 32 \cdot 3 + 3 = 547$. `model.train()` / `model.eval()` switch dropout and BatchNorm behaviour; `torch.no_grad()` saves memory and time when gradients aren't needed; `state_dict()` (an ordered dict of parameter and buffer tensors) is the recommended way to save and load.

GPU: `device = "cuda" if torch.cuda.is_available() else "cpu"` (Apple Silicon: `"mps"`); move the model and every batch with `.to(device)`. Mixed precision (`torch.autocast`) and `torch.compile(model)` speed up real workloads.

Custom Dataset: for data on disk, subclass `Dataset` with `__len__` and `__getitem__(i)` returning one (x, y) pair; `DataLoader(ds, batch_size=64, shuffle=True, num_workers=4)` batches, shuffles and loads in parallel.

Common mistakes

- Forgetting `optimizer.zero_grad()`, or `model.eval()` / `torch.no_grad()` at evaluation.
- dtype mismatches: `CrossEntropyLoss` wants `long` class labels; `BCEWithLogitsLoss` wants float targets shaped like the logits.
- Calling `.item()` / `.numpy()` every step on a GPU — it forces a synchronisation.
- Not shuffling the training loader.

Real-world examples

PyTorch underlies Meta's production models, OpenAI's research code and most Hugging Face models. Wrappers — PyTorch Lightning, Hugging Face `Trainer`, fastai — hide the loop but run exactly these five steps.

In the interview

30-second answer: "Subclass nn.Module and write forward; wrap a Dataset in a DataLoader for batching and shuffling; each step is forward, loss, zero_grad, backward, optimizer.step. Switch model.train()/eval() for dropout and BatchNorm, use torch.no_grad() for inference, and save the state_dict."

- *Q: Why must you call zero_grad?* A: `.backward()` adds into `.grad`; without zeroing you'd sum gradients from several batches (sometimes done on purpose: gradient accumulation).
- *Q: Difference between `nn.ReLU` and `F.relu`?* A: Same maths; the module form can sit in `nn.Sequential`, the functional form is called inside `forward`.

Self-check

1. The five lines of a training step? → forward, loss, zero_grad, backward, step.
2. What does `model.eval()` change? → Dropout off; BatchNorm uses running statistics.
3. Parameters of `nn.Linear(100, 10)`? → 100·10 + 10 = 1,010.

TL;DR

Tensors + autograd + nn.Module + DataLoader + optimizer. Loop: forward → loss → zero_grad → backward → step. train()/eval(), no_grad(), save state_dict.

Chapter TL;DR

- **Neuron** = weighted sum + bias + activation; **MLP** = stacked layers building a feature hierarchy; one hidden layer solves XOR.
- **Activations/losses:** ReLU/GELU inside; identity/sigmoid/softmax out; MSE, BCE, cross-entropy on logits.
- **Backprop** = chain rule to every weight; then $\theta \leftarrow \theta - \eta \nabla L$ with SGD/momentum/Adam(W) and a warmup + cosine schedule.
- **Deep-net stability:** ReLU, He/Xavier init, normalization, residuals, clipping. **Regularization:** dropout, weight decay, early stopping, augmentation, more data.
- **Practice:** chance-level first loss, overfit one batch, LR finder, read the curves; PyTorch's forward/loss/zero_grad/backward/step loop.

Next: Chapter 11 applies these ideas to images with convolutional networks; Chapter 12 to sequences with RNNs and attention.

Chapter 11 — CNNs and Computer Vision

An image is just a grid of numbers — a 224×224 colour photo is 224 × 224 × 3 = 150,528 numbers. Feeding that into the dense networks of Chapter 10 works badly: far too many weights, and no notion that neighbouring pixels belong together or that a cat is still a cat when it moves ten pixels to the left. **Convolutional neural networks (CNNs)** fix both problems with one idea: slide a small learnable filter across the image and reuse it everywhere. That idea, plus depth, took computer vision from "barely works" to superhuman on some benchmarks within a decade.

What you'll learn

1. Convolution: kernels, stride, padding, channels, output-size formula — with a worked 3×3 example.
2. Pooling, and how conv + pool + dense layers make a CNN architecture.
3. Receptive field and why stacks of 3×3 convolutions work.
4. The famous architectures: LeNet → AlexNet → VGG → Inception → ResNet (skip connections) → MobileNet/EfficientNet → Vision Transformer.
5. Transfer learning and fine-tuning.
6. Data augmentation for images.
7. Vision tasks beyond classification: object detection (IoU, NMS, the YOLO idea) and segmentation (the U-Net idea).
8. A small PyTorch CNN on scikit-learn's 8×8 digits, trained on CPU in seconds.

Prerequisites: Chapter 10 (neurons, backprop, optimizers, BatchNorm, dropout, PyTorch training loop). All code is in `code/`; outputs are real.

11.1 Convolution

What it is

A **convolutional layer** slides a small grid of weights — a **kernel** or **filter**, typically 3×3 — over the image. At each position it multiplies the kernel element-wise with the patch of pixels underneath and sums the result into a single number. Doing this at every position produces a new grid called a **feature map** (or activation map). A layer has many kernels, each producing its own feature map, each detecting a different pattern.

Analogy: a flashlight with a stencil. You sweep the same stencil across the whole picture; wherever the picture matches the stencil's pattern, the light shines bright.

The problem it solves

Problem with dense layers on images	How convolution fixes it
Huge parameter counts (150k inputs × 1,000 units = 150M weights for one layer)	**Weight sharing**: one 3×3×3 kernel has 27 weights (+1 bias) regardless of image size
Ignores spatial structure — pixel order could be shuffled	**Local connectivity**: each output only looks at a small neighbourhood
A pattern learned in one place must be re-learned in every other place	**Translation equivariance**: shift the input, and the feature map shifts the same way — the same detector works everywhere

Parameter comparison from `code/conv_by_hand.py`:

```
Conv2d(3->16, 3x3) params: 448  vs Linear(32*32*3 -> 16) params: 49168
```

$(3 \cdot 3 \cdot 3 + 1) \cdot 16 = 448$ versus $(3{,}072 + 1) \cdot 16 = 49{,}168$ — and the conv layer's count doesn't change with image size.

How it works — worked 3×3 example

Input X (5×5, one channel) and a vertical-edge kernel K:

```
X =  1 2 0 1 3       K =  1 0 -1
     0 1 3 2 1            1 0 -1
     2 1 0 1 0            1 0 -1
     1 0 2 3 1
     0 2 1 0 2
```

1. Place K on the top-left 3×3 patch `[[1,2,0],[0,1,3],[2,1,0]]`. Multiply element by element and add: left column (1+0+2)·1 + middle column ·0 + right column (0+3+0)·(−1) = 3 − 3 = **0**.
2. Slide one step right and repeat; at the end of the row move one step down.
3. The patch centred on the middle pixel is `[[1,3,2],[1,0,1],[0,2,3]]`: left column 1+1+0 = 2, right column 2+1+3 = 6, so the output is 2 − 6 = **−4** — a strong response: it's brighter on the right than on the left, a vertical edge.
4. Add the bias (0 here) and, in a real network, apply an activation such as ReLU.

The full output with stride 1 and no padding is 3×3:

```
stride=1 padding=0 -> output (3, 3) (formula 3), matches torch: True
[[ 0.  0. -1.]
 [-2. -4.  3.]
 [ 0. -1.  0.]]
```

Strictly, deep-learning "convolution" is **cross-correlation** — the kernel isn't flipped as in the mathematical definition. Since kernels are learned, the distinction doesn't matter; PyTorch's `F.conv2d` does exactly what the loop below does.

```python
# code/conv_by_hand.py (core)
def conv2d(x, k, stride=1, padding=0):
    x = np.pad(x, padding)
    kh, kw = k.shape
    oh = (x.shape[0] - kh) // stride + 1
    ow = (x.shape[1] - kw) // stride + 1
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            patch = x[i * stride:i * stride + kh, j * stride:j * stride + kw]
            out[i, j] = np.sum(patch * k)          # element-wise multiply, then sum
    return out
```

Stride and padding

- **Stride** s: how many pixels the kernel jumps each move. Stride 2 halves the output's height and width — a cheap way to downsample.
- **Padding** p: extra border (usually zeros) around the input. Without it, every 3×3 conv shrinks the image by 2 pixels and edge pixels are under-used. **"Same" padding** (p = 1 for a 3×3 kernel, p = 2 for 5×5) keeps the size unchanged at stride 1. **"Valid"** means no padding.

Output size formula (per spatial dimension):

out = ⌊(n + 2p − k) / s⌋ + 1

n	k	s	p	out	Check
5	3	1	0	(5 − 3)/1 + 1 = 3	valid conv shrinks by 2
5	3	2	0	⌊2/2⌋ + 1 = 2	stride 2 downsamples
5	3	1	1	(5 + 2 − 3)/1 + 1 = 5	same padding
224	7	2	3	⌊(224 + 6 − 7)/2⌋ + 1 = 112	ResNet's first layer

```
stride=2 padding=0 -> output (2, 2) (formula 2), matches torch: True
[[ 0. -1.]
 [ 0.  0.]]

stride=1 padding=1 -> output (5, 5) (formula 5), matches torch: True
[[-3. -2.  0. -1.  3.]
 [-4.  0.  0. -1.  4.]
 [-2. -2. -4.  3.  6.]
 [-3.  0. -1.  0.  4.]
 [-2. -2. -1.  0.  3.]]
```

Channels

Real images have 3 **channels** (red, green, blue), and hidden feature maps have many (64, 128, 512…). A kernel always spans **all input channels**: a 3×3 kernel on a 64-channel input is really 3×3×64 = 576 weights, producing one output channel. A layer with C_out kernels produces C_out channels.

- Weights in a conv layer: **k × k × C_in × C_out** (+ C_out biases).
- PyTorch tensor layout: **N × C × H × W** (batch, channels, height, width). `nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`.
- A **1×1 convolution** mixes channels at each pixel without looking at neighbours — used to cheaply change the number of channels (Inception, ResNet bottlenecks).
- A **depthwise-separable convolution** splits a conv into a per-channel k×k spatial conv plus a 1×1 conv that mixes channels (MobileNet, Xception, EfficientNet). Cost per output pixel: standard = $k^2 \cdot C_{in} \cdot C_{out}$; separable = $k^2 \cdot C_{in} + C_{in} \cdot C_{out}$. The ratio is $1/C_{out} + 1/k^2$ — for k = 3, C_in = C_out = 256: 1/256 + 1/9 ≈ 0.115, about 8.7× cheaper.

What do kernels detect?

`code/edge_filters.py` applies classic hand-designed kernels to a digit. A CNN **learns** kernels like these — first-layer filters of trained networks look like edge, colour-blob and stripe detectors.

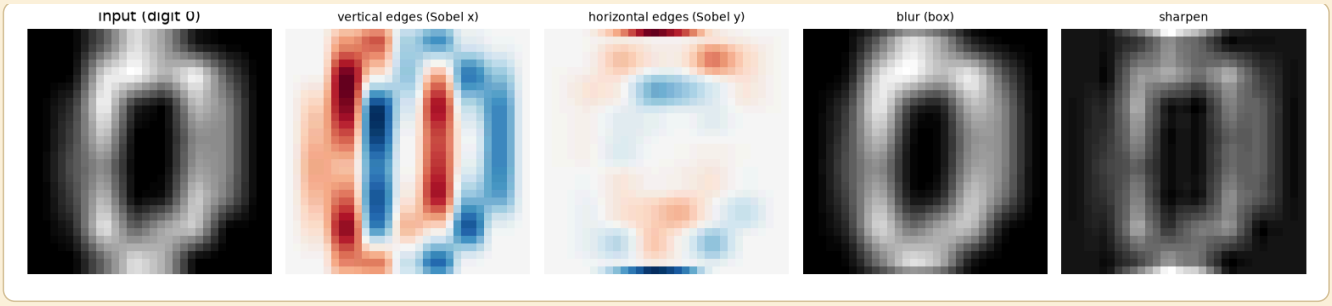


Figure — A "0" digit and the feature maps from Sobel vertical/horizontal edge detectors (red = positive response, blue = negative), a blur and a sharpen kernel.

Common mistakes

- Mixing up N×C×H×W (PyTorch) and N×H×W×C (TensorFlow/Keras default, NumPy images) — transpose with `permute(0, 3, 1, 2)`.
- Forgetting padding and being surprised when the image shrinks layer after layer.
- Computing the flatten size by hand wrongly — pass a dummy tensor through the conv part and print its shape, or use `nn.LazyLinear`.

In the interview

30-second answer: "A conv layer slides small learned kernels, each spanning all input channels, across the image and sums the element-wise products into a feature map. Weight sharing makes the parameter count independent of image size, and locality plus sharing give translation equivariance. Output size is ⌊(n + 2p − k)/s⌋ + 1; stride downsamples, padding preserves size; 1×1 convs mix channels cheaply."

- *Q: Output of a 224×224×3 image through `Conv2d(3, 64, 7, stride=2, padding=3)`?* A: 64 × 112 × 112; weights 7·7·3·64 + 64 = 9,472.
- *Q: Is convolution translation invariant?* A: Equivariant — the feature map shifts with the input; pooling and global pooling add (approximate) invariance.

Self-check

1. 5×5 input, 3×3 kernel, stride 1, padding 0 — output size? → 3×3.
2. Weights in a 3×3 conv from 64 to 64 channels (no bias)? → 3·3·64·64 = 36,864.
3. Why is "convolution" in deep learning really cross-correlation? → The kernel isn't flipped; since it is learned, it makes no difference.

TL;DR

A conv layer slides shared k×k×C_in kernels over the input; out = ⌊(n + 2p − k)/s⌋ + 1; weights = k·k·C_in·C_out. Weight sharing + locality = far fewer parameters and translation equivariance.

11.2 Pooling and the CNN architecture

What it is

A **pooling layer** downsamples each feature map independently by summarising small windows:

- **Max pooling** (most common, 2×2 window, stride 2): keep the largest value — "was the feature present anywhere in this window?"
- **Average pooling**: take the mean.
- **Global average pooling (GAP)**: average each whole feature map to a single number — used at the end of modern CNNs instead of big dense layers.

Pooling has **no learnable parameters**. It shrinks H and W (2×2/2 halves them), cutting compute, growing the receptive field, and adding a little tolerance to small shifts (**local translation invariance**).

Max-pool 2×2 stride 2 of the 5×5 padded-conv output above (the last row and column are dropped because the 2×2 windows don't fit):

```
max-pool 2x2/2 of the padded-conv output (5x5 -> 2x2):
 [[0. 0.]
 [0. 3.]]
avg-pool via torch:
 [[-2.25 -0.5 ]
 [-1.75 -0.5 ]]
```

Top-left window [[−3, −2], [−4, 0]] → max 0, mean −2.25.

Many modern networks replace pooling with **stride-2 convolutions** (learned downsampling) and use pooling only at the very end (GAP).

The standard CNN architecture



1. **Feature extractor ("backbone")**: repeated blocks of conv → normalization → activation, with periodic downsampling. As you go deeper, spatial size **shrinks** and the number of channels **grows** (e.g. 64 → 128 → 256 → 512): fewer, more abstract, more numerous features.
2. **Classifier ("head")**: flatten or global-average-pool, then one or more dense layers producing class logits; softmax + cross-entropy for training.

The learned features form a hierarchy (Zeiler & Fergus visualised this in 2013): **edges and colours → textures and corners → object parts → whole objects**.

Receptive field

The **receptive field (RF)** of a unit is the region of the input image that can influence it. A 3×3 conv has RF 3. Stacking layers grows it. Track two numbers per layer — the RF and the **jump** (how many input pixels one step in this layer's output corresponds to, i.e. the product of all strides so far):

$RF_{new} = RF_{old} + (k - 1) \cdot jump_{old}$, $jump_{new} = jump_{old} \cdot s$

Worked through the VGG pattern below: conv3 → 1 + 2·1 = 3; conv3 → 3 + 2·1 = 5; maxpool 2/2 → 5 + 1·1 = 6, jump becomes 2; conv3 → 6 + 2·2 = 10; conv3 → 10 + 2·2 = 14.

`code/receptive_field.py`:

```
Three 3x3 convs (stride 1) vs one 7x7 conv, on a 32x32 input
layer                  out size   RF  jump
conv3x3                      32    3     1
conv3x3                      32    5     1
conv3x3                      32    7     1
layer                  out size   RF  jump
conv7x7                      32    7     1
weights (C=64 channels in and out, no bias): three 3x3 = 110,592  vs one 7x7 = 200,704

VGG-style block pattern: conv, conv, pool, ... on 224x224
layer                  out size   RF  jump
conv3x3                     224    3     1
conv3x3                     224    5     1
maxpool2                    112    6     2
conv3x3                     112   10     2
conv3x3                     112   14     2
maxpool2                     56   16     4
conv3x3                      56   24     4
conv3x3                      56   32     4
maxpool2                     28   36     8
```

Key insight (VGG, 2014): three stacked 3×3 convs see the same 7×7 region as one 7×7 conv, but use $27C^2$ instead of $49C^2$ weights and add two extra nonlinearities. That's why almost every modern CNN uses 3×3 kernels. Downsampling makes the RF grow much faster ("jump" doubles after each pool), which is how deep layers come to "see" the whole object. (The *effective* receptive field — where influence is actually strong — is smaller and roughly Gaussian-shaped.)

Self-check

1. Output size of a 32×32 input after conv(k=5, s=1, p=0)? → 28×28.
2. Parameters of `Conv2d(64, 128, 3)`? → 3·3·64·128 + 128 = 73,856.
3. Why do channel counts grow as spatial size shrinks? → To keep representational capacity (and roughly the compute per layer) while features become more abstract and numerous.
4. RF of two 3×3 convs (stride 1)? → 5×5.

TL;DR

Pooling (usually 2×2 max) halves H and W with no parameters. CNN = backbone of conv-BN-ReLU blocks with downsampling + head of GAP and dense layers. Stacked 3×3 convs grow the receptive field cheaply.

11.3 A small CNN on 8×8 digits (PyTorch, CPU)

scikit-learn's `load_digits` has 1,797 grayscale 8×8 images of handwritten digits (pixel values 0–16). It's tiny, so a CNN trains on a laptop CPU in a few seconds.

```python
# code/digits_cnn.py (core)
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),   # 1x8x8  -> 16x8x8
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),                              # 16x8x8 -> 16x4x4
            nn.Conv2d(16, 32, kernel_size=3, padding=1),  # -> 32x4x4
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                              # -> 32x2x2
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),                                 # -> 128
            nn.Dropout(0.3),
            nn.Linear(32 * 2 * 2, 10),                    # 10 class logits
        )
    def forward(self, x):
        return self.classifier(self.features(x))

X = (X / 16.0).reshape(-1, 1, 8, 8).astype(np.float32)   # N x C x H x W, scaled to [0, 1]
# ... stratified 70/30 split, DataLoader(batch_size=64, shuffle=True)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-3)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=3e-3, total_steps=epochs * len(train_dl))
# ... standard loop: forward, cross_entropy, zero_grad, backward, step, sched.step()
```

The script trains three models for 40 epochs — an MLP with a similar parameter budget, the CNN, and the CNN with **shift augmentation** (each training image randomly moved up to 1 pixel) — then tests all three on the normal test set and on the test set **shifted 1 pixel right**.

Output:

```
train (1257, 1, 8, 8)  test (540, 1, 8, 8)
MLP                        params= 7210  test accuracy=0.9759
CNN                        params= 6186  test accuracy=0.9907
CNN + shift augmentation   params= 6186  test accuracy=0.9852
MLP                        accuracy on test images shifted 1px right = 0.4722
CNN                        accuracy on test images shifted 1px right = 0.8574
CNN + shift augmentation   accuracy on test images shifted 1px right = 0.9796
misclassified (count, true, predicted): [(2, 8, 1), (2, 3, 9), (1, 9, 7), (1, 9, 1), (1, 8, 9)]
```

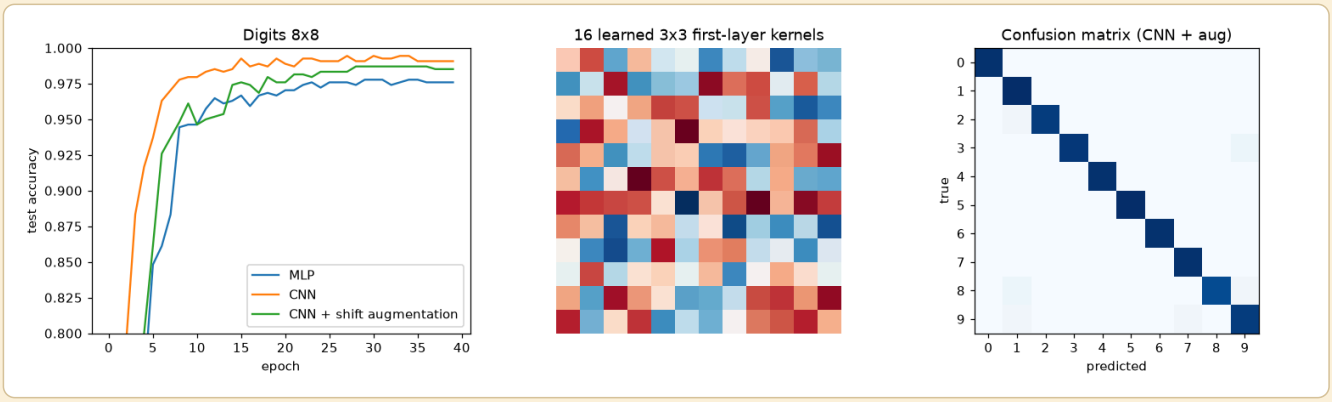


Figure — Left: test accuracy per epoch. Middle: the 16 first-layer kernels the CNN learned. Right: confusion matrix — nearly everything on the diagonal; the few errors are 8↔1 and 3↔9 type confusions.

What it shows:

- **Fewer parameters, higher accuracy**: the CNN (6.2k parameters) beats the MLP (7.2k) — 99.1% vs 97.6%.
- **Built-in spatial bias**: shift every test image by one pixel and the MLP collapses to 47% (every "pixel feature" moved), while the CNN keeps 86% thanks to shared kernels and pooling.
- **Augmentation buys robustness**: trained with random shifts, the CNN keeps 98% on shifted images — at the cost of half a point on the unshifted test set here (a small, typical trade-off on a tiny dataset).

TL;DR

On tiny 8×8 digits a two-block CNN beats a larger MLP (99.1% vs 97.6%) and is far more robust to shifts; shift augmentation makes it nearly shift-proof.

For real images (28×28 MNIST, 32×32 CIFAR-10, 224×224 ImageNet), the same code works with more blocks and channels; you'd load data with `torchvision.datasets` and run on a GPU.

Common mistakes

- Forgetting the channel dimension (8×8 images must be reshaped to 1×8×8).
- Evaluating with the model still in `train()` mode (BatchNorm and dropout behave differently).
- Too much downsampling for small images: 8×8 can only be halved three times.

11.4 Famous architectures — a timeline

What it is

A handful of architectures, most of them winners of the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** — 1.28 million training images across 1,000 classes — each introduced an idea that is still in use.

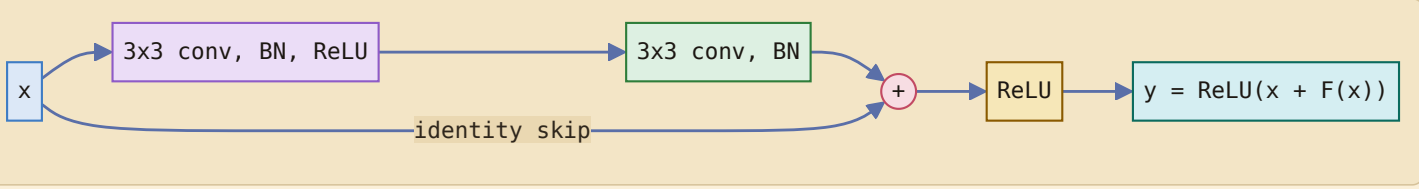
Year	Model	Depth / params	Key idea	Result
1998	**LeNet-5** (LeCun et al.)	7 layers, ~60k	Conv → pool → conv → pool → dense; trained with backprop	Read handwritten digits on bank cheques in production
2012	**AlexNet** (Krizhevsky, Sutskever, Hinton)	8 layers, ~60M	ReLU, dropout, heavy augmentation, trained on 2 GPUs	ILSVRC top-5 error 15.3% vs 26.2% runner-up — started the deep-learning boom
2014	**VGG-16/19** (Simonyan & Zisserman)	16–19 layers, ~138M (VGG-16)	Only 3×3 convs, stacked; very uniform	ILSVRC 2014 runner-up, 7.3% top-5; still a popular feature extractor
2014	**GoogLeNet / Inception** (Szegedy et al.)	22 layers, ~7M	Inception module: parallel 1×1, 3×3, 5×5 convs; 1×1 bottlenecks; GAP instead of big dense layers	ILSVRC 2014 winner, 6.7% top-5
2015	**ResNet** (He et al.)	18–152 layers (ResNet-50 ≈ 25.6M)	**Residual (skip) connections** + BatchNorm	ILSVRC 2015 winner, 3.57% top-5 (ensemble); made 100+ layer nets trainable
2017	**MobileNet** (Howard et al.)	small	Depthwise-separable convs for phones	Efficient on-device vision
2019	**EfficientNet** (Tan & Le)	B0–B7 (5.3M–66M)	**Compound scaling**: grow depth, width and input resolution together with fixed ratios; SiLU, MBConv blocks	B7: 84.3% top-1 ImageNet, 8.4× smaller than the previous best (GPipe)
2020	**Vision Transformer (ViT)** (Dosovitskiy et al.)	86M+ (ViT-Base)	Split the image into 16×16 patches, treat them as tokens, apply a transformer	Beats CNNs when pre-trained on very large datasets
2022	**ConvNeXt** (Liu et al.)	ResNet-sized	A ResNet modernised with transformer-era design choices	Matches ViTs, showing CNNs remain competitive

ResNet and skip connections

The problem. Before 2015, making plain CNNs deeper beyond ~20 layers made them *worse* — even on the **training** set. This "degradation" isn't overfitting; very deep plain stacks are simply hard to optimise (signals and gradients get distorted through many layers).

The idea. Instead of asking a block to learn a mapping H(x) directly, let it learn the **residual** F(x) = H(x) − x and add the input back: **y = ReLU(x + F(x))**, where F = conv → BN → ReLU → conv → BN

If the best thing a block can do is nothing, it just learns F ≈ 0 — easy. And in backprop, ∂y/∂x includes an identity term, so gradients have a direct **highway** back to early layers. When the shapes differ (stride 2 or more channels), the skip path uses a 1×1 conv to match them.




```python
# code/resnet_block.py (core)
class ResidualBlock(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c))
    def forward(self, x):
        return torch.relu(x + self.f(x))                # the skip connection
```

Output — gradient norm reaching the first conv, and a 30-conv-layer plain net vs a 30-conv-layer residual net trained on digits for 5 epochs:

```
conv layers  plain |grad|  residual |grad|
          2      2.58e-02         4.78e-02
         10      1.11e-01         1.18e-01
         30      3.54e+00         4.14e-01
         60      3.08e+02         9.41e-01

Training 30-conv-layer nets on digits (SGD+momentum lr 0.05, 5 epochs):
  plain    training accuracy 0.575  training loss 1.121
  residual training accuracy 0.957  training loss 0.159
```

With BatchNorm, the plain net's gradients don't vanish here — they become erratic and **explode** as depth grows (3×10^2 at 60 layers), while the residual net's stay near 1. Trained with identical settings, the plain net reaches only 58% **training** accuracy (the degradation problem) while the residual net reaches 96%. Skip connections are now everywhere: DenseNet, U-Net, and every transformer block.

EfficientNet in one paragraph

If you have more compute, how should you grow a CNN — deeper, wider (more channels), or with higher-resolution inputs? EfficientNet found that scaling all three together with a fixed ratio (depth × α^φ, width × β^φ, resolution × γ^φ) gives far better accuracy per FLOP than scaling any one. The base network B0 was found by neural architecture search; B1–B7 are compound-scaled versions.

Vision Transformer (brief)

ViT cuts an image into fixed-size patches (e.g. 16×16 pixels → a 224×224 image becomes 14 × 14 = 196 patches), flattens and linearly projects each patch into a vector (a "token"), adds position embeddings and a learnable [CLS] token, and runs a standard transformer encoder (self-attention: Chapter 12). With no built-in locality or translation equivariance, ViTs need much more data (the original used 300M-image pre-training) but scale extremely well. Hybrid and hierarchical designs (Swin Transformer) and modern CNNs (ConvNeXt) now compete closely; for small datasets, a pretrained CNN or pretrained ViT fine-tuned is the practical choice.

Trade-offs

Need	Good choice
Strong, well-understood baseline	ResNet-50 (or ResNet-18 for speed)
Mobile / edge devices	MobileNetV3, EfficientNet-B0
Best accuracy with large pretraining	ViT / Swin / ConvNeXt, EfficientNetV2
Tiny dataset	Any pretrained backbone, fine-tuned

In the interview

30-second answer: "LeNet showed conv-pool stacks work; AlexNet scaled them with ReLU, dropout and GPUs and won ImageNet 2012; VGG showed stacks of 3×3 convs are enough; Inception used parallel branches and 1×1 bottlenecks; ResNet added identity skip connections so 100+ layer networks train — each block learns a residual and gradients flow through the identity path; EfficientNet scales depth, width and resolution together; ViT applies a transformer to image patches and wins with large-scale pretraining."

- *Q: Why do residual connections help?* A: They make the identity easy to represent (F = 0) and give gradients a direct path, fixing the degradation problem of deep plain nets.
- *Q: What is a 1×1 convolution for?* A: Mixing channels and changing channel count cheaply (bottlenecks).
- *Q: Why does ViT need more data than a CNN?* A: It lacks the CNN's built-in inductive biases (locality, weight sharing across positions), so it must learn them from data.

Self-check

1. Which architecture introduced skip connections? → ResNet (2015).
2. What three dimensions does EfficientNet scale? → Depth, width, input resolution.
3. How many patches does ViT make from a 224×224 image with 16×16 patches? → 196.
4. Name two ideas AlexNet popularised. → ReLU activations and dropout (also GPU training and heavy augmentation).

TL;DR

LeNet → AlexNet (ReLU, dropout, GPUs) → VGG (3×3 stacks) → Inception (1×1, multi-branch) → ResNet (skip connections: y = x + F(x)) → MobileNet/EfficientNet (efficiency, compound scaling) → ViT (patches + transformer).

11.5 Transfer learning and fine-tuning

What it is

Transfer learning reuses a model trained on a large **source** task (typically ImageNet classification) as the starting point for your **target** task. The early layers of any vision model learn general features — edges, textures, shapes — that are useful for almost every image problem, so you don't have to learn them again from a few hundred images.

Analogy: hiring an experienced photographer to sort your product photos. They already know how to see; you only teach them your categories.

Why it matters

Training a ResNet-50 from scratch on ImageNet takes days of GPU time and 1.28M labelled images. Fine-tuning a pretrained one on 2,000 of your own images takes minutes and usually gives far better accuracy than training from scratch on those 2,000. It's the default for almost every practical vision project.

How it works — the two strategies

1. **Feature extraction (frozen backbone).** Load pretrained weights, **freeze** the backbone (`requires_grad = False`), replace the final layer with a new head for your classes, and train only the head. Fast, needs little data, low risk of overfitting.
2. **Fine-tuning.** Start from the pretrained weights and also update some or all backbone layers with a **small learning rate** (often 10× smaller than for the head, "discriminative learning rates"), so useful features are adjusted rather than destroyed. A common recipe: train the new head first with the backbone frozen, then unfreeze the top blocks (or all) and continue with a low LR.

Your data	Similar to ImageNet	Very different (X-rays, satellite, spectrograms)
Small	Feature extraction	Feature extraction from earlier layers, or fine-tune gently; augment heavily
Large	Fine-tune all layers	Fine-tune all layers (pretraining still usually helps convergence)

The real code with torchvision (needs a download of weights — not run here):

```
import torch
from torch import nn
from torchvision import models

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)   # ImageNet-pretrained
for p in model.parameters():
    p.requires_grad = False                                         # 1) freeze backbone
model.fc = nn.Linear(model.fc.in_features, num_classes)             # new head (trainable)
opt = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# ... train a few epochs, then 2) unfreeze and fine-tune with a smaller lr:
for p in model.parameters():
    p.requires_grad = True
opt = torch.optim.AdamW([{"params": model.fc.parameters(), "lr": 1e-3},
                         {"params": [p for n, p in model.named_parameters() if not n.startswith("fc.")], "lr": 1e-4}])
# Inputs must be preprocessed like the pretraining data: resize/crop to 224 and normalise with
# ImageNet mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225) - the weights object's
# .transforms() gives the exact pipeline.
```

A runnable demonstration (no downloads)

`code/transfer_learning.py` mimics the setting on digits: pretrain a CNN on **clean** digits (half the dataset), then adapt it to the other half **"scanned by a different device"** (Gaussian noise added), using only k labelled images per class.

```
def make_model(strategy, pretrained):
    bb, head = backbone(), nn.Linear(128, 10)
    model = nn.Sequential(bb, head)
    if strategy == "frozen backbone":                   # pretrained backbone, NEW head
        bb.load_state_dict({k[2:]: v for k, v in pretrained.items() if k.startswith("0.")})
    elif strategy == "fine-tune all":                   # start from the whole source model
        model.load_state_dict(pretrained)
    if strategy == "frozen backbone":
        for p in bb.parameters():
            p.requires_grad = False                     # freeze: no gradients, no updates
        return model, head.parameters()
    if strategy == "fine-tune all":
        return model, [{"params": model.parameters(), "lr": 1e-4}]   # small lr: nudge, don't overwrite
    return model, model.parameters()
```

Output:

```
source model: train acc on clean digits 0.996, acc on noisy target digits with NO adaptation 0.839

labels/class     from scratch  frozen backbone    fine-tune all   (mean test acc over 3 seeds)
           2            0.530            0.650            0.840
           5            0.673            0.766            0.834
          10            0.738            0.779            0.847
          30            0.810            0.824            0.857

trainable params: from scratch / fine-tune = 6090 | frozen backbone = 1290
```

Reading it:

- With **2 labels per class**, from-scratch gets 53%; a frozen pretrained backbone with a new head gets 65%; gently fine-tuning the full pretrained model gets 84%.
- The advantage **shrinks as target data grows** (30 per class: 81% vs 86%) — transfer matters most when labels are scarce.
- **Always measure the pretrained model before adapting it**: here the source model already scores 83.9% on the target with no training. Fine-tuning with a *large* learning rate or a randomly re-initialised head can easily do *worse* than that baseline — which is why fine-

tuning uses small learning rates.

- Transfer is not magic: in a variant of this experiment (not included in the script) where the target classes were *different digits* (pretrain on 0–4, adapt to 5–9) on these tiny 8×8 images, pretraining did not help — the features weren't general enough. Real ImageNet backbones, trained on 1,000 diverse classes, transfer far more broadly.

When not to use it

- When your inputs are nothing like natural images *and* you have plenty of data (pretraining may still help convergence, but check).
- When the license of the pretrained weights forbids your use.

Common mistakes

- Forgetting to preprocess inputs exactly like the pretraining data (size, normalisation).
- Fine-tuning everything with a large learning rate on a small dataset — "catastrophic forgetting" of the useful features.
- Leaving BatchNorm layers updating their running statistics on tiny batches during fine-tuning (often better kept in eval mode / frozen).

Real-world examples

Most production vision models start from pretrained backbones: medical-imaging classifiers fine-tuned from ImageNet models, retail product recognition, defect detection on factory lines. Detection models (YOLO, Faster R-CNN) use pretrained classification backbones. The same pattern — pretrain big, fine-tune small — is how LLMs are adapted (AI Handbook, Chapter 8).

In the interview

> **30-second answer:** "Take a model pretrained on a large dataset like ImageNet, replace its head, and either freeze the backbone and train only the head (feature extraction — small data) or fine-tune some or all layers with a small learning rate (more data or a different domain). Preprocess inputs like the pretraining data. It beats training from scratch whenever labelled data is limited."

- *Q: Which layers transfer best?* A: Early layers (generic edges/textures); later layers are more task-specific, so they're the first to unfreeze.
- *Q: Why a smaller LR for the backbone?* A: To adjust, not overwrite, useful pretrained features.

Self-check

1. What does freezing do in PyTorch? → `requires_grad = False` — no gradients, so the optimizer doesn't change those weights.
2. Small dataset similar to ImageNet — which strategy? → Feature extraction (frozen backbone, new head), maybe light fine-tuning later.
3. Why did transfer help less with 30 labels per class than with 2? → With more target data, a model trained from scratch catches up; pretraining matters most when data is scarce.

TL;DR

Start from pretrained weights; new head; freeze then fine-tune with a small LR; preprocess like the pretraining data; biggest gains when labels are scarce.

11.6 Data augmentation for images

What it is

Generating new training images from existing ones by applying transformations that **don't change the label**: a flipped, slightly rotated or recoloured cat is still a cat. The network sees more variety, so it memorises less and becomes robust to the variations it will meet in the real world.

How it works — common transforms (torchvision.transforms.v2)

Transform	What it does	Watch out
`RandomResizedCrop(224)`	Random crop of random size/aspect, resized	The standard ImageNet training transform
`RandomHorizontalFlip()`	Mirror left-right	Not for text, digits, or where left/right matters
`RandomRotation(15)`	Small rotations	Large rotations turn 6 into 9
`ColorJitter(0.4, 0.4, 0.4)`	Brightness, contrast, saturation changes	Not if colour is the label (ripe vs unripe fruit)
`RandomErasing()` / Cutout	Blank a random rectangle	Forces use of several parts of the object
`GaussianBlur`, noise	Blur or noise	Mimic camera quality
Mixup / CutMix	Blend two images (or paste a patch) and blend their labels	Strong regularizers for large models
RandAugment / AutoAugment / TrivialAugment	Automatically chosen policies of transforms	Good defaults in modern training recipes

```python
from torchvision.transforms import v2 as T   # (torchvision not needed for the rest of this chapter)
train_tf = T.Compose([T.RandomResizedCrop(224), T.RandomHorizontalFlip(), T.ColorJitter(0.2, 0.2, 0.2),
                      T.ToImage(), T.ToDtype(torch.float32, scale=True),
                      T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
test_tf = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToImage(), T.ToDtype(torch.float32, scale=True),
                     T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

Augmentation happens **on the fly** in the training `Dataset`/`DataLoader`, differently every epoch — only for training data. Validation/test use a fixed deterministic resize/crop. **Test-time augmentation (TTA)** — averaging predictions over a few flipped/cropped versions — can add a little accuracy at inference cost.

In §11.3, shift augmentation raised accuracy on shifted digits from 86% to 98%.

For **detection and segmentation**, geometric transforms must be applied to the boxes/masks too (torchvision v2 transforms and the Albumentations library handle this).

Common mistakes

- Augmenting validation/test data randomly.
- Augmentations that change the label (flipping digits, rotating road signs that mean different things when mirrored).
- Forgetting that augmentation slows each epoch (CPU-bound) — use `num_workers` in `DataLoader`.

In the interview

30-second answer: "Augmentation applies random, label-preserving transforms — crops, flips, colour jitter, small rotations, Mixup/CutMix — to training images on the fly, so the model sees more variety and overfits less. Validation and test use a fixed resize and crop. For detection and segmentation the same geometric transform must be applied to boxes and masks."

Self-check

1. Why not horizontally flip digit images? → Mirrored digits (e.g. a mirrored 2 or 7) aren't valid digits — the label no longer holds.
2. What does Mixup do to the label? → Blends the two one-hot labels with the same ratio as the images.
3. Is test-time augmentation training or inference? → Inference: average predictions over a few transformed copies.

TL;DR

Random label-preserving transforms on training images only; flips, crops, colour jitter, rotations, Mixup/CutMix; the cheapest regularizer in vision.

11.7 Vision tasks: classification, detection, segmentation

What it is

Task	Question	Output	Typical models
Image classification	What is in this image?	One label (or a label per image)	ResNet, EfficientNet, ViT
Object detection	What objects are where?	A box (x, y, w, h) + class + confidence per object	YOLO, SSD, Faster R-CNN, DETR
Semantic segmentation	Which class is each pixel?	A class per pixel	U-Net, DeepLab, FCN
Instance segmentation	Which object does each pixel belong to?	A mask per object	Mask R-CNN, YOLO-seg, SAM (promptable)

Object detection

IoU (Intersection over Union) measures box overlap: area of intersection ÷ area of union. It's used to match predictions to ground truth (e.g. a detection counts as correct if IoU ≥ 0.5) and in NMS.

Worked example (`code/iou_nms.py`): ground truth (50, 50)–(150, 150) and prediction (70, 60)–(170, 160), boxes as (x1, y1)–(x2, y2). Each area = 100 × 100 = 10,000. Intersection = (150 − 70) × (150 − 60) = 80 × 90 = 7,200. Union = 10,000 + 10,000 − 7,200 = 12,800. **IoU = 7,200 / 12,800 = 0.5625.**

NMS (non-maximum suppression) removes duplicate detections of the same object:

1. Sort boxes by confidence.
2. Keep the highest-scoring box; delete every remaining box whose IoU with it exceeds a threshold (e.g. 0.5).
3. Repeat with the next remaining box.

```python
def my_nms(boxes, scores, thr):
    order = sorted(range(len(boxes)), key=lambda i: -scores[i])
    keep = []
    while order:
        best = order.pop(0)
        keep.append(best)
        order = [i for i in order if iou(boxes[best], boxes[i]) < thr]   # drop heavy overlaps
    return keep
```

Output:

```
IoU worked example: 0.562
pairwise IoU of the 5 boxes:
[[1.   0.89 0.65 0.   0.  ]
 [0.89 1.   0.69 0.   0.  ]
 [0.65 0.69 1.   0.   0.  ]
 [0.   0.   0.   1.   0.74]
 [0.   0.   0.   0.74 1.  ]]
NMS (IoU threshold 0.5) keeps boxes [0, 3] with scores [0.9, 0.8]
```

Five raw boxes around two objects → two final detections.

Two families of detectors:

- **Two-stage** (R-CNN → Fast R-CNN → **Faster R-CNN**, 2015): a Region Proposal Network suggests candidate boxes, then a second stage classifies and refines each. Accurate, slower.
- **One-stage** (**YOLO**, SSD, RetinaNet): predict all boxes in a single forward pass. Fast enough for real-time video.

The YOLO idea ("You Only Look Once", Redmon et al., 2016): divide the image into an S × S grid. Each grid cell predicts a few bounding boxes (x, y, w, h, and an objectness confidence) and class probabilities for an object whose **centre** falls in that cell — all as one tensor from one CNN pass. Then threshold on confidence and run NMS. The original ran at about 45 frames per second. Later versions (YOLOv3 onward, and today's Ultralytics YOLO releases) added multi-scale predictions, anchor boxes (and later anchor-free heads) and better backbones, and remain the go-to for real-time detection. **DETR** (2020) reframed detection with a transformer that predicts a set of boxes directly, removing NMS. For a full system built on a detector — blurring faces and plates in street imagery, where recall per person matters more than mAP — see case study 18.7.

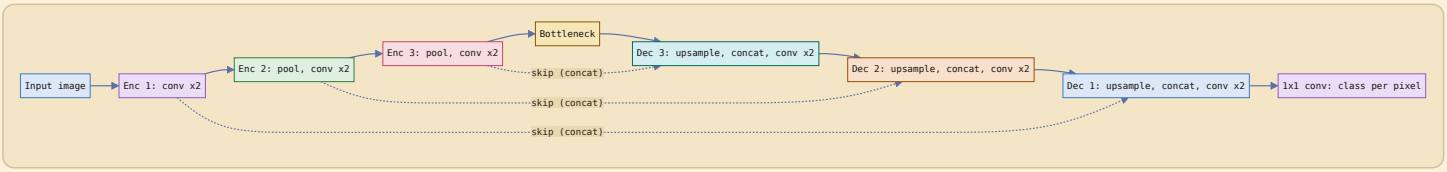
Metric: mAP (mean average precision) — area under the precision–recall curve per class, averaged over classes; PASCAL VOC uses IoU ≥ 0.5, COCO averages over IoU thresholds 0.5 to 0.95.

Segmentation — the U-Net idea

Segmentation needs a prediction for **every pixel**, so the network must go down to a compact representation (to understand *what*) and back up to full resolution (to say exactly *where*).

U-Net (Ronneberger et al., 2015, for biomedical images):

1. **Encoder (contracting path)**: conv blocks + downsampling — spatial size shrinks, channels grow (like a classifier backbone).
2. **Decoder (expanding path)**: upsampling (transposed convolutions) back to the input resolution.
3. **Skip connections**: at each level, the encoder's feature map is **concatenated** onto the decoder's — giving back the fine spatial detail lost during downsampling.
4. A final 1×1 conv outputs class scores per pixel; train with per-pixel cross-entropy (often plus Dice loss for imbalanced masks).



It's shaped like a "U", hence the name. U-Net-style networks also sit at the core of image-generation diffusion models (e.g. Stable Diffusion v1/v2 use a U-Net denoiser).

Real-world examples

- **Classification**: Google Photos search, product recognition in retail apps, content moderation.
- **Detection**: self-driving perception (cars, pedestrians, signs), warehouse robots, retail shelf auditing, face detection in phone cameras.
- **Segmentation**: medical imaging (tumour and organ outlines), background removal in video calls, satellite land-use maps, portrait mode.

In the interview

30-second answer: "Classification gives one label per image; detection gives boxes plus classes; segmentation gives a label per pixel. Detection uses IoU to match boxes and NMS to remove duplicates; two-stage detectors like Faster R-CNN propose regions then classify them, one-stage detectors like YOLO predict all boxes in one pass over a grid, which makes them real-time. Segmentation uses encoder-decoder networks like U-Net, whose skip connections restore spatial detail. Metrics: accuracy/top-k, mAP at IoU thresholds, and IoU/Dice for masks."

- *Q: Why is NMS needed?* A: Detectors output many overlapping boxes for the same object; NMS keeps the most confident and removes overlaps above an IoU threshold.
- *Q: Semantic vs instance segmentation?* A: Semantic labels pixels by class (all people are "person"); instance separates each object (person 1, person 2).

Self-check

1. IoU of two identical boxes? Of disjoint boxes? → 1 and 0.
2. In YOLO, which grid cell is responsible for an object? → The cell containing the object's centre.
3. What do U-Net's skip connections carry? → High-resolution encoder feature maps, concatenated onto the decoder to recover spatial detail.

TL;DR

Classification → label; detection → boxes (IoU, NMS, YOLO = one pass over a grid, Faster R-CNN = propose then classify); segmentation → per-pixel labels (U-Net = encoder-decoder + skip connections).

11.8 Putting it together — trade-offs, mistakes, interview

When not to use a CNN

- Tabular data (use gradient-boosted trees).
- Very small image datasets **without** a pretrained model — collect more data or use classical features.
- When a pretrained ViT/CLIP-style foundation model with zero-shot prompts already solves the problem.

Common mistakes (whole chapter)

- Training from scratch when a pretrained backbone is available.
- Wrong tensor layout or normalisation.
- Label-changing augmentations.
- Judging a detector by accuracy instead of mAP.

In the interview — big picture

> **30-second answer:** "CNNs slide shared small kernels over the image, giving far fewer parameters and translation equivariance. Stacked 3×3 convs with downsampling build a hierarchy of features and a growing receptive field; a head of global pooling plus a dense layer classifies. ResNet's skip connections enable very deep networks. In practice I'd start from a pretrained backbone, fine-tune with augmentation, and choose the head by task — classification, YOLO-style detection or U-Net-style segmentation."

Self-check

1. Two properties of convolution that make it suit images? → Weight sharing (few parameters) and local connectivity/translation equivariance.
2. What does global average pooling replace? → The large flattened dense layers at the end of older CNNs.
3. What's the first thing to try for a new 5,000-image classification problem? → Fine-tune a pretrained backbone (e.g. ResNet-50 or EfficientNet) with augmentation.

Chapter TL;DR

- **Convolution**: shared k×k kernels over all input channels; out = ⌊(n + 2p − k)/s⌋ + 1; params = k·k·C_in·C_out + C_out. Stride downsamples; padding preserves size.
- **Pooling**: 2×2 max-pool halves H and W, no parameters; global average pooling before the classifier.
- **Architecture**: conv-BN-ReLU blocks, shrinking space and growing channels, then GAP + dense. Stacked 3×3 convs grow the receptive field cheaply.
- **History**: LeNet → AlexNet (ReLU, dropout, GPU) → VGG → Inception → **ResNet (skip connections)** → MobileNet/EfficientNet → ViT.
- **Our CNN**: 99.1% on 8×8 digits with fewer parameters than an MLP; far more robust to shifts, especially with augmentation.
- **Transfer learning**: pretrained backbone + new head; freeze, then fine-tune with a small LR; biggest gains with few labels.
- **Augmentation**: label-preserving random transforms, training only.
- **Detection**: IoU, NMS, YOLO (one pass, grid), Faster R-CNN (two-stage), mAP. **Segmentation**: U-Net (encoder-decoder + skip connections).

Next: Chapter 12 moves from grids of pixels to sequences — RNNs, LSTMs and attention.

Chapter 12 — Sequences: RNNs, LSTMs and Attention

Text, speech, stock prices, sensor readings, click streams, DNA — much of the world's data comes as **sequences**, where order carries meaning. "Dog bites man" and "man bites dog" contain the same words. The networks of Chapters 10 and 11 take a fixed-size input all at once; sequence models read data **step by step** (RNNs) or **all positions against each other** (attention), so they can handle variable lengths and dependencies across time.

This chapter follows the historical path, because each step fixes the previous one's weakness: RNNs → vanishing gradients → LSTM/GRU gates → seq2seq encoder-decoders → the bottleneck problem → attention → self-attention and the transformer. Along the way we cover how text becomes numbers (tokenization and word embeddings) and a short introduction to time-series forecasting.

What you'll learn

1. What makes sequence data special, and the input/output patterns (one-to-many, many-to-one, many-to-many).
2. Recurrent neural networks: the recurrence, unrolling, backpropagation through time (BPTT).
3. Why gradients vanish or explode through time.
4. LSTM and GRU: gate intuition and equations.
5. Sequence-to-sequence (encoder-decoder) models, teacher forcing, greedy and beam decoding.
6. The attention mechanism — worked with small numbers.
7. Self-attention and the transformer at a high level. The step-by-step treatment is in the AI Handbook, Chapter 3 — Transformers and Embeddings.
8. Text preprocessing and tokenization (word, character, subword/BPE) and word embeddings (word2vec intuition).
9. Time-series forecasting: lags, baselines, ARIMA vs ML.
10. Runnable PyTorch: an LSTM on two toy sequence tasks, a seq2seq model with and without attention, a tiny word2vec.

All code is in `code/`, runs on CPU in seconds, and every output shown is real.

12.1 Sequence data

What it is

A **sequence** is an ordered list $x_1, x_2, \ldots, x_T$ where position matters and T can vary. Each element (a **time step** or **token**) may be a number (temperature), a vector (a video frame's features), or a symbol (a word).

Data	Element	Typical task
Text	word / subword token	classification, translation, generation
Speech	audio frame (e.g. 10–25 ms)	speech recognition
Time series	measurement per time unit	forecasting, anomaly detection
User events	click, purchase	next-item recommendation, churn
Biology	nucleotide / amino acid	structure and function prediction

Why ordinary networks struggle

- **Variable length**: an MLP needs a fixed input size.
- **Order and context**: the meaning of "bank" depends on nearby words; a bag of features loses order.
- **Parameter sharing across time**: a pattern ("not good") should be recognised wherever it appears, just as a CNN kernel is shared across image positions.

Input/output patterns

Pattern	Example
One-to-many	Image captioning (image in, word sequence out)
Many-to-one	Sentiment classification, "is this transaction sequence fraud?", next-value forecast
Many-to-many, aligned	Part-of-speech tagging, per-frame video labels
Many-to-many, not aligned (seq2seq)	Machine translation, summarisation, speech-to-text

TL;DR

Sequences are ordered and variable-length; models must share parameters across positions and carry context along.

12.2 Recurrent neural networks (RNNs)

What it is

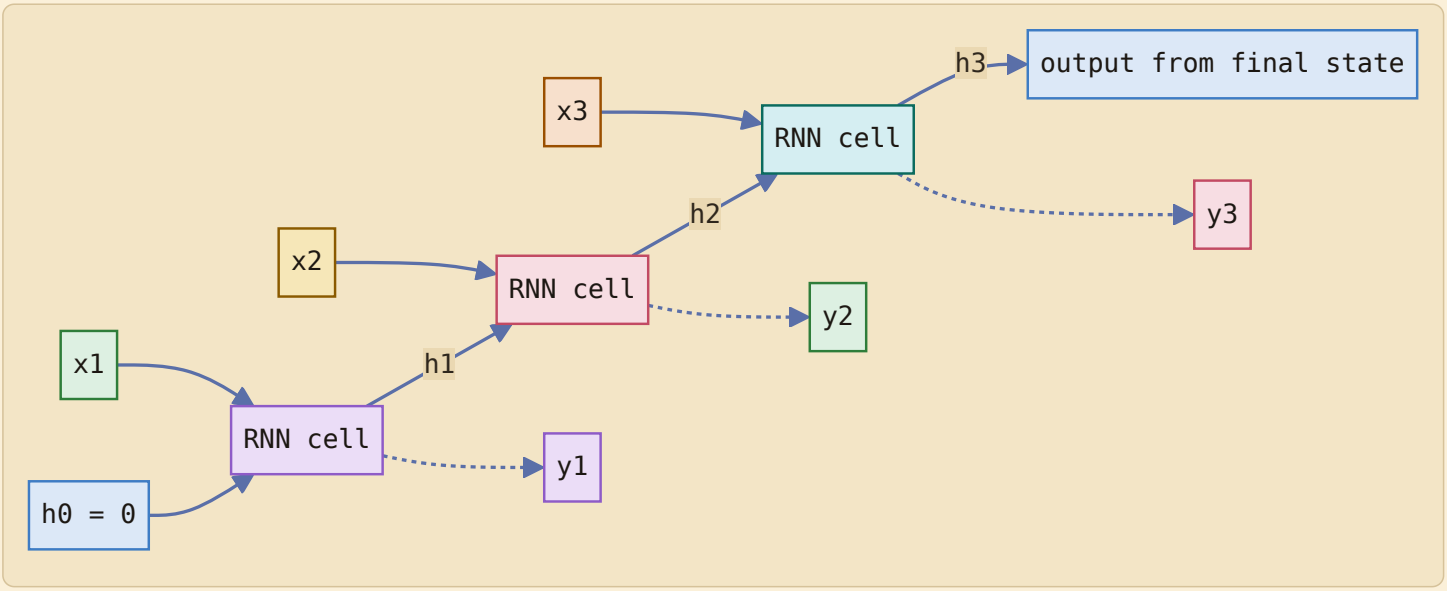
A **recurrent neural network** reads a sequence one step at a time and keeps a **hidden state** h_t — a vector summarising everything read so far. At each step it combines the new input with the previous hidden state using the **same weights**:

h_t = tanh(W_xh x_t + W_hh h_{t−1} + b_h) **ŷ_t = W_hy h_t + b_y** (an output, if the task needs one at this step)

Analogy: reading a novel and keeping a running mental summary. Each new sentence updates the summary; you never re-read the whole book — you carry the summary forward.

How it works — unrolling

"Unrolling" draws the loop as a chain with one copy of the cell per time step; all copies share W_xh, W_hh and b_h (Elman, 1990, popularised this "simple RNN").



Worked numbers from `code/rnn_by_hand.py` — 1 input, 2 hidden units, W_xh = [0.5, −0.3]ᵀ, W_hh = [[0.4, 0.1], [−0.2, 0.3]], b = 0, inputs 1.0, 0.5, −1.0:

t	x_t	W_xh x_t + W_hh h_{t−1}	h_t = tanh(…)
1	1.0	(0.5, −0.3) + 0	(0.4621, −0.2913)
2	0.5	(0.25, −0.15) + (0.4·0.4621 + 0.1·(−0.2913), −0.2·0.4621 + 0.3·(−0.2913)) = (0.4057, −0.3298)	(0.3848, −0.3184)
3	−1.0	(−0.3779, 0.1275)	(−0.3609, 0.1268)

```
torch.nn.RNN final hidden: [-0.3609  0.1268]
```

PyTorch's `nn.RNN` with the same weights gives the same final state.

Parameters don't grow with sequence length: here 2 + 4 + 2 = 8 numbers process sequences of any length.

Backpropagation through time (BPTT)

To train, unroll the network over the sequence, compute the loss (summed over steps that have targets), and run ordinary backprop through the unrolled graph. Because the weights are shared, each weight's gradient is the **sum** of its gradients from every time step: $\partial L/\partial W_{hh} = \Sigma_t$ ($\partial L/\partial h_t$ · local derivative at step t), where $\partial L/\partial h_t$ itself collects contributions from all later steps. This is **backpropagation through time**. For long sequences, **truncated BPTT** backpropagates only k steps (e.g. 100) at a time while still carrying the hidden state forward — cheaper, but it can't learn dependencies longer than k.

Variants

- **Stacked (deep) RNNs**: the hidden-state sequence of one RNN layer is the input sequence of the next (`num_layers=2`).
- **Bidirectional RNNs**: one RNN reads left-to-right, another right-to-left, and their states are concatenated — each position sees both past and future context. Good for tagging and classification; impossible for real-time generation (the future isn't known yet).

In PyTorch: `nn.RNN(input_size, hidden_size, num_layers=1, batch_first=True, bidirectional=False)`; input shape (batch, time, features); it returns `output` (every step's top-layer h) and `h_n` (final state per layer).

Self-check

1. What does the hidden state h_t hold? → A fixed-size summary of everything read up to step t.
2. How many parameters does an RNN with input size 1 and hidden size 2 (one bias) need for a 1,000-step sequence? → Still 8 — weights are shared across steps.
3. Why can't a bidirectional RNN do live generation? → Its backward half needs future inputs that don't exist yet.

TL;DR

RNN = the same cell applied at every step, carrying a hidden state; unroll it and backprop through time; shared weights mean fixed parameter count for any length.

12.3 Vanishing and exploding gradients through time

The problem

The gradient of a late loss with respect to an early hidden state is a **product of T Jacobians**, one per step: $\partial h_T/\partial h_0 = \prod_t \text{diag}(\tanh'(\cdot)) \cdot W_{hh}$. Like depth in Chapter 10, but now "depth" is sequence length — often hundreds of steps. If the recurrent weights' largest singular value (roughly: how much W_hh stretches vectors) combined with tanh' ≤ 1 is below 1, gradients shrink **exponentially** with distance; above 1, they can grow exponentially. Bengio, Simard and Frasconi analysed this in 1994; in practice a simple RNN struggles to learn dependencies more than a few tens of steps apart.

`code/rnn_by_hand.py` measures $\|\partial h_T/\partial h_0\|$ in a 32-unit tanh RNN for different recurrent-weight scales:

```
scale 0.5: |d h_T / d h_0| at T=1: 2.64e+00  T=10: 6.51e-03  T=60: 3.54e-17
scale 1.0: |d h_T / d h_0| at T=1: 5.13e+00  T=10: 5.22e+00  T=60: 2.58e+01
scale 1.5: |d h_T / d h_0| at T=1: 9.44e+00  T=10: 2.15e+01  T=60: 1.90e+02
scale 3.0: |d h_T / d h_0| at T=1: 1.60e+01  T=10: 3.90e+02  T=60: 1.29e+07
```

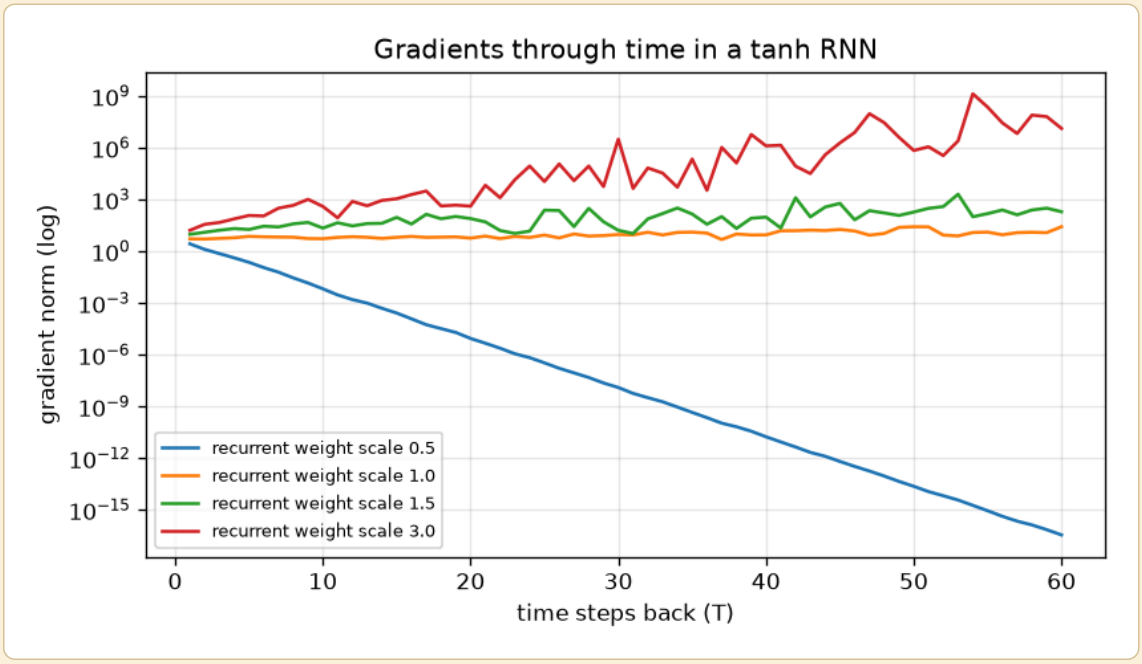


Figure — With small recurrent weights the gradient from 60 steps back is ~10^{-17} (vanished); with large weights it grows to 10^{7} (exploded). Only a narrow band in between is usable, and it's hard to stay there during training.

Fixes

Problem	Fix
Exploding	**Gradient clipping**: rescale the gradient if its norm exceeds a threshold — `torch.nn.utils.clip_grad_norm_(params, 1.0)`. Standard for all recurrent models (and transformers).
Vanishing	**Gated cells (LSTM, GRU)** with an additive memory path; careful init (e.g. orthogonal W_hh); shorter paths via **attention** or skip connections.

In the interview

30-second answer: "BPTT multiplies one Jacobian per time step, so gradients from distant steps shrink or grow exponentially with the distance — vanishing or exploding. Exploding is fixed by gradient clipping. Vanishing is fixed architecturally: LSTM/GRU gates create an additive path for information and gradients, and attention gives every position a direct connection to every other."

Self-check

1. Why is a long sequence like a deep network? → Backprop multiplies one factor per time step, just as it does per layer.
2. What does gradient clipping do? → Scales the whole gradient down when its norm exceeds a threshold, preventing huge updates.
3. Does clipping fix vanishing gradients? → No — it only caps large gradients.

TL;DR

Gradients through time are products of per-step factors → vanish or explode. Clip for exploding; use gates or attention for vanishing.

12.4 LSTM and GRU

What it is

Long Short-Term Memory (Hochreiter & Schmidhuber, 1997; the forget gate was added by Gers et al., 2000) adds a separate **cell state** c_t — a memory "conveyor belt" — and three **gates** that decide what to erase, what to write, and what to reveal. A **gate** is a vector of numbers between 0 and 1 (from a sigmoid) that multiplies another vector element-wise: 0 = block, 1 = let through.

Analogy: a notebook you carry through a lecture. At each moment you decide what to **cross out** (forget gate), what **new notes** to add (input gate), and which parts of your notes to **say aloud** right now (output gate).

How it works — LSTM equations

Let [h_{t−1}, x_t] be the previous hidden state and current input stacked into one vector; σ = sigmoid; ⊙ = element-wise product.

Step	Equation	Meaning
Forget gate	f_t = σ(W_f [h_{t−1}, x_t] + b_f)	Which memory entries to keep (1) or erase (0)
Input gate	i_t = σ(W_i [h_{t−1}, x_t] + b_i)	Which entries to update
Candidate	*č_t = tanh(W_c [h_{t−1}, x_t] + b_c)*	Proposed new content, in (−1, 1)
Cell update	**c_t = f_t ⊙ c_{t−1} + i_t ⊙ č_t**	Keep part of the old memory, add part of the new
Output gate	o_t = σ(W_o [h_{t−1}, x_t] + b_o)	Which parts of memory to expose
Hidden state	h_t = o_t ⊙ tanh(c_t)	What the rest of the network sees

Why it fixes vanishing gradients: the cell update is **additive**. Along the cell path, ∂c_t/∂c_{t−1} = f_t (element-wise) — no repeated squashing by tanh' or multiplication by W_hh. When the forget gate is near 1, information and gradients flow across many steps almost unchanged (the "constant error carousel"). The network *learns* when to remember and when to forget.

Tiny numeric example for one memory unit: previous c = 2.0; the gates output f = 0.9, i = 0.2, and the candidate is č = 0.5. Then c_new = 0.9·2.0 + 0.2·0.5 = 1.9 — mostly kept, slightly updated. With o = 0.7, h = 0.7·tanh(1.9) = 0.7·0.956 = 0.669.

An LSTM has **4 weight blocks** (f, i, č, o), so about 4× the parameters of a simple RNN of the same width.

GRU

The **Gated Recurrent Unit** (Cho et al., 2014) merges cell and hidden state and uses two gates:

Step	Equation	Meaning
Update gate	z_t = σ(W_z [h_{t−1}, x_t] + b_z)	How much to replace the old state
Reset gate	r_t = σ(W_r [h_{t−1}, x_t] + b_r)	How much of the old state to use when proposing new content
Candidate	*h̃_t = tanh(W_h [r_t ⊙ h_{t−1}, x_t] + b_h)*	Proposed new state
New state	h_t = (1 − z_t) ⊙ h_{t−1} + z_t ⊙ h̃_t	Interpolate old and new

(Cho et al. and PyTorch write the last line with z and 1 − z swapped — the same model with the gate's meaning flipped. PyTorch also applies r after multiplying h_{t−1} by its weight matrix, a minor variant.) A GRU has 3 weight blocks — fewer parameters than an LSTM, often similar accuracy.

Code — an LSTM on two toy tasks

`code/lstm_toy.py`.

A) Forecasting a noisy signal. The LSTM reads the last 30 values of a sum of two sine waves plus noise and predicts the next value (many-to-one regression). Training uses the first 900 windows; testing uses the **later** part of the series (never shuffle time across the train/test boundary).

```
class Forecaster(nn.Module):
    def __init__(self, hidden=32):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=hidden, batch_first=True)  # (batch, time, feat)
        self.head = nn.Linear(hidden, 1)

    def forward(self, x):                       # x: (batch, 30, 1)
        out, (h_n, c_n) = self.lstm(x)          # out: every step's h; h_n/c_n: final hidden & cell state
        return self.head(out[:, -1])            # predict from the last hidden state
```

B) Long memory. Read T random symbols (1–8) and output the **first** one. The answer must survive T steps of distracting input, which isolates the vanishing-gradient problem. Each model embeds symbols (`nn.Embedding`), runs a 32-unit RNN / LSTM / GRU, and classifies from the last hidden state; gradients are clipped at norm 1. The LSTM's forget-gate bias is initialised to 1 ("remember by default") — a widely used trick (Jozefowicz et al., 2015) that noticeably helps LSTMs start learning long dependencies.

```
if cell == "LSTM":                                    # forget-gate bias = 1: "remember by default"
    with torch.no_grad():                             # PyTorch gate order: input, forget, cell, output
        self.rnn.bias_ih_l0[hidden:2 * hidden].fill_(1.0)
        self.rnn.bias_hh_l0[hidden:2 * hidden].fill_(0.0)
```

Output:

```
A) LSTM test MSE 0.0190  vs naive last-value MSE 0.0442  (noise variance = 0.0100)
B) recall the first of T symbols (test accuracy, chance 0.125)
    T     RNN    LSTM     GRU
    5   1.000   1.000   1.000
   15   0.505   1.000   1.000
recurrent-layer parameters (hidden 32): {'RNN': 1600, 'LSTM': 6400, 'GRU': 4800}
saved img/lstm_sine_forecast.png
```

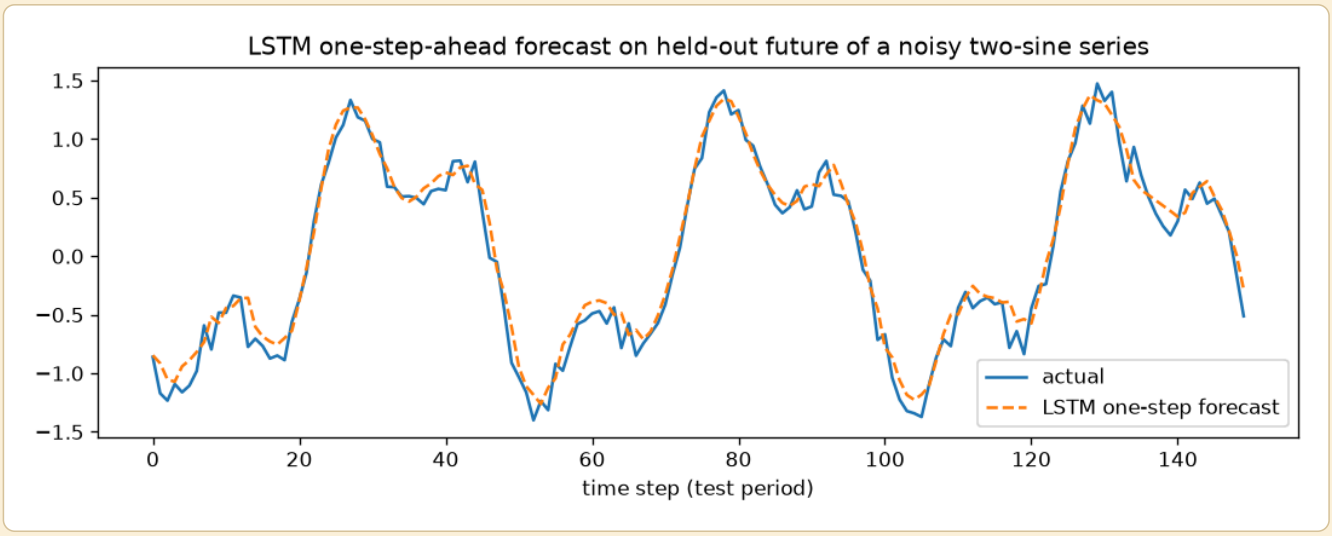


Figure — One-step-ahead LSTM forecasts on the held-out future. The error (MSE 0.019) is less than half the naive "next = last" baseline and approaches the noise floor (0.010), which no model can beat.

Reading part B: with 5 steps all three cells solve the task. With 15 steps the plain RNN stalls near 50% while LSTM and GRU reach 100%. (With much longer gaps, even gated cells need more training steps — and this is exactly where attention, which connects positions directly, shines.) Parameter counts confirm the 1 : 4 : 3 ratio of RNN : LSTM : GRU weight blocks (16·32 + 32·32 + 2·32 = 1,600 for the RNN).

Trade-offs

	Simple RNN	LSTM	GRU	Transformer (12.7)
Long-range memory	Poor (a few tens of steps at best)	Good	Good	Excellent (direct connections)
Parameters (same width)	1×	4×	3×	Different design
Training parallelism across time	None (sequential)	None	None	Full
Inference memory for long sequences	Constant	Constant	Constant	Grows with length (KV cache)
Typical use today	Teaching	Small/streaming models, time series, on-device	Same as LSTM	NLP, vision, audio, most large models

When not to use it

- Large-scale NLP: transformers win on quality and trainability (they parallelise over time on GPUs).
- Tabular time series with good lag features: gradient-boosted trees are often better and simpler (12.10).

Common mistakes

- Forgetting `batch_first=True` and feeding (batch, time, features) to a layer expecting (time, batch, features).
- Using `out[:, -1]` on **padded** batches — the last position may be padding. Either pack the batch (`nn.utils.rnn.pack_padded_sequence`) or pick out the real final step of every sequence using its length.
- Not clipping gradients.
- Shuffling time-series data across the train/test boundary (future leaks into training).

Real-world examples

LSTMs powered Google's voice-search speech recognition (2015), Google Neural Machine Translation (2016) and many handwriting-recognition systems before transformers took over. They remain common in on-device and streaming settings and in industrial time-series forecasting

and anomaly detection.

In the interview

30-second answer: "An LSTM adds a cell state updated additively — c_t = f ⊙ c_{t−1} + i ⊙ c̃ — controlled by sigmoid gates: forget, input and output. Because the cell path isn't repeatedly squashed or multiplied by the recurrent matrix, gradients survive many steps, fixing the vanishing-gradient problem of simple RNNs. A GRU does the same with two gates (update and reset) and no separate cell, so it's lighter. Both still process tokens sequentially, which is why transformers replaced them for large-scale NLP."

- *Q: What does the forget gate do, and why initialise its bias to 1?* A: It scales the old cell state; bias 1 makes f ≈ 0.73 at the start, so memory persists by default and gradients flow early in training.
- *Q: LSTM vs GRU?* A: GRU is simpler (3 vs 4 blocks), trains faster; accuracy is usually similar — try both.
- *Q: Why can't you parallelise an RNN over time?* A: h_t depends on h_{t−1}, so steps must run in order.

Self-check

1. Which LSTM equation makes the gradient path additive? → c_t = f_t ⊙ c_{t−1} + i_t ⊙ c̃_t.
2. Range of gate values and why? → (0, 1), from a sigmoid — they act as soft switches.
3. Parameters of an LSTM layer vs a simple RNN of the same size? → About 4×.
4. In the toy task, why did the RNN fail at T = 15? → The gradient from the answer back to the first symbol vanished across 15 steps.

TL;DR

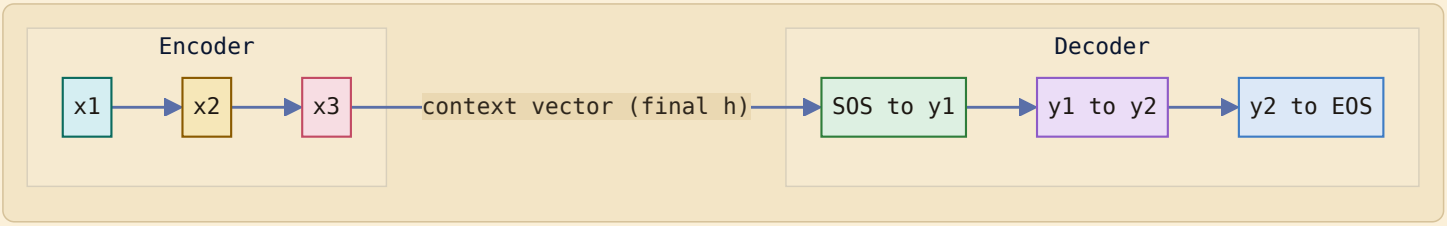
LSTM = cell state + forget/input/output gates, additive memory → long-range dependencies. GRU = update/reset gates, lighter. Clip gradients; use forget-bias 1.

12.5 Sequence-to-sequence (encoder-decoder) models

What it is

Seq2seq (Sutskever, Vinyals & Le; Cho et al.; both 2014) maps an input sequence to an output sequence of a *different* length — translation, summarisation, speech-to-text:

1. **Encoder**: an RNN reads the whole input and ends with a hidden state — a fixed-size "summary" (context vector).
2. **Decoder**: another RNN, initialised with that summary, generates the output one token at a time, each step taking the previously generated token as input, until it emits an end-of-sequence token.



How it works

- **Training with teacher forcing**: the decoder's input at step t is the **true** previous token (the target sequence shifted right, starting with a start-of-sequence token SOS), not its own prediction. This makes training fast and stable. The loss is cross-entropy at every output step.
- **Inference (decoding)**: the true tokens aren't available, so the decoder feeds back its own predictions.
  - **Greedy decoding**: pick the most likely token each step.
  - **Beam search**: keep the k most probable partial sequences (the "beam"), extend each, keep the best k again. Finds higher-probability outputs than greedy; standard in translation.
  - **Sampling** (temperature, top-k, top-p): for creative generation (AI Handbook, Chapter 4).
- **Exposure bias**: the model is trained on perfect prefixes but tested on its own, possibly wrong, prefixes; errors can compound. Scheduled sampling is one mitigation.

The bottleneck problem

The whole input must be squeezed into one fixed-size vector. For long sentences, early information gets lost — translation quality of early seq2seq models dropped sharply with sentence length. Sutskever et al. found that simply **reversing the source sentence** helped (it shortens the distance between the first source and first target words) — a hint that shorter paths matter. The real fix is attention.

In the interview

30-second answer: "An encoder RNN reads the input into a context vector; a decoder RNN generates the output token by token from it. Training uses teacher forcing — feeding the true previous token — and inference uses greedy or beam search on the model's own outputs. The weakness is the fixed-size bottleneck, which attention removes."

- *Q: Beam search with k = 1?* A: It is greedy decoding.

- *Q: What is exposure bias?* A: The train/test mismatch from teacher forcing: at inference the decoder conditions on its own, possibly wrong, previous tokens.

Self-check

1. Decoder input at step t during training? → The true token t − 1 (teacher forcing), starting from SOS.
2. What limits plain seq2seq on long inputs? → The single fixed-size context vector.

TL;DR

Seq2seq = encoder → context vector → decoder; teacher forcing to train, greedy/beam search to decode; the bottleneck motivates attention.

12.6 The attention mechanism

What it is

Attention (Bahdanau, Cho & Bengio, 2014) lets the decoder, at every output step, **look back at all encoder states** and take a weighted average of them, with weights computed from how relevant each input position is right now. Instead of one summary vector, the decoder gets a fresh, targeted **context vector** at each step.

Analogy: translating a sentence, you don't memorise the whole source and then write; your eyes jump back to the relevant source words for each word you write.

How it works — queries, keys and values

Modern notation (from the transformer paper) describes attention as a soft dictionary lookup:

- **Query (q)**: what I'm looking for (the decoder's current state).
- **Keys (k)**: a label for each item that can be found (one per encoder position).
- **Values (v)**: the content returned (often the same encoder states).

Steps:

1. **Score** each key against the query. Dot-product (Luong, 2015): score = q·k. Additive (Bahdanau): score = $v^T \tanh(W_1 q + W_2 k)$. Transformers use **scaled dot-product**: score = q·k / √d_k.
2. **Softmax** the scores into weights that are positive and sum to 1.
3. **Output** = Σ $\text{weight}_i \cdot v_i$ — a weighted average of the values.

In matrix form for many queries at once: **$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k})\, V$**.

Worked example with small numbers

Three tokens ("the", "cat", "sat"), d_k = 2. (In a real model Q, K and V are the token vectors multiplied by learned matrices W_Q, W_K, W_V; here we pick them directly.)

Q = K = [[1, 0], [0, 1], [1, 1]], V = [[1, 2], [3, 4], [5, 6]].

Query 1 ("the", q = [1, 0]):

1. Dot products with the keys: [1,0]·[1,0] = 1, [1,0]·[0,1] = 0, [1,0]·[1,1] = 1.
2. Scale by 1/√2: (0.707, 0, 0.707).
3. Softmax: e^0.707 = 2.028, e^0 = 1, e^0.707 = 2.028; sum = 5.056 → weights (0.401, 0.198, 0.401).
4. Output = 0.401·[1, 2] + 0.198·[3, 4] + 0.401·[5, 6] = [3.0, 4.0].

All three queries (`code/attention_by_hand.py`):

```
scores = Q K^T / sqrt(2):
 [[0.707 0.    0.707]
 [0.    0.707 0.707]
 [0.707 0.707 1.414]]
attention weights (softmax per row):
 [[0.401 0.198 0.401]
 [0.198 0.401 0.401]
 [0.248 0.248 0.503]]
output = weights @ V:
 [[3.    4.   ]
 [3.407 4.407]
 [3.51  4.51 ]]
matches torch.nn.functional.scaled_dot_product_attention: True
```

```python
def attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)                 # similarity of every query with every key
    if mask is not None:
        scores = np.where(mask, scores, -np.inf)    # hide disallowed positions
    weights = softmax(scores)                       # each row sums to 1
    return weights @ V, scores, weights             # weighted average of values
```

Why divide by √d_k? For random vectors with unit-variance entries, the dot product's standard deviation grows like √d_k. Large scores push softmax into a nearly one-hot, saturated regime with tiny gradients. Scaling keeps scores around unit variance:

```
d_k=  4: std of q.k =   1.99   std after / sqrt(d_k) = 1.00
d_k= 64: std of q.k =   8.05   std after / sqrt(d_k) = 1.01
d_k=512: std of q.k =  22.44   std after / sqrt(d_k) = 0.99
```

Attention fixes seq2seq — runnable demo

`code/seq2seq_attention.py` trains two GRU encoder-decoders (1,500 steps each, teacher forcing) to **reverse a 12-digit sequence**. One decoder sees only the encoder's final state; the other also attends over all encoder states with scaled dot-product attention (query = decoder state, keys = values = encoder states).

```
def forward(self, src, tgt_in):
    enc_out, h = self.encoder(self.src_emb(src))               # enc_out: (B, L, H) - one vector per input token
    dec_out, _ = self.decoder(self.tgt_emb(tgt_in), h)         # decoder starts from the encoder's summary h
    if not self.use_attention:
        return self.out(dec_out), None
    scores = dec_out @ enc_out.transpose(1, 2) / H ** 0.5      # (B, L_tgt, L_src): query=decoder, keys=encoder
    weights = scores.softmax(-1)
    context = weights @ enc_out                                # weighted sum of encoder states
    return self.out(torch.cat([dec_out, context], -1)), weights
```

Output:

```
no attention  : token accuracy 0.669, whole-sequence accuracy 0.156
   source [7, 7, 5, 1, 0, 0, 0, 3, 6, 9, 0, 0]
   output [0, 0, 9, 6, 0, 3, 0, 3, 1, 5, 0, 7]
with attention: token accuracy 0.991, whole-sequence accuracy 0.980
   source [7, 7, 5, 1, 0, 0, 0, 3, 6, 9, 0, 0]
   output [0, 0, 9, 6, 3, 0, 0, 0, 1, 5, 7, 7]
   where each output step looks (argmax of attention over source positions): [11, 11, 10, 9, 8, 7, 6, 6, 4, 3, 2, 1]
```

Without attention, the model gets the first few outputs (recent inputs) right and then garbles the rest — the bottleneck. With attention, 98% of sequences are perfectly reversed, and the attention pattern is interpretable: the peak moves from the last source position (11) down towards the first — roughly a reversed diagonal — so the model **learned to read the input backwards**. The peak is not always exactly on the digit being copied: where neighbouring source digits are identical (the 0s at positions 10–11 and 4–6) either one will do, and for the last few steps it sits one position to the right (it looks at 4, 3, 2, 1 while copying positions 3, 2, 1, 0). That still works because the encoder is a left-to-right GRU: its state at position j + 1 has just read x_j and still carries it.

In the interview

30-second answer: "Attention scores a query against every key, softmaxes the scores into weights, and returns the weighted average of the values: softmax($QK^T/\sqrt{d_k}$)V. In seq2seq the decoder state is the query and the encoder states are keys and values, so each output step gets its own context vector. Dividing by √d_k keeps the scores at unit variance so the softmax doesn't saturate."

- *Q: Additive vs dot-product attention?* A: Additive (Bahdanau) scores with a small MLP; dot-product (Luong, transformers) is a matrix multiply — faster and, with scaling, as good.

Self-check

1. What are the three steps of attention? → Score (q·k/√d), softmax to weights, weighted sum of values.
2. What problem did attention solve in seq2seq? → The fixed-size bottleneck: the decoder can look at every encoder state directly.
3. Why scale by √d_k? → To keep the variance of scores ≈ 1 so softmax doesn't saturate.

TL;DR

Attention = softmax($QK^T/\sqrt{d_k}$)·V — a learned, differentiable, weighted lookup. In seq2seq it removes the bottleneck and gives interpretable alignments.

12.7 Self-attention and the transformer (high level)

What it is

In **self-attention**, queries, keys and values all come from the **same** sequence: every token looks at every other token of its own sentence to build a context-aware representation. ("The animal didn't cross the street because **it** was too tired" — self-attention lets "it" draw information from "animal".) The **transformer** (Vaswani et al., 2017, "Attention Is All You Need") drops recurrence entirely and builds a network from self-attention plus small feed-forward layers.

Why it matters

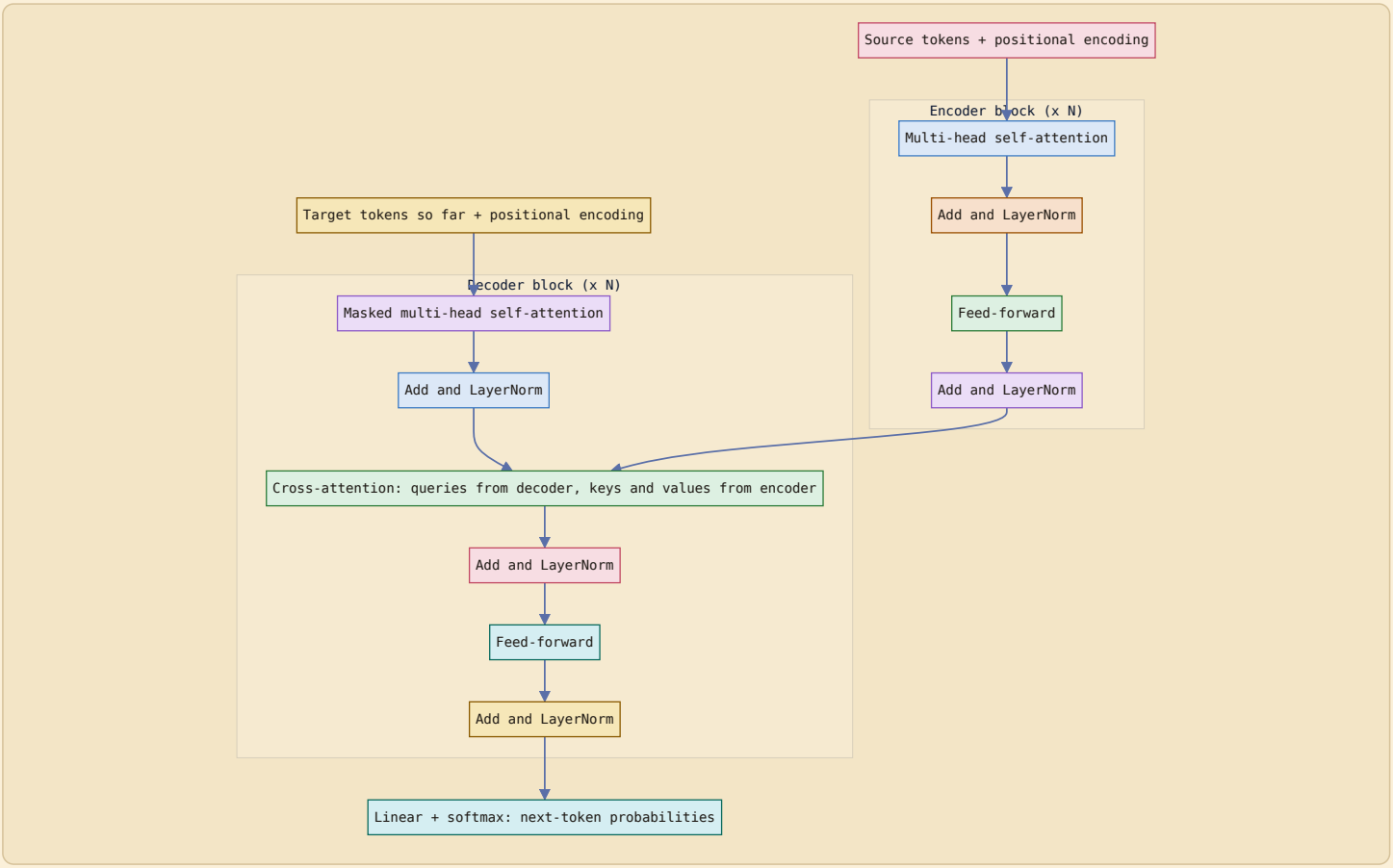
- **Parallel**: all positions are computed at once — no step-by-step dependency — so it trains efficiently on GPUs over huge datasets.
- **Short paths**: any two positions are one attention step apart, so long-range dependencies are easy to learn (no vanishing through time).

- **Scales**: it is the architecture behind BERT, GPT, T5, Claude, Gemini, LLaMA, Whisper (speech) and ViT (vision, Chapter 11).

How it works — the building blocks

1. **Token embeddings**: each token id becomes a vector of size d_model (512 in the original paper).
2. **Positional encoding**: self-attention alone ignores order (shuffle the tokens and the outputs are just shuffled), so position information is added to each embedding — originally fixed sine/cosine waves of different frequencies, PE(pos, 2i) = sin(pos / 10000^(2i/d)) and PE(pos, 2i+1) = cos(pos / 10000^(2i/d)); later models learn positions or use rotary embeddings (RoPE).
3. **Multi-head attention**: h attention "heads" run in parallel, each with its own W_Q, W_K, W_V projecting to d_model/h dimensions; their outputs are concatenated and projected back.
4. **Feed-forward network**: a 2-layer MLP applied to each position independently (typically 4× wider inside).
5. **Residual connections + layer normalization** around each sub-layer (the ResNet idea from Chapter 11).
6. Stack N blocks (original: 6 encoder + 6 decoder, 8 heads).

Each of these pieces is worked through with numbers and code in the AI Handbook, Chapter 3 (§3.3 positions, §3.5 multi-head, §3.6 the block, §3.7 causal masking, §3.8 the three families); here we only need the big picture.



- **Encoder**: bidirectional self-attention — each token sees the whole input.
- **Decoder**: **masked** (causal) self-attention — token i may only attend to tokens ≤ i, so it can't peek at the future it must predict — plus **cross-attention** to the encoder's outputs (exactly the seq2seq attention of 12.6).
- Three families: **encoder-only** (BERT — understanding/classification), **decoder-only** (GPT, Claude, LLaMA — generation), **encoder-decoder** (original transformer, T5, Whisper — translation, summarisation, speech-to-text).

Causal masking in the worked example (the upper triangle is set to −∞ before softmax):

```
causal (decoder) weights:
 [[1.    0.    0.   ]
 [0.33  0.67  0.   ]
 [0.248 0.248 0.503]]
causal output matches torch (is_causal=True): True
MultiheadAttention: input (2, 5, 16) -> output (2, 5, 16) | averaged attention weights (2, 5, 5)
positional encoding rows 0 and 1 (first 4 dims): [0. 1. 0. 1.] [0.841 0.54  0.682 0.732]
```

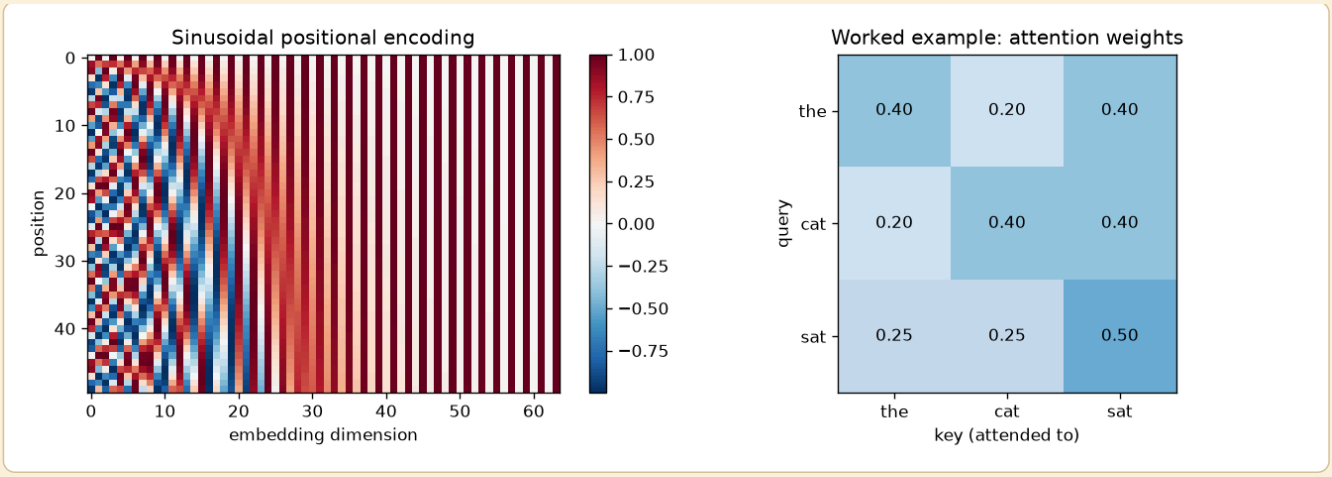


Figure — Left: sinusoidal positional encodings for 50 positions × 64 dimensions; low dimensions vary fast, high ones slowly. Right: the attention weights from the worked example.

Trade-offs

	RNN/LSTM	Transformer
Compute per layer	$O(n \cdot d^2)$	$O(n^2 \cdot d)$ for attention — quadratic in sequence length n
Parallel over sequence	No	Yes
Path length between distant tokens	$O(n)$	$O(1)$
Memory at inference	Constant state	Key/value cache grows with context

Long contexts are expensive (n^2), which motivates efficient-attention variants (FlashAttention computes exact attention faster and with less memory; sparse/linear attention and state-space models such as Mamba trade off differently).

Where to go next: the AI Handbook's Chapter 3 — Transformers and Embeddings builds the transformer step by step, from tokens to next-token probabilities; its Chapter 4 — Large Language Models covers training objectives, scaling, fine-tuning and RLHF. This book stops at the ML foundations.

In the interview

> **30-second answer:** "Self-attention lets each token compute a weighted average of all tokens' value vectors, with weights from softmax of query-key dot products scaled by √d. A transformer stacks multi-head self-attention and position-wise feed-forward layers with residuals and layer norm, adding positional encodings because attention is order-agnostic. Encoders attend bidirectionally; decoders use causal masks and cross-attention to the encoder. It replaced RNNs because it's parallel over the sequence and connects any two positions in one step; its cost is quadratic in sequence length."

- *Q: Why multiple heads?* A: Each head attends in a different learned subspace, capturing different relations; concatenation combines them at the same total cost as one full-width head.
- *Q: Why is a causal mask needed in a decoder?* A: During training all positions are processed at once; the mask stops position i from seeing the tokens it's supposed to predict.
- *Q: Why positional encodings?* A: Self-attention is permutation-equivariant — without positions, "dog bites man" = "man bites dog".

Self-check

1. In self-attention where do Q, K, V come from? → The same sequence (via three learned projections).
2. What's cross-attention? → Queries from the decoder, keys and values from the encoder output.
3. Complexity of self-attention in sequence length n? → $O(n^2)$ time and memory for the score matrix.
4. Which family is GPT? BERT? → Decoder-only; encoder-only.

TL;DR

Transformer = embeddings + positional encoding + N × (multi-head self-attention + feed-forward, each with residual + LayerNorm). Encoder bidirectional, decoder causal + cross-attention. Parallel, short paths, $O(n^2)$. Details: AI Handbook, Chapter 3.

12.8 Text preprocessing and tokenization

What it is

Neural networks need numbers. **Tokenization** splits text into units (**tokens**) and maps each to an integer id from a **vocabulary**; an embedding layer then maps ids to vectors.

How it works — a classic pipeline

1. **Normalise**: lowercase (sometimes), Unicode normalisation, replace URLs/numbers with placeholders, strip markup.

2. **Tokenize**: split into tokens.
3. **Build a vocabulary** from the training data, usually with a frequency cut-off; add special tokens: `<pad>` (padding), `<unk>` (unknown/out-of-vocabulary), and for seq2seq `<sos>` / `<eos>`.
4. **Encode**: tokens → ids; unknown words → `<unk>`.
5. **Pad/truncate** to a fixed length so sequences can be batched (and use masks so padding is ignored).

Classic NLP also used **stop-word removal** ("the", "a") and **stemming/lemmatization** ("running" → "run"). These help bag-of-words models (TF-IDF + logistic regression) but are usually **not** used with neural models — the model benefits from every word, and "not" is a stop word you really don't want to drop in sentiment analysis.

```
def normalize(text):
    text = text.lower()
    text = re.sub(r"https?://\S+", "<url>", text)          # replace URLs with a placeholder
    return re.findall(r"<url>|[a-z0-9]+(?:'[a-z]+)?|[.,!?]", text)

def encode(tokens, stoi, max_len):
    ids = [stoi.get(t, stoi["<unk>"]) for t in tokens][:max_len]   # unknown words -> <unk>, truncate
    return ids + [stoi["<pad>"]] * (max_len - len(ids))            # pad to a fixed length
```

Output (`code/text_and_embeddings.py`):

```
'The cat sat on the mat!'                          -> ['the', 'cat', 'sat', 'on', 'the', 'mat', '!']
"Dogs don't like cats, see https://example.com"    -> ['dogs', "don't", 'like', 'cats', ',', 'see', '<url>']
'The mat is red.'                                  -> ['the', 'mat', 'is', 'red', '.']
vocab size 18 | encode('the dog sat') = [17, 1, 15, 0, 0, 0]
characters of 'mat': ['m', 'a', 't'] | bytes of 'café' (UTF-8): [99, 97, 102, 195, 169]
```

"dog" was never seen (only "dogs"), so it became id 1 = `<unk>` — the weakness of word-level vocabularies.

Word vs character vs subword tokens

Level	Vocabulary	Sequence length	Unknown words	Used by
Word	Huge (100k+), still misses rare words and typos	Short	`<unk>` problem	Classic NLP, word2vec
Character / byte	Tiny (≈100 chars, 256 bytes)	Very long	None	Some specialised models (ByT5)
Subword (BPE, WordPiece, Unigram)	Medium (≈30k–200k)	Medium	Rare words split into known pieces	Essentially all modern LLMs

Byte-Pair Encoding (BPE) (applied to NLP by Sennrich et al., 2016): start from characters; repeatedly find the **most frequent adjacent pair** of symbols in the training corpus and merge it into a new symbol; stop after a set number of merges (= vocabulary size). Frequent words become single tokens; rare words split into meaningful pieces ("lowest" → "low" + "est").

```
BPE merges learned: ['lo', 'low', 'er', 'er</w>', 'low</w>', 'es']
words as subword tokens: ['low er</w>', 'low es t </w>', 'low</w>', 'n e w er</w>', 'n e w es t </w>', 'w i d er</w>', 'w i d es t </w>']
```

(`</w>` marks the end of a word.) After six merges, "low" is a single token and the suffixes "er" and "es" are emerging; more merges would produce "est" and whole frequent words. GPT-2 onward use **byte-level BPE** (start from bytes, so any string — any language, emoji, code — can be encoded with no `<unk>`); BERT uses **WordPiece**; T5 and many multilingual models use **SentencePiece** (Unigram or BPE). As a rough rule, one token is about three-quarters of an English word (≈4 characters), which is why API prices and context windows are quoted in tokens.

Common mistakes

- Building the vocabulary (or fitting a tokenizer) on validation/test text.
- Using a different tokenizer at inference than in training — with a pretrained model, always load its own tokenizer.
- Removing stop words/negations for neural sentiment models.
- Forgetting attention/padding masks, so the model "reads" padding.

Self-check

1. Why does a word-level vocabulary need `<unk>`? → Words not seen in training have no id.
2. What does one BPE merge do? → Replaces the most frequent adjacent symbol pair with a new symbol.
3. Why is byte-level BPE free of unknown tokens? → Any text is a byte sequence and all 256 bytes are in the base vocabulary.

TL;DR

Normalise → tokenize → vocabulary with `<pad>` / `<unk>` → ids → pad. Modern models use subword tokenizers (BPE/WordPiece/SentencePiece), which have no out-of-vocabulary problem.

12.9 Word embeddings

What it is

A **word embedding** is a dense vector (e.g. 100–300 numbers) representing a word such that words used in similar contexts get similar vectors. It replaces **one-hot encoding** (a 50,000-long vector with a single 1), in which every word is equally different from every other ("cat" is as far from "kitten" as from "carburettor").

Intuition — the **distributional hypothesis**: "you shall know a word by the company it keeps" (J. R. Firth, 1957). Words that appear with the same neighbours ("pet", "fur", "vet") end up close together.

How word2vec works (Mikolov et al., 2013)

Two variants, both shallow networks trained on raw text:

- **Skip-gram**: from a centre word, predict each word in a window around it.
- **CBOW** (continuous bag of words): from the surrounding words, predict the centre word.

Skip-gram with **negative sampling** in steps:

1. Slide a window (e.g. ±2 words) over the corpus; every (centre, context) pair is a positive example.
2. For each positive pair, draw k random "negative" words from the vocabulary (frequency^0.75 distribution).
3. Each word has an input vector and a context vector. Train so σ(v_centre · u_context) → 1 for real pairs and → 0 for negatives (binary cross-entropy).
4. Keep the input vectors as the embeddings.

Because words that share contexts receive similar gradient updates, their vectors converge. The famous side effect: vector arithmetic captures relations — vec("king") − vec("man") + vec("woman") is close to vec("queen") in embeddings trained on large corpora.

```
class SkipGram(nn.Module):
    def __init__(self, n_words, dim=16):
        super().__init__()
        self.inp = nn.Embedding(n_words, dim)      # the word vectors we keep
        self.ctx = nn.Embedding(n_words, dim)      # separate vectors for words used as context
    def forward(self, center, context, negatives):
        c = self.inp(center)                                          # (B, d)
        pos = (c * self.ctx(context)).sum(-1)                         # score for the real pair
        neg = torch.bmm(self.ctx(negatives), c.unsqueeze(-1)).squeeze(-1)  # scores for random pairs
        return -(nn.functional.logsigmoid(pos) + nn.functional.logsigmoid(-neg).sum(-1)).mean()
```

Trained on a synthetic 3,000-sentence corpus where royal, animal and fruit nouns each appear with their own context words:

```
word2vec: 3000 sentences, vocab 31, final batch loss 1.709
  nearest to 'king': [('kingdom', 0.78), ('throne', 0.6), ('queen', 0.59)]
  nearest to 'dog': [('chases', 0.7), ('pet', 0.68), ('cat', 0.55)]
  nearest to 'mango': [('apple', 0.72), ('cherry', 0.71), ('banana', 0.7)]
  mean cosine: same-group nouns 0.50  vs  different-group nouns 0.20
```

The model was never told the groups: "king" ends up near "queen", "dog" near "cat", "mango" near other fruits purely from co-occurrence. (Topical context words like "kingdom" also rank high — embeddings capture relatedness as well as similarity.)

Other embeddings

Method	Idea	Note
GloVe (Stanford, 2014)	Factorise global word co-occurrence statistics	Similar quality to word2vec
fastText (Facebook, 2016)	Word vector = sum of character n-gram vectors	Handles rare/misspelled words
Contextual embeddings (ELMo 2018, BERT 2018, all transformers)	A word's vector depends on the sentence — "bank" (river) ≠ "bank" (money)	Static word2vec gives one vector per word regardless of meaning
Sentence/document embeddings	One vector per text	Semantic search and RAG (AI Handbook, Chapter 7)

In PyTorch every NLP model starts with `nn.Embedding(vocab_size, dim)` — a trainable lookup table that is learned with the rest of the model (optionally initialised from pretrained vectors).

Real-world examples

Embeddings of words, products, users and songs drive recommendation (e.g. Airbnb published listing embeddings learned word2vec-style from users' browsing sessions, KDD 2018), search ranking, and today's vector databases for retrieval-augmented generation.

In the interview

30-second answer: "Embeddings map discrete tokens to dense vectors where geometry reflects meaning. word2vec learns them by predicting context words from a centre word (skip-gram) or vice versa (CBOW), using negative sampling to avoid a full softmax; words with similar contexts get similar vectors. Static embeddings give one vector per word; transformers produce contextual embeddings that depend on the sentence."

- *Q: Why negative sampling?* A: A softmax over the whole vocabulary per pair is expensive; turning it into k+1 binary classifications is cheap.
- *Q: Limitation of word2vec?* A: One vector per word regardless of sense; no vectors for unseen words (fastText helps).

Self-check

1. What's wrong with one-hot vectors? → Huge, sparse, and every pair of words is equally dissimilar.
2. Skip-gram predicts what from what? → Context words from the centre word.
3. What does `nn.Embedding(10000, 128)` hold? → A 10,000 × 128 trainable table, one row per token id.

TL;DR

Embeddings = learned dense vectors; similar contexts → similar vectors (word2vec skip-gram/CBOW with negative sampling). Modern models learn contextual embeddings.

12.10 Time-series forecasting — an introduction

What it is

Forecasting predicts future values of a sequence of measurements taken over time (daily sales, hourly energy demand, server load). Special care is needed because observations are **ordered and dependent**, and the future may not look like the past.

Vocabulary:

- **Trend**: long-term increase/decrease. **Seasonality**: repeating patterns (weekly, yearly). **Noise**: the unpredictable rest.
- **Lag**: a past value; lag_7 at day t is the value at t − 7.
- **Horizon**: how far ahead you predict (one-step vs multi-step).
- **Stationarity**: statistical properties (mean, variance) don't change over time — many classical methods assume it; **differencing** (y_t − y_{t−1}) removes trends.

How it works — framing forecasting as supervised learning

1. Build a table where each row is a time t and features are **only past information**: lags, rolling means (shifted so the current value doesn't leak), calendar features (day of week, holiday).
2. Target = the value at t (or t + h for horizon h).
3. Split **chronologically**: train on the past, test on the future. For cross-validation use expanding/rolling windows (`sklearn.model_selection.TimeSeriesSplit`), never random K-fold.
4. Always compare with **baselines**: naive (repeat the last value) and seasonal naive (repeat the value from one season ago).

`code/time_series.py` — a synthetic daily series with an upward trend, a weekly cycle and noise (std 2):

```
def lag_features(s, lags=(1, 2, 3, 7, 14)):
    df = pd.DataFrame({"y": s})
    for L in lags:
        df[f"lag_{L}"] = s.shift(L)                      # value L days earlier (only past info!)
    df["rolling_mean_7"] = s.shift(1).rolling(7).mean()   # shift first so today's value doesn't leak
    df["dayofweek"] = s.index.dayofweek
    return df.dropna()
```

Output:

```
feature table (first 3 rows):
                y  lag_1  lag_2  lag_3  lag_7  lag_14  rolling_mean_7  dayofweek
2024-01-15   98.9   93.0   86.8   96.8  102.6   100.3            99.8          0
2024-01-16  107.9   98.9   93.0   86.8  107.2   107.7            99.3          1
2024-01-17  110.3  107.9   98.9   93.0  108.1   111.2            99.4          2

one-step-ahead MAE on the last 60 days (noise std = 2.0):
  naive (yesterday)                          5.94
  seasonal naive (same weekday last week)    2.20
  linear AR on lag features                  1.80
  gradient boosting on lag features          2.77
  gradient boosting on differences           1.90
train target range 91-154, test range 134-160
```

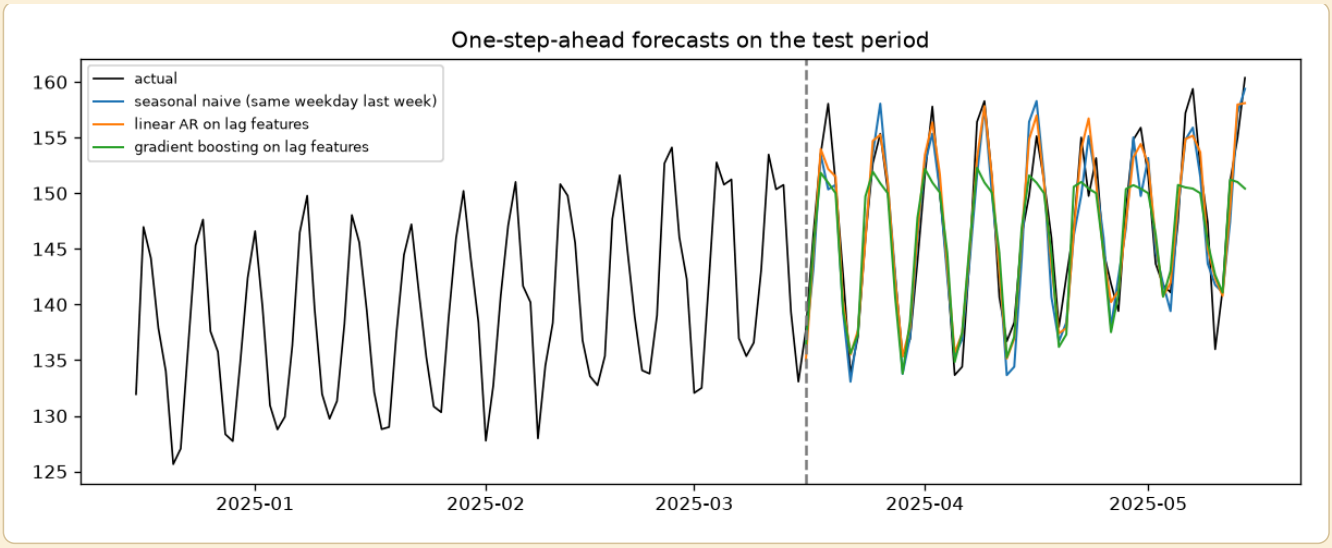


Figure — Last 150 days with one-step-ahead forecasts over the 60-day test period (dashed line = start of test).

Lessons:

- The **seasonal naive** baseline is already strong (2.20); the naive one is poor because it ignores the weekly cycle.
- A **linear autoregression** on lags wins here (1.80, close to the noise floor of ≈1.6 = the MAE of pure N(0, 2) noise) — the data is linear.
- **Gradient boosting on raw levels is worse than the baseline** (2.77): trees can't **extrapolate** — test values (up to 160) exceed most of the training range, and a tree can only output values it saw. Modelling the **change** (differences) fixes it (1.90). This is a classic trap.

ARIMA vs ML — brief

	ARIMA / exponential smoothing (classical statistics)	**ML with lag features (GBM, linear)**	**Deep learning (LSTM, transformers, N-BEATS…)**
Model	ARIMA(p, d, q): **AR** — regression on p own lags; **I** — difference d times for stationarity; **MA** — regression on q past forecast errors. SARIMA adds seasonal terms. ETS/Holt-Winters smooth level, trend, season	Any regressor on engineered lag/calendar features	Learn features from raw windows
Series handled	One at a time (univariate)	Many series in one global model	Many series, global
Extra features (price, promotions, weather)	Limited (ARIMAX)	Easy	Easy
Data needed	Small (dozens–hundreds of points)	Medium	Large
Uncertainty	Built-in prediction intervals	Needs quantile loss or conformal methods	Needs extra work
Tools	`statsmodels` (ARIMA, ETS), `pmdarima`, Prophet	scikit-learn, LightGBM, XGBoost	PyTorch, `darts`, `neuralforecast`

Practical guidance: start with seasonal-naive and ETS/ARIMA baselines; for many related series with extra features, a global gradient-boosting model on lag features is usually the strongest practical choice (LightGBM-based approaches dominated the M5 retail-forecasting competition in 2020); deep models pay off with lots of data (the M4 competition in 2018 was won by a hybrid of exponential smoothing and an RNN). Pretrained forecasting foundation models (e.g. Amazon's Chronos, Google's TimesFM) are an emerging option.

Common mistakes

- Random train/test splits or K-fold on time series (future leaks into training).
- Rolling features that include the current value (leakage).
- Skipping baselines — a fancy model that loses to seasonal naive is common.
- Trees on trending levels (no extrapolation) — difference or detrend first.
- Evaluating one-step-ahead when the business needs 30 days ahead (multi-step errors compound).

In the interview

30-second answer: "I frame forecasting as supervised learning with only past information as features — lags, shifted rolling statistics, calendar effects — and split chronologically with rolling-origin validation. I always beat naive and seasonal-naive baselines first. ARIMA models a single series with autoregressive, differencing and moving-average terms and gives intervals; ML models like LightGBM handle many series and external features but can't extrapolate trends unless I model differences. Deep models help with lots of data."

- *Q: What does the I in ARIMA do?* A: Differencing to remove trends so the series is closer to stationary.

- *Q: How do you cross-validate a forecaster?* A: Expanding or sliding windows — always train on earlier data and validate on the following period (`TimeSeriesSplit`).

Self-check

1. What's a seasonal naive forecast for daily data with a weekly pattern? → The value from the same weekday last week (lag 7).
2. Why must a rolling mean be shifted by one? → Otherwise it includes the value being predicted — leakage.
3. Why did the GBM on levels underperform? → Test values exceeded the training range and trees can't extrapolate.

TL;DR

Past-only features (lags, shifted rolling stats, calendar), chronological splits, strong baselines. ARIMA = AR + differencing + MA for single series; GBMs with lag features for many series (model differences to handle trend); deep models for lots of data.

Chapter TL;DR

- **Sequences** are ordered, variable-length; models must share parameters over time.
- **RNN**: $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$; unroll and train with **BPTT**; gradients through time **vanish/explode** (clip for explosions).
- **LSTM**: cell state with forget/input/output gates; $c_t = f \odot c_{t-1} + i \odot \tilde{c}$ gives an additive gradient path. **GRU**: update/reset gates, lighter. In our toy task the RNN failed at 15 steps; LSTM/GRU reached 100%.
- **Seq2seq**: encoder → context → decoder; teacher forcing for training, greedy/beam search for inference; suffers a fixed-size bottleneck.
- **Attention**: $\text{softmax}(QK^T/\sqrt{d_k})V$ — the decoder looks back at all encoder states; our reversal task went from 16% to 98% perfect sequences.
- **Transformer**: self-attention + feed-forward blocks with residuals and LayerNorm; positional encodings; encoder (bidirectional), decoder (causal + cross-attention); parallel, $O(n^2)$. Deep dive: AI Handbook, Chapter 3.
- **Text**: normalise → tokenize (subword BPE/WordPiece in modern models) → ids → embeddings. **word2vec** learns vectors from context (skip-gram/CBOW + negative sampling); transformers give contextual embeddings.
- **Time series**: lag features, chronological splits, seasonal-naive baselines; ARIMA for single series, GBMs on lags (watch extrapolation), deep models with lots of data.

Chapter 13 — Recommender Systems and Embeddings

When Netflix fills a row with shows, YouTube picks the next video or Amazon shows "customers who bought this also bought", a **recommender system** is at work: it predicts which items (movies, songs, products, posts) a particular user is most likely to want, from a catalogue far too large to browse. This chapter builds recommenders from a ratings matrix and a similarity formula, through matrix factorisation and learned **embeddings**, up to the **two-tower + ranking funnel** used in production — and shows how to evaluate them honestly.

What you'll learn

1. Framing (rating prediction vs top-K ranking), the user–item matrix, explicit vs implicit feedback.
2. Similarity measures; content-based vs collaborative filtering (user- and item-based) vs hybrids.
3. Matrix factorisation with SGD and ALS (implemented), plus its implicit-feedback version.
4. Embeddings, word2vec and item2vec (implemented).
5. Two-tower retrieval: in-batch negatives, serving from a nearest-neighbour index, freshness, versioning.
6. The funnel (retrieval → ranking → re-ranking with MMR) and approximate nearest neighbour search (IVF implemented, HNSW, PQ).
7. Cold start, popularity bias and feedback loops.
8. Evaluation: precision/recall@k, MAP, NDCG, hit rate, coverage, diversity, novelty, A/B tests — and how Netflix, YouTube, Spotify and Amazon do it.

All code is in `code/` (numpy, scikit-learn, PyTorch; no downloads — `code/data.py` generates a small dataset shaped like **MovieLens**, the classic public movie-ratings set from GroupLens). Outputs below are real.

13.1 The recommendation problem

What it is

We have a set of **users** U, a set of **items** I, and a log of **interactions** (ratings, clicks, plays, purchases). The job: for a user u, produce a short **ranked list** of items u has not yet seen and is likely to value.

Analogy: a good bookshop clerk. They remember what you bought before (your history), know which books are alike (item similarity), and notice that people with your taste loved a particular new novel (collaborative signal). A recommender automates that clerk for millions of customers at once.

Why it matters

- **Choice overload**: millions of items, about ten fit on a phone screen.
- **Business value**: Gomez-Uribe & Hunt (2015) wrote that Netflix's recommender influences the choice behind about 80% of hours streamed, with most of the rest coming from search. Recommendations drive discovery, engagement and sales.
- **Two-sided value**: they help creators and sellers in the "long tail" (the many niche items) find an audience.

How to frame it

There are two classic formulations:

Framing	Question the model answers	Typical loss / metric	Example
Rating prediction (regression)	"How many stars would u give i?"	Squared error / RMSE	The Netflix Prize (2006–2009)
Top-K ranking (the real product)	"Which K items should u see, in what order?"	Ranking losses / precision@K, recall@K, NDCG	Home feeds, "Up next", playlists

Modern systems almost always optimise the **ranking** framing: users only see the top of the list, and most feedback is implicit (clicks, watch time), not stars. Before modelling, answer the framing questions an interviewer expects:

1. **What is a "good" outcome?** A click is cheap; watch time, a purchase or a "save" is closer to real value. Clicks alone reward clickbait.
2. **What signals exist?** Ratings, views, dwell time, skips, purchases, searches, item metadata, user profile, context (device, time, location).
3. **Scale and latency.** 1,000 items can be scored exhaustively; 100 million cannot — hence the funnel of §13.8.
4. **Freshness** (news goes stale in hours, movies last years) and **constraints** (diversity, creator fairness, safety filters, "don't show what they already bought").

The user–item matrix

Users are rows, items are columns; cell (u, i) holds the interaction (stars, or 1 for "watched") or is empty.

```
        Movie1  Movie2  Movie3  Movie4
Ana        5       4       .       1
Ben        4       5       1       .
Cy         1       .       5       4
```

Two facts define the whole field:

- The matrix is **huge**: users × items can be $10^9 \times 10^8$.
- It is **extremely sparse**: a typical user touches a vanishing fraction of items. Even our small synthetic dataset has only ~9% of cells filled; real ones are often below 0.1%.

Recommendation = **filling in the missing cells** well enough that the highest-scoring empty cells in each row are items the user would actually enjoy.

`code/data.py` builds the dataset used throughout the chapter: 600 users, 400 movies, 6 genres. Each user has hidden genre tastes; each movie has 1–2 genres and a hidden quality; popular movies are seen much more often (a long tail), like real catalogues.

```
affinity = taste @ item_genres.T / item_genres.sum(1) + quality     # hidden "true" liking
stars = clip(round(3 + 1.2 * affinity + noise), 1, 5)                # what a rating would be
p_seen = popularity[None, :] * exp(1.5 * affinity)                   # people watch popular + liked
observed = rng.random(p_seen.shape) < p_seen                          # only these cells are known
```

Output:

```
users=600 items=400 ratings=21216
density = 8.840%
top 10% of items receive 38% of all ratings
star histogram: {1: 47, 2: 624, 3: 3054, 4: 6335, 5: 11156}
```

The star histogram is skewed high because people mostly watch (and rate) what they expect to like. This **selection bias** (data "missing not at random") holds in real data too: observed ratings are not a random sample of opinions.

TL;DR

Recommenders fill in a huge, sparse user–item matrix and show each user the top-scoring empty cells. Frame it as **top-K ranking** on a well-chosen outcome (not just clicks), and state scale, latency, freshness and constraints before choosing a model.

13.2 Explicit vs implicit feedback

What it is

- **Explicit feedback**: the user tells you directly — 1–5 stars, thumbs up/down, a review.
- **Implicit feedback**: you infer preference from behaviour — clicks, plays, watch time, purchases, add-to-cart, skips, dwell time.

Why it matters

Implicit data is **plentiful** (every session produces it) while explicit data is **scarce** (few people rate anything). Netflix itself replaced 5-star ratings with thumbs up/down in 2017, saying far more members were willing to give a thumb than a star rating. So most production recommenders learn mainly from implicit signals — which changes the maths.

	Explicit	Implicit
Volume	Low	Very high
Negative signal	Yes (1 star = dislike)	Mostly **no**: "not watched" may mean "never saw it"
Noise	Rating scale differs by person	A click may be accidental; a play may be background noise
Strength	Clear preference	Can be graded: watch 90% vs 5%, played 30 times
Modelling	Regress on the observed cells only	Treat all cells: observed = positive, unobserved = weak negative, with **confidence** weights

How it works — turning behaviour into training data

1. Define the **positive event** (watched > 50%, purchased, saved a track).
2. Optionally turn intensity into **confidence**: $c = 1 + \alpha \cdot$(play count), α a tuning constant.
3. **Sample negatives**: random unobserved items, or — much stronger — items shown but not clicked ("impressions without clicks").
4. Correct **position bias** (top slots get clicks just for being on top): log the position, or randomise a small slice of traffic.

Common mistakes

- Treating every unobserved cell as a full-weight "dislike".
- Optimising clicks when the real goal is satisfaction (clickbait wins).
- Ignoring the **negative implicit signals** you do have: early skips, "not interested", short dwell.

TL;DR

Explicit = stars (scarce, has real negatives). Implicit = behaviour (abundant, positives only, unobserved ≠ negative). Use confidence weights and sampled negatives for implicit data.

13.3 Similarity measures

What it is

A **similarity** function gives a number saying how alike two vectors (two users' rating rows, two items' columns, two embeddings) or two sets are.

Measure	Formula	Range	Use when
Dot product	$a \cdot b = \Sigma\, a_k b_k$	$(-\infty, \infty)$	Embeddings where vector length carries meaning (e.g. popularity)
Cosine	$a \cdot b \,/\, (\|a\|\|b\|)$	$[-1, 1]$	Direction matters, length does not (text, rating vectors)
Adjusted cosine / Pearson	cosine after subtracting each user's mean rating	$[-1, 1]$	Ratings, where some users rate everything high
Jaccard	$\|A \cap B\| \,/\, \|A \cup B\|$	$[0, 1]$	Sets / binary data: "users who bought both"
Euclidean distance	$\|a - b\|$ (smaller = more similar)	$[0, \infty)$	Dense embeddings; for unit vectors $\|a-b\|^2 = 2 - 2\cdot\cos$

How it works — worked numbers

Using the tiny matrix from §13.1 (0 = not rated):

- cos(Ana, Ben) = (5·4 + 4·5 + 0·1 + 1·0) / (√42 · √42) = 40 / 42 = **0.952** → very similar taste.
- cos(Ana, Cy) = (5·1 + 0 + 0 + 1·4) / 42 = 9 / 42 = **0.214** → different taste.
- Jaccard on the sets of movies each *liked* (≥ 4 stars): Ana {1, 2}, Ben {1, 2} → 2/2 = **1.0**; Ana vs Cy {3, 4} → 0/4 = **0**.

Why subtract the mean? A generous rater gives 4–5 to everything, a harsh one 2–3; raw cosine sees them as different even when they order movies identically. Mean-centring first (adjusted cosine ≈ Pearson correlation) compares *relative* preferences.

Dot product vs cosine. A dot product keeps vector length, so an item whose vector grew long because it is broadly popular scores higher for everyone — a real signal or a bias, depending on your goal.

Common mistakes

- Raw cosine where 0 means "missing" — zeros drag similarities down. Mean-centre on observed entries only.
- Trusting similarities built on 2 co-rated items — **shrink** them (pull toward 0 when overlap is small) or require a minimum overlap.
- Training with dot product but serving with cosine (or vice versa) — rankings differ.

Self-check

1. Two users rated the same 3 movies [5, 4, 3] and [3, 2, 1]. Raw cosine? Adjusted cosine? → Raw ≈ 0.98 (both positive vectors). After subtracting means (4 and 2): [1, 0, −1] and [1, 0, −1] → **1.0**, same relative taste.
2. When is Jaccard the right choice? → Binary/set data such as "bought" or "followed", with no graded strength.
3. For unit-length vectors, how do cosine and Euclidean rankings relate? → Identical: $\|a-b\|^2 = 2 - 2\cos(a,b)$.

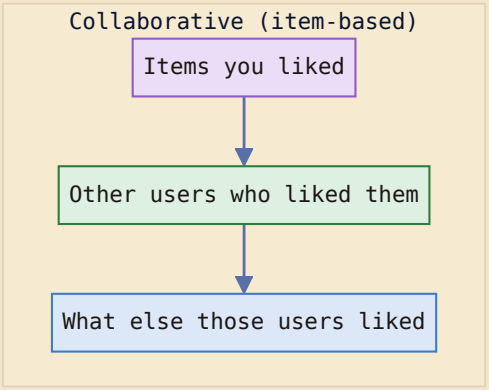
TL;DR

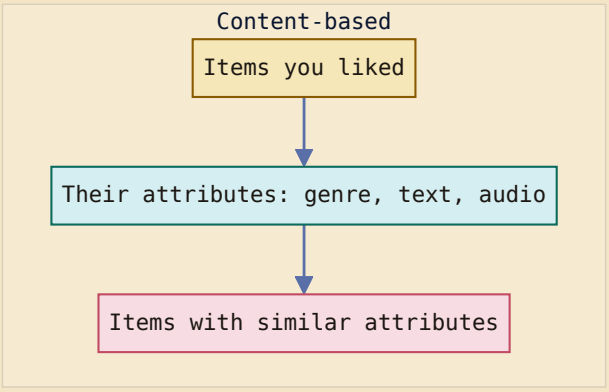
Cosine for direction, dot product when magnitude matters, adjusted cosine for ratings, Jaccard for sets. Mean-centre ratings and shrink similarities built on small overlaps.

13.4 Content-based vs collaborative filtering

What it is

- **Content-based filtering** recommends items **similar in their attributes** to what the user liked: genres, actors, text description, audio features, image. "You watched three sci-fi films → here is another sci-fi film."
- **Collaborative filtering (CF)** ignores item attributes and uses **other users' behaviour**: "people who liked what you liked also liked X." The word *collaborative* means users implicitly help each other.
  - **User-based CF**: find users similar to u, recommend what they liked.
  - **Item-based CF**: for each item u liked, find items that *the same users* tended to like, recommend the nearest ones. Amazon made this famous.
- **Hybrid**: combine both (weighted blend, or feed both signals into one model).





How item-based CF predicts a rating

To predict how user u would rate item i:

1. Compute item–item similarity S(i, j) for all pairs (adjusted cosine over the users who rated both).
2. Take the k items most similar to i **that u has rated**.
3. Predict a similarity-weighted average of u's (mean-centred) ratings on them: r̂(u, i) = mean(u) + Σⱼ S(i, j)·(r(u, j) − mean(u)) / Σⱼ |S(i, j)| (the absolute value keeps the denominator positive if some similarities are negative; our code simply keeps only positive neighbours.)

Item-based usually beats user-based in practice: items have many more ratings each than users do, item–item similarities are more stable over time, and they can be precomputed offline.

Code — `code/similarity_cf.py`

```
# collaborative: item-item adjusted cosine on mean-centred ratings
Rc = np.where(mask, R - user_mean[:, None], 0.0)
norms = np.linalg.norm(Rc, axis=0) + 1e-12
S_cf = (Rc.T @ Rc) / np.outer(norms, norms)
# content-based: item-item cosine on genre vectors
Gn = G / np.linalg.norm(G, axis=1, keepdims=True)
S_cb = Gn @ Gn.T
# ... predict each held-out rating with the k=20 most similar items the user rated
```

Output:

```
== Rating prediction on held-out ratings (RMSE, lower is better) ==
predict global mean      : 0.833
predict user's mean      : 0.811
content-based item-kNN   : 0.6872
collaborative item-kNN   : 0.6820
hybrid (average of both) : 0.6711

Movie 51 (Drama) - 5 nearest neighbours
  collaborative: ['40:Drama', '341:Drama', '321:Drama', '325:Drama', '270:Drama']
  content-based: ['109:Drama', '40:Drama', '52:Drama', '56:Drama', '341:Drama']
```

What this shows:

- Both beat the naive baselines clearly, and the **hybrid** beats both — they capture partly different information.
- **CF discovered "Drama" by itself** from who-rated-what alone, never seeing a genre label.
- Here genre drives taste, so content does well. On real data CF usually wins for items with plenty of history (behaviour captures quality and mood that metadata misses); content wins for new items.

Trade-offs

	Content-based	Collaborative filtering
Needs	Item attributes / content	Many user interactions
New item (no history)	✅ Works immediately	❌ Cold start
New user	⚠️ Needs a few likes	❌ Cold start
Serendipity	Low — "more of the same" (filter bubble)	High — surprising cross-genre finds
Captures quality/taste beyond metadata	❌	✅
Explainability	"Because it's sci-fi like X"	"People like you also liked X"
Popularity bias	Low	High (popular items have more co-occurrences)

When not to use it

- Pure CF when the catalogue turns over daily (news, listings) — most items are always cold.
- Pure content-based when metadata is poor or quality varies wildly within a genre.

Common mistakes

- Re-recommending items the user already consumed.
- User–user similarity over 100 M users ($O(U^2)$) — go item-based, or embeddings + approximate nearest neighbours.
- A random split for a "predict what they do next" product — use a **time-based** split (§13.11).

Real-world examples

- **Amazon** "Customers who bought this item also bought": item-to-item CF (Linden, Smith & York, *IEEE Internet Computing*, 2003). Its key idea was precomputing item–item similarities offline so the online step was a cheap lookup.
- **Pandora**'s Music Genome Project: content-based, with trained analysts tagging songs on hundreds of musical attributes.
- Most modern systems are **hybrids**.

In the interview

> **30-second answer:** "Content-based matches item attributes to what the user liked — it handles new items but over-specialises. Collaborative filtering uses co-behaviour across users — item-based CF is the scalable classic — and finds surprising, quality-aware matches but has cold start and popularity bias. Production systems combine both, often in an embedding model whose item side uses ID and content features."

- *Why item-based over user-based?* More interactions per item, more stable similarities, precomputable offline.
- *How do you make CF scale?* Precompute top-N neighbours per item, or learn embeddings and use an ANN index.
- *Rich metadata, sparse interactions?* Start content-heavy; blend in CF as data grows.

Self-check

1. What does CF need that content-based does not? → Interaction data from many users.
2. Which handles a movie released today? → Content-based (it only needs attributes).
3. Why is CF prone to popularity bias? → Popular items co-occur with everything, so they look similar to everything.

TL;DR

Content-based = "similar attributes"; collaborative = "similar people's behaviour". CF finds structure (even genres) from behaviour alone but needs history; content covers cold items. Hybrids win.

13.5 Matrix factorisation

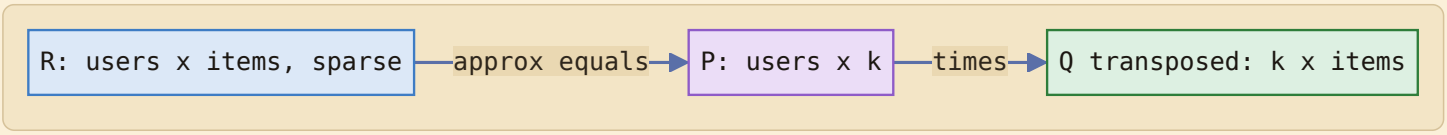
What it is

Matrix factorisation (MF) assumes each user and each item can be described by a short vector of k hidden **latent factors** ($k \approx 10$–200). A user's affinity for an item is the dot product of their vectors:

$$\hat{r}(u, i) = \mu + b_u + b_i + p_u \cdot q_i$$

- μ = global average rating; b_u = user bias (a generous rater); b_i = item bias (a universally loved film).
- $p_u \in \mathbb{R}^k$ = user factor vector; $q_i \in \mathbb{R}^k$ = item factor vector.

So the giant R (users × items) ≈ **P (users × k) · Q^T (k × items)** — two thin matrices. Analogy: instead of storing everyone's opinion of every song, store each person's "taste coordinates" and each song's "style coordinates" on a few hidden axes (say acoustic ↔ electronic); training discovers the axes, nobody names them.



Why it matters

- It was the star of the **Netflix Prize** (Simon Funk's 2006 SGD "SVD" blog post; Koren, Bell & Volinsky, 2009).
- It **generalises** CF: two users with no co-rated movie can still be similar if their vectors point the same way.
- Its vectors p_u, q_i are **embeddings** — the bridge to two-tower models (§13.7) — and storage drops from U·I to (U + I)·k.

How it works — the objective

Minimise the squared error on the **observed** ratings only, plus L2 **regularisation** so the vectors don't overfit:

$L = \Sigma_{(u,i)}$ observed $[(r - \hat{r})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)]$

(λ = regularisation strength; the penalty sits inside the sum, as in Koren, Bell & Volinsky 2009, which is what the per-rating SGD updates below minimise. The constant factor 2 from differentiating the square is absorbed into the learning rate η.)

This is not classical SVD: SVD needs a full matrix, and filling missing cells with zeros would tell the model "everyone hates everything unseen". We optimise only over known cells. Two ways to minimise it:

1. Stochastic gradient descent (SGD) — loop over ratings one at a time:

1. err = r − r̂(u, i)
2. $b_u \leftarrow b_u + \eta(\text{err} - \lambda b_u)$; $b_i \leftarrow b_i + \eta(\text{err} - \lambda b_i)$
3. $p_u \leftarrow p_u + \eta(\text{err}\cdot q_i - \lambda p_u)$; $q_i \leftarrow q_i + \eta(\text{err}\cdot p_u - \lambda q_i)$ (use the old p_u in the second update)

2. Alternating least squares (ALS) — the objective is not convex in P and Q together, but it **is** a plain ridge regression if you freeze one of them:

1. Freeze Q. For each user, solve $p_u = (Q_u^T Q_u + \lambda I)^{-1} Q_u^T r_u$ exactly (Q_u = rows of Q for the items u rated; r_u = u's ratings on them, minus μ in our bias-free version). This is the ridge-regression formula with Q_u as the inputs and r_u as the targets.
2. Freeze P. Solve each q_i the same way.
3. Repeat until the error stops falling (usually 10–20 rounds).

Each user's solve is independent, so ALS **parallelises** easily (Spark MLlib ships it).

	SGD	ALS
Update	One rating at a time	Whole vector solved exactly
Parallelism	Harder (conflicting updates)	Embarrassingly parallel
Tuning	Learning rate, epochs	Few knobs, few iterations
Implicit data (all cells)	Needs negative sampling	Efficient (Hu et al. 2008)
Extra features, custom losses	Flexible	Less flexible

Code — `code/matrix_factorization.py`

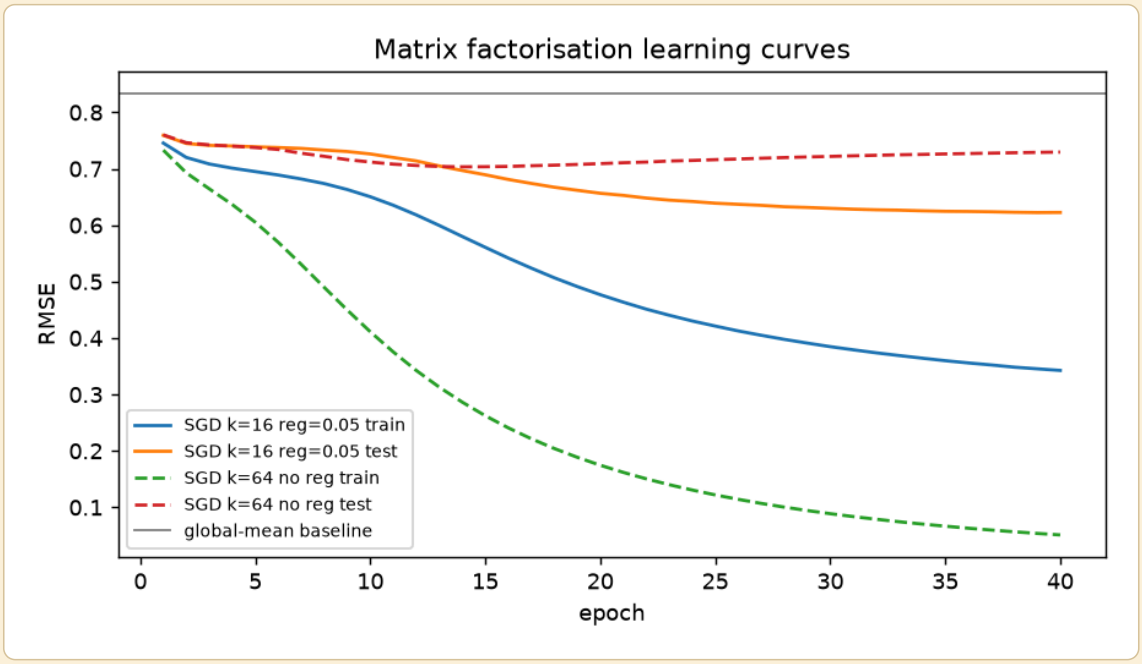
```
def mf_sgd(train, test, n_users, n_items, k=16, lr=0.02, reg=0.05, epochs=40, seed=0):
    ...
    for _ in range(epochs):
        for idx in rng.permutation(len(tr)):          # one rating at a time
            u, i, r = tu[idx], ti[idx], tr[idx]
            err = r - (mu + bu[u] + bi[i] + P[u] @ Q[i])
            bu[u] += lr * (err - reg * bu[u])
            bi[i] += lr * (err - reg * bi[i])
            pu = P[u].copy()
            P[u] += lr * (err * Q[i] - reg * P[u])
            Q[i] += lr * (err * pu - reg * Q[i])

def mf_als(train, test, n_users, n_items, k=16, reg=5.0, iters=10, seed=0):
    ...
        for u, idx in enumerate(by_user):              # fix Q, ridge-solve each user
            Qi = Q[ti[idx]]
            P[u] = np.linalg.solve(Qi.T @ Qi + I_k, Qi.T @ (tr[idx] - mu))
        # ... then the same loop over items with P fixed
```

Output:

```
baseline (global mean) test RMSE: 0.833
SGD epoch  1: train RMSE 0.746  test RMSE 0.759
SGD epoch  5: train RMSE 0.695  test RMSE 0.739
SGD epoch 10: train RMSE 0.650  test RMSE 0.726
SGD epoch 20: train RMSE 0.477  test RMSE 0.657
SGD epoch 40: train RMSE 0.343  test RMSE 0.623
SGD k=64, no regularisation, epoch 40: train 0.051  test 0.730
ALS iter   1: train RMSE 0.769  test RMSE 0.818
ALS iter   2: train RMSE 0.576  test RMSE 0.724
ALS iter   5: train RMSE 0.496  test RMSE 0.677
ALS iter  10: train RMSE 0.488  test RMSE 0.675

Top-5 for user 0: [98, 325, 236, 40, 314] predicted [5.16 5.14 5.11 4.98 4.93]
true (hidden) stars of those movies: [5. 5. 5. 5. 4.]
```



Reading the results:

- Regularised SGD reaches **0.623** test RMSE — better than item-kNN (0.682) and the mean (0.833).
- ALS converges in ~5 iterations (each step is an exact solve); it ends a little worse here because this simple version has no bias terms.
- The dashed lines show **overfitting**: 64 factors with no regularisation push training error to 0.05 while test error climbs back up to 0.73. With many parameters and sparse data, λ is not optional.
- User 0's top-5 are movies the generator says they would truly rate 5, 5, 5, 5, 4.

Implicit-feedback MF (weighted ALS)

For clicks/plays, Hu, Koren & Volinsky (2008) changed the objective to cover **every** cell:

- preference p_{ui} = 1 if u interacted with i, else 0;
- confidence c_{ui} = 1 + α·(count), so observed cells matter a lot and unobserved ones a little;
- minimise Σ over **all** (u, i) of $c_{ui}(p_{ui} - x_u \cdot y_i)^2 + \lambda(\ldots)$.

It is solved with ALS. The speed trick: the user solve needs $Y^T C_u Y$ (Y = item-factor matrix, C_u = diagonal matrix of u's confidences), rewritten as $Y^T Y + Y^T(C_u - I)Y$ — $Y^T Y$ is shared by all users, and the second term only involves the few items u actually touched. `code/implicit_eval.py` uses the plain dense form (fine for small data); libraries: `implicit` (Python), Spark MLlib `ALS(implicitPrefs=True)`.

Trade-offs / when not to use it

- MF sees only IDs: cold-start items have no vector, and context (time, device) is hard to add — use feature-based models (factorisation machines, two-towers).
- A dot product is a limited interaction function, yet well-tuned MF stays a strong baseline: Rendle et al. (2020) found a tuned dot product beat an MLP-based "neural collaborative filtering" model.

Common mistakes

- Zero-filling missing cells and running `np.linalg.svd` — teaches "unseen = hated".
- No regularisation, or tuning k on the test set.
- Dropping bias terms — much of the signal is just "this user rates high" and "this movie is loved".

Real-world examples

- **Netflix Prize**: *BellKor's Pragmatic Chaos* won in 2009 with the required 10% RMSE improvement over Cinematch, using a large blend of factor and neighbourhood models. Netflix wrote in 2012 that it productionised two components from an earlier progress prize (an SVD-style factorisation and restricted Boltzmann machines) but not the final blend — the extra accuracy did not justify the engineering cost.
- **Spotify** used implicit-feedback MF on listening data as one of the pillars behind Discover Weekly.

In the interview

> **30-second answer:** "MF gives every user and item a k-dimensional vector and predicts affinity as a dot product plus biases, trained on observed entries with L2 — by SGD, or by ALS, which becomes many small parallel ridge regressions. For implicit data I'd use weighted ALS with confidence weights. Weak spots: cold start and no context, which feature-based two-tower models fix."

- *Why not plain SVD?* It needs a complete matrix; imputing zeros biases toward "dislike".
- *How to choose k?* Validation sweep; larger k needs more regularisation.

Self-check

1. What do b_u and b_i capture? → A user's general generosity; an item's general appeal.
2. Why regularise? → Few ratings per user/item vs many parameters → overfitting (see the dashed curve).
3. What makes ALS easy to distribute, and how does the implicit version differ? → With one side fixed, each user (or item) is an independent least-squares solve; implicit ALS covers all cells with 0/1 preferences and confidence weights.

TL;DR

MF = users and items as short vectors, affinity = dot product (+ biases). Train on observed cells with L2, by SGD (flexible) or ALS (parallel, great for implicit data). Strong baseline; fails on cold start.

13.6 Embeddings

What it is

An **embedding** is a short, dense, learned vector of numbers (typically tens to a few hundred values) that represents a discrete thing — a user ID, an item ID, a word, a category. The factor vectors p_u and q_i from MF are embeddings.

The problem it solves

Raw IDs carry no meaning: `item_5521` is not "closer" to `item_5522` than to `item_9`. The naive encoding, **one-hot** (a vector as long as the vocabulary with a single 1), is enormous for 100 million items, and every pair of one-hot vectors is equally far apart — it says nothing about which items resemble each other.

An embedding is small, and training places the vectors so that **distance reflects the training objective**: items that serve the same users end up near each other. Mechanically, an embedding layer is a lookup table (one row per ID); multiplying a one-hot vector by it selects one row. Caution: "close" means close *for that objective* — two co-watched items may still differ in price, language or quality.

Four ways to train them

Route	Idea	Best when
Matrix factorisation	Factor the interaction matrix (§13.5)	Only interactions available; strong baseline
Two-tower (contrastive)	Real (user, item) pairs must outscore sampled negatives (§13.7)	Large-scale retrieval with features
Graph embeddings	Random walks or graph neural networks over users, items, follows, co-purchases	Relationships are the signal (e.g. Pinterest's PinSage)
Pretrained content encoders	A pretrained text/image/audio model embeds the item's content	Brand-new items; text/image-heavy catalogues

This chapter focuses on vectors learned from *your own behaviour logs*. Text embeddings from pretrained language models (for semantic search and RAG) are covered in the AI book (embeddings, vector databases and RAG); the geometry is the same.

word2vec intuition → item2vec

word2vec (Mikolov et al., 2013) learns word vectors from one idea: *a word is known by the company it keeps*. The **skip-gram** version:

1. Slide a window over text. For each centre word, the words within ±w positions are its **context**.
2. Train so that σ(centre vector · true context vector) → 1 (σ = sigmoid), and for k random **negative** words σ → 0. This is **negative sampling**; negatives are drawn in proportion to frequency^0.75, giving rare words a little more weight than raw frequency would.
3. Words used in similar contexts end up with similar vectors (the famous "king − man + woman ≈ queen").

item2vec (Barkan & Koenigstein, 2016) treats a session, playlist or basket as a "sentence" and its items as "words", so items consumed together get nearby vectors — no labels needed. Airbnb (listing embeddings from users' search click sessions, Grbovic & Cheng, KDD 2018) and Spotify (tracks co-occurring in playlists) have described using this idea. Case study 18.9 builds a similar-listings carousel on it.

`code/item2vec.py` implements skip-gram with negative sampling in numpy:

```
pos = sig(np.sum(vc * W_out[o], 1))                 # centre . true context  -> want 1
ngs = sig(np.einsum("bd,bkd->bk", vc, W_out[negs]))  # centre . 5 negatives   -> want 0
g_pos = (pos - 1)[:, None]                          # gradient of -log(pos)
g_neg = ngs[:, :, None]                             # gradient of -log(1 - ngs)
grad_c = g_pos * W_out[o] + np.sum(g_neg * W_out[negs], 1)
np.add.at(W_out, o, -lr * g_pos * vc)               # np.add.at handles repeated ids
np.add.at(W_out, negs, -lr * g_neg * vc[:, None, :])
np.add.at(W_in, c, -lr * grad_c)
```

Output:

```
epoch  1  mean loss per pair 2.884
epoch 10  mean loss per pair 2.581
top-5 neighbours sharing the main genre: 79% (random would give ~17%)
item 51 (Drama) neighbours: ['113:Drama', '236:Drama', '281:Action', '321:Drama', '146:Drama']
```

The model only saw lists of item IDs, yet 79% of each item's nearest neighbours share its genre (chance: 17%). That is the core promise of embeddings: **meaningful geometry learned from behaviour**.

When not to use embeddings

- **Small catalogues** where exact scoring of everything already fits the budget — learned retrieval plus an ANN index is extra machinery.
- **Numeric inputs** (price, distance, account age) are already meaningful: scale them and use them directly.
- **Decisions that must be explained**: embedding dimensions have no human meaning, so credit and fraud systems keep interpretable features alongside.

Common mistakes

- Comparing vectors from two different trainings — each run has its own coordinate system (§13.7).
- A separate embedding for every rare ID (noisy) — use a minimum count, hashing, or content features.
- Assuming "near" means "similar in every way".

Self-check

1. Why is one-hot bad for 100 M IDs? → Enormous and every pair is equally distant — no similarity information.
2. What is an embedding layer mechanically? → A trainable lookup table; row i is ID i's vector.
3. What plays the role of "sentence" in item2vec? → A session, playlist or basket.

TL;DR

Embeddings are short learned vectors that give IDs a geometry where near = related (for the training objective). Learn them by MF, two-tower, graphs or content encoders; item2vec = word2vec on sessions.

13.7 Two-tower retrieval

What it is

A **two-tower model** (also *dual encoder*) has two separate neural networks:

- the **user (query) tower** reads user features — ID, recent watches/clicks, search terms, context such as device and hour — and outputs a vector;
- the **item (candidate) tower** reads item features — ID, category, text/image content — and outputs a vector of the **same length** (commonly 64–256 numbers).

The score is simply the **dot product** (or cosine) of the two vectors.

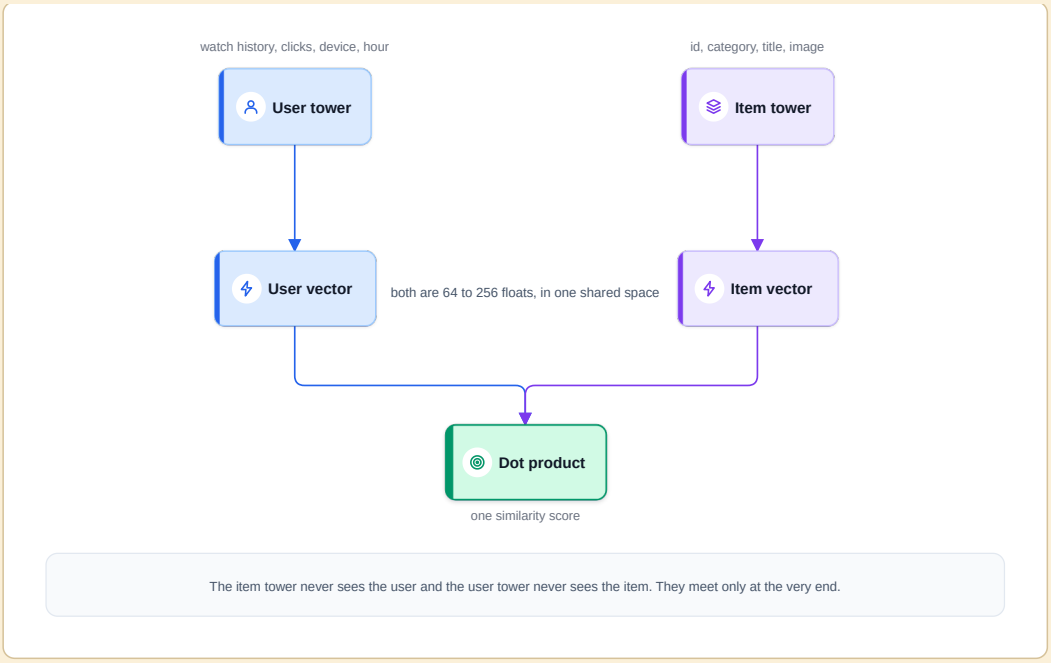


Figure — The user tower and item tower run independently and meet only at the final dot product (course diagram).

Why it matters — the key property

The two towers **never see each other's inputs**; they meet only in the final dot product. So the item tower can embed the **entire catalogue ahead of time** and load the vectors into a nearest-neighbour index; when a request arrives, the system computes just the user's vector with the **user tower** and asks the index for its nearest items — 100 million items searched in milliseconds.

A **cross model**, which reads user and item features together, can learn interactions such as "this user × this item's price" and is more accurate, but nothing can be precomputed: it would have to run once per catalogue item per request. Hence two-towers do **retrieval** and cross models do **ranking** (§13.8).

How it is trained

The logs already hold positive pairs: (user, video watched), (user, product bought).

1. Sample a batch of B positive pairs (u_j, i_j) and encode them → B user vectors, B item vectors.
2. Score every user against every item: a B × B matrix of dot products, divided by a **temperature** τ (e.g. 0.05–0.1; a small τ sharpens the softmax).
3. For row j the correct "class" is column j; the other B − 1 items are **negatives**. Apply softmax cross-entropy. This is **in-batch negative sampling** — negatives for free.
4. **Popularity correction**: a hit item appears as somebody's negative in batch after batch and gets pushed down too hard. The **logQ correction** (Yi et al., 2019) subtracts log(item's sampling probability) from its logit.
5. Optionally add a few **hard negatives** (close but wrong, e.g. shown-but-not-clicked) — useful, but too many or mislabelled ones hurt. Add them last, after the first two steps work.

Code — `code/two_tower.py`

```
class Tower(nn.Module):
    def __init__(self, n_ids, n_feats, id_dropout=0.0):
        super().__init__()
        self.id_emb = nn.Embedding(n_ids, 32)
        self.mlp = nn.Sequential(nn.Linear(32 + n_feats, 64), nn.ReLU(), nn.Linear(64, DIM))
        self.id_dropout = id_dropout   # randomly hide the id so the tower also learns from content

    def forward(self, ids, feats, use_id=True):
        e = self.id_emb(ids)
        if not use_id:
            e = torch.zeros_like(e)
        elif self.training and self.id_dropout:
            e = e * (torch.rand(len(ids), 1) > self.id_dropout).float()
        return F.normalize(self.mlp(torch.cat([e, feats], 1)), dim=1)

# training step
uv = user_tower(u, U_feat[u])                # user tower: id + genre profile of history
iv = item_tower(i, G[i])                     # item tower: id + genre multi-hot
logits = uv @ iv.T / temperature             # (B, B) in-batch scores
logits = logits - log_q[i][None, :]          # logQ popularity correction
loss = F.cross_entropy(logits, torch.arange(len(idx)))
```

40 items are held out of training entirely to act as brand-new items; for them we use the item tower with the ID part zeroed (content only).

Output:

```
epoch  1  in-batch softmax loss 6.637
epoch  5  in-batch softmax loss 6.022
epoch 15  in-batch softmax loss 5.798

warm items  recall@50: two-tower 0.649 | popularity 0.510
cold items  hit@10 among the 40 new items: 0.586 (random = 0.250)
```

- Known items: recall@50 is 0.65 vs 0.51 for "show the most popular".
- Items **never seen in training**: the content half of the item tower still puts the right new item in the top 10 of 40 for 59% of users (random: 25%) — two-towers soften **item cold start**. The `id_dropout` trick (randomly hiding the ID in training) is what makes the content-only vector meaningful.
- (Batches hold 512 pairs, so plain uniform guessing would give a loss of ln 512 ≈ 6.24. The logQ term shifts the logits, so the absolute numbers are not directly comparable with that — read the downward trend, and judge the model by the recall figures.)

How it serves

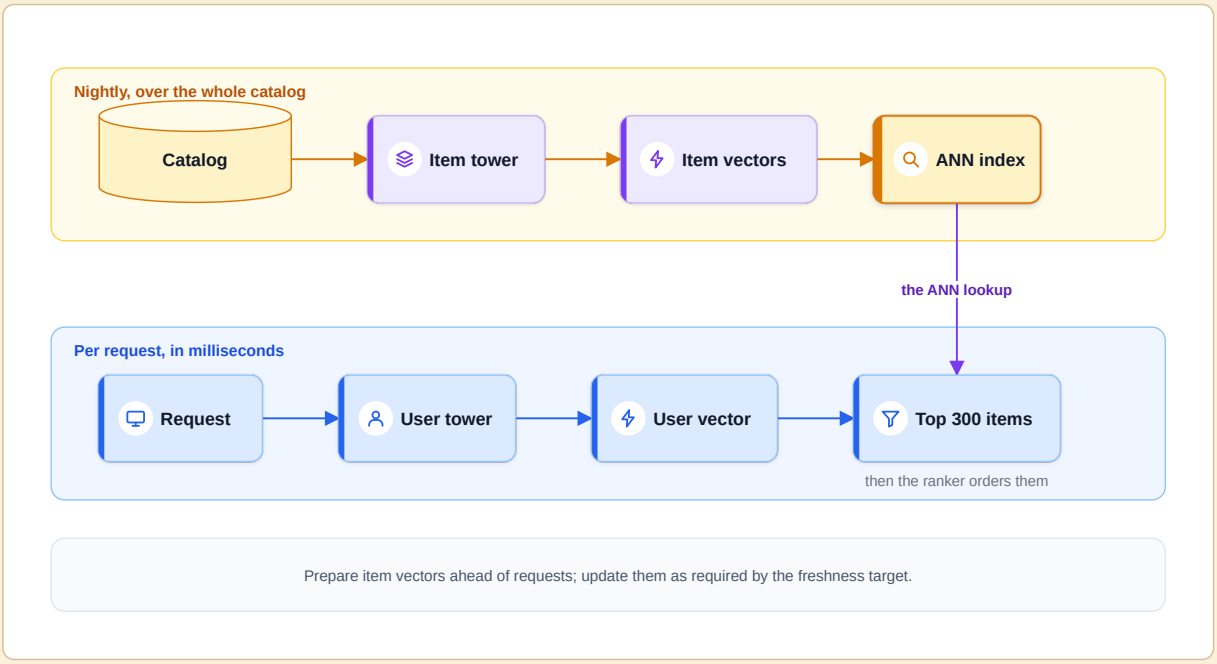


Figure — Offline item embedding and index build vs the online per-request path (course diagram).

1. **Offline (e.g. nightly):** embed the catalogue with the item tower and rebuild an **ANN index** (approximate nearest neighbour — returns *almost* the exact closest vectors, much faster; §13.9).
2. **Per request:** run the user tower (or reuse a recently cached user vector).
3. **Lookup:** the index returns a few hundred nearest items in milliseconds — the **candidates** the ranker will order.

Freshness differs by tower. Item vectors are refreshed as often as the catalogue and measured retrieval quality demand (nightly is an example, not a rule). The user side must be fresher — someone who just watched three cooking videos should get cooking candidates *now* — so the user tower reads **real-time features** from an online feature store at request time.

The versioning trap

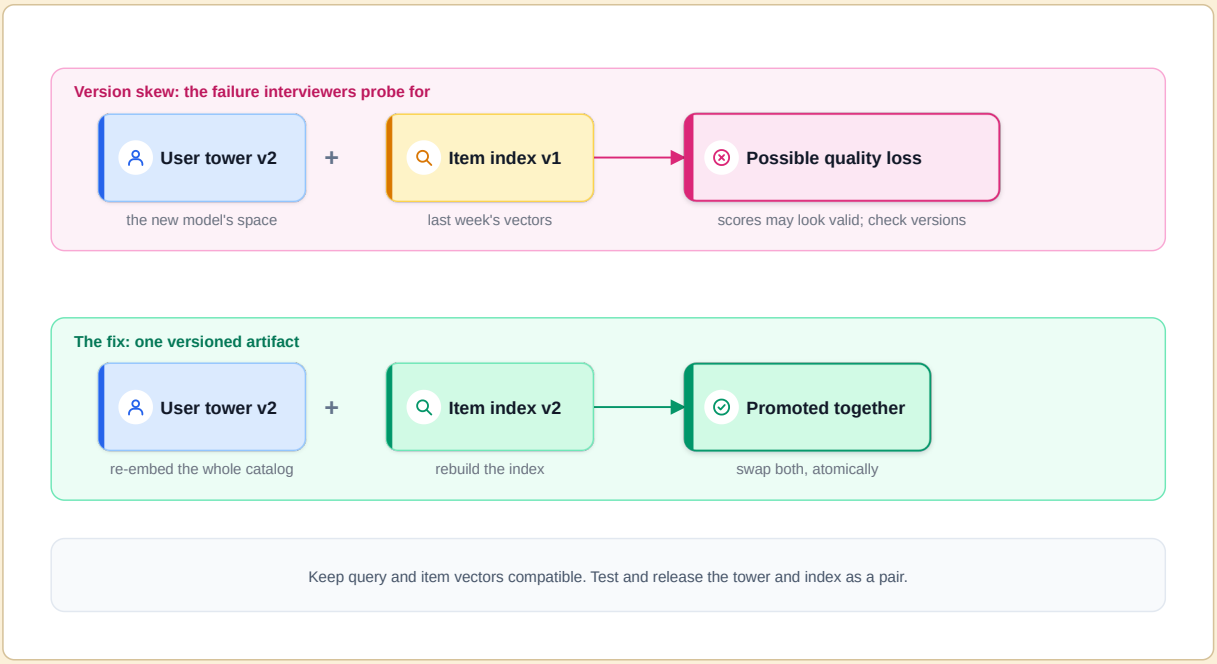


Figure — Version skew: a new user tower querying an index built by an old item tower (course diagram).

An embedding only has meaning **inside the space its model created**; retraining generally rotates and rescales that space. So upgrading is a coordinated migration: (1) re-embed the whole catalogue with the **new** item tower, (2) build a **new** index, (3) switch the new user tower and new index **together, atomically**, as a versioned pair in the model registry, keeping the old pair for rollback.

Version skew (new user tower querying an old index, or the reverse) is dangerous because nothing crashes — scores are valid numbers, latency looks normal, only quality silently drops. Defences: version tags on requests and indexes, routing to compatible pairs, offline validation of the pair, retrieval-quality monitoring during rollout. Budget the re-embedding compute too (100 M items through a new tower is not free).

Trade-offs

	Two-tower	Cross model (ranker)
User–item feature interactions	Only via final dot product	Full (any feature crosses)
Precompute items	✅	❌
Items scored per request	Whole catalogue (via ANN)	A few hundred
Role	Retrieval / candidate generation	Ranking

When not to use it

Small catalogues (score everything exactly), or when simple co-occurrence is good enough and the training + indexing + versioning machinery isn't affordable.

Common mistakes

- The same item twice in a batch becomes its own false "negative" — deduplicate or mask.
- No popularity correction → hits pushed down (or, with uniform random negatives, pushed up).
- Mixing tower and index versions; leaking the future into user features (events after the label).

Real-world examples

- **YouTube**: Covington, Adams & Sargin (2016) — a candidate-generation network trained as a huge softmax over videos and served by nearest-neighbour search, then a ranking network; Yi et al. (2019) — a two-tower model with streaming logQ correction.
- **Pinterest, LinkedIn, Airbnb** and many e-commerce sites describe embedding-based retrieval in engineering blogs.

In the interview

30-second answer ("How do you find candidates among 100 M items?"): "A two-tower model trained with in-batch negatives and logQ correction. Item vectors are computed offline into an ANN index; per request I compute a fresh user vector and pull a few hundred nearest items for the ranker. I'd watch retrieval recall and refresh cadence, and ship tower and index as a tested, versioned pair."

- *What stops the two sides drifting apart?* Ship tower + index as one versioned artifact, switched atomically, with a rollback pair.
- *Can retrieval surface an item with zero interactions?* Yes, through the item tower's content features; add a "new arrivals" slot or exploration traffic.
- *Why not one network over user and item together?* It can't be precomputed — use it as the ranker on the retrieved few hundred.

Self-check

1. What property lets item vectors be precomputed? → The item tower never sees user inputs.
2. What are in-batch negatives? → The other positives' items in the same batch, used as negatives for each user.
3. Why is version skew hard to detect? → Scores are still valid numbers; only quality degrades.

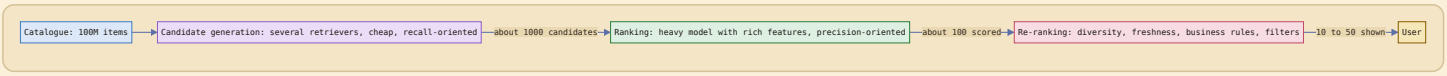
TL;DR

Two towers encode users and items independently into one space; they meet only at a dot product, so items are embedded offline into an ANN index and only the user tower runs per request. Train with in-batch negatives + logQ correction; release tower and index together.

13.8 The funnel: candidate generation → ranking → re-ranking

What it is

Large recommenders are **multi-stage funnels**. Each stage sees fewer items and can afford a heavier model.



Stage	Input → output	Model	Optimises
Candidate generation (retrieval)	$10^8 \to 10^3$	Several merged sources: two-tower + ANN, item-kNN, co-visitation, trending, followed creators	**Recall**: don't lose good items
Ranking	$10^3 \to$ ordered 10^2	Gradient-boosted trees or deep nets (Wide & Deep, DCN, DLRM) on user × item × context features; predicts P(click), watch time, P(purchase)… combined into one score	**Precision** at the top
Re-ranking	$10^2 \to 10$–50	Rules + algorithms (MMR, quotas)	Diversity, freshness, fairness, policy, de-duplication

Some systems add a **pre-ranking** ("light ranking") stage between retrieval and ranking when candidate counts are very large.

Why it matters

No single model is both cheap enough for 100 M items and rich enough for detailed feature interactions, so the funnel spends compute where it matters. Retrieval sets a **ceiling**: an item not retrieved can never be ranked.

Maximal Marginal Relevance (MMR)

A ranker scores items independently, so the top 10 are often near-duplicates (ten episodes of one show). **MMR** (Carbonell & Goldstein, 1998) re-ranks greedily:

next item = argmax over remaining j of $[\ \lambda \cdot \text{relevance}(j) - (1-\lambda) \cdot \max \text{similarity}(j, \text{already chosen})\ ]$

$\lambda = 1$ is pure relevance; smaller λ trades relevance for variety.

In the interview

> **30-second answer:** "No single model is both cheap enough for the whole catalogue and rich enough for fine feature crosses, so I'd build a funnel: several high-recall retrievers to about a thousand candidates, a heavy ranker trained on logged impressions to order them, and a re-ranker for diversity, freshness and policy. I'd track retrieval recall separately, since it caps everything after it."

- *Why several retrievers?* Each covers a different need (similar items, trending, followed creators, fresh items); merging raises recall.
- *Where does diversity go?* In re-ranking (MMR or quotas), because the ranker scores items one at a time.

Code — `code/funnel_rerank.py`

Stage 1: item-kNN retrieves 100 candidates. Stage 2: a logistic-regression ranker, trained on half the users, scores them with richer features (kNN score, a heavier implicit-ALS score, popularity, genre match with the user's profile, average star rating). Stage 3: MMR with $\lambda = 0.7$ on the ranker's top 30.

```
def mmr(cands, relevance, sim, lam=0.7, k=K):
    chosen, pool = [], list(range(len(cands)))
    while len(chosen) < k:
        best = max(pool, key=lambda j: lam * relevance[j]
                   - (1 - lam) * max((sim[cands[j], cands[c]] for c in chosen), default=0))
        chosen.append(best); pool.remove(best)
    return [cands[j] for j in chosen]
```

Output:

```
stage 1 candidate recall@100 = 0.787  (ceiling for later stages)
stage 1 only (kNN order)           recall@10 0.375  NDCG@10 0.398  diversity 0.728
stage 2 ranker (logistic regr.)    recall@10 0.393  NDCG@10 0.409  diversity 0.524
stage 3 re-rank (MMR, lambda=0.7)  recall@10 0.375  NDCG@10 0.391  diversity 0.683
ranker weights: {'knn_score': 0.18, 'als_score': 0.63, 'log_popularity': 0.36, 'genre_match': 0.62, 'avg_star': 0.07}
```

- Retrieval keeps 79% of the held-out items within 100 candidates — the most any later stage can recover.
- The ranker improves accuracy (NDCG 0.398 → 0.409) by combining signals; it leans on the heavier ALS score and genre match.
- But the ranker made lists much **less diverse** (0.728 → 0.524), piling onto the favourite genre. MMR restores diversity (0.683) at a small offline accuracy cost; online, users often reward variety — an A/B test decides.

Common mistakes

- Training the ranker only on what the *old* system showed (exposure bias) — log impressions and some randomised exposure.
- Evaluating the ranker but not retrieval recall.
- One objective (clicks) when the business wants long-term satisfaction — use a multi-task ranker (click, watch time, like, "not interested") with tuned weights.

TL;DR

Funnel = cheap high-recall retrieval ($10^8 \to 10^3$) → heavy precise ranking ($10^3 \to 10^2$) → re-ranking for diversity/business rules ($\to 10$). Retrieval caps what ranking can achieve; MMR buys variety for a little relevance.

13.9 Approximate nearest neighbour (ANN) search

What it is

k-nearest-neighbour search returns the k item vectors most similar to a query q. **Exact** (brute-force) search compares q with all N vectors of d dimensions: O(N·d) per query. **Approximate** nearest neighbour (ANN) search returns *almost* the same top-k while looking at a small fraction.

Why it matters

10^8 items × 128 dimensions ≈ $1.3 \cdot 10^{10}$ multiply-adds per query — far too slow at thousands of requests per second. ANN trades a little **recall** (fraction of the true top-k found) for large speed-ups (often 10–1000×).

How the main families work

1. IVF — inverted file index (clustering).

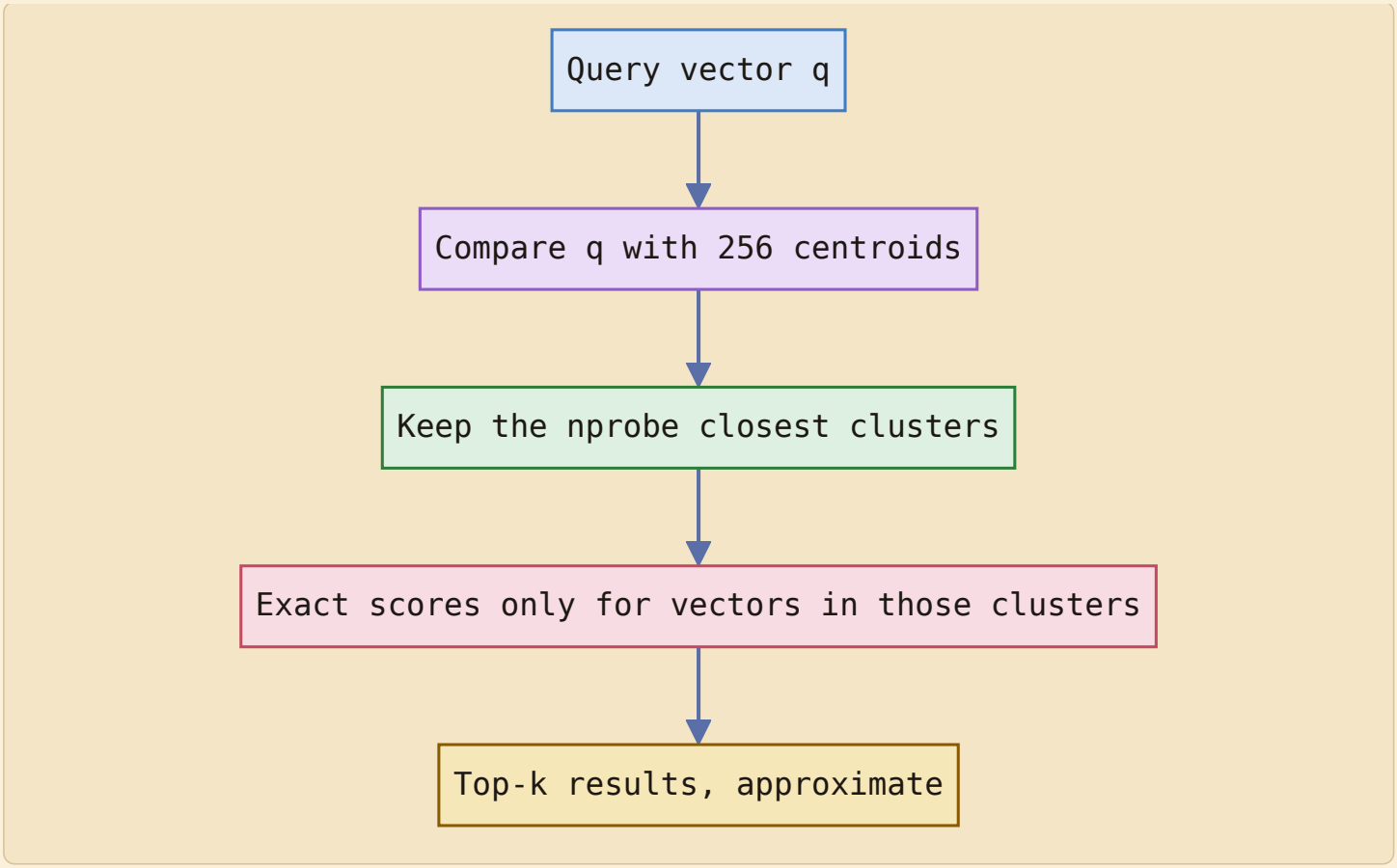
1. Offline: run k-means on the vectors to get `nlist` centroids; store each vector in the list of its nearest centroid.
2. Query: find the `nprobe` centroids nearest to q; scan only those lists exactly.
3. Knob: larger `nprobe` → higher recall, slower.

2. HNSW — Hierarchical Navigable Small World graphs (Malkov & Yashunin, 2016/2018).

1. Each vector is a node linked to about M close neighbours; upper layers hold few nodes ("highways"), the bottom layer holds all — like zooming from a country map to a street map.
2. Query: start at the top and greedily hop to whichever neighbour is closer to q; when none is closer, drop a layer; at the bottom keep a candidate list of size `ef_search` and return the best k.
3. Knobs: M, `ef_construction`, `ef_search` (recall vs speed). Excellent recall/latency and incremental inserts, but more memory than IVF.

3. Product quantisation (PQ) compresses vectors (e.g. 128 floats = 512 bytes → 16 bytes) by splitting each into sub-vectors and replacing each sub-vector by the ID of its nearest sub-centroid. Distances are computed from small lookup tables. Often combined: **IVF-PQ**.

4. Tree forests (e.g. Spotify's Annoy): random hyperplane splits build several trees; search a few.

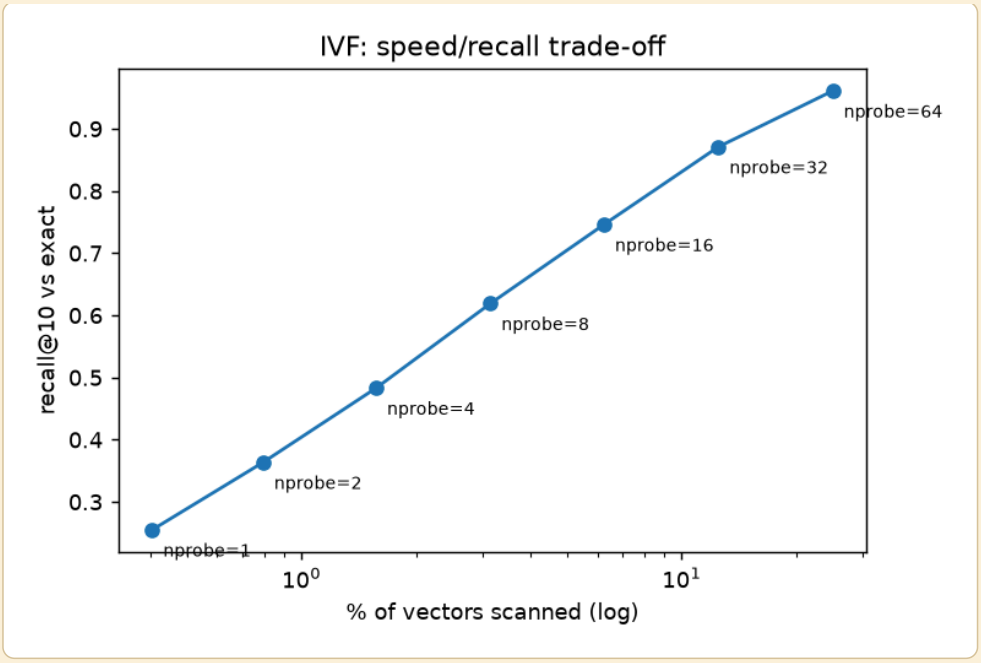


Code — `code/ann_demo.py` (brute force vs IVF)

```
exact = np.argsort(-(queries @ X.T), axis=1)[:, :K]          # brute force: score every vector
C, _ = kmeans(X[rng.choice(N, 20_000, replace=False)], 256)  # train centroids on a sample
assign = np.argmax(X @ C.T, axis=1)
lists = [np.flatnonzero(assign == c) for c in range(256)]    # the "inverted file"
for q in queries:
    probe = np.argsort(-(C @ q))[:nprobe]                    # closest clusters
    cand = np.concatenate([lists[c] for c in probe])
    top = cand[np.argsort(-(X[cand] @ q))[:K]]
```

Output (100,000 unit vectors, 32 dimensions; timings depend on the machine):

```
brute force: scans 100% of 100,000 vectors, 6.13 ms/query, recall@10 = 1.000
IVF build (k-means, 256 lists): 0.1 s
IVF nprobe= 1: scans  0.41%, 0.06 ms/query, recall@10 = 0.255
IVF nprobe= 2: scans  0.79%, 0.08 ms/query, recall@10 = 0.363
IVF nprobe= 4: scans  1.57%, 0.14 ms/query, recall@10 = 0.483
IVF nprobe= 8: scans  3.14%, 0.27 ms/query, recall@10 = 0.619
IVF nprobe=16: scans  6.25%, 0.56 ms/query, recall@10 = 0.746
IVF nprobe=32: scans 12.49%, 1.21 ms/query, recall@10 = 0.871
IVF nprobe=64: scans 24.96%, 2.56 ms/query, recall@10 = 0.961
```



The curve is the whole story of ANN: scanning 12% of the data gives 87% recall at ~5× speed; 25% gives 96%. (This data is deliberately hard — overlapping clusters; real embeddings usually reach high recall at smaller `nprobe`, and production libraries are far faster than this numpy loop.)

Trade-offs

Method	Recall/speed	Memory	Build	Updates
Brute force	Exact, O(N)	Vectors only	None	Trivial
IVF	Good, tune `nprobe`	Low	k-means (fast)	Add to lists; re-train centroids occasionally
IVF-PQ	Good at huge scale	Very low (compressed)	Slower	As IVF
HNSW	Excellent	Higher (graph links)	Slowest	Incremental inserts; deletes are awkward

Libraries: **FAISS** (Meta), **ScaNN** (Google), **hnswlib**, **Annoy/Voyager** (Spotify), plus vector databases (Milvus, Weaviate, Qdrant, pgvector, OpenSearch).

When not to use it

Up to around a million vectors, a batched brute-force matrix multiply (especially on a GPU) may already meet the latency budget — exact and simple. Measure first.

Common mistakes

- A cosine index fed un-normalised vectors (or vice versa).
- Reporting latency without recall@k measured against exact search.
- Filtering (country, availability) *after* ANN and ending with too few results — use filtered search or over-retrieve.

Self-check

1. What does `nprobe` control? → How many clusters are scanned; recall ↑, speed ↓.
2. Why does HNSW have upper layers? → Long-range "express" hops to reach the right region quickly.
3. What does PQ buy you? → Much smaller memory per vector, with approximate distances.

TL;DR

Exact k-NN is O(N) per query; ANN (IVF clustering, HNSW graphs, PQ compression) scans a fraction for a small recall loss. Always report recall with latency.

13.10 Cold start

What it is

Cold start = making recommendations with little or no history.

- **New user**: we know nothing about their taste.
- **New item**: nobody has interacted with it yet, so CF/MF have no vector.
- **New system**: no data at all.

How to handle it

Case	Strategies
New user	Popular/trending items by region or device; short onboarding ("pick 3 genres"); context (location, time, referrer); fast updates from the first clicks (session-based models)
New item	Content features in the item tower (59% vs 25% random in `two_tower.py`); creator/brand signals; **exploration** — a small controlled share of traffic (bandits, Chapter 14) and a "new arrivals" slot
New system	Rules + popularity + content similarity, then collect data

Exploration is essential: if only proven items are shown, new items never get to prove themselves. Multi-armed bandits (ε-greedy, UCB, Thompson sampling) balance exploiting proven items with exploring newcomers.

Common mistakes

- Letting cold items sink forever because their score rests on zero data; asking for long onboarding questionnaires few people finish.

TL;DR

New users: popularity + onboarding + context + fast session updates. New items: content features + controlled exploration. Never let "no data" mean "never shown".

13.11 Evaluation

What it is

Recommenders are evaluated **offline** (on held-out historical data, fast and cheap) and **online** (A/B tests on real users, the final judge).

Offline accuracy metrics (top-K)

For one user, with a ranked list `recs` and a set of `relevant` held-out items:

Metric	Formula	Meaning
Precision@k	(# relevant in top k) / k	How much of what we showed was good
Recall@k	(# relevant in top k) / (# relevant)	How much of the good stuff we found
Hit rate@k	1 if any relevant item is in top k, else 0	Did we get at least one right? (common with leave-one-out)
AP@k (average precision)	(sum of precision@i over the ranks i that hold a hit) / min(#relevant, k)	Rewards putting hits early
MAP@k	mean AP@k over users	
NDCG@k	DCG / ideal DCG, where DCG = Σ_i rel_i / $\log_2(i + 1)$ (i = rank; a common variant uses 2^rel_i − 1 as the gain)	Position-discounted gain, normalised to [0, 1]; supports graded relevance
MRR	mean of 1 / (rank of first hit)	Only the first hit matters (search-like)
RMSE / MAE	error on predicted ratings	Only for the rating-prediction framing

Worked example (`implicit_eval.py`): recs = A B C D E, relevant = {A, C, F}, k = 5.

- Hits at ranks 1 (A) and 3 (C). Precision@5 = 2/5 = **0.400**. Recall@5 = 2/3 = **0.667**. Hit@5 = **1**.
- AP@5 = (precision@1 + precision@3) / min(3, 5) = (1/1 + 2/3) / 3 = **0.556**.
- DCG = 1/$\log_2$2 + 1/$\log_2$4 = 1 + 0.5 = 1.5. Ideal (3 hits at ranks 1–3) = 1 + 0.631 + 0.5 = 2.131. NDCG@5 = 1.5 / 2.131 = **0.704**.

```python
def average_precision_at_k(recs, relevant, k=K):
    hits, total = 0, 0.0
    for rank, item in enumerate(recs[:k], start=1):
        if item in relevant:
            hits += 1
            total += hits / rank          # precision at each position where we hit
    return total / min(len(relevant), k)

def ndcg_at_k(recs, relevant, k=K):
    dcg = sum(1 / np.log2(rank + 1) for rank, it in enumerate(recs[:k], 1) if it in relevant)
    ideal = sum(1 / np.log2(rank + 1) for rank in range(1, min(len(relevant), k) + 1))
    return dcg / ideal
```

Beyond-accuracy metrics

- **Catalogue coverage**: fraction of all items shown to *anyone* (low = only the hits).
- **Intra-list diversity**: 1 − average pairwise similarity within one list (here: genre cosine).
- **Novelty**: average popularity of recommended items (lower = more novel); **serendipity** = relevant *and* unexpected.
- **Fairness / exposure**: how impressions spread across creators or sellers.

Code — comparing three models on implicit data (`code/implicit_eval.py`)

Each user's interactions are binarised to "watched"; 5 random watched items per user are hidden as the **test** set, and models must rank them among all unseen items. The ALS settings are chosen on a separate **validation** split (5 more items per user hidden from the training data), never on the test items.

Output:

```
ALS chosen on validation split: alpha=10, lambda=50 (validation NDCG@10 0.331)

model              P@10   R@10   MAP@10 NDCG@10 HR@10  cover  divers  avgPop
Popularity         0.107  0.215  0.104  0.197  0.708  0.07   0.741   219
Item-kNN (cosine)  0.184  0.367  0.256  0.388  0.882  0.34   0.733   186
Implicit ALS       0.195  0.390  0.282  0.416  0.893  0.42   0.591   154
(average item popularity in the catalogue = 46 interactions)
```

- **Popularity** is a strong baseline (hit rate 71%) — always include it. But it covers 7% of the catalogue, with items ~5× more popular than average.
- **Implicit ALS** wins every accuracy metric, covers 42% and recommends less popular items — but its lists are less genre-diverse. Accuracy, coverage, novelty and diversity pull apart; choose deliberately.
- The validation sweep ($\alpha \in \{5, 10, 20\}$, $\lambda \in \{10, 50, 100\}$, $k = 16$) picked $\alpha = 10$, $\lambda = 50$; the model was then refit on all training data and scored once on the test items.

Offline evaluation pitfalls

- **Random splits leak the future.** Use a **time-based split** (train before day T, test after), matching real use.
- **Sampled metrics** (true item vs only 100 random negatives) inflate scores and can even flip which model looks best (Krichene & Rendle, 2020). Rank against the full catalogue when possible.
- **Logs are biased by the old system** (feedback only on what it showed). **Inverse propensity scoring (IPS)** reweights each logged example by 1 / (probability the old system showed it), removing that bias at the cost of higher variance.

Online evaluation

- **A/B test**: randomly split users; compare business metrics (watch time, retention, purchases) plus guardrails (latency, complaints, diversity); run long enough for novelty effects to fade.
- **Interleaving**: merge two rankers' lists and see whose items get clicked — detects a preference with far less traffic; Netflix uses it to pre-screen ideas.
- Offline metrics are a filter, not the verdict.

In the interview

30-second answer: "Offline: a time-based split, recall@k for retrieval and NDCG@k or MAP@k for ranking over the full catalogue, plus coverage, diversity and novelty, versus a popularity baseline. Then an A/B test on the real objective, such as retention, with latency and diversity guardrails."

- *Precision@k vs recall@k?* Precision = quality of what's shown; recall = how much of the relevant set was found. Retrieval cares about recall, the final list about precision/NDCG.
- *Why NDCG over precision?* It rewards putting good items higher and supports graded relevance.
- *Why can offline and online disagree?* Logs reflect the old policy and ignore presentation and long-term effects.

Self-check

1. Relevant = {X}, X at rank 4. NDCG@5? MRR? → $1/\log_2 5 \approx 0.431$ (ideal = 1); MRR = 1/4 = 0.25.
2. Why is a random train/test split risky? → The model sees the future (later interactions) during training.
3. Model A: higher NDCG, 5% coverage. Model B: slightly lower NDCG, 40% coverage. Which? → Depends on goals; usually test B online — low coverage starves the long tail and feeds the popularity loop.

TL;DR

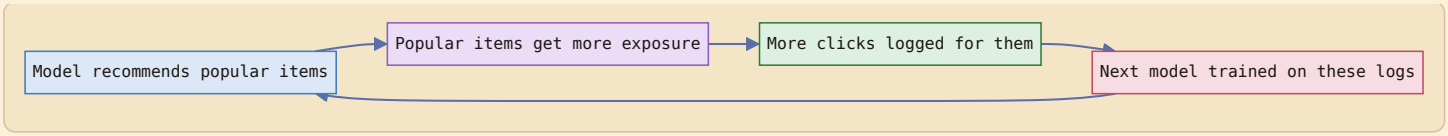
Offline: precision/recall/hit rate@k, MAP, NDCG on a time-based split, full-catalogue ranking, plus coverage/diversity/novelty and a popularity baseline. Online: A/B tests (or interleaving) on the true objective.

13.12 Popularity bias and feedback loops

What it is

Popularity bias: models over-recommend already-popular items. Causes: the data (more interactions → more gradient updates and co-occurrences) and the model (dot products favour long vectors).

Feedback loop: popular items are shown → get more clicks → the next model, trained on those clicks, rates them even higher → shows them more. Long-tail items rarely get shown, so never collect the data to prove themselves. Result: less diversity, "filter bubbles", struggling niche creators.



How to counter it

1. **Measure**: average popularity of recommendations, coverage, Gini coefficient of exposure (0 = perfectly even, 1 = all to one item).
2. **Correct in training**: logQ (§13.7), down-weighting popular positives, IPS.
3. **Explore**: a little bandit traffic for new and long-tail items.
4. **Re-rank**: MMR, novelty boosts, exposure quotas for new creators.
5. **Log impressions** so training can tell "not shown" from "shown and ignored".

Common mistakes

- Celebrating an offline win that comes only from recommending more popular items (test sets are popularity-biased too).
- Removing popularity entirely — it is real information; aim for calibration, not elimination.

TL;DR

Popular items get more data, so models over-recommend them and the loop reinforces itself. Measure it, correct it in training (logQ, IPS), explore, and re-rank for diversity.

13.13 Real-world systems

Amazon's item-to-item CF is covered in §13.4; the others are collected here.

Company	Publicly described approach	Ideas to remember
Netflix	Netflix Prize (2006–2009, $1M, won by BellKor's Pragmatic Chaos); personalised rows and ranking; artwork chosen per member with contextual bandits; recommender influences ~80% of hours streamed (2015)	Moved from star prediction to ranking on implicit signals; heavy A/B testing plus interleaving
YouTube	Deep candidate generation + a ranking net predicting expected watch time (Covington et al., 2016); two-tower with streaming logQ correction (Yi et al., 2019); multi-task ranking (Zhao et al., 2019)	Two-stage funnel; watch time rather than clicks; an "example age" feature for freshness
Spotify	Discover Weekly (2015) blends collaborative filtering on listening and playlists, NLP on text about music, and audio models; bandits on home shelves; open-sourced Annoy and Voyager	Playlists as co-occurrence data; skips and saves as implicit feedback
Pinterest	PinSage (Ying et al., 2018): graph convolutional network over billions of pins and boards	The graph-embedding route

---

Chapter TL;DR

- **Frame** it as top-K ranking on a meaningful outcome; implicit feedback dominates, and unobserved ≠ negative.
- **Content-based** covers new items; **collaborative** finds quality and surprise but needs history. Hybrids win.
- **Matrix factorisation** ($\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$, SGD or ALS; weighted ALS for implicit) is the strong baseline; its vectors are **embeddings**.
- **Two-tower** retrieval: items precomputed into an ANN index (IVF, HNSW, PQ), only the user tower runs per request; ship tower and index as one version.
- **Funnel**: retrieval (recall ceiling) → ranking (precision) → re-ranking (diversity, rules).
- **Evaluate** on a time-based split over the full catalogue (recall, NDCG, MAP + coverage, diversity, novelty) against a popularity baseline, then A/B test. Handle cold start with content and exploration; counter popularity bias's feedback loop.

Chapter 14 — Reinforcement Learning

Supervised learning learns from an answer key; unsupervised learning finds structure without one. **Reinforcement learning (RL)** is the third way: an agent learns *what to do* by acting in a world, seeing what happens, and collecting rewards — no answer key, just a score that arrives, often late. It is how AlphaGo learned to beat a world champion at Go, how robots learn to walk in simulation, how some recommenders decide what to try next, and how chat models such as ChatGPT and Claude were tuned to be more helpful (RLHF).

What you'll learn

1. The core vocabulary: agent, environment, state, action, reward, episode, policy.
2. Markov decision processes, the return and the discount factor — with worked numbers on a treasure-hunt grid.
3. Value functions V and Q, and the Bellman equations.
4. Exploration vs exploitation: multi-armed bandits with ε-greedy, UCB and Thompson sampling (implemented).
5. Dynamic programming when the rules are known: policy evaluation, policy iteration, value iteration (implemented on a gridworld and a FrozenLake-style lake).
6. Learning from experience: Monte Carlo vs temporal-difference (TD) learning.
7. Q-learning and SARSA from scratch in numpy — no Gym needed — and why they behave differently near a cliff.
8. Deep RL: DQN with a replay buffer and a target network (implemented in PyTorch).
9. Policy gradients: REINFORCE, baselines and actor-critic (implemented), plus PPO in one paragraph.
10. RLHF — the bridge from RL to large language models.
11. Real-world uses (AlphaGo, robotics, recommender bandits, ads) and pitfalls (reward hacking, sample inefficiency).

All code lives in `code/` and uses only numpy, matplotlib and PyTorch. `code/gridworld.py` is a tiny environment written for this chapter (FrozenLake-style lake, treasure grid, cliff walk); its action numbering (0 = left, 1 = down, 2 = right, 3 = up) matches the popular Gymnasium library so you can move to it later. Outputs below are real.

14.1 What reinforcement learning is

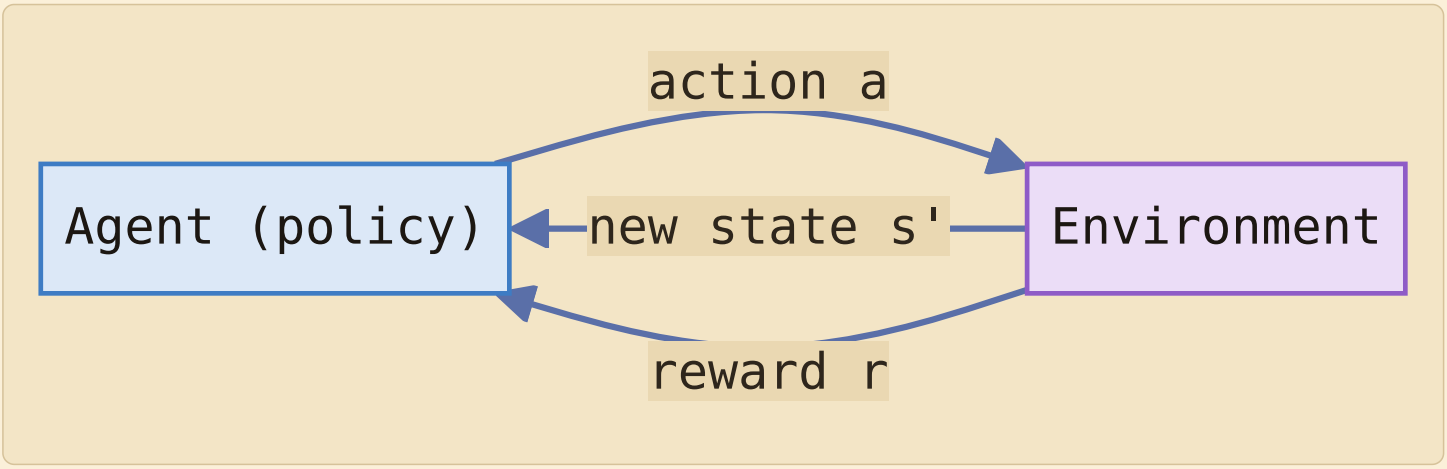
What it is

In RL a learner called the **agent** repeatedly:

1. looks at the current **state** of its **environment**,
2. picks an **action**,
3. receives a **reward** (a number: positive = good, negative = a penalty) and finds itself in a new state.

Its goal is not to get the next reward, but to collect **as much total reward as possible over time**. The strategy it learns — "in this situation, do that" — is called a **policy**.

Term	Meaning	Treasure-hunt example
Agent	The learner and decision-maker	A robot, a game player, a program
Environment	Everything the agent interacts with; it reacts to actions	The house, the maze, the game board
State (s)	A description of the current situation	The robot's position on the map
Action (a)	A choice available to the agent	Move up / down / left / right
Reward (r)	Immediate numeric feedback after an action	+10 treasure, −5 trap, −1 per step
Episode	One run from a start to an end (goal, failure or time limit)	One attempt at the maze
Policy (π)	The agent's rule for choosing actions in each state	"In the hallway, go left"
Goal	Maximise total (discounted) reward	Reach the treasure by the best route



Trial and error — the intuition

Think of learning to ride a bicycle. Nobody hands you the correct muscle movements. You try, wobble, fall, adjust, and gradually the attempts that kept you upright become habit. RL agents learn the same way: **act → observe the outcome → receive feedback → shift future choices** toward what paid off.

Everyday framings of the same loop:

- **A robot looking for a cookie** in a house: small rewards for getting closer, penalties for bumping into furniture; after many attempts it settles on the route that earned the most.
- **Tic-tac-toe**: early games are random and mostly lost (negative reward); the occasional win (positive reward) gradually tilts the agent toward the moves that appeared in winning games.
- **A grid game** (right = +5, left = −2) or **a snack chooser** (fruit rewarded, candy penalised): after sampling each option a few times, the agent comes to prefer the rewarded one, even if it looked unattractive before trying it.

How RL differs from other learning

	Supervised	Unsupervised	Reinforcement
Data	Inputs with correct labels	Inputs only	Experience generated by the agent's own actions
Feedback	"The right answer was X"	None	"That was worth +3" — how good, not what was right
Timing	Immediate, per example	—	Often **delayed** (the win comes 40 moves later)
Data distribution	Fixed dataset	Fixed dataset	**Depends on the policy** — change behaviour, change the data
Key challenge	Generalisation	Finding structure	Exploration and **credit assignment** (which earlier action earned the reward?)

Why it matters

RL fits problems that are **sequential decisions under uncertainty**, where actions change what happens next: games, robot control, resource allocation (cooling, bidding, inventory), dialogue, and sequential recommendation. It can discover strategies humans never wrote down (AlphaGo's famous "move 37").

When not to use it

- If you have labelled examples of the right action, supervised learning (behaviour cloning) is simpler and far more data-efficient.
- If each decision does not affect future states, a **contextual bandit** (§14.4) is enough.
- If you cannot simulate the environment and real trials are expensive or dangerous (medicine, finance), pure online RL is risky; look at offline RL or stay supervised.

TL;DR

RL = an agent learns a policy by trial and error, choosing actions in states to maximise long-run reward. Unlike supervised learning, feedback is a score (not the right answer), often delayed, and the agent's own behaviour decides what data it sees.

14.2 Markov decision processes, returns and discounting

What it is

A **Markov decision process (MDP)** is the mathematical model of an RL problem: a tuple (S, A, P, R, γ).

- **S**: the set of states. **A**: the set of actions.
- **P(s' | s, a)**: the **transition probability** of landing in s' after doing a in s. (Deterministic worlds put probability 1 on one outcome; a slippery lake spreads it.)
- **R(s, a, s')**: the reward for that transition.
- **γ** (gamma, $0 \le \gamma \le 1$): the **discount factor**.

The **Markov property**: the next state and reward depend only on the current state and action, not on the full history. "The present summarises the past." If your state description misses something important (e.g. a car's speed, not just its position), the problem is not Markov in that state — add the missing information (or stack recent observations, as DQN stacks 4 Atari frames).

Return and discount

The **return** G_t is the total reward from time t onward, with later rewards discounted:

$$G_t = r_{t+1} + \gamma \cdot r_{t+2} + \gamma^2 \cdot r_{t+3} + \ldots = \Sigma_k \, \gamma^k \cdot r_{t+k+1}$$

Why discount?

- **Mathematical**: keeps sums finite for tasks that never end.
- **Preference**: a reward now is worth more than the same reward later (like interest rates); future rewards are also more uncertain.
- $\gamma = 0$ → completely short-sighted (only the next reward counts). $\gamma \to 1$ → far-sighted. With $\gamma = 0.9$ a reward 10 steps away counts $0.9^{10} \approx 0.35$ of its face value; a rough "horizon" is $1/(1-\gamma) = 10$ steps.

A useful identity: $G_t = r_{t+1} + \gamma \cdot G_{t+1}$ — the return is "this reward plus the discounted return from the next step". Everything in §14.3 builds on it.

Episodic tasks end (a game, a maze run); **continuing** tasks never do (a thermostat, a stock of servers) and need $\gamma < 1$ or an average-reward formulation.

Worked example — the treasure grid

Here is the paper game "draw a grid, place a treasure and some traps, and score different routes", turned into code (`code/treasure_grid.py`):

```
S.-..      S = start, G = treasure (+10, ends the episode)
..-..      - = trap (-5, you keep going)
...-.      every other move costs -1
.-...      (walking into the edge = you stay put, still -1)
....G
```

```python
def play(env, actions, gamma=0.9):
    s = env.reset()
    rewards = []
    for a in actions:
        s, r, done, _ = env.step(a)
        rewards.append(r)
        if done:
            break
    G = sum(gamma ** t * r for t, r in enumerate(rewards))   # discounted return
    return rewards, sum(rewards), G, done
```

Output:

```
A: right along the top, then down    rewards=[-1, -5, -1, -1, -1, -1, -1, 10]  total=-1  discounted G(gamma=0.9)=-4.03  reached=True
B: down the left edge, then right    rewards=[-1, -1, -1, -1, -1, -1, -1, 10]  total=+3  discounted G(gamma=0.9)=-0.43  reached=True
C: zig-zag through the middle        rewards=[-1, -1, -1, -1, -1, -1, -1, 10]  total=+3  discounted G(gamma=0.9)=-0.43  reached=True

random tries that reached the treasure within 30 moves: 307 / 2000
best of 2,000 random tries: total=-1  moves=RDDRRDRD
```

Checking path B by hand: seven -1 steps, then $+10$ on step 8. $G = -(1 + 0.9 + 0.9^2 + \dots + 0.9^6) + 10 \cdot 0.9^7 = -(1 - 0.9^7)/(1 - 0.9) + 10 \cdot 0.478 = -5.22 + 4.78 = \mathbf{-0.43}$.

Lessons:

- Path A walks through a trap and scores worse; B and C avoid all traps and tie on the best possible total, +3 (8 moves is the minimum).
- Even with discounting the ranking is the same here, but the discounted value of reaching +10 eight steps away is only 4.78 — distant rewards count less.
- **Blind trial and error is weak**: 2,000 random attempts never found a +3 route, and only 15% reached the treasure at all. The agent needs to *learn from* its trials — remember which actions paid off and prefer them. That is what value functions and Q-learning do.

Self-check

1. $\gamma = 0.5$ and the next three rewards are 1, 1, 1 (then the episode ends). G_t? → $1 + 0.5 + 0.25 = \mathbf{1.75}$.
2. Why is a car's position alone not a Markov state? → The next position also depends on speed, which the state omits.
3. Rough horizon for $\gamma = 0.99$? → $1/(1 - 0.99) = \mathbf{100}$ steps.

TL;DR

An MDP = states, actions, transition probabilities, rewards, discount γ. The return $G_t = \Sigma\, \gamma^k\, r_{t+k+1} = r_{t+1} + \gamma G_{t+1}$. γ trades short-term for long-term reward. Random exploration alone rarely finds the best plan.

14.3 Policies, value functions and the Bellman equations

What it is

- A **policy** π tells the agent what to do. **Deterministic**: $\pi(s) = a$. **Stochastic**: $\pi(a \mid s)$ = probability of choosing a in s.
- The **state-value function** $V\pi(s)$ = expected return starting in s and then following π. "How good is it to be here, if I keep behaving like this?"
- The **action-value function** $Q\pi(s, a)$ = expected return if you take action a in s, then follow π. "How good is doing *this* here?"
- The **optimal** functions $V^*(s)$ and $Q^*(s, a)$ are the best achievable by any policy. Once you know Q^*, acting optimally is trivial: $\pi(s) = \mathit{argmax}_a\ Q(s, a)$** (be *greedy* with respect to Q^*).

Why Q is so useful: choosing the best action from V requires knowing the transition model (to look one step ahead), but choosing from Q does not — just compare numbers. That is why model-free methods (Q-learning, DQN) learn Q.

The Bellman equations

Plug the identity $G_t = r_{t+1} + \gamma G_{t+1}$ into the definitions:

Equation	Formula	Used by
Bellman expectation (for a given π)	$V^{\pi}(s) = \Sigma_a\ \pi(a\|s)\ \Sigma_{s'}\ P(s'\|s,a)\ [R(s,a,s') + \gamma V^{\pi}(s')]$	Policy evaluation
Bellman optimality for V	$V^*(s) = \max_a\ \Sigma_{s'}\ P(s'\|s,a)\ [R(s,a,s') + \gamma V^*(s')]$	Value iteration
Bellman optimality for Q	$Q^*(s,a) = \Sigma_{s'}\ P(s'\|s,a)\ [R(s,a,s') + \gamma\ \max_{a'}\ Q^*(s',a')]$	Q-learning, DQN

(V of a terminal state is 0 — nothing more can be earned after the episode ends.)

In words: *the value of where you are = the reward you get now + the discounted value of where you land* (averaged over what might happen, and, for the optimal version, choosing the best action).

Worked numbers on the treasure grid (γ = 0.9, deterministic):

- The cell just above G: move down → +10 and the episode ends. V* = **10**.
- One more cell up: V* = −1 + 0.9·10 = **8.0**.
- One more: −1 + 0.9·8 = **6.2**; then −1 + 0.9·6.2 = **4.58**; …; at the start, **−0.43** — exactly path B's discounted return.

Self-check

1. What is the difference between V(s) and Q(s, a)? → V averages over the policy's action choice; Q fixes the first action.
2. If you know Q*, how do you act? → Pick $\text{argmax}_a\ Q^*(s, a)$.
3. Compute V* for a cell two steps from G with reward −1 per step, γ = 0.5. → −1 + 0.5·10 = 4.

TL;DR

Policy = behaviour; V = how good a state is; Q = how good an action in a state is. Bellman: value now = reward now + γ × value next. Greedy on Q* is optimal.

14.4 Exploration vs exploitation — multi-armed bandits

What it is

The **exploration–exploitation dilemma**: should you **exploit** the best option you know, or **explore** others that might be better? Always exploiting can lock you into a mediocre choice forever; always exploring wastes reward.

The simplest setting that isolates it is the **multi-armed bandit**: k slot machines ("one-armed bandits"), each paying off with an unknown probability. There is only one state — no consequences beyond the immediate reward — so it is RL with the sequential part removed. Real versions: which of 10 ad banners, headlines or thumbnails to show; which new item to give traffic to.

Regret = reward you could have collected by always pulling the best arm, minus what you actually got. Good strategies make regret grow slowly (logarithmically in time).

The strategies

1. **Greedy**: try each arm once, then always pull the arm with the highest average so far. Can get stuck on an arm that got lucky early.
2. **ε-greedy**: with probability ε (e.g. 0.1) pull a random arm, otherwise the best-looking one. Simple and robust, but keeps exploring uselessly at rate ε forever (decay ε over time to fix).
3. **UCB (Upper Confidence Bound)** — *optimism in the face of uncertainty*: pick $\text{argmax}_a\ [\ Q(a) + c\cdot\sqrt{(\ln t\ /\ N_a)}\ ]$, where N_a = pulls of arm a so far. Rarely tried arms get a big bonus; the bonus shrinks as evidence accumulates. UCB1 uses c = √2 (from theory, for rewards in [0, 1]).
4. **Thompson sampling** — *probability matching*: keep a belief distribution over each arm's payoff (for 0/1 rewards, a Beta(wins + 1, losses + 1) distribution), draw one sample per arm, pull the arm whose sample is highest. Arms that *might* be best get tried in proportion to that chance.

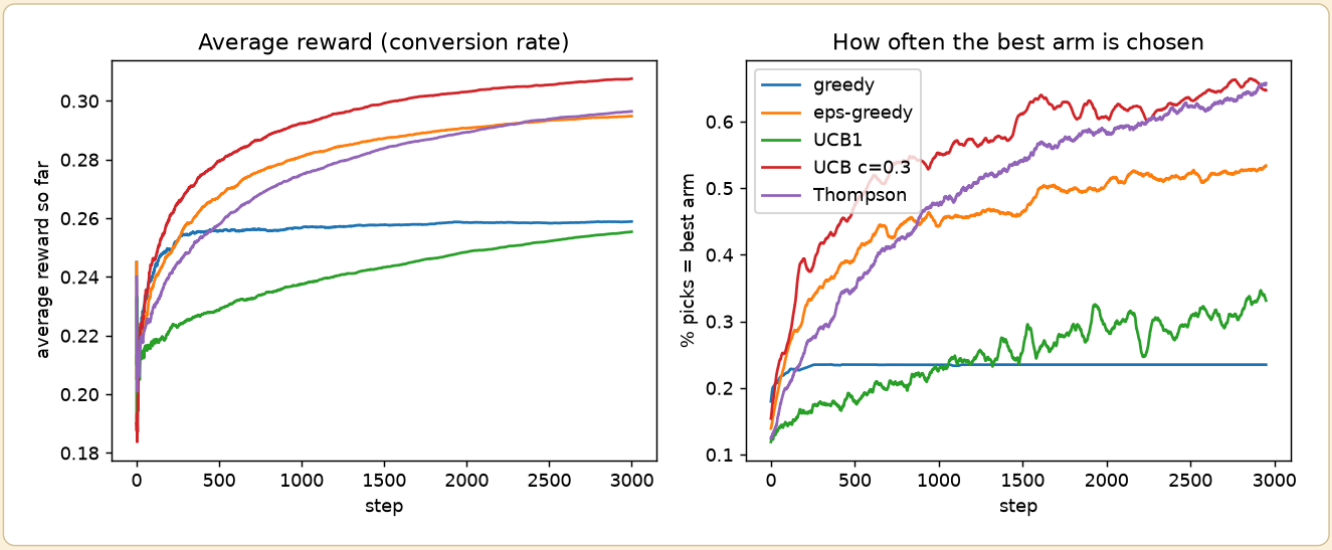
Code — `code/bandits.py`

10 arms with conversion rates drawn from 5–35%, 3,000 pulls, averaged over 200 random problems.

```
if policy == "eps-greedy":
    a = int(rng.integers(N_ARMS)) if rng.random() < eps else int(np.argmax(q))
elif policy.startswith("UCB"):
    c = np.sqrt(2) if policy == "UCB1" else 0.3
    a = t if t < N_ARMS else int(np.argmax(q + c * np.sqrt(np.log(t + 1) / n)))
elif policy == "Thompson":
    a = int(np.argmax(rng.beta(wins, losses)))           # sample a belief, act on it
r = float(rng.random() < true_p[a])                      # conversion (1) or not (0)
n[a] += 1
q[a] += (r - q[a]) / n[a]                                # incremental mean
```

Output:

```
policy      avg reward  % optimal arm (last 500)  total regret
greedy      0.2589       23.5%                197.0
eps-greedy  0.2948       52.4%                 89.1
UCB1        0.2554       31.5%                207.4
UCB c=0.3   0.3076       64.9%                 50.9
Thompson    0.2964       63.4%                 84.4
```



What the results show:

- **Greedy** locks in early: it finds the best arm in only 23.5% of problems, and its curve goes flat.
- **ε-greedy** and **Thompson** both learn steadily; Thompson picks the best arm most often among the untuned methods and needs no ε to tune.
- **UCB1** with its textbook c = √2 explores *too much* at this horizon: the bonus is large compared with rewards around 0.2–0.3, so after 3,000 pulls it is still sampling bad arms. A smaller constant (c = 0.3) gives the lowest regret here. Lesson: theoretical constants guarantee long-run behaviour, not the best finite-horizon performance — tune on simulations of your own traffic.

Contextual bandits

Add **context** (user features, time, device): choose the arm based on context, e.g. LinUCB (a linear model per arm with a UCB bonus). Yahoo! used LinUCB for news-article recommendation on its front page (Li et al., 2010). Netflix has described using contextual bandits to choose which artwork to show each member. Contextual bandits are arguably the most widely deployed form of RL in industry, because they are simple, relatively safe and easy to measure.

Common mistakes

- Using a fixed ε forever — decay it, or use Thompson/UCB.
- Ignoring **non-stationarity**: click rates drift; use a sliding window or constant step size instead of an all-time average.
- Declaring a winner from a handful of pulls (the greedy trap).

In the interview

30-second answer: "Exploration vs exploitation is the trade-off between using the best-known option and gathering information about others. Bandits isolate it. ε-greedy explores randomly at a fixed rate; UCB adds an optimism bonus that shrinks with the number of tries; Thompson sampling samples from a posterior over each arm's reward and acts greedily on the sample. In production I'd use a contextual bandit, often Thompson or LinUCB, for things like headline or thumbnail selection."

- *Bandit vs A/B test?* A/B splits traffic evenly until the end; a bandit shifts traffic toward winners during the test, reducing regret — at the cost of less clean statistics.
- *Why does Thompson work?* An arm is chosen with the probability that it is the best, given the evidence.

Self-check

1. Why is pure greedy bad? → An unlucky early estimate of the best arm is never corrected.
2. In UCB, what happens to an arm's bonus as it is pulled more? → It shrinks like $1/\sqrt{N_a}$.
3. What distribution does Thompson use for 0/1 rewards? → Beta(successes + 1, failures + 1).

TL;DR

Explore to learn, exploit to earn. ε-greedy (random exploration), UCB (optimism bonus), Thompson (posterior sampling). Contextual bandits are RL's biggest industrial success.

14.5 Dynamic programming — planning when the model is known

What it is

If you **know** the MDP (P and R), you can compute optimal values without any trial and error, by repeatedly applying the Bellman equations. This family is called **dynamic programming (DP)** (Bellman, 1950s).

- **Policy evaluation**: given π, start with V = 0 and repeatedly replace each V(s) by the Bellman *expectation* right-hand side until it stops changing.
- **Policy improvement**: make π greedy with respect to the current V (look one step ahead with the model).
- **Policy iteration**: alternate evaluation and improvement until the policy stops changing. Converges in few rounds.
- **Value iteration**: skip full evaluation — apply the Bellman *optimality* update (with the max) directly, sweep after sweep, then read off the greedy policy.

Code — `code/value_iteration.py`

```python
def q_from_v(env, V, s, gamma):
    """One-step look-ahead: Q(s,a) = sum_s' P(s'|s,a) * [r + gamma * V(s') * (not done)]."""
    return np.array([sum(p * (r + gamma * V[s2] * (not done)) for p, s2, r, done in env.transitions(s, a))
                     for a in range(env.nA)])

def value_iteration(env, gamma, tol=1e-8):
    V, deltas = np.zeros(env.nS), []
    while True:
        delta = 0.0
        for s in range(env.nS):
            if env.is_terminal(s):
                continue
            best = q_from_v(env, V, s, gamma).max()      # Bellman OPTIMALITY backup
            delta = max(delta, abs(best - V[s]))
            V[s] = best
        deltas.append(delta)
        if delta < tol:
            break
    policy = np.array([np.argmax(q_from_v(env, V, s, gamma)) for s in range(env.nS)])
    return V, policy, deltas
```

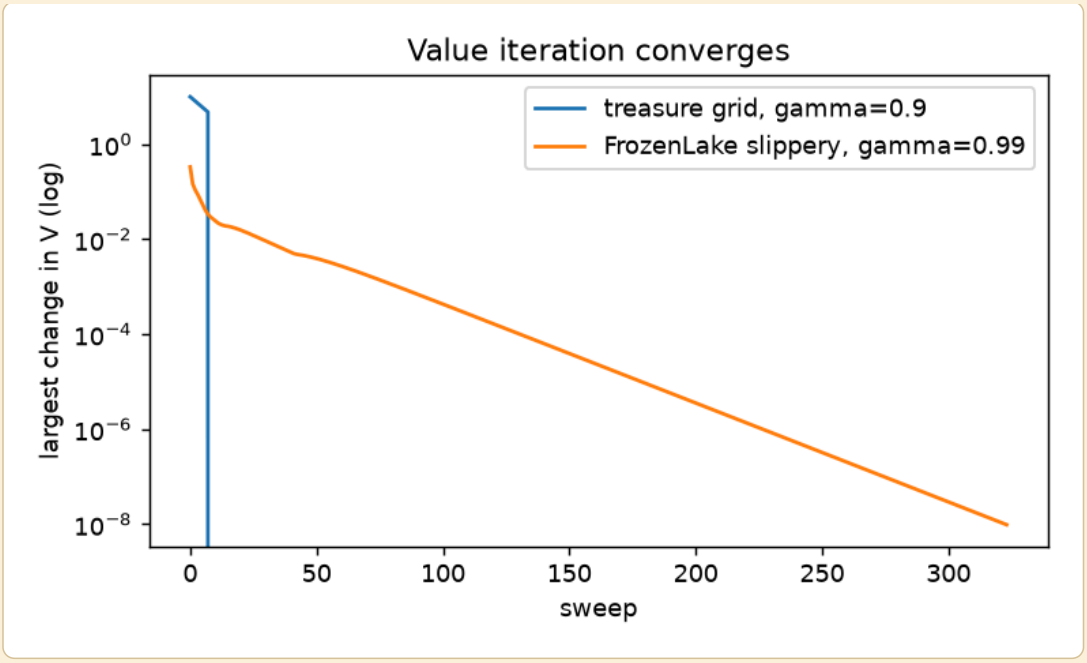
The **FrozenLake** world (a classic RL toy): walk across a frozen lake from S to the goal G without falling into holes H. It is **slippery**: whatever direction you choose, you move that way only 1/3 of the time and slide to either side 1/3 each. Reward 1 only for reaching G.

```
SFFF      F = frozen (safe), H = hole (episode ends, reward 0)
FHFH
FFFH
HFFG
```

Output:

```
== Treasure grid (deterministic), gamma = 0.9 ==
value iteration converged in 9 sweeps
V*:
 -0.43   0.63   1.81   3.12   4.58
  0.63   1.81   3.12   4.58   6.20
  1.81   3.12   4.58   6.20   8.00
  3.12   4.58   6.20   8.00  10.00
  4.58   6.20   8.00  10.00   0.00
greedy policy:
vv>vv
vvv>v
v>vvv
vvvvv
>>>>G

== FrozenLake 4x4, slippery (2/3 of moves slide sideways), gamma = 0.99 ==
value iteration converged in 324 sweeps
V*:
  0.54   0.50   0.47   0.46
  0.56   0.00   0.36   0.00
  0.59   0.64   0.62   0.00
  0.00   0.74   0.86   0.00
policy:
<^^^
<H<H
^v<H
H>vG
policy iteration: 7 improvement rounds, same policy as VI: True
simulated success rate of this policy: 73.8%
a random fixed policy succeeds:         2.9%
```



Reading it:

- On the treasure grid the values rise smoothly toward G (the Bellman numbers from §14.3), and the greedy policy routes around every trap.
- On the slippery lake the optimal policy looks **strange**: from the start it presses *left* (into the wall!). Because you only slide sideways, pushing against the wall means you either stay put or slide down — never toward a hole. Top-row cells press *up* for the same reason. The optimal plan under uncertainty is about **avoiding risk**, not the shortest path. It reaches the goal about 74% of the time within the 100-step limit, vs 3% for a random fixed policy.
- Value iteration needed 324 sweeps on the lake (γ = 0.99, slow propagation) but only 9 on the deterministic grid; policy iteration needed just 7 improvement rounds.

Trade-offs / when not to use it

DP needs the **full model** and sweeps **every state** each iteration — fine for thousands of states, impossible for Go (≈10^{170} positions) or for continuous states. It is the foundation that model-free methods approximate from samples.

Self-check

1. Value iteration vs policy iteration? → VI applies the max-backup repeatedly and reads off the policy at the end; PI fully evaluates a policy, improves it greedily, and repeats until the policy stops changing.
2. Why does the lake policy press *left* at the start? → Slips only go sideways, so pushing into the wall can never slide the agent into a hole.
3. Why can't DP solve Go? → It needs the full model and a sweep over every state — about 10^{170} of them.

TL;DR

With a known model, value iteration (repeat the Bellman optimality backup) or policy iteration (evaluate, then act greedily; repeat) compute optimal values and policies exactly. Great for small, fully known problems.

14.6 Learning from experience: Monte Carlo vs temporal difference

What it is

Usually we **don't** know P and R — we can only act and observe. **Model-free** methods estimate values from sampled experience. To estimate V_π:

	Monte Carlo (MC)	**Temporal difference, TD(0)**
Update target	Actual return G_t observed at the end of the episode	$r_{t+1} + \gamma V(s_{t+1})$ — one real reward + current estimate of the rest
Update rule	$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$	$V(s) \leftarrow V(s) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s)]$
When it can update	After the episode ends	After every step (works for never-ending tasks)
Bias	Unbiased (real return)	Biased (uses its own estimate — **bootstrapping**)
Variance	High (return depends on many random steps)	Low (one random step)
Usually	Slower, simpler	Faster learning in practice

The quantity $\delta = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the **TD error**: "how much better or worse did this step go than I expected?" It is one of the central quantities in RL (and has a well-known parallel in dopamine signalling in the brain, per Schultz, Dayan & Montague, 1997). α is the **step size** (learning rate).

Analogy: predicting your commute time. MC waits until you arrive, then updates the estimate. TD updates at every landmark: "I'm at the bridge 5 minutes late → my estimate for the whole trip was too low, adjust now."

Code — `code/mc_vs_td.py` (the classic 5-state random walk)

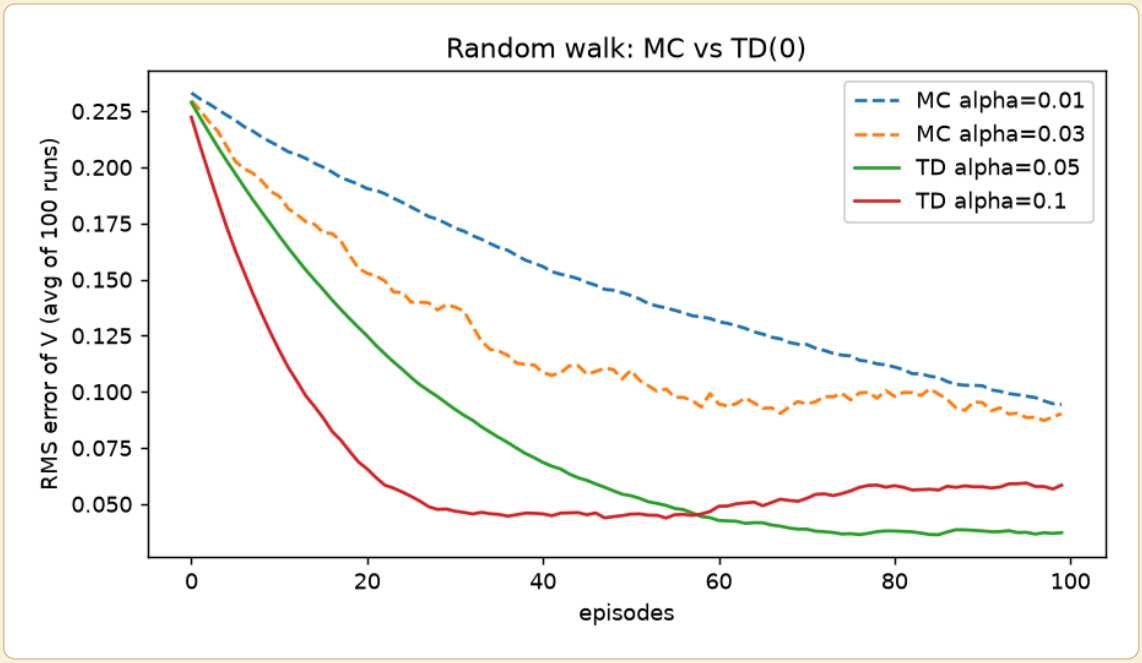
States A–E in a row; start at C; step left or right at random; reaching the right end gives +1, the left end 0. True values are 1/6, 2/6, …, 5/6.

```
if method == "MC":        # wait until the end, then move each V(s) toward the actual return G
    for s in states:
        V[s] += alpha * (G - V[s])
else:                     # TD(0): after every step, move V(s) toward r + gamma * V(s')
    for t, s in enumerate(states):
        target = gamma * V[states[t + 1]] if t + 1 < len(states) else final_r
        V[s] += alpha * (target - V[s])
```

(The file writes the target explicitly with the zero intermediate reward; this is every-visit, constant-α Monte Carlo.)

Output:

```
MC alpha=0.01  RMS error after 10 eps 0.211, after 100 eps 0.094
MC alpha=0.03  RMS error after 10 eps 0.189, after 100 eps 0.090
TD alpha=0.05  RMS error after 10 eps 0.175, after 100 eps 0.037
TD alpha=0.1   RMS error after 10 eps 0.126, after 100 eps 0.058
TD estimate after 300 episodes: [0.22 0.41 0.51 0.67 0.84]  true: [0.17 0.33 0.5  0.67 0.83]
```



TD learns faster than MC here, as in Sutton & Barto's version of this experiment. Note the step-size trade-off: a larger α learns faster early but leaves more noise at the end (TD α = 0.1 plateaus above α = 0.05). With a constant α the estimates keep jittering around the truth rather than converging exactly.

n-step returns and TD(λ) blend the two: use n real rewards before bootstrapping (n = 1 is TD, n = ∞ is MC), or average the n-step targets for all n with weights $(1-\lambda)\lambda^{n-1}$ (λ = 0 gives TD(0), λ = 1 gives MC). Modern deep RL (e.g. GAE in PPO) uses this idea.

Self-check

1. TD(0) step: V(s) = 0.5, reward 0, V(s') = 0.6, γ = 1, α = 0.1. New V(s)? → δ = 0 + 0.6 − 0.5 = 0.1, so V(s) = 0.5 + 0.1·0.1 = **0.51**.
2. Which method can learn in a task that never ends? → TD — MC must wait for an episode's end.
3. Why is MC unbiased but noisy? → Its target is the real return, which sums many random steps.

TL;DR

MC updates toward the actual return (unbiased, noisy, waits for the end). TD updates toward reward + γ·V(next) (bootstraps, lower variance, learns every step). The TD error δ drives most of RL.

14.7 Q-learning and SARSA — tabular control

What it is

To **act** without a model we learn **Q(s, a)** from experience, using TD updates, while behaving ε-greedily.

Q-learning (Watkins, 1989) — *off-policy*: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\,]$

SARSA (Rummery & Niranjan, 1994) — *on-policy*, named after the quintuple (s, a, r, s', a') it uses: Q(s, a) ← Q(s, a) + α [r + γ · Q(s', a') − Q(s, a)], where a' is the action the agent will actually take next.

Off-policy means the agent learns about one policy (the greedy, optimal one — the max) while following another (the exploratory ε-greedy one). **On-policy** means it learns the value of the policy it is actually following, exploration included.

How it works (Q-learning episode loop)

1. Initialise Q(s, a) = 0 for all s, a.
2. For each episode: s ← start.
3. Choose a with ε-greedy on Q(s, ·).
4. Take a; observe r, s'.
5. Update Q(s, a) toward r + γ·max$_{a}$'Q(s', a') (use 0 instead of the max if s' is terminal).
6. s ← s'; repeat until the episode ends.
7. Decay ε over episodes so the agent gradually shifts from exploring to exploiting.

Under standard conditions (every state–action pair visited infinitely often, step sizes decaying appropriately) tabular Q-learning converges to Q*.

Code — `code/q_learning_sarsa.py`

```
def train(env, method, episodes, alpha, gamma, eps_schedule, seed=0):
    rng = np.random.default_rng(seed)
    Q = np.zeros((env.nS, env.nA))
    returns = []
    for ep in range(episodes):
        eps = eps_schedule(ep)
        s = env.reset()
        a = epsilon_greedy(Q, s, eps, rng)
        total, done, trunc = 0.0, False, False
        while not (done or trunc):
            s2, r, done, trunc = env.step(a)
            a2 = epsilon_greedy(Q, s2, eps, rng)
            if method == "Q-learning":           # off-policy: bootstrap from the BEST next action
                target = r + gamma * Q[s2].max() * (not done)
            else:                                # SARSA, on-policy: from the action we WILL take
                target = r + gamma * Q[s2, a2] * (not done)
            Q[s, a] += alpha * (target - Q[s, a])
            s, a, total = s2, a2, total + r
        returns.append(total)
    return Q, np.array(returns)
```

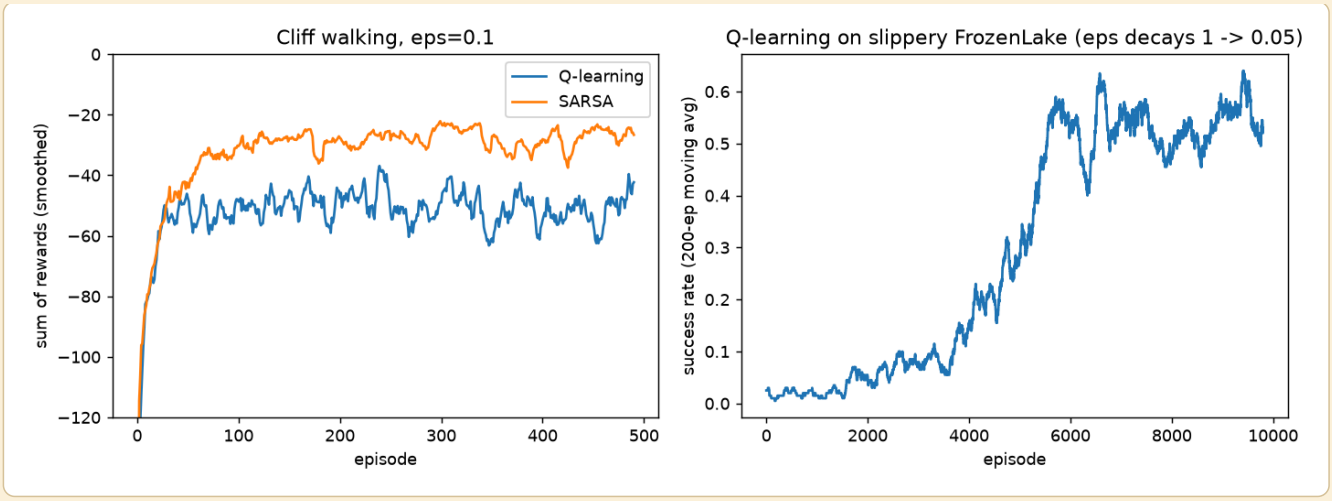
Part 1 — Cliff walking (Sutton & Barto, Example 6.6): walk from S to G along the bottom of a 4×12 grid; the cells between them (C) are a cliff: stepping in costs −100 and sends you back to S. Every step costs −1. ε stays at 0.1.

Part 2 — slippery FrozenLake with ε decaying from 1 to 0.05 over 6,000 of 10,000 episodes.

Output:

```
Q-learning avg return per episode (last 100, while exploring eps=0.1):   -51.7
greedy path learned:
>^>>>>>>>>>v
>>v>>v>>>>vv
>>>>>>>>>>v
^CCCCCCCCCCG
SARSA      avg return per episode (last 100, while exploring eps=0.1):   -27.2
greedy path learned:
>>>>>>>>>>v
^>>^^^^^>^vv
^^^^^>>>^>v
^CCCCCCCCCCG

FrozenLake Q-learning policy:
<^^^
<H<H
^v<H
H>vG
greedy success rate: Q-learning 73.4% | value-iteration optimum 74.0%
```



Reading it:

- **Q-learning** learns the *optimal* path — right along the cliff edge (row 3: `>>>>>>>>>>>v`). But while it still explores with ε = 0.1, a random step sometimes drops it off the cliff, so its average return during training is worse (−51.7).
- **SARSA** learns the value of what it actually does, *including* its random slips, so it prefers the **safe path** along the top — longer, but it earns more while exploring (−27.2).
- Which is "right"? If exploration will continue in deployment (a robot that sometimes slips), SARSA's caution is valuable; if you will act greedily once trained, Q-learning finds the better final policy.
- On FrozenLake, Q-learning — with no knowledge of the slip probabilities — **rediscovers the exact optimal policy** found by value iteration (identical arrows, 73–74% success in simulation). It needed thousands of episodes to do what DP did from the model: that is **sample inefficiency** in a nutshell.

Trade-offs

	Q-learning	SARSA
Type	Off-policy (learns greedy policy)	On-policy (learns the behaviour policy)
Target	max over next actions	the actually chosen next action
During training	Riskier (optimistic about its own future)	Safer
Learns from others' data / replay	Yes (basis of DQN)	Not directly
Known issue	Maximisation bias (fixed by Double Q-learning)	Converges to optimal only as ε → 0

Common mistakes

- Bootstrapping from a terminal state (must use 0 after the episode ends — the `(not done)` factor). Also, a **time-limit** truncation is not a real terminal state; ideally keep bootstrapping there.
- Never decaying ε, or decaying it so fast the agent never finds the goal (sparse rewards!).
- Breaking ties by always taking action 0 when Q is all zeros — the agent walks into a wall forever. Break ties randomly.
- Tabular methods on large state spaces — the table never fills in; use function approximation (next section).

In the interview

> **30-second answer:** "Q-learning is an off-policy TD method: it updates Q(s,a) toward r + γ·max Q(s',·), so it learns the optimal greedy policy while exploring with ε-greedy. SARSA is on-policy: it bootstraps from the action it actually takes next, so it learns the value of its exploratory behaviour and tends to be safer, as the cliff-walking example shows."

- *Why does Q-learning overestimate?* The max over noisy estimates is biased upward; Double Q-learning uses one estimator to select and another to evaluate.
- *What does off-policy enable?* Learning from old data or other agents' data — replay buffers, offline RL.

Self-check

1. Which target uses max, and why is that "off-policy"? → Q-learning; it evaluates the greedy policy regardless of the exploratory action actually taken.
2. On the cliff, why does SARSA take the long way? → Its values include the chance of random slips off the edge.
3. What does Q converge to under Q-learning? → Q*, given enough exploration and suitable step sizes.

TL;DR

Q-learning: off-policy, target r + γ·max Q(s') → learns the optimal policy. SARSA: on-policy, target r + γ·Q(s', a') → learns what it actually does, safer. Both are TD control with ε-greedy exploration.

14.8 Deep reinforcement learning — DQN

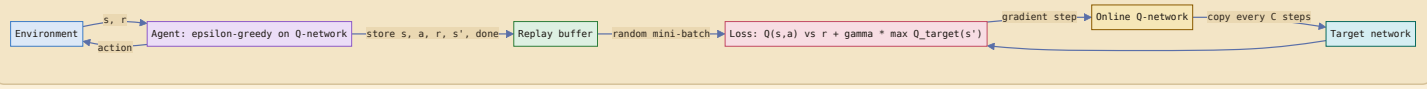
What it is

A Q-table needs one entry per state–action pair. Atari screens, robot joint angles or board positions have far too many states. **Function approximation** replaces the table with a model Q(s, a; θ) — typically a neural network that takes the state and outputs one Q-value per action. That network generalises: similar states get similar values.

DQN (Deep Q-Network) — DeepMind, Mnih et al., *Nature* 2015 — learned to play 49 Atari 2600 games from raw pixels and score, with a single architecture and hyperparameter set, reaching human-level performance on many of them. Naively combining Q-learning with a neural network is unstable; DQN made it work with two tricks:

1. **Experience replay**: store transitions (s, a, r, s', done) in a large buffer; train on **random mini-batches** from it. This breaks the strong correlation between consecutive steps (which violates the i.i.d. assumption of SGD) and reuses each experience many times.
2. **Target network**: compute the TD target $r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-)$ with a **frozen copy** θ^- of the network, copied from the online network only every C steps. Otherwise every update moves the target it is chasing ("a dog chasing its tail").

Plus: ε-greedy exploration, the Huber loss (less sensitive to large errors), frame stacking for motion.



The **deadly triad** (Sutton & Barto): function approximation + bootstrapping + off-policy learning can diverge. DQN's tricks are mitigations, not guarantees.

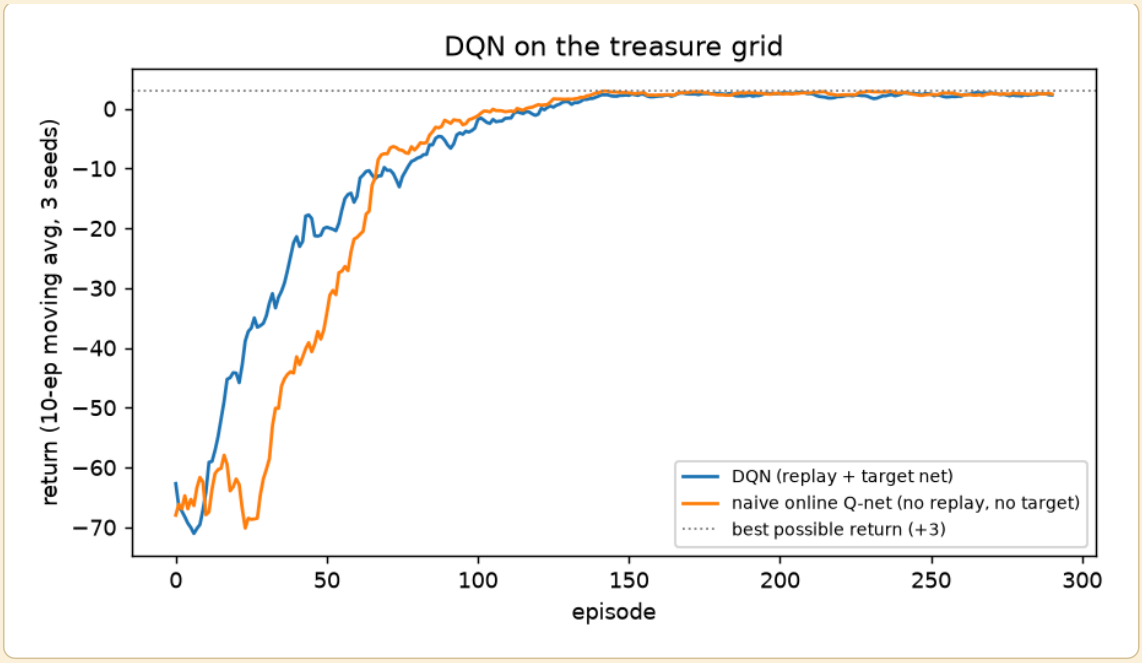
Code — `code/dqn.py` (DQN on the treasure grid)

The state is one-hot encoded (25 inputs); a 64-unit MLP outputs 4 Q-values.

```
buffer = deque(maxlen=5000)                              # experience replay memory
...
buffer.append((s, a, r, s2, done))
batch = random.sample(buffer, min(64, len(buffer))) if use_replay else [buffer[-1]]
...
with torch.no_grad():                                    # TD target from the frozen network
    nxt = (target_net if use_target else q_net)(S2).max(1).values
    y = R + gamma * nxt * (1 - D)
q = q_net(S).gather(1, A[:, None]).squeeze(1)
loss = nn.functional.smooth_l1_loss(q, y)                # Huber loss
opt.zero_grad(); loss.backward(); opt.step()
if use_target and steps % 200 == 0:
    target_net.load_state_dict(q_net.state_dict())       # periodic hard copy
```

Output:

```
DQN (replay + target net)                 mean return, last 50 episodes: +2.31
naive online Q-net (no replay, no target)  mean return, last 50 episodes: +2.42
greedy policy of the DQN (seed 0):
vv>>v
vv>>v
>>vvv
vvvvv
>>>>G
```



- The DQN learns a trap-free 8-move route (return +3; the last-50 average is a bit lower because ε = 0.05 exploration continues).
- With replay it learns noticeably **faster** (it reuses each experience in many mini-batches).
- On this tiny, 25-state problem the naive version also gets there eventually. Honest takeaway: replay and target networks matter most at scale — large networks, pixel inputs, long training — where the naive version often oscillates or diverges.

Beyond DQN (one line each)

- **Double DQN**: select the next action with the online net, evaluate it with the target net → less overestimation.
- **Dueling DQN**: separate streams for V(s) and the advantage A(s, a).
- **Prioritised replay**: sample transitions with large TD error more often.
- **Rainbow** (2017): combines these and more.
- DQN handles **discrete** actions only; continuous control (robot torques) uses actor-critic methods such as DDPG, TD3 and SAC.

In the interview

30-second answer: "DQN replaces the Q-table with a neural network trained toward the Q-learning target r + γ·max Q(s', ·). Two tricks make it stable: an experience-replay buffer sampled at random, which breaks correlations and reuses data, and a target network, a frozen copy that is refreshed only every C steps so the target stops moving. It works for discrete actions; continuous control uses actor-critic methods such as SAC or TD3."

- *Why does DQN overestimate, and the fix?* The max over noisy estimates is biased upward; Double DQN picks the action with the online net and evaluates it with the target net.
- *Why not use DQN for robot torques?* Taking the max over infinitely many continuous actions isn't practical.

Self-check

1. What goes wrong without a target network? → Every update shifts the target it is chasing, which can cause oscillation or divergence.
2. What does the replay buffer store? → Transitions (s, a, r, s', done).
3. Name the deadly triad. → Function approximation, bootstrapping, off-policy learning.

TL;DR

Deep RL replaces the Q-table with a neural network. DQN stabilises Q-learning with experience replay (decorrelate, reuse) and a target network (freeze the target). Discrete actions only; beware the deadly triad.

14.9 Policy gradients, REINFORCE and actor-critic

What it is

Value-based methods learn Q and derive a policy from it. **Policy-gradient** methods learn the policy **directly**: a parameterised π(a | s; θ) (e.g. a softmax over action scores, or a neural net outputting a Gaussian over torques), improved by gradient ascent on expected return J(θ).

Why bother?

- Works naturally with **continuous** and huge action spaces (no max over actions needed).
- Can learn **stochastic** policies (useful in games like rock-paper-scissors and under partial observability).
- Small parameter changes → smooth policy changes (more stable than a greedy argmax that can flip).

The policy-gradient theorem, intuitively

$\nabla\theta\ J(\theta) = E[\ \nabla\theta \log \pi(a \mid s; \theta) \cdot (\text{return that followed})\ ]$

Read it as: *make actions that led to high returns more likely, and actions that led to low returns less likely, in proportion to how good the outcome was.* $\nabla \log \pi(a|s)$ is the direction in parameter space that raises the probability of the action taken. For a softmax policy with scores $\theta[s, \cdot]$, that gradient is simply (one-hot(a) − $\pi(\cdot|s)$).

REINFORCE (Williams, 1992)

1. Run an episode with the current policy; record (s, a, r) at every step.
2. For each step t compute the return G_t.
3. Update $\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla\theta \log \pi(a_t \mid s_t; \theta)$.

(The textbook version also multiplies by γ^t; most practical implementations drop that factor, and so does our code.)

Problem: variance. G_t depends on everything that happened afterwards, so the gradient estimate is extremely noisy. If all returns are negative (as with a −1 per step cost), every action taken gets pushed down, and learning can lock into bad behaviour.

Fix: a baseline. Subtract b(s) — usually a learned estimate V(s) — from the return: use the **advantage** $A_t = G_t - V(s_t)$, "was this better or worse than usual from here?". It does not change the expected gradient but greatly reduces its variance.

Actor-critic

Go one step further: instead of waiting for the full return, use the **TD error** as the advantage: $\delta = r + \gamma V(s') - V(s)$.

- The **actor** (policy π) chooses actions and is updated by $\delta \cdot \nabla \log \pi$.
- The **critic** (value function V) judges them and is updated by TD learning. Updates happen every step, with lower variance (and some bias). Modern variants: **A2C/A3C**, **PPO**, **SAC**.

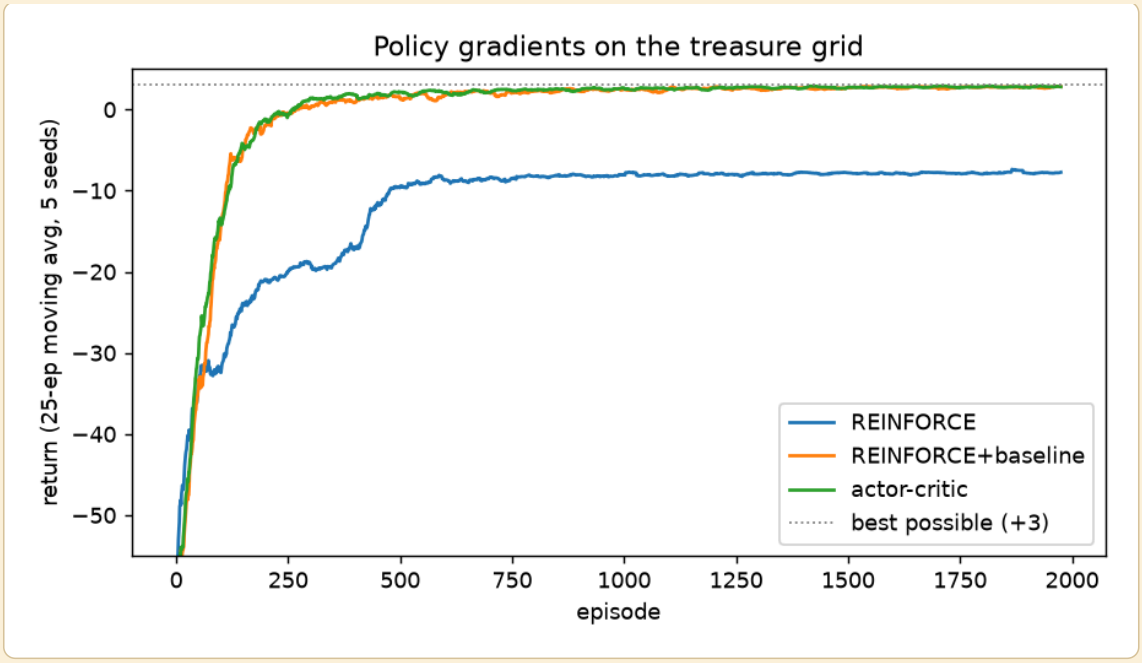
PPO (Proximal Policy Optimization, Schulman et al., 2017) is today's default workhorse: an actor-critic that **clips** how far each update may move the policy from the one that collected the data. With the probability ratio $\rho = \pi_new(a|s) / \pi_old(a|s)$ and advantage A, it maximises $\min(\rho \cdot A, \text{clip}(\rho, 1 - \varepsilon, 1 + \varepsilon) \cdot A)$ with $\varepsilon \approx 0.2$, so pushing ρ outside that band earns nothing extra — preventing destructive large steps. It was the RL algorithm in InstructGPT (2022), the RLHF recipe behind early ChatGPT.

Code — `code/policy_gradient.py` (tabular softmax policy on the treasure grid)

```
p = softmax(theta[s])
a = rng.choice(4, p=p)
s2, r, done, trunc = env.step(a)
if method == "actor-critic":          # update every step using the TD error
    delta = r + GAMMA * V[s2] * (not done) - V[s]
    V[s] += lr_v * delta              # critic
    grad = -p; grad[a] += 1           # d log pi(a|s) / d theta[s]
    theta[s] += lr * delta * grad     # actor
...
# REINFORCE (after the episode), going backwards to accumulate the return
G = r + GAMMA * G
adv = G if method == "REINFORCE" else G - V[s]   # baseline version subtracts V(s)
theta[s] += lr * adv * grad
```

Output (5 random seeds, 2,000 episodes each):

```
REINFORCE           mean return: episodes 1-100 -40.96 | last 100 -7.80 | std across seeds (last 100) 21.10
REINFORCE+baseline  mean return: episodes 1-100 -42.48 | last 100 +2.75 | std across seeds (last 100) 0.08
actor-critic        mean return: episodes 1-100 -39.86 | last 100 +2.80 | std across seeds (last 100) 0.02
```



- **Plain REINFORCE** is unreliable: four of the five seeds learn a good route, but one gets stuck in a policy that never reaches the treasure (−50 per episode) — hence the average of −7.8 and the huge spread across seeds (21.1). That is the variance problem in action.
- **Adding a baseline** fixes it — every seed reaches close to the optimal +3.
- **Actor-critic** learns about as fast, with the smallest spread, because it updates every step from the lower-variance TD error.

Value-based vs policy-based

	Value-based (Q-learning, DQN)	Policy-based (REINFORCE, PPO)	Actor-critic
Learns	Q(s, a)	π(a | s)	Both
Action spaces	Discrete	Discrete or continuous	Both
Sample efficiency	Higher (off-policy replay)	Lower (on-policy)	Middle
Stability	Can diverge (deadly triad)	High variance	Better balanced
Examples	DQN (Atari)	REINFORCE	PPO (RLHF, games, robotics), SAC (robotics)

In the interview

30-second answer: "Policy-gradient methods adjust the policy's parameters directly so that actions followed by high returns become more likely: the gradient is the average of ∇log π(a|s) times the return. REINFORCE uses the full return and is noisy; subtracting a baseline V(s) gives the advantage and cuts variance without adding bias; actor-critic uses the TD error as the advantage and updates every step. PPO limits each update by clipping the probability ratio, which makes it stable enough to be the default."

- *Why policy gradients over DQN?* Continuous or huge action spaces, and stochastic policies.
- *Why is PPO on-policy a cost?* Data from old policies can't be reused much, so it needs more samples than replay-based methods.

Self-check

1. Softmax policy π(·|s) = [0.7, 0.2, 0.1] and action 0 was taken. ∇ of log π(a|s) with respect to the three scores? → one-hot − π = **[0.3, −0.2, −0.1]**.
2. Does subtracting a baseline change the expected gradient? → No — it only lowers the variance.
3. What does PPO clip? → The ratio π_new(a|s) / π_old(a|s), to the range [1 − ε, 1 + ε].

TL;DR

Policy gradients push up the log-probability of actions in proportion to how good their outcome was. REINFORCE is simple but noisy; a baseline (advantage = G − V) cuts variance; actor-critic uses the TD error for step-by-step updates. PPO = clipped actor-critic, the default in practice.

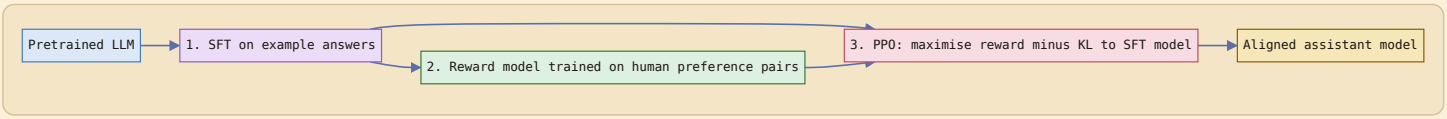
14.10 RLHF — the bridge to large language models

What it is

Reinforcement Learning from Human Feedback (RLHF) tunes a pretrained language model so that its responses match what people prefer (helpful, honest, harmless). The "environment" is a prompt; the "action" is the generated response (a sequence of token choices); the "reward" comes from a model trained on human preferences.

How it works (the InstructGPT recipe, Ouyang et al., 2022)

1. **Supervised fine-tuning (SFT)**: fine-tune the pretrained model on human-written example answers.
2. **Reward model (RM)**: show people two (or more) responses to the same prompt and ask which is better. Train a model to give higher scores to preferred responses (a pairwise loss: maximise log σ(score_preferred − score_rejected)). Comparisons are easier and more consistent for people than absolute scores.
3. **RL fine-tuning with PPO**: generate responses, score them with the RM, and update the policy (the LLM) to increase reward, **minus a KL penalty** that keeps it close to the SFT model. Without that penalty the policy drifts into gibberish or reward-model exploits — a textbook case of reward hacking (§14.12).



Variants you'll hear about

- **RLAIF / Constitutional AI** (Anthropic, Bai et al., 2022): an AI model, guided by a written set of principles, produces the preference labels, reducing the need for human labels on harmful content.
- **DPO** (Direct Preference Optimization, Rafailov et al., 2023): skips the separate reward model and the RL loop — optimises the policy directly on preference pairs with a classification-style loss. Simpler and popular for open models.
- **RL with verifiable rewards** for reasoning: reward = "did the maths answer / code test pass?". DeepSeek's GRPO (introduced in DeepSeekMath, 2024; used for DeepSeek-R1, 2025) is a PPO variant that samples a group of answers per prompt and scores each one relative to the group (reward minus the group mean, divided by the group's standard deviation) instead of using a learned critic.

Why it is still "RL"

The model's outputs change what data it learns from (on-policy sampling), the reward is a scalar judgement not a correct answer, and credit must be assigned across many token choices — all the defining features of RL. And all the RL pitfalls apply: reward hacking (verbose or flattering answers that score well with the RM), instability, and cost.

Self-check

1. Why ask people to compare two answers rather than score one? → Comparisons are easier and more consistent than absolute scores.
2. What does DPO remove from the pipeline? → The separate reward model and the RL loop.
3. What is a "verifiable reward"? → An automatic check, such as a passing unit test or a correct final answer.

TL;DR

RLHF = SFT → reward model from human preference comparisons → PPO against the reward model with a KL leash to the SFT model. DPO removes the RL loop; RLAIF replaces human labels with AI feedback; verifiable-reward RL powers reasoning models. The AI book's large language models chapter places RLHF in the full LLM training pipeline.

14.11 Real-world RL

Domain	Example (publicly described)	What kind of RL
Games	**AlphaGo** (DeepMind) beat Lee Sedol 4–1 in March 2016. Policy and value networks, first trained on human expert games, then improved by self-play RL, combined with Monte Carlo tree search. **AlphaGo Zero** (2017) learned from self-play alone; **AlphaZero** (2018) mastered chess, shogi and Go with one algorithm; **MuZero** (Nature, 2020) matched this without being told the rules, by learning its own model of the game's dynamics. Also: DQN on Atari (2015), OpenAI Five (Dota 2, 2019), AlphaStar (StarCraft II, 2019)	Self-play, deep RL + planning
Robotics	OpenAI's Dactyl hand manipulated a Rubik's Cube to a solved state (2019) with a policy trained entirely in simulation, using automatic **domain randomisation** (randomising physics and visuals so the policy copes with the real hand). The RL policy did the finger control; the sequence of face turns came from a standard cube-solving algorithm (Kociemba's). Legged-robot locomotion is now commonly trained in simulation (sim-to-real)	Deep RL (PPO/SAC) in simulation
Recommenders (Chapter 13)	Yahoo! front-page news with LinUCB (2010); Netflix artwork personalisation with contextual bandits; Spotify homepage shelves with bandits; research on RL for long-term engagement (e.g. Google's REINFORCE-based YouTube recommender, Chen et al., 2019)	Contextual bandits (most common), off-policy RL
Ads & pricing	Ad selection/creative testing with bandits; real-time bidding and **budget pacing** formulated as sequential decisions (several published industrial papers)	Bandits, constrained RL
Operations	DeepMind reported (2016) that neural-network recommendations cut the energy used for cooling in a Google data centre by up to 40%; a 2018 follow-up let the system control cooling directly	Control / RL-style optimisation
LLMs	RLHF / RLAIF for ChatGPT, Claude and others; verifiable-reward RL for reasoning models	PPO, DPO, GRPO

The honest pattern: RL shines when a **simulator or cheap trials** exist (games, simulated robots) or when the decision is **one step** (bandits). Full sequential RL in live products is rarer, because exploration there costs real money or user trust.

14.12 Pitfalls

Reward hacking (specification gaming)

The agent maximises **the reward you wrote**, not the goal you meant. `code/reward_hacking.py` adds a "+3 bonus" tile next to the start, intended to encourage exploration:

```
S+-..
..-..
...-.
.-...
....G
```

Output:

```
bonus tile worth +3: first 12 moves RUUUUUUUUUUU reward +36 reached treasure: False   V(start) = 30.00
bonus tile worth +0: first 12 moves RDDRDDRR     reward +4 reached treasure: True   V(start) = 0.57
```

The optimal policy for the buggy reward steps onto the bonus and then **walks into the wall forever** (U, U, U…): bumping the edge keeps it on the tile, collecting +3 every step — worth $3/(1-0.9) = 30$, far more than the treasure. It is not a bug in the learner; it's a bug in the reward. Real cases: a boat-racing agent (OpenAI's CoastRunners example, 2016) that circled to hit respawning targets instead of finishing the race; chat models learning that longer or more flattering answers get higher reward-model scores.

Defences: reward the outcome, not a proxy (only once, and not re-collectable); penalise unwanted side effects; constrain the policy (e.g. RLHF's KL penalty); watch the agent's actual behaviour, not only the reward curve; red-team the reward.

Other pitfalls

Pitfall	What it means	Mitigations
Sample inefficiency	Q-learning needed thousands of episodes for a 16-cell lake; deep RL on Atari used hundreds of millions of frames in some variants	Simulators, model-based RL, replay, offline data, imitation learning to start
Sparse rewards	The only reward is at the far goal; random exploration never finds it	Reward shaping (carefully — see hacking), curriculum, curiosity bonuses, demonstrations
Credit assignment	Which of 200 moves earned the win?	Discounting, TD learning, advantage estimation (GAE)
Instability & seed sensitivity	Results swing between runs (see plain REINFORCE's std of 21)	Report many seeds with confidence intervals; target nets, clipping (PPO), normalisation
Sim-to-real gap	Policies overfit simulator quirks	Domain randomisation, system identification, fine-tune on real data
Safe exploration	Random actions can break robots or lose money	Constrained RL, safety layers, offline RL, human oversight
Distribution shift (offline RL)	Learning only from logs: the policy wants actions the logs never tried	Conservative methods (CQL, IQL), stay close to the data policy
Non-stationarity	The world (users, markets, opponents) changes	Continual learning, sliding windows, monitoring
Evaluation is hard	You can't A/B test every idea; off-policy evaluation is noisy	IPS/doubly robust estimators, shadow mode, staged rollouts

In the interview

30-second answer ("When would you use RL, and what goes wrong?"): "I'd use RL when decisions are sequential and change future states, and when I can simulate or safely experiment — otherwise supervised learning or a contextual bandit is simpler. Typical problems are reward hacking, sample inefficiency, sparse rewards, instability across seeds and the sim-to-real gap. I'd start with a bandit or offline evaluation, carefully design and audit the reward, and roll out in stages with guardrails."

- *Bandit or full RL for a recommender?* A contextual bandit if you optimise the immediate response; full RL only if you can model long-term effects (retention) and evaluate off-policy credibly.
- *How to evaluate an RL policy before launch?* Simulation; off-policy evaluation (IPS, doubly robust) on logged data with known propensities; shadow mode; small canary traffic.

Self-check

1. Why did the bonus-tile agent never reach the treasure? → Repeatedly re-collectable reward worth 30 in discounted value beats a one-off +10.
2. What is the deadly triad? → Function approximation + bootstrapping + off-policy → possible divergence.
3. Why does RLHF include a KL penalty? → To keep the model close to the SFT model and limit reward-model exploitation.

Chapter TL;DR

- **RL** = agent learns by trial and error: state → action → reward + next state; goal = maximise long-run (discounted) return. Feedback is evaluative and often delayed; the agent's behaviour shapes its data.
- **MDP** (S, A, P, R, γ); **return** $G_t = r_{t+1} + \gamma G_{t+1}$; γ sets the horizon.
- **V(s)**, **Q(s, a)**; **Bellman**: value = reward now + γ × value next; greedy on Q* is optimal.
- **Bandits**: ε-greedy, UCB (optimism bonus), Thompson (posterior sampling); contextual bandits are RL's biggest industrial use. Tune exploration for your horizon.
- **Dynamic programming** (model known): value iteration / policy iteration — solved the slippery lake exactly (≈74% success, "hug the wall" policy).
- **MC vs TD**: actual return (unbiased, noisy) vs bootstrapped target (lower variance, learns every step).
- **Q-learning** (off-policy, max target → optimal, risky while exploring) vs **SARSA** (on-policy → safer path on the cliff). Both learned from scratch without a model.
- **DQN**: neural Q-function + experience replay + target network; discrete actions.
- **Policy gradients**: REINFORCE (noisy) → baseline/advantage (much better) → actor-critic → PPO (clipped, default).
- **RLHF**: SFT → reward model from preference pairs → PPO with a KL leash; DPO and verifiable-reward RL are the newer variants.
- **Pitfalls**: reward hacking (our agent farmed a bonus tile forever), sample inefficiency, sparse rewards, instability across seeds, sim-to-real gap, unsafe exploration.

Chapter 15 — ML System Design: The Interview Framework

"Design YouTube's recommendations." "Design a fraud detector." "Add AI to our support desk." These questions now sit next to the classic "Design Twitter" in system design loops, and they are asked of ordinary backend engineers, not only ML specialists. The good news: everything you know about load balancers, caches, queues and databases still applies. The new part is that one box in your diagram **learns from data** — so its output can be wrong while the request succeeds, its quality can decay while the code stays the same, and each call may cost real money.

This chapter gives you a repeatable way to answer such questions in 45 minutes. Chapter 16 then goes deep on each production building block (training pipelines, feature stores, serving, evaluation, monitoring, A/B tests, MLOps).

What you'll learn

1. The five shapes an AI/ML design question takes, and what each role is expected to cover.
2. What changes when a system contains a model — four assumptions from classic design that break.
3. The three question families (predictive, generative, agentic) and the one-line test that tells them apart.
4. A seven-step framework with a minute-by-minute plan: frame → success metrics → data → baseline model → serving → keep-it-working loop → failures and cost.
5. How to turn a **business goal** into an **ML objective**, labels, features, offline and online metrics.
6. How to scope a vague "use AI to improve X" question in the first ten minutes (full worked capstone).
7. The mistakes that sink most answers.

Code for this chapter is in `code/` and runs with the shared environment. Outputs below are real.

15.1 ML design vs classic system design

What it is

An **ML (or AI) system design question** asks you to design a product in which at least one component is a **model** — a function learned from data rather than written as explicit rules. It takes an input (a user's watch history, a transaction, a typed question) and returns an output (a score, a label, a number, generated text).

Two words you will use constantly:

Term	Meaning	Example
Training	Fitting or updating the model from data. Usually a background job, separate from user traffic.	Nightly job that learns from yesterday's orders
Inference	Running the trained model on an input to get an output.	Scoring one transaction at checkout
Online inference	Inference inside a live request; must meet the request's latency limit.	Fraud score before a card payment is approved
Batch inference	Inference over many inputs in a scheduled job; results stored for later.	Tonight's recommendation lists for every active user

Analogy: a model is like a new hire who learned the job from past examples. You still need an office (infrastructure), a way to check their work (evaluation), a way to notice when they get worse (monitoring), a budget (cost), and a plan for when they make a mistake (fallbacks). Knowing how their brain works (model architecture) helps, but it is not what the design interview is mostly about.

Why it matters

A common preparation mistake is to spend weeks on model internals — transformer attention, loss derivations — and then freeze when asked "where does the training data come from?" or "what happens when the model times out?". In a system design round, the interviewer mostly grades the **whole system** around the model.

The five forms of the question

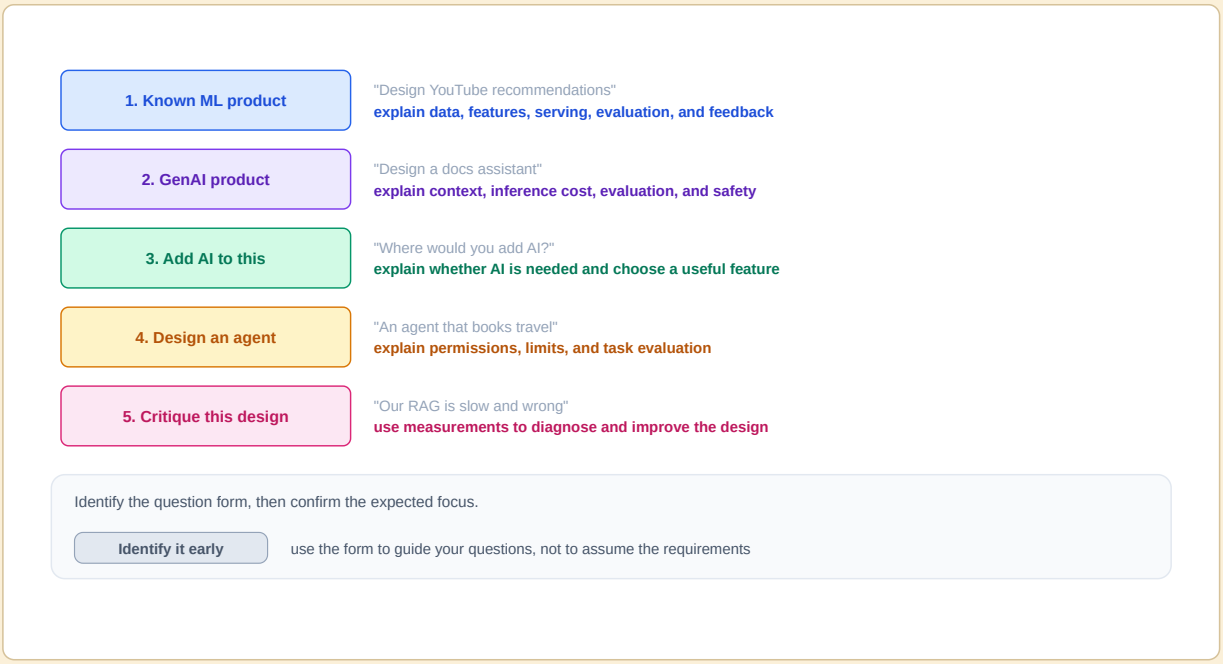


Figure — Five shapes of AI design prompts (course diagram)

#	Form	Example prompts	What to emphasise
1	**Known ML product**	"Design YouTube recommendations", "Design search autocomplete"	Data, features, training, serving, evaluation, feedback loop
2	**GenAI product**	"Design ChatGPT", "Design a docs Q&A assistant"	Retrieval, context assembly, inference cost, output evaluation, safety
3	**Add AI to an existing system**	"Our online store already runs fine — which part of it should get AI?"	Whether AI is needed at all, choosing one useful feature, simplest viable design
4	**Design an agent**	"An agent that books travel for users"	Tool permissions, approvals, step/cost limits, stop conditions, task-success evaluation
5	**Critique / scale an AI design**	"Our RAG system is slow and wrong — fix it"	Diagnose with measurements, logs and failed examples *before* changing anything

Terms: a **feature** is one piece of information fed to the model (watch history, hour of day); the **feature pipeline** computes and delivers them; the **feedback loop** collects outcomes (clicks, resolutions) to evaluate and improve the system. **GenAI** = generative AI (produces text, code, images). **LLM** = large language model. **RAG** = retrieval-augmented generation (fetch relevant documents and put them in the model's input — this helps ground answers but does not guarantee correctness). **Fine-tuning** = extra training of an existing model for a narrower task.

Who gets asked what

Expected depth depends on role and company — treat this as a preparation guide, and **ask**.

Role	Forms to prepare	Depth expected
Software engineer (senior+)	2, 3, sometimes 1	Architecture, serving, cost, failure handling; model inputs/outputs/limits
ML engineer	1, 2, 5	Above + model choice, features, training pipelines, evaluation metrics
AI / ML platform engineer	2, 4, 5	Inference infrastructure: GPUs, batching, caching, latency under load
Engineering manager / staff+	3, 5	Build vs buy, cost limits, ownership, risk, gradual rollout

A useful question in the first minute: *"Would you like me to go deep on model development, or on the system that serves and maintains the model?"*

What interviewers grade (and what they don't)

Graded	Usually secondary in a system design round
Requirements discipline — users, latency, quality measure, what's out of scope, numeric assumptions	Deriving the transformer or backprop by hand
A complete loop, not just the request path — how outcomes come back and improve the system	Naming this month's newest model
Cost and latency realism — state assumptions, estimate, then optimise	Framework trivia ("use library X") without saying why the component is needed
Failure honesty — a model can be confidently wrong with HTTP 200; detection, blast-radius limits, fallbacks	

> **How it works — four assumptions that break when a model enters the system**

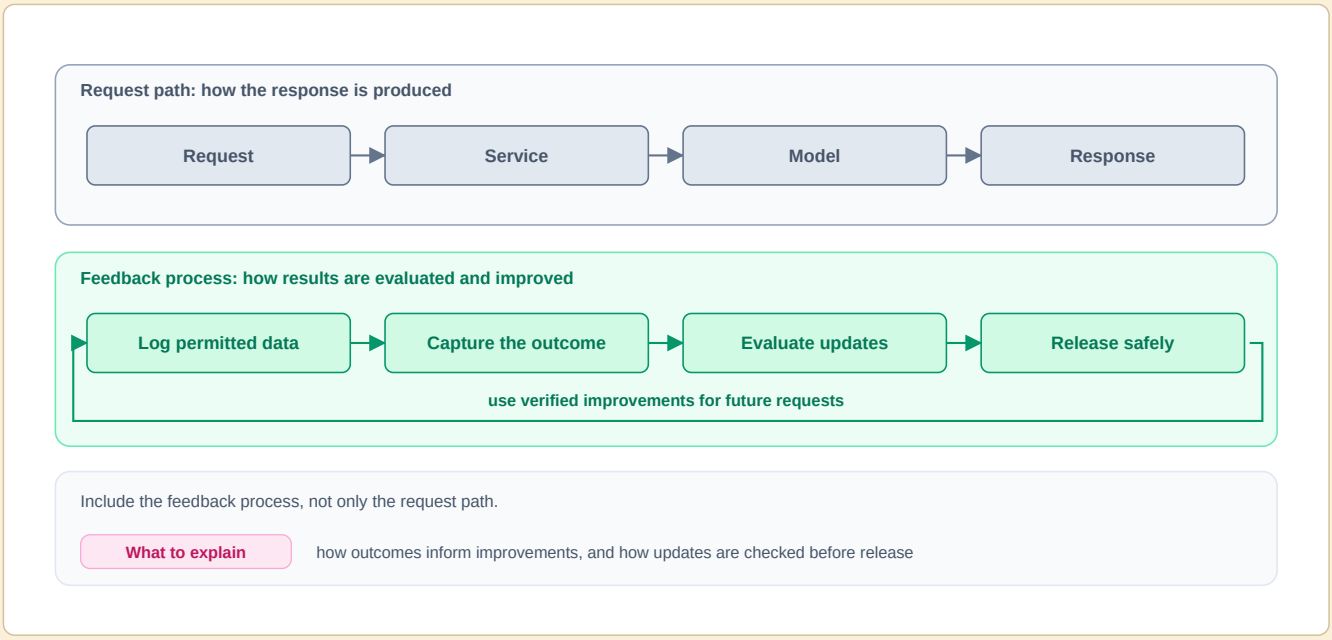


Figure — Serving path on top, improvement loop underneath (course diagram)

These are contrasts, not absolutes — non-ML systems also need quality checks and cost control — but they are where incomplete AI designs usually fall down.

#	Classic assumption	What changes with a model	Design response
1	**Correctness is binary** — a payment is either the authorised amount or a bug	Output quality is a spectrum: a recommendation can render fine and still be irrelevant	Define **quality metrics** (CTR, resolution rate, suggestion-acceptance rate) *alongside* latency and availability SLOs; log predictions and join them with later outcomes
2	**The code is the behaviour**	Behaviour also depends on data. **Drift** — a change in input patterns or in the input→outcome relationship — can degrade a model whose code never changed (new fraud tactics, changed tastes)	A feedback process: collect outcomes, investigate quality changes, evaluate a candidate against the current version, release gradually with rollback. Retrain because evidence says so, not on every alert
3	**Compute is cheap**	A model call can cost orders of magnitude more than a DB read, depending on model size, input/output length, hardware and utilisation	Put a number on it: 10M requests/day × $0.01 = $100,000/day ≈ $3M/month (illustrative). Then consider caching (only when permissions/freshness allow), routing easy requests to a smaller model (only if it meets the quality bar), and precomputing in batch
4	**Failures are loud** (500s, timeouts)	A wrong answer arrives as `200 OK`: an invented citation, a missed fraud, harmful text	Assume anything a user types, or that retrieval pulls in, may be hostile (prompt injection can't be fully stopped by input filters — enforce permissions outside the model), check outputs, require human approval for high-impact actions, define fallbacks (unpersonalised results, hand-off to a human)

Question to ask yourself for every design: **"Who is hurt by a wrong output, and what limits the damage?"**

Build, host, or call an API?

The other big decision is how you obtain and run the model. The options overlap (you can fine-tune and then use managed hosting).

Option	Pros	Cons / what you still own
Hosted API (pay per use)	Fastest to launch, no GPU fleet	Data handling terms, rate limits, provider availability, per-call cost at scale; you still build integration, evaluation, monitoring
Self-host open-weights model (weights downloadable under a licence)	Control over data, deployment, latency; can be cheaper at high, steady utilisation	Capacity, security, upgrades, on-call, often GPUs; savings are not guaranteed
Train / fine-tune your own	Needed when your own interaction data is the product (ranking, fraud, recommendations); fine-tuning can help a narrow generative task	Data preparation, training infra, ongoing maintenance

Compare them on quality, data sensitivity, volume, latency, total cost and team skills. Start with the simplest option that meets requirements and say what evidence would make you switch.

Trade-offs

Going ML at all is itself a trade-off. Google's *Rules of ML* open with **Rule #1: don't be afraid to launch without ML** — a heuristic may be good enough to start, and it produces the data you'll later learn from. **Rule #3: choose ML over a complex heuristic** — once hand-written rules become a tangle, a learned model is easier to maintain.

Common mistakes

- Answering an SWE design question like a research talk (15 minutes on fine-tuning objectives, nothing on serving or evaluation).
- Drawing only the request path — no logging, no outcome join, no retraining or evaluation loop.
- Assuming model calls are free or as fast as a cache read.
- Treating user feedback (thumbs up/down) as automatically trustworthy training labels.
- Claiming a fixed percentage of traffic can go to a small model without evidence.

Real-world examples

- **Netflix, YouTube, TikTok** feeds: predictive ranking systems (form 1).
- **GitHub Copilot** style code assistants: generative products measured by suggestion acceptance and retention (form 2).
- **Stripe Radar**: a fraud model embedded into an existing payments flow (form 3 in spirit).
- Customer-support agents that can issue refunds through tools with limits (form 4).

In the interview

30-second answer — *"What changes compared with an ordinary backend service?" → "Everything from distributed systems still holds. I add three things: quality metrics that sit beside the latency and uptime SLOs; a loop that notices when quality slips and tests any new version before rolling it out step by step; and a budget for per-call latency and spend. Finally, safeguards for bad outputs — validation, fallbacks, and a human reviewer where the stakes warrant one."*

- **"What do you mean by quality, concretely?"** → Name a task metric: for a feed, CTR plus a longer-term satisfaction signal (watch time, return visits); for fraud, precision and recall at the chosen threshold; for support, correct resolution rate plus CSAT and repeat-contact rate.
- **"Would you fine-tune?"** → Not first. Check whether an existing model with good instructions (and retrieval if private data is needed) meets the quality and cost bar on an evaluation set; fine-tune only if evaluation shows a specific gap that simpler changes don't close.
- **"Assume the model already exists."** → Then spend the time on integration, serving, cost, evaluation and feedback; bring training up only when some requirement depends on it.

Self-check

1. Why can a model fail "silently"? → Because a wrong prediction is returned successfully (HTTP 200); error-rate dashboards don't see it — you need quality metrics joined to outcomes.
2. Name the four broken assumptions. → Binary correctness, code = behaviour, cheap compute, loud failures.
3. When is a hosted API a poor fit? → Strict data-residency/sensitivity rules, very high steady volume where per-call pricing dominates, or latency needs the provider can't meet.
4. What does Rules of ML #1 say, and why? → You may launch without ML; a simple heuristic gets the product and the data pipeline going, and gives a baseline.

TL;DR

- Classic design still applies; a model adds **quality measurement, drift, cost, and silent failure**.
- Five question forms; confirm how deep to go on models vs system.
- Graded: requirements, full loop, cost/latency realism, failure honesty.
- Build vs host vs API is a requirements-driven choice — start simplest.

15.2 The three question families

What it is

Before choosing an architecture, classify what the system mainly has to **produce**:

Family	The model produces…	Typical questions	Focus of your answer	Classic failure
1. Predictive	A score, number or label that the system acts on	Recommendations, feed/search ranking, ads CTR, fraud/spam, ETA, churn	Data & features, candidate generation + ranking, offline & online metrics, feedback loop	Picking a model (e.g. "an LLM ranks the feed") without data, latency or cost reasoning
2. Generative	Content: an answer, summary, code, image	ChatGPT-style assistant, docs support bot, code assistant, meeting summaries	Retrieval & context assembly, inference latency/cost (streaming, caching, model routing), output evaluation, safety (prompt injection, hallucination)	Proposing to train a foundation model from scratch; ignoring cost; no quality measurement
3. Agentic	A decision about which **action** to take next (call a tool, read result, decide again)	Refund-issuing support agent, travel booking, coding agent opening PRs	Tool interface & permissions, step/time/cost limits, stop conditions, approval for risky actions, task-level evaluation	Unlimited tool access and no stop rule — or, the opposite, human approval for every harmless step

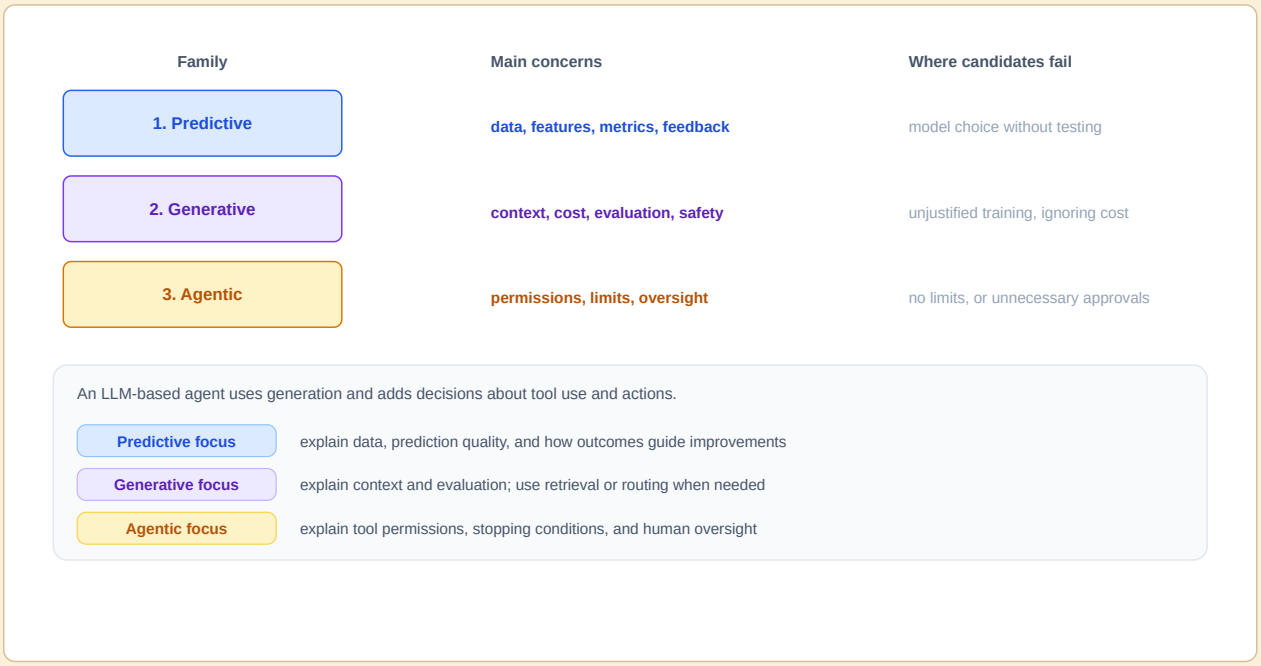


Figure — Predictive, generative and agentic compared (course diagram)

Terms: a **label** is the category or target value to predict (fraud / not fraud). **Pretrained** = trained before you use it for your task. A **foundation model** is a large model trained on broad data, adaptable to many tasks. A **token** is the chunk of text an LLM reads and writes — often a whole word, sometimes a word fragment or a punctuation mark. **Hallucination** = generated content that is false or unsupported. **Prompt injection** = untrusted text trying to redirect the model's instructions. **Streaming** = showing output as it is generated. **Model routing** = choosing which model handles a request based on difficulty/cost.

Why it matters

"Design an AI shopping experience" and "Design fraud detection" need completely different designs, metrics and risks. Classifying early prevents you from spending 30 minutes on the wrong system.

How it works — the output test

1. Ask: *does the system mainly output a prediction, content, or an action?*
2. If the prompt is vague, say what you think it is and confirm.
3. If the product mixes families, split it into subsystems. Example: a streaming service ranks the homepage (predictive) and writes "why you might like this" blurbs (generative). A support platform routes tickets (predictive), drafts replies (generative), and issues refunds through an authorised tool (agentic).
4. Name each subsystem, say how they pass work to each other, and ask which one to go deep on — usually the one carrying the biggest risk.

This book (ML Handbook) focuses mostly on **family 1**; the AI Handbook covers LLM serving, RAG and agents in depth. The framework below works for all three.

Common mistakes

- Using an LLM for everything, e.g. calling it once per item to rank a 50M-item catalog. Sensible LLM roles are narrower — enriching item descriptions offline, or reordering a short list of finalists — and even then benchmark it against a standard learned ranker on quality, speed and spend.

- Choosing a multi-agent design when a fixed workflow would do (if the steps are known, hard-code them).

In the interview

> **30-second answer** to an unclear prompt: *"First I want to pin down what the system outputs. A ranking or score means my time goes to data, features and evaluation. Generated text shifts it to context building, per-call spend and answer quality. Actions put tools, permissions and limits at the centre. My read is that this is mostly ranking items per user — shall I go that way?"*

- **"It's all three."** → List the subsystems and their interfaces, propose to go deep on the riskiest ("wrong refunds cost money, so the agentic part"), and let the interviewer redirect.
- **"Why not let an LLM rank?"** → Evaluate it: quality vs a learned ranker on interaction data, per-item latency and cost. Plausible as a reranker on the top ~50, not as a scorer over millions.

Self-check

1. "Summarise support tickets and auto-close duplicates." Family? → Generative (summaries) plus an action (closing tickets) — a small agentic part that needs permission and review rules.
2. What's graded most in predictive questions? → Features/data, candidate generation and ranking, offline and online metrics, the feedback loop.
3. Why is "LLM ranks the feed" weak? → It ignores personalisation data, evaluation, latency and cost at catalog scale.

TL;DR

Predictions → data/features/metrics. Content → context/cost/quality/safety. Actions → permissions/limits/oversight. Mixed products: split, connect, pick a focus.

15.3 The seven-step framework

What it is

A fixed order of decisions, so that requirements and data drive the architecture instead of the other way round. It is the classic design structure (requirements → estimates → API → high-level design → deep dive → wrap-up) with two ML-specific concerns made explicit: **define quality early** and **show how the system keeps working after launch**.

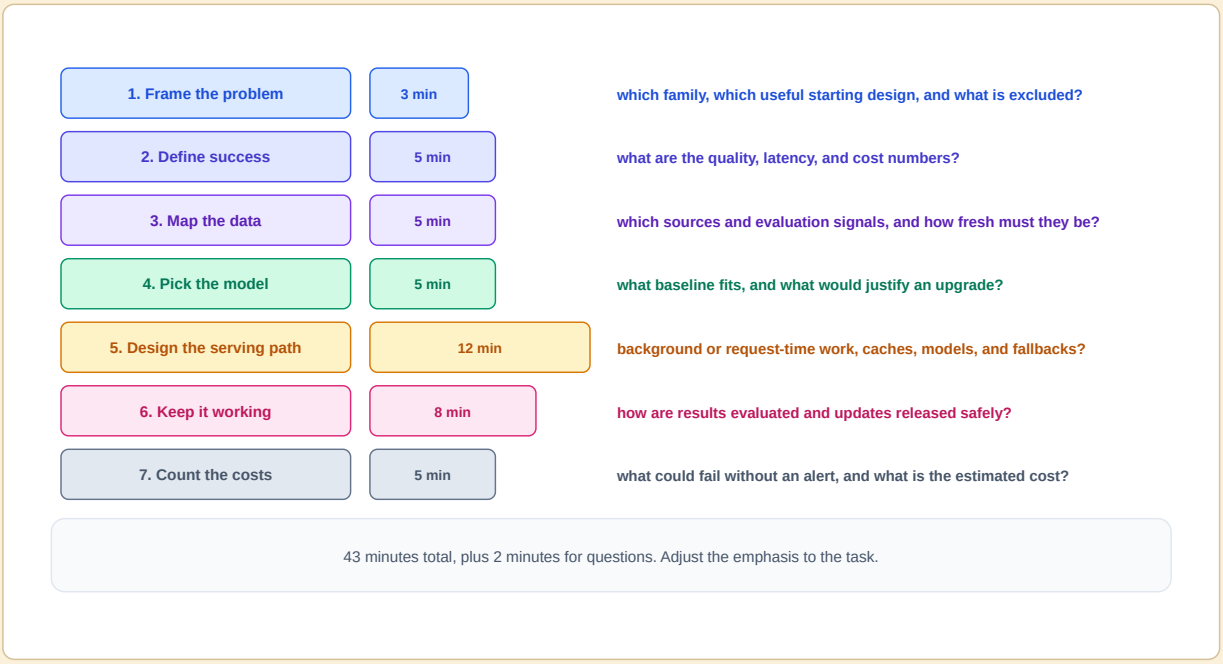
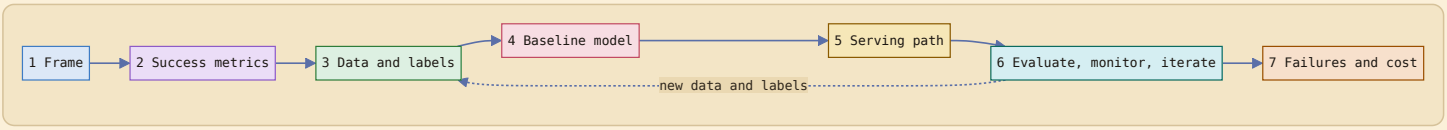


Figure — Seven steps with their guiding questions and minutes (course diagram)

Step	Question it answers	Minutes (45-min round)
1. Frame the problem	What family, what known system does it resemble, what's in/out of scope?	3
2. Define success	What are the quality, latency, scale and cost targets?	5
3. Map the data	Which sources, where do labels come from, how fresh must data be?	5
4. Pick the model	What's the simplest baseline, and what would justify upgrading?	5
5. Design the serving path	What runs offline vs per request, caches, cheaper paths, failure handling?	12
6. Keep it working	How are outcomes logged, quality measured, changes evaluated and rolled out?	8
7. Count failures and cost	What breaks, what controls exist, what does it cost vs budget?	5
Buffer	Questions	2

The same structure appears in Aminian & Xu's *ML System Design Interview* (clarify requirements → frame as ML task → data preparation → model development → evaluation → deployment/serving → monitoring) and in Chip Huyen's *Designing ML Systems* lifecycle (scoping → data engineering → model development → deployment → monitoring and continual learning → business analysis). Use whichever wording you like; cover every step. Shift time toward data and models for predictive questions, toward inference and context for generative ones, and toward permissions for agents.



Step 1 — Frame the problem (and frame it as an ML task)

1. **Name the family** with the output test.
2. **Name the known system** it resembles: "this is a recommender where items are podcasts", "this is document Q&A with stricter accuracy". Then say which requirements differ — don't paste a memorised design.
3. **State scope** in and out: "English text support first; voice and other languages later. OK?"
4. For predictive problems, **frame it as an ML task** — this is the heart of Aminian & Xu's step 2:
   - **Input and output** of the model (e.g. input = user, video, context; output = probability the user watches ≥ 30 s).
   - **ML category**: binary/multiclass classification, regression, ranking (learning-to-rank), retrieval, clustering, or generation.
   - **ML objective**: the quantity the model is trained to optimise, chosen because it moves the business metric.

Business metric → ML objective (worked examples)

Business goal / metric	ML task	ML objective (what the model predicts)	Why not the obvious one
Increase time spent on video platform	Ranking	P(watch ≥ X seconds) or expected watch time	Optimising raw clicks rewards clickbait
Reduce fraud losses without annoying customers	Binary classification	P(transaction is fraud) → thresholds set by expected cost	Accuracy is meaningless at 0.2% fraud
Accurate delivery promise at checkout	Regression	Minutes from order to delivery	Business also needs a *promise-kept rate*, handled by a display buffer, not the model
Improve marketplace search revenue	Ranking	P(purchase | query, item), possibly × price	Relevance-only ranking may ignore conversion
Reduce subscription churn	Binary classification	P(cancel within 30 days)	The label arrives 30 days late — shapes monitoring

Rules of ML adds two useful reminders: **Rule #12 — don't overthink which objective you directly optimise** at first, and **Rule #13 — pick a simple, observable and attributable metric** for your first objective (a click is observable and attributable to the item shown; "user happiness" is not).

Step 2 — Define success

Turn the goal into numbers; when the interviewer doesn't give them, state assumptions.

1. **Quality** — a *business / online metric* and an *offline ML metric* that correlates with it:
   - "Users find videos they enjoy" → online: watch time, CTR, with guardrails for satisfaction surveys and policy-violating content; offline: NDCG / recall@k on held-out interactions.
   - "Helpful support bot" → correct resolution rate, CSAT, repeat-contact rate.
2. **Latency** — a feed might need to load in a few hundred ms; a chatbot has a time-to-first-token target *and* a full-answer target; a batch job needs a completion deadline and a freshness target. Latency decides what must be precomputed.
3. **Scale and cost** — users, requests/day, peak QPS, cost per request or per *completed task*. A free feature needs a tiny per-request cost; a support bot can cost more per conversation if it genuinely resolves tickets. Use a concrete budget, never "near zero".

Rules of ML **#2: design and implement metrics first** — before building anything, log what you'll need to measure success.

Step 3 — Map the data

List data sources: interaction logs (clicks, watches, purchases), catalog/content data, user profiles (where use is permitted), documents for generation. Then answer two things:

1. **Label source.** A label is the target value for a supervised training example, and labels are often late or imperfect:
   - Fraud: confirmed investigations and chargebacks — slow (weeks) and not every chargeback is fraud.
   - Feeds: clicks — instant but only a proxy for satisfaction.
   - Support: reviewed conversations, verified resolutions; a thumbs-down is feedback, not a reliable correctness label.
   - Self-supervised learning and pretrained models reduce the need for hand-labelled data, but you still need labelled *evaluation* data.
2. **Freshness.** Daily may be enough for long-term tastes; trending content and fraud need minutes or seconds. Freshness decides batch vs streaming vs request-time computation.

Briefly name **feature groups** here (entity, recent-behaviour aggregates, request context, cross features — see Chapter 16.4), and flag **leakage** risk (features that wouldn't exist at prediction time) and **privacy** limits.

Step 4 — Choose a model, beginning with a baseline

Begin from a **baseline**: the least elaborate approach that satisfies the basic requirements. It tests the pipeline end to end, and it's the yardstick for every later model (*Rules of ML #4: keep the first model simple and get the infrastructure right*).

Family	Baseline	Upgrade path	Trigger to upgrade
Predictive	Popularity / rules / per-entity averages; logistic regression	GBDT on tabular features → deep models (two-tower retrieval, multitask rankers)	Baseline misses the quality target on the offline metric and the gain is worth the complexity
Generative	Existing model + clear instructions; add retrieval if answers need private/fresh data	Better retrieval, model routing, fine-tuning	Evaluation shows a specific failure that prompts/retrieval don't fix
Agentic	Fixed workflow if the steps are known; otherwise one agent with a few narrow tools	More tools, then multiple agents	The task clearly benefits from separation; extra coordination cost is justified

Always say the condition: *"If the baseline misses the target, I'll compare a GBDT against it on the same held-out weeks."*

Code — baseline first (`code/baseline_first.py`)

A synthetic delivery-time task: naive baseline → quick heuristic → GBDT, then translate model error into the business metric "orders delivered within the promised time" by adding a display buffer.

```
# 1) naive baseline: everyone gets the global average
rows.append(report("global mean", yte, np.full(len(yte), ytr.mean())))

# 2) heuristic: per-restaurant average + a distance term (shippable in a week)
rest_avg = {r: ytr[Xtr[:, 0] == r].mean() for r in np.unique(Xtr[:, 0])}
resid = ytr - np.array([rest_avg[r] for r in Xtr[:, 0]])
dist_fit = LinearRegression().fit(Xtr[:, [1]], resid)
heur = np.array([rest_avg.get(r, ytr.mean()) for r in Xte[:, 0]]) + dist_fit.predict(Xte[:, [1]])

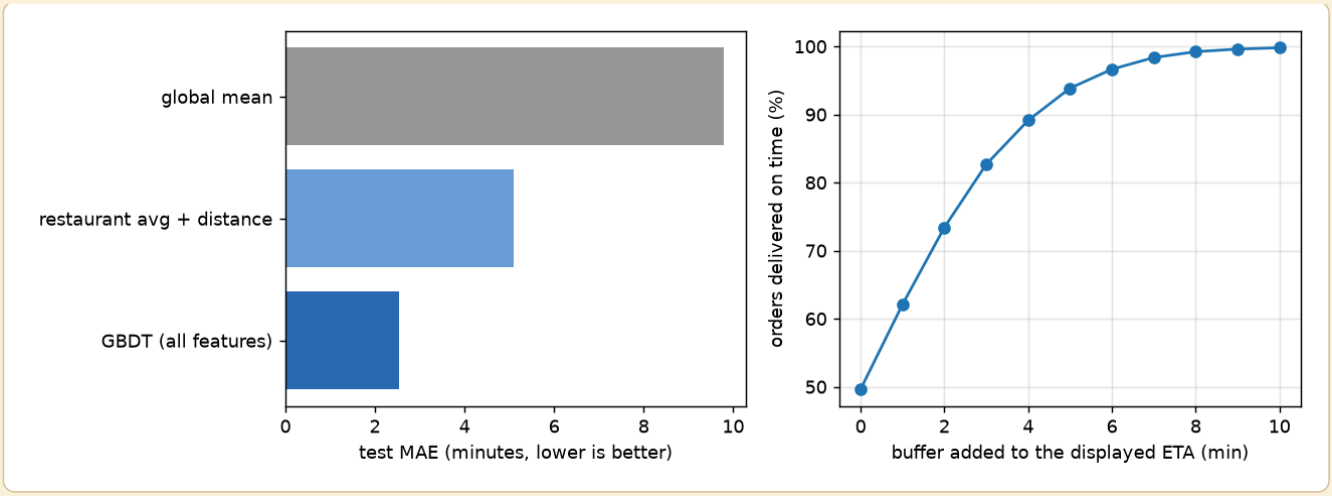
# 3) learned model: gradient-boosted trees on all features (restaurant id as categorical)
gbdt = HistGradientBoostingRegressor(random_state=0, categorical_features=[0]).fit(Xtr, ytr)
pred = gbdt.predict(Xte)

# business metric: % of orders that arrive within (prediction + buffer)
kept = [(yte <= pred + b).mean() * 100 for b in buffers]
```

Output:

```
model                          MAE  p90 abs err
global mean                   9.80        19.92
restaurant avg + distance     5.11        10.45
GBDT (all features)           2.54         5.31

buffer (min) -> promise kept % / avg promised minutes   [GBDT]
  + 0 min ->  49.6%  /  37.8
  + 3 min ->  82.7%  /  40.8
  + 5 min ->  93.9%  /  42.8
  + 8 min ->  99.2%  /  45.8
```



What it shows:

- Every number is judged **against a baseline**. The heuristic halves the naive error; the GBDT halves it again (on synthetic data — real gains are usually smaller).
- An unbiased model is late about **half the time** (49.6% kept with no buffer). The business metric depends on a **product decision** (the buffer), not only on model accuracy: +5 min keeps ~94% of promises but makes the displayed ETA look slower. Keep that knob outside the model so product can own it.

Step 5 — Design the serving path

Draw the architecture and trace one request. Whatever the shape (retrieval → ranking, retrieval → generation, agent loop), answer the same four questions:

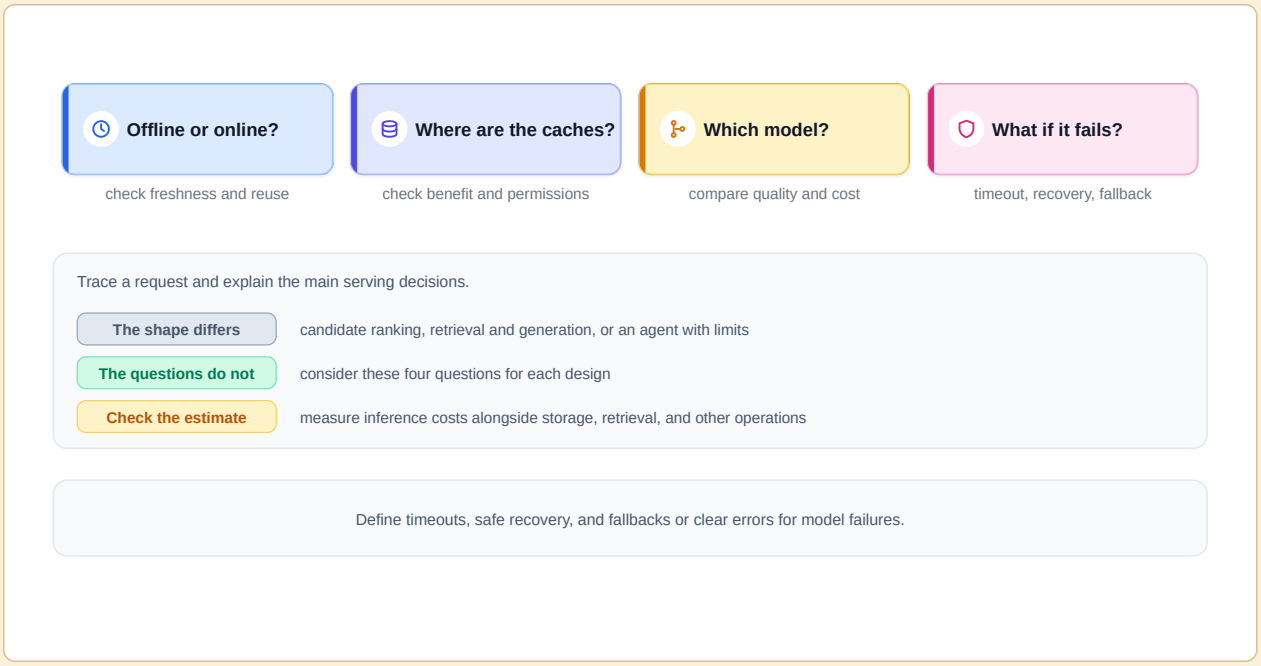


Figure — Step 5 checklist: offline/online, caches, model choice, failure (course diagram)

1. **Offline or online?** What can be precomputed (batch) and what needs request-time data? (Chapter 16.5.)
2. **Where are the caches?** What do they save in latency/cost, and what are the freshness, invalidation and **permission** rules so one user never sees another's cached result?
3. **Which requests can take a cheaper path** — a smaller model, cached answer, or rule — and which need the expensive one?
4. **What if the model is slow, down, or rate-limited?** Timeouts, bounded retries (only when safe), a fallback (previous model, cached score, heuristic, unpersonalised list) or a clear failure. For batch jobs: deadlines and recovery.

A typical predictive design for a large catalog is a **funnel**:



Step 6 — Keep it working (evaluation, monitoring, iteration)

Show the feedback process **in the diagram**, not just in words:

1. **Log** inputs, outputs, features used and model version (with privacy controls and retention limits).
2. **Capture outcomes**: clicks, ratings, escalations, confirmed fraud, actual delivery time.
3. **Measure** the step-2 metrics on live outcomes and on human-reviewed samples.

4. **Improve** data, features, retrieval, prompts or the model.
5. **Release safely**: offline evaluation → **shadow deployment** (new version scores copied traffic, outputs not shown; disable side effects) and/or a **controlled experiment** (A/B test) → gradual ramp with explicit stop/rollback conditions.

Offline vs online evaluation, in one table (details in Chapter 16.7 and 16.9):

	Offline evaluation	Online evaluation
Data	Held-out historical data (time-based split)	Live users
Metrics	ML metrics: PR-AUC, NDCG, recall@k, MAE, calibration	Business metrics: CTR, watch time, revenue, resolution rate + guardrails
Cost / risk	Cheap, no user risk	Uses real traffic, needs time and statistical power
Blind spots	Proxy labels, feedback loops, serving-time degradation	Slow, noisy, needs good experiment design
Role	Filter candidates	Decide the launch

Monitoring in one line: system health (latency, errors) → input feature distributions → prediction distributions → outcome quality (as late as labels arrive). Retrain on a schedule justified by measured decay, or after an investigation — never promote a model automatically just because an alert fired.

Step 7 — Failure modes and the bill

Failure list — specific to your system: unsupported answers, drifting data, unsafe actions, feedback loops that reinforce bad behaviour (optimising only clicks → clickbait). For each, a control (thresholds, quality monitors, content checks, permission limits, a kill switch) and what the control *cannot* guarantee.

Cost list — model cost ≈ requests × share hitting each model × calls per request × cost per call. Add retrieval, storage, retries and human escalation.

Code — napkin math (`code/napkin_math.py`)

```python
def qps(per_day, peak_factor=3.0):
    avg = per_day / 86_400
    return avg, avg * peak_factor

def model_cost_per_day(requests_per_day, share, calls_per_request, cost_per_call):
    return requests_per_day * share * calls_per_request * cost_per_call
```

Output:

```
10M req/day -> avg 116 QPS, peak ~347 QPS (3x peak factor assumed)
all requests x $0.01        -> $100,000/day, $3,000,000/30-day month
20% to large model x $0.01  -> $20,000/day (excludes small model, retrieval, retries)
support: 40,000 conversations/day x $0.10 = $4,000/day model cost
  compare per resolved issue, not per call: human ticket assumed $4.00
  resolve 30%: expected cost/ticket $2.90 vs $4.00 human-only
  resolve 50%: expected cost/ticket $2.10 vs $4.00 human-only
  resolve 70%: expected cost/ticket $1.30 vs $4.00 human-only
precompute 25M users x 500 recs = 12.5B rows; active-only (5M) = 2.5B rows
```

The important habit is the **unit**: compare cost *per resolved issue* (AI cost + human cost for the unresolved share), not "one model call vs one human ticket". All prices are illustrative assumptions, not quotes from any provider.

> Short on time? Squeeze steps 6–7 into two sentences: *"Every prediction gets matched to its eventual outcome; when quality moves I dig in, and any new version passes offline tests, then shadow or canary, before a staged ramp — with cost per request on the dashboard. The two risks I'd watch hardest are X and Y."*

Trade-offs of the framework

Benefit	Cost
Decisions follow requirements and data	Can feel slow if the interviewer wants the diagram now — then state assumptions in one sentence each and jump to step 5
Guarantees you don't forget evaluation/monitoring/cost	Rigidly following it when the interviewer steers elsewhere looks robotic — adapt order and depth

When not to use it rigidly

- "Skip to the architecture": state quality target and latency limit as assumptions, draw, and return to any open requirement at the moment it changes a choice.
- "You have 10 minutes": scope, main architecture, biggest risk, cost — and one sentence each on quality target, data, and evaluation.

In the interview

30-second opener: *"Plan: agree the task and its boundaries; put numbers on quality, latency and spend; find the data and where labels come from. Next a baseline and the serving path. I'll close with how we evaluate, roll out safely, handle failures, and what it all costs. To*

start — this reads like a ranking problem…"

- **"Why start with a baseline if you know GBDT will win?"** → It validates the pipeline and logging, gives a number to beat, and sometimes it's good enough (Rules of ML #1, #4).
- **"What's your ML objective, and why that one?"** → Name the prediction target, show how it moves the business metric, and name the proxy risk (e.g. clicks vs satisfaction) and the guardrail that catches it.
- **"How will you know it's working after launch?"** → Four monitoring layers, outcome join, per-segment quality, drift alerts that trigger investigation, gated retraining.

Self-check

1. Which steps are ML-specific additions to the classic framework? → Early quality definition (step 2's quality metric), data/labels (3), baseline model (4), and the keep-it-working loop (6).
2. Give a business metric → ML objective pair and its proxy risk. → Watch time → P(watch ≥ 30 s); risk: short-clip bias; guardrail: satisfaction surveys, long-term retention.
3. What are the four serving questions? → Offline vs online, caches (and permissions), cheaper paths, failure handling.
4. Why was the "no-buffer" promise kept only ~50% of the time? → An unbiased regressor's errors are roughly symmetric, so about half the deliveries are later than predicted.
5. What goes into model cost? → Requests × share per model × calls per request × price, plus retrieval, retries, storage and escalations.

TL;DR

Frame (3) → Success (5) → Data (5) → Baseline (5) → Serving (12) → Keep it working (8) → Failures & cost (5). Translate the business goal into an observable ML objective, beat a baseline, show the feedback loop, and put numbers on cost.

15.4 Capstone — scoping a vague AI question

The exercise

Hypothetical prompt (numbers are exercise assumptions):

> "We're a food-delivery company: ~25 million monthly users, 500,000 orders a day. Leadership wants AI to improve customer experience. Design something."

The feature, the success metric and the system family are all unspecified. Your job in the first ten minutes is to turn this into an agreed, measurable design problem — not to draw the architecture yet.

Step A — Generate options by family, then choose

Spend ~30 seconds listing options, grouped with the output test:

Family	Options
Predictive	Delivery-time (ETA) prediction; restaurant/dish recommendations; fraud detection on refund claims; predictions that feed courier assignment (the assignment itself also needs optimisation/rules)
Generative	Support assistant explaining order status and refund policy; AI-written dish descriptions; review summaries
Agentic	"Get me dinner for four under $60" ordering assistant that can place an order after checks and confirmation

Pick using **impact × feasibility × risk** (feasibility = can the team build and run it with the data, time and people available):

- **ETA prediction** — touches every order; delivered orders provide labels (check completeness).
- **Support assistant** — savings show up directly in support spend; the order records and policy documents it needs probably exist today; risk limited by restricting actions and escalating uncertain cases.
- **Ordering agent** — spends users' money; needs payment authorisation, confirmation and cancellation flows. Feasible, but more controls.

There is no universally right pick — say which you prefer under these assumptions and what would change your mind.

Step B — Run the framework opening (support assistant chosen)

Numbers are assumptions to confirm aloud.

1. **Quality**: primary = *correct resolution without human escalation*; also CSAT and repeat contacts for the same issue (a closed ticket isn't proof of a fix). Assume a human-handled ticket costs $4. Target: resolve ≥ 50% of eligible **tier-1** tickets (routine: order status, coupon questions, simple cancellations) with CSAT no worse than humans. Note: explaining the cancellation policy is one thing; actually *cancelling* is another, and doing things requires its own authorisation and rule checks.
2. **Latency**: stream the reply; assume p95 time-to-first-token ≤ 1 s and p95 completion ≤ 5 s for short answers (p95 = 95% of requests meet it); tool calls may get separate targets.
3. **Scale and cost**: 500,000 orders × 8% contact rate = **40,000 conversations/day** (average; peaks and repeats estimated separately). Assume 10 model calls per conversation at $0.01 → $0.10 per conversation → **$4,000/day** in model cost. Compare *total cost per correctly resolved issue*, including retrieval, infrastructure, retries and escalations — not one model call against one human ticket.
4. **Out of scope**: voice, non-English. Refunds above an assumed $10 go to a human; refunds below it still need identity, eligibility and duplicate-payment checks enforced **outside** the model.

Step C — Classify and preview the shape

"At its core this generates answers, grounded in order data the user is allowed to see and in retrieved policy text. Giving the model a refund tool adds an agentic slice, so reading data and moving money get separate safeguards. My deep dives: how accurate retrieval is, how we grade answers, what each conversation costs, and the backend checks that guard refunds of $10 or less."

Sample opening (paraphrased dialogue)

- **Interviewer:** "Use AI to improve customer experience."
- **You:** "Three candidates: ETA prediction, a support assistant, or an ordering agent — the agent needs payment and confirmation controls. I'd start with support: we can measure savings against current support costs and likely already have order data and policies. Support, or would you prefer ETA?"
- **Interviewer:** "Support works."
- **You:** "Target: correctly resolve at least half of eligible routine tickets, CSAT at least human-level, no rise in repeat contacts. Assuming $4 per human ticket, 500k orders and 8% contact rate gives ~40k conversations a day. For short answers, p95 first output within 1 s and completion within 5 s. At an assumed ten calls of one cent each, model spend is ten cents a conversation, and more once retrieval and infrastructure are added — so the number we track is total spend per issue actually resolved. Version one skips voice and languages other than English. A person handles any refund above $10; smaller ones still go through authorisation and policy checks. So: answer generation over orders and policies, plus a refund action. I'd spend the deep dive on retrieval, grading answers, spend, and guarding that action — agreed?"

Now you (10 minutes each, out loud)

Cover: options by family, one choice with reasons, quality/latency/scale/cost targets with assumptions flagged, in/out of scope, the family and key design decisions.

1. "Our music-streaming app wants listeners to discover more. Where does AI help?"
2. "Bank leadership has asked for something AI-powered in the mobile app." (Weigh the damage a wrong output could do: a bot that gives personal money advice is far riskier than one that explains why a purchase was put in a category, and forecasts or fraud warnings still need someone watching them. Don't assume any family is automatically safe — say what the feature may do, must not do, and when a human reviews.)
3. "Candidates and jobs on our job board are poorly matched. Fix it with AI."

> **TL;DR**
>
> Vague prompt → list options by family → choose on benefit, data, feasibility, risk → state quality/latency/scale/cost numbers in comparable units → confirm scope, name the family and the key decisions.

15.5 Common mistakes (the whole interview)

Mistake	Better
Jumping to "I'd use a transformer" in minute 2	Frame, define success and data first; baseline before deep models
Optimising a proxy with no guardrail (clicks → clickbait)	Pair the objective with guardrail metrics (satisfaction, retention, policy violations)
"We'll have labels"	Say where labels come from, how late, how noisy, who pays
Random train/test split on time-dependent data	Time-based split; point-in-time-correct features (Rules of ML #33)
Accuracy on imbalanced data	Precision/recall at a threshold, PR-AUC; threshold owned by product/risk
No fallback	Timeouts + fallback prediction chosen with product and risk owners
Only the request path	Logging → outcome join → evaluation → gated retraining → staged rollout
Training features computed differently from serving features	One feature definition, log serving features and train on them (Rules of ML #29, #32)
"GPU cluster" for a small tabular model	Benchmark on CPU first
Retraining automatically on every drift alert	Alert → investigate (often a broken pipeline) → validated retrain → gates
Hand-waving cost	Requests × share × calls × price, compared per completed task
Ignoring the interviewer's redirect	Adapt order and depth; the framework is a checklist, not a script

Chapter TL;DR

- An ML design answer = classic system design **plus** quality metrics, a data/label story, a baseline-first model plan, a feedback loop, and cost realism.
- Identify the **form** (known product, GenAI, add-AI, agent, critique) and the **family** (prediction, content, action) in the first minute; confirm depth.
- Seven steps in ~43 minutes: frame (3), success (5), data (5), model (5), serving (12), keep-it-working (8), failures and cost (5).
- Translate the business metric into an observable ML objective, and keep product knobs (buffers, thresholds) outside the model.
- Every number means something only next to a baseline; every design needs a fallback; every cost goes in the same unit as the benefit.

- Next: Chapter 16 builds each production piece — training vs serving, labels, feature stores, batch vs online inference, serving, offline metrics, monitoring and drift, A/B testing and MLOps — and assembles them into an ETA platform.

Chapter 16 — ML in Production: Pipelines, Serving, Evaluation, Monitoring and MLOps

Fitting a model in a notebook is a small slice of the job. The rest is machinery: trustworthy labels, identical features for training and live traffic, deciding when predictions are computed, fast serving with a fallback, the metric that proves one model beats another, alarms for quiet decay, and safe releases. This chapter builds each piece for **predictive** models (scores, labels, numbers), then assembles them into a delivery-time (ETA) platform.

What you'll learn

1. The five pieces of a predictive ML system and the offline/online divide.
2. Training vs serving, and training–serving skew (leakage demo).
3. Labels: natural, behavioural, human, LLM-assisted; imbalance and recalibration; negatives; splits; privacy.
4. Feature groups, encodings, batch vs streaming features, feature stores, request-time features.
5. Batch, online and streaming inference; precompute-then-rerank.
6. Serving: latency budgets, batching, caching, rollout, hardware, compression, fallbacks.
7. Offline metrics for classification, ranking and regression; thresholds; calibration; baselines.
8. Monitoring, drift (PSI, KS), retraining, feedback loops.
9. A/B tests: power, peeking, guardrails, interleaving, switchbacks, bandits.
10. MLOps: tracking, registry, data validation, CI/CD/CT, shadow and canary.
11. Capstone: an ETA prediction platform.

Embeddings and two-tower retrieval belong to Chapter 13. All code is in `code/`; outputs below are real.

16.1 The five pieces and the offline/online divide

What it is

Recommenders, fraud detectors and ETA predictors all decompose into five parts:

Piece	Job
1. **Training pipeline**	Builds a dataset, fits a model and scores it, on a schedule or when triggered
2. **Feature layer**	Code plus storage converting raw events into model inputs
3. **Inference path**	Everything that happens between a request and its score
4. **Serving platform**	The machines, routing and autoscaling behind live traffic
5. **Monitoring loop**	Signals that reveal degradation and help you diagnose it

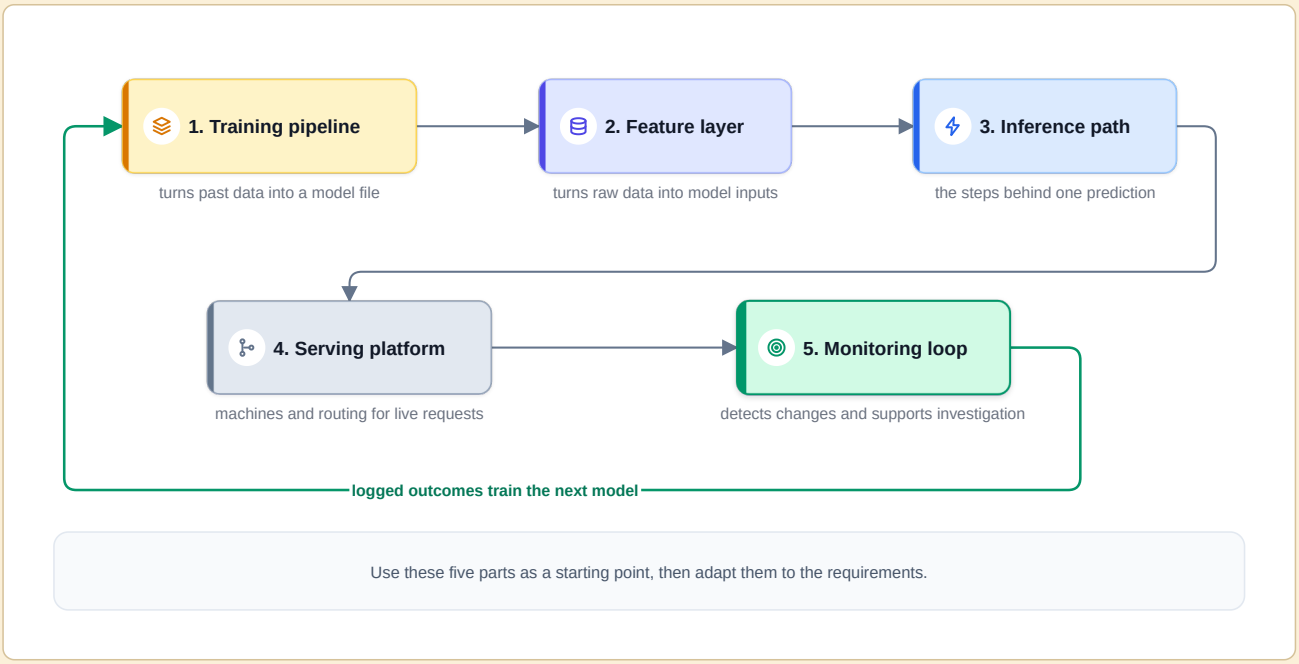


Figure — Five building blocks of a predictive ML system (course diagram)

How it works — one prediction, start to finish

Delivery-time estimate at checkout:

1. The customer opens checkout; the app calls the prediction service.

2. The service gathers features — some from storage, some computed from the request.
3. It arranges them in exactly the order and format the model expects.
4. The model returns a number, say 34 minutes.
5. Business rules adjust it (e.g. add a buffer) and it's shown.
6. The service logs features, prediction, model version and order id.
7. ~40 minutes later the food arrives — the true delivery time (the **label**) is now known.
8. Overnight, the training pipeline joins logged features with actual times.
9. A fresh candidate is fitted and evaluated; it replaces the current model only if it wins.

Steps 1–6 live inside a **latency budget** — the product decides how many milliseconds the whole prediction path may take. Steps 7–9 run hours or days afterwards, and that gap is the thread running through this chapter.

The offline/online divide

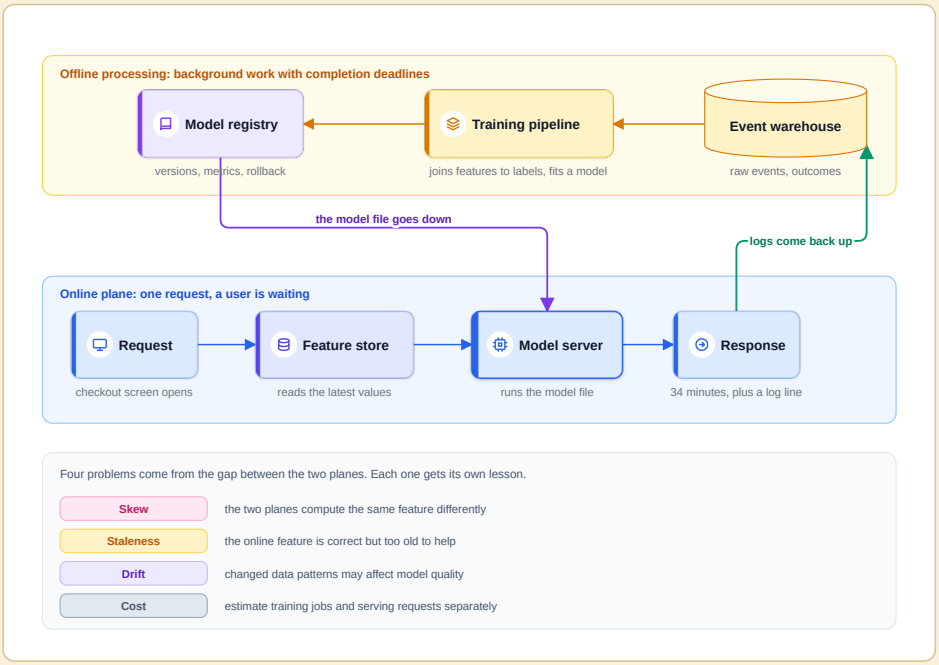


Figure — Offline and online planes: model goes down, logs come up (course diagram)

- **Offline plane**: background jobs over history (training, batch inference) — throughput and completion deadlines.
- **Online plane**: live requests under a response-time limit.
- The **model artifact** crosses offline → online; **prediction logs + outcomes** cross back. Schemas, feature definitions and config are shared.

Four problems live at this boundary:

Problem	Meaning	Section
Skew	A feature is computed differently in training and serving	16.2
Staleness	A feature is correct but old (hourly average when the kitchen got busy 10 min ago)	16.4
Drift	Data or input→outcome relationships change; quality falls while the service is "up"	16.8
Cost	Training cost (job size × frequency) and serving cost (traffic × hardware × utilisation) are separate and both need estimates	16.5, 16.6

Vocabulary: a **feature** is a model input derived from raw data (order events are raw; "orders this restaurant took in the past half hour" is a feature). A **label** is the outcome being predicted — delivery minutes, a click — and may be imperfect (a chargeback is not always fraud). A **training example** is one row of features plus its label.

Self-check

1. Which two things cross the offline/online boundary? → The model artifact (to serving) and prediction/outcome logs (to training).
2. Why can't the label be used at prediction time? → It doesn't exist yet — the delivery hasn't happened.
3. Name the four divide problems. → Skew, staleness, drift, cost.

TL;DR

Five pieces; two planes; model goes down, logs come back up; skew, staleness, drift and cost live at the boundary.

16.2 Training vs serving

What it is

Two systems with different jobs that share artifacts. **Training** consumes history and emits a model artifact; **serving** picks up that artifact and responds to live traffic.

	Training	Serving
Runs	Scheduled (nightly/weekly) or triggered	Continuously, per request
Optimises for	Throughput over large datasets	Latency and availability
Hardware	Big GPU or high-memory clusters; cheap spot machines work if the job checkpoints and resumes	Always-on CPUs or small GPUs, sized reliably for the SLO
When it breaks	The new model is late and the previous one stays live (how urgent depends on how quickly it goes stale)	Users see errors and on-call is paged — a safe fallback is required
Data	Months/years of history	This request's inputs, now

Draw them as two separate groups from the start.

How it works — the training pipeline

A scheduled workflow (Airflow, Kubeflow Pipelines, Dagster, cloud equivalents) with four stages:

1. **Data preparation** — read raw events from the warehouse or lake and assemble rows that pair feature values **as of the prediction moment** with the label that showed up later. Test this hard: one wrong join quietly poisons everything downstream.
2. **Training** — for tabular data a **GBDT** (gradient-boosted decision trees: many small trees, each fixing the previous ones' errors) is the strong default; a modest neural net is the other common choice. Too much data or time for one machine → distributed training: **data parallelism** (examples split across workers, each holding the full model) or **model parallelism** (the model itself split across devices).
3. **Evaluation** — score the candidate on **held-out** data and compare with production on quality, latency, cost and safety. Equally good but cheaper is also a valid release.
4. **Registration** — store the approved model in a **model registry** with version, metrics and **lineage** (which data and code produced it). Rollback needs compatible features, dependencies and config too — not just the old file.

Propose a retraining cadence with a reason ("daily — labels arrive within the hour, demand shifts weekly") and say you'll validate it against label availability, measured decay and compute cost.

How it works — the serving path

Request → features fetched/computed → model runs → prediction returned and logged. Budget the **entire** path; with small models, fetching features often takes longer than inference. Service hygiene still applies: **timeouts**, a **fallback** score (default or cached), horizontal scaling, versioned releases.

One product decision to state explicitly: does the model **fail closed** (block when unsure/unavailable — safer for high-risk payments) or **fail open** (allow — better customer experience)? Name it and why.

Training–serving skew

Skew means the inputs at serving time differ from the inputs the model learned from. Quality drops while the error logs stay empty. Three sources:

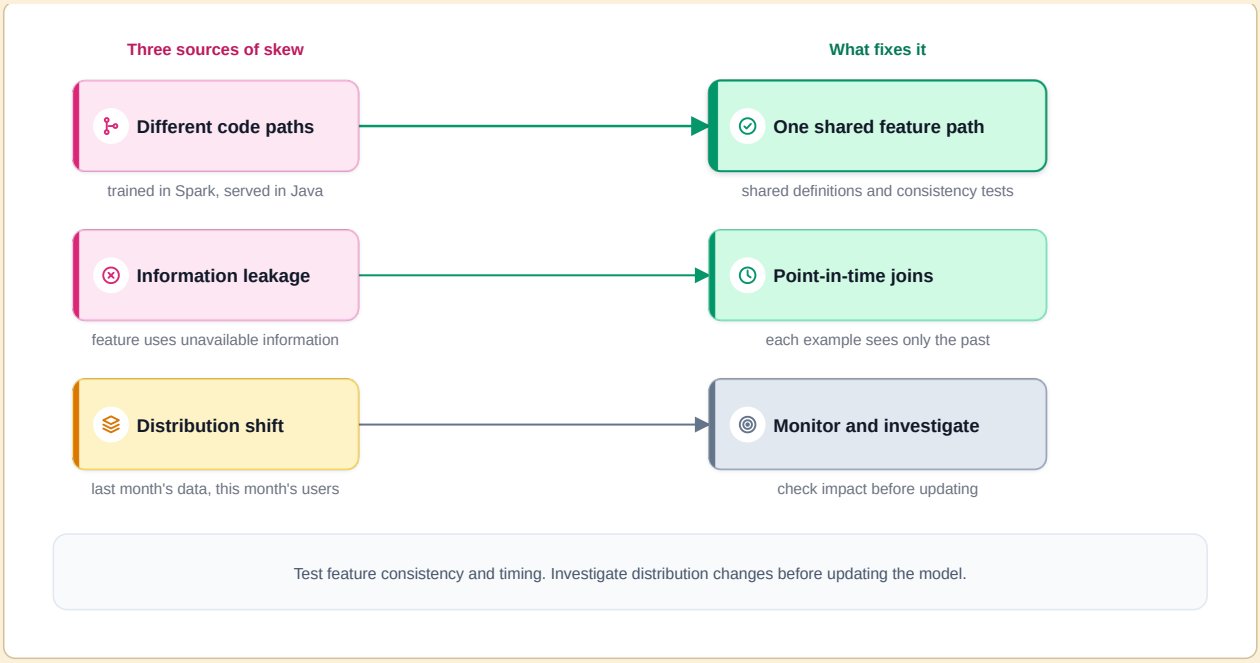


Figure — Skew: causes and remedies (course diagram)

Source	Example	Fix
Different code paths	Spark/Python feature for training, rewritten in Java for serving, with a different null default, rounding or time zone	Shared, tested feature definitions; consistency tests comparing both paths
Information leakage	A "today's average delivery time for this restaurant" feature built from the full day's orders, so it already contains the order being predicted	**Point-in-time-correct joins**: each example sees only data that existed at its timestamp
Distribution shift	Trained on last month, serving this month	Not a bug, same symptom — monitoring and retraining (16.8)

Rules of ML: **#29** — log the features used at serving time and train on those logs; **#32** — reuse code between training and serving; **#37** — measure training/serving skew. Logs can still be incomplete or mis-joined, but this removes the biggest source.

Code — leakage vs point-in-time join (`code/point_in_time_join.py`)

The as-of join uses `pandas.merge_asof`: for each order, take the running mean of same-restaurant, same-day orders that had **finished before** this one was placed.

```
def pit_feature(df):
    done = df[["rest", "day", "done_h", "minutes"]].sort_values("done_h").copy()
    done["cum_sum"] = done.groupby(["rest", "day"])["minutes"].cumsum()
    done["cum_n"] = done.groupby(["rest", "day"]).cumcount() + 1
    left = df[["rest", "day", "placed_h"]].reset_index().sort_values("placed_h")
    j = pd.merge_asof(left, done[["rest", "day", "done_h", "cum_sum", "cum_n"]],
                      left_on="placed_h", right_on="done_h", by=["rest", "day"],
                      direction="backward", allow_exact_matches=False)   # strictly before
    j = j.set_index("index").sort_index()
    return (j["cum_sum"] / j["cum_n"]).to_numpy()     # NaN = nothing finished yet today

df["avg_today_LEAKY"] = df.groupby(["rest", "day"])["minutes"].transform("mean")  # whole day!
df["avg_today_PIT"] = pit_feature(df)
# ... temporal split (last 12 days = test), fit one GBDT per feature version
```

Output:

```
   placed_h  rest  minutes  avg_today_LEAKY  avg_today_PIT
0      10.0    20     31.3             34.1            NaN
1      10.0    31     36.6             44.5            NaN
...
PIT feature missing (no finished orders yet today): 15.0%

setup                                    MAE (min)
distance only (no daily feature)         5.88
LEAKY model, offline test (looks amazing)  3.60
LEAKY model, fed real serving features     4.91
PIT model, offline = what serving will see  4.67
```

The leaky model wins offline (3.60) but, fed the values that exist at serving time, it's *worse* (4.91) than the point-in-time model (4.67), whose offline number honestly forecasts production. The honest feature is **missing** 15% of the time (first orders of the day), so the model must learn to handle that.

Trade-offs

Choice	Benefit	Cost
Frequent retraining	Fresher model	Compute, evaluation load, release risk
Train on logged serving features	Removes most skew	Log volume, privacy review, no logs yet for new features
Separate training/serving teams	Specialisation	Handoff skew — needs shared definitions

Common mistakes

- Evaluating on a random split of time-ordered data.
- Rewriting feature logic for serving "because it's faster in Java".
- No fallback: the checkout hangs when the model service is slow.

In the interview

> **30-second answer** ("walk me through the ML architecture"): *"Offline, a scheduled job assembles examples with features frozen at prediction time, fits a candidate, compares it with what's live, and registers it with lineage. Online, a stateless service computes features from the same definitions, calls the model within a time budget with timeout and fallback, and logs features, output and version. Artifacts travel to serving; logs and outcomes travel back."*

- **"Why not train continuously?"** → Freshness value vs label delay, compute and evaluation burden; online learning needs guards (16.8).
- **"First model with no logs?"** → Historical data if any; otherwise a rule-based baseline that starts collecting data — with selection bias (you only see outcomes for what the rules allowed).

Self-check

1. What's a point-in-time-correct join? → Joining each example only with feature values that existed at its prediction timestamp.
2. Why did the leaky model look better offline? → Its feature partly contained the label (the day's average included this order and later ones).
3. What does a registry store besides the file? → Version, metrics, lineage (data + code), stage.
4. Fail open vs closed for card fraud? → Risk appetite: often open for small amounts with rules as backstop, closed/review for high-risk ones — say who decides.

TL;DR

Four-stage pipeline ending in a registry; serving with budget, timeout, fallback, logging; kill skew with shared definitions, point-in-time joins and training on logged serving features.

16.3 Where training data comes from

What it is

A supervised model is only as good as the pipeline that attaches an outcome to each feature row. Expect probes on the labels' source, delay and cost.

Source	Examples	Cost	Delay	Main weakness
Natural labels (observed outcomes)	Delivery duration, chargeback, cancellation	Logging + validation	Minutes to weeks	Late, incomplete; a chargeback is evidence, not proof
Behavioural labels	Clicks, skips, saves, watch time	Logging + processing	Often seconds	Indirect (click ≠ satisfaction); only covers what the system chose to show
Human annotation	Relevance grades, moderation decisions	Roughly a few cents up to a few dollars each	Hours to days	Budget caps how much gets labelled; reviewers need quality control

Chip Huyen calls the prediction-to-label gap the **feedback loop length** — short for ETAs and clicks, long for fraud and churn. Label delay doesn't stall training; it means only "mature" examples (old enough to have a label) are used, and outcome monitoring lags by the same amount.

Human labelling is a system, not a task

- **Versioned guidelines** — a label means what the guideline says; record its version in dataset lineage, because redefining "spam" breaks metric comparisons across that change.
- **Inter-annotator agreement** — several reviewers label the same items (e.g. Cohen's κ for two raters). Low agreement usually signals an unclear guideline, hard items or missing context; high agreement ≠ correctness.
- **Gold questions** — items with known answers seeded into the queue track each reviewer's accuracy over time.
- **Active learning** — spend labels where they're worth most: items the model is least sure about, disagreements, new product areas.
- **Reuse existing queues** — fraud analysts already make decisions; record context and uncertainty, validate and version before training on them.
- **Weak supervision** (e.g. Snorkel-style labelling functions — heuristics that vote) yields many noisy labels cheaply; audit them the same way.

LLM-assisted labelling

Given the written guideline, an LLM proposes labels fast. In production it's **tiered**, not a replacement: the LLM labels eligible items, humans check a random sample plus low-confidence or contested ones, and accuracy is measured against human-reviewed sets **per important subgroup**. The catch: LLM errors are **correlated** — one blind spot repeated a million times (dialects, specialist topics, adversarial content). Keep human audit slices, re-measure whenever prompt, model or policy changes, and check privacy rules before sending data to any model.

Class imbalance

Suppose 0.2% of transactions are fraud (1 in 500): always predicting "legit" scores 99.8% accuracy, and most compute goes on easy negatives.

- **Downsample negatives**: keep every positive and a fraction *w* of negatives (say 10%). Data shrinks ~10×; positive share rises from 0.2% to ~2%.
- **Scores are now inflated** (fraud looks ~10× more common). If scores are used as probabilities (dollar thresholds, auctions), **recalibrate**: the odds were multiplied by 1/w, so **p = w·q / (w·q + 1 − q)**, where q is the downsampled model's score — or fit Platt/isotonic calibration on held-out data at the real base rate. (Rules of ML **#30**: importance-weight sampled data, don't drop it arbitrarily.)
- **Reweighting** is the alternative: keep all data but weight positives more in the **loss function** (the quantity training minimises). More compute, no resampling.
- Evaluate with **precision and recall**, not accuracy (16.7).

Code — downsample, recalibrate, check (`code/imbalance_calibration.py`)

```
w = 0.10
keep = (ytr == 1) | (rng.random(len(ytr)) < w)              # all fraud, 10% of legit
model = LogisticRegression(max_iter=1000).fit(Xtr[keep], ytr[keep])
q = model.predict_proba(Xte)[:, 1]
p_corrected = w * q / (w * q + 1 - q)                       # undo the 1/w odds inflation

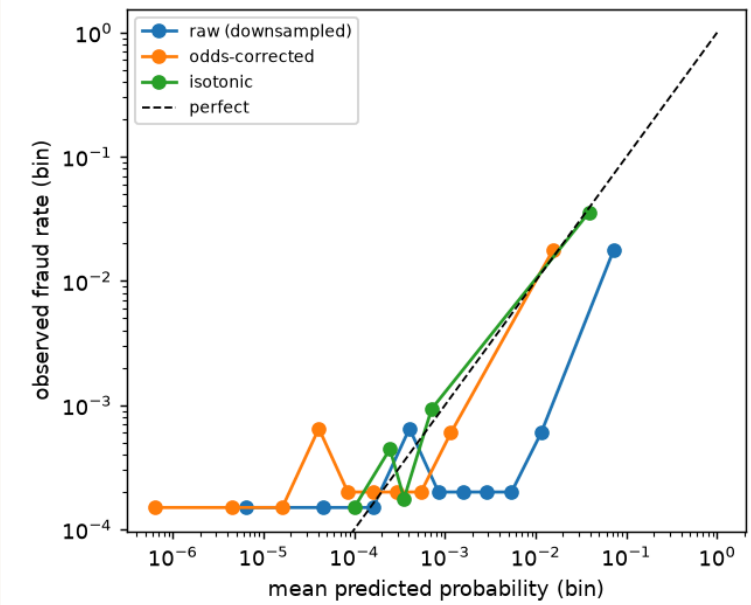
# or learn the mapping on held-out data with the true base rate (scikit-learn >= 1.6)
cal_platt = CalibratedClassifierCV(FrozenEstimator(model), method="sigmoid").fit(Xcal, ycal)
cal_iso = CalibratedClassifierCV(FrozenEstimator(model), method="isotonic").fit(Xcal, ycal)
```

Output:

```
base rate: 0.200%
'always legit' accuracy: 99.80%  (catches 0 fraud)

downsampled train: 30,732 rows (was 300,000), positive share 1.94%

scores                         mean  PR-AUC     Brier  logloss      ECE
(true rate)                  0.0020
raw (downsampled)            0.0095   0.653  0.001610   0.0119  0.00758
odds-corrected (w=0.1)       0.0018   0.653  0.001093   0.0063  0.00044
Platt on held-out            0.0021   0.653  0.001016   0.0062  0.00038
isotonic on held-out         0.0021   0.624  0.001020   0.0061  0.00020
-> ranking (PR-AUC) is identical for raw and corrected: calibration is a separate property
```



Reading it: raw scores average 0.95% against a true 0.2% (≈5× too high); the formula and Platt restore ~0.2% and nearly halve log-loss. Monotonic fixes leave PR-AUC unchanged (ranking ≠ calibration); isotonic's small drop comes from ties in its step function. (`FrozenEstimator` replaced `cv="prefit"` in scikit-learn 1.6.) **ECE** (expected calibration error) = weighted average gap between mean score and observed rate per bin; **Brier** = mean squared error of probabilities.

Manufacturing negatives (retrieval)

Logs show clicks, never the vast catalogue a user ignored, so retrieval models (two-tower, Chapter 13) need manufactured negatives: **random** (easy, teaches little about close calls); **in-batch** (other users' positives in the batch — cheap, but popular items are over-represented, so apply a **popularity/sampling-bias correction** or the model learns to demote popular items); **hard** (shown-but-not-clicked, or near neighbours — sharpen the boundary, but some are false negatives). Mix them and tune the ratio.

Split hygiene and the leakage checklist



Figure — Time-based vs random split (course diagram)

- **Temporal splits** for future performance (nearly always): train weeks 1–10, validate 11, test 12. A random split lets the model train on week 12 and "predict" week 3. (Rules of ML **#33**: trained through Jan 5 → test from Jan 6.) Generalising to new users needs **entity-based splits** (all of a user's rows in one split).
- **Point-in-time features** within every example (16.2).
- Leakage checklist:
  1. Aggregates whose window includes the label moment ("merchant fraud rate" including this transaction).
  2. **Label-adjacent fields** that only get filled because the outcome happened (refund flag, case-resolution code).
  3. Near-duplicates across splits (same document twice; decide whether the same user may appear in both, based on what you're testing).

Privacy, in one paragraph

Minimise **PII** (personally identifiable information) at ingestion: collect only what's needed, restrict access, set retention. Hashed IDs are still linkable. Define how deletion requests reach raw data, derived datasets, embeddings and models (versioned data, deletion tracking, scheduled rebuilds — and involve privacy/legal). Fraud verdicts and moderation decisions are sensitive too.

In the interview

> **30-second answer** ("where does your training data come from?"): *"Outcomes, behaviour and human judgments, each with a cost and delay. I validate labels, join them to point-in-time features, split by time, downsample and recalibrate for imbalance, and audit LLM labels against human slices."*

- **"Fraud outcomes take six weeks. What do you train on today?"** → Matured rows (the window stops six weeks back), plus provisional signals — rule hits, analyst verdicts — for recent traffic. Outcome monitoring lags six weeks; score-distribution monitors cover the gap.
- **"Annotators disagree 30% of the time."** → A guideline bug first: cluster disagreements, add examples, re-version, re-measure. For truly ambiguous items, collect several judgments and train on the majority or the distribution.
- **"Why not let an LLM label everything?"** → Its errors are correlated and invisible from inside; human audit slices catch them.

Self-check

1. Three label sources and their main weakness? → Natural (late), behavioural (biased proxy), human (costly, needs QA).
2. After keeping 10% of negatives, a model outputs 0.05. Corrected probability? → $0.1 \cdot 0.05 / (0.1 \cdot 0.05 + 0.95) \approx 0.0052$.
3. Why popularity-correct in-batch negatives? → Popular items appear as negatives too often, so the model learns to penalise popularity itself.

4. Name two leakage patterns. → Windows that include the label time; label-adjacent fields.

TL;DR

Outcomes (slow), behaviour (fast, biased), humans (costly, need QA). Downsample then recalibrate. Split by time, join point-in-time, dedupe, minimise PII.

16.4 Features and feature stores

What it is

A **feature store** computes, stores and serves features consistently for training (history) and serving (latest values). For predictive designs, say for each feature what it is, where it comes from, how fresh it must be and what it costs.

How it works — four feature groups (ETA example)

Group	Meaning	ETA examples	Typical compute path
Entity	Attributes of a restaurant, courier or customer that rarely change	Typical prep time at the restaurant, the courier's vehicle, the customer's building type	Batch (nightly)
Behavioural aggregates	Counts/averages over a recent window	Restaurant orders last 30 min, courier speed today, dish median prep over 7 days	Streaming for short windows, batch for long
Contextual	The request moment	Time, weekday, weather, distance, big match on TV	Request time
Cross	Two entities together	This courier at this restaurant; this customer in this cuisine	Batch or streaming

A second checklist, by **signal source**, helps find gaps: the item's content, the person acting, their recent behaviour, graph links between entities, and the circumstances of this particular request.

Turning raw signals into model inputs

Kind of input	Usual encoding	Example
Huge ID vocabularies	Learned **embedding tables** (Chapter 13); when the set of IDs is unbounded, the **hashing trick** maps each ID to one of a fixed number of buckets	user, item, merchant IDs
Numbers	Bucketing, normalisation, and a log transform when a few values are enormous and most are small (**heavy tails**)	order counts, km, prices
Short category lists	One-hot vectors or tiny embeddings	device type, country
Pairs that interact	Hand-built crosses, or a deep model that discovers them	user × category

Trees don't need scaling; linear models and neural nets do.

Freshness: batch path + streaming path

Short-window aggregates are worth having only while they're current: a half-hour order count refreshed once a night tells the model nothing. So the feature layer may have two arms:

- **Batch path** — Spark/SQL jobs compute slow features for every entity (e.g. nightly); easy to backfill.
- **Streaming path** — Flink or Spark Structured Streaming reading Kafka keeps windowed aggregates current within seconds; more operational work.

This is the **lambda architecture** (Nathan Marz); a streaming-only design is **kappa**. Run both arms only when freshness requires it. Rules of ML **#8: know the freshness requirements of your system**.

What a feature store actually is

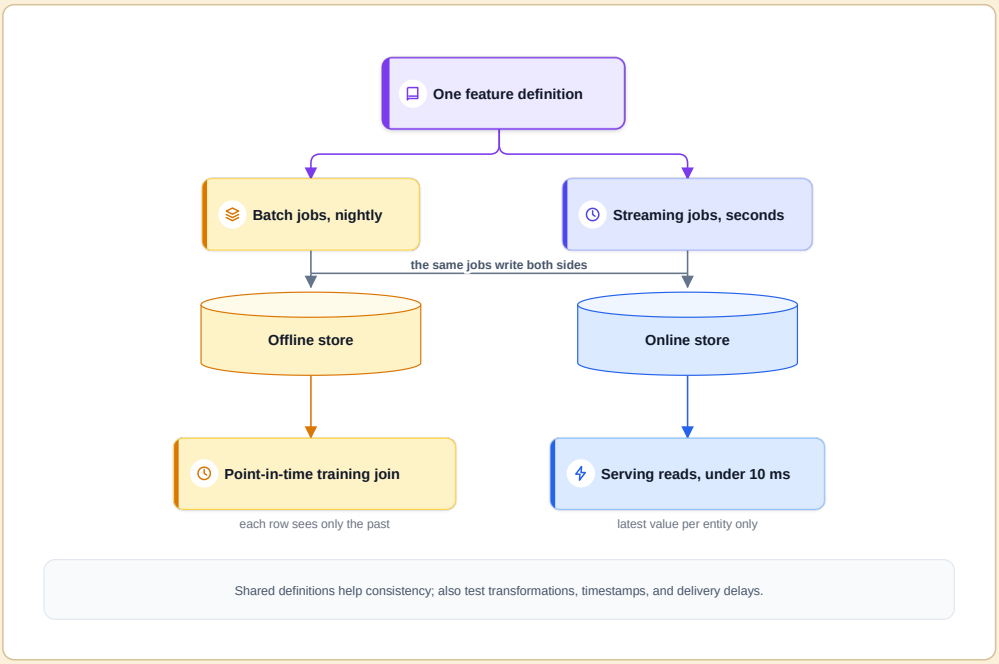


Figure — Feature store: shared definitions, offline history, online latest (course diagram)

Side	Storage	Holds	Used for
Offline store	Warehouse / lake (Parquet, BigQuery, Snowflake…)	Timestamped history of feature values	Training sets via **point-in-time joins** — a row from 6:02 pm on 3 March gets the values as of then, not today's
Online store	Low-latency key-value store (Redis, DynamoDB, Cassandra, Bigtable)	Just the **current** value of each feature for each entity	Serving: "40 features for restaurant 812, courier 3407, this route — in < 10 ms"

One set of definitions feeds both sides, and the store doubles as a **registry** so teams reuse a feature instead of re-deriving a slightly different one (which recreates skew). Still test transforms, timestamps, late data and empty values. Products: **Feast** (open source), **Tecton**, **Databricks**, **SageMaker** and **Vertex AI** feature stores; Uber's **Michelangelo** popularised the pattern.

Request-time features

Some values exist only at request time (route distance, current cart); serving computes them inline. The trap is an expensive call (routing service, third-party API) with no latency plan. If it eats 80 ms of a 100 ms budget:

1. **Cache** a recent answer.
2. **Approximate** (straight-line instead of road distance).
3. **Default** a "missing" value on timeout.

Train and evaluate with those degraded inputs so the fallback is rehearsed and its quality cost known. As a budget line: *"15 ms to assemble features — a single batched lookup and two computed inline; if any of them is late, it takes the default value the model already met during training."*

Trade-offs

	Batch features	Streaming features	Request-time features
Freshness	Hours	Seconds	Instant
Cost/complexity	Low	High (state, late events, backfills)	Latency on the critical path
Backfill for training	Easy	Harder	Must be logged at serving

Common mistakes

- Computing 7-day aggregates at request time (millions of events scanned per call).
- Silent staleness: the stream falls behind and no alarm goes off. Put **feature freshness** on the main dashboard with its own alert, and consider feeding "feature age" to the model.
- A second team re-implements a feature → skew.

In the interview

> **30-second answer**: *"Entity features in nightly batch, short-window aggregates in streaming, request context inline under a budget. A feature store gives point-in-time history for training and latest values for serving from one definition, and I'd test that both paths agree."*

- **"Why not compute everything per request?"** → Too slow and costly. Precompute what tolerates it, stream what must be current, and leave for request time only values that can't be known sooner.

- **"The streaming job lags?"** → Alert on freshness, fall back to trained-in defaults, include feature age.

Self-check

1. Which store serves a point-in-time training join? → Offline store.
2. What does the online store keep? → Latest value per feature per entity.
3. Three fixes for a slow request-time feature? → Cache, approximate, default — all rehearsed in training.

TL;DR

Four feature groups; batch for slow, streaming for fresh, inline for request-only; feature store = offline history + online latest; budget request-time features.

16.5 Batch vs online vs streaming inference

What it is

Batch inference precomputes predictions on a schedule into a key-value store, so a request becomes a lookup. **Online inference** computes per request. **Streaming inference** scores events as they arrive through a queue.

	Batch	Online
When computed	On schedule, for all (or active) entities	Per request
Request-time work	KV lookup, ~1 ms	Feature fetch + model, ~10–100 ms
Infra	Scheduled job + KV store	Always-on serving capacity, autoscaling
Freshness	As old as the last run	As fresh as the inputs
Can use request context?	No	Yes (query, cart, route)
Cost shape	Batch size × reuse × hardware	Scales with traffic, peaks at busy hours

How it works — the deciding question

Does the prediction need information that only exists at request time? Then weigh freshness and reuse.

- ETA: yes (route, cart, live queue) → online.
- Fraud scoring: yes — the transaction *is* the input → online (or streaming).
- "Videos you might like" shelf: mostly no → batch (last night's top-500 is fine).
- Search ranking: query is request-time, but per-item quality scores can be precomputed → hybrid.

Waste: 25M users × 500 recommendations = 12.5 billion rows, mostly unread if 80% don't visit this week. Mitigations: precompute for **active users only** (5M → 2.5B rows), keep **fewer items per user** and reorder them cheaply online, or precompute per **segment** instead of per user.

The hybrid: precompute-then-rerank

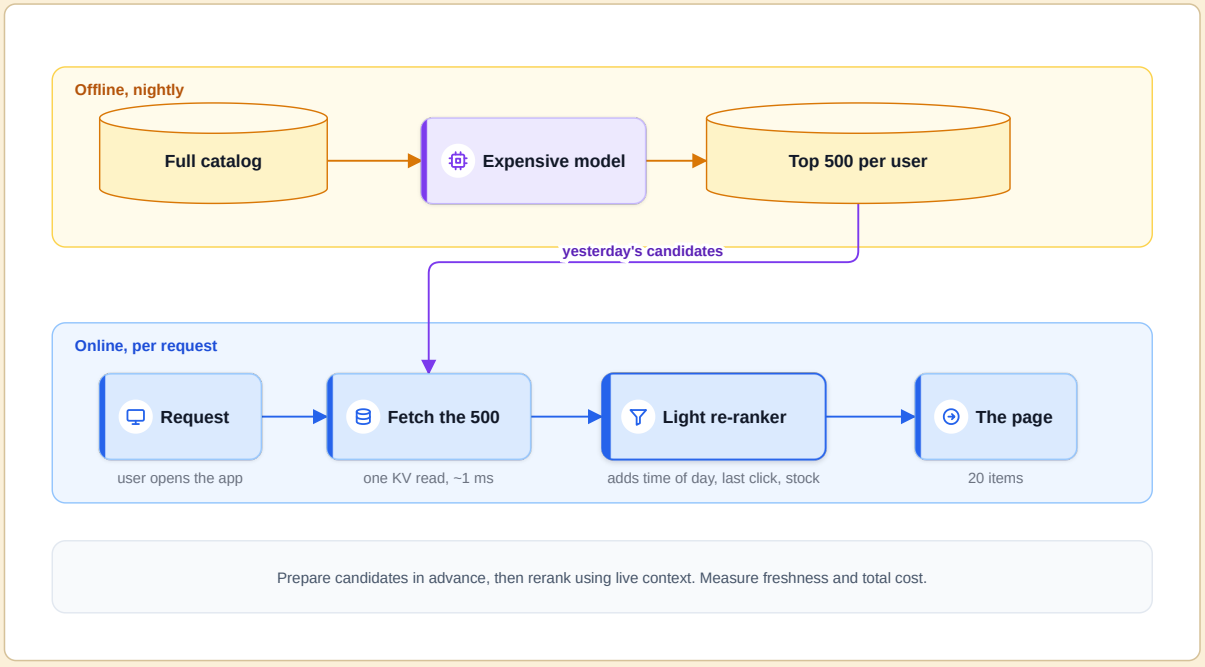


Figure — Nightly candidates, live rerank (course diagram)

Offline, an expensive model picks ~500 candidates per user; online, a small model reorders them using the moment (time of day, last click, stock). New items missing from the list need an online retrieval path. Failures degrade gracefully: yesterday's candidates if the nightly job fails, unreranked candidates if the reranker fails.

Streaming inference

Events trigger predictions: each uploaded video scored for policy violations, each payment scored in the pipeline. It's online inference behind a queue; handle queue delay, retries, **duplicate events** (idempotent scoring) and **backpressure**.

Trade-offs / when not to use

- Batch is wrong when inputs exist only at request time or staleness hurts (fraud, ETA).
- Online is wasteful when predictions are stable and heavily reused.
- Streaming is wrong when a caller is waiting synchronously for the answer.

In the interview

> **30-second answer**: *"Does it need request-time inputs, how fresh must it be, how often is a stored prediction reused? Stable and reused → batch; request-dependent → online; event-driven async → streaming. For feeds, precompute candidates for active users and rerank online."*

- **"The nightly job failed."** → Yesterday's predictions keep serving — degraded, not down; alert on staleness and rerun. An online failure is an immediate incident.
- **"How fresh is fresh enough?"** → Measure it: plot CTR against the age of the stored candidates; how fast it falls tells you how often to rerun the batch.

Self-check

1. Deciding question? → Does the prediction need request-time information?
2. Why precompute only for active users? → Most stored predictions for inactive users are never read.
3. What's streaming inference architecturally? → Online inference with a queue in front, no synchronous caller.

TL;DR

Batch = lookup, stale, cheap per request; online = fresh, request context, pay at peak; streaming = event-triggered async. Hybrid precompute-then-rerank is the default for large catalogs.

16.6 Model serving basics

What it is

A typical predictive model server is **stateless**: on startup it reads the model into memory, then turns each feature vector into a score. It scales like any web tier — replicas behind a load balancer, autoscaling on CPU load or queue length, rolling deploys, health checks. The ML-specific parts: where the artifact comes from, the latency budget, safe rollout, and hardware.

The latency budget, itemised (illustrative small-model numbers)

Stage	Rough cost	Comment
Model inference	1–10 ms	Trees and small tabular models are quick; deep nets need tuning
Online-store lookup	2–10 ms	Do it as a single batched read; frequently the slowest step
Features computed inline	0–15 ms	Each has its own allowance and a default if it runs over
Routing, auth, parsing the request	1–5 ms	The usual web-service overhead
Writing the log, sending the reply	1–2 ms	Logging is asynchronous so it doesn't block

Profile the full path before optimising. Two standard tools:

- **Batching** (dynamic/micro-batching) — group requests for a few ms to use hardware better, especially GPUs; trades a small wait for throughput, so test p99.
- **Feature caching** — hold the vectors of the most-requested entities (busy restaurants, trending videos) in process memory, so the bulk of traffic never touches the online store.

Shipping a new model safely

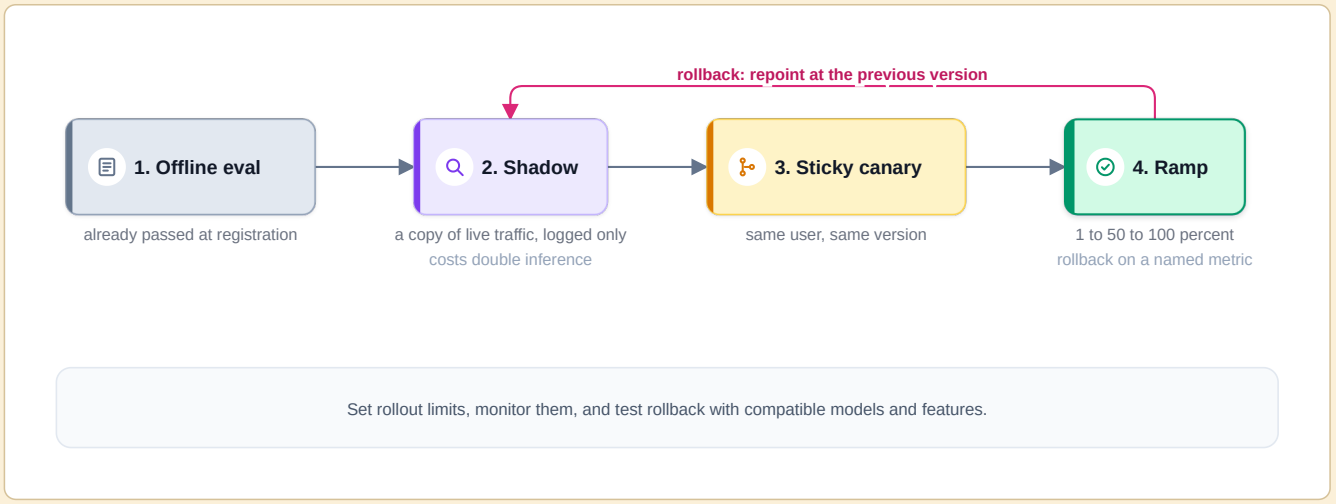


Figure — Rollout stages: offline eval, shadow, canary, ramp (course diagram)

1. **Offline evaluation** — already passed (that's how it got registered).
2. **Shadow deployment** — the candidate scores a copy of live traffic; outputs are logged, never used. Catches serving bugs, latency problems and big behaviour differences; isolate its capacity and disable side effects.
3. **Canary / A/B** — a small traffic share gets the new model, usually **sticky per user** (sessions, orgs, regions or time windows fit other cases).
4. **Ramp up, with a pre-agreed abort rule** — e.g. "revert if session CTR falls 2% or p99 exceeds 150 ms". Rollback must restore a compatible model *and* features and config; rehearse it and know how long it takes.

Hardware

Tree models and small neural nets serve well on CPU — say *"I'd benchmark the GBDT on CPU replicas first"*, not "GPU cluster" by reflex; GPUs pay off for large neural models with batching. Runtimes (ONNX Runtime, NVIDIA Triton, TensorFlow Serving) merit a passing mention.

Shrinking the model

Technique	Idea	Watch out
Distillation	A small *student* learns to mimic a large *teacher*	Measure the quality loss
Quantisation	Fewer bits per weight/activation (int8 vs float32)	Needs runtime support; check accuracy
Pruning	Drop weights that contribute little	Speedups need sparse-aware runtimes

Compare compression with simply changing hardware, counting engineering effort and quality loss. (Central for LLMs — see the AI Handbook.)

Degradation is part of the spec

Agree the **fallback prediction** with product and risk owners: the previous version kept warm, a cached prediction, a heuristic (average ETA; rules-only fraud), or a product default (unpersonalised shelf). For fraud it may be review or decline, not "allow". Name the owner and how long degraded mode may last before paging.

Model-in-service vs model-as-a-service

Embedding the model in the application saves a network hop but couples deploys and scaling. A standalone model service costs an extra network hop (a few ms) but gives independent rollout, sharing across teams and right-sized hardware — the default at scale.

Common mistakes

- Forgetting timeouts and autoscaling once a model is in the diagram.
- No model version logged per prediction — A/B results can't be attributed.
- Rollback = "redeploy the old file" after the feature schema changed.

In the interview

30-second answer: *"A stateless, versioned model service loading from the registry, scaled like a web tier. I itemise the latency budget and profile before adding batching or caches. Releases: offline eval → shadow → sticky canary → ramp with a named rollback metric. Fallback: previous version or a heuristic agreed with product."*

- **"A/B two models cheaply?"** → One service with both versions loaded, a sticky per-user router, version logged per prediction; shadow first, since experiments cost weeks of traffic.

Self-check

1. Why is the model server easy to scale? → It's stateless.
2. What's the usual latency bottleneck for small models? → Feature fetching.
3. Shadow vs canary? → Shadow: real traffic, outputs unused; canary: a real user slice gets outputs.

TL;DR

Stateless versioned service; itemised budget; batching and hot caches; shadow → canary → ramp; CPU first; distill/quantise/prune; explicit fallback.

16.7 Offline evaluation: choosing the metric

What it is

Offline metrics are the *first gate*: candidates are compared on held-out data before any user sees them; the online experiment (16.9) is the second gate. The definitions — precision/recall, ROC-AUC vs PR-AUC, MAE/RMSE, recall@k/NDCG/MRR, calibration — are all in Chapter 6 (§6.2 regression, §6.3 classification and thresholds, §6.4 ranking, §6.5 calibration). This section only adds what changes once the model is headed for production.

Problem	Report	Production twist
Rare-event classification (fraud, abuse)	P/R at the shipping threshold, PR-AUC next to the base rate	Thresholds are business policy; often two of them (approve / review / block)
Retrieval → ranking	recall@k for candidate generation, NDCG@k or MRR for the ordering stage	Each funnel stage gets its own metric; click labels are position-biased
Regression (ETA, demand)	MAE or RMSE plus p90 error	Break it down per cohort — new cities or segments hide inside a good average
Any score used as a number	Calibration (reliability diagram, ECE)	Recheck after downsampling/re-weighting (16.3) and after population shifts

Production checklist

1. **Fix the evaluation set to look like traffic**: time-based split, the real base rate, no leaked features (16.2, §4.7–4.8).
2. **Report the operating point, not only an area**: "92% precision at 60% recall at threshold 0.05", together with the review workload it creates (alerts per day).
3. **Let the cost owner pick the threshold**: product/risk teams price blocked good customers against missed fraud; ML supplies the PR curve and expected costs.
4. **Slice**: per country, device, new vs returning user, new vs mature market. Ship only if no important slice regresses.
5. **Beat two baselines**: a naive one (majority class, popularity, "same hour last week") and the current production model. A 6-minute MAE is impressive against a naive 8, not against a naive 6.2.
6. **Label source matters for ranking**: judged sets (human graders following guidelines) plus behavioural labels (clicks, purchases) with position-bias correction.

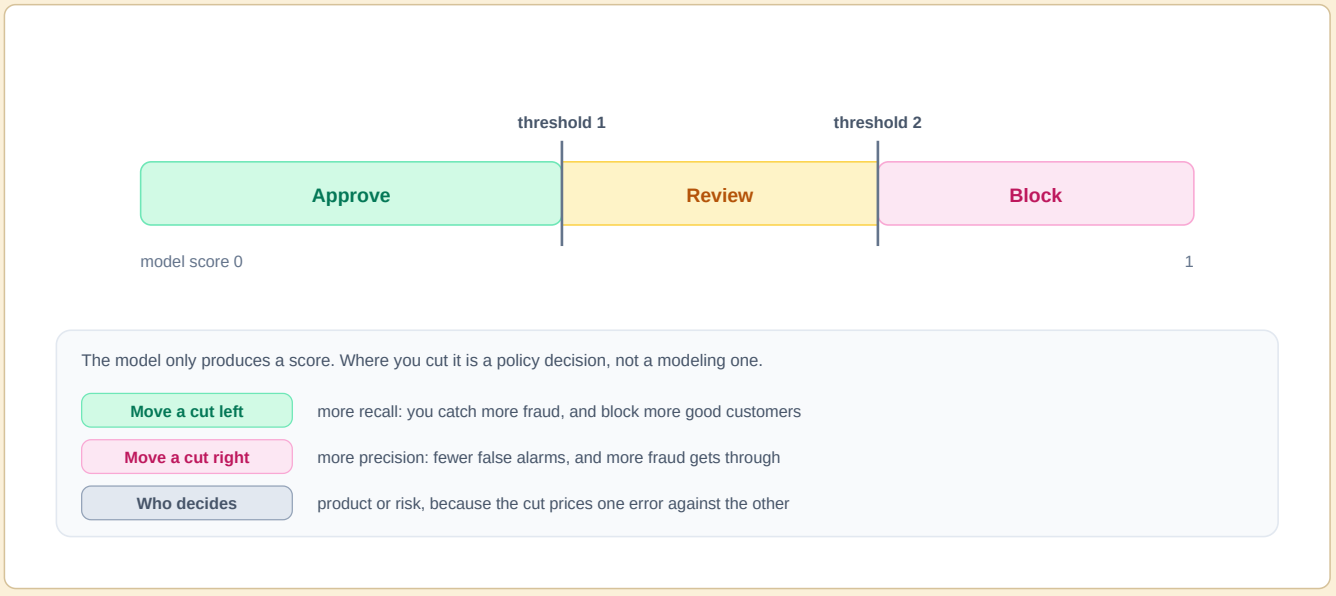


Figure — Approve / review / block bands from two cut-offs (course diagram)

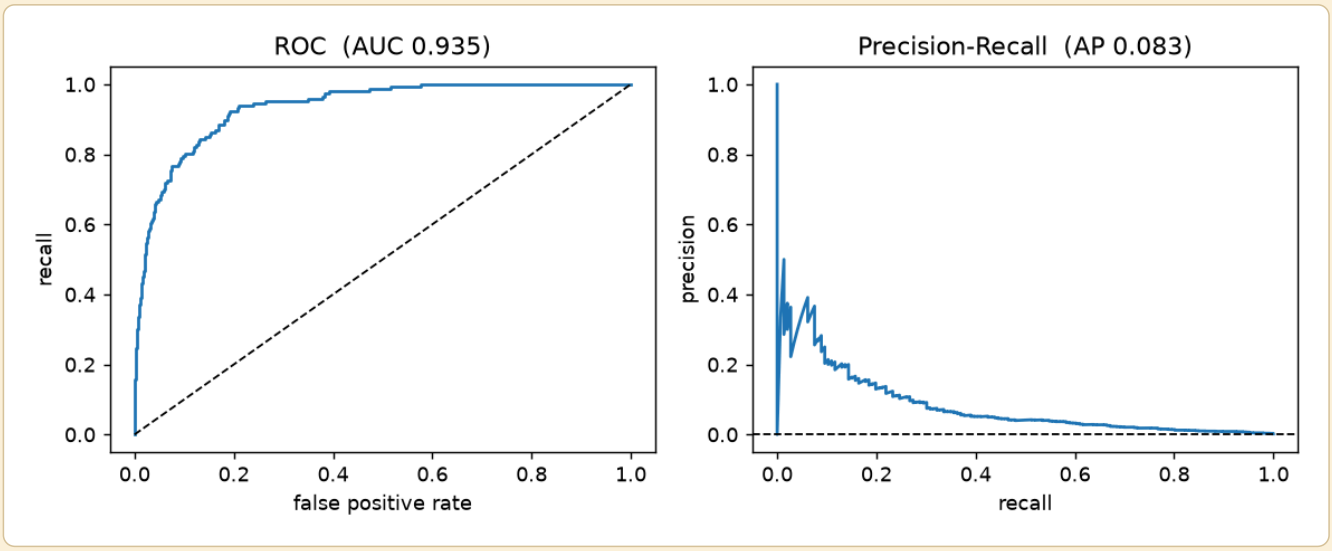
Code — one production-sized run per family (`code/offline_metrics.py`)

The script trains a logistic regression on synthetic transactions with 0.2% fraud, sweeps thresholds, builds the two-threshold bands, checks a by-hand NDCG against scikit-learn, and splits ETA errors by city cohort.

Output (abridged):

```
== classification (rare positives) ==
test rows 80,000, positives 146 (base rate 0.18%)
ROC-AUC 0.935   PR-AUC (average precision) 0.083   PR-AUC of a random model = base rate = 0.002
threshold  flagged   TP     FP precision  recall     FPR
    0.005     5776  106   5670      0.02    0.73  0.0710
    0.050      462   42    420      0.09    0.29  0.0053
  approve :  77081 txns, fraud rate in band 0.07%
  review  :   2457 txns, fraud rate in band 1.91%
  block   :    462 txns, fraud rate in band 9.09%

== regression (ETA minutes) ==
overall  MAE  3.71  RMSE  5.27  p90|err|  7.41
mature   MAE  3.17  RMSE  3.99  p90|err|  6.55  share 93%
new      MAE 10.93  RMSE 13.69  p90|err| 22.66  share 7%
naive baseline (predict the mean) MAE  9.46
```



What it shows in production terms: a 0.935 ROC-AUC would pass a dashboard review, but catching 73% of fraud means 5,670 false alarms for 106 catches — the analyst queue, not the AUC, is what the business feels. The two bands put 9% fraud in "block" and 1.9% in "review", so humans look at 2,457 cases instead of 80,000. The overall ETA error of 3.7 minutes hides 10.9 minutes in new cities.

In the interview

30-second answer: *"I pick the offline metric from the task (Chapter 6 details), but in production I report it at the shipping threshold with the workload it creates, sliced by cohort, against a naive baseline and the current model. The cost owner chooses the threshold. Offline wins only earn an A/B test."*

- **"Offline metric up, A/B flat?"** → The offline metric is a proxy: weak label (click vs watch time), test set no longer like traffic, or serving-time degradation (skew). Check experiment power and logging first.
- **"Which threshold would you ship?"** → Show the PR curve with expected costs and alert volume; let the cost owner pick; for fraud, two thresholds with a review band.

TL;DR

Metric definitions live in §6.2–6.5. In production: traffic-like eval set, operating point + workload, owner-chosen thresholds, cohort slices, naive and production baselines — then an A/B test.

16.8 Monitoring, drift and retraining

What it is

A deployed model loses quality as the world changes — fraudsters adapt, a pandemic reshapes delivery times, new slang fools a toxicity classifier. Decay rates differ; measure them.

Drift type	What changes	Example	Also called (Huyen)
Data drift	Inputs P(X)	Orders shift from lunch to dinner	Covariate shift
Label / prior shift	Outcomes P(Y)	Fraud rate doubles in a holiday	Label shift
Concept drift	The relationship P(Y | X)	Same features, different outcomes — new fraud tactics, a restaurant changes kitchens	Concept drift

Monitoring must also tell drift from plain **bugs** (a broken upstream pipeline looks like drift).

How it works — the four-layer monitoring stack

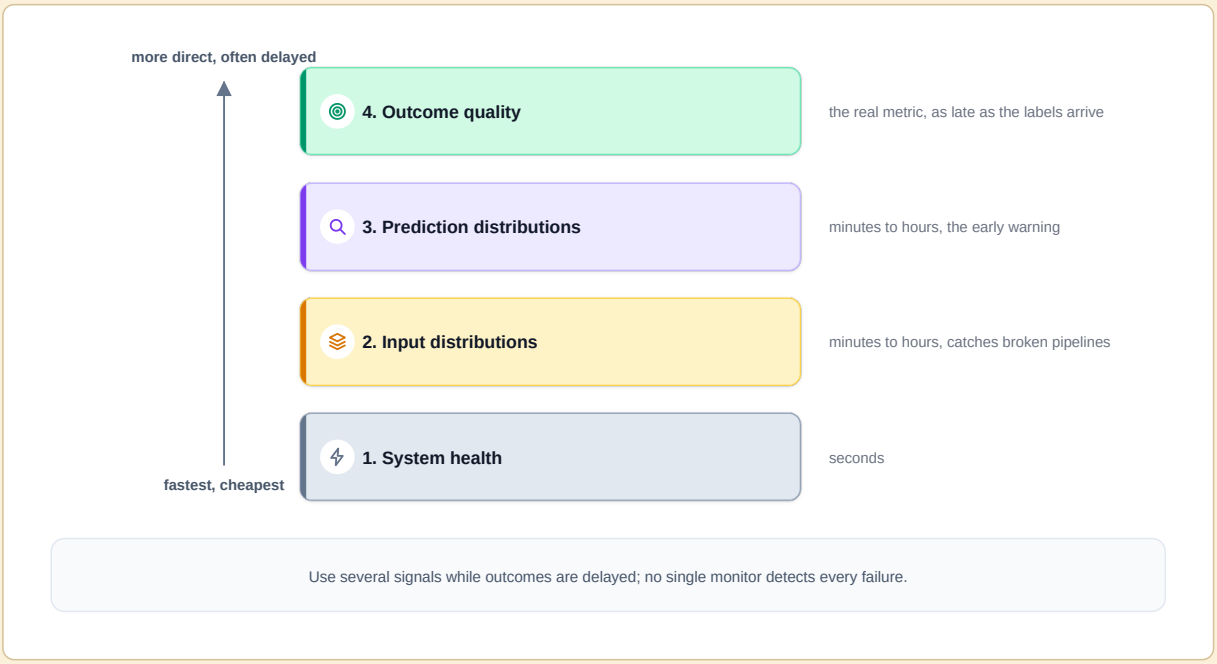


Figure — Monitoring layers from fast/cheap to slow/direct (course diagram)

Layer	How quickly it reacts	Signals tracked	Typical findings
1. Service health	Within seconds	Traffic, latency, error rate, how old the feature-store values are	Outages; pipelines that stopped updating
2. Inputs	Minutes to hours	Each feature compared with its training reference — share of nulls, averages, PSI or KS distance	Data drift; a producer sending garbage
3. Outputs	Minutes to hours	Shape of the score histogram; share of positive decisions	The model behaving differently, visible before any label exists
4. Outcomes	Only once labels land — seconds for clicks, about an hour for ETAs, weeks for chargebacks	CTR, realised ETA error, recall on confirmed fraud	Concept drift and genuine quality loss

Layers 2–3 exist because layer 4 is late; some attacks move neither inputs nor scores, so combine signals. Rules of ML **#9: detect problems before exporting models**; **#10: watch for silent failures** (e.g. a joined table that stopped updating). *Reliable Machine Learning* (Chen, Murphy et al., O'Reilly 2022) treats model quality as SLOs with incident playbooks, like service SLOs.

Drift statistics

- **PSI (Population Stability Index)**: bin the reference (training) distribution into quantiles; PSI = Σ (cur% − ref%) · ln(cur% / ref%). Credit-scoring convention: < 0.1 stable, 0.1–0.25 watch, > 0.25 investigate.
- **Two-sample Kolmogorov–Smirnov (KS) test**: D = max gap between the two empirical CDFs, plus a p-value. With large samples trivial shifts become "significant" — look at D (effect size).
- **Null rates and schema checks** — PSI on non-null values can't see a producer that started sending 30% nulls.
- Categorical features: chi-square test or PSI over categories. Tools: Evidently, WhyLabs, NannyML, SageMaker Model Monitor, Vertex AI Model Monitoring, TFDV.

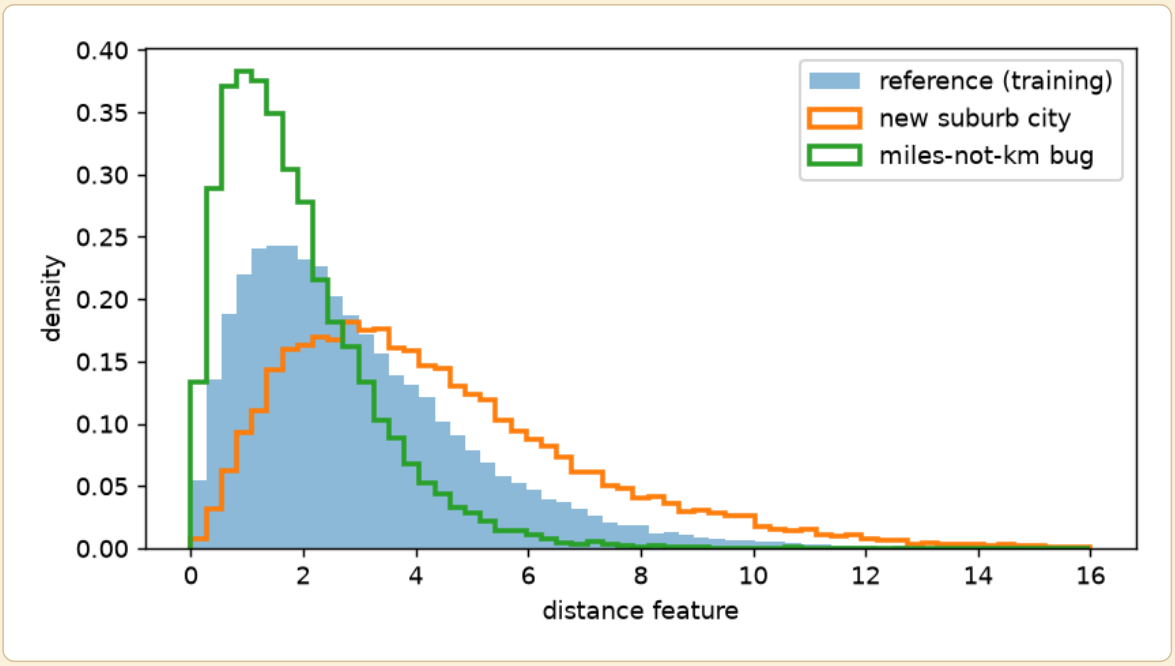
Code — PSI and KS (`code/drift_psi_ks.py`)

```
def psi(reference, current, bins=10, eps=1e-6):
    ref, cur = reference[~np.isnan(reference)], current[~np.isnan(current)]
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf            # catch values outside the training range
    r = np.histogram(ref, edges)[0] / len(ref)
    c = np.histogram(cur, edges)[0] / len(cur)
    r, c = np.clip(r, eps, None), np.clip(c, eps, None)
    return float(np.sum((c - r) * np.log(c / r)))

ks = ks_2samp(reference, current)                    # scipy.stats
```

Output:

```
same distribution (new sample)     PSI  0.001  KS D 0.007  p  4.9e-01  nulls  0.0%  -> stable
tiny shift (+3%) -- big n          PSI  0.002  KS D 0.018  p  2.5e-11  nulls  0.0%  -> stable
new suburb city (longer trips)     PSI  0.361  KS D 0.241  p  0.0e+00  nulls  0.0%  -> INVESTIGATE
upstream bug: miles not km         PSI  0.426  KS D 0.251  p  0.0e+00  nulls  0.0%  -> INVESTIGATE
30% nulls (broken producer)        PSI  0.000  KS D 0.006  p  7.5e-01  nulls 30.2%  -> stable
```



Lessons: (1) a 3% shift on large samples gives $p \approx 10^{-11}$ but PSI 0.002 — don't page on p-values; (2) a new city and a unit bug look the **same** to the statistic — alerts start investigations, they don't prove drift; (3) PSI/KS missed the 30% nulls — check null rates separately.

Retraining: cadence, trigger, gates

Retraining is one possible response — first fix broken data, policy or serving bugs.

1. **Scheduled** (nightly/weekly) — simple, right for gradual drift. Set the cadence from measured decay — e.g. if a weekly-refreshed model has shed 2% AUC by its seventh day, move to twice a week.
2. **Triggered** — monitor fires → investigation → if validated data exists, a candidate run. Never auto-promote because an alert fired.
3. **Online / continual learning** — incremental updates (Huyen: **stateless retraining** from scratch vs **stateful training** from the last checkpoint). Suits fast-changing tasks; needs bounded updates, validation and rollback — a bad stream corrupts an online learner fast.

Every candidate passes the same gates: offline vs production (overall **and per segment**) → shadow → canary, approved by a named owner or automated limits. The history window should cover seasonality yet drop stale patterns — treat it as a hyperparameter, weighting recent data in fast-drift domains.

Feedback loops that damage your data

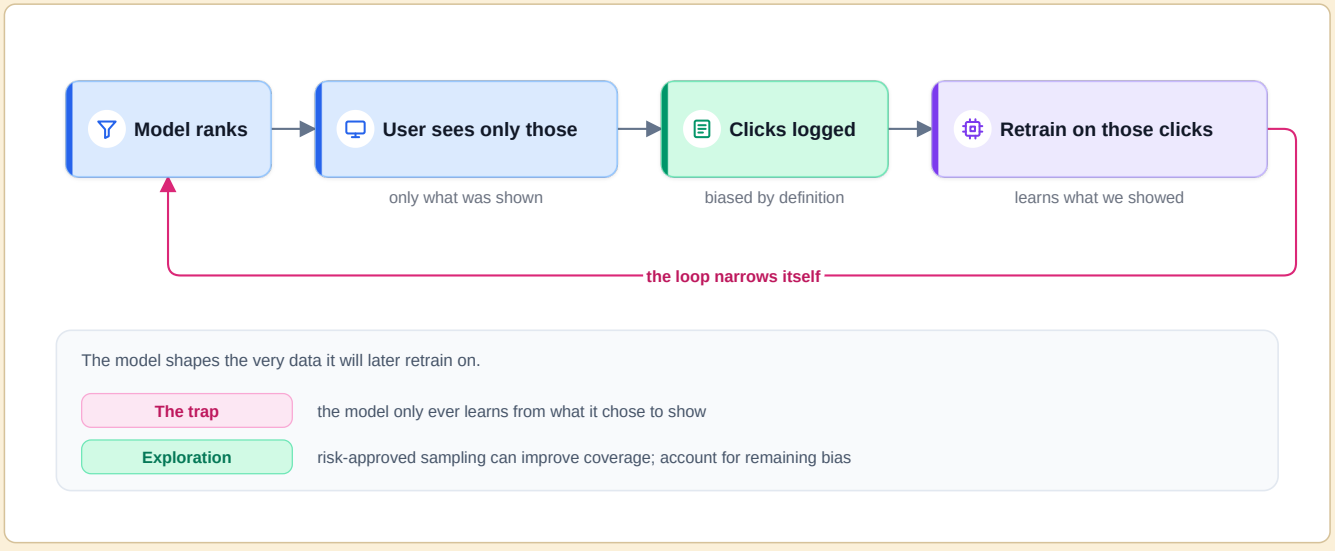


Figure — How a ranker narrows its own training data (course diagram)

The model shapes the data it later learns from — Huyen's **degenerate feedback loop**. A feed shows what it predicts you'll like, you can only click what was shown, and the retrained model learns "what we showed gets clicked", narrowing further (popularity bias, filter bubbles). In fraud, blocked transactions never produce an outcome (investigator evidence gives partial labels — record source and uncertainty).

Mitigations:

- **Controlled exploration** — in low-risk recommenders, show a small approved share of traffic randomised candidates and **log selection probabilities** to correct later (inverse propensity weighting); bandits formalise this.
- For fraud, never let suspicious payments through merely to learn their outcome; rely on reviews the risk team has signed off, simulations, or small experiments under strict limits.
- Positional features: log position at training, set it to a fixed value at serving (Rules of ML **#36**: avoid feedback loops with positional features).
- Exploration improves coverage but doesn't remove all bias; measure its benefit and user cost.

In the interview

> **30-second answer**: *"Four layers — service health, inputs, outputs, real outcomes; labels are late, so the first three warn early. An alert starts an investigation; often it's a broken pipeline, not the model. Retraining runs on a cadence set by measured decay or after an investigation, and every candidate clears offline, shadow and canary gates. I'd guard against the model skewing its own labels with a small, risk-approved exploration budget."*

- **"Quality dropped overnight — debug it."** → Bottom up: pipeline freshness and feature nulls (most likely, fastest) → prediction histogram → segment the drop (one country? one app version?) → only then the model.
- **"Data drift vs concept drift?"** → Inputs change vs the input→outcome relationship changes; the first may not hurt accuracy, the second always needs new labels.

Self-check

1. Why monitor prediction distributions? → They move before delayed labels arrive.
2. PSI = 0.4 — retrain? → Investigate first; could be a unit bug or a new market, not necessarily a stale model.
3. What's a degenerate feedback loop? → The model's outputs determine its future training data, reinforcing its own biases.
4. Why log selection probabilities during exploration? → To reweight (inverse propensity) and get unbiased estimates.

TL;DR

Data/label/concept drift; four monitoring layers; PSI/KS + null checks; gated retraining by schedule or after investigation; beware models that shape their own data.

16.9 Online experiments and A/B testing

What it is

Offline evaluation asks whether the new model predicts held-out labels better; production asks whether it makes the **product** better. An **A/B test** (controlled experiment) randomly splits units between control (A, current model) and treatment (B, new model) and compares a business metric.

Why offline numbers aren't enough

1. **The label is a proxy** — a model can win on click-AUC while lowering watch time (clickbait predicts clicks well).
2. **Deploying the model alters the behaviour it is judged on** — a new ranker changes what people see, and so what they click; a frozen test set can't reflect that.
3. **Serving isn't scoring** — live traffic has timeouts, stale caches and fallbacks that offline evaluation never saw.

How it works — two gates, and shadow in between

1. **Offline gate** — predefined quality, cost, safety and per-subgroup criteria.
2. **Shadow mode** (when failures are expensive) — score live traffic in parallel without showing it; compare on identical requests.
3. **Online gate** — A/B against production on the **business** metric (some products can't A/B safely or lack traffic).

A/B mechanics

1. **Randomise by unit** — usually users (hash user id + experiment name for sticky assignment); sessions, orgs, regions or time windows when users influence each other (**interference**) or effects carry over.
2. **One primary metric, chosen in advance** — everything else is a guardrail or diagnostic.
3. **Size the test for enough power** — fix the smallest lift worth detecting (**minimum detectable effect**, MDE), significance level α (false-positive rate, typically 0.05) and **power** (probability of detecting a true effect of that size, typically 0.8). Sample size per arm for two proportions: $n = [z_{1-\alpha/2}\cdot\sqrt{(2\cdot\bar{p}(1-\bar{p}))} + z_{1-\beta}\cdot\sqrt{(p_1(1-p_1) + p_2(1-p_2))}]^2 \,/\, (p_2 - p_1)^2$ Rare events, small effects and noisy metrics need big samples.
4. **Duration** — from sample size, outcome delay and business cycles (whole weeks); watch **novelty effects** and seasonality.
5. **Don't peek** — stopping at the first daily $p < 0.05$ inflates false positives. Commit to an end date, or use a **sequential test** built for repeated checks (alpha-spending, always-valid methods). **CUPED** (pre-experiment behaviour as a covariate) reduces variance and so sample size.

Code — sample size, z-test, peeking (`code/ab_test.py`)

```python
def assign(user_id, experiment, treatment_share=0.5):
    h = int(hashlib.sha256(f"{experiment}:{user_id}".encode()).hexdigest(), 16)
    return "B" if (h % 10_000) / 10_000 < treatment_share else "A"      # sticky

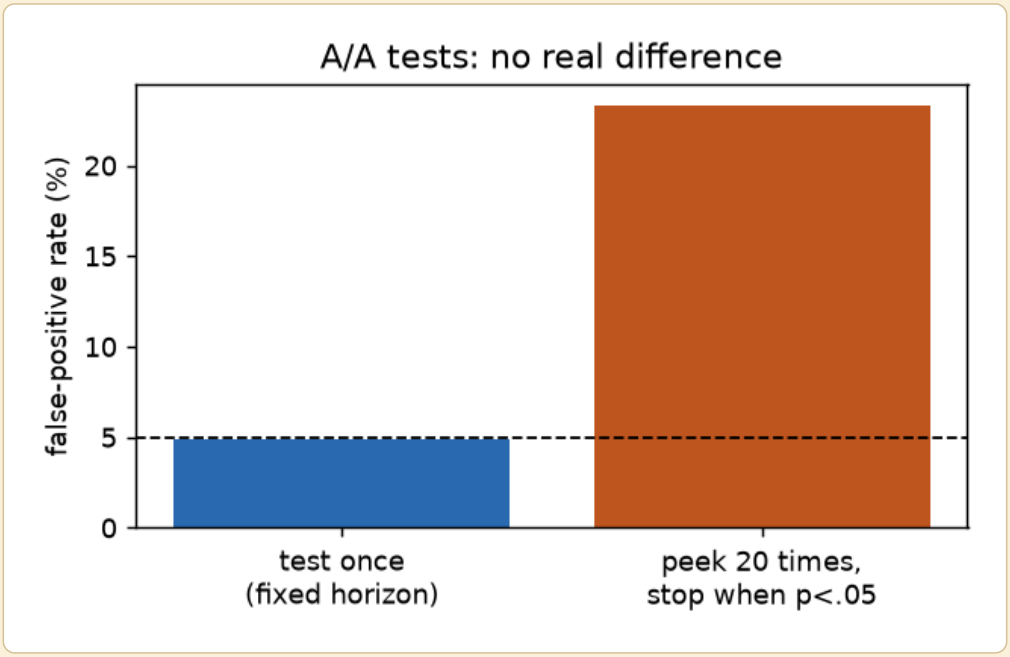
def two_prop_ztest(x_a, n_a, x_b, n_b):
    pa, pb = x_a / n_a, x_b / n_b
    pooled = (x_a + x_b) / (n_a + n_b)
    se0 = np.sqrt(pooled * (1 - pooled) * (1 / n_a + 1 / n_b))        # SE under H0
    z = (pb - pa) / se0
    p = 2 * (1 - norm.cdf(abs(z)))
    se = np.sqrt(pa * (1 - pa) / n_a + pb * (1 - pb) / n_b)           # SE for the CI
    return pa, pb, z, p, (pb - pa - 1.96 * se, pb - pa + 1.96 * se)
# ... sample_size_per_arm(), and 2,000 simulated A/A tests with 20 peeks each
```

Output:

```
sticky assignment: ['A', 'B', 'A', 'A', 'A', 'A', 'B', 'B'] | again: True
baseline CTR 5%, detect +1% relative lift (alpha .05, power .8): 2,996,698 users per arm
baseline CTR 5%, detect +2% relative lift (alpha .05, power .8): 752,703 users per arm
baseline CTR 5%, detect +5% relative lift (alpha .05, power .8): 122,124 users per arm

A: 19,906/400,000 = 4.9765%   B: 20,575/400,000 = 5.1437%
diff +0.1673% (relative +3.4%), z = 3.41, p = 0.0006, 95% CI [+0.0712%, +0.2633%]

A/A tests, alpha 0.05: false-positive rate testing once at the end = 4.9%; stopping at the first significant of 20 peeks = 23.3%
```



Halving the detectable lift roughly **quadruples** the sample ($n \propto 1/\delta^2$): a 1% relative CTR lift needs ~3M users per arm. The finished test is significant (CI excludes zero). Peeking 20 times turns a 5% false-positive rate into ~23%.

Guardrails and the metric hierarchy

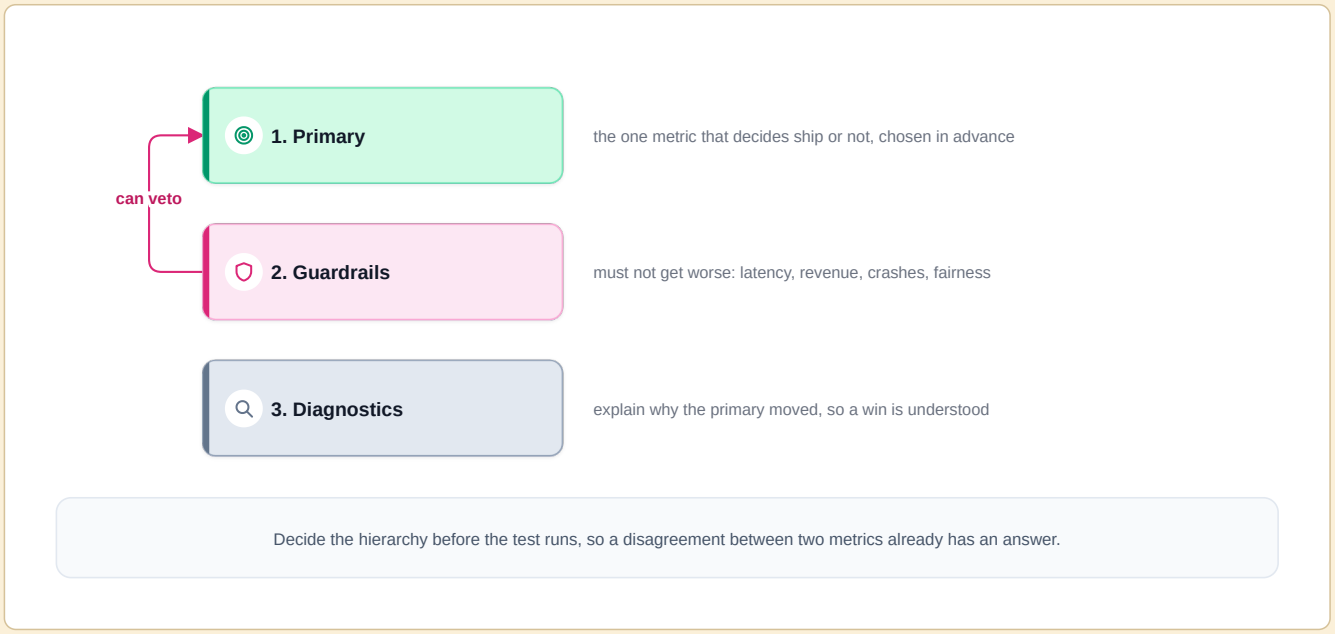


Figure — Metric tiers: primary, guardrails, diagnostics (course diagram)

1. **Primary** — decides ship/no-ship (e.g. watch time per session).
2. **Guardrails** — must not degrade: latency, crash rate, unsubscribes, revenue, fairness across segments. A primary win doesn't override a breached guardrail.
3. **Diagnostics** — explain *why* the primary moved.

Fixing the hierarchy beforehand settles disagreements in advance. Also check **sample ratio mismatch** (a 50/50 split arriving as 52/48 means broken assignment or logging).

Beyond the simple A/B

Variant	Use when	How
Interleaving	Ranking, low traffic	Merge two rankers' results into one list per user (e.g. team-draft); see whose items get clicked. Much more sensitive, but measures preference, not long-term business impact
Switchback	Marketplaces with network effects (dispatch, surge pricing)	Flip the whole region between variants over time windows, because treating some riders affects the drivers everyone shares
Multi-armed bandits	Ongoing selection where you want reward during the test	Shift traffic toward the winning arm as evidence accrues (Thompson sampling, UCB); analysis differs from a fixed-split test
Holdback	Measuring durable long-term effect	Keep a small slice on the old experience for months

Real-world examples

Microsoft (Bing/ExP), Netflix, Booking.com, LinkedIn and Airbnb run thousands of experiments a year; Netflix and Airbnb have published on interleaving for ranking, DoorDash and Lyft on switchback experiments for marketplaces; CUPED originated at Microsoft (Deng et al., 2013).

In the interview

> **30-second answer**: *"Offline filters candidates; shadow catches serving problems; then an A/B test with a predeclared unit, primary metric and guardrails, sample size from MDE and power, whole weekly cycles, and a fixed-horizon or sequential stopping rule — then a gradual rollout with rollback conditions."*

- **"Very little traffic?"** → More sensitive metrics, interleaving, longer runs, variance reduction, bolder changes — and admit some rare-event effects can't be A/B tested; rely on shadow plus careful ramps.
- **"Metric improved for a month, then reverted."** → Novelty effect, seasonality, population change or data issues; run past the novelty window and keep a long-term holdback.

Self-check

1. Why pick the primary metric beforehand? → To avoid choosing the metric that happens to look good (p-hacking).
2. How does sample size scale with MDE? → ~$1/\text{MDE}^2$; half the effect → ~4× the users.
3. What's wrong with peeking? → Many looks at noise inflate the false-positive rate (~23% in the demo).
4. When use a switchback? → When treated and control units interfere, e.g. shared drivers in a marketplace.

TL;DR

Offline filters, online decides: right unit, one primary metric, vetoing guardrails, powered sample, whole weeks, no peeking; interleaving, switchbacks, bandits, holdbacks for special cases.

16.10 MLOps: the glue that makes it repeatable

What it is

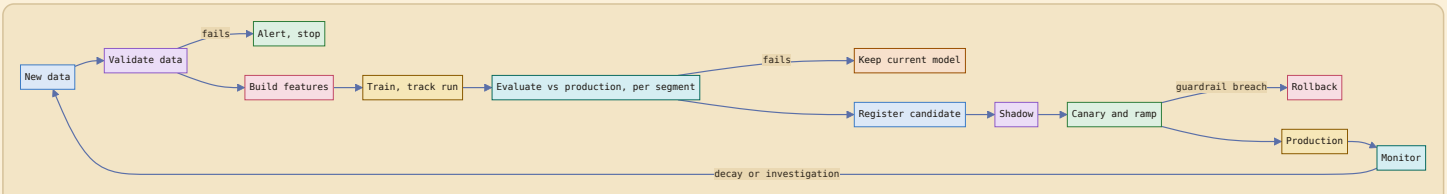
MLOps applies DevOps practices — version control, automated testing, CI/CD, monitoring — to ML, where the "code" includes **data** and **models**. Google's MLOps maturity levels are a handy vocabulary:

Level	What's automated	Typical symptom
0 — Manual	Nothing: notebooks, hand-exported models, rare releases	"Only Alice can retrain it"
1 — ML pipeline automation	The training pipeline runs automatically on new data (**continuous training**, CT), with data & model validation and a registry	Models refresh; pipeline code still deployed by hand
2 — CI/CD pipeline automation	Pipeline code itself is built, tested and deployed automatically	New features/models go from commit to production safely

The components

Component	Job	Examples
Experiment tracking	Log each run's code version, params, data hash, metrics, artifacts — reproducible and comparable	MLflow Tracking, Weights & Biases, Neptune
Data & model versioning	Snapshot datasets and link them to models (lineage)	DVC, lakeFS, Delta Lake time travel
Data validation	Schema, ranges, null rates, distribution checks **before** training and at serving; fail the pipeline on bad data	TFDV, Great Expectations, Pandera
Model registry	Versions, stages (candidate/staging/production/archived), metrics, approvals, rollback target	MLflow Model Registry, SageMaker, Vertex AI
Orchestration	DAG of ingest → validate → features → train → evaluate → register	Airflow, Kubeflow Pipelines, Dagster, Metaflow, TFX
CI for ML	Feature-code unit tests, training–serving consistency tests, a "trains and beats baseline" smoke test, validation gate	GitHub Actions + pytest, etc.
CD for ML	Package model + serving code, deploy via shadow → canary → ramp, auto-rollback on guardrails	KServe, Seldon, BentoML, SageMaker endpoints
Monitoring	16.8's four layers, feedback into CT	Evidently, Arize, WhyLabs, Prometheus/Grafana

Rules of ML **#5: test the infrastructure independently from the ML** — e.g. push a trivial model through and confirm identical offline and online predictions before trusting any accuracy number. Google's "ML Test Score" paper (Breck et al., 2017) is a checklist for this.



Code — a toy end-to-end loop (`code/mlops_pipeline.py`)

Data validation, tracked runs with data hashes, a registry with stages, a per-segment promotion gate, shadow comparison, a canary with automatic rollback, and a skew check — in ~170 lines of plain Python.

```
def gate(cand, prod, tolerance=0.01):
    c, p = cand["metrics"], prod["metrics"]
    if c["mae"] >= p["mae"]:
        return False, f"overall MAE {c['mae']:.2f} not better than {p['mae']:.2f}"
    for k in ("mae_mature", "mae_new_city"):                        # no segment may regress
        if c[k] > p[k] * (1 + tolerance):
            return False, f"segment {k} regressed {p[k]:.2f} -> {c[k]:.2f}"
    return True, f"MAE {p['mae']:.2f} -> {c['mae']:.2f}, no segment regressed"

def canary(cand_model, prod_model, live, ylive, cand_features=None):
    for share in (0.01, 0.05, 0.25, 0.50):
        # ... compare canary MAE with production MAE on the traffic slice
        if mae_c > mae_p * 1.05:
            return False            # ROLLBACK
    return True
```

Output:

```
1) validation of a clean batch: OK
   validation of a broken batch: ['distance_km: 99.9% out of range [0.0, 80.0]', 'hour: 2.0% nulls']

2-3) train + track + register; v1 (ridge) bootstraps production
   v1 ridge            MAE 2.94 (mature 2.84, new city 3.79)
   v2 gbdt             MAE 2.43 (mature 2.43, new city 2.40)
   v3 gbdt_no_new_city MAE 2.58 (mature 2.43, new city 3.85)
   tracked run files: ['run_1.json', 'run_2.json', 'run_3.json']

4) promotion gate vs production (v1)
   v2 gbdt             -> PASS: MAE 2.94 -> 2.43, no segment regressed
   v3 gbdt_no_new_city -> FAIL: segment mae_new_city regressed 3.79 -> 3.85

5) shadow v2: 20,000 live requests, mean |prod - cand| = 1.49 min, max = 27.3 min (logged, never served)

6) canary v2, first attempt (serving container has the UTC-hour bug):
   canary   1%: MAE 2.95 vs prod 2.93 -> continue ramp
   canary   5%: MAE 3.47 vs prod 2.92 -> ROLLBACK (guardrail: >5% worse)
7) skew check on logged features: 'hour' differs from the training pipeline on 100% of rows -> fix the serving code path
   canary v2, after the fix:
   canary   1%: MAE 2.20 vs prod 2.93 -> continue ramp
   canary   5%: MAE 2.48 vs prod 2.93 -> continue ramp
   canary  25%: MAE 2.42 vs prod 2.93 -> continue ramp
   canary  50%: MAE 2.46 vs prod 2.92 -> continue ramp
   registry: [(1, 'ridge', 'archived'), (2, 'gbdt', 'production'), (3, 'gbdt_no_new_city', 'candidate')]
```

What it shows: validation stops a km→m unit bug; v3 beats v1 on average but is blocked for regressing on new cities; the canary catches a **serving-side** bug (hour in UTC) invisible offline — 1% was too noisy, 5% caught it; comparing logged serving features with training finds the cause; after the fix v2 ramps and is promoted.

Common mistakes

- Metrics tracked without the data version → unreproducible models.
- A registry without stages or owners → nobody knows what's live.
- Auto-retrain and auto-promote without gates.
- CI that never checks training and serving produce the same features.

In the interview

30-second answer: *"Every run tracked with code, data hash, params and metrics; data validated before training; candidates registered and promoted only through an evaluation gate, shadow and auto-rollback canary. The pipeline retrains on schedule or trigger (CT), and its code ships through CI/CD with training–serving consistency tests."*

Self-check

1. MLOps level 1 vs 2? → Level 1 automates retraining (CT); level 2 also automates building/testing/deploying the pipeline code.
2. Why log the data hash with a run? → Reproducibility and lineage: same code + same data → same model.
3. What did the canary catch that offline eval couldn't? → A serving-path feature bug (skew).

TL;DR

Track runs, version data and models, validate, register with stages, orchestrate, CI/CD/CT with gates, shadow → canary → rollback, monitor, loop.

16.11 Capstone — an ML platform for ETA prediction

The exercise

> "Show a delivery-time estimate at checkout and keep it accurate as the business grows. 500K orders/day, peak 30 orders/s at dinner. The estimate appears as soon as the customer opens checkout."

(Numbers are illustrative.) Try it with the Chapter 15 framework first.

Steps 2–4: success, data, baseline (first ~12 minutes)

- **Metrics**: MAE against real delivery time, **p90 absolute error** (customers remember the 25-minute promise that took 70), and a **promise-kept rate** — share of orders arriving inside the window shown. The business may trade a slightly longer displayed time for promises that almost never break.
- **Latency**: assume < 100 ms. Note 30 *orders*/s ≠ checkout views/s — estimate views, abandoned sessions and retries separately.
- **Labels**: each completed order reveals its delivery time within about an hour — excellent labels. Validate timestamps, cancellations, missing events, and the gap between checkout-view time and order time.
- **Sources**: the order event stream, courier location pings, when restaurants accept and finish orders, and past weather.

- **Features**: entity (restaurant median prep, courier vehicle); streaming (restaurant queue now, courier supply in zone, prep-time drift last hour); request context (route distance, cart size, time, rain).
- **Baseline**: per-restaurant average + distance term — ships fast, tests logging and evaluation. **Upgrade**: GBDT on all features, benchmarked on CPU.

Step 5: architecture (minutes 12–30)

Online inference: nobody knows the kitchen queue or the route until the customer is at checkout.

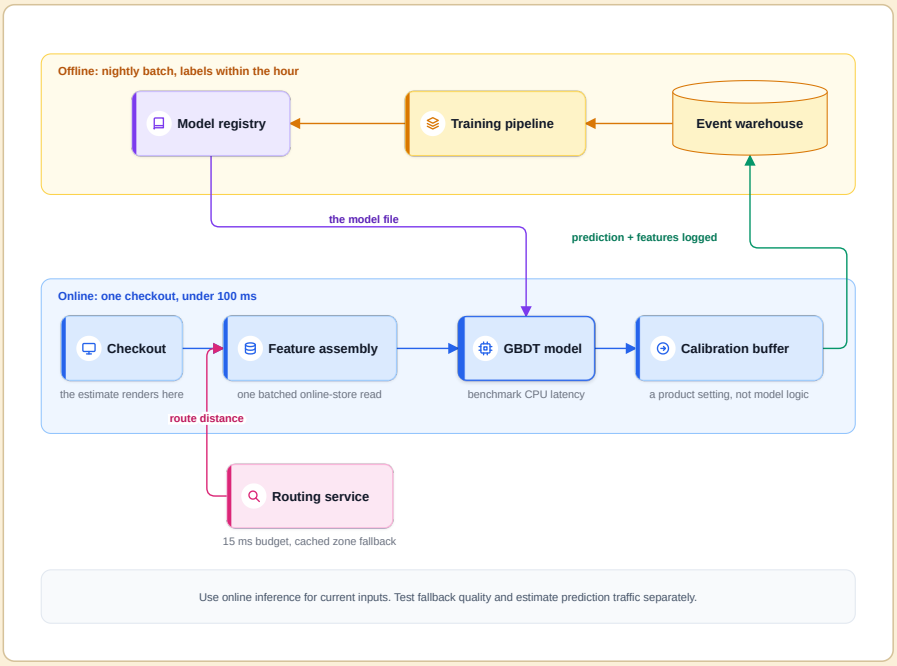
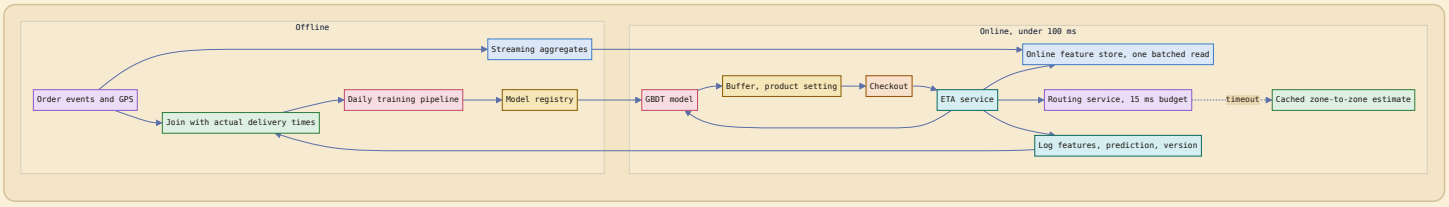


Figure — ETA platform end to end (course diagram)



Three points to say out loud:

1. **Routing is where latency goes wrong** — an external dependency on every checkout. Give it a budget (15 ms) with a cached zone-to-zone fallback, and train/evaluate with that approximation so degradation is measured.
2. **Log predictions with the exact features used** — kills skew; join outcomes when orders complete, validating timestamps, ID matches, duplicates and late events.
3. **A buffer layer after the model** — displayed ETA = prediction + buffer. A bigger buffer keeps more promises but makes the restaurant look slower than rivals; it's a **product** setting, not hidden in the model (see Chapter 15's buffer curve).

Steps 6–7: loop, failures, cost (minutes 30–40)

- **Monitoring**: freshness & nulls → input distributions (a new city skews distances) → prediction distribution (if the mean promised time keeps rising, the business needs to hear about it) → MAE on completed orders by prediction hour, **per city, restaurant type, weather**.
- **Retraining**: daily to start, checked against data maturity, measured decay and cost. Evaluate **per segment** before promotion — a better average can hide a worse Manhattan dinner rush.
- **Failures and fallbacks** (rehearsed, never improvised):

Failure	Fallback
Routing service down	Cached zone estimates + wider buffer
Feature store stale	Trained-in defaults; freshness monitor fires
Model service down	Warm, versioned per-restaurant heuristic table; never block checkout indefinitely — show the fallback or "estimate unavailable"

- **Cost**: count prediction requests separately from orders; benchmark CPU, replicas, storage, stream processing, routing calls. For a small GBDT, features and routing may cost more than inference — the opposite of LLM systems.

What each level sounds like

- **Mid**: the right components with prompting (GBDT, feature store, daily retrain, dashboard), but gaps between them — features not logged, routing unbudgeted, MAE only global.
- **Senior**: unprompted label timing and validation, features logged with predictions, routing budget + trained-in fallback, per-segment promotion via shadow and canary.

- **Staff**: separates model from promise (business owns the buffer), starts with the baseline to prove logging and evaluation, compares inference vs pipeline vs routing cost, builds a platform for future models.

Follow-ups to drill

- **p99 creeps past 100 ms, profiling blames feature reads** → one batched read, in-process cache for hot restaurants, defaults for the slowest aggregates, and drop features that cost more milliseconds than they add accuracy.
- **Product wants ETAs on the restaurant list (100× checkout traffic)** → a hundredfold routing load breaks the latency and cost budgets. Serve browsing from batch estimates per zone and restaurant; keep the live path for checkout, where the figure becomes a commitment.
- **New city, zero history** → distance baseline + priors from the most similar city; monitor it as its own segment; labels arrive within hours, so retrains fold it in within days.
- **A storm doubles delivery times this evening** → input drift fires (weather), then prediction drift, then hourly MAE; training data may hold no storm this severe. Fastest lever: widen the buffer (product decision) while a triggered retrain absorbs tonight's labels.

Same structure, different problem

- **Delivery date on a retailer's product page** → labels take days, monitoring slows, the buffer matters more; batch per zone × warehouse, online only at checkout.
- **Freight ETA across a country** → multi-day horizons, sparse GPS, thousands (not millions) of predictions; less QPS work, more label quality, per-segment accuracy and dispatcher overrides.

Now you (40 minutes each, out loud)

1. **Subscription churn** — "churned" is known only 30 days later; let that reshape training windows, monitoring and retraining.
2. **Ride-hailing price estimate** — surge makes freshness paramount; seconds-level aggregates; routing on every screen render. For both: when labels arrive, what happens with no prediction, and which component dominates cost.

> **TL;DR**

Per-request inference (live queue, route); labels within the hour; routing budgeted, cached, its absence rehearsed in training; features logged with predictions; buffer owned by product; per-segment monitoring and promotion; features and external calls outweigh small-model inference cost.

Chapter TL;DR

- **Five pieces, two planes**; models flow to serving, logs flow back.
- **Skew**: one set of feature definitions, as-of joins, train on logged serving features (Rules of ML #29, #32, #37).
- **Labels** have a source, cost and delay; downsample then recalibrate ($p = wq / (wq + 1 - q)$); split by time.
- **Features**: entity / aggregates / context / cross; batch, streaming, request-time; feature store = offline history + online latest.
- **Inference**: request-time inputs → online; stable and reused → batch; events → streaming; big catalogs → precompute-then-rerank.
- **Serving**: stateless, budgeted, CPU first, shadow → canary → ramp, explicit fallback.
- **Offline metrics**: PR-AUC + thresholds, recall@k + NDCG, MAE/RMSE + p90 + cohorts, calibration, always vs baseline.
- **Monitoring**: health → inputs → predictions → outcomes; investigate before retraining; watch feedback loops.
- **A/B tests**: predeclared unit and metrics, powered sample, whole weeks, no peeking.
- **MLOps** makes all of this happen every time: tracking, versioning, validation, registry, CI/CD/CT, gated promotion.

Chapter 17 — ML Case Studies I: Recommendations, Feed Ranking, Search

Chapter 15 gave you a seven-step framework and Chapter 16 built the production pieces behind it. This chapter and the next put both to work on the questions interviewers actually ask. The same handful of designs come up again and again — recommendations, feeds, search, ads, fraud, content moderation, "people you may know" — and each one teaches a different lesson about data, serving and evaluation. Learn these well and a new question ("design job matching", "design a podcast feed") becomes a matter of saying which parts transfer and which requirement is new.

This chapter covers the four **retrieval-and-ranking** designs: video recommendations, social feed ranking, e-commerce search and text-to-video search. Chapter 18 covers ads, fraud, harmful content, people-you-may-know, music discovery, visual search, privacy blurring, event recommendation and similar listings, and ends with a comparison table across all thirteen case studies.

What you'll learn

1. A single case-study template you can run for any predictive design question (17.0).
2. **Recommendations**: the retrieval → ranking → re-ranking funnel, two-tower embeddings with an ANN index, cold start, position bias and the proxy trap (17.1).
3. **Feed ranking**: a social candidate pool, multi-task models with one head per reaction, a product-owned value formula, and an integrity layer (17.2).
4. **Search ranking**: query understanding, hybrid lexical + semantic retrieval, L1/L2 rankers, NDCG, and click bias correction with inverse-propensity weighting (17.3).
5. **Text-to-video search**: indexing what videos say (ASR) and show (CLIP-style frame embeddings), fusing evidence channels, segment vectors for moments (17.4).
6. Four small runnable toys (synthetic data) that show the core modelling idea of each design.

Code for this chapter is in `code/` and runs with the shared environment. All outputs below are real.

17.0 The case-study template

What it is

One fixed order of questions for every design in Chapters 17–18. It is the seven-step framework from Chapter 15.3, split into finer pieces so the ML parts get their own slot:

#	Section	The question it answers	Ch. 15 step
1	Clarify requirements	What product surface, what scale, what latency, what's out of scope?	1–2
2	Frame as an ML task	What goes in, what comes out, which ML category, which objective?	1
3	Data and labels	Which logs and catalogs exist, where do labels come from, how biased/late are they?	3
4	Features	Which feature groups, how fresh, what leakage risk?	3
5	Model: baseline → advanced	What's the simplest thing that works, and what's worth upgrading to?	4
6	Offline metrics	How do we pick models before exposing users?	2, 6
7	Online metrics and A/B	What decides a launch, and what may never get worse (guardrails)?	2, 6
8	Serving architecture	What runs offline vs per request; draw the request path	5
9	Scale numbers	QPS, candidates scored per second, memory — with the arithmetic shown	5, 7
10	Monitoring and failure modes	What breaks silently, what are the fallbacks?	6–7
11	Interview follow-ups	The questions you'll get, with answers	—

Each case study then adds a **toy** (a small runnable script), **common mistakes**, **same design, other questions** (variants that reuse the design), a **self-check**, and a **TL;DR**.

Generic material is taught once elsewhere and only referenced here: A/B test mechanics — sticky randomisation, one primary metric, guardrails, novelty effects, interleaving, holdbacks — are in Chapter 16.9; drift, monitoring layers and retraining triggers in 16.8; shadow → canary → ramp releases in 16.6. The case studies only add what is *specific* to each design.

The shared shape: a funnel (for retrieval and ranking designs)

Recommendations, feeds and search all face the same arithmetic: too many items to score each one carefully on every request. The answer is a **funnel** — a series of stages, each handling fewer items and so able to spend more compute per item:

1. **Retrieve** (also called **candidate generation**): cheap methods pick a few hundred plausible items from millions. It is graded on **recall**: what fraction of the things this person would have enjoyed made it into the shortlist? Anything missing from the shortlist is lost for good, because later stages can only reorder what they receive.
2. **Rank**: a richer model scores those few hundred and sorts them. It is graded on **top-of-list precision**: among the first few results on screen, what share are genuinely good? Those few slots are nearly all a person ever looks at.
3. **Re-rank / apply rules**: a final layer adjusts the top items for business reasons — freshness, variety, policy, "no two ads in a row".

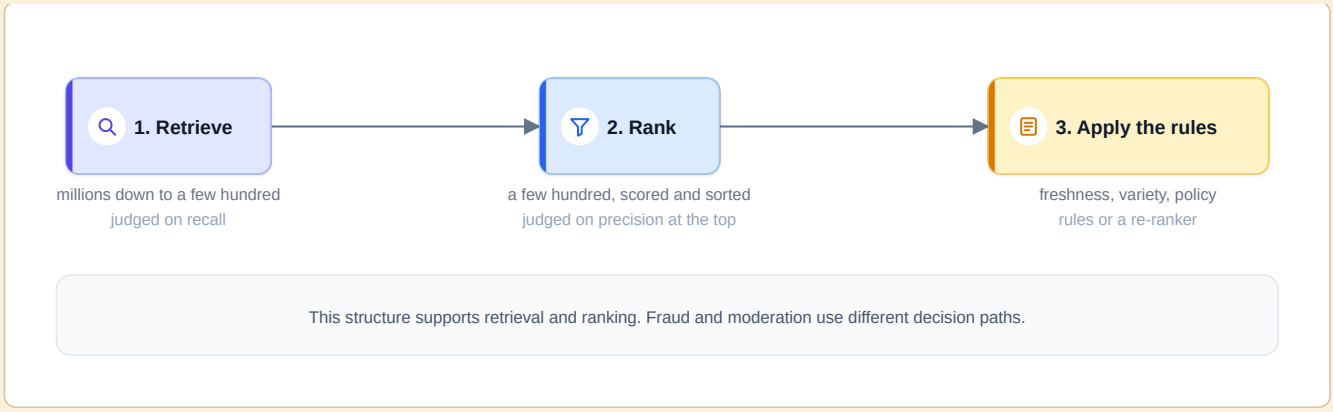


Figure — Retrieve, rank, apply rules: the funnel shared by recommendations and search (course diagram)

Not every design is a funnel. **Fraud detection** scores one transaction at a time, and **harmful content detection** uses a **cascade**: cheap checks on everything, expensive checks (bigger models, humans) only on what the cheap checks flag. **Privacy blurring** is not a funnel either: a batch detection pipeline that runs once per image. Chapter 18 covers all three.

Embedding (used everywhere below): a list of numbers (a vector) that represents an item or a user, learned so that similar things get nearby vectors. "Similar" then becomes a distance you can compute.

What each case study adds

Case study	New idea it teaches	Chapter
Recommendations	Multi-source retrieval, ranking, rule layer, cold start	17.1
Feed ranking	Many goals at once (like, comment, hide, report) → value formula	17.2
Search ranking	A query as input; clicks are biased by position	17.3
Text-to-video search	Documents with no reliable text: index speech and pictures, fuse channels	17.4
Ad click prediction	Probabilities feed an auction, so **calibration** matters	18.1
Fraud detection	An adversary who adapts; asymmetric, per-transaction costs	18.2
Harmful content	Platform-wide scale, high stakes both ways, human review	18.3
People you may know	Graph-based candidates, two-sided acceptance	18.4
Music discovery	Sequences, sessions, exploration vs familiarity	18.5
Visual search	Image embeddings and nearest-neighbour search at scale	18.6
Privacy blurring	Detection where a miss is a privacy breach: recall per person, fail closed	18.7
Event recommendation	Every item is new, perishable, tied to a place and time	18.8
Similar listings	Item-to-item similarity learned from browsing sessions	18.9

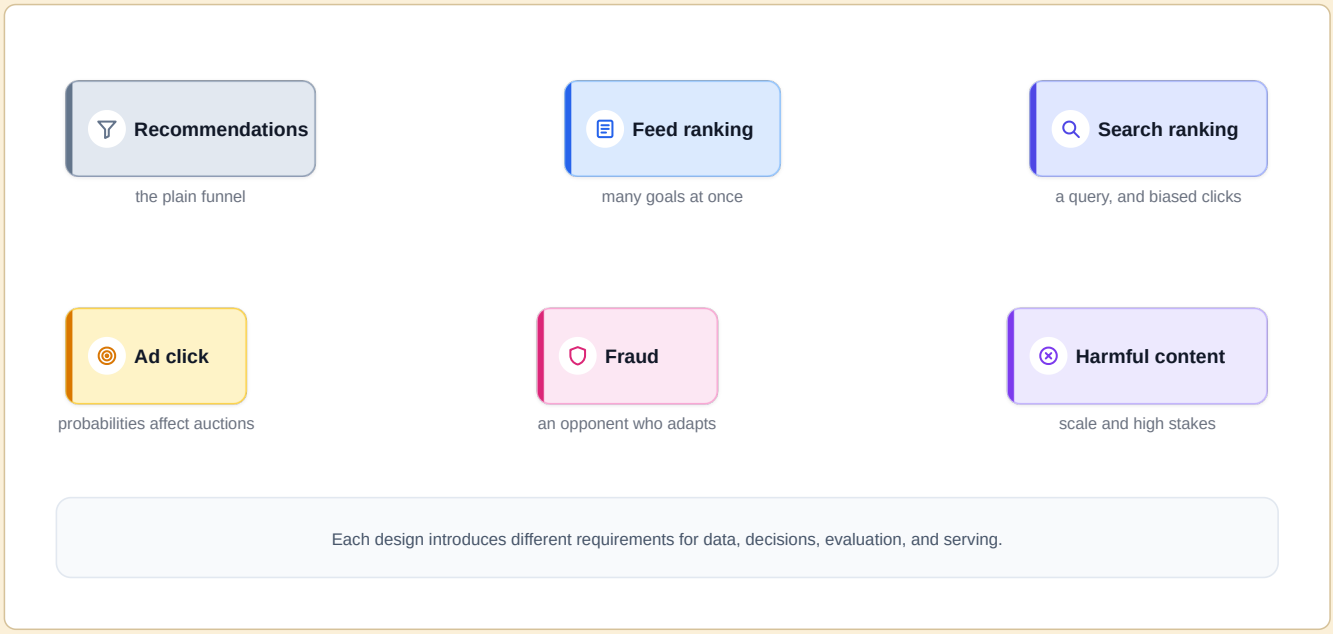


Figure — The main focus of each of the six core designs (course diagram)

Warm-up (do this before 17.1)

Open any app that recommends things to you. Compare its catalog size with how many items you see per session — that ratio is why a retrieval stage exists. Then recall an irrelevant item it showed you: if better items were never pulled in, that's a **retrieval failure**; if they were pulled in but ranked too low, a **ranking failure** (logs or experiments tell which).

17.1 Case study: video recommendations

> "Design the homepage recommendations for a video platform (think YouTube)."

1. Clarify requirements

Ask, then state assumptions out loud:

Question	Assumption used below
Which surface?	Homepage feed only. Search, "up next" and notifications are out of scope.
Catalog size?	50 million videos, thousands uploaded per hour
Users?	100 million daily active users (DAU)
Latency?	Whole page in about 200 ms
Page size?	About 20 videos
Business goal?	People come back and watch more

The scale *is* the design: running a careful model on 50 million videos for each request would blow both the 200 ms budget and the cost budget by orders of magnitude. So say early: *"This is a prediction problem with a ranking-funnel shape."*

2. Frame it as an ML task

- **Input**: user (history, profile), context (time, device, location), candidate video.
- **Output**: a score per (user, video) — e.g. P(click) and expected watch time, or P(watch ≥ 30 s).
- **Category**: retrieval + **learning-to-rank** (sorting items by predicted value).
- **Objective**: predicted engagement per **impression** (one item shown to one user once). Engagement arrives within minutes, so the model can learn fast, while the true business goal (daily return rate, long-term retention) takes weeks.

Why not optimise one proxy? Clicks alone reward misleading thumbnails; watch time alone rewards needlessly long videos. This is the **proxy trap** — a *proxy* being a stand-in metric you optimise because the real one is too slow. The mature framing combines several predicted outcomes and checks the long-term metric in experiments.

3. Data and labels

Three sources:

1. **Interaction logs** — the source of labels. Log every impression with: user and item IDs, timestamp, position on the page, model version, the features used for ranking, then join outcomes (click, watch seconds, like, "not interested") within a defined **observation window** (e.g. 24 h). Respect privacy and retention rules.
2. **Catalog content** — title, tags, description, channel, audio/visual embeddings.
3. **Signup signals** — language, country, declared interests, device.

Labels: an engaged impression is a positive. A skipped impression is only a *weak* negative — the user may never have noticed it. Two flaws to name unprompted:

- **Exposure bias** — you only observe outcomes for items you showed. The model may generalise to unshown items, but their outcomes are never directly measured.
- **Position bias** — an item in slot 1 gets more clicks than the same item in slot 15, so clicks mix relevance with placement. (Fixes in section 10 and in 17.3.)

Content and signup data exist precisely for what logs can't cover: brand-new users and brand-new videos.

4. Features

Group	Examples	Freshness
User	Watch history (IDs + embeddings), subscriptions, language, country	Nightly + in-session updates
Short-term user state	Last 10 videos watched this session, topics skipped	Seconds (streaming)
Item	Channel, topic, length, upload age, embedding, **CTR** (click-through rate = clicks ÷ impressions) and watch-rate aggregates	Stable fields nightly; counts every few minutes
User × item (cross)	Has the user watched this channel before? Similarity of item embedding to the user's recent watches	Request time
Context	Device, hour of day, day of week, network speed	Request time

Leakage to avoid: any feature computed with data from after the impression (e.g. total watch count including the watch you're predicting). Use point-in-time-correct joins (Chapter 16).

5. Model: baseline → advanced

One model per funnel stage.

Baseline: "most popular in your region + newest from your subscriptions." It ships in a week, tests the whole logging pipeline, and gives a number to beat.

Candidate generation — several cheap sources running in parallel, results merged:

1. **Two-tower embeddings (collaborative filtering)** — the main source. One neural "tower" turns user features into a vector, a second turns item features into a vector of the same length; training pushes a user's vector close to the vectors of items they engaged with (and away from sampled negatives). Relevance = dot product. Items are precomputed; only the user vector is computed per request.
2. **Co-occurrence** (viewers of video X frequently went on to watch Y) — item-to-item lists precomputed offline and looked up from the user's recent watches. Cheap, explainable, surprisingly strong. Amazon's item-to-item collaborative filtering (Linden, Smith & York, 2003) is the classic example.
3. **Non-personalised sources** — trending now, fresh uploads, popular in region. They serve new users and keep the page fresh.

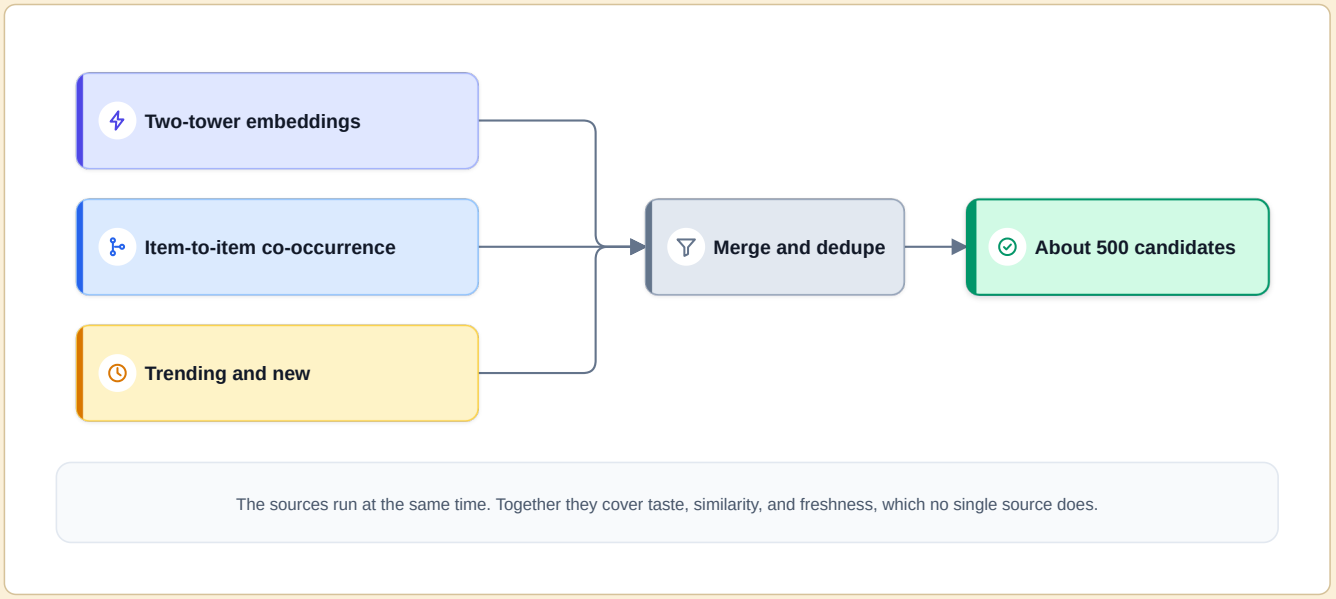


Figure — Three cheap candidate sources running at the same time (course diagram)

Ranker — the candidate list is now small (~500), so use rich features and a stronger model: gradient-boosted decision trees (**GBDT**) as the first ranker, then a deep network predicting several engagement events (click, watch time, like). Most modelling effort goes here. YouTube's published systems follow exactly this split: a candidate-generation network and a separate ranking network (Covington et al., 2016), later a multi-task ranker with a separate small "shallow tower" that models position bias (Zhao et al., 2019).

6. Offline metrics

- **Retrieval**: **recall@K** — take the videos a person really watched during a later, held-out week; what share of them show up in the K retrieved candidates?
- **Ranking**: **AUC** (probability a random positive is scored above a random negative) or a ranking loss; **NDCG@k** if you have graded outcomes.
- Always use a **time-based split** (train on weeks 1–4, test on week 5) — random splits leak the future.

Offline metrics only *select candidates for an online test*; they don't decide launches.

7. Online metrics and A/B testing

- **Primary**: watch time per user, daily return rate (retention) over a long enough window.
- **Guardrails**: "not interested" / dislike rate, policy-violating impressions, latency, creator-side diversity.
- Standard user-level A/B test (16.9); run it long enough to see return behaviour, not just first-week novelty.

8. Serving architecture

Stage by stage, with example numbers:

1. **Candidate generation: 50 M → ~500.** Item vectors are computed offline and loaded into an **approximate nearest neighbour (ANN)** index — a data structure such as **HNSW** (a layered graph of vectors; Malkov & Yashunin, 2016) that finds close vectors without comparing against every item. At request time: fetch the user vector, pull the nearest few hundred in a few ms, merge with co-occurrence and trending results. ANN trades a little recall for huge speed; measure that recall loss and choose index settings (memory, update speed, latency) from benchmarks.
2. **Ranking: ~500 → ~50.** All candidates go to the model in one batch. Treat 20–40 ms for this step as an illustrative budget and verify it with your model, hardware and concurrent load.
3. **Re-ranking: ~50 → ~20 on screen.** Rules: cap slots per channel, lift recent uploads, apply policy demotions, drop just-seen items, honour editor-pinned slots. Keep mandatory rules **outside** the model so policy changes need no retraining; re-check fast-changing restrictions just before display.

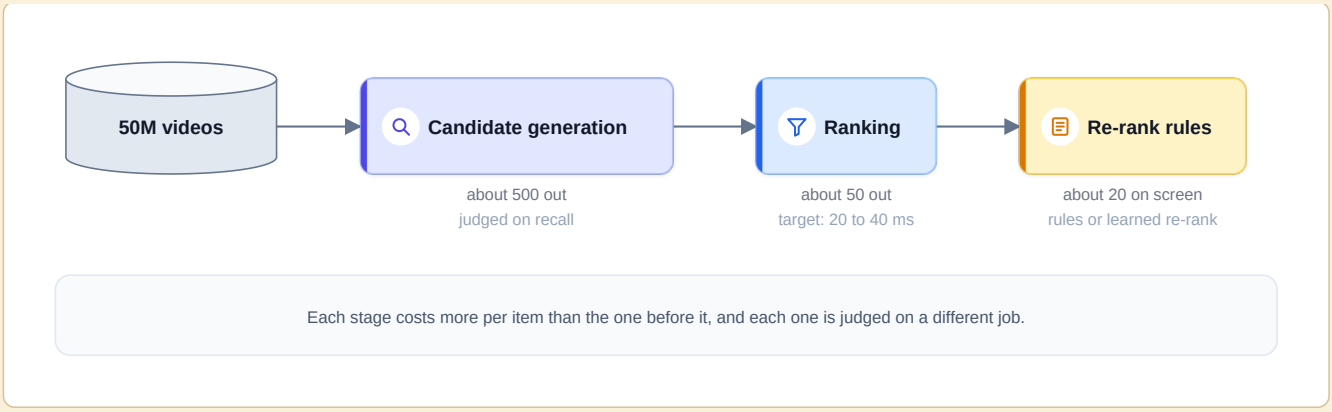


Figure — Example candidate counts and latency targets for the recommendation pipeline (course diagram)



Cold start (bring it up yourself, both kinds):

- **New user**: no history, so collaborative sources are weak. Serve what is popular overall and in their region, lean on what signup told you (language, declared interests, platform), and learn fast in the first session — weight early clicks heavily and update session state in seconds, not overnight.
- **New video**: no engagement yet. Use **content features** (title, tags, audio/visual embeddings) so it can be retrieved by looking like established videos, and give it a small, controlled **exploration boost**. Without that, established videos soak up almost all exposure and new ones never collect data.

9. Scale numbers

Quantity	Arithmetic	Result
Homepage QPS	100 M DAU × 5 loads/day ÷ 86,400 s (assumed 5 loads)	≈ 5,800 avg; ~17 K at a 3× peak
Items scored by ranker	17 K QPS × 500	≈ 8.7 M scores/s at peak
Raw item vectors	50 M × 128 dims × 4 bytes	25.6 GB (before index structure, IDs, metadata, replicas)

The design deliberately moves heavy work offline (embedding and index builds, co-occurrence lists) so each request is cheap. Incremental updates avoid recomputing unchanged items.

10. Monitoring and failure modes

Update loops at three speeds (validate each interval against measured freshness needs): ranker retrained daily; item embeddings and ANN index rebuilt nightly with incremental inserts for new uploads; user short-term state updated in-session via streaming.

Training traps:

- **Position bias** → either model how position affects *examination* separately from relevance (e.g. a separate position tower used only in training), or use **inverse-propensity weighting (IPW)**: weight each example by 1 / (estimated probability the user examined that slot). Validate the propensity estimates and cap large weights. A common shortcut — feed slot number in as a feature during training, then fix it to one value when serving — does not reliably remove the bias. (Worked toy in 17.3.)
- **Proxy trap** → multiple objectives + long-term guardrails.

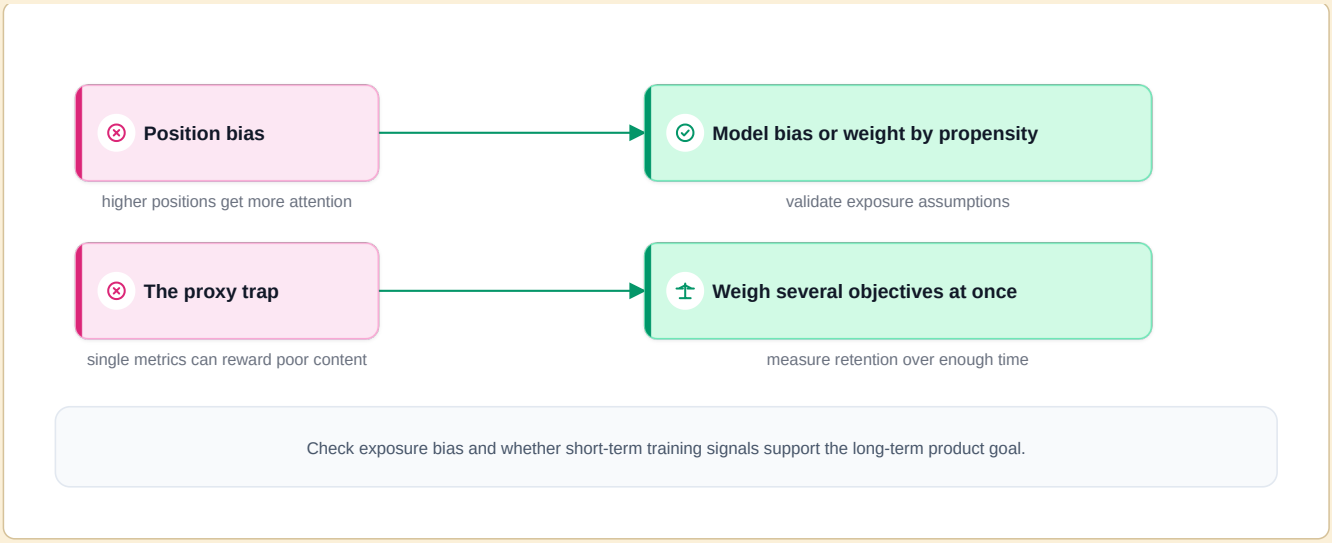


Figure — The two training traps, and the fix for each (course diagram)

Failures and fallbacks:

Failure	Fallback	Monitor
Nightly index build fails	Keep serving the last validated index up to an age limit; still enforce current availability and policy filters	Index age (alert severity rises with staleness)
Ranker times out	Candidate order or trending list, same required filters	Fallback rate, latency
Diversity collapse (feedback loop)	Re-rank diversity rules, exploration	Watch time by category over time narrowing

11. Interview follow-ups

- **"Why not one end-to-end model?"** → Any model smart enough to order items well costs too much to evaluate 50 M times per page load. Stages fix that, each with its own yardstick (recall for the shortlist, top precision for the final order).
- **"Recall looks fine but users seem bored."** → Check the re-rank layer and the objective per user segment: diversity may have collapsed into one taste cluster, or the proxy over-rewards short-term clicks. A shrinking week-by-week category mix is the warning sign.
- **"How do you do real-time personalisation?"** → Session events stream into short-term features within seconds and nudge the user vector (cheap version: a recency-weighted average of the last few videos' vectors); full retraining stays daily.
- **"Someone proposes a new candidate source. Include it?"** → Measure its **marginal** value — watched items it finds that no other source does — then shadow, then a small experiment slice. (The toy below computes exactly this.)

Toy — multi-source retrieval and marginal value (`code/rec_funnel_toy.py`)

Synthetic users and videos live in a hidden 8-dimensional "taste space"; each user watches 35 videos sampled by taste + popularity; the last 5 are held out. Three candidate sources each return K = 100 items; then we take their union.

```
def two_tower(train, k, dim=16, epochs=15, lr=0.05, reg=1e-4, n_neg=5):
    """Matrix factorisation = the simplest 'two-tower': user tower and item tower are embedding tables."""
    U = 0.1 * rng.normal(size=(N_USERS, dim))
    V = 0.1 * rng.normal(size=(N_ITEMS, dim))
    # ... for each (user, watched item): 1 positive + 5 random negatives, logistic loss, SGD
            scores = V[items] @ U[u]
            g = 1 / (1 + np.exp(-scores)) - labels           # logistic-loss gradient
    # ...
        s = V @ U[u]                                          # in production: ANN search, not a full scan
        s[row] = -np.inf                                      # never re-recommend watched items

union = [np.unique(np.concatenate(x)) for x in zip(pop, co, tt)]
# marginal value: share of held-out hits that ONLY this source found
```

Output:

```
2000 users, 3000 videos, 30 past watches each, last 5 held out

candidate source          size  recall@K
popularity                 100     0.129
co-occurrence              100     0.177
two-tower (MF)             100     0.203
union of the three         223     0.306
hits found ONLY by popularity      : 0.022 of held-out watches
hits found ONLY by co-occurrence   : 0.043 of held-out watches
hits found ONLY by two-tower (MF)  : 0.096 of held-out watches
```

What it shows:

- The learned embeddings beat popularity and co-occurrence, but **the union** (≈223 unique candidates) reaches clearly higher recall than any single source — sources overlap only partly, which is why real systems run several.
- **Marginal value** is how to judge a source: popularity finds 12.9% of held-out watches, but only 2.2% that nobody else finds. It still earns its place as the only source that works for brand-new users — a benefit this offline test can't see. (Numbers are synthetic.)

Common mistakes

Weak	Strong
Opens by naming a model architecture	Opens with 50 M items vs 200 ms, which forces the funnel
Treats clicks as ground truth	Raises position bias unprompted, offers a validated correction
Hides diversity and policy inside the model	Separate rule layer that product can change without retraining
Waits to be asked about cold start	Covers both kinds, with different fixes

Same design, other questions

- **"Customers also bought"** — item-to-item co-occurrence carries most of retrieval; the target becomes purchase (rarer, but far less ambiguous). Funnel, cold start and bias story unchanged.
- **Podcast discovery** — hour-long episodes make completion signals late and weak, so audio/transcript embeddings carry more of retrieval and new-item handling becomes the main path.

Self-check

1. Why is retrieval judged on recall and ranking on precision at the top? → Retrieval losses are unrecoverable; ranking only matters for the few items the user sees.
2. Two kinds of cold start, one fix each? → New user: popularity + signup signals + in-session learning. New item: content embeddings + exploration boost.
3. Why is a skipped impression a weak negative? → The user may never have examined it.

TL;DR

Shape	Multi-source retrieval (~500) → rich ranker (~50) → rule re-rank (~20 on screen)
Retrieval	Two-tower embeddings in an ANN index + co-occurrence + trending/new
Cold start	Fresh accounts: popular/regional lists, then learn during the first visit. Fresh uploads: content embeddings plus a small exploration budget
Traps	Position bias (examination model or IPW); proxy trap (multi-objective + guardrails)
Metrics	Offline recall@K / AUC choose what to test; online watch time and retention decide
Cost	Precompute heavy, serve cheap; 25.6 GB of raw vectors; fall back to yesterday's index

---

17.2 Case study: social feed ranking

> "Design the ranking for Instagram's (or LinkedIn's) home feed."

1. Clarify requirements

Say in one sentence that the recommendation funnel carries over — a candidate stage, a ranker over hundreds of items, a rule layer — and then spend your time on what a feed changes:

1. **The candidate pool is social.** It comes mostly from who you follow, not from a 50 M catalog.
2. **"Good" is plural.** There are several positive reactions and several negative ones, and the ranking must weigh them together.
3. **Integrity moves inside ranking.** Engagement alone can reward bait and misinformation, so quality and safety checks sit in the ranking path, not beside it.

Question	Assumption used below
Users	500 M DAU
Candidates per session	A few hundred to a few thousand posts (graph-limited), plus suggested posts from a larger pool
Latency	< 200 ms per ranking call
Calls per session	Several — infinite scroll re-ranks as you scroll (assume 3)
Freshness	New eligible posts rankable within seconds

2. Frame it as an ML task

- **Input**: viewer, post (author, content, age, engagement so far), viewer–author relationship, context.

- **Output**: a *vector* of predictions per post: P(like), P(comment), P(reshare), E[dwell time], P(profile visit), and negatives P(hide), P("see less"), P(unfollow), P(report). (P(x) = the model's estimated probability of event x; E[·] = expected value.)
- **Category**: **multi-task** classification/regression feeding a ranking.
- **Objective**: rank by a **value formula** that combines the predictions — the formula, not any single probability, is what you optimise.

3. Data and labels

- **Retrieval data = the social graph.** Candidates come from (a) **fan-in**: recent posts by accounts you follow; (b) **engagement spillover**: posts that people you're connected to reacted to, written by accounts outside your follow list; (c) a slice of **out-of-network suggestions** retrieved with the same two-tower + ANN machinery as 17.1.
- **Labels = the event stream.** Each (impression, event kind) pair becomes a row, stamped with the feature values the ranker actually saw. Log positives (like, comment, reshare, dwell) *and* negatives (hide, report).
- **Negative events are valuable but not ground truth.** A hide is a much clearer preference signal than a scroll-past, but its meaning depends on context; a report needs validation — it's not automatic proof of a violation.
- **Survey labels**: ask a sample of users "was this post worth your time?" and train a satisfaction head on the answers. Useful beyond engagement, but check response bias, sample size and differences across user groups.

4. Features

Group	Examples
Viewer	Interests, past engagement rates by content type, language, session depth so far
Author	Follower count, posting frequency, historical engagement and hide rates, integrity history
Viewer × author	Interaction history (DMs, comments, profile visits), closeness, how recently they interacted
Post content	Type (photo/video/text/link), text and image embeddings, topic, integrity classifier scores
Post engagement (streaming)	Likes/comments/hides in the last 5 min, 1 h — sparse for new posts
Session	Posts already seen this session (dedupe), in-session reactions

A brand-new post has zero reactions to count, so at first the ranker relies on who wrote it, what it contains and who its likely audience is, until first-hour engagement streams in.

5. Model: baseline → advanced

- **Baseline**: reverse-chronological fan-in, or a single logistic regression predicting P(like). It works, and it's the "Following" tab many apps still offer.
- **Advanced — multi-task learning (MTL)**: one network with a **shared body** (layers that read viewer, post and context once) and a small **head** per event on top. Shared layers learn a common representation, so rare events (reports, reshares) borrow statistical strength from common ones (likes). Variants such as MMoE (multi-gate mixture of experts) help when tasks conflict.
- **Value formula** (the product's definition of "good"):

```
score = w1·P(like) + w2·P(comment) + w3·P(reshare) + w4·E[dwell]  −  w5·P(hide) − w6·P(report)
```

Harm terms are *subtracted*, so a post you'd likely hide ranks lower. Rules for the weights:

- Product, safety and ML teams choose them together — they encode policy.
- **Normalise units**: dwell is measured in seconds while the other heads output values between 0 and 1, so rescale before adding.
- Heads must be **calibrated** (a predicted 0.1 means ~10% happen), or the weighted sum is meaningless.
- Weights live **outside** the model, so many policy changes are a formula change + A/B test, no retraining. Some changes still need new labels or features.

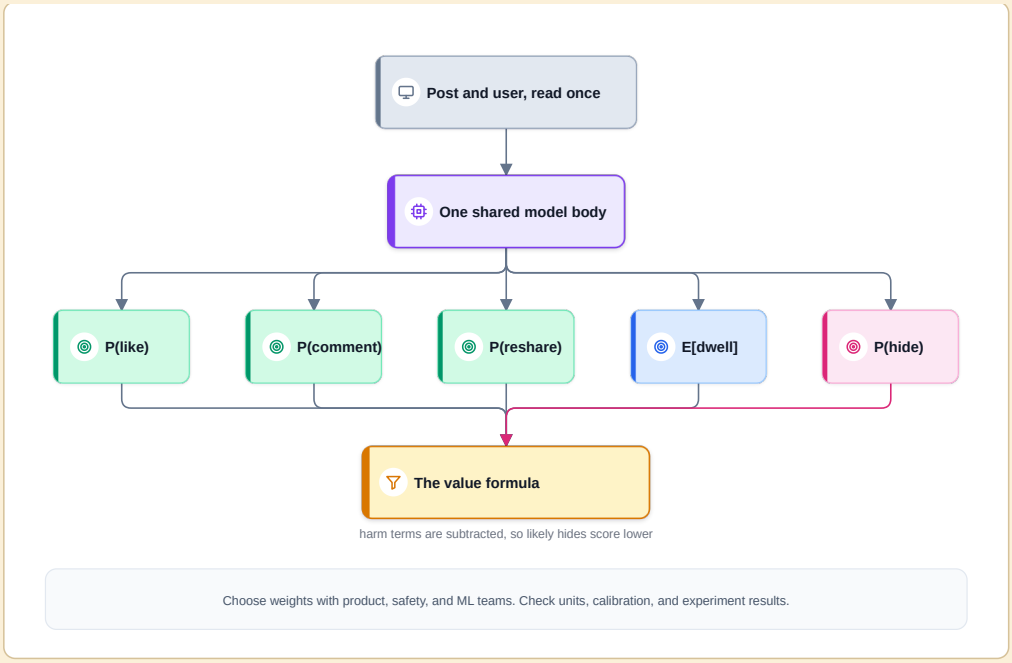


Figure — One shared body, one head per event, one value formula (course diagram)

Real-world: Facebook's 2018 "meaningful social interactions" change re-weighted the feed toward comments and reshares among friends; Twitter's open-sourced 2023 "heavy ranker" predicted about ten engagement probabilities and combined them with fixed weights — e.g. +0.5 for a like, −74 for negative feedback ("show less", mute, block) and −369 for a report.

6. Offline metrics

- Per-head **AUC** and **log loss**; **calibration** per head (reliability curve, expected-vs-actual rate) because the formula adds them up.
- Ranking-level: offline replay of the value formula — expected likes/comments/hides in the top-k under each weight setting (the toy below does this).
- Slice by viewer segment, content type and new vs established authors.

7. Online metrics and A/B testing

- **Primary metric** (one, decides the launch): sessions per user per day, or the share of survey answers saying a post was "worth your time".
- **Guardrails** (can veto a win on their own, limits fixed in advance — 16.9): hide rate, report rate, integrity-violating impressions, creator-side distribution.
- Weight changes are A/B tested against the same primary + guardrails; urgent safety changes may use a faster release process.

8. Serving architecture

Getting posts into candidate lists — the classic news-feed systems question:

- **Push (fan-out on write)**: each new post is written into every follower's precomputed inbox. Fast reads; but a celebrity with 100 M followers triggers 100 M writes.
- **Pull (fan-out on read)**: gather followed accounts' posts when the app opens. Cheap writes; hot accounts get enormous read load.
- **Hybrid**: push for normal accounts, pull for mega-accounts. Mention it, offer to go deeper, and return to the ML if the interviewer declines.

Two-pass ranking: a **light ranker** (a small **distilled** model — trained to imitate the heavy one) trims ~2,000 candidates to ~500; the **heavy multi-task ranker** scores those; the value formula orders them; then the **integrity layer**:

- demote clickbait, engagement bait (posts engineered to farm reactions), misinformation and borderline content using probabilistic classifiers;
- **per-author caps** so one prolific account can't take over a session;
- **diversity** interleaving across friends, groups and suggested content;
- durable **per-user suppression** after explicit "see less".

The classifiers feeding this layer are a separate adversarial ML system (compare 18.2, 18.3).

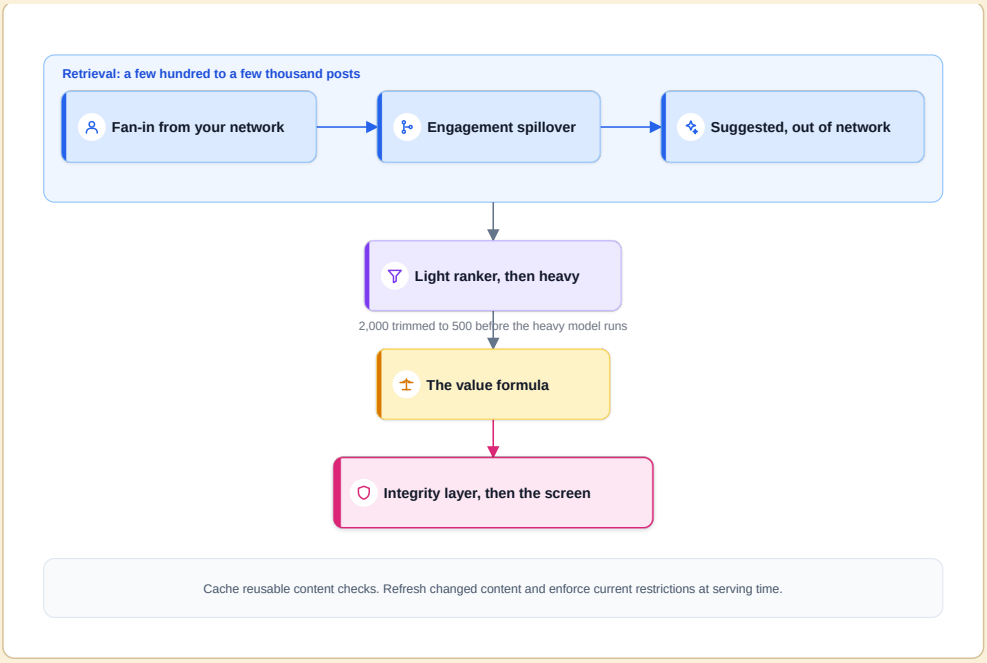
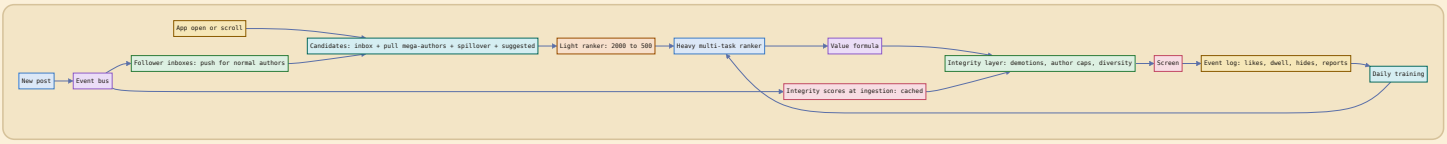


Figure — The feed serving path, from fan-in to the screen (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Ranking calls, average	500 M DAU × 5 sessions × 3 calls ÷ 86,400	≈ 86,800 /s
Peak (assume 2×)	86,800 × 2	≈ 174,000 /s
Posts scored, average	86,800 × 500	≈ 43.4 M scores/s (provision for peak)

What makes this affordable: the two-pass funnel (the light ranker absorbs most volume), **micro-batching** (many scoring requests in one model call), and **item-side feature caching** — very effective because one viral post is scored in millions of feeds. Post-only integrity scores are computed at upload and cached (redone on edit or policy change); viewer-dependent checks still run per request.

10. Monitoring and failure modes

Three loop speeds (examples — set from data availability, measured quality and cost): session state in seconds, engagement aggregates in minutes, model retrained daily. Candidate freshness comes from the streaming path, not a nightly build.

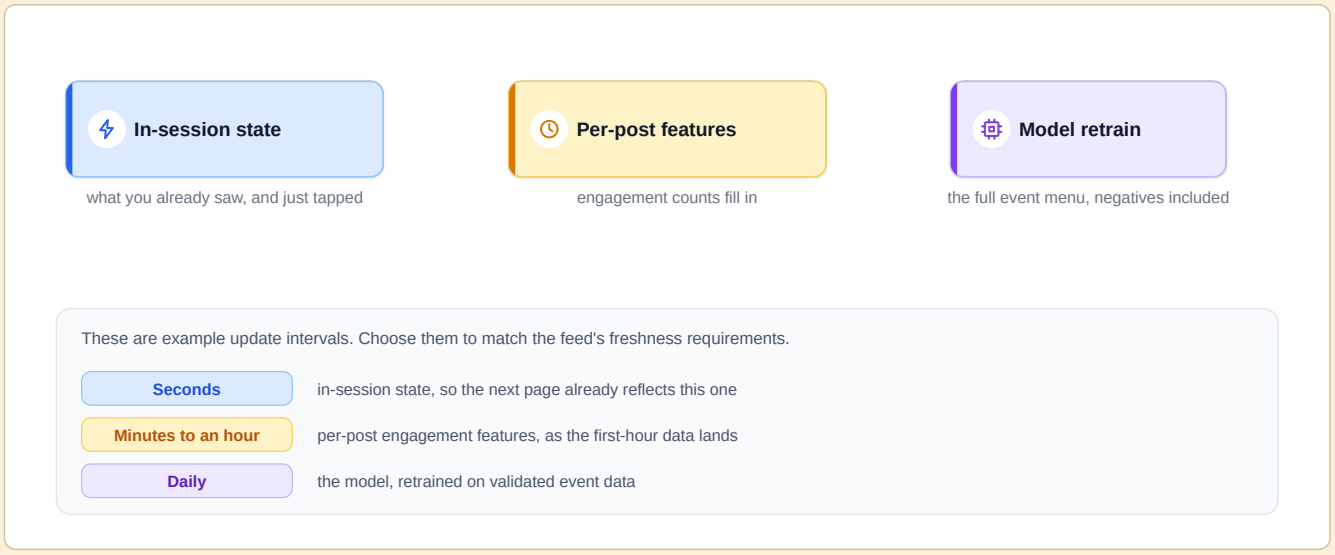


Figure — Three loops running at three different speeds (course diagram)

Failure	Response
Heavy ranker times out at peak	Serve the order the light ranker produced; with no ranking available at all, sort followed posts newest-first. **Tag degraded sessions** so their engagement doesn't pollute training. Privacy, eligibility and safety checks stay on in every path — if they can't run, suppress.
Feedback loop (silent)	The feed shows what it predicts, people react to what they're shown, the model retrains on those reactions — which can amplify low-quality engagement. Monitor satisfaction surveys and content-quality metrics, keep guardrails and the integrity layer.
Calibration drift in one head	The formula silently re-weights; monitor predicted vs actual rate per head.
Recency mis-weighting	Breaking news must surface in seconds; an old personal update may still be valuable. Let the model learn recency per content type.

11. Interview follow-ups

- **"Defend ranking over chronological."** → Most users follow enough accounts that newest-first buries the few posts they'd value; test that ranking improves interaction *and* satisfaction. Concede the downsides (less transparent order, possible bubbles) and offer a newest-first tab plus a "why am I seeing this" link.
- **"A post went up seconds ago — how can it be ranked?"** → The streaming path adds it to candidate pools at once; with no reaction counts, the model uses the author's history, a content embedding and audience priors; a modest freshness boost buys exploration data.
- **"Who sets the weights, and how?"** → Offline sweeps propose settings, A/B tests judge them against the primary metric and guardrails, product leadership signs off because weights are policy. No retraining needed while the heads still predict the right events.

Toy — multi-task heads + value formula (`code/feed_multitask_toy.py`)

Synthetic impressions have 6 features; some posts are "bait" that attract likes *and* hides. A small PyTorch network with a shared body and three heads (like, comment, hide) is trained with one binary cross-entropy loss per head, summed. Then 500 simulated users each get 200 candidates, and we compare what the top 10 contains under three rankings.

```
class MultiTask(nn.Module):
    def __init__(self):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(N_FEAT, 32), nn.ReLU(), nn.Linear(32, 16), nn.ReLU())
        self.heads = nn.ModuleList([nn.Linear(16, 1) for _ in range(3)])   # like, comment, hide

    def forward(self, x):
        h = self.body(x)                                  # shared representation, computed once
        return torch.cat([head(h) for head in self.heads], 1)   # 3 logits

# training: loss = sum(BCEWithLogits(logits[:, k], y[:, k]) for k in range(3))
formulas = {"P(like) only": np.array([1.0, 0.0, 0.0]),
            "gentle 1/2/-3": np.array([1.0, 2.0, -3.0]),
            "strong 1/4/-10": np.array([1.0, 4.0, -10.0])}
top = np.argsort(-(ph @ w))[:10]          # ph = predicted [P(like), P(comment), P(hide)]
```

Output:

```
event rates in logs  like=0.259 comment=0.077 hide=0.060
head like     AUC=0.818  mean pred=0.244 vs actual=0.252
head comment  AUC=0.782  mean pred=0.071 vs actual=0.082
head hide     AUC=0.868  mean pred=0.057 vs actual=0.061

expected events per user in the top 10 (true probabilities):
ranking           likes  comments  hides
P(like) only       8.16      1.08   2.34
gentle 1/2/-3      5.37      2.79   0.33
strong 1/4/-10     3.51      3.18   0.17
```

What it shows:

- Ranking by P(like) alone maximises likes but fills the top 10 with bait: 2.34 expected hides per user.
- The **gentle** formula cuts hides by ~86% (2.34 → 0.33) and more than doubles comments, at the cost of about a third of the likes; the **strong** one goes further. Which is "right" is a **product decision** settled by A/B test — exactly why the weights sit outside the model.
- Mean predictions are close to actual rates for each head — the calibration check you'd run before summing heads.

Common mistakes

Weak	Strong
Rebuilds the whole recommendation funnel for ten minutes	One sentence "it transfers", then spends the time on what's feed-specific
Ranks on one P(click)	Multi-task heads + a product-owned value formula
Filters hides and reports out of training as noise	Trains on negative feedback, weights it by evidence, applies durable suppressions
Puts integrity into the training objective only	Separate post-formula integrity layer that policy teams can tune

Same design, other questions

- **Notification ranking** — each candidate is push-or-hold and users tolerate only a few pings a day; predicted annoyance (e.g. P(turns notifications off)) gets a big negative weight and the rule layer is mostly a frequency cap.
- **A "Following" tab next to "For You"** — pure fan-in, near-pure recency; the effort goes into logging both tabs identically so one training and experimentation pipeline serves both.

> **Self-check**

1. Why must the heads be calibrated? → The formula adds weighted probabilities; a head that systematically over-predicts silently gets more weight.
2. What's wrong with push fan-out for celebrities? → One post = one write per follower (100 M writes); hybrid push/pull fixes it.
3. Why tag degraded sessions? → Their engagement came from a fallback ranking; training on them unmarked teaches the model the wrong thing.

> **TL;DR**

Reused from 17.1	Retrieve → rank → rules; mention it in a sentence and move to what's new
Candidates	Social graph fan-in + spillover + suggested (embedding retrieval); push/pull hybrid
Ranker	Multi-task heads, one per event; value formula owned by product
Integrity	Post-formula demotions, author caps, diversity; negative feedback trained on
Freshness	Streaming candidates; content/author features for brand-new posts; session state
Metrics	One primary session metric + guardrails (hides, reports) + worthwhileness surveys
Scale	~86.8 K avg ranking calls/s, ~43 M scores/s; two-pass, micro-batching, caching

---

17.3 Case study: e-commerce search ranking

> "Design product search for an online store (think Amazon)."

1. Clarify requirements

It's the funnel again, reshaped by one new input: the **query**. Recommendations *infer* what you want from history and context; a query *states* your current goal — though often incompletely or ambiguously ("apple", "case"). The query touches every stage, so the system has three jobs in order: **understand the query → retrieve against it → rank with it**. Say that early: *"recommendation funnel plus a query-understanding stage at the front."*

Question	Assumption used below
Catalog	100 M products
Traffic	10 K queries/s at peak
Latency	~300 ms end to end: a few ms for query understanding, the biggest slice for the heavy ranker
Scope	Search results page; autocomplete out of scope unless asked
Business	Search drives purchases directly, so revenue matters as well as relevance

2. Frame it as an ML task

- **Input**: query text, user, context (locale, device), candidate product.
- **Output**: a relevance/purchase score per (query, product) — e.g. P(purchase | query, product), or a graded relevance estimate.
- **Category**: **learning-to-rank**. Three standard loss styles: **pointwise** (predict each item's label independently), **pairwise** (predict which of two items is better — e.g. RankNet), **listwise** (optimise a whole-list metric such as NDCG). **LambdaMART** — gradient-boosted trees trained on pairs whose weights come from how much swapping them would change NDCG — sits between the last two and is the classic industrial default.
- **Supporting ML tasks** inside query understanding: spelling correction (sequence-to-sequence or noisy-channel), query classification, attribute extraction (named-entity-style tagging).

3. Data and labels

1. **Catalog**: titles, descriptions, attributes (brand, colour, size), category tree, price, ratings, stock.
2. **Search logs**: query, results page with positions, clicks, add-to-carts, purchases, reformulations.
3. **Human relevance judgments**: raters grade (query, product) pairs from *irrelevant* to *highly relevant* — expensive but unbiased, used for evaluation and for training heads.
4. **The query itself** is data. Query frequencies follow a roughly **Zipf-like distribution** — a few head queries are very common, the long tail is enormous — which matters for caching (section 8).

Labels: graded judgments are the clean labels; behaviour (clicks < add-to-cart < purchase) gives cheap, plentiful, biased ones. **Position bias** is strongest here: people look at the top results, so top results get clicked partly *because* they're on top. Training on raw clicks

teaches the new ranker to copy the old one.

4. Features

The ranker's defining ingredient is **query–item features** — how well *this* item matches *this* query:

Group	Examples
Query–item match	BM25 text score per field (title, brand, description), embedding similarity, attribute agreement (query says "red", item is red), category match
Item quality	Rating, review count, sales velocity, return rate, in stock, shipping speed
Query	Length, class (product / brand / help), head vs tail, predicted category
Personalisation	User's brand and price-range history, past purchases (capped influence)
Context	Locale, device, time, season

5. Model: baseline → advanced

Query understanding (runs first, a few ms, results cached for repeated queries):

1. **Normalise and spell-correct**: "iphon 15 cas" → "iphone 15 case". Correction models are trained on **reformulation logs** — pairs where users retyped a query.
2. **Expand with synonyms**: "sofa" also matches "couch", "running shoes" matches "sneakers". Historically hand lists plus **co-click mining** (two words are synonyms if their queries lead to clicks on the same products); increasingly handled by embeddings.
3. **Classify intent and extract structure**: product vs brand vs help query; "red nike shoes size 10" → brand=nike, colour=red, size=10, category=shoes. **Reliable, explicit** constraints become **hard filters**; uncertain interpretations become ranking features (or prompt a clarification).

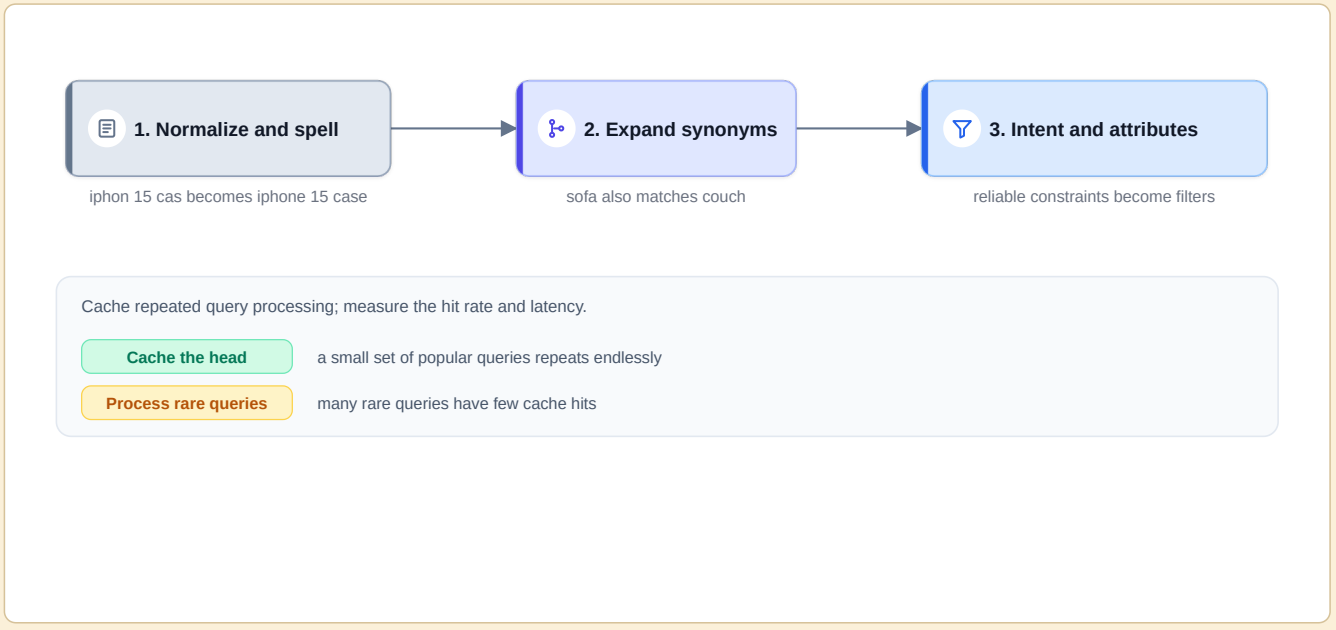


Figure — The three jobs of the query-understanding stage (course diagram)

Retrieval — two engines in parallel:

- **Lexical** via an **inverted index** (like a book's index for the whole catalog: for each word, the list of products containing it), scored with **BM25** — a formula that rewards query words appearing in the document, more for rare words (IDF), with diminishing returns for repeats and a correction for document length. It shines on literal hits — part numbers, brand names, tokens such as "X200-B" — and is fast and explainable. Its blind spot is **vocabulary mismatch**: a shopper typing "something warm for the snow" misses a parka described only as a "thermal winter jacket", because no word matches.
- **Semantic** via embeddings: a two-tower model trained on (query, clicked product) pairs encodes the query; an ANN index returns the nearest products. It captures meaning across wordings but is unreliable on exact codes and can confidently return a plausible wrong item.

Ranking — two levels at this volume: a light **L1** ranker (e.g. a small GBDT on cheap features) trims the union; a heavy **L2** ranker (LambdaMART-style GBDT or a deep model with query–item features) orders the final list. Optionally, a **cross-encoder** (a transformer that reads query and product text *together* — slower, sharper) re-ranks only the top ~20 if the latency budget allows.

Baseline: BM25 + popularity + in-stock filter. Everything above is the upgrade path.

6. Offline metrics

NDCG@k (normalised discounted cumulative gain) — rewards putting the most relevant results near the top:

1. DCG@k = Σ over positions i = 1..k of $(2^{rel_i} - 1) / \log_2(i + 1)$
2. NDCG@k = DCG@k / IDCG@k, where IDCG is the DCG of the perfect ordering → range 0 to 1.

Worked example (k = 3): results with relevance [3, 0, 2]: DCG = 7/1 + 0/1.585 + 3/2 = 8.5. Ideal order [3, 2, 0]: IDCG = 7 + 3/1.585 + 0 = 8.89. NDCG = 8.5 / 8.89 ≈ 0.956. Swapping the 0 and the 2 would reach 1.0.

Also: **MRR** (mean reciprocal rank of the first relevant result) for navigational queries, and recall of the retrieval stage. Compute on human-graded sets; use behaviour-based labels only after bias correction.

7. Online metrics and A/B testing

- **Primary**: session success rate (a search session that ends in add-to-cart or purchase), revenue per search.
- **Also**: add-to-cart rate, click-through rate on results.
- **Dissatisfaction signals**: **reformulation**, **abandonment** and **zero-results** rates — interpret with care (a reformulation can be exploration).
- **Guardrails**: relevance of sponsored slots, latency, zero-results rate by query segment.
- **Interleaving** (mix rankers A and B in one list, see which side gets clicks; 16.9) is a more sensitive way to compare rankers.

8. Serving architecture

Run both retrieval engines **in parallel** and take the **union** (a few hundred candidates). This isn't redundancy: each catches what the other misses. The same **hybrid retrieval** pattern (BM25 + vectors) reappears in RAG systems in the AI book. Each index needs a defined **update path** (both inverted and ANN indexes can be updated incrementally, depending on the implementation) — measure how long a new or changed product takes to become searchable.

The **business layer** matters more in search than anywhere else: anything out of stock or not shippable to the user is removed outright; boosts for margin or strategy are applied afterwards; **paid placements** appear only in marked slots and only if they clear a minimum **relevance floor**. Money and relevance genuinely pull in different directions — present that as a decision product leadership owns, protected by guardrail metrics.

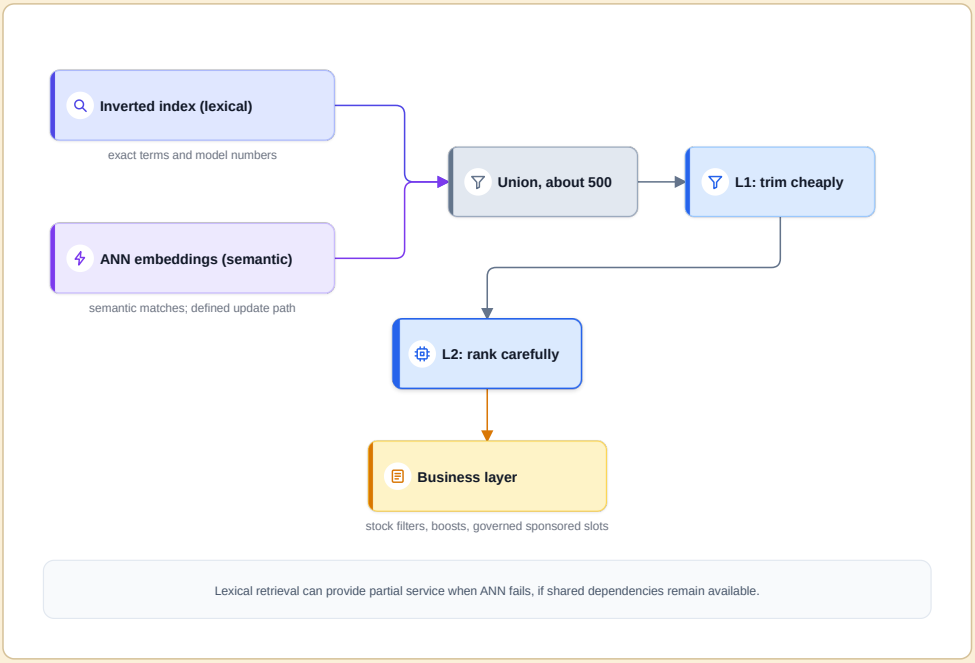
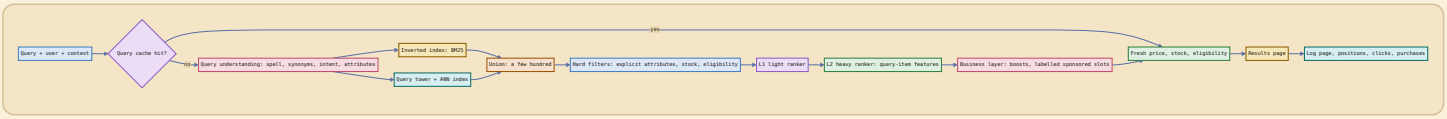


Figure — Two retrieval engines, one union, then two rankers (course diagram)



Caching done right: cache the **ranked list of IDs** for head queries — with locale, permissions and personalisation in the cache key — while the fast-changing fields — **price, stock, delivery eligibility** — are looked up live as the page is assembled, then checked once more at checkout. Inventory events invalidate the filter layer. A cache holding yesterday's availability sells products you can't ship: a correctness bug even though the service "works".

9. Scale numbers

Quantity	Arithmetic	Result
Peak QPS	given	10,000 /s
Cache misses (assume 40% miss — *measure it*)	10,000 × 0.4	4,000 /s pay full price
L2 scores	4,000 × 200 candidates after L1	800 K scores/s
Per-item LLM ranking	4,000 × 200 LLM calls	800 K LLM calls/s — not viable; benchmark before any LLM on the request path

The head is cheap (cached); the long tail pays for query understanding, two retrievals and two rankers. LLMs still help **offline**: generating training labels, enriching product text, rewriting queries.

10. Monitoring and failure modes

Training loop: log every results page with its features and positions → retrain rankers on **position-corrected clicks** and **within-session preference pairs** (e.g. a purchased result ranked below a skipped one) → gate releases on NDCG offline and session success online. Preference pairs help but aren't automatically unbiased: a skip doesn't prove the user saw the item.

Position-bias fixes (same family as 17.1, sharpened):

- Model examination separately from relevance (a click happens if the user *examined* the slot *and* found the item relevant).
- **IPW**: weight each click by 1 / P(examined at its position). Estimate propensities with randomisation experiments (e.g. swapping adjacent results for a small share of traffic) or from multiple rankers' logs; validate them and cap large weights. (Joachims, Swaminathan & Schnabel, 2017, is the standard reference for unbiased learning-to-rank.)

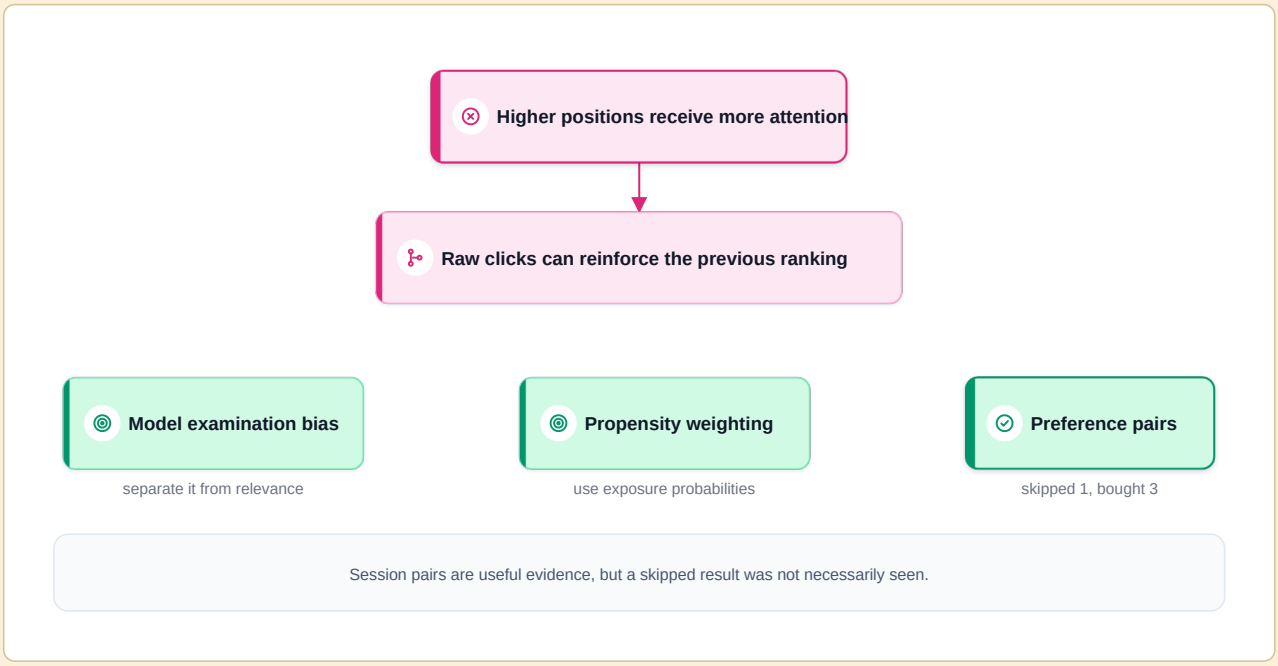


Figure — Position affects click labels; each correction needs validated assumptions (course diagram)

Monitor by query segment: reformulation and zero-results rates reveal problems that aggregate revenue hides (e.g. a broken synonym list for one category).

Failure	Fallback / effect
Embedding service / ANN index down	Lexical still serves exact and head queries (if it doesn't share the failed infrastructure); long-tail natural-language queries degrade — watch reformulation and zero-results rates as the outage dashboard
Query understanding down	Search the raw query
Stale cache	Wrong availability → refresh volatile fields at render, validate at checkout

11. Interview follow-ups

- **"A search returns nothing. What do you do?"** → First the cause: no inventory, or vocabulary mismatch? Then fallbacks — relax *uncertain inferred* filters (never explicit ones or eligibility), try spelling variants, retry semantic-only, offer category browsing — each logged as a health metric.
- **"How far would you personalise search?"** → Personal signals may reorder results that already match the query, never outrank what the user explicitly typed. Personal features go into L2 with a capped effect; check each user segment in experiments.
- **"Why not let an LLM rank everything?"** → At 10 K QPS per-item LLM calls blow latency and cost (section 9). Use LLMs offline for labels and enrichment, and at most a small cross-encoder over the top 20.

Toy — position bias and IPW (`code/search_position_bias_toy.py`)

Each synthetic query has 20 products with 3 features: x0 and x1 drive true relevance (graded 0–3); x2 is a "promoted" score the **old ranker** leaned on, with **no effect** on relevance. Users click only if they **examine** the slot (P = 1/position) **and** are attracted (more likely for higher relevance). We train a linear scorer on the logs three ways and measure NDCG@10 against true relevance on fresh queries.

```
PROPENSITY = 1.0 / POS                                  # P(examined at rank k) = 1/k

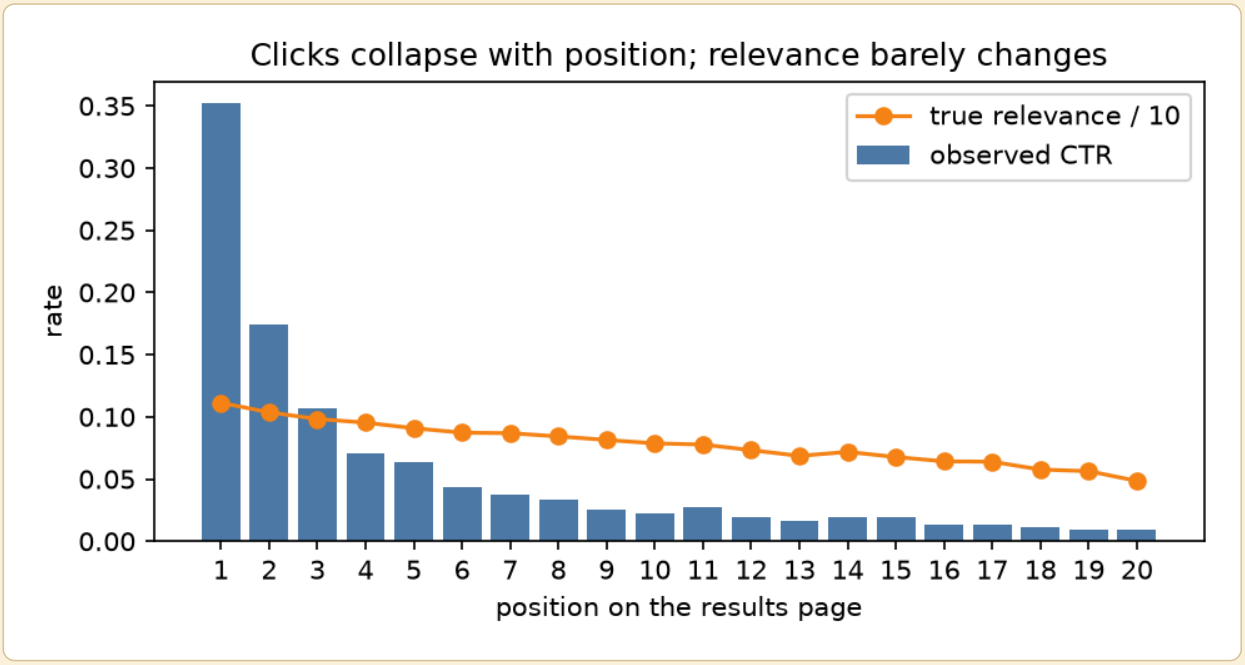
def fit_linear(x, target, weight_pos, weight_neg, epochs=300):
    """Weighted logistic loss: w_pos*log(s) + w_neg*log(1-s), full-batch Adam."""
    # ...
        loss = -(wp * logsigmoid(z) + wn * logsigmoid(-z)).mean()

ipw = clicks / np.clip(prop, 0.1, None)                 # clip: cap weights at 10x
models = {
    "naive clicks": fit_linear(xs, clicks, clicks, 1 - clicks),
    "IPW clicks":   fit_linear(xs, clicks, ipw, 1 - ipw),   # unbiased pointwise loss (weights may be < 0)
    "oracle (true rel)": fit_linear(xs, None, (rs >= 2).astype(float), (rs < 2).astype(float)),
}
```

Output:

```
CTR by position (1..10): [0.352 0.174 0.107 0.071 0.064 0.044 0.038 0.033 0.026 0.022]
mean true relevance by position (1..10): [1.11 1.04 0.98 0.95 0.91 0.87 0.87 0.84 0.81 0.79]

model                 w_x0   w_x1   w_x2  NDCG@10
naive clicks          1.03   0.50   0.82    0.820
IPW clicks            1.28   0.79   0.22    0.948
oracle (true rel)     3.72   2.48   0.01    0.956
old ranker                                   0.514
saved img/search_ctr_by_position.png
```



What it shows:

- CTR falls 16× from position 1 to 10 while true relevance barely moves (1.11 → 0.79) — clicks mostly measure **where** an item was shown.
- The **naive** model gives the irrelevant promoted feature x2 a large weight (0.82): it learned to copy the old ranker. **IPW** shrinks that to 0.22 and reaches NDCG 0.948, close to the **oracle** (0.956) trained on true labels we'd never have in real life.
- IPW falls short of the oracle partly because weights are clipped at 10× — a little bias traded for much lower variance — and its guarantee holds only if the propensities are right.
- Why it works: with click c and propensity p, the weight c/p has expected value P(attracted), so on average the loss equals the loss on true attraction labels. The negative weight 1 − c/p is below zero for clicks below slot 1 — fine in a custom PyTorch loss, rejected by libraries that forbid negative weights.

Common mistakes

Weak	Strong
Jumps straight to embedding retrieval	Runs keyword and vector retrieval side by side and names each one's blind spot (part numbers vs paraphrased needs)
Treats the query as an opaque string	A whole stage for spelling, synonyms, intent, attributes; reliable constraints as filters, uncertain ones as features
Trains on raw clicks	Names position bias; validated correction; treats session pairs as imperfect evidence
Pretends sponsored results don't exist	Names revenue vs relevance as a governed trade-off with a relevance floor

Same design, other questions

- **App-store search** — small catalog, extreme head skew, exact title/brand matching dominates; success = install-and-retain.
- **Job search** — **two-sided** relevance (seeker fits the job *and* employer wants the seeker); qualifications as hard filters; postings expire, so freshness matters.

Self-check

1. Why run lexical and semantic retrieval together? → Lexical handles exact identifiers and rare tokens; semantic handles vocabulary mismatch.
2. Compute NDCG@2 for relevances [0, 3]. → DCG = 0 + 7/1.585 = 4.42; IDCG = 7 + 0 = 7; NDCG ≈ 0.63.
3. What does IPW weight each click by? → 1 / P(examined at that position), with weights capped.

TL;DR

New stage	Query understanding (fix typos, add synonyms, detect intent, turn reliable attributes into filters); frequent queries are cached
Retrieval	Hybrid: inverted index + BM25 (lexical) and two-tower + ANN (semantic), union
Ranking	L1 → L2 with query–item features; optional cross-encoder on the top ~20
Biased clicks	Position bias → examination model or IPW; NDCG on graded judgments offline; session success online
Business	Hard filters, boosts, labelled sponsored slots with a relevance floor
Failure	Lexical as fallback; refresh volatile fields; zero-results and reformulation by segment

---

17.4 Case study: text-to-video search

> "Users type a query such as 'how to fold a fitted sheet' or 'goal from last night's match'. Design the search system that returns videos (think YouTube search)."

1. Clarify requirements

The recognition sentence: **"This is the search funnel of 17.3, but the documents are videos, and most of what a video contains is not written down anywhere."** A video's title may be empty, in another language, or clickbait; what it shows lives in its **frames** and what it says in its **audio**. So the new work is on the *indexing* side: turn pictures and speech into something a text query can match. Query understanding, hybrid retrieval, L1/L2 ranking and position bias carry over from 17.3.

Question	Assumption used below
Corpus	1 B searchable videos; ~1 M new uploads/day, average 10 minutes long
Traffic	20 K queries/s at peak
Latency	~300 ms for the results page
In scope	Text query → ranked videos; a "jump to the moment" timestamp when the match is one scene of a long video
Out of scope	Query by image or clip (that is 18.6), autocomplete, ads
Freshness	A new upload should be searchable within minutes by its title, and within hours by its content

Business goal: a **satisfied** watch — a click followed by 5 seconds of viewing and a return to the results is a failure.

2. Frame it as an ML task

- **Input**: query text, user and context (language, region, device), candidate video with all its evidence (text fields, transcript, frames).
- **Output**: a relevance score per (query, video), then a ranking score that also predicts **satisfied watch** (e.g. P(watch ≥ 30 s or ≥ 50% of a short video | shown for this query)).
- **Category**: **cross-modal retrieval** (the query is text, the item is pictures + sound + text) followed by **learning-to-rank**.
- **Two supporting ML tasks run at upload time**, not query time: **speech recognition** (**ASR**, automatic speech recognition: audio → transcript) and **visual embedding** (frames → vectors in the same space as text). Optionally **OCR** (optical character recognition: text burned into the frames, such as slide titles or scoreboards).

3. Data and labels

1. **Video metadata**: title, description, tags, channel, language, upload time, duration, chapters if the creator added them.
2. **Derived content**: ASR transcript with timestamps, OCR text, sampled frames (e.g. one per second, or one per detected shot — a **shot** is a continuous run of frames from one camera cut).
3. **Search logs**: query, results with positions, clicks, **watch time after the click**, returns to the results page ("pogo-sticking"), reformulations.
4. **Human relevance ratings** on sampled (query, video) pairs — the clean, unbiased evaluation set, split by query type.

Labels:

- For the **ranker**: satisfied watches after a search are positives; short clicks followed by a quick return are strong negatives; skipped results are weak negatives. Position bias applies exactly as in 17.3, so log positions and correct for it.
- For the **text–video encoder**: (query, satisfyingly watched video) pairs are best. Before logs exist, **weak pairs** do the job: (title, video) and (transcript sentence, frames at that timestamp). Research datasets work the same way: MSR-VTT pairs clips with human captions (Xu et al., 2016); HowTo100M uses the narration of 1.2 M instructional videos as free, noisy captions (Miech et al., 2019).

4. Features

Group	Examples
Query–text match	BM25 on title, description, tags, transcript and OCR (separate score per field); text-embedding similarity
Query–visual match	Similarity between the query vector and the video-level visual vector; **best-matching segment** score and its timestamp
Channel availability	Has transcript? Title empty or very short? Language of speech vs language of query
Video quality	Satisfied-watch rate for similar queries, likes vs dislikes, completion rate, channel authority, upload age
Query	Class (navigational: a channel or a specific video; informational: "how to…"; fresh: news, sport results), language, head vs tail
Personalisation	Subscribed channel, languages the user watches in (capped, as in 17.3)

Availability features matter: a ranker cannot tell a bad title match from a missing title unless it is told the title is missing (see the toy).

5. Model: baseline → advanced

Baseline: BM25 over title + description + tags, times popularity. It fails on exactly the videos this design is about: thin or misleading text.

Upgrade 1 — index what is said. Run ASR on every upload and index the transcript (with timestamps) as another text field. Modern ASR such as Whisper (Radford et al., 2022, trained on 680 K hours of weakly labelled audio) makes speech searchable across many languages. A large gain for tutorials and talks; nothing for music or silent footage.

Upgrade 2 — index what is shown. A **shared text–image embedding space** in the style of CLIP (Radford et al., 2021; contrastive training on image–caption pairs so that a caption and its picture land close together — the same idea as 18.6). Encode sampled frames with the image tower and the query with the text tower; similarity is a dot product. From frames to a video:

- **Mean-pool** all frame vectors into one video vector — one entry per video in the ANN index, cheap, but a short scene gets diluted.
- **Segment vectors** — one vector per shot or per few seconds — find moments inside long videos, at many times the index size.
- **Temporal models** (a small transformer over frame vectors) learn which frames matter. CLIP4Clip (Luo et al., 2022) compared these choices on top of CLIP: parameter-free mean-pooling is strong on small datasets; learned temporal models need more training data. Fine-tune the towers contrastively (InfoNCE, 18.6) on (query, satisfied-watch video) pairs once search logs exist.

Fusion. Each channel (lexical text, text embedding, visual embedding) returns its own top few hundred. Merge them:

- **Reciprocal rank fusion** (RRF; Cormack, Clarke & Büttcher, 2009): score = Σ over channels of 1 / (k + rank), with k ≈ 60. It needs no training and ignores the fact that the channels' raw scores are on different scales.
- **A learned ranker** (L1 then L2, as in 17.3) with every channel's score *and* the availability features as inputs. This is where the real gain is: it learns that a missing transcript is not evidence against a video.

Ranking objective: multi-task heads — P(click) and P(satisfied watch | click) — combined into one score, the product of the two (the 17.2 value-formula idea), plus a freshness term for queries classified as fresh.

6. Offline metrics

- **Retrieval**: recall@K **per channel and after fusion**, on the rated set. Also **marginal recall** per channel (videos only that channel found — the 17.1 test for adding a source).
- **Ranking**: NDCG@10 on graded ratings; MRR for navigational queries.
- **Slices** that aggregate numbers hide: query class, language, videos with no title / no speech, long videos with moment-level matches, fresh uploads.
- **Moment accuracy**: for rated moment queries, is the returned timestamp within a few seconds of the right scene?

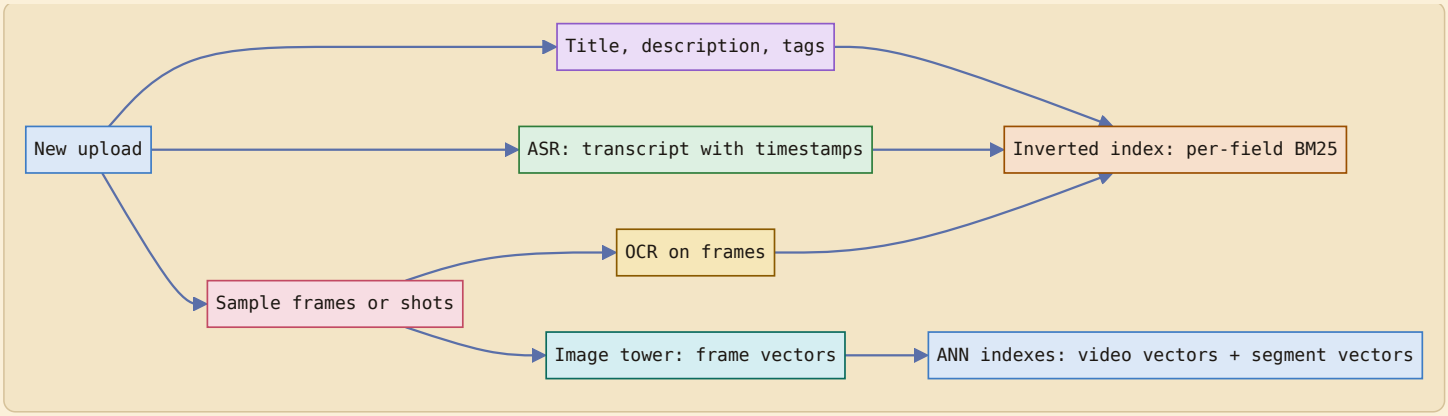
7. Online metrics and A/B testing

- **Primary**: search success rate = share of searches ending in a satisfied watch; **time to first satisfied watch**.
- **Dissatisfaction**: quick returns to the results page, reformulations, abandonment.
- **Guardrails**: harmful or misleading videos in results (18.3), latency, creator diversity, freshness on fresh queries. **Interleaving** (16.9) compares rankers with far fewer users.

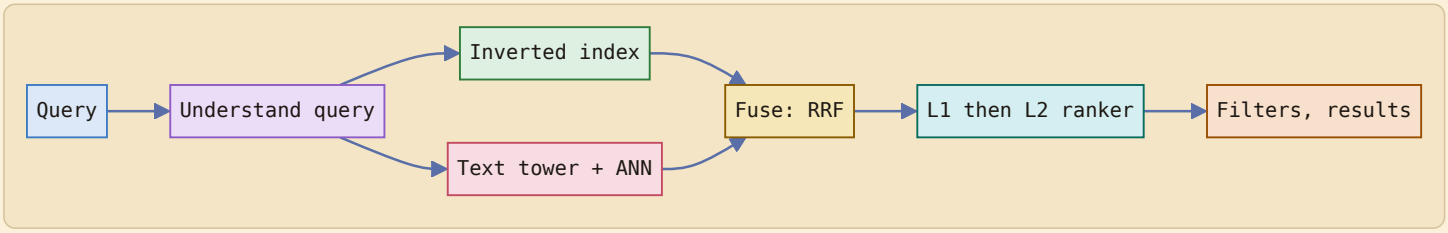
8. Serving architecture

Two pipelines: an *indexing* pipeline that runs once per upload, and a *query* path that runs per search.

Indexing path (per upload):



Query path (per search):



Query understanding = spelling, language, query class, freshness intent (17.3). The ANN lookup covers video-level and segment-level vectors; fusion keeps ~1,000 candidates; the L2 ranker predicts click × satisfied watch; policy, safety and region filters run last, and results carry a moment timestamp when a segment matched.

Staged freshness: title and description are indexed within minutes; ASR and visual vectors follow in minutes to hours. Until then the ranker sees "transcript: not yet available", not "no match". For fresh queries a small **real-time index** of the last day's uploads is searched alongside the main one.

Caching: head queries' ranked ID lists are cached (language and region in the key; short TTLs for fresh queries); policy status (taken down, age-restricted, region-blocked) is checked at render time.

9. Scale numbers

Quantity	Arithmetic	Result
Video-level vectors	1 B × 512 dims × 2 bytes (float16)	1 TB raw → shard it; with PQ (18.6) at 64 B/vector, 64 GB
Segment vectors	long videos only, say 100 M × 30 segments × 64 B (PQ)	192 GB — 3× the video-level index, so index segments selectively
Frame encoding	1 M uploads/day × 600 s ÷ 2 s per sampled frame	300 M frames/day ≈ 3,500/s — a steady GPU fleet
ASR	1 M × 10 min of audio	10 M audio-minutes/day to transcribe
Query side	20 K QPS × (1 text-tower pass + 3 index lookups)	cheap compared with indexing; L2 on 20 K × 300 = 6 M scores/s

The expensive half of this design is **indexing**, not querying — the reverse of 17.3, where documents are cheap to index and queries are expensive to rank.

10. Monitoring and failure modes

- **Indexing lag**: time from upload to (a) title-searchable, (b) transcript-searchable, (c) visually searchable. Alert on each.
- **Coverage**: share of videos with a transcript, with visual vectors, per language. A broken ASR language model silently makes a whole language unsearchable by speech.
- **Encoder upgrades**: a new visual encoder needs a full re-embed and an atomic swap of model + index (the 18.6 rule); budget weeks of GPU time for 1 B videos.
- **Keyword stuffing**: creators pack popular words into titles and tags; content channels plus satisfied-watch labels are the defence.

Failure	Fallback / effect
Visual ANN index down	Text channels still serve; visually-only-findable videos disappear — watch success rate on "no title / no speech" slice
ASR backlog	Serve without transcripts; ranker treats them as missing, not as no-match
Stale cache on a taken-down video	Render-time policy check drops it

11. Interview follow-ups

- **"Why not just search titles and descriptions?"** → A large share of videos have empty, foreign-language or misleading titles; what a video shows and says is the only reliable evidence. Show recall per channel and the marginal recall of each.
- **"How would you let users jump to the right moment?"** → Segment-level vectors and transcript timestamps; return the best segment's start time. Index segments only for long videos to control index size.

- **"How do you handle breaking-news queries?"** → Classify the query as fresh, search a small real-time index of recent uploads in parallel, and add a freshness feature to the ranker; rely on title/ASR first because visual embedding may lag.

Toy — three evidence channels, fusion and moments (`code/video_search_toy.py`)

Each synthetic video has a hidden topic vector. We assume a CLIP-style encoder already maps text and frames into one space, so each channel gives a vector comparable to the query: a **title** vector (31% of videos have empty titles, 7% clickbait), **16 frame** vectors (each very noisy, always present) and a **transcript** vector (only 44% have speech). Queries are noisy views of a video's topic.

```
s_rrf = sum(1.0 / (K + 1 + ranks(s)) for s in (s_title, s_visual, s_trans))      # K = 60

def feats(i):   # learned fusion: channel scores gated by availability + availability flags
    has_t, has_s = (~empty).astype(float), speech.astype(float)
    return np.stack([s_title[i] * has_t, s_visual[i], np.where(speech, s_trans[i], 0) * has_s,
                     has_t, has_s], axis=1)
lr = LogisticRegression(max_iter=1000).fit(X_train, y_train)    # target vs hard + random negatives

seg = unit(frames.reshape(N, F // 2, 2, D).mean(axis=2))         # 8 segments of 2 frames
seg_scores = np.einsum("qd,nsd->qns", mq, seg).max(axis=2)       # best segment per video
```

Output:

```
5,000 videos; 31% empty titles, 7% clickbait titles, 44% with speech

main-topic queries, recall@10 on 500 test queries   (all | target title empty, n=132)
  title/tags only           0.672 | 0.000
  visual (mean of frames)   0.652 | 0.606
  transcript only           0.462 | 0.424
  average of raw scores     0.714 | 0.288
  reciprocal rank fusion    0.740 | 0.379
  learned fusion (LR)       0.896 | 0.886
  learned weights [title, visual, transcript, has_title, has_speech]: [ 9.7  8.  10.8 -5.7 -6. ]

'moment' queries (scene lasts 2 of 16 frames), recall@10
  mean-pool whole video       0.138
  max over 2-frame segments   0.978
  index size: 1 vector/video = 5,000; 8 segments/video = 40,000
```

What it shows:

- **Title-only search never finds a video with an empty title** (0.000 on that slice). The visual channel is present for every video, so it keeps working there (0.606).
- **Untrained fusion helps overall but hurts the empty-title slice**: averaging raw scores (0.288) and RRF (0.379) both let a missing title drag a video down, below what visual search alone achieves on that slice (0.606).
- **A learned fusion with availability flags** fixes it: 0.896 overall and 0.886 on empty-title videos. The negative weights on `has_title` and `has_speech` are the model's way of saying "an available channel raises the bar; a missing one is not evidence against the video."
- **Moments**: when the queried scene is 2 of 16 frames, mean-pooling dilutes it (0.138); taking the best of 8 short segments finds it (0.978) — at 8× the index size. The toy's scene frames are clean, so real gains are smaller, but the direction and the cost are the same.

Common mistakes

Weak	Strong
Indexes titles and descriptions only	Indexes what is said (ASR), shown (visual embeddings) and written on screen (OCR), each as a channel
Treats a missing channel as a low score	Availability features and a learned fusion; "not yet indexed" is different from "no match"
One vector per video for everything	Video-level vectors for topic queries, segment vectors (selectively) for moments
Optimises clicks	Satisfied watch after search; quick returns to results are strong negatives

Same design, other questions

- **Podcast or meeting-recording search** — the same design with the visual channel replaced by speaker identification; ASR quality is everything, timestamps become the product.
- **Stock-footage search** — no speech, weak titles, so the visual channel carries the design; buyers search for moments, so segment vectors are core.

Self-check

1. Why does video search need an indexing pipeline that product search mostly doesn't? → The content is in pictures and audio; ASR, OCR and visual encoders must turn it into searchable text and vectors at upload time.
2. What does reciprocal rank fusion add up, and why not raw scores? → 1/(k + rank) per channel; raw scores from different channels are on different scales.
3. Why can mean-pooling miss a query about a short scene? → The scene's frames are a small share of the average, so their signal is diluted.

TL;DR

Big picture	17.3's search funnel where documents are videos; the new work is indexing pictures and speech
Channels	Lexical (title, description, ASR transcript, OCR) + shared text–image embeddings (CLIP-style) at video and segment level
Fusion	RRF as a start; a learned ranker with availability features is what handles missing titles and speech
Objective	Satisfied watch after search, not clicks; position bias corrected as in 17.3
Cost	Indexing dominates (frames, ASR per upload); segment vectors multiply the index — use selectively
Freshness	Title in minutes, content in hours, a real-time index for fresh queries

---

Chapter comparison (17.1–17.4)

	Recommendations	Feed ranking	Search ranking	Text-to-video search
Task type	Retrieval + learning-to-rank	Multi-task ranking	Query understanding + learning-to-rank	Cross-modal retrieval + learning-to-rank
Candidate source	Two-tower ANN, co-occurrence, trending	Social graph fan-in, spillover, suggested	Inverted index (BM25) + embedding ANN	Inverted index over title/ASR/OCR + text–visual ANN (video and segment level)
Key features	Watch history, item embeddings, user×item similarity	Viewer–author closeness, post content, live engagement	Query–item match (BM25, embedding sim, attributes), item quality	Per-channel match scores, channel availability, satisfied-watch rate, freshness
Model	GBDT → deep multi-objective ranker	Shared-body multi-task net + value formula	L1 GBDT → L2 LambdaMART / deep; cross-encoder top-k	BM25 → + ASR → + CLIP-style encoders; RRF → learned fusion ranker
Offline metric	Recall@K, AUC	Per-head AUC, calibration, formula replay	NDCG@k, MRR	Recall@K per channel, NDCG@10, moment accuracy
Online metric	Watch time, return rate	Sessions/user or worthwhileness + hide/report guardrails	Session success, revenue per search, reformulation	Searches ending in a satisfied watch, quick returns
Hardest challenge	Cold start + position bias	Balancing many goals without rewarding bait	Biased clicks + revenue vs relevance	Indexing cost; missing or misleading text

The full thirteen-way table is at the end of Chapter 18.

Chapter TL;DR

- One template for every case: requirements → ML framing → data/labels → features → baseline → advanced → offline metrics → online metrics → serving → scale → monitoring → follow-ups.
- Retrieval-and-ranking problems share a **funnel**: cheap high-recall retrieval, expensive high-precision ranking, rules kept outside the model.
- **Recommendations**: multi-source retrieval judged by marginal recall, cold start, precompute-heavy serving.
- **Feeds**: calibrated per-event heads blended by a product-owned **value formula**, integrity layer after it, negative feedback in training.
- **Search**: query understanding, hybrid lexical + semantic retrieval, NDCG, and **position-biased clicks** corrected with an examination model or IPW.
- **Video search**: index what videos say (ASR) and show (CLIP-style frame vectors); a learned fusion that knows when a channel is missing; segment vectors for moments.
- In all four, clicks are a biased proxy: log positions and features, correct for bias, and let long-term online metrics with guardrails decide.

Chapter 18 — ML Case Studies II: Ads, Fraud, Moderation, PYMK, Music, Visual Search, Blurring, Events, Similar Listings

Chapter 17 worked through the four retrieval-and-ranking designs. This chapter adds nine more, each bringing one new pressure: money riding on every probability (ads), an opponent who adapts (fraud), platform-wide scale with harm in both directions (moderation), a two-sided graph problem (people you may know), two serving modes for one product (music), an index that changes every minute (visual search), a detector whose misses are privacy breaches (street-imagery blurring), a catalogue where every item is new and perishable (events), and similarity learned from browsing sessions (similar listings). Each uses the **eleven-part template** of 17.0 and ends with a runnable toy on synthetic data. Generic A/B-testing and monitoring mechanics are not repeated — see Chapter 16 (16.8 monitoring, 16.9 experiments).

What you'll learn

1. **Ad click prediction**: why the auction needs *calibrated* probabilities, down-sampling correction, Platt/isotonic calibration, delayed conversions, pacing (18.1).
2. **Fraud detection**: extreme imbalance, velocity and graph features, rules + model, two cost-based thresholds, censored labels (18.2).
3. **Harmful content** (including the platform-scale version): the risk cascade, the action ladder, prevalence of violating views, reviewer capacity, reach-aware re-scoring (18.3).
4. **People You May Know**: friends-of-friends retrieval, Adamic-Adar, the two-sided objective P(send) × P(accept), privacy that fails closed (18.4).
5. **Music discovery**: weekly batch playlists vs online radio, audio embeddings for new tracks, session context (18.5).
6. **Visual search**: a shared image-text space (CLIP-style), contrastive training, IVF/HNSW, index churn and encoder upgrades (18.6).
7. **Privacy blurring**: recall-first detection, object-level recall across views, box padding, audits instead of A/B tests, failing closed (18.7).
8. **Event recommendation**: an all-cold catalogue scored by content, place, time and social features; point-in-time features; a two-sided marketplace (18.8).
9. **Similar listings**: skip-gram listing embeddings from click sessions, booked-listing context, same-market negatives, cold-start averaging (18.9).
10. A comparison table across all thirteen case studies of Chapters 17–18.

Code is in `code/`; every output shown is real.

18.1 Case study: ad click prediction

> "Design the system that picks which ad to show and predicts whether it will be clicked."

1. Clarify requirements

Ranking again, plus auctions, budgets and billing. The key difference: **the model's output is part of a financial transaction**. In a simplified cost-per-click (CPC) setup ads are ranked by **expected value per impression = bid × pCTR** (**pCTR** = predicted click-through rate, CTR = clicks ÷ impressions). A wrong probability changes who wins and what the platform earns.

Question	Assumption used below
Traffic	100 K ad requests/s at peak
Latency	50–100 ms for the whole auction (eligibility → retrieval → scoring → auction)
Pricing	CPC and cost-per-conversion campaigns both exist
Auction	State one (e.g. quality-weighted second-price); real auctions add reserves
Freshness	Campaigns start, pause and change budgets all day

2. Frame it as an ML task

- **Input**: user and context (page, device, time), ad and creative, advertiser history.
- **Output**: **pCTR** = P(click | impression) and, for conversion campaigns, **pCVR** = P(conversion | click) — a conversion being the purchase or sign-up the advertiser actually wants.
- **Category**: binary classification whose output must be a **calibrated probability**, not just a good ordering; minimise **log loss** (cross-entropy), which rewards accurate probabilities.

Calibration — what this question is really about. A model is calibrated if, of many impressions given pCTR = 2%, about 2% are clicked. **AUC** only checks ordering, so a model can have excellent AUC while every probability is 3× too high — tolerable in a feed, not in an auction that multiplies the probability by money.

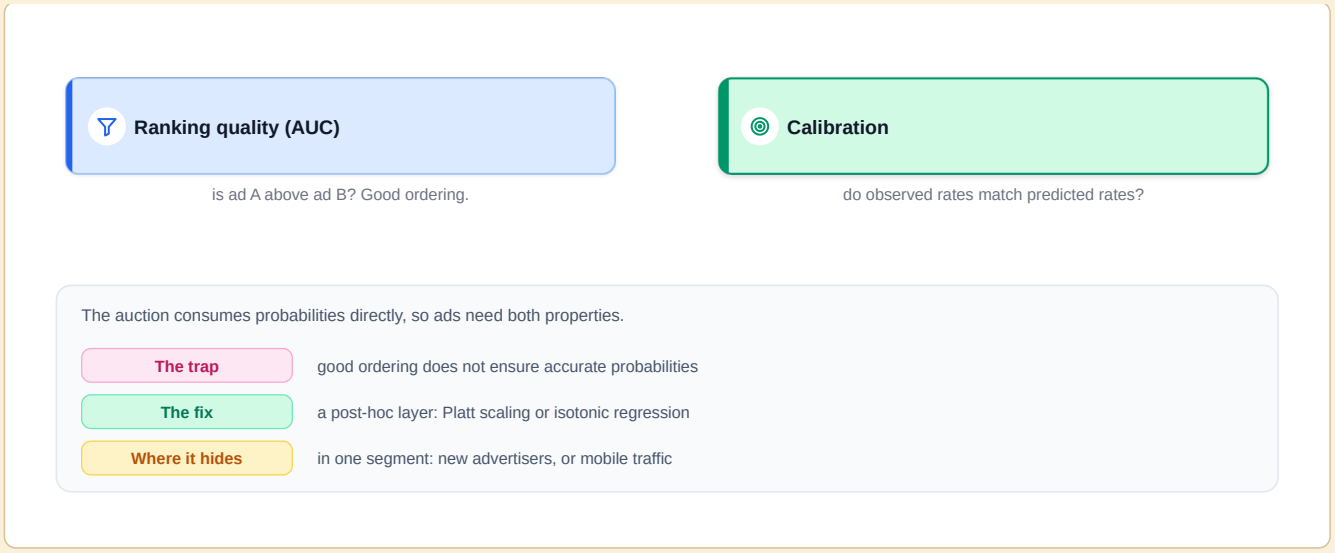


Figure — Good ranking and accurate probabilities are different properties (course diagram)

3. Data and labels

Two clocks:

- **Clicks** arrive within seconds, but a *non-click* is only final after an **observation window** — otherwise late clicks get recorded as negatives.
- **Conversions** arrive hours to days later, and some never do. A recent click with no conversion yet is **unresolved**, not negative; treating it as negative biases pCVR downward (**delayed feedback**).

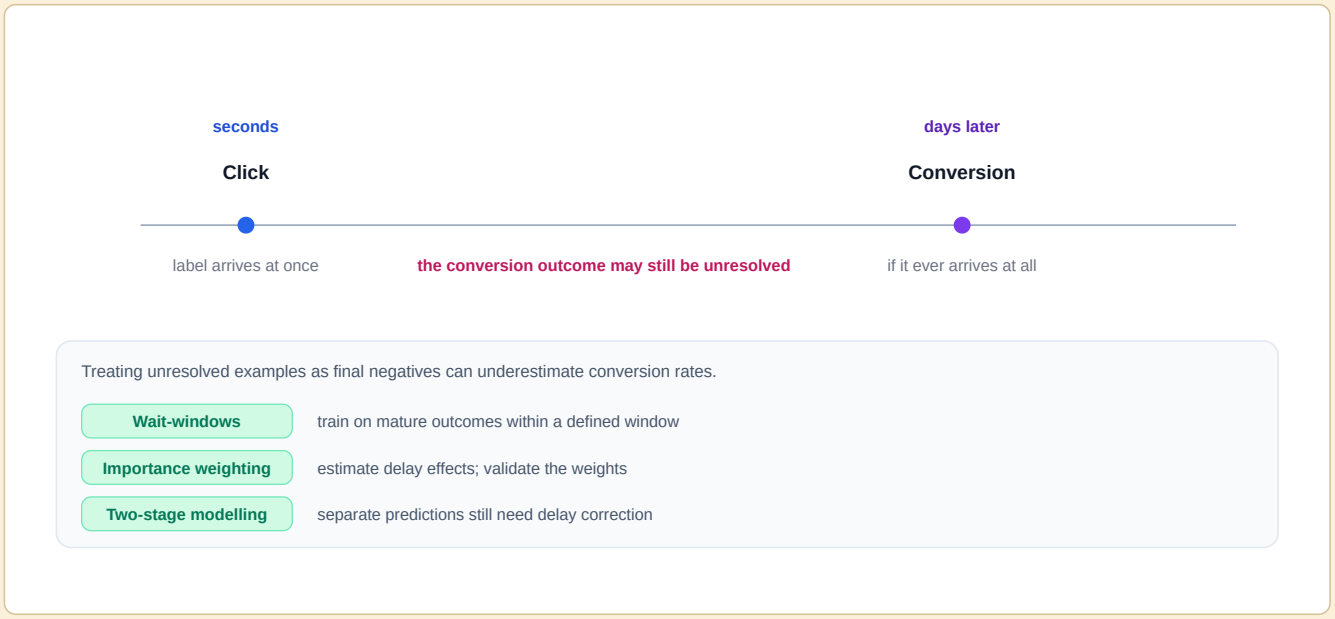


Figure — A click is instant; the conversion it may cause is not (course diagram)

Also: **eligibility data** (targeting, remaining budget, policy approval) for a hard pre-model filter, and **invalid-traffic** signals (bots, click farms) removed before billing *and* training — with later credits and label fixes for what's found late.

4. Features

Group	Examples
User / context	Interest segments, recent activity, device, OS, placement, page topic, time, country
Ad / creative	Format, text and image embeddings, landing-page category
Advertiser history	Historical CTR by placement/device, account age, policy record
Cross features	User segment × ad category, placement × format (hand-made for LR, learned by deep models)
Counters	Impressions/clicks per ad in the last hour/day (streaming)

Features are sparse (millions of IDs): **embedding tables** or **feature hashing** (map each ID into a fixed number of buckets), all cached for the 50–100 ms budget.

5. Model: baseline → advanced

- **Baseline**: logistic regression on hashed and crossed features — cheap, usually well calibrated under log loss, the historical workhorse (Google's search-ads system: online LR with FTRL-Proximal, McMahan et al., 2013).
- **Next**: boosted-tree outputs used as features for a logistic model (Facebook, He et al., 2014), then deep CTR models that learn feature crosses — factorisation machines (Rendle, 2010), Wide & Deep (Cheng et al., 2016, Google Play), DeepFM, DCN.
- **Multi-head**: shared body, pCTR and pCVR heads (17.2's pattern). Alibaba's ESMM (Ma et al., 2018) trains on *all* impressions by predicting pCTCVR = pCTR × pCVR, so the conversion head isn't learned only from the biased "clicked" subset.
- **Calibration layer** (part of the model, not an afterthought): map raw scores to probabilities on held-out data with **Platt scaling** (a one-feature logistic regression on the score) or **isotonic regression** (a non-decreasing step function). If you **down-sample negatives**, keeping a fraction w of them to save training cost, correct analytically: $q = p / (p + (1 - p)/w)$ (the formula in He et al., 2014).

6. Offline metrics

- **Log loss** and **normalised entropy** — log loss divided by the log loss of always predicting the average CTR (Facebook's metric; lower is better) — both sensitive to calibration.
- **AUC** for ordering.
- **Calibration**: predicted/observed ratio, reliability curves, and **expected calibration error (ECE)** — bucket predictions, then average $|\text{mean prediction} - \text{observed rate}|$ across buckets weighted by bucket size. Break it down **per segment** (advertiser size, slot, device, market): a model can be calibrated on average while over-predicting for new advertisers and under-predicting on mobile.

7. Online metrics and A/B testing

- **Platform**: revenue per thousand impressions (RPM), CTR. **Advertiser**: cost per click / conversion, return on ad spend — advertisers who lose money leave. **User guardrails**: ad load, ad hide/report rate, page latency.
- Ads-specific trap: **budget interference** — both arms spend the same campaign budgets, so one can starve the other; split budgets per arm (A/B basics: 16.9).

8. Serving architecture

1. **Eligibility filter first** (targeting, budget, policy) — serving outside targeting or budget breaks the advertiser contract, far worse than a poor match.
2. **Retrieval** narrows eligible ads by relevance (embedding or rule-based).
3. **Scoring**: calibrated pCTR / pCVR in one batched call.
4. **Auction**: rank by expected value (bid × pCTR, plus quality terms and reserves) and price by the chosen rule. In a one-slot **second-price** (Vickrey) auction the winner pays the second-highest bid, making truthful bidding optimal; the multi-slot **generalised second-price (GSP)** auction does *not* guarantee that (Edelman, Ostrovsky & Schwarz, 2007), and Google Ad Manager moved to a **first-price** auction in 2019.
5. **Pacing**: a control loop compares spend rate with the campaign's plan and throttles or boosts participation — otherwise budgets burn out by mid-morning.

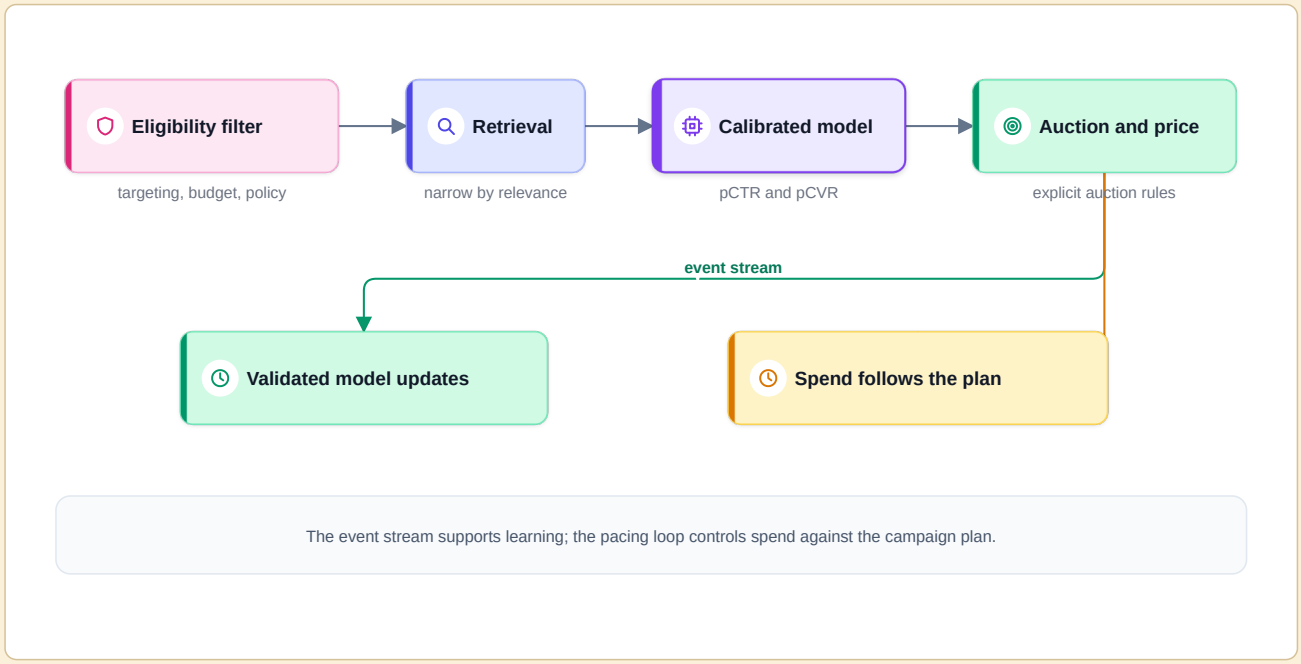
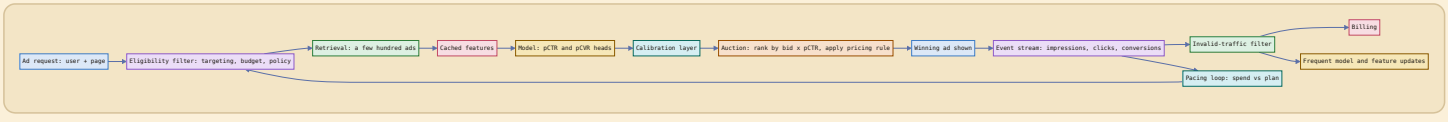


Figure — An example auction path with a 50–100 ms target (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Scores per second at peak	100 K requests × 300 candidates	30 M scores/s
Cost sensitivity	30 M/s × 86,400 s	~2.6 trillion scores/day at peak rate — a microsecond per score matters
Outage cost	revenue/day ÷ 86,400	An ads outage is priced in dollars per second

10. Monitoring and failure modes

- **Update loop**: among the fastest in ML — incremental updates every minutes to hours, each validated with a tested rollback.
- **Cold start is permanent**: start new campaigns from **priors** (guesses from advertiser, category, creative) plus a small, capped **exploration** budget.
- **Delayed conversions**: train on **mature** examples inside an attribution window, or model the delay (Chapelle, 2014).

Failure	Response
pCTR service degraded	Warm fallbacks: last-known-good model → segment-average CTR tables; eligibility and budget checks never skipped
Calibration drift in one segment	Per-segment alerts; recalibrate; investigate the feature pipeline
Invalid-traffic spike	Filter before billing and training; credits and label fixes for what's found later
Budget over-delivery	Pacing plus hard budget accounting

11. Interview follow-ups

- **"Second-price or first-price?"** → Describe the incentives: truthfulness holds only for single-slot second-price, not GSP; calibration matters in every variant.
- **"An advertiser says they're overcharged."** → Compare predicted vs realised CTR for their segment, then auction logs, invalid-click records and billing — recalibration fixes predictions, billing errors need a separate correction.
- **"Conversions are scarce — how does pCVR learn?"** → Factor into click × conversion-given-click (ESMM-style) so each head uses the densest labels; correct for delay; priors for thin advertisers.

Toy — calibration, down-sampling and the auction (`code/ads_calibration_toy.py`)

Synthetic impressions, ~4% CTR; a GBDT trained on all clicks but only 10% of non-clicks. We correct the probabilities, check calibration per segment, and run a two-advertiser auction.

```
keep = (ytr == 1) | (rng.random(len(ytr)) < w)          # all clicks, 10% of non-clicks
model = HistGradientBoostingClassifier(max_iter=200, random_state=0).fit(Xtr[keep], ytr[keep])
raw = model.predict_proba(Xte)[:, 1]
corrected = raw / (raw + (1 - raw) / w)                  # analytic down-sampling correction
platt = LogisticRegression().fit(np.log(raw_cal / (1 - raw_cal)).reshape(-1, 1), ycal)   # Platt on logit
iso = IsotonicRegression(out_of_bounds="clip").fit(raw_cal, ycal)                       # isotonic
```

Output:

```
true CTR in test traffic: 0.0397

pCTR source               AUC  mean pCTR  log loss     ECE
raw (down-sampled)      0.788     0.2368    0.3217  0.1971
analytic correction     0.788     0.0404    0.1438  0.0016
Platt scaling           0.788     0.0402    0.1438  0.0017
isotonic                0.787     0.0403    0.1444  0.0020

segment check (skewed model)   overall pred/actual = 0.0403/0.0397
  mobile   predicted 0.0601  observed 0.0460  ratio 1.31
  desktop  predicted 0.0203  observed 0.0334  ratio 0.61
auction, calibrated pCTR      : A wins 49.8% of mobile / 48.9% of desktop, expected value $1,064
auction, A inflated on mobile : A wins 75.9% of mobile / 48.9% of desktop, expected value $1,040
```

What it shows:

- The raw model has the **same AUC** (0.788) as the corrected ones yet predicts a 23.7% CTR for 4% traffic. AUC can't see this; log loss and ECE can.
- The analytic formula and both calibrators restore the ≈4% mean and cut ECE about 100×. A model can match the overall rate (0.0403 vs 0.0397) while being 31% high on mobile and 39% low on desktop — only **per-segment** checks reveal it.
- Inflating one advertiser's mobile pCTR lifts its mobile win share from 50% to 76% and lowers expected value ($1,040 vs $1,064): impressions go to the ad less likely to be clicked.

Common mistakes

Weak	Strong
Optimises AUC and stops	Calibration layer plus per-segment calibration monitoring
Starts with relevance retrieval	Eligibility (targeting, budget, policy) is the first, hard filter
Diagram ends at the winning ad	Delayed conversions, invalid-traffic filtering, pacing, warm fallbacks

Same design, other questions

- **Sponsored search ads** — an ad must clear a query-relevance bar before bid × pCTR applies (17.3's retrieval). **Native feed ads** — a quality floor no bid can buy past, and an ad-load cap on slots shared with organic posts.

Self-check

1. Why can't an ads model be judged on AUC alone? → The auction multiplies the probability by a bid; AUC ignores whether probabilities are accurate.
2. You kept 5% of negatives and the model outputs p = 0.5. Corrected probability? → q = 0.5 / (0.5 + 0.5/0.05) = 0.5 / 10.5 ≈ 0.048.
3. Platt vs isotonic? → Platt fits a smooth sigmoid (2 parameters, little data needed); isotonic fits any monotone step function (more flexible, needs more data, can overfit).

TL;DR

Core idea	Probabilities are priced: rank by bid × **calibrated** pCTR; check calibration per segment
Path	Eligibility → retrieval → pCTR/pCVR heads → calibration → auction → pacing
Labels	Delayed conversions via windows or delay models; invalid traffic stripped before billing and training
Cold start	Permanent: priors + capped exploration

18.2 Case study: payment fraud detection

> "Design fraud detection for a payments platform."

1. Clarify requirements

No funnel: no catalog to narrow, just a stream of transactions, each needing a decision while the payment waits. The new ingredient is an **adversary** — fraudsters change behaviour to get past whatever you deploy, so some patterns die within days.

Question	Assumption used below
Volume	10 K transactions/s at peak
Fraud rate	Below 1% (the toy uses ~0.2%)
Latency	~50 ms inside the payment authorisation call, ~10 ms of it for the model
Actions	Approve, step-up authentication (3-D Secure, one-time passcode), review, block
Costs	Missed fraud → loss + **chargeback** fees (a disputed payment reversed to the cardholder, at the merchant's cost); false decline → lost sale and trust; review → analyst time

Class imbalance: at 0.2% fraud, "approve everything" is 99.8% accurate and catches nothing — accuracy is off the table from minute one.

2. Frame it as an ML task

- **Input**: transaction (amount, merchant, channel), card/account, device, recent activity, graph links.
- **Output**: a risk score ≈ P(fraud). **Category**: binary classification on heavily imbalanced data.
- **Decision** — *separate from the score*: an **action policy** maps score (plus amount, review capacity, mandatory rules) to **approve** (below the low threshold — the vast majority), **extra checks** between thresholds (step-up, or human review via a pending/delayed-capture state, since humans can't answer in 50 ms), or **decline** (above the high threshold).

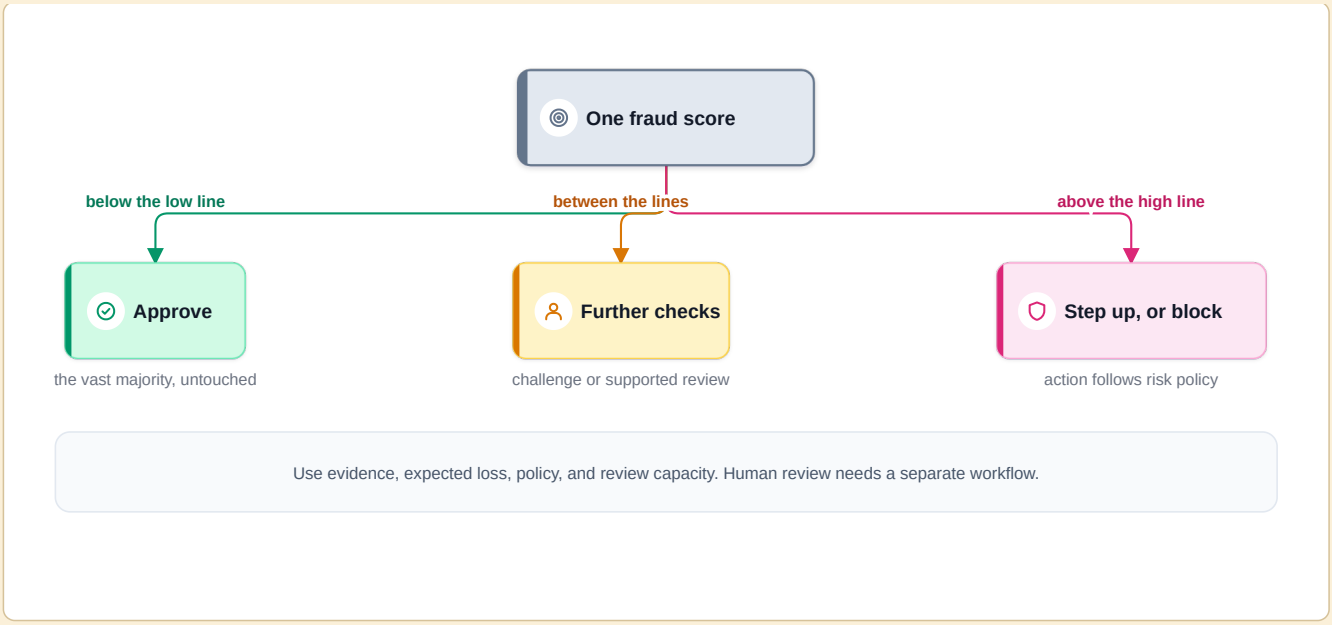


Figure — The decision is three-way, set by two thresholds (course diagram)

Thresholds trade **precision** (share of flagged that is fraud) against **recall** (share of fraud caught); set them from **expected cost**, review capacity and policy.

3. Data and labels

Three label problems — name all three:

1. **Delayed labels.** Chargebacks and fraud reports arrive weeks later (assume 2–8 weeks); not every chargeback is fraud, not every fraud is reported. Fresh recall *doesn't exist yet*, so early warning comes from score distributions and rule-hit rates.
2. **Censored labels.** A blocked payment never produces an outcome, so the data has holes exactly where the model sees most risk. Partial remedies: analyst investigations, verification results, appeals — record each label's source. **Letting likely fraud through to learn its outcome is off the table** (any deliberate test needs risk-owner sign-off and a loss cap).
3. **Imbalance.** Class weights or down-sampling in training (then recalibrate — 18.1); report **PR-AUC**, precision/recall at the operating thresholds, or best, **expected dollar loss**.

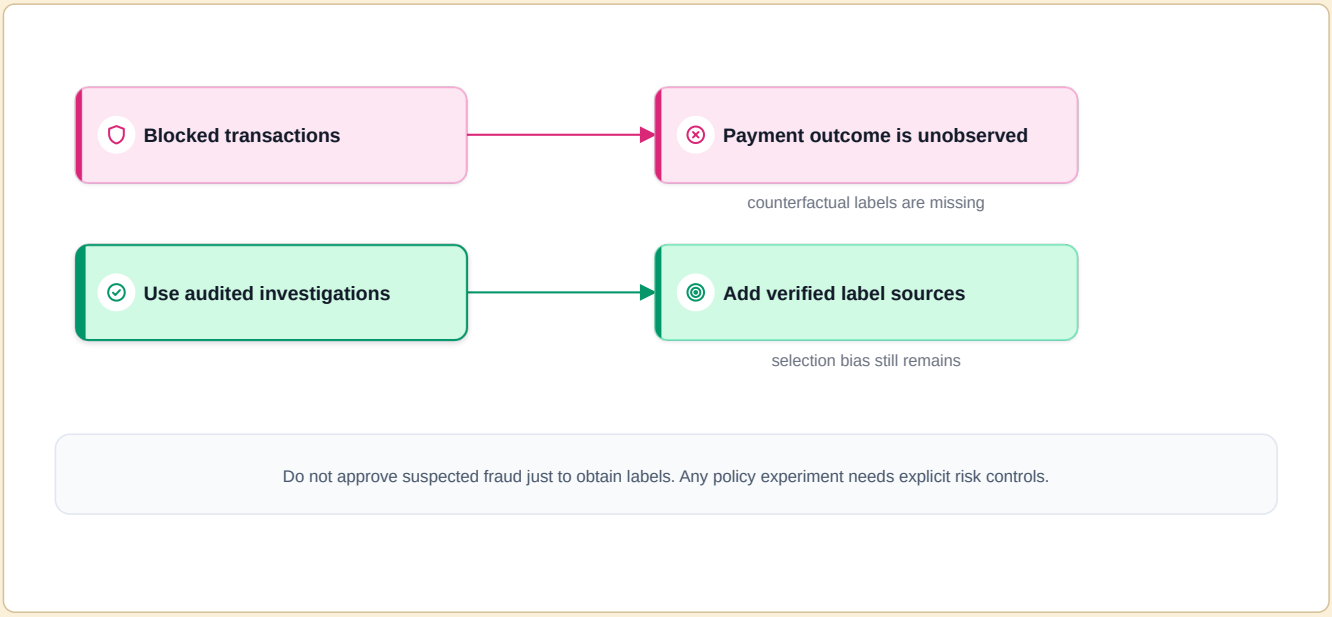


Figure — Blocked payments have missing outcomes; audited evidence helps but doesn't remove all bias (course diagram)

4. Features

Dominated by **velocity** (how much activity, how fast) and **graph** signals:

Group	Examples	Freshness
Velocity	Transactions on this card in the last 10 min / 1 h; distinct merchants today; amount vs this card's usual	Seconds (streaming)
"Impossible travel"	Distance between consecutive in-person transactions ÷ time between them	Request time
Device / session	New device? Device seen with other cards? Emulator, proxy, typing cadence	Request time
Graph	Device linked to a confirmed-stolen card; many accounts sharing one address or IP	Minutes
Merchant / account	Merchant fraud rate (from *past* windows only — leakage risk), account age	Hourly–daily

Fraud rings show up as dense clusters in the card–device–address graph before any single transaction looks odd. Shared devices also occur innocently (families, offices), so a graph link is evidence, not proof.

5. Model: baseline → advanced

Rules and model, together on purpose. A **rules engine** (explicit, human-written conditions) gives three things a learned score can't: **hard constraints** (sanctions checks, verified blocklists aren't probabilities), **speed** against a new attack (a rule ships in minutes; a model has no training data for a scheme that appeared today), and **explainability** for regulators and dispute teams. The **model** catches combinations of weak signals no short rule list captures.

- **Baseline**: rules + logistic regression on velocity features.
- **Standard**: **GBDT** on tabular features (the common choice for card fraud) plus graph features or a graph neural network for rings; optionally an anomaly score (e.g. isolation forest) for never-seen patterns.
- **Rule hygiene**: each rule gets an owner, a precision/recall dashboard and an **expiry date**, or stale rules pile up (**rule sprawl**).

Real-world: Stripe Radar combines ML risk scores with merchant-written rules; in Europe, PSD2's strong customer authentication makes 3-D Secure step-up a standard middle action.

6. Offline metrics

- **PR-AUC** (area under the precision–recall curve) — meaningful under heavy imbalance, where ROC-AUC can look excellent while the flagged queue is mostly false alarms.
- **Recall at a fixed false-positive rate**, or **precision at a fixed review capacity**.
- **Expected cost** of the whole policy (missed fraud + false declines + review), on a **time-based** split with **mature** labels only — a **backtest** on past windows whose outcomes are known.

7. Online metrics and A/B testing

- **Business**: fraud loss in **basis points** of volume (1 bp = 0.01%), chargeback rate, false-decline rate, review-queue size and SLA (promised turnaround).
- You can't A/B a fraud model like a feed ranker (letting fraud through on purpose is unacceptable). Rollout: backtest → **shadow scoring** (score live traffic silently; compare distributions and reviewer agreement) → **canary** under approved risk limits → wait for mature labels → full rollout (16.6).

8. Serving architecture

~50 ms end to end, set by the payment network: **rules** → **feature assembly** (online store + streaming velocity) → **model** (~10 ms) → **policy**. Labels return as three streams at different speeds: analyst findings, verification results, late chargebacks.

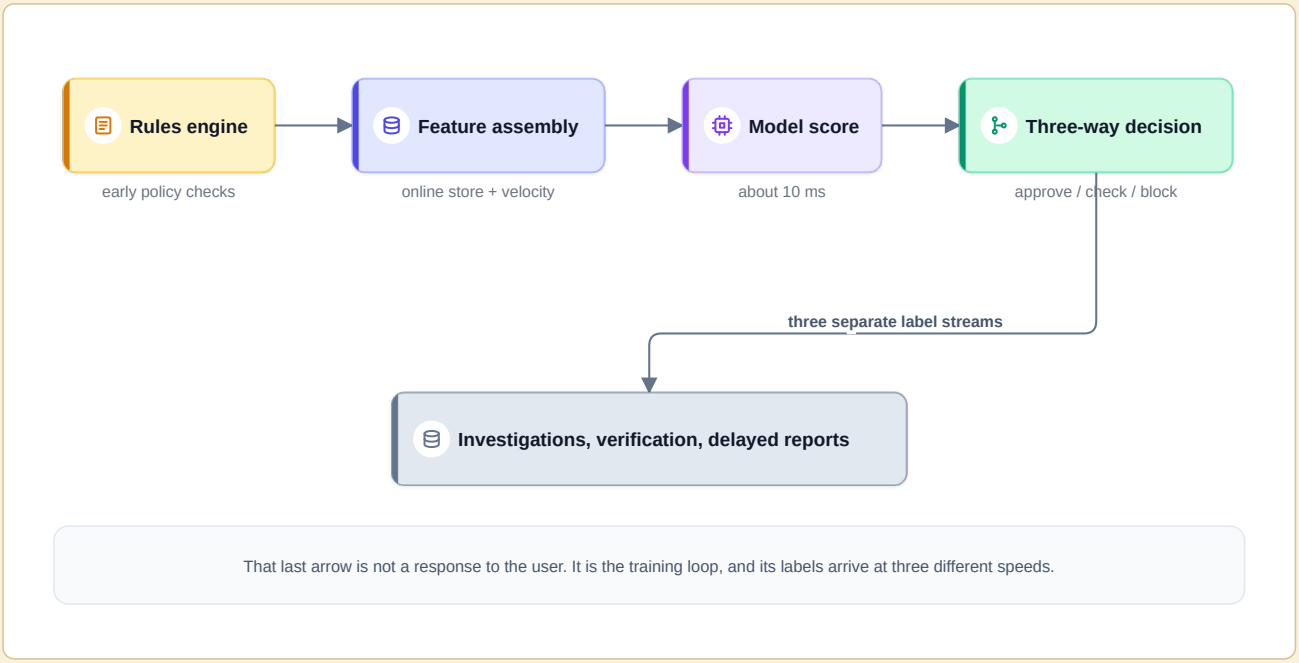
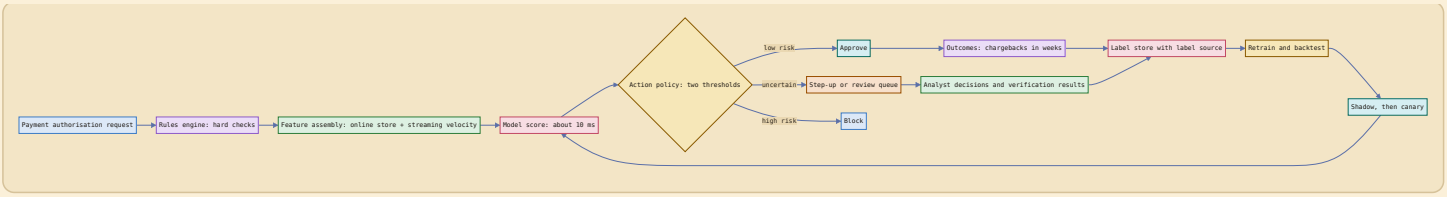


Figure — An example 50 ms automated decision path, with labels arriving separately (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Fraud events	10 K/s × 0.2%	~20 fraudulent attempts/s at peak
Review load	0.5% in the middle band: 10 K × 0.005 × 3,600	180 K reviews per peak hour — impossible for humans, so step-up handles most of the band and review is kept for high-value cases
Velocity state	200 M active cards × 4 counters × 8 bytes	≈ 6.4 GB in memory (plus keys, replicas), updated per transaction

10. Monitoring and failure modes

- **Early warning** while labels are weeks late: score distributions, rule-hit rates, approval rate — per merchant, country, channel (drift basics: 16.8).
- **Adversarial drift**: watch for bursts of small "test" transactions on new cards. A **sudden drop** in the flagged rate is a possible attack or a broken feature, not good news.
- **Timeout policy**: neither global **fail open** (approve if scoring fails) nor global **fail closed** (decline) — decide in advance by risk: small amounts from long-standing accounts pass, large or risky ones are challenged or held.

Failure	First check
False-decline rate doubles overnight	Data feeds before the model: a velocity counter that stopped updating moves every score. One segment spiking suggests a rule or feature; everything spiking suggests upstream
Fraud volume ×10, analysts ×1	Narrow the review band to high-value cases, lean on step-up; never weaken mandatory controls

11. Interview follow-ups

- **"Once the model is strong, can the rules go?"** → No: speed during an attack, hard constraints, readable logic. Manage rules with metrics and expiry dates instead.
- **"How do you catch a fraud ring?"** → Graph features, community detection, and tools to expand from one confirmed case to its neighbours — checking for innocent shared infrastructure.

Toy — velocity features and two thresholds (`code/fraud_thresholds_toy.py`)

One synthetic week of card transactions (~0.23% fraud); stolen cards are used in quick bursts. Train on days 1–5, test on days 6–7 (time split), then pick approve/review/block thresholds by expected dollar cost under a review-capacity limit.

```
def velocity(card, t, window=10.0):
    """# of earlier transactions on the same card within `window` minutes (a streaming aggregate)."""
    # ... keep, per card, the timestamps still inside the window

def cost(lo, hi):
    block, review = p >= hi, (p >= lo) & (p < hi)
    missed = (yt == 1) & ~block & ~review                       # approved fraud: lose the amount
    c = amt[missed].sum() + COST_FALSE_DECLINE * ((yt == 0) & block).sum() + COST_REVIEW * review.sum()
    c += amt[(yt == 1) & review].sum() * 0.1                    # reviewers miss ~10% of what they see
```

Output:

```
360,769 transactions, fraud rate 0.230%
'approve everything' accuracy = 99.770%  (catches 0 fraud)

amount+foreign+hour    ROC-AUC 0.838   PR-AUC 0.054
+ velocity (10 min)    ROC-AUC 0.849   PR-AUC 0.408

no model (approve all): cost $33,052
single threshold 0.5   lo=0.50 hi=0.50: cost $17,258  blocked 163  reviewed 0  fraud caught 50%
best two thresholds    lo=0.02 hi=0.70: cost $15,554  blocked 158  reviewed 293  fraud caught 57%
```

What it shows:

- **Accuracy is useless**: 99.77% for a policy that catches nothing.
- **ROC-AUC barely moves** (0.838 → 0.849) with velocity, but **PR-AUC jumps ~7.6×** (0.054 → 0.408) — ROC-AUC hides false alarms under imbalance.
- A **review band** (within 300-review capacity) catches more fraud at lower cost than one threshold. (In practice pick thresholds on a separate validation window.)

Common mistakes

Weak	Strong
Binary classifier, one threshold	Three-way decision, two cost-based thresholds, step-up in the middle
"Retire the rules once the model is good"	Rules for speed, constraints, explainability; model for complex patterns
Quotes accuracy at a 0.2% base rate	PR-AUC, precision/recall at operating points, expected dollar loss

Same design, other questions

- **Account takeover at login** — the unit is a session; device fingerprint, location velocity and typing rhythm dominate; step-up is the main action. **Promo/referral abuse** — audit labels in days, so a faster loop; graph features dominate because abuse comes from rings.

Self-check

1. Why is ROC-AUC misleading here? → With 500 legitimate per fraud, a small false-positive *rate* is still a large *number* of false alarms; PR-AUC shows it.
2. What's a censored label? → An outcome never observed because your own action (blocking) prevented it.
3. A sudden drop in flagged transactions — good news? → Treat it first as possible evasion or a broken feature.

TL;DR

New ingredient	An adaptive adversary: patterns expire, probing is constant
Decision	Approve / step-up or review / block via two cost-derived thresholds
Hybrid	Rules (speed, constraints, explainability) + GBDT/graph model
Labels	Weeks late, censored by blocks, extremely imbalanced; PR-AUC and dollar loss

---

18.3 Case study: harmful content detection (and running it at platform scale)

> "Design a system to detect harmful content — hate speech, harassment, spam, graphic violence, misinformation." Scale-up version: trust and safety for a short-video app receiving ~200 M new items daily (clips, sound, captions, comments), with a rulebook from CSAM (zero tolerance, legally mandated handling) to grey-zone health misinformation.

1. Clarify requirements

Fraud's adaptive adversary and unequal error costs, plus three things:

1. **Multimodal input** — a harmless photo plus a hateful caption is harmful though neither half is: several models (per modality or harm) plus a policy layer combining their scores.
2. **Costs both ways, not only money.** A **false negative** (missed violation) exposes people to harm; a **false positive** (wrongful removal) silences a legitimate user. Thresholds become an explicit policy conversation.
3. **Huge scale, low base rates** that differ by harm type and surface.

Question	Assumption used below
Volume	200 M uploads/day ≈ 2,300/s average (tens of thousands/s at peak across all item types)
Violation rate	Below 1% for any given category
Latency	Livestream near-real-time (score rolling few-second windows); uploads a second or two; back-catalogue re-scans offline
Scope	Public posts; ads and private messages have their own pipelines and privacy rules
Jurisdiction	One policy engine keyed by **harm category and country**, since legal lines differ

The highest-harm category first. CSAM (child sexual abuse material) needs a dedicated, tightly governed process: hash matching against approved databases of known material (Microsoft's **PhotoDNA**; Meta open-sourced **PDQ** and **TMK+PDQF** in 2019), restricted-access review, evidence preservation, and legally required reporting. In the US, for example, providers must report apparent CSAM to NCMEC's CyberTipline once they know of it (18 U.S.C. § 2258A); that statute does not itself require proactive scanning, and duties differ by country — name who owns policy and escalation. Hashes find *known* material only.

2. Frame it as an ML task

- **Input**: the item (frames, audio, text), its context (account, where posted, replies) and its distribution (views, share velocity).
- **Output**: per-category risk scores P(violates policy c). **Category**: multi-label classification run as a **cascade**, then a policy decision — an **action ladder**, not remove/allow (section 8).

The metric that matters: prevalence of violating views = (views landing on violating content) ÷ (all views), estimated by human-labelling a random sample of *views*, so each miss counts by its reach. Report it **per category, with confidence intervals** (Meta's Community Standards Enforcement Report gives violating views per 10,000). **Opposite-direction guardrails**: **false-removal rate** (audits) and **appeal-overturn rate**; plus **time to action** and **views before action**.

3. Data and labels

1. **Rare, expensive labels.** Reviewer time is the bottleneck. **Active learning** sends the most useful items (uncertain, diverse, high reach) to reviewers; a *separate* random sample measures prevalence, since active-learning samples are biased by design.
2. **Three label feeds:** reviewer decisions (best, scarcest); **appeals** (false-positive evidence, but self-selected — add random audits); **user reports** (huge, noisy, manipulable — weight by **reporter reputation**, the historical precision of that reporter's reports; evidence, never verdicts).
3. **Adversarial evasion** — drift driven by an opponent: **leetspeak** ("fr33"), **homoglyphs** (Cyrillic "a" for Latin "a"), **text inside images** (needs **OCR**, optical character recognition), **splitting** the attack into the comment under an innocent post, **re-encoding / cropping / mirroring** to break hashes, and **coded language** that rotates faster than training cycles.
4. **Consistency and context.** Guidelines plus **inter-rater agreement** checks (how often independent reviewers agree); the same image can be news or gore.

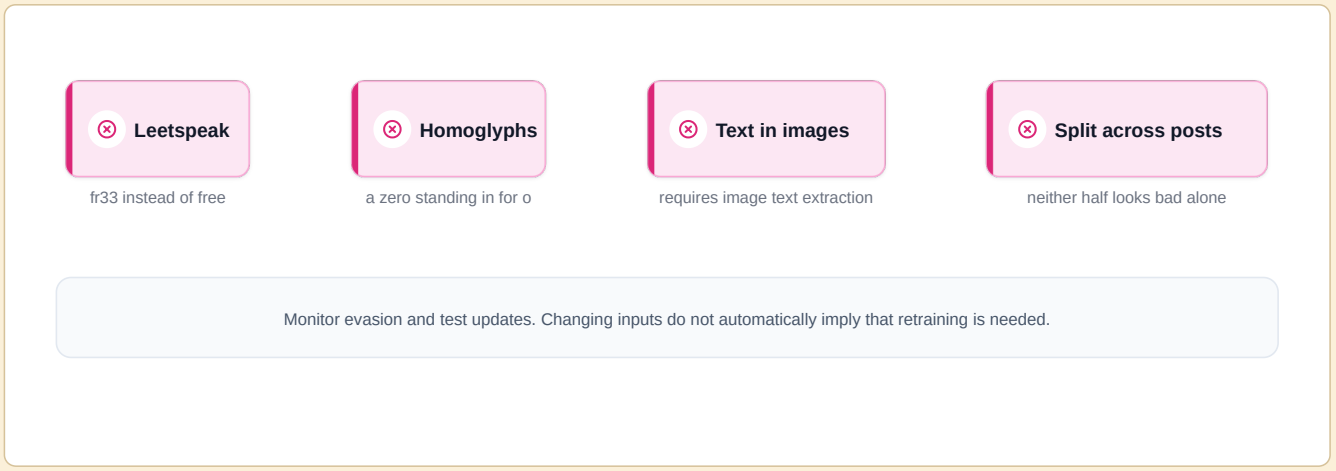


Figure — Four forms of adversarial evasion (course diagram)

4. Features

Group	Examples
Content	Text embeddings (after normalising leetspeak/homoglyphs), image and frame embeddings, OCR text, **ASR** (speech-to-text) transcript, audio embeddings
Hashes	Exact hash, perceptual hash of keyframes, audio fingerprint
Account / context	Account age, prior strikes, posting rate; surface (comment, post, live), the thread it replies to
Distribution	Views, **share velocity** (how fast it's spreading), audience
Reports	Count weighted by reporter reputation, report-graph clustering

5. Model: baseline → advanced (one model per stage)

1. **Hashes and high-confidence rules** — **exact hashes** catch identical files; **perceptual hashes** tolerate some edits (test robustness to crops and mirroring; audio fingerprints survive a mirrored video). Hash-sharing programmes exist for some categories (e.g. GIFCT's database for terrorist content).
2. **Fast classifiers on everything** — small per-modality models (a distilled text transformer, a small image CNN) whose only job is triage.
3. **Heavy multimodal models on the flagged slice** — large image-text models and LLM-based classifiers for nuanced text and memes.
4. **Human review** — borderline and appealed cases; also the label source.

Baseline for a new category: rules + hashes + report routing; then a fast classifier as labels accumulate; heavy models last. **Upgrade** by distilling the heavy model into the fast one.

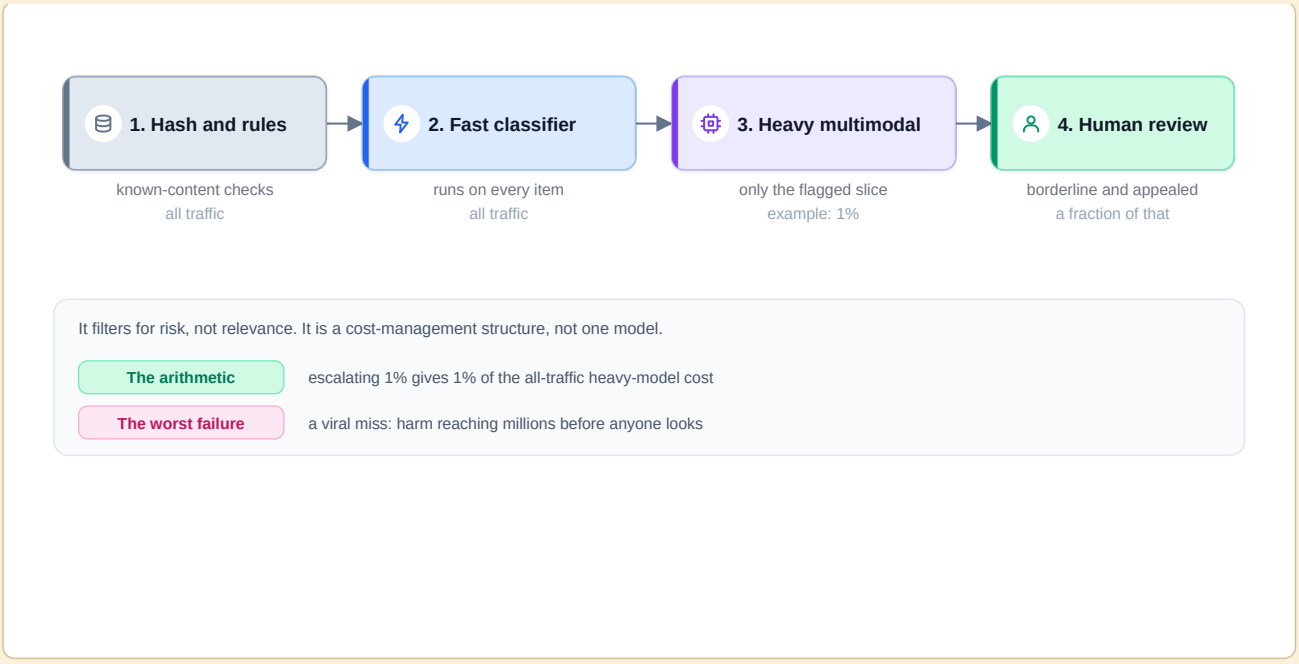


Figure — The risk funnel spends compute in proportion to risk (course diagram)

6. Offline metrics

- Per-category **precision and recall at the operating thresholds** on a **fully reviewed** evaluation set, sliced by language, region and content type.
- **Recall of the whole cascade**, not of each stage: an item wrongly passed by stage 2 never reaches stage 3.
- **Hash recall under mutation**: a test suite of known items transformed by current evasion tricks.

7. Online metrics and A/B testing

Metrics as in section 2, plus creator churn and queue age. You can't ethically randomise *leaving harm up*, so rollouts go backtest → shadow scoring → **independent adjudication** of disagreements (agreement with the old system isn't proof) → canary → sampled prevalence.

8. Serving architecture

The action ladder — policy turned into callable actions, per category and region:

Rung	When	Notes
Auto-remove	Confidently violating	Tuned so false removals stay rare; user notified, can appeal
Human review	Uncertain cases, appeals	Prioritised queue
Reduce / age-gate / warn	Borderline or allowed-but-restricted	Stays up but isn't amplified (the feed's integrity layer, 17.2), hidden from minors, or behind a warning
Auto-allow	Clearly fine — the vast majority	

Thresholds are owned by Trust & Safety and Legal (not the model team), set from measured error rates, severity, reviewer capacity and law.

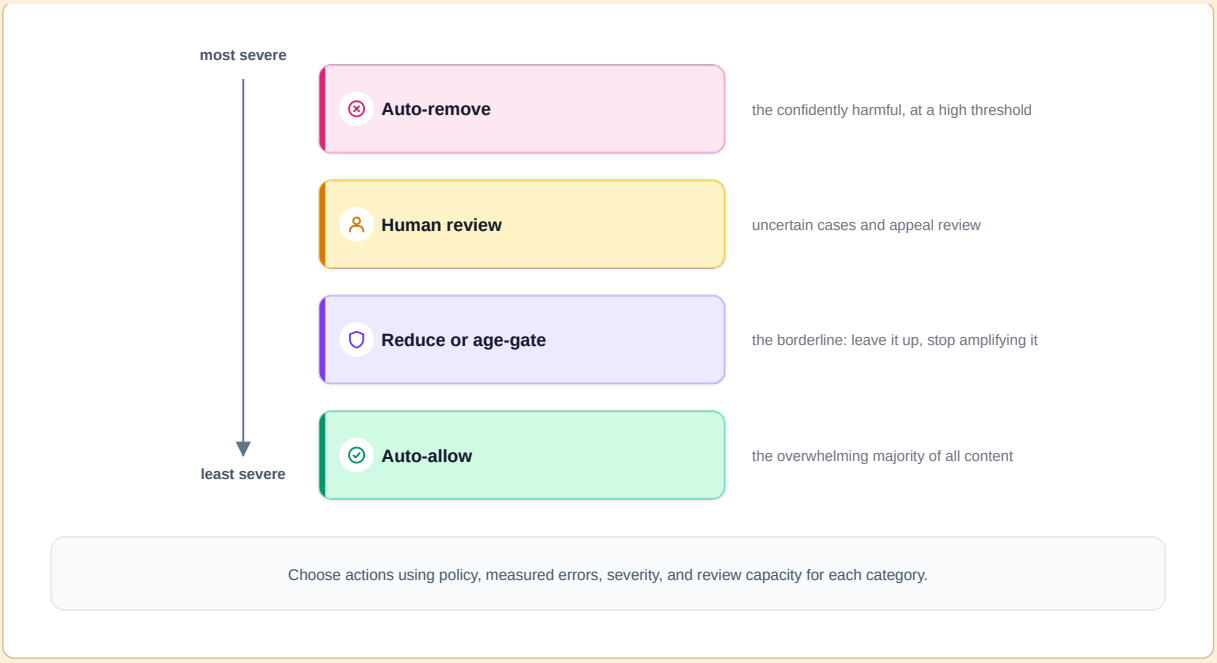


Figure — The action ladder, not a binary (course diagram)

Review-queue priority = f(severity, expected reach, uncertainty, deadline), with capacity reserved for random audits and appeals.

Reach-aware re-scoring — against the **viral miss** (harm reaching millions before anyone looks): each view tier crossed (10 K, 100 K, 1 M) triggers a re-check by a stronger model. Affordable because few items climb the tiers.

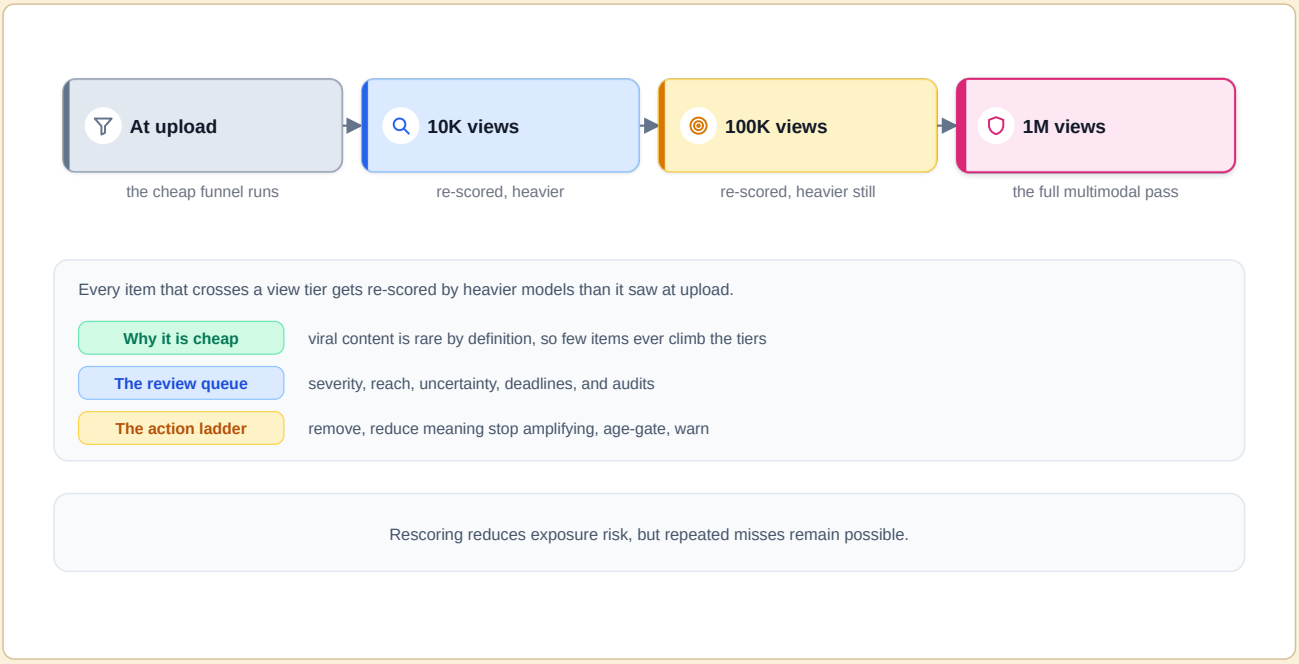
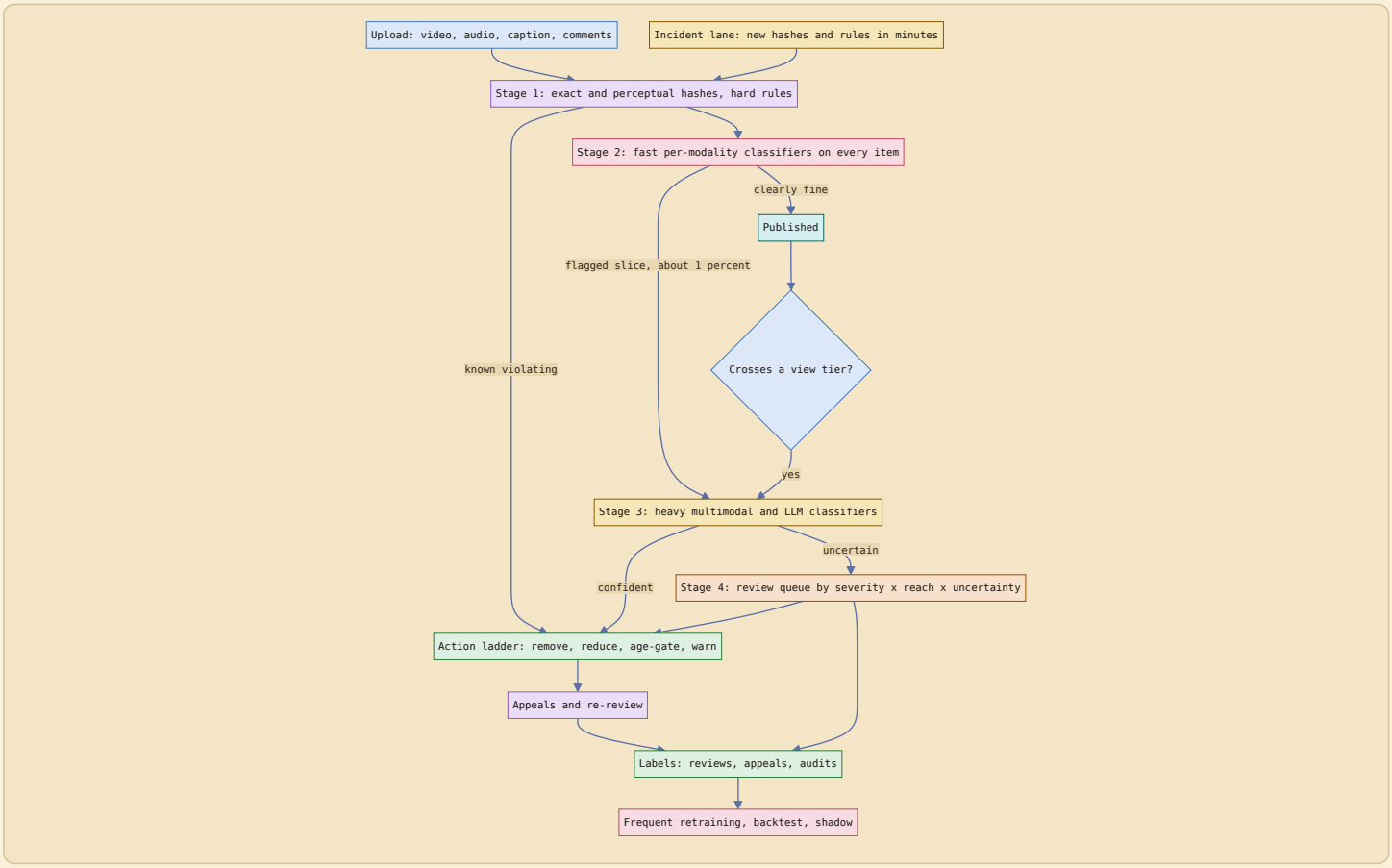


Figure — Reach-aware re-scoring: scrutiny grows with the audience (course diagram)

Incident lane: in a crisis (e.g. a live-streamed attack being re-uploaded) new fingerprints and rules go live in minutes — no retrain is that fast.



9. Scale numbers

Quantity	Arithmetic	Result
Fast stage	200 M/day ÷ 86,400	≈ 2,300 items/s average → small models, strict budgets
Heavy stage at 1% flag rate	200 M × 0.01	2 M/day ≈ 23/s average (then add media length, decoding, peaks, replicas)
Cost of the dial	+0.1 percentage point flag rate	+200,000 heavy inferences/day
Human review at 0.1%	200 M × 0.001	200,000 decisions/day
Reviewers	200,000 ÷ ~300 decisions per reviewer-day	≈ 670 reviewer-days per day → with shifts and leave, many hundreds of staff

The flag rate is the GPU bill; the review threshold is a **headcount** setting.

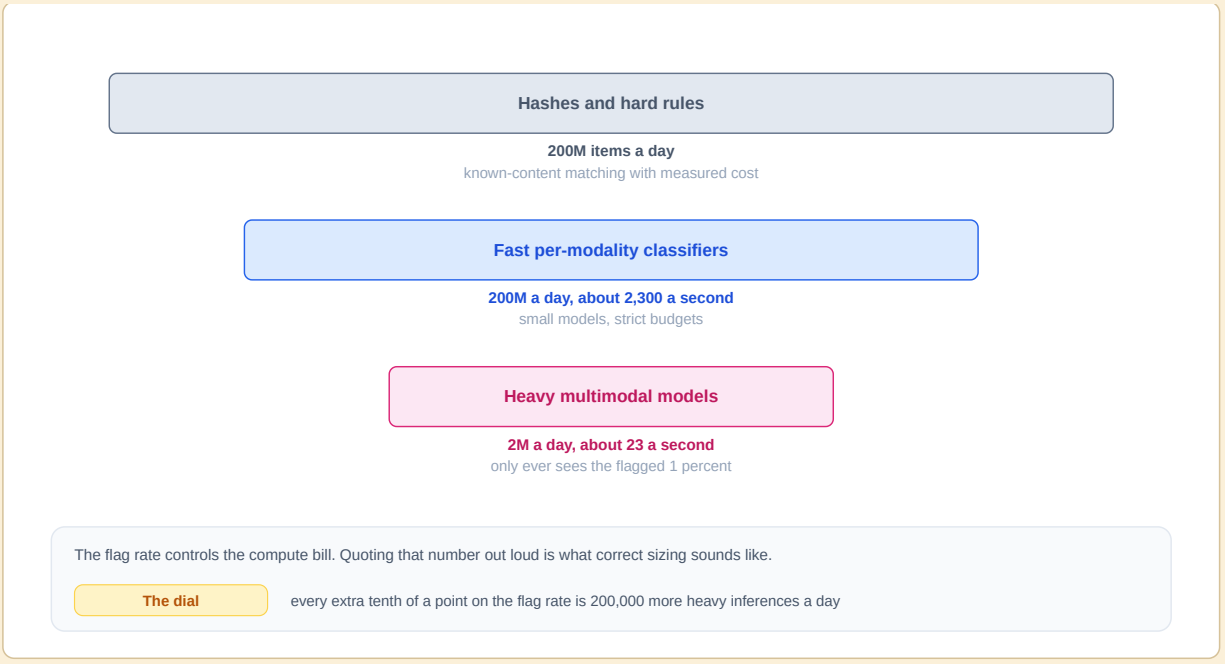


Figure — The risk funnel, with the volume said out loud at every stage (course diagram)

10. Monitoring and failure modes

- A sudden **drop** in flag rate means possible evasion, not improvement.
- **Policy changes trigger a back-catalogue re-scan** — yesterday's allowed content can violate today's rule.
- **Outage policy per category**: never treat unscored content as low-risk; high-risk surfaces hold (fail closed), low-risk ones may publish (fail open).
- **Moderator well-being is designed capacity**: exposure limits per shift, rotation off graphic queues, blurred-first media, attrition in the headcount plan.

Failure	Loud or quiet?	Defence
Viral miss	Loud	Re-check at view tiers; for the worst harms, escalate generously (accept extra false alarms)
Over-removal	Quiet (overturns, creator churn)	Audited false-removal and overturn rates beside prevalence
Evasion wave	Quiet at first	Flag-rate drop alerts, near-duplicate clustering, fast rules

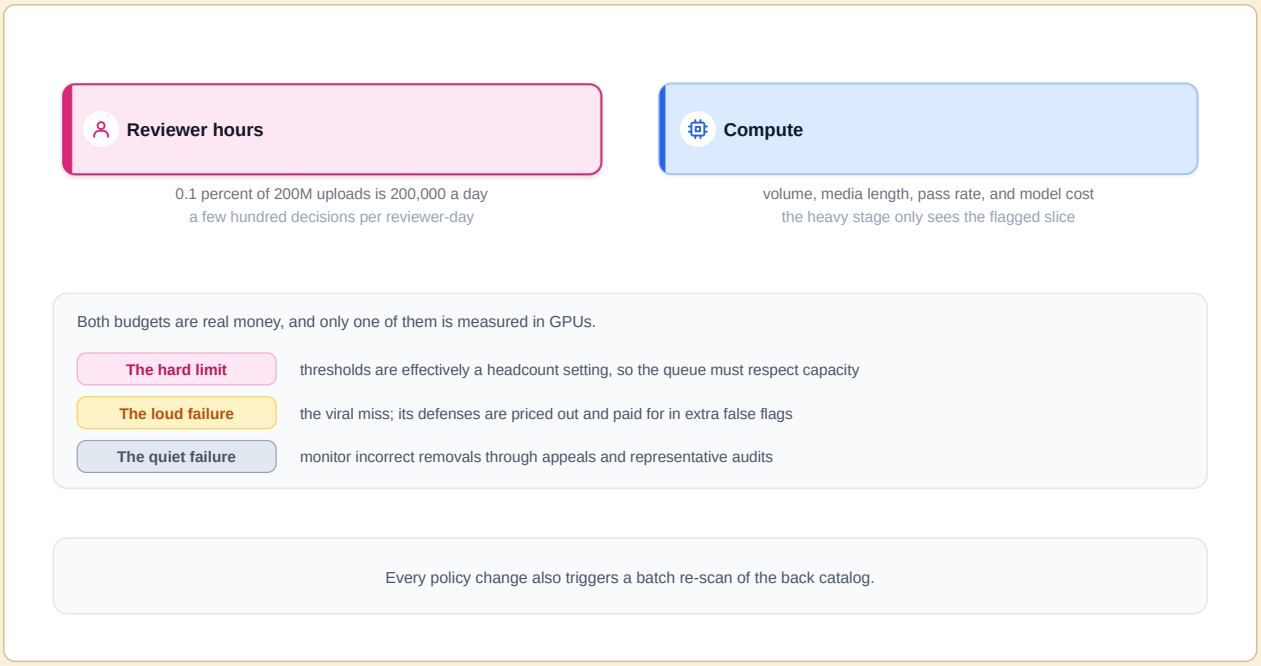


Figure — Two budgets, and only one of them is measured in GPUs (course diagram)

11. Interview follow-ups

- **"Something harmful is spreading fast right now."** → Share velocity is an input: fast spreaders jump to deeper checks and the head of the queue, and every view tier triggers another pass.
- **"A brigade mass-reports one creator overnight."** → Reports raise priority but can't remove; reputation weighting and report-graph clustering (same target, correlated timing, shared origin) expose the campaign — a fraud-ring problem in moderation clothing.

- **"A government bans a new category within a week."** → Rules, hashes and report routing on day one, classifier in weeks; the region-keyed policy engine scopes it to one jurisdiction.

Toy — the cascade dial and prevalence sampling (`code/moderation_cascade_toy.py`)

One million uploads, 0.5% violating, heavy-tailed views. The fast classifier scores everything; the heavy model (100× the cost) scores the flagged slice; removal needs a high heavy score. We vary the flag rate, add view-tier re-scoring, then estimate prevalence by sampling views. (Simplification: removals happen before any views.)

```
def run(bad, views, fast, heavy, flag_rate, viral_tier=None):
    flagged = fast >= np.quantile(fast, 1 - flag_rate)
    if viral_tier is not None:
        flagged |= views >= viral_tier                          # reach-aware re-scoring
    removed = flagged & (heavy > 2.5)
    compute = N * 1 + flagged.sum() * HEAVY_COST                # fast on all + heavy on flagged
    viol_views = views[bad & ~removed].sum() / views[~removed].sum() # prevalence of violating views
```

Output:

```
violating uploads 0.49%, violating share of views before action 0.45%; top 1% of items get 62% of views

policy                             heavy runs  compute*  recall  viol. views false rem.
flag top 0.5%                           5,000     0.015   30.4%       0.345%         24
flag top 1.0%                          10,000     0.020   41.3%       0.288%         49
flag top 2.0%                          20,000     0.030   52.4%       0.172%        103
flag top 5.0%                          50,000     0.060   67.6%       0.136%        281
flag top 1.0% + re-score >=10K views     12,903     0.023   41.5%       0.136%         67
* compute relative to running the heavy model on every item

true prevalence of violating views: 0.136%
  sample  5,000 labelled views -> estimate 0.120%  (95% CI 0.024% to 0.216%)
  sample 50,000 labelled views -> estimate 0.152%  (95% CI 0.118% to 0.186%)
```

What it shows:

- The cascade costs 1.5–6% of "heavy model on everything"; a higher flag rate buys recall with diminishing returns.
- **Reach-aware re-scoring** matches the 5% flag rate's violating-view prevalence (0.136%) at well under half the compute (0.023 vs 0.060) and a quarter of the false removals, because 1% of items get 62% of views. Upload recall barely moves (41.5%) — the view metric is the one that tracks harm.
- Measuring 0.1% prevalence is hard: 5,000 labelled views give a CI of 0.02–0.22%, 50,000 narrow it to 0.12–0.19% — metric precision costs reviewer hours.

Common mistakes

Weak	Strong
One big classifier over everything	Cascade: expensive compute only on the slice that needs it
Remove vs allow	Full action ladder, including reduce and age-gate
Thresholds read off a PR curve	Thresholds as policy, per category and region, informed by measured errors

Same design, other questions

- **Marketplace counterfeits** — photos, text and price that don't fit (a luxury bag at a tenth of retail); actions hit listings and sellers. **DM spam** — private content, so triage uses sending behaviour and reports; rate limits before removal.

Self-check

1. Why measure prevalence on views rather than uploads? → Harm scales with exposure; an unseen violating item does little harm, a viral one a lot.
2. Why can't appeals alone estimate the false-positive rate? → They're self-selected; use random audits.

TL;DR

Shape	Four-stage cascade: fingerprints and rules → cheap triage on everything → big models on the suspicious few → people for hard cases
Metric	Violating-view prevalence per category with error bars; false-removal and overturn rates as the opposite guardrail
Decision	Action ladder; thresholds are policy per category and region
Scale	Flag rate = compute bill; review threshold = headcount; reach-aware re-scoring against viral misses

---

18.4 Case study: People You May Know (PYMK)

> A LinkedIn-style network with 900 M members wants a connection-suggestion module, shown on the home page and during onboarding. Design it.

1. Clarify requirements

Question	Assumption used below
Members	900 M registered (not daily active)
Surfaces	Home-page module; post-sign-up onboarding
Latency	~200 ms for the module; heavy work precomputed
Privacy	Suggestions can reveal sensitive relationships or invite unwanted contact — privacy controls are core requirements

Success has three layers, because each simple metric is a trap: invites sent ignores whether the recipient wants the connection; acceptance rate alone rewards suggesting obvious existing acquaintances; the real goal is **network value** — connections that go on to interact.

Layer	Metric
Primary (online)	Accepted invitations per suggestion impression
Guardrails (can veto)	Invitation-ignore rate; "I don't know this person" reports (weighted heavily in training, like hides in 17.2)
Long-term (weeks later)	Interaction between newly connected pairs

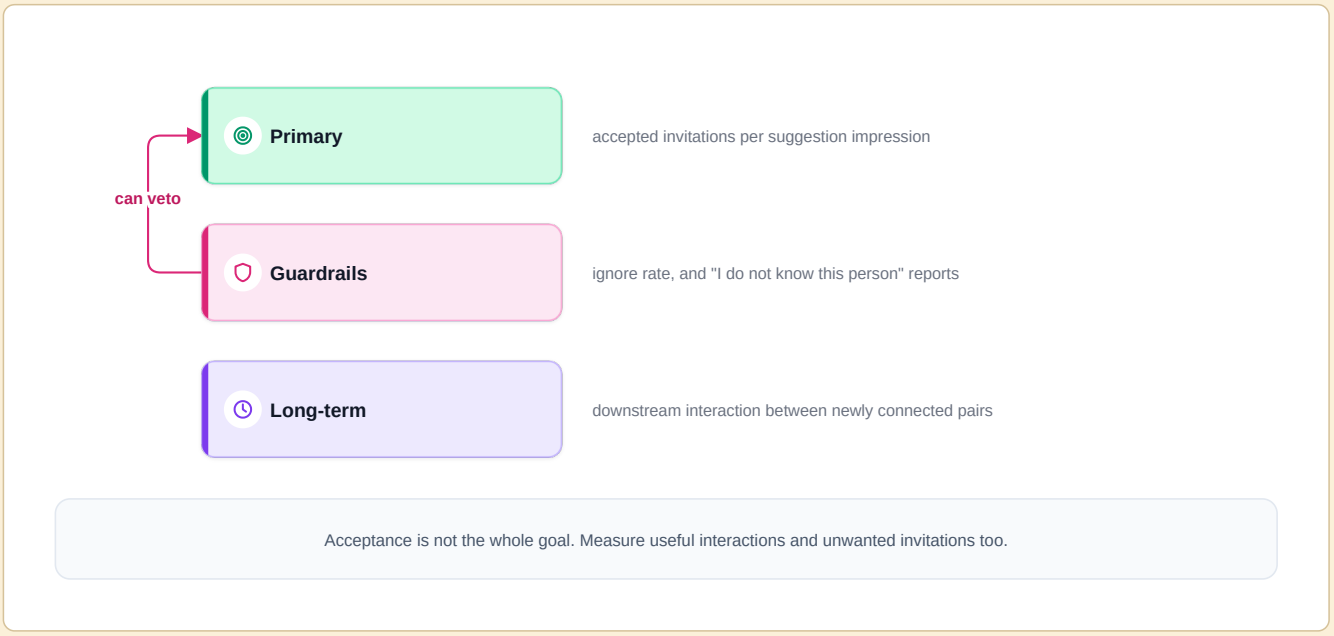


Figure — Three layers of metric, because acceptance alone is a trap (course diagram)

2. Frame it as an ML task

- **Input**: viewer, candidate member, the graph around them, context (new vs tenured member).
- **Output**: two probabilities per pair — **P(send)** = P(invite sent | shown), does the *viewer* want it? — and **P(accept | sent)**, does the *candidate* want it?
- **Ranking score** = P(send) × P(accept | sent) = probability of an accepted invitation per impression.
- **Category**: **link prediction** (predict which edges will form in a graph) + two-sided ranking.

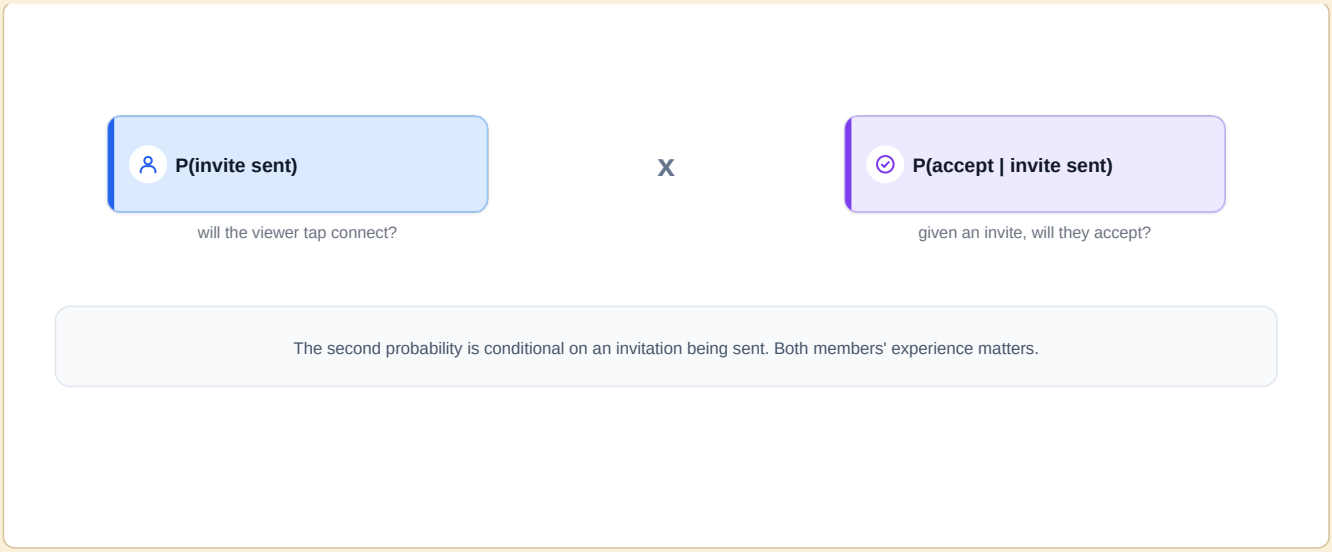


Figure — The score multiplies the viewer's intent by the recipient's willingness (course diagram)

3. Data and labels

- **Graph**: connections, follows, past invites (sent, accepted, ignored), dismissals, blocks. **Profiles**: employer, school, groups, skills, location, with dates (overlap in time matters).
- **Imported contacts** (only with permission): minimise collection and retention, support opt-out and deletion; hashing identifiers does *not* make them anonymous.
- **Labels**: invite sent arrives in-session; acceptance takes days. A pending invite is *unresolved*, not rejected — use an observation window (the delayed-label idea of 18.1).

4. Features

Group	Examples
Graph	Mutual-connection count and strength; **Adamic-Adar** score — sum over mutual friends of 1 / log(that friend's number of connections), so a shared friend with few connections counts more than one who knows everyone; graph-embedding similarity
Affinity	Same employer/school/group with overlapping dates, similar skills, profile views of each other
Behavioural	Viewer's historic invite-acceptance rate; candidate's accept rate and inbox load
Context	New vs tenured member; surface; times this pair was already shown

5. Model: baseline → advanced

Candidate generation — four sources, unioned and deduped (17.1's multi-source retrieval, with people as the catalog):

1. **Friends-of-friends (FoF) / triangle closing** — if A–B and B–C are connected, C is a candidate for A. The main source, precomputed and **bounded**: expand only top neighbours by interaction, sample very-high-degree accounts' lists, cap candidates per member.
2. **Shared affiliations** — same company/school/group with overlapping dates; catches the new coworker with no mutuals yet.
3. **Imported contacts** — permission-based matches.
4. **Embedding similarity** — two-tower member vectors from profile and activity, or graph embeddings (node2vec-style random walks, graph neural networks), queried through an ANN index.

Hard filters at retrieval (re-checked before display): already connected, blocked, dismissed, opted out. **Impression discounting** lowers candidates shown many times without action — softer than an explicit, permanent dismissal.

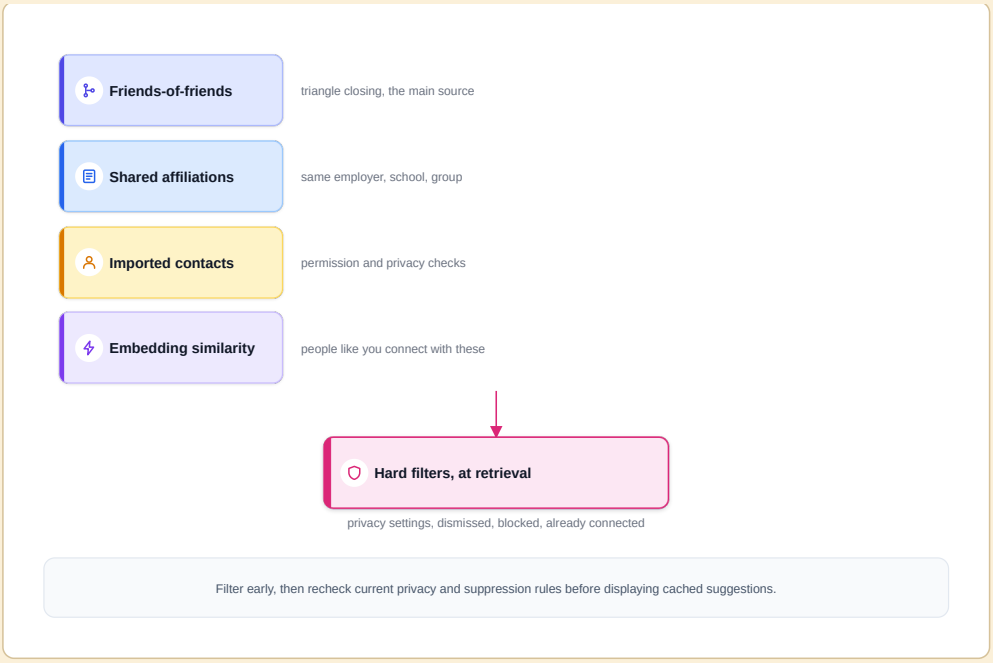


Figure — Four retrieval sources, then hard filters before ranking (course diagram)

Ranker: baseline = sort FoF candidates by mutual count or Adamic-Adar; advanced = GBDT or a two-head network predicting P(send) and P(accept | sent), multiplied.

6. Offline metrics

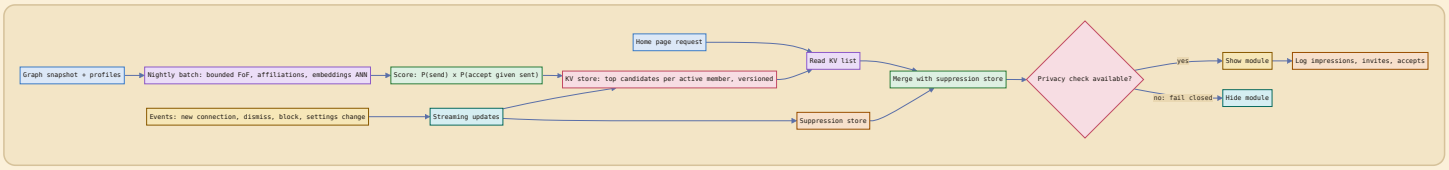
- **Recall@K of future edges**: hide connections formed after a cutoff and check whether retrieval would have produced them.
- **AUC / log loss** per head, plus calibration — the heads are multiplied.
- Diversity: share of suggestions going to the top-1% most-connected members; share crossing community boundaries.

7. Online metrics and A/B testing

- The three metric layers, measured on **both sides** (inviter and recipient).
- **Network interference**: a treated viewer invites a control member, so user-level randomisation leaks. Use **graph-cluster randomisation** (assign whole communities to one arm) or model the interference.
- The interaction metric takes weeks, so keep a long-running **holdback** (16.9).

8. Serving architecture

- **Nightly batch**: bounded FoF, affiliations, scoring → top candidates per active member in a **key-value (KV) store**, with **idempotent, versioned** writes (a retry can't overwrite a newer run).
- **Streaming path**: new connections, dismissals and blocks update a **suppression store** within minutes (seconds for dismissals).
- **Request time**: read the list, merge with suppressions, re-check current privacy settings, render.
- **New members**: contacts and affiliations carry the first session, refreshed within minutes as first connections form.



9. Scale numbers

Quantity	Arithmetic	Result
Batch scope	Active in last 30 days only (assume 300 M) instead of 900 M	~⅓ of the graph work; dormant members refreshed weekly
FoF blow-up	500 connections, each with 500 → up to 250,000 two-hop paths	why expansion is capped to top neighbours
KV storage	300 M members × 100 candidates × ~16 bytes	≈ 480 GB before replication

The dominant cost is usually the **graph computation**, not inference.

10. Monitoring and failure modes

Graph-shaped feedback loops: **rich-get-richer** (well-connected members sit in more triangles, get suggested more, grow more connected — cap exposure per candidate) and **cluster amplification** (triangle closing reinforces silos — monitor cross-cluster suggestions).

Failure	Response
Nightly graph job fails	Serve last validated candidates within a freshness limit, still enforcing *current* privacy, blocks, dismissals; alert on age
Privacy filter unavailable	**Fail closed**: render nothing rather than unverified suggestions
Dismissed person reappears an hour later	Design bug: dismissals must go through the streamed suppression store merged at render time

11. Interview follow-ups

- **"FoF for 900 M members without melting the cluster?"** → Top neighbours only, 30-day-active members nightly, sampled celebrity neighbour lists, capped outputs — never the full two-hop closure.
- **"Legal asks you to prove opted-out members never appear."** → Auditable filters at retrieval *and* before display, fail-closed on timeouts, prompt cache invalidation, filter-version logging, tests for stale caches and races.

Toy — FoF retrieval, Adamic-Adar, two-sided ranking (`code/pymk_toy.py`)

A 4,000-member clustered graph (networkx `powerlaw_cluster_graph`) with 10% of edges hidden as "future". Part 1 ranks FoF candidates three ways; part 2 simulates viewers who like inviting popular members who rarely accept.

```
def fof_candidates(G, u):
    """Two-hop neighbours with their mutual-connection lists (bounded in production: top neighbours only)."""
    mutual = {}
    for f in G[u]:
        for c in G[f]:
            if c != u and not G.has_edge(u, c):
                mutual.setdefault(c, []).append(f)
    return mutual

scores = {
    "random FoF": {c: random.random() for c in cands},
    "common neighbours": {c: len(m) for c, m in mutual.items()},
    "Adamic-Adar": {c: sum(1 / math.log(deg[f]) for f in m if deg[f] > 1) for c, m in mutual.items()},
}
# part 2: rank the top 10 by p_send alone vs p_send[c] * p_acc[c]
```

Output:

```
4000 members, 15977 connections, 1597 hidden as future; max degree 264, median 5
avg FoF candidates per member 155; FoF retrieval covers 65.0% of future connections

random FoF           recall@10 = 0.080
common neighbours    recall@10 = 0.216
Adamic-Adar          recall@10 = 0.287

ranking (top 10 per member) invites sent  accepted accept rate  to top-1% members
P(send) only                       4.44      0.80       17.9%              65.4%
P(send) x P(accept)                2.48      1.35       54.6%               1.5%
```

What it shows:

- FoF covers 65% of future connections — a third come from elsewhere, hence four sources. Within FoF, common neighbours is 2.7× better than random, and **Adamic-Adar** adds another third.
- Ranking by P(send) alone sends 65% of suggestions to the top 1% most-connected members, who accept only 18%. The **two-sided score** sends fewer invites but yields 69% more accepted connections (1.35 vs 0.80 per member). (Behaviour is simulated; a real system learns both heads.)

Common mistakes

Weak	Strong
Optimises invites sent or acceptance alone	P(send) × P(accept | sent), ignore/report guardrails, long-term interaction
Privacy filtering only at display	Filters at retrieval *and* before display; fails closed
Dismissals wait for the nightly batch	Streamed suppression merged at render time

Same design, other questions

- **Groups to join** — one side is an aggregate, so predict join × stay-and-participate; big groups dominate triangles, so a diversity cap is mandatory. **CRM contact suggestions** — shared accounts matter more than mutuals; the second side is "will this contact reply".

Self-check

1. Why multiply P(send) by P(accept | sent)? → The product is the probability of an accepted invite per impression — both sides must want it.
2. Why randomise by graph cluster? → Members interact across arms, so individual randomisation leaks treatment into control.
3. What if the privacy check can't run? → Hide the module (fail closed).

TL;DR

Objective	P(send) × P(accept | sent); ignores and "don't know" reports as guardrails; lasting interaction as the goal
Retrieval	FoF + affiliations + contacts + embeddings; hard privacy/dismissal filters
Serving	Nightly bounded batch → versioned KV; streamed suppressions
Non-negotiables	Privacy fails closed; dismissals take effect immediately; watch rich-get-richer

---

18.5 Case study: music discovery (Spotify-style)

> "Design discovery for a music streaming service with 400 million users and 100 million tracks: personalised weekly playlists like Discover Weekly, plus a 'radio' that plays endlessly from any seed song."

1. Clarify requirements

The recognition sentence: **"It's the recommendation funnel used twice — once in batch, once online."**

- **Batch** = computed ahead of time on a schedule. Discover Weekly is one precomputed playlist per user per week (launched by Spotify in 2015; refreshed every Monday).
- **Online** = computed at request time. Radio picks the next track from the current session.

Question	Assumption used below
Users / tracks	400 M users (≈ 250 M weekly active), 100 M tracks, many uploads daily (pick a rate for capacity planning)
Discover Weekly	Batch with a deadline (ready before Monday) plus a serving cache
Radio	Next-track decision in ~100–200 ms using session context
Out of scope	Search, social features, licensing economics

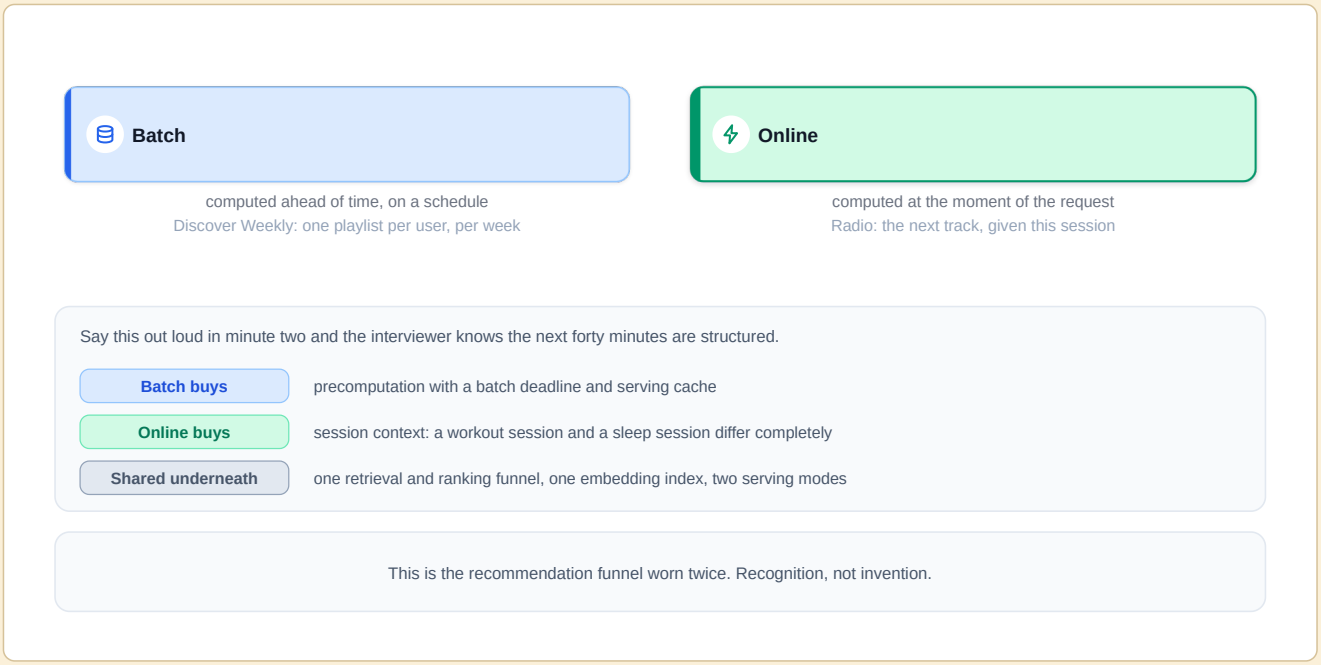


Figure — The same machinery, used as batch and as online (course diagram)

2. Frame it as an ML task

- **Discover Weekly**: retrieval + ranking of tracks the user *hasn't heard* but will like — recommendation with a novelty constraint.
- **Radio**: **sequential recommendation** — predict the next track from the seed and the session so far.
- **Labels**: listens past **30 seconds**, saves, playlist adds, artist follows = positive; an **early skip** = fast but noisy negative (interruptions, repetition, a change of activity).
- **Launch metric** (Discover Weekly): tracks **saved** per eligible user. **Guardrail**: **diversity and artist fairness** — discovery that keeps serving the same hits fails even when engagement looks fine.

3. Data and labels

Plays, skips (with position in the track), saves and session sequences; catalogue metadata (artist, genre, release date, explicit flag, market availability); **the audio itself**; and billions of **user-made playlists** — tracks on the same playlists go together. Labels arrive fast (a skip in seconds).

4. Features

Three signal families — and what each does for **cold start**:

Signal	What it captures	New track with zero plays?
Collaborative (listening co-occurrence, matrix factorisation)	Who listens to what together	Useless until plays accumulate
Audio content embeddings (model over the raw audio)	Tempo, timbre (texture of the sound), mood, instrumentation	Works from day zero
Text / metadata embeddings (tags, playlist titles, editorial, web text)	Genre, scene, cultural context	Works from day zero

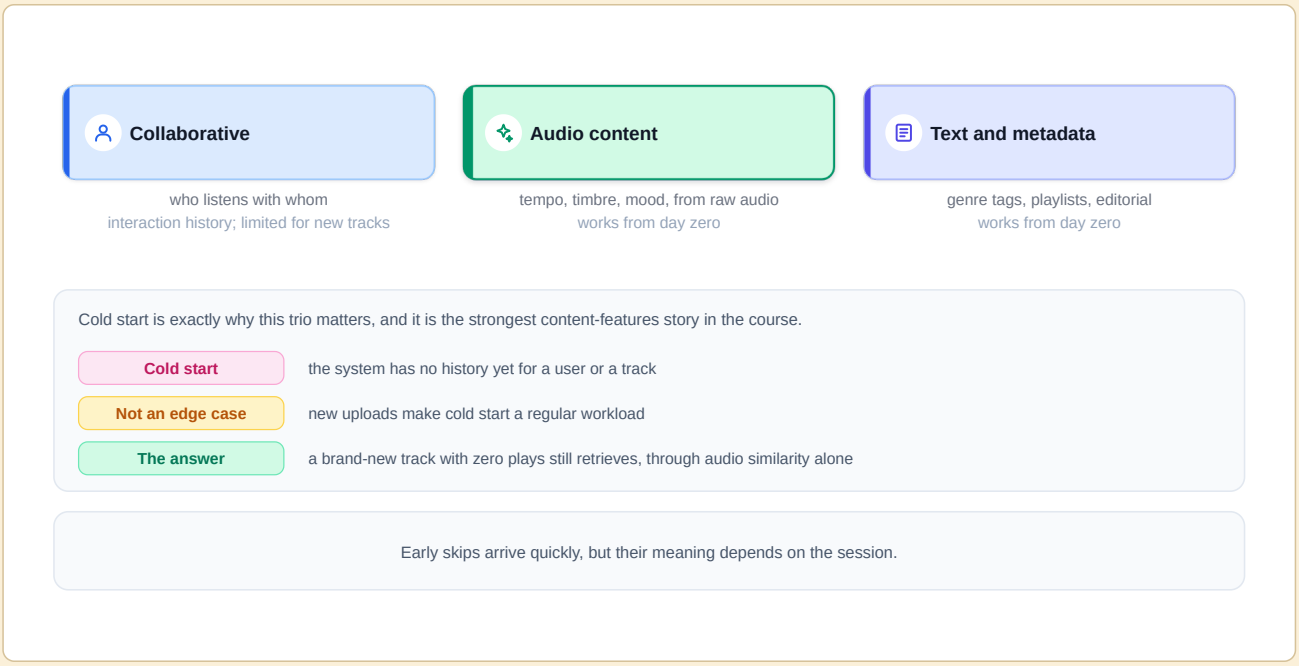


Figure — Three signal sources, and what each does at cold start (course diagram)

Radio session features: seed track, last N tracks, skips this session, time of day, device. **Session context is first-class** — the same user wants different music at the gym and at bedtime.

5. Model: baseline → advanced

1. **Baseline**: popularity within a **cohort** (a group of similar users, e.g. new listeners in one country who like a genre).
2. **Collaborative embeddings**: matrix factorisation on implicit feedback (implicit ALS — Hu, Koren & Volinsky, 2008) or a two-tower model (Chapter 13).
3. **Content encoders**: a CNN or audio transformer over spectrograms (van den Oord, Dieleman & Schrauwen, 2013, showed CNNs can predict collaborative-filtering vectors from audio); a text model for metadata. Map content embeddings into the collaborative space so new tracks are retrieved like established ones.
4. **Sequence model for radio**: from item-to-item transitions, to recurrent GRU4Rec (Hidasi et al., 2016), to self-attention SASRec (Kang & McAuley, 2018).
5. **Ranker**: GBDT or neural, with features from all three families plus session context.

6. Offline metrics

- **Recall@K / Hit@K** on held-out listens (**Hit@K** = share of cases where the true next item is in the top K); **MRR** for radio's next track.
- **Coverage and diversity**: share of catalogue ever recommended, artist concentration (e.g. **Gini coefficient** of exposure — 0 = perfectly even, 1 = one artist gets everything), novelty.
- **Cold-start slice**: tracks released after the training cutoff.

7. Online metrics and A/B testing

- **Discover Weekly**: saves per user, 30-second-listen rate, return next week. **Radio**: skip rate, session length. **Longitudinal truth**: repeat listening of discovered tracks a month later.
- **Exploration**: a small randomised slice gives less-exposed tracks a chance — **artist-ecosystem policy** as much as ML. If it costs skip rate, bound it and judge it on a long-term holdback (16.9) rather than let the short-term metric veto it.

8. Serving architecture

1. **The weekly batch runs only on weekly-active users** — scoring all 400 M when ~250 M are active wastes over a third of the compute. Output → KV store, served from cache.
2. **Radio prefetches the next 2–3 tracks**, hiding ranking latency; a skip **re-scores the queue immediately**, and a backup queue covers skip storms.
3. **A rules layer outside the model**: artist spacing, explicit-content filters, market licensing.

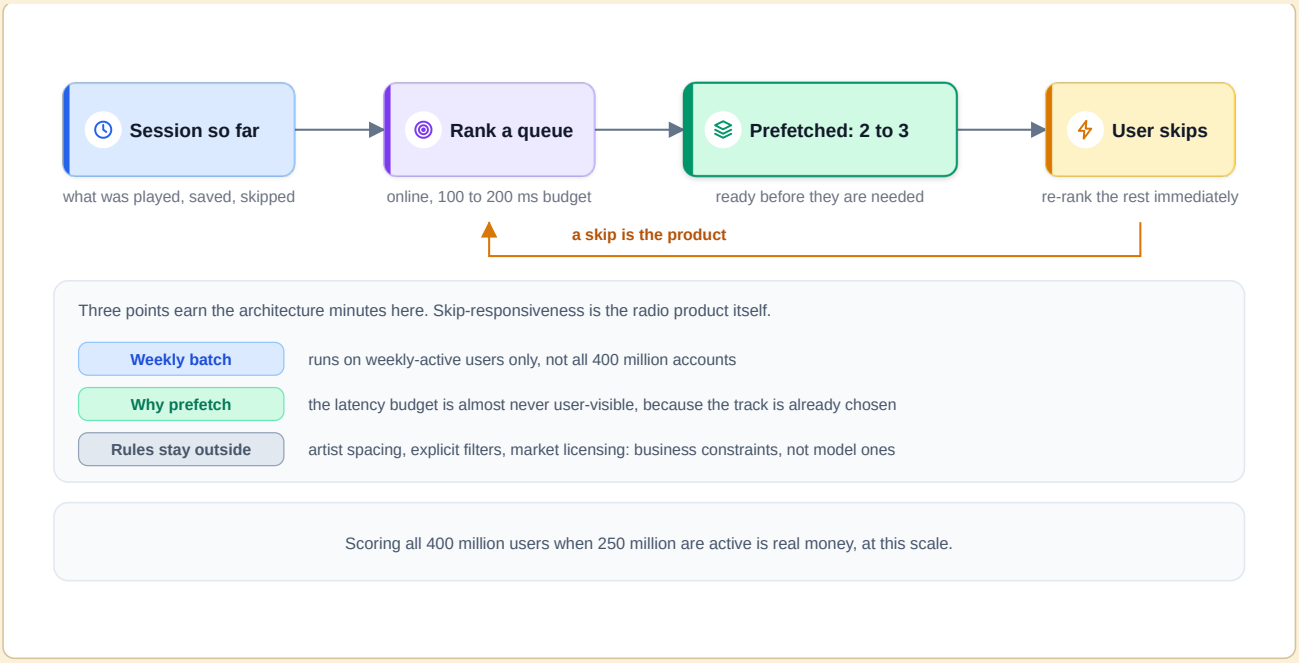
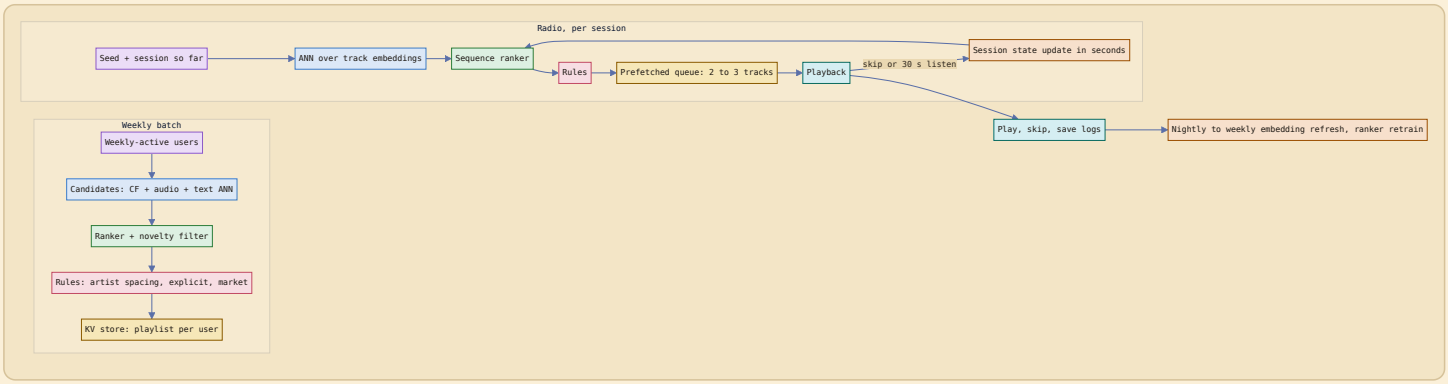


Figure — Radio prefetches the queue, so a skip re-ranks instantly (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Weekly batch	250 M users × 30 tracks	7.5 B playlist entries/week — a compute job with a deadline, not a latency problem
Waste avoided	150 M inactive users not scored	~37.5% of the batch
Track embeddings	100 M × 128 dims × 4 bytes	51.2 GB raw per embedding space (before index overhead and replicas)
Radio QPS	e.g. 20 M concurrent radio listeners ÷ ~200 s per track	≈ 100 K next-track decisions/s

10. Monitoring and failure modes

- **Loops**: skips update session state in seconds; embeddings refresh nightly to weekly; ranker training uses **position-bias-corrected** logs (17.3), or it copies its own previous ordering.
- **Funnel observability**: log per track, per stage — retrieved → ranked where → shown → played. If a label claims its artists are buried, you can show which stage dropped them.

Failure	Fallback
Weekly batch fails	Last week's playlists keep serving; alert on staleness
ANN index or ranker down during radio	Cached queue → genre-station heuristic. **Never silence**
Embedding refresh out of sync	Version index + vectors as a pair, promote atomically, check versions per request

11. Interview follow-ups

- **"Power users are bored by midweek."** → Keep the weekly ritual; add a daily shelf re-ranked on this week's listens from the same precomputed candidates — cheap freshness.
- **"Radio for a brand-new user?"** → Audio neighbours of the seed carry the session, market priors fill the ranker, the first skips teach fast — radio is the *best* cold-start surface.

Toy — session context and audio embeddings for the next track (`code/music_session_toy.py`)

2,000 tracks, each with a genre and a mood (workout/sleep/focus) encoded in a synthetic "audio embedding". Users like two genres, each session has one mood, 20% of tracks are released after training. Predict a session's 4th track from its first 3 (Hit@10).

```
elif method == "session transitions":
    score = trans[prefix[-1]] + 1e-3 * plays          # item-to-item "played B right after A"
elif method == "session audio embedding":
    score = audio @ audio[prefix].mean(0)             # nearest to the session's average sound
elif method == "audio + user taste":
    score = audio @ audio[prefix].mean(0) + 0.3 * (user_genre[u][genre] > 0)
```

Output:

```
method                       Hit@10  Hit@10 on NEW tracks
popularity                    0.004                 0.000
user taste (no session)       0.020                 0.000
session transitions           0.039                 0.000
session audio embedding       0.080                 0.107
audio + user taste            0.081                 0.107
ceiling (true pool, random)   0.092                 0.073
(205 of 1000 test targets are tracks released after training)
```

What it shows:

- **Ignoring the session** scores 0.020 (right genres, usually the wrong mood). **Session transitions** double that but, like every play-count method, score **zero on new tracks**.
- **Session audio embeddings** reach 0.080, near the ceiling of knowing the exact eligible pool (0.092), and alone find new tracks (0.107) — cold start in miniature. (Numbers are low because each pool holds ~130 equally likely tracks.)

Common mistakes

Weak	Strong
One serving mode for both products	Batch weekly playlist, online sequential radio — said in minute two
Collaborative filtering only	Audio + text embeddings so new tracks are retrievable from day zero
Engagement only	Diversity/artist-fairness guardrail and a defended exploration slice

Same design, other questions

- **Podcasts** — same funnel; transcript and speech embeddings, completion curves as labels, and a higher precision bar (each pick costs an hour of trust). **News "For You"** — recency is everything, so overnight precomputation is nearly worthless: streaming signals and per-request ranking.

Self-check

1. Which product is batch and which online, and why? → Discover Weekly: one list per week (deadline, cache). Radio: needs session context per request.
2. Which signal handles a track uploaded today? → Audio and text/metadata embeddings.
3. What does prefetching buy? → Hides ranking latency; must re-rank on skip and keep a fallback queue.

TL;DR

Big picture	One recommender served two ways: a precomputed weekly list and a live next-track picker
What's special	Audio vectors make unplayed tracks findable; skips are fast but noisy; the session shapes every pick
Architecture	Active-users-only batch → KV; radio = ANN + sequence ranker + prefetch, re-rank on skip
Guardrails	Diversity and artist fairness beside engagement; continuous audio fallback

---

18.6 Case study: visual search for a second-hand marketplace

> A used-goods marketplace has about 100 M active listings, and each one disappears when it sells, so the catalogue turns over constantly. Shoppers should be able to snap a picture of something they like — say a vintage lamp or a coat a friend is wearing — and see comparable items on sale. A second request: let people type a description to find things among listings they have saved.

1. Clarify requirements

The recognition sentence: **"This is the search funnel of 17.3, except the query is a photo."** The front stage becomes an **encoder** and retrieval runs in an embedding space, not an inverted index (which can search captions but not visual similarity). The hard part is **the index**: 100 M listings where every sale is a deletion. The second request is a clue: if photos and text share **one embedding space** (encoders trained so a photo and its description land close together), a single index answers both.

Question	Assumption used below
Listings	100 M live, millions of new listings/day with several photos each
Latency	~300 ms photo-to-results — say whether upload and compression count
In scope	Picture-to-listing lookup; typed lookup within a user's *saved* items
Out of scope	Exact-product recognition, AR try-on, anything generative
Signature failure	Showing items that are already sold

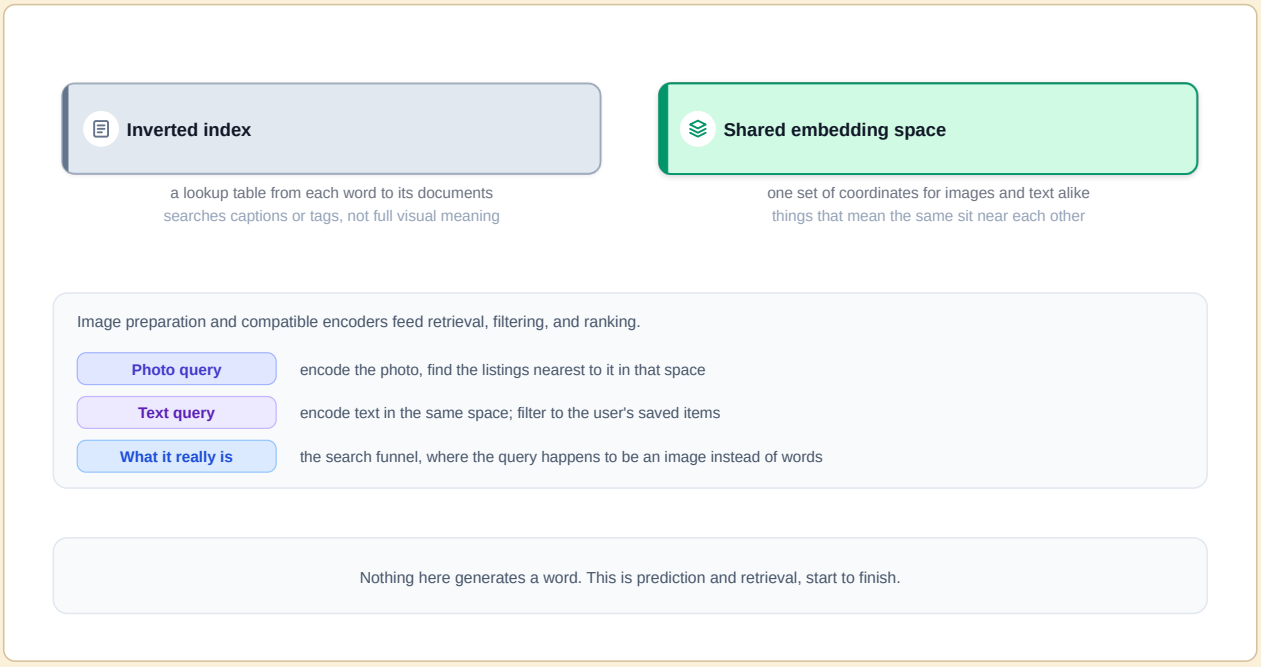


Figure — Image-text embeddings support visual retrieval beyond caption matching (course diagram)

2. Frame it as an ML task

- **Input**: query photo (or text); candidate listings (photos, title, attributes).
- **Output**: similarity in the shared space, then a purchase-likelihood ranking score.
- **Category**: **representation (metric) learning** — an embedding where distance = similarity — trained with a **contrastive loss** (pull matching pairs together, push others apart), plus nearest-neighbour retrieval and ranking.

3. Data and labels

- **Photos**: a few per listing, amateur phone shots; **query photos are often worse** — include them in training and evaluation. **Text and attributes**: title, description, brand, size, condition, price, category.
- **Labels in two phases** (at launch there are no behaviour logs — the **bootstrap problem**):
  1. **Human-judged pairs**: raters mark (query photo, listing) as close match, partial or miss; scored with NDCG. The judged set becomes the permanent **regression suite**.
  2. **Behaviour** later: clicks, then **contact-seller** events, then transactions — with position-bias correction on the results grid (17.3).
- **Free positive pairs**: different photos of the same single-item listing (after screening out bundles and stock images).

4. Features

Stage	Features
Retrieval	Query image (or text) embedding vs listing image embeddings
Filters	Category, price range, location/shipping, live status (not sold)
Ranker	Embedding similarity, **price proximity**, condition, seller rating, listing freshness, distance/shipping cost

Retrieval finds lookalikes; the ranker decides what's worth buying.

5. Model: baseline → advanced

- **Baseline**: a **pretrained image-text model**, no fine-tuning — CLIP-style (OpenAI's CLIP, Radford et al., 2021, trained contrastively on ~400 M image-caption pairs). Ship it against the judged set in week one.
- **Upgrade — fine-tune on marketplace pairs** with **InfoNCE** (the contrastive loss CLIP uses: a softmax over similarities that should pick the true partner out of the batch):
  - positives: two photos of the same listing; photo ↔ its title/description;
  - **in-batch negatives**: other items in the batch — cheap, but near-duplicates become false negatives;
  - **category-hard negatives**: a *different sofa* rather than a toaster — sharpens distinctions buyers actually make.
- **Light ranker** (GBDT) on the structured features.

- **Query preprocessing**: a light object detector crops the dominant item (plus tap-to-select). **Composed queries** ("this jacket, but in black"): fuse image and text scores, route colour/size to filters, evaluate them explicitly.

6. Offline metrics

- **NDCG@k and recall@k** on the judged set, **per category** — an embedding can excel on sofas while quietly failing on sneakers.
- **ANN recall vs exact search** at the chosen index settings. Include poor query photos, or offline numbers flatter the model.

7. Online metrics and A/B testing

- **Primary**: contact-seller rate per search. **Guardrails**: sold-item impressions (target ≈ 0), zero-results rate, latency, offensive-adjacency reports.
- **Testing an encoder that needs a full re-embed**: judged-suite gate → index one category with it → **interleave** for a sliver of users → only then re-embed everything.

8. Serving architecture

Request path: upload → crop → image encoder (e.g. a 20 ms GPU budget — benchmark it) → **ANN search** with filters → light ranker → **live-status hydration** → results.

ANN index families (details in Chapter 13.9):

- **IVF** (inverted file): k-means groups vectors into buckets; a query scans only the `nprobe` nearest buckets. Often combined with **product quantisation (PQ)**, which compresses each vector into a few bytes (FAISS, Johnson, Douze & Jégou, 2017).
- **HNSW**: a layered proximity graph walked greedily from coarse to fine (17.1).

Filtered ANN: a very selective post-filter can leave too few results — fall back to more candidates, partitioned indexes or exact search on small subsets.

The index path is the heart of this design:

- **Embed at creation**, streamed into the index within minutes — a listing unfindable for a day may already be sold.
- **Tombstone on sale** (mark deleted) and **check live status at render time** — index updates alone can't guarantee no sold items.
- **Version model and index together, promote atomically**: a new encoder against an old index returns nonsense even if dimensions match, so upgrades mean a planned, budgeted **re-embed of the corpus**, not a deploy.

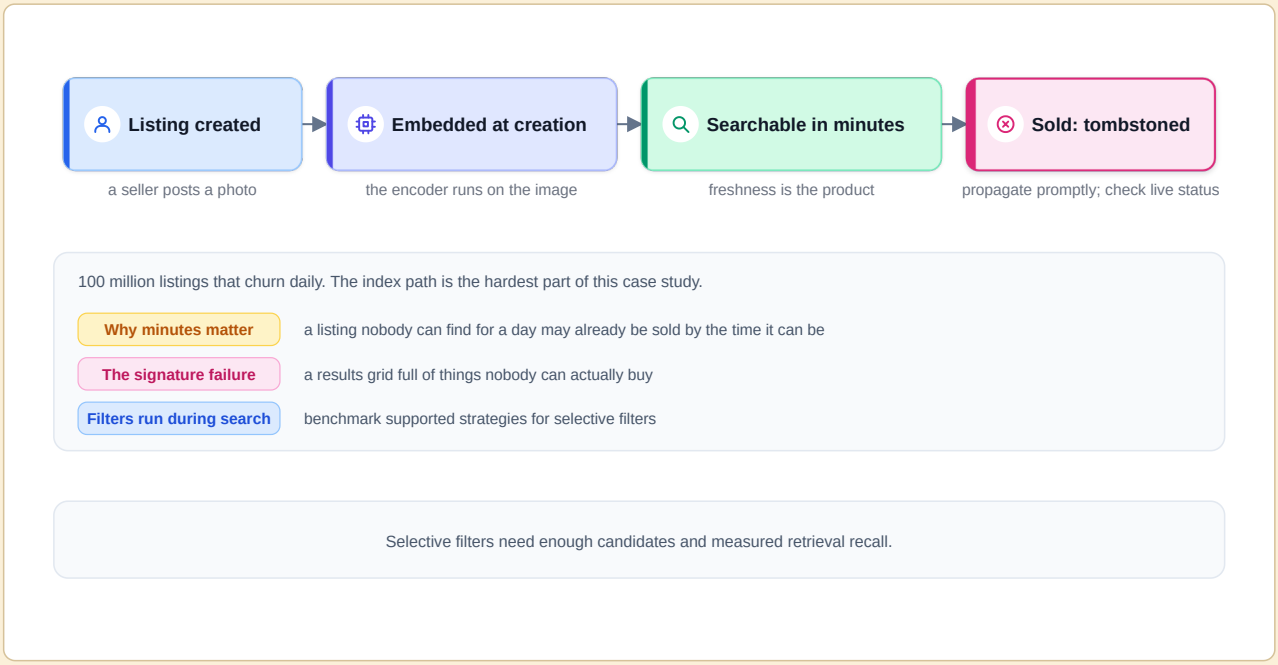


Figure — Index churn: every sale is a deletion (course diagram)

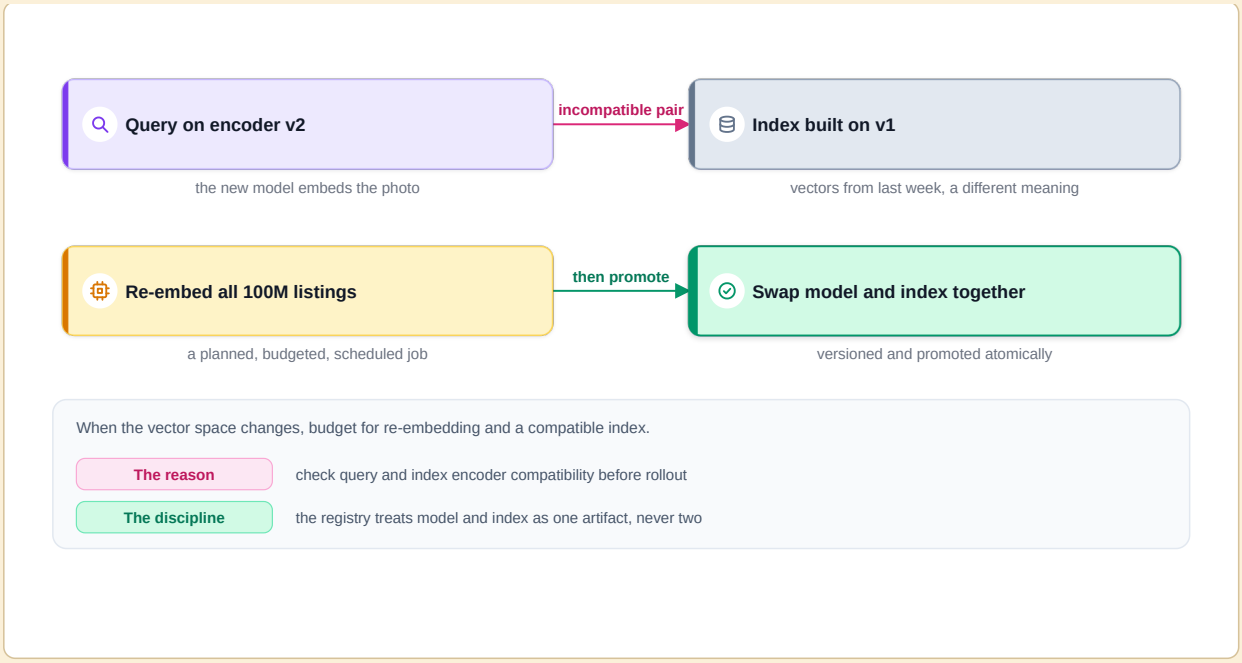
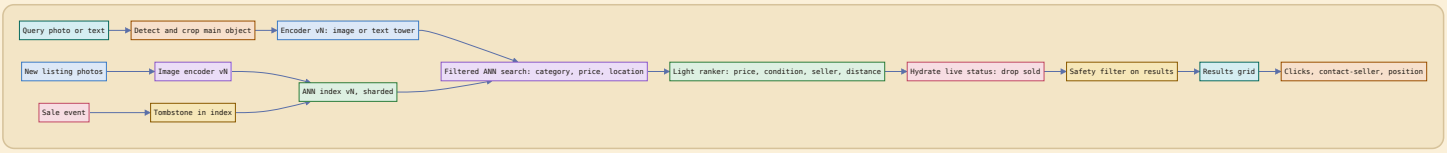


Figure — An incompatible encoder upgrade requires a compatible replacement index (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Index memory	100 M × 512 dims × 4 bytes (float32)	204.8 GB raw, before graph overhead, metadata, replicas; more if several photos per listing are indexed
With PQ compression	e.g. 64 bytes per vector	6.4 GB — at some recall cost
Encoding load	e.g. 5 M new listings/day × 4 photos	20 M encodings/day ≈ 230/s — possibly more than query traffic
Re-embed on upgrade	100 M listings × several photos	a multi-day batch plus temporary dual-index capacity

Shard the index; a lost shard means reduced recall, backed up by text search or category browse — degrade, don't error.

10. Monitoring and failure modes

- Every encoder version replays the growing judged suite; online, watch contact-seller rate **per category**. **Seasonal drift** (coat season → swimwear) means budgeted periodic retrains and index rebuilds.

Failure	Defence
Model–index **version skew**	Atomic promotion + per-query version check
Tombstone lag (sold items shown)	Render-time live-status hydration — the stale-cache rule of 17.3 applied to "sold"
Offensive adjacency (similar-looking but inappropriate result)	Safety filter on results, using the moderation machinery of 18.3
Counterfeits via stock photos	Near-duplicate detection against catalogue imagery at listing time

11. Interview follow-ups

- **"Why not predict a category and search inside it?"** → Classifiers break on items straddling categories (a futon: couch or bed?) while embeddings degrade gently; use the taxonomy as a filter and feature, not the retrieval.
- **"Results look right but nobody contacts sellers."** → Relevance ≠ desirability: overpriced, worn, distant or zero-history lookalikes. That's the ranker's job — measure on contact-seller rate, not clicks.

Toy — contrastive image-text space + IVF search (`code/visual_search_toy.py`)

Synthetic "photos" are noisy 64-d views of a hidden item vector plus nuisance directions (lighting, background); "descriptions" are noisy 24-d views. Two small encoders are trained with InfoNCE on 30 K listings and tested on 5 K *new* ones with a noisier "buyer photo" as query; then IVF over 100 K listings.

```
def info_nce(a, b, temp=0.07):
    a, b = nn.functional.normalize(a, dim=1), nn.functional.normalize(b, dim=1)
    logits = a @ b.T / temp                                          # every other row = an in-batch negative
    target = torch.arange(len(a))
    return (nn.functional.cross_entropy(logits, target) + nn.functional.cross_entropy(logits.T, target)) / 2

loss = info_nce(f_img(p1[b]), f_img(p2[b])) + info_nce(f_img(p1[b]), f_txt(d[b]))   # photo-photo + photo-text

# IVF: k-means buckets (plain numpy Lloyd's, 10 iterations), scan only the nprobe nearest buckets
centers, labels = kmeans(idx_vec, 256)
buckets = np.argsort(-(centers @ q))[:nprobe]
cand = np.concatenate([lists[c] for c in buckets])
```

Output (timings vary run to run):

```
recall@10 among 5,000 listings
  raw features, photo -> listing     : 0.070
  trained encoder, photo -> listing  : 0.527
  trained encoder, text  -> listing  : 0.381   (same index, text query)

IVF over 100,000 vectors, 256 buckets (exact search: 6.99 ms/query)
  nprobe= 1: scans  0.4% of index, recall vs exact 0.270, 0.05 ms/query
  nprobe= 4: scans  1.6% of index, recall vs exact 0.596, 0.15 ms/query
  nprobe=16: scans  6.3% of index, recall vs exact 0.916, 0.62 ms/query

without a live-status check, 4.9% of returned results are already sold -> filter tombstones
```

What it shows:

- Raw features barely work (0.070) — lighting and clutter dominate distance; contrastive training learns to ignore them (7.5× better on unseen listings). The **same index** answers **text queries** (0.381), so the saved-items request comes almost free.
- **IVF** trades recall for speed: scanning 6.3% of the index recovers 92% of the exact top-10 in about a tenth of the time (a Python loop; FAISS is far faster). `nprobe` is the dial.
- ~5% of results are already sold — why render-time hydration is non-negotiable.

Common mistakes

Weak	Strong
Treats the index as a static corpus	Embed at creation, tombstone on sale, hydrate live status at render
Swaps in a new encoder like a normal deploy	Versioned model+index pairs, atomic promotion, budgeted re-embed
Stops at "nearest neighbours"	A ranker with price, condition, seller, distance — lookalikes aren't purchases

Same design, other questions

- **On-device photo search** — distilled, quantised encoder and a small index on the phone; battery is the cost, nothing leaves the device. **Video moment search** — chunk videos into timestamped shots; the corpus multiplies while churn nearly disappears.

Self-check

1. Why does one index serve photo and text queries? → Both encoders map into one shared space.
2. What's an in-batch negative, and its risk? → Other batch items used as non-matches; near-duplicates become false negatives.
3. What does `nprobe` control in IVF? → How many buckets are scanned: higher = better recall, more latency.

TL;DR

Big picture	Search where the query is a picture; a shared image-text space lets one index answer photos and words
Twist	The index churns: embed at creation, tombstone on sale, check live status at render
Model	Pretrained CLIP-style → contrastive fine-tune with hard negatives → light ranker on structured signals
Versioning	Model and index promoted together; incompatible upgrades need a budgeted re-embed

---

18.7 Case study: privacy blurring in street imagery

> A mapping service drives camera cars through cities and publishes 360° street-level panoramas. Before any image is published, every face and every licence plate in it must be blurred. Design the system.

1. Clarify requirements

The recognition sentence: **"This is object detection where one kind of error is a privacy breach and the other is a blurred poster — so it is a recall problem first, and the unit that matters is the person, not the box."** It is also a **batch** design: nothing is served per user

request, so latency means "time from capture to publication", not milliseconds.

Question	Assumption used below
Volume	5 M new panoramas/day, each about 32 megapixels (8,192 × 4,096)
Objects	Faces (any angle, including tiny ones far away) and licence plates (many countries' formats)
Deadline	Published within a few days of capture; nothing unblurred ever leaves the pipeline
In scope	Detection, blurring, quality audits, a "report a problem" path from viewers
Out of scope	Recognising *who* someone is — the system must never identify people, only locate faces
Legal	Privacy law (e.g. the EU's GDPR) treats faces and plates as personal data; retention of the unblurred originals is a policy decision for legal, not the ML team

The two errors are not symmetric:

- **Miss** (false negative) → a recognisable face or readable plate is published. Harm to a real person, legal exposure, press coverage.
- **False alarm** (false positive) → a face on a billboard, a statue, a street-sign number gets blurred. Mildly ugly; blurring shop signs and house numbers does reduce the map's usefulness, so it is a real but smaller cost.

Say the target out loud: *"maximise recall per person; spend precision only where it hurts the product."* The published system Google described for Street View (Frome et al., 2009) framed it exactly this way and reported automatically blurring about 89% of faces and 94–96% of licence plates on their evaluation sets at the time — which is also why a manual **report path** is a permanent part of the design, not a temporary patch.

2. Frame it as an ML task

- **Input**: a panorama (plus the neighbouring panoramas along the drive, camera pose, GPS).
- **Output**: a set of regions to blur, each from a detected box with a class (face / plate) and a confidence.
- **Category**: **object detection** (11.7): predict boxes + classes + confidences, then **NMS** to remove duplicates.
- **Objective**: at the chosen threshold, maximise **object-level recall** (every view of every person is covered) subject to a budget on falsely blurred area.

3. Data and labels

- **Labelled panoramas**: boxes on faces and plates, including tiny, blurry, side-on, partly hidden and reflected ones. Guidelines matter: is a face on a poster a face? (Usually yes — blurring it is cheap.) Decide the minimum identifiable size and write it down.
- **Hard-case mining**: the rare cases (night, rain, crowds, motorbike plates, foreign plates) are where errors live. Mine them by running the current model and sending low-confidence and disagreement regions to labellers — **active learning**.
- **Public datasets** for pre-training: WIDER FACE (Yang et al., 2016) was built for faces with large variation in scale, pose and occlusion, including many tiny faces; plate datasets exist per region (e.g. CCPD, Xu et al., 2018, for Chinese plates).
- **The training data is itself sensitive**: annotators see unblurred faces. Restrict access, log it, and delete labelled originals on the retention schedule.

Labels from users: every "please blur this" report is a missed detection with a location — the most valuable label you get. Feed confirmed reports back into training as hard positives.

4. Features

This is an image model, so there is no feature table; the design choices are about **what the model sees**:

Choice	Why
Tiling the panorama into overlapping perspective views (e.g. cube faces), not the raw equirectangular image	Panoramas stretch objects near the top and bottom; detectors are trained on normal photos
Multiple scales: run once at normal resolution and once on 2× upscaled tiles near the horizon	Far-away faces are 10–20 pixels tall — too small for a detector's default input size
Overlap between tiles	An object cut by a tile edge is missed in both tiles
Neighbouring panoramas (the car took another picture 5–10 m later)	The same person appears in 2–4 consecutive panoramas; one view is often easier than the others
Geometry priors from camera height and the horizon line	A "face" floating in the sky or a "plate" 3 m above the road is probably a false alarm

5. Model: baseline → advanced

Baseline: off-the-shelf face and plate detectors on tiles, threshold 0.5, blur every box. The audit set shows where it misses (small, side-on, dark).

Upgrade 1 — one fine-tuned detector for both classes. A one-stage detector (the YOLO family, Redmon et al., 2016; or RetinaNet, Lin et al., 2017, whose **focal loss** down-weights the millions of easy background locations so training focuses on hard examples) fine-tuned on in-domain panoramas with tiling and multi-scale training. Transformer detectors such as DETR (Carion et al., 2020) predict a set of boxes directly without NMS — worth benchmarking, but one-stage CNN detectors remain the usual choice for high-resolution, high-throughput batch work.

Upgrade 2 — a cascade (18.3's idea) for recall at a sane false-alarm rate.

1. Stage 1 runs at a **very low threshold** (high recall, many false alarms).
2. Stage 2 is a small classifier on each candidate crop *with more context around it* (is this a face on a person, a face on a poster, or a pattern?). It removes most false alarms stage 1 created. Frome et al. (2009) used this shape: sensitive detectors first, then a second model combining detector scores with extra features to decide.
3. Anything stage 2 is unsure about goes to **human review** before publication.

Upgrade 3 — multi-view fusion. Track objects across consecutive panoramas using camera motion: a face detected confidently in one view is projected into its neighbours, and blurred there even if the detector missed it. The unit of success becomes the *person*, not the box.

Blurring itself: pad each box (localisation is imperfect — see the toy), blur with a method that destroys information (a strong Gaussian blur or solid fill; light pixelation of text can sometimes be reversed), and apply it to the published copy only.

6. Offline metrics

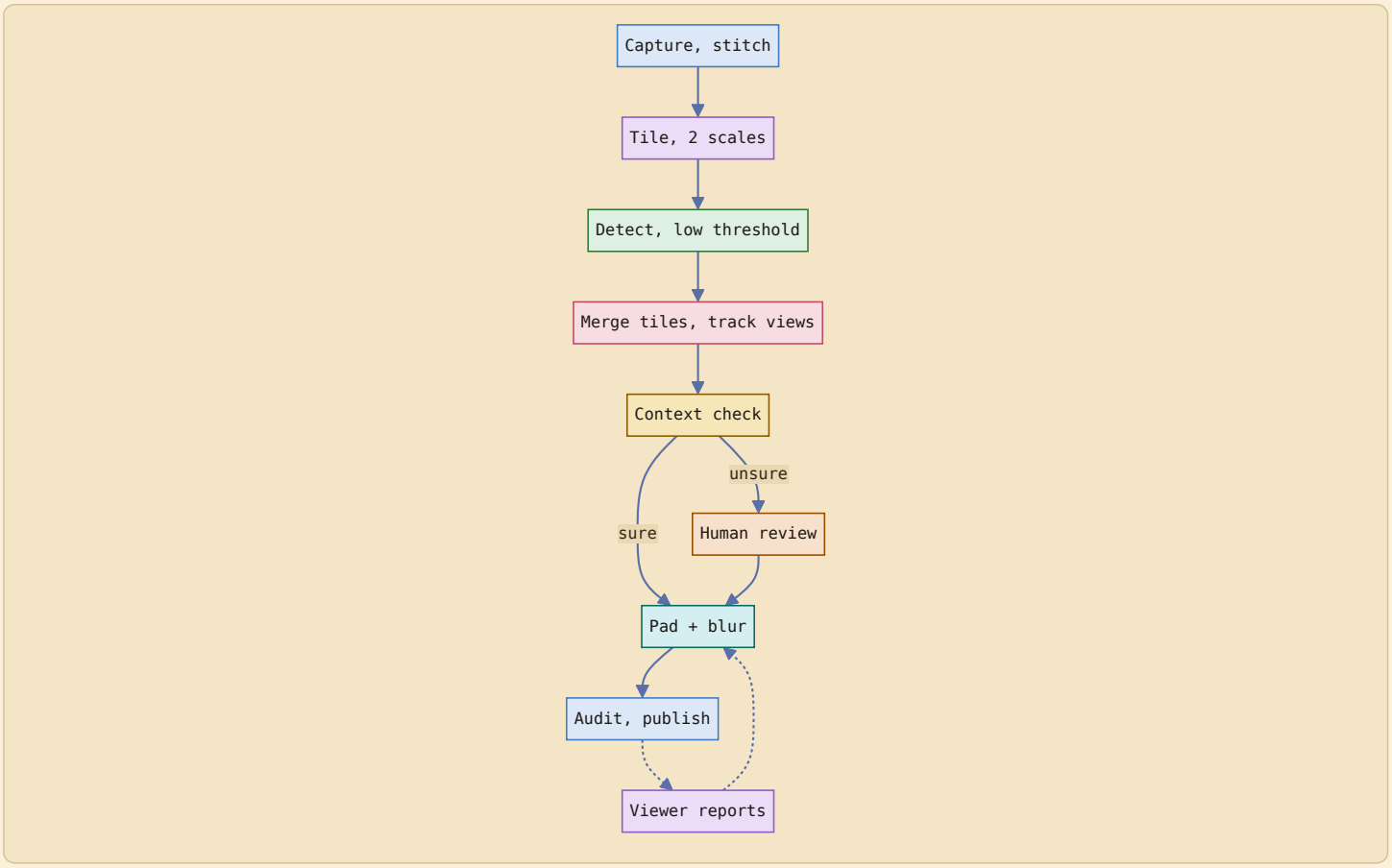
- **Recall by size bucket, class, region and lighting** at the operating threshold — overall recall hides the tiny-object failure.
- **Object-level recall**: share of physical people/plates covered in *every* view they appear in.
- **Pixel coverage**: share of each true object's area inside a blur region; count an object as **leaked** if more than, say, 5% of it is uncovered. A detection that is found but misplaced still leaks.
- **Cost of false alarms**: false blurs per panorama and share of image area blurred wrongly; separately track blurred **text** (signs, house numbers) because that is the useful content.
- mAP (11.7) is still useful for comparing detectors, but it is not the launch metric — it averages over thresholds nobody will use.

7. Online metrics and A/B testing

There is no A/B test on privacy: you cannot publish a less-safe variant to half the users. Instead:

- **Audits**: a random sample of published panoramas reviewed by humans each week → estimated **missed faces and plates per 1,000 panoramas**, with confidence intervals. This is the headline number (the same "prevalence by sampling" idea as 18.3).
- **Report rate**: user privacy reports per million panorama views, and **time to fix** a confirmed report.
- **Product guardrails**: complaints about unreadable signs, and the share of panoramas routed to human review (that is headcount).
- A new model is compared on the frozen audit set and in **shadow** on new captures before it replaces the old one.

8. Serving architecture



Stage by stage: panoramas are cut into overlapping perspective tiles at two scales; the stage-1 detector runs on GPUs in batches; boxes are mapped back to panorama coordinates, merged with NMS across tiles and the seam, and propagated to neighbouring panoramas by tracking; the context classifier either confirms or sends a region to human review; blurring is padded and irreversible; a weekly sample is audited. Confirmed viewer reports are blurred by hand within hours and become hard positives for training, as do audit misses.

Details to say out loud:

- **The seam**: a 360° panorama wraps around; a face cut by the left/right edge must be merged across it.

- **Fail closed**: if detection fails or times out for a panorama, it is **not published** (the opposite of most serving designs, which degrade and carry on).
- **Irreversibility and originals**: the published copy is blurred destructively; whether an unblurred original is retained, for how long and who can access it, is a legal policy — the pipeline must make deletion possible.

9. Scale numbers

Quantity	Arithmetic	Result
Tiles	5 M panoramas × 24 tiles (1,024², incl. overlap and a 2× horizon band)	120 M tile inferences/day ≈ 1,400/s
GPU time	120 M × 10 ms per tile (batched; benchmark it)	1.2 M GPU-seconds/day ≈ 14 GPUs busy all day; ~40 with headroom for re-processing and peaks
Stage 2	~2 low-confidence candidates per panorama × 5 M	10 M small crops/day — cheap
Human review	0.5% of panoramas × 5 M × 20 s each	25 K reviews/day ≈ 140 reviewer-hours/day (~18 full-time reviewers)
Re-processing	a better model re-run on a 10 B-panorama archive at the same rate	~5.4 years on 14 GPUs → needs a separate, much larger budget; prioritise by views

The last row is the hidden cost: improving the model helps new captures immediately, but the published archive only improves if you pay to re-process it.

10. Monitoring and failure modes

- **Drift**: new countries (plate formats), new camera hardware, winter clothing, night drives → track audited recall **per country and camera model**; report spikes from one city are an early alarm.
- **Model upgrades** can raise mAP while lowering tiny-face recall: gate releases on per-size-bucket recall.

Failure	Defence
Detector misses tiny/side-on faces	Multi-scale tiling, low stage-1 threshold, multi-view propagation, audits by size bucket
Found but misplaced box leaks part of a face	Padding (toy: 20% padding cuts leaks from ~80% of objects to ~6%)
Face cut by a tile edge or the 360° seam	Overlapping tiles, merge across the seam
Pipeline error on a panorama	Fail closed: do not publish
Over-blurring signs and numbers	Stage 2 context classifier; measure blurred text separately

11. Interview follow-ups

- **"Can you guarantee 100% of faces are blurred?"** → No detector reaches 100%. Layer the defences: high-recall detector, multi-view tracking, padding, human review of uncertain cases, audits, and a fast report-and-fix path — and state the audited miss rate honestly.
- **"Why not blur every person entirely with a segmentation model?"** → Possible (segmenting whole bodies, 11.7), and some operators blur people fully. It costs more of the image and more compute; faces and plates are the identifying parts. It is a policy choice to name, not a default.
- **"Should the model recognise people who asked to be removed?"** → No — that would require face *recognition*, a far more sensitive capability. Handle such requests by location (blur this house, this spot), not by identity.

Toy — the recall decisions around a detector (`code/blur_recall_toy.py`)

No images: we simulate what a detector outputs so we can study the decisions around it. Each of 13,333 physical objects appears in 3 consecutive frames at a different size; confidence rises with object size (small, far-away objects are hard), background patches give low-confidence false alarms, and predicted box corners are off by about 8% of the object size.

```
score = sig(2.2 * (np.log(h) - np.log(14)) + rng.normal(0, 1.2, h.shape))   # true objects: bigger = easier
det = score >= t
all_views = det.all(axis=1)                                    # a person is safe only if blurred in EVERY view
tracked = det | (det.any(axis=1, keepdims=True) & (rng.random(det.shape) < 0.9))   # propagate between views
bx = pred + np.stack([-pad * w_, -pad * h_, pad * w_, pad * h_], 1)                # pad the blur box
```

Output:

```
13,333 physical objects x 3 views; 21% of views are under 16 px tall

threshold  recall/view  recall <16px  FP per frame  % frame blurred by FP
      0.7        0.670         0.137          0.02                0.003%
      0.5        0.796         0.321          0.09                0.015%
      0.3        0.889         0.556          0.31                0.047%
      0.1        0.971         0.868          1.19                0.188%
     0.05        0.990         0.953          1.94                0.307%

threshold 0.1: per-view recall 0.971
  objects blurred in ALL 3 views, per-frame detection only : 0.929
  ... with box propagation between neighbouring views       : 0.992
  objects never detected in any view (tracking cannot help) : 0.001

box padding vs leaked pixels (detected objects only)
 padding    objects with >5% of pixels unblurred   blur area vs true box
      0%                                  79.8%                  0.99x
     10%                                  27.1%                  1.43x
     20%                                   6.4%                  1.94x
     30%                                   1.6%                  2.54x
```

What it shows:

- **The default 0.5 threshold is badly wrong here**: it misses 20% of views and 68% of tiny ones. Dropping to 0.05 reaches 99% (95% of tiny ones) at a cost of about 2 false blurs per frame covering 0.3% of the image — cheap. Recall-first thresholds are the right call when misses are the expensive error.
- **Per-view recall overstates safety**: 97.1% per view becomes only 92.9% of people blurred in all 3 views. Propagating boxes between neighbouring views lifts it to 99.2%; only people missed in *every* view (0.1%) remain.
- **Found is not covered**: with exact boxes, 80% of detected objects still leak more than 5% of their pixels because the box is slightly off. 20% padding cuts that to 6.4% at about twice the blurred area — which is why padding is part of the blur step.

Common mistakes

Weak	Strong
"Train a detector, use threshold 0.5"	Recall-first threshold chosen from the cost of a miss vs a false blur; cascade to recover precision
Reports overall recall or mAP	Recall by size, country, lighting; object-level across views; pixel coverage
Treats each image independently	Tracks people across neighbouring panoramas and propagates boxes
Blurs the exact box	Pads boxes, uses an irreversible blur, handles tile edges and the 360° seam
Fails open when the pipeline errors	Fails closed: nothing unverified is published

Same design, other questions

- **Redacting documents or screenshots** (credit-card numbers, names, addresses) before sharing — detection of text regions + classification; the same recall-first threshold, padding and fail-closed rules.
- **Blurring bystanders in body-camera or dash-cam video** — video adds tracking over dozens of frames; the per-person, all-frames recall becomes even more important.

Self-check

1. Why is per-view recall a misleading metric here? → A person appears in several views and is exposed if any one of them misses; object-level recall across all views is the real measure.
2. Why pad the blur box? → Detected boxes are slightly misplaced; without padding, many detected faces still show part of the face.
3. What does "fail closed" mean in this pipeline? → If detection fails for an image, the image is not published.

TL;DR

Big picture	Object detection where a miss is a privacy breach: recall first, per person, per pixel
Model	Tiled, multi-scale one-stage detector at a low threshold → context classifier (cascade) → human review for uncertain cases
Views	Track across neighbouring panoramas and propagate boxes; per-view recall overstates safety
Blur step	Pad boxes, irreversible blur, merge across tile edges and the seam, fail closed
Measuring	Weekly audit samples → missed objects per 1,000 panoramas; reports and time-to-fix; no privacy A/B tests
Cost	GPU batch for new captures; re-processing the archive is a separate, large budget

---

18.8 Case study: event recommendation

An events platform (think Eventbrite or Meetup) lists concerts, workshops, meetups and classes created by organisers. Design the

> "events for you" recommendations on the home page and in a weekly email.

1. Clarify requirements

The recognition sentence: **"This is the recommendation funnel of 17.1, except every item is new, perishable and tied to a place and a time."** A video can collect views for years; an event is created, sells tickets for a few weeks, happens once and is gone. So there is almost never enough interaction history on an event to use collaborative filtering on its ID. The design has to score events by **what they are, where and when they are, and who else is going**.

Question	Assumption used below
Users	50 M registered, 10 M monthly active; many are occasional (a few events a year)
Catalogue	~1 M upcoming events at any time; ~50 K new per day; most live 2–6 weeks
Surfaces	Home page (online, ~200 ms) and a weekly email (batch)
Actions	Register (free RSVP or paid ticket); later, attend (check-in)
Constraints	Must be in the future, not sold out, reachable (distance, or online)
Business	Registrations and ticket revenue, and organisers' success — an event with zero registrations loses the organiser

It is a **two-sided marketplace** (like PYMK's two sides in 18.4, but between users and organisers): showing only already-popular events makes attendees happy and starves new organisers.

2. Frame it as an ML task

- **Input**: user (location, history, friends), context (time, device, surface), candidate event.
- **Output**: P(register | shown), optionally P(attend | registered); the ranking score is their product for free events (no-shows waste capacity) or P(register) × price for revenue-weighted surfaces.
- **Category**: **pointwise learning-to-rank** on a **content- and context-based** model. Item-ID embeddings are nearly useless because each item's life is short.
- **Hard constraints first**: time window, distance or online, capacity, age restrictions — filters, not features.

3. Data and labels

1. **Events**: title, description, category, organiser, venue location, start time, price, capacity and **remaining capacity**, online/in-person.
2. **Organisers**: past events, past registration and attendance rates, ratings, followers.
3. **Users**: home location (or current location with permission), past registrations and check-ins, categories browsed, organisers followed, stated interests at sign-up.
4. **Social graph** (if the platform has one, or imported with consent): who is friends with whom, and who has registered for what — shown to others only if those users allow it.
5. **Impression logs**: every event shown, position, surface, then registration within an observation window.

Labels:

- A registration within, say, 7 days of an impression is a positive. Many registrations happen days after the first view (people check calendars), so use a **delayed-label window** (18.1) rather than same-session clicks.
- **Attendance** is known only after the event date — weeks late — and only for events that check people in. Use it as a second head or as a sample weight, not as the only label.
- Point-in-time correctness is critical: features like "registrations so far" and "friends going" must be computed **as of the impression time**, not at training time when the event has already filled up. This is the most common leakage bug in this design.

4. Features

Group	Examples
User–event match	Topic affinity (user's category history vs event category; text-embedding similarity of description to the user's past events), price vs the user's usual spend
Place	**Distance** or travel time from user to venue; same city/neighbourhood; online flag
Time	Days until the event, weekday/weekend and hour vs the user's past attendance pattern, clash with events the user already registered for
Social	Friends registered (count, closeness), organiser followed by user or by friends
Event quality	Organiser's past attendance rate and rating, description completeness, photos
Momentum	Registrations in the last 24 h, **remaining capacity**, sell-out velocity (all as of impression time)
Context	Surface (home vs email), device, time of day

5. Model: baseline → advanced

Baseline: rules — upcoming events within 20 km, in categories the user has attended, sorted by registrations so far. It ships in days and it is a fair number to beat. Its weakness: "registrations so far" is zero for every new event, so the ranking is decided by events that are already popular (see the toy).

Main model — gradient-boosted trees on the features above. GBDT (e.g. XGBoost, Chen & Guestrin, 2016) handles this mix of distances, counts, prices and categorical fields well without heavy feature engineering, and it generalises to events it has never seen because it scores **features of the event**, not its ID. Research on event-based social networks took the same route: Macedo, Marinho & Santos (2015) combined content, social, location and time signals with learning-to-rank on Meetup data, precisely because collaborative filtering struggles when every event is new.

Candidate generation for 1 M events:

- **Geo + time index**: events within the user's radius (a geohash or S2 cell lookup — the cells of a hierarchical grid over the map) in the next N weeks, plus online events. This alone cuts 1 M to a few thousand.
- **Two-tower retrieval** (13.7) where the event tower uses **content only** (category, text embedding, organiser features, price) — so a brand-new event gets a vector the moment it is created. Retrieve a few hundred from the geo-filtered pool.
- **Social source**: events friends registered for; events by followed organisers.
- **Exploration source**: a small quota of new events from organisers with little history, so they can collect data at all (the cold-start boost of 17.1).

Advanced ranker: a deep model with the description's text embedding and multi-task heads (register, attend). Upgrade only when GBDT stops improving — the features, not the model, carry this design.

Re-ranking: spread categories and dates (not five pub quizzes on the same night), respect an organiser-exposure floor, drop events the user already registered for, and never show sold-out events.

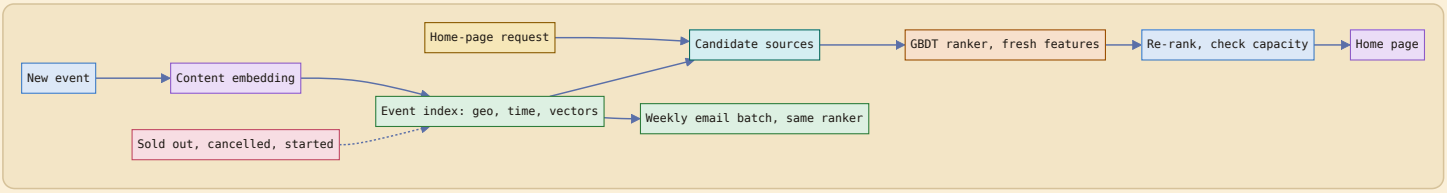
6. Offline metrics

- **Time-based split by event creation date**: train on events from weeks 1–4, test on events **created in week 5**. A random split puts the same event in train and test and flatters any model that memorises IDs.
- **AUC / log loss** of P(register); **hit@k / NDCG@k** per user on the events actually shown.
- **Slices**: new users, new organisers, events with zero registrations at impression time, online vs in-person, small towns vs big cities.
- **Calibration** if the weekly email uses a threshold ("send only if at least 3 events score above p").

7. Online metrics and A/B testing

- **Primary**: registrations per active user per week; for paid events, ticket revenue.
- **Quality**: **attendance rate** of recommended registrations (registrations that don't turn up are weak wins), post-event ratings.
- **Marketplace health (guardrails)**: share of events that get at least one registration, organiser retention, concentration of registrations on the top 1% of events.
- **Email guardrails**: unsubscribe and spam-report rates.
- Events are local, so randomise **by user** but check effects by city; an experiment that only works in the three largest cities is not a win everywhere.

8. Serving architecture



Candidate sources = geo + time filter, content-only two-tower ANN, events friends registered for, followed organisers, and an exploration quota. The ranker's momentum features (registrations in the last hour, remaining capacity) come from a stream processor, computed as of now; impressions, registrations and check-ins are logged for training.

- **Event lifecycle drives the index**: an event is embedded and indexed **when it is created** (minutes, not nightly — a new event matters most in its first days), and removed when it sells out, is cancelled or starts. Remaining capacity is **hydrated at render time**, the same rule as sold listings in 18.6.
- **Streaming features**: registrations in the last hour and remaining capacity come from a stream processor; they change fastest just when an event is most in demand.
- **Email** runs the same ranker in batch over active users once a week — one model, two serving modes, as in 18.5.

9. Scale numbers

Quantity	Arithmetic	Result
Home-page QPS	10 M MAU × ~10 visits/month ÷ (30 × 86,400 s)	≈ 40/s average; ~150/s at a weekend-evening peak
Ranker load	150 QPS × 500 candidates	75 K scores/s — small for a GBDT
Event vectors	1 M × 128 dims × 4 B	512 MB — fits in memory per region, no sharding needed
Weekly email	10 M users × 500 candidates	5 B scores per week in batch — a few hours on a modest cluster
New-event embedding	50 K/day	< 1/s — trivial

This is a small system by 17.1's standards: the difficulty is **correctness** (freshness, leakage, capacity), not throughput.

10. Monitoring and failure modes

- **Seasonality**: holidays, summer festivals, and the week before a long weekend change behaviour sharply; compare against the same week last year, not last week.
- **New-city launches**: no local history at all — fall back to category popularity from similar cities and lean on the exploration quota.
- **Feature freshness**: if the capacity stream lags, sold-out events get recommended; alert on stream lag.

Failure	Fallback / effect
Ranker down	Rules baseline (nearby + category + date), still capacity-filtered
Two-tower / ANN down	Geo + time filter + followed organisers; quality drops most for users with sparse history
Capacity stream stale	Render-time check against the ticketing service; never sell a sold-out seat
Leakage in training ("registrations so far" computed late)	Offline AUC jumps suspiciously, online gains vanish → point-in-time feature store (16.4)

11. Interview follow-ups

- **"Why not matrix factorisation or item-ID embeddings?"** → Each event's life is a few weeks and most get few registrations; its ID has too little history to learn a vector. Model users and events through **features** that transfer across events: category, organiser, place, time, price, text.
- **"A brand-new user opens the app."** → Location plus the interests picked at sign-up; popular events nearby this week; learn from the first clicks within the session.
- **"Should we show 'your friend is going'?"** → Only if the friend's privacy setting allows it; when it does, it is one of the strongest signals (the toy: 7.6% vs 1.7% register rate), but it only exists for a small share of impressions.
- **"How do you treat online events?"** → Distance becomes meaningless: set a separate flag, use time zone instead of distance, and give them their own candidate source so they neither crowd out local events nor get buried.

Toy — every item is cold (`code/event_rec_toy.py`)

A synthetic city of 4,000 users with a home location, topic tastes and price sensitivity; friends live nearby. Each week brings 300 **new** events. True P(register) rises with topic match, falls with distance and price, rises with friends already registered and host quality. We train on four weeks of impressions and test on week 5's events, none of which appear in training.

```
def true_p(u, ev, j, n_friends):
    dist = np.linalg.norm(u_loc[u] - ev["loc"][j], axis=-1)
    z = (-3.0 + 6.0 * u_topic[u, ev["topic"][j]] - 0.18 * dist + 1.2 * np.log1p(n_friends)
         - 0.03 * u_price[u] * ev["price"][j] + 0.5 * ev["host"][j])
    return sig(z), dist

for drop in [None, "distance_km", "friends_registered", "user_topic_affinity"]:     # ablations
    keep = [i for i, c in enumerate(COLS) if c != drop]
    m = HistGradientBoostingClassifier(max_iter=200, random_state=0).fit(Xtr[:, keep], ytr)
```

Output:

```
train: 240,000 impressions on 1200 past events, register rate 2.0%
test : 60,000 impressions on 300 NEW events (0 overlap with training IDs)

4.3% of test impressions have >=1 friend already registered (register rate 7.6% vs 1.7% without)

model                                AUC   hit@3  AUC, friend-going slice
random / item-ID CF (all cold)     0.496   0.204
popularity (early registrations)   0.679   0.430
GBDT, all features                 0.894   0.797                   0.845
GBDT without distance_km           0.837   0.714                   0.791
GBDT without friends_registered    0.892   0.785                   0.844
GBDT without user_topic_affinity   0.810   0.650                   0.676
```

What it shows:

- **Item-ID collaborative filtering has nothing to work with**: every test event is new, so it is no better than random (AUC ≈ 0.5).
- **Popularity helps but needs a warm-up**: it only works because the toy lets early registrations accumulate first; at the moment an event is posted it is zero for everyone.
- **The feature-based GBDT generalises to unseen events** (AUC 0.894, hit@3 0.797) because it scores *what, where, when and who*, not *which ID*.
- **Ablations price each feature group**: removing topic affinity costs the most (0.894 → 0.810), distance next (→ 0.837). The social feature is powerful where it exists (about 4.5× the register rate) but present on only 4% of impressions, so it barely moves the overall AUC — measure such features on their own slice, not only in aggregate.

Common mistakes

Weak	Strong
Uses item-ID collaborative filtering or MF	Content- and context-based model that scores events it has never seen
Ignores location and time	Distance, travel time and date fit as core features; time window and reachability as filters
Computes "registrations so far" at training time	Point-in-time features as of the impression; leakage checked
Ranks by popularity	Exploration quota and organiser-exposure floor for a healthy two-sided marketplace
Counts registrations only	Attendance, ratings and organiser success as quality and guardrail metrics

Same design, other questions

- **Restaurant or class booking recommendations** — same place-and-time constraints; items are longer-lived, so collaborative signals help more.
- **Job recommendations** — perishable postings (weeks), location and qualifications as filters, two-sided (the employer must want the candidate).
- **Live-stream recommendations** — the event is happening *now*: momentum (viewers joining per minute) dominates and the "index" churns every few minutes.

Self-check

1. Why is item-ID collaborative filtering weak for events? → Each event lives a few weeks and collects little history; there is no time to learn an ID vector.
2. What leakage bug is typical here? → Momentum features (registrations so far, remaining capacity) computed at training time instead of as of the impression.
3. Why an exploration quota? → New organisers' events start with no signals; without guaranteed exposure they never collect any, and the marketplace loses organisers.

TL;DR

Big picture	17.1's funnel where every item is new, perishable, and tied to a place and a time
Candidates	Geo + time filter, content-only two-tower, friends going, followed organisers, exploration
Model	Rules → GBDT on user–event, place, time, social, organiser and momentum features → deep multi-task (register, attend)
Traps	Point-in-time features; delayed registrations; attendance known weeks later
Marketplace	Organiser-exposure floor, events-with-≥1-registration guardrail
Serving	Index on creation, drop on sell-out/start, hydrate capacity at render; same ranker for home and weekly email

---

18.9 Case study: similar listings from session embeddings

> A vacation-rental marketplace (think Airbnb) shows a "similar listings" carousel on every listing page. A guest looking at a flat in Lisbon for 12–15 June should see good alternatives. Design it.

1. Clarify requirements

The recognition sentence: **"This is item-to-item retrieval: the anchor is the listing on screen, and 'similar' should mean *what guests consider as alternatives*, not just matching attributes."** Two listings can share a price, size and neighbourhood and still attract different guests (a design loft vs a family flat); two listings with different attributes can be natural substitutes. Browsing sessions record those judgements for free: guests click through several listings they are comparing. Learn from that — the item2vec idea of 13.6 applied to listings.

Question	Assumption used below
Listings	5 M active, spread over ~50 K markets (a **market** = a destination: a city or region)
Traffic	50 M listing-page views/day
Latency	~100 ms for the carousel (it loads alongside the page)
Hard constraints	Same destination; available for the guest's dates; fits the guest count
Goal	Bookings — ideally the guest books *something*, even if not the anchor
Out of scope	The main search ranking (a sibling system; see "Same design" below)

Watch the goal: a carousel that steals clicks from a listing the guest would have booked anyway is not a win. The launch metric is **bookings on the whole site**, not carousel clicks.

2. Frame it as an ML task

- **Input**: anchor listing; guest context (dates, guests, recent clicks in this session).
- **Output**: a ranked list of ~12 listings.
- **Category**: **representation learning** (listing embeddings where distance = "considered together") + nearest-neighbour **retrieval** + a small **ranker**.
- **Objective for the embedding**: listings clicked close together in the same session should be close; listings eventually booked should pull the session's clicks toward them.

3. Data and labels

1. **Click sessions**: the sequence of listing pages a guest opens during one search. Clean them: count only clicks with a meaningful dwell time (e.g. ≥ 30 s, so mis-taps don't count), and split a user's activity into sessions at gaps of 30+ minutes.
2. **Bookings**: which listing each session ended in (a minority of sessions book).
3. **Listing attributes**: location, type (entire home, private room), price, capacity, bedrooms, amenities, photos, review score, host responsiveness.
4. **Availability calendar**: which nights are free — changes constantly.
5. **Carousel impression logs** (once the carousel exists): shown, clicked, booked, position.

Labels: there are no labelled "similar pairs". The embedding learns from **co-occurrence** in sessions (self-supervised, like word2vec learning from sentences). The later ranker learns from carousel impressions → clicks → bookings.

4. Features

Stage	Features
Retrieval	Cosine similarity of listing embeddings (anchor vs candidate)
Filters	Same market, available for the dates, capacity ≥ guests, not the anchor itself, not hidden by the host
Ranker	Embedding similarity, **similarity to the guest's session** (mean of recently clicked listings' vectors), distance to the anchor, price ratio to the anchor, type/capacity match, review score and count, host response rate, instant-book, listing age

Retrieval defines "alternatives"; the ranker decides which alternatives are worth booking for this guest and these dates.

5. Model: baseline → advanced

Baseline — attribute nearest neighbours. Same market, same type, capacity ≥ guests, price within ±30%, sort by distance to the anchor. Explainable, zero training, and good enough to launch the carousel and start logging. It misses intangible taste — style, vibe, "feels like a hotel" vs "feels like home".

Upgrade 1 — session-based listing embeddings (skip-gram with negative sampling). Treat each click session as a sentence and each listing as a word, and train word2vec's skip-gram (Mikolov et al., 2013; item2vec, Barkan & Koenigstein, 2016; 13.6): for each clicked listing, predict the listings clicked just before and after it, contrasting them with random **negative** listings. Grbovic & Cheng (2018) described doing this at Airbnb for similar-listing recommendations and search personalisation, with two adaptations that fit this problem well:

- **The booked listing as global context**: in sessions that end in a booking, every clicked listing is also trained to predict the booked one — it pulls embeddings toward what guests actually *book*, not just what they look at.
- **Negatives from the same market**: sessions almost never leave one destination, so random negatives are nearly always from other markets. The model can then minimise the loss by learning *"Lisbon vs Paris"* and little else. Adding negatives drawn from the anchor's own market forces it to learn distinctions *inside* the market — which is all the carousel needs, since the market is a hard filter anyway (see the toy).

Cold start — new listings have no clicks. Give a new listing the average vector of a few existing listings that are nearby, of the same type and in the same price bucket (the approach Grbovic & Cheng describe). Longer term, train a **content encoder** (photos, text, attributes → vector) to predict the session embedding, so every new listing gets a vector the moment it is published.

Upgrade 2 — a ranker on top. GBDT on the ranker features, trained on carousel impressions with **booking** as the main label (clicks as an auxiliary one). This is where dates, price and quality enter — a similar listing that is double the price or has 3-star reviews should not lead.

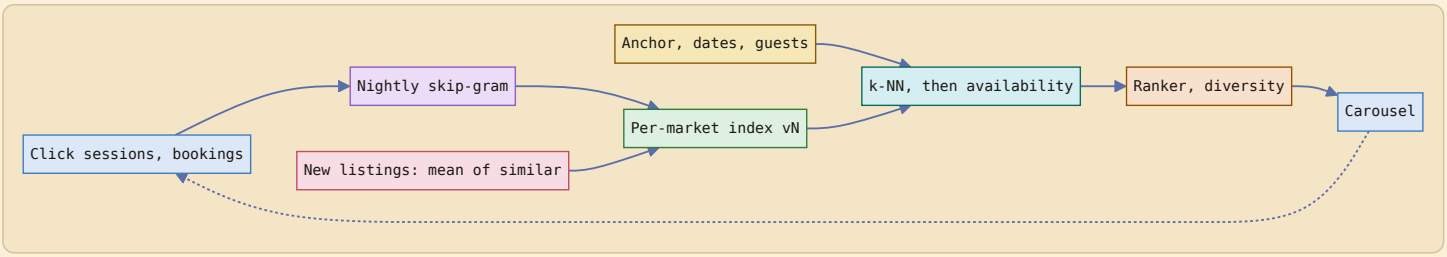
6. Offline metrics

- **Next-click hit rate@K** on held-out sessions: from a clicked listing, is the next listing the guest clicked among the K most similar (after the same filters serving will apply)?
- **Booking hit rate@K**: from the session's first click, is the eventually booked listing among the K most similar? Closer to the business goal.
- **Sanity checks by eye and by numbers**: nearest neighbours should share the market (unfiltered), usually the type and a similar price; plot a 2-D projection per city. Embeddings that pass metrics but fail the eye test usually have a data bug.
- **Slices**: big vs small markets, cold-start listings, luxury and budget ends.

7. Online metrics and A/B testing

- **Primary**: **site-wide bookings per visitor** (captures cannibalisation).
- **Secondary**: carousel click-through, carousel-attributed bookings.
- **Guardrails**: bookings of the anchor listing (a big drop means the carousel distracts), host-side exposure concentration (a few listings appearing in every carousel), latency, empty-carousel rate.
- Randomise by **guest**; for marketplace-wide effects (hosts losing exposure), check by market as well.

8. Serving architecture



The skip-gram uses the booked listing as global context and same-market negatives; k-NN fetches ~200 neighbours in the anchor's market, the calendar service removes those unavailable for the dates or too small for the guests, and a GBDT ranker (similarity, session similarity, price ratio, quality) plus a host-diversity rule picks ~12.

- **Per-market indexes**: similarity search never crosses a market, so build one small index per market. Most markets hold a few hundred to a few thousand listings — exact (brute-force) search is fast enough; only the largest need ANN (13.9).
- **Availability is the expensive filter**: fetch 200 neighbours, then check the calendar service for the dates, then rank. In peak season many neighbours are booked, so fetch more when the filter empties the list — the filtered-ANN issue of 18.6.
- **Versioning**: a retrained skip-gram model produces a *new, unrelated* vector space (it can come out rotated). Swap vectors and index together, atomically (the 18.6 rule), or warm-start each night's training from the previous vectors to keep neighbours stable.
- **Precompute for popular anchors**: the top-200 neighbour list per listing can be computed nightly and cached; only availability filtering and ranking run per request.

9. Scale numbers

Quantity	Arithmetic	Result
Carousel QPS	50 M page views/day ÷ 86,400	≈ 580/s average, ~1,700/s at a 3× peak
Vectors	5 M × 32 dims × 4 B	640 MB — the whole catalogue fits in memory on each server
Largest market	e.g. 60 K listings × 32 dims	~2 M multiply-adds per exact k-NN query — well under a millisecond
Training pairs	e.g. 300 M sessions/month × 8 clicks × 4 context listings	~10 B pairs/month — a few hours per nightly run on a small cluster, or incremental training
Availability checks	1,700/s × 200 candidates	340 K listing-date lookups/s → cache calendars; precompute "available next 30 days" bitmaps

Small vectors (32 dimensions) are enough here: the space only needs to separate listings *within* a market.

10. Monitoring and failure modes

- **Neighbour stability**: overlap of each listing's top-10 neighbours between yesterday's and today's model; a sudden drop means a training bug or a rotated space served with an old index.
- **Cold-start share**: the fraction of carousel slots filled by listings with averaged (not learned) vectors, and their booking rate.
- **Seasonality**: summer vs winter sessions differ (beach houses vs ski chalets); train on a recent window, and watch per-market hit rate.

Failure	Fallback / effect
Embedding job fails	Keep serving yesterday's vectors and index (both, as a pair)
Calendar service slow	Serve precomputed availability bitmaps; confirm at booking
Empty carousel after filters (peak season)	Widen: more neighbours, then attribute nearest neighbours, then nearby markets
Model–index version skew	Atomic promotion; per-request version check

11. Interview follow-ups

- **"Why not just use attributes?"** → Attributes miss taste. Sessions capture what guests treat as substitutes — style, neighbourhood feel, the kind of trip — without anyone labelling it. Attributes remain as filters, ranker features and the cold-start fallback.
- **"Why are same-market negatives needed if you filter by market anyway?"** → Because the filter is exactly why they're needed: with random negatives, the model spends its capacity separating markets (which the filter already does) instead of ordering listings inside one.
- **"How would you personalise the carousel?"** → Add the guest's **in-session** vector (mean of recently clicked listings) as a ranker feature; Grbovic & Cheng used similar in-session embedding features for search ranking. Long-term preferences (a family that always books whole homes) come from the guest's booking history.
- **"How does this relate to search ranking?"** → Same embeddings, different anchor: search ranks listings for a query and dates; the carousel ranks them for an anchor listing. Share the embedding pipeline, keep separate rankers.

Toy — skip-gram listing vectors, market negatives, cold start (`code/listing_embeddings_toy.py`)

20 synthetic cities with 100 listings each; each listing has hidden attributes (price tier, style, size). A session stays in one city and clicks 8 listings close to the guest's hidden taste; 30% end in a booking. We train skip-gram (window 2, 5 negatives) three ways and evaluate on held-out sessions, applying the same-city filter serving would apply.

```
n = torch.randint(0, N, (len(b), neg))                          # catalogue-wide negatives
if same_city_neg:                                               # half the negatives from a's own city
    local = city_t[a][:, None] * PER + torch.randint(0, PER, (len(b), neg // 2 + 1))
    n = torch.cat([n[:, : neg - local.shape[1]], local], 1)
loss = (-logsigmoid((va * emb_out(p)).sum(1)).mean()               # pull real co-clicks together
        - logsigmoid(-(va[:, None] * emb_out(n)).sum(2)).mean())   # push negatives apart

# cold start: a new listing = mean vector of the 3 closest known listings in its city by attributes
near = cand[np.argsort(((obs[cand] - obs[i]) ** 2).sum(1))[:3]]
E[i] = E[near].mean(0)
```

Output:

```
2000 listings in 20 cities; 20,000 training sessions, 2,000 held out

                                          top-10 in          after same-city filter
variant                                   same city next-click HR@10  booking HR@10
random vectors                                0.054            0.100          0.075
a) skip-gram, catalogue-wide negatives        1.000            0.337          0.310
b) + same-city negatives                      0.957            0.380          0.387
c) + booked listing as global context         0.969            0.378          0.383

cold start: 200 listings treated as new (no clicks); next-click HR@10 starting from them
  random vector                        0.089
  mean of 3 similar nearby listings    0.326
  (vector learned from real clicks     0.353)
```

What it shows:

- **Sessions alone learn geography perfectly**: with catalogue-wide negatives, 100% of each listing's top-10 neighbours are in its own city — nobody told the model about cities.
- **Same-city negatives trade that for better within-city order**: next-click hit rate 0.337 → 0.380 and booking hit rate 0.310 → 0.387 after the city filter. A few neighbours now come from other cities (95.7% same city), which doesn't matter because serving filters by market — it was capacity spent on the wrong distinction.
- **The booked-listing context adds nothing here** (0.378 vs 0.380) — honestly, because in this toy the booked listing is a random one of the clicked listings and carries no extra signal. In real logs bookings differ from clicks (guests click attractive photos and book good value), which is why the technique exists; test it on your data rather than assuming it helps.
- **Cold start by averaging** recovers most of the quality: 0.326 vs 0.353 for a vector learned from real clicks, and far above a random vector (0.089).

Common mistakes

Weak	Strong
Defines similarity by attributes only	Learns "considered together" from sessions; attributes as filters, features and cold-start fallback
Random negatives from the whole catalogue	Adds same-market negatives so the space orders listings inside a market
Ignores new listings	Averaged neighbour vectors on day one, a content encoder longer term
Optimises carousel clicks	Site-wide bookings as the launch metric; anchor bookings as a cannibalisation guardrail
Forgets dates	Availability filter per request; fetch more neighbours in peak season
Retrains embeddings like a normal model deploy	Versions vectors and index together; monitors neighbour stability

Same design, other questions

- **"Customers also viewed" in e-commerce** — the same session embeddings; categories play the role of markets, and stock replaces availability.
- **Next-song or next-video "up next"** — sessions become sequences to *predict* (18.5's radio), not just co-occurrence to embed.
- **Hotel search personalisation** — the same listing vectors as ranker features, compared with the guest's in-session vector.

Self-check

1. What plays the role of a "sentence" and a "word" here? → A click session; a listing.
2. Why do random negatives waste capacity in this setting? → Sessions stay in one market, so random negatives are almost always from another market; the model learns market identity, which the market filter already provides.
3. How does a brand-new listing get a vector? → The mean of the vectors of a few nearby listings of the same type and price bucket (later, a content encoder).

TL;DR

Big picture	Item-to-item retrieval where "similar" = what guests consider together, learned from click sessions
Model	Attribute k-NN baseline → skip-gram listing embeddings (booked-listing context, same-market negatives) → GBDT ranker on bookings
Cold start	Average vectors of similar nearby listings; content encoder later
Serving	Per-market index, availability filter per request, precomputed neighbour lists, model + index versioned together
Measuring	Next-click and booking hit rate offline; site-wide bookings online with an anchor-cannibalisation guardrail

---

Comparison across all thirteen case studies (Chapters 17–18)

Case study	Task type	Key features	Model (baseline → advanced)	Main metric (offline / online)	Hardest challenge
Video recommendations (17.1)	Retrieval + learning-to-rank	Watch history, item embeddings, user×item similarity, session state	Popularity → two-tower ANN + co-occurrence → GBDT / deep multi-objective ranker	Recall@K, AUC / watch time, return rate	Cold start and position bias; clickbait proxy trap
Feed ranking (17.2)	Multi-task ranking over a social candidate pool	Viewer–author closeness, content embeddings, live engagement counts, integrity scores	Chronological / P(like) LR → shared-body multi-task net + value formula	Per-head AUC + calibration / sessions per user or worthwhileness, with hide/report guardrails	Balancing many goals without rewarding bait; ~87 K ranking calls/s
Search ranking (17.3)	Query understanding + learning-to-rank	Query–item match (BM25, embedding similarity, attributes), item quality, capped personalisation	BM25 + popularity → hybrid retrieval, L1 GBDT → L2 LambdaMART/deep, cross-encoder on top-k	NDCG@k / session success, revenue per search, reformulation	Position-biased clicks; revenue vs relevance
Text-to-video search (17.4)	Cross-modal retrieval + learning-to-rank	Per-channel match (title, ASR transcript, OCR, visual), channel availability, satisfied-watch rate, freshness	BM25 on metadata → + ASR transcripts → + CLIP-style text–frame embeddings (video + segment vectors) → learned fusion ranker	Recall@K per channel, NDCG@10 / searches ending in a satisfied watch	Indexing cost (frames, ASR per upload); missing or misleading text; moments inside long videos
Ad click prediction (18.1)	Binary classification with calibrated probabilities feeding an auction	Sparse user/context/ad IDs, crosses, advertiser history, streaming counters	Logistic regression → GBDT+LR / deep CTR, pCTR+pCVR heads, calibration layer	Log loss, calibration by segment / RPM, advertiser ROI	Calibration per segment; delayed conversions; 100 K QPS in 50–100 ms
Fraud detection (18.2)	Imbalanced binary classification + cost-based action policy	Velocity counters, impossible travel, device and graph links	Rules + LR → GBDT + graph features (GNN for rings)	PR-AUC, expected $ loss / fraud bps, false-decline rate	Adaptive adversary; weeks-late and censored labels
Harmful content (18.3)	Multi-label multimodal classification in a cost cascade	Text/image/audio/OCR embeddings, hashes, account history, share velocity, weighted reports	Hashes + rules → fast per-modality classifiers → heavy multimodal/LLM → humans	Per-category precision/recall / prevalence of violating views, false-removal & overturn rates	Harm both ways at huge scale; reviewer capacity; evasion
People You May Know (18.4)	Link prediction + two-sided ranking	Mutual count, Adamic-Adar, shared affiliations, accept history, graph embeddings	Mutual-count sort → two-head P(send)×P(accept) model over FoF+affiliation+contacts+ANN candidates	Recall of future edges / accepted invites per impression, "don't know" reports	Two-sided objective; privacy fail-closed; graph-scale batch; rich-get-richer
Music discovery (18.5)	Batch recommendation + online sequential recommendation	Collaborative vectors, audio embeddings, metadata/playlist text, session context	Cohort popularity → MF/two-tower + audio encoder → sequence model (GRU4Rec/SASRec) for radio	Recall/Hit@K, diversity / saves, skip rate, long-term repeat listening	New-track cold start; skip-responsive radio; diversity and artist fairness
Visual search (18.6)	Representation learning + ANN retrieval + ranking	Image and text embeddings, price proximity, condition, seller quality, live status	Pretrained CLIP-style → contrastive fine-tune with hard negatives + light ranker	Judged NDCG per category / contact-seller rate, sold-item impressions	Index churn (sold = deleted), encoder upgrades needing re-embeds
Privacy blurring (18.7)	Object detection (batch), recall-first	Tiled multi-scale panoramas, neighbouring views, geometry priors	Off-the-shelf detectors → fine-tuned one-stage detector at a low threshold → context classifier (cascade) + multi-view tracking + human review	Recall by size/region, object-level recall, pixel coverage / audited misses per 1,000 panoramas, report rate	Misses are privacy breaches; tiny faces; per-person recall across views; no A/B tests
Event recommendation (18.8)	Content- and context-based pointwise ranking	Topic affinity, distance, date fit, friends going, organiser quality, momentum (as of impression)	Nearby-popular rules → GBDT on features → content two-tower retrieval + deep multi-task (register, attend)	AUC, hit@k on newly created events / registrations per user, attendance, events with ≥1 registration	Every item is cold and perishable; point-in-time leakage; two-sided marketplace
Similar listings (18.9)	Item-to-item representation learning + ranking	Session-embedding similarity, in-session vector, price ratio,	Attribute k-NN → skip-gram on click sessions (booked context, same-market negatives) → GBDT ranker on bookings	Next-click / booking hit rate / site-wide bookings, anchor cannibalisation	Learning 'substitutes' without labels; cold-start

Case study	Task type	Key features	Model (baseline → advanced)	Main metric (offline / online)	Hardest challenge
		distance, availability, quality			listings; availability filtering

Patterns that repeat — learn these once, reuse everywhere:

- **Funnels and cascades**: cheap high-recall stage → expensive precise stage (17.1–17.4, 18.3, 18.5–18.9).
- **Rules outside the model**: re-rank rules, integrity layers, eligibility filters, fraud rules, action ladders, availability and capacity filters — so policy changes don't need retraining.
- **Biased and late labels**: position bias (search, recs, visual grids), delayed conversions (ads, PYMK acceptance, event registrations), weeks-late chargebacks and attendance, censored blocks, selective appeals.
- **Two-sided or multi-goal objectives**: value formulas (feeds), bid × pCTR (ads), P(send) × P(accept) (PYMK), organiser exposure (events), costs of both error types (fraud, moderation, blurring).
- **Fail-safe defaults decided in advance**: yesterday's index, light-ranker order, segment CTR tables, risk-based timeout policy, privacy fail-closed (PYMK, blurring), genre station.
- **Perishable items and render-time checks**: sold listings, sold-out events, unavailable dates, taken-down videos — filter in the index *and* re-check when the page is built (17.4, 18.6, 18.8, 18.9).

Chapter TL;DR

- **Ads**: the probability is priced — calibrate, check per segment, handle delayed conversions, pace budgets.
- **Fraud**: accuracy is meaningless at 0.2%; velocity + graph features, rules + GBDT, two cost-based thresholds; labels late and censored.
- **Harmful content**: flag rate = compute bill, review threshold = headcount; measure prevalence of *views* per category; action ladder; reach-aware re-scoring.
- **PYMK**: four candidate sources ranked by P(send) × P(accept); privacy fails closed; streamed dismissals.
- **Music**: one funnel, two serving modes; audio embeddings make new tracks findable; radio is session-driven.
- **Visual search**: shared image-text space; the index is perishable — embed at creation, tombstone on sale, version model and index together.
- **Privacy blurring**: recall first, per person across views and per pixel; low threshold + cascade + tracking + padding; audits and reports, fail closed.
- **Events**: every item is new — score by content, place, time and friends with GBDT; point-in-time features; keep organisers alive with exploration.
- **Similar listings**: skip-gram on click sessions with booked-listing context and same-market negatives; averaged vectors for new listings; launch on site-wide bookings.
- Across all thirteen: name the shape, tie the ML objective to the business goal, be honest about labels, keep policy outside the model, decide failure behaviour in advance.

Chapter 19 — Responsible ML: Fairness, Explainability, Privacy and Security

Every chapter so far asked "is the model accurate?". This one asks the questions that decide whether a model *should ship*: Is it fair to every group it touches? Can we explain its decisions? Does it leak the private data it learned from? Can someone fool it, poison it, or steal it? Which laws apply?

These are not side topics. A credit model that quietly rejects qualified applicants from one neighbourhood, or a medical model that reveals who was in its training set, is *broken* however good its AUC. Such failures have measurable symptoms and concrete fixes, and this chapter treats them as engineering problems.

What you'll learn

1. Where bias enters an ML system, from historical data to feedback loops.
2. The main fairness definitions, why they cannot all hold at once, and how to measure them in code.
3. Mitigation before, during and after training: reweighing, fairness penalties, group thresholds.
4. Explainability: permutation importance, partial dependence, SHAP (computed exactly) and LIME.
5. Privacy: why "anonymised" data often isn't, k-anonymity, differential privacy, federated learning.
6. Security: adversarial examples (FGSM), poisoning, model extraction and inversion, membership inference, generative-AI misuse.
7. Model cards, datasheets, regulation (GDPR, EU AI Act, US rules), real incidents and a pre-launch checklist.

All code is in `code/` (one script per section), figures in `img/`. Every output shown is real (Python 3.14, scikit-learn 1.9, PyTorch 2.14, NumPy 2.5). The optional libraries `shap`, `lime` and `fairlearn` are not installed in the book environment, so their snippets are marked **(not run)**; each has a runnable from-scratch equivalent.

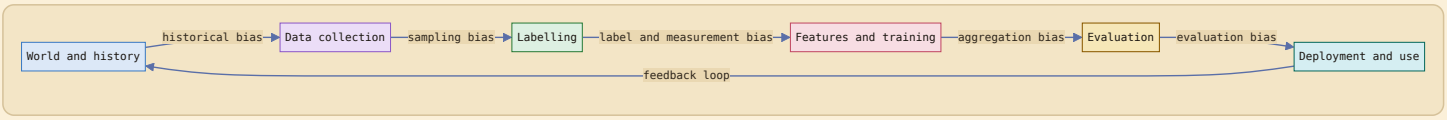


Figure — bias can enter at every stage of the ML lifecycle, and deployment feeds back into the next round of data.

19.1 Where bias comes from

What it is

In this chapter **bias** means a *systematic* difference in how a model treats or performs for different groups of people that is not justified by the task. (This is different from the statistical "bias" in the bias–variance trade-off of Chapter 6.) A **protected** or **sensitive attribute** is a characteristic that anti-discrimination law or ethics says should not drive decisions — e.g. race, sex, age, religion, disability. A **group** is the set of people sharing a value of that attribute.

Analogy: a model is an apprentice who learns only by watching past decisions. If those decisions were unfair, or it only ever watched one kind of customer, it reproduces the unfairness — at scale and with an air of objectivity.

Why it matters

Data records human choices and who was measured. Three mechanisms matter most:

1. **Data encodes past decisions.** Hiring, lending and arrest records carry the inequalities of their time: a résumé screener trained on "who we hired" learns "who we used to prefer", not "who does the job well".
2. **Some groups are thin in the data.** The model sees few of their examples, and the training loss barely notices errors on them. Classic result: the *Gender Shades* study (Buolamwini & Gebru, 2018) found commercial gender classifiers erred on up to 34.7% of darker-skinned women versus at most 0.8% of lighter-skinned men.
3. **Stereotyped associations get learned and amplified.** Word embeddings trained on web text place "nurse" near "she" and "programmer" near "he" (Bolukbasi et al., 2016). Anything built on top — job recommendations, ad targeting, autocomplete — inherits the association.

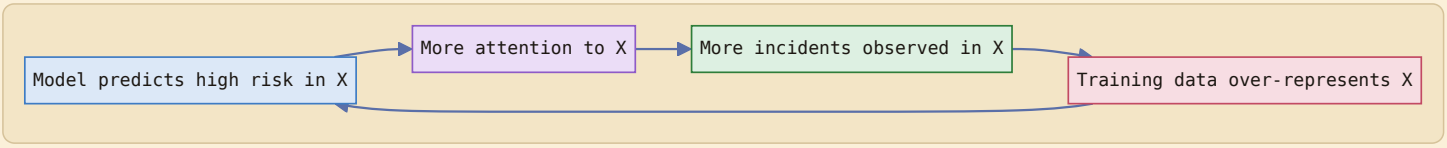
The harm (denied loans, misidentification, worse service) tends to land on people who were already disadvantaged; for the organisation it means legal exposure and lost trust.

How it works — a taxonomy of sources

(Names follow Suresh & Guttag's widely used framework.)

Source	What goes wrong	Example
Historical bias	The world itself was unequal; even perfect data reproduces it	Past hiring favoured men, so "good hire" labels skew male
Sampling / representation bias	Some groups are under-sampled or missing	Face datasets dominated by lighter-skinned faces; a voice assistant trained mostly on one accent
Measurement bias	A feature or proxy is measured differently, or means different things, across groups	"Number of prior arrests" reflects policing intensity, not just behaviour
Label bias	The target is a flawed proxy, or is recorded unevenly	Health *cost* used as a stand-in for health *need* (Obermeyer et al., 2019): less was spent on Black patients at the same level of illness, so the algorithm under-rated their need
Aggregation bias	One model for groups whose feature–outcome relationship differs	A single HbA1c threshold across ethnicities with different typical values
Evaluation bias	The benchmark doesn't represent the users; only an overall metric is reported	95% overall accuracy hiding 70% accuracy on a minority group
Deployment bias	The model is used for a purpose or population it was not built for	A risk score designed to target help being used to set punishment
Feedback loops	The model's decisions shape the next training data	Predictive policing (below)

Feedback loops. Predictive policing is the textbook case (Lum & Isaac, 2016, simulated it on Oakland drug-crime data): the model flags district X, partly because X was patrolled more in the past; more officers go there, so more incidents are *recorded* there; the next training set shows even more crime in X. Nothing in the loop checks whether X really has more crime — the data only reflects where you looked. The same pattern appears in lending (people never approved never get a chance to show they would repay — the **selective labels** problem) and in recommenders (items never shown never get clicks).



Representative data is the first defence. A model only works for people it has seen: coverage across ages, accents, skin tones, dialects, devices and regions gives it examples of every kind of real user and of rare cases.

Common mistakes

- Assuming a model is neutral because the algorithm is "just maths" — the data carries the bias.
- Checking only overall accuracy; per-group performance is where bias shows.
- Assuming more data fixes it — more of the same skewed data makes the model *more confident* in the skew.
- Ignoring the label: if the target itself is biased (arrests, past hires, cost), no model choice rescues it.

In the interview

30-second answer: "Bias usually comes from data, not the algorithm: past decisions, thin coverage of some groups, proxy labels or features, and feedback loops. I audit per-group coverage and metrics, question whether the label measures what we care about, and watch for loops after launch."

- *How do you break a feedback loop?* → Keep a small randomised exploration slice, log what you *didn't* show or approve, and monitor whether input distributions drift towards the model's own predictions.

Self-check

1. A hiring model is trained on which applicants managers promoted. Which bias is most likely? → Historical and label bias: "promoted" encodes past managers' preferences.
2. Your speech model works well overall but poorly for Scottish accents. Which bias? → Sampling/representation bias (and evaluation bias if nobody measured per-accent accuracy).
3. Why can't you fix a feedback loop just by retraining more often? → Retraining on the loop's own output reinforces it; you need exploration or external ground truth.

TL;DR

Bias enters at every stage, from history to feedback loops. Representative data and per-group evaluation come first; no algorithm fixes a biased label.

19.2 Fairness definitions and the impossibility trade-off

What it is

A **fairness criterion** is a precise statistical condition on a model's predictions across groups. Notation used below: **A** is the group, **Y** the true outcome (1 = good, e.g. repays), **Ŷ** the decision (1 = approve), **S** the model's score. TPR (true-positive rate, recall), FPR (false-positive rate), FNR = 1 − TPR and PPV (precision) are the confusion-matrix rates from Chapter 6, computed separately per group. Most criteria fall into three families (Barocas, Hardt & Narayanan's *Fairness and Machine Learning*):

- **Independence** — the decision doesn't depend on the group.
- **Separation** — errors don't depend on the group, given the true outcome.
- **Sufficiency** — a given score means the same thing in every group.

Criterion	Family	Condition (equal across groups)	Plain meaning
Demographic (statistical) parity	Independence	P(Ŷ=1 | A=a)	Same approval rate in every group
Disparate impact ratio	Independence	P(Ŷ=1 | B) / P(Ŷ=1 | A) ≥ 0.8	The US "four-fifths rule" of thumb from the 1978 Uniform Guidelines on employee selection
Equal opportunity	Separation (half)	TPR = P(Ŷ=1 | Y=1, A=a)	Qualified people are approved at the same rate
Equalized odds	Separation	TPR *and* FPR = P(Ŷ=1 | Y=0, A=a)	Same hit rate and same false-alarm rate
Predictive parity	Sufficiency (at the threshold)	PPV = P(Y=1 | Ŷ=1, A=a)	An approval is equally reliable in each group
Calibration within groups	Sufficiency	P(Y=1 | S=s, A=a) = s for all s	A score of 0.7 means a 70% chance, whatever the group

Two other ideas you should know by name:

- **Individual fairness** (Dwork et al., 2012): similar individuals should get similar predictions. Hard in practice because you must define "similar".
- **Counterfactual fairness** (Kusner et al., 2017): the prediction should not change if the person's group were different, with everything causally downstream of it adjusted. Needs a causal model of the data.

Why it matters

"Make it fair" is not actionable; "equalise TPR between groups within 2 points" is. Picking a definition is a **values decision** tied to the harm you care about:

- Hiring shortlist where the base rates are themselves the product of discrimination → **demographic parity** may be justified.
- Loans, medical screening, where missing a qualified person is the key harm → **equal opportunity**.
- Criminal justice, where both false accusations and missed risks matter → **equalized odds**.
- Risk scores read by humans (a doctor, a judge) → **calibration**, so a number means the same thing for everyone.

How it works — the impossibility result

When groups have **different base rates** (different true rates of Y=1), you generally cannot have calibration *and* equal FPR *and* equal FNR at the same time, unless the model is perfect. This was shown independently by Kleinberg, Mullainathan & Raghavan (2016) and Chouldechova (2017), prompted by the COMPAS debate (§19.10).

Chouldechova's identity makes it concrete. For any group with base rate p:

FPR = p / (1 − p) · (1 − PPV) / PPV · (1 − FNR)

If PPV and FNR are equal across two groups but p differs, the right-hand side differs, so FPR *must* differ.

`code/impossibility.py` builds scores that are **perfectly calibrated by construction** (each person's outcome is drawn with probability equal to their score) for two groups whose base rates are 0.6 and 0.4:

```
groups = {"A": rng.beta(6, 4, n), "B": rng.beta(4, 6, n)}   # mean score 0.6 vs 0.4
data = {g: (s, (rng.random(n) < s).astype(int)) for g, s in groups.items()}
# ... one shared threshold 0.5, then group thresholds chosen so FPR = 0.20 in both
```

Output:

```
group  base_rate  calib(0.7-0.8 bin)  FPR    FNR    PPV   selection
  A      0.601      0.746->0.748     0.624  0.171  0.667  0.747
  B      0.399      0.737->0.728     0.171  0.623  0.594  0.253

Group thresholds chosen so FPR = 0.20 in both groups:
  A: threshold=0.673  FPR=0.200  FNR=0.574  PPV=0.762
  B: threshold=0.483  FPR=0.200  FNR=0.582  PPV=0.582
```

Reading it: both groups are calibrated (scores near 0.74 come true about 74% of the time), yet one shared threshold gives FPR 0.62 for A and 0.17 for B. Group thresholds equalise FPR (and here FNR too), but then PPV splits, 0.76 vs 0.58: an approval "means" different things

per group. Check the identity for group A at threshold 0.5: 0.601/0.399 × 0.333/0.667 × 0.829 ≈ 1.506 × 0.499 × 0.829 ≈ **0.623** — matches the measured FPR of 0.624.

Trade-offs

If you enforce…	You typically give up…
Demographic parity	Calibration and accuracy when base rates truly differ; may approve less-qualified people in one group
Equalized odds	Calibration / predictive parity; usually needs group-specific thresholds
Calibration	Equal error rates when base rates differ
Any constraint	Some overall accuracy (often small, see §19.4)

When not to use a given metric

- Demographic parity, when base-rate differences are real and legitimate (disease prevalence by age).
- Equal opportunity, when the labels themselves are biased — you would equalise against a distorted target.
- The 4/5 rule as a legal safe harbour; it is only a screening heuristic.

Common mistakes

- Reporting one fairness number without saying which definition.
- Comparing groups with tiny sample sizes — gaps of 10 points can be noise; report confidence intervals.
- Forgetting **intersectional** groups (e.g. older women), where gaps are often largest.

In the interview

> **30-second answer:** "Criteria fall into independence (demographic parity), separation (equal opportunity, equalized odds) and sufficiency (predictive parity, calibration). With different base rates, calibration and equal error rates can't both hold (Kleinberg; Chouldechova), so I pick the criterion that matches the harm, document why, and report the rest."

- *Which would you pick for loan approvals?* → Usually equal opportunity (qualified applicants approved at equal rates), plus calibration checks on the score; confirm with legal and domain experts.
- *Does the impossibility mean fairness is hopeless?* → No; trade-offs must be chosen explicitly, and a more accurate model shrinks all the gaps.

Self-check

1. Group A approval 50%, group B 35%. Disparate impact ratio? → 0.35/0.50 = 0.70, below 0.8.
2. Equalized odds requires equality of which two rates? → TPR and FPR.
3. Why can't a calibrated model have equal FPRs when base rates differ? → By FPR = p/(1−p)·(1−PPV)/PPV·(1−FNR), different p forces a different FPR, PPV or FNR.

TL;DR

Parity = equal approval rates; equal opportunity = equal TPR; equalized odds = equal TPR and FPR; calibration = scores mean the same everywhere. With unequal base rates they conflict: choose by harm and report the rest.

19.3 Measuring fairness in code

What it is

A **fairness audit** computes the confusion-matrix rates separately per group and compares them. It needs the true label, the model's decision (and score) and the group, even if the model never sees the group.

The problem it solves

Overall metrics hide group gaps; an audit makes them visible and trackable, like accuracy.

How it works

1. Hold out a test set that includes the sensitive attribute (collected with consent for auditing).
2. Per group compute n, base rate, selection rate, TPR, FPR, PPV and accuracy.
3. Compute the gaps: demographic-parity difference, disparate-impact ratio, equal-opportunity difference, equalized-odds difference (the larger of the TPR and FPR gaps), predictive-parity difference.
4. Check calibration per group, and check for **proxies**: how well do the other features predict the group?

The dataset. `code/credit_data.py` generates 20,000 synthetic loan applicants whose true repayment chance depends *only on finances*. Two biases are injected: group B has lower average income (historical inequality), and in the *recorded* labels 20% of B applicants who actually repaid are marked as defaults (label bias). `zip_region` correlates strongly with group — a proxy.

```
       income_k  zip_region  y_true      y
group
0        58.048       0.145   0.534  0.534
1        45.310       0.796   0.425  0.340
```

The core of `code/fairness_metrics.py`:

```python
def group_report(y, y_hat, group):
    rows = []
    for g in np.unique(group):
        m = group == g
        yt, yp = y[m], y_hat[m]
        tp = np.sum((yt == 1) & (yp == 1)); fp = np.sum((yt == 0) & (yp == 1))
        fn = np.sum((yt == 1) & (yp == 0)); tn = np.sum((yt == 0) & (yp == 0))
        rows.append({"group": "AB"[g], "n": m.sum(), "base_rate": yt.mean(),
                     "selection_rate": yp.mean(), "TPR": tp / (tp + fn),
                     "FPR": fp / (fp + tn), "PPV": tp / (tp + fp),
                     "accuracy": (tp + tn) / m.sum()})
    return pd.DataFrame(rows).set_index("group")

def fairness_summary(r):
    a, b = r.loc["A"], r.loc["B"]
    return {"demographic_parity_diff": a.selection_rate - b.selection_rate,
            "disparate_impact_ratio": b.selection_rate / a.selection_rate,
            "equal_opportunity_diff": a.TPR - b.TPR,
            "equalized_odds_diff": max(abs(a.TPR - b.TPR), abs(a.FPR - b.FPR)),
            "predictive_parity_diff": a.PPV - b.PPV}

# the model is "group-blind": FEATURES excludes `group`
model = make_pipeline(StandardScaler(), LogisticRegression()).fit(train[FEATURES], train.y)
```

Output:

```
Group report vs RECORDED labels (what an auditor can see):
          n  base_rate  selection_rate    TPR    FPR    PPV  accuracy
group
A      4245      0.538           0.516  0.709  0.292  0.739     0.709
B      1755      0.349           0.281  0.436  0.198  0.542     0.675
  demographic_parity_diff  +0.235
  disparate_impact_ratio   +0.544
  equal_opportunity_diff   +0.273
  equalized_odds_diff      +0.273
  predictive_parity_diff   +0.197

Same predictions vs the hidden TRUE labels:
       base_rate  selection_rate    TPR    FPR
group
A          0.538           0.516  0.709  0.292
B          0.435           0.281  0.439  0.159

AUC for predicting group from the 'group-blind' features: 0.886
calibration A: (0.13->0.13) (0.30->0.33) (0.50->0.53) (0.70->0.73) (0.89->0.89)
calibration B: (0.12->0.14) (0.30->0.30) (0.50->0.45) (0.69->0.59) (0.87->0.66)
```

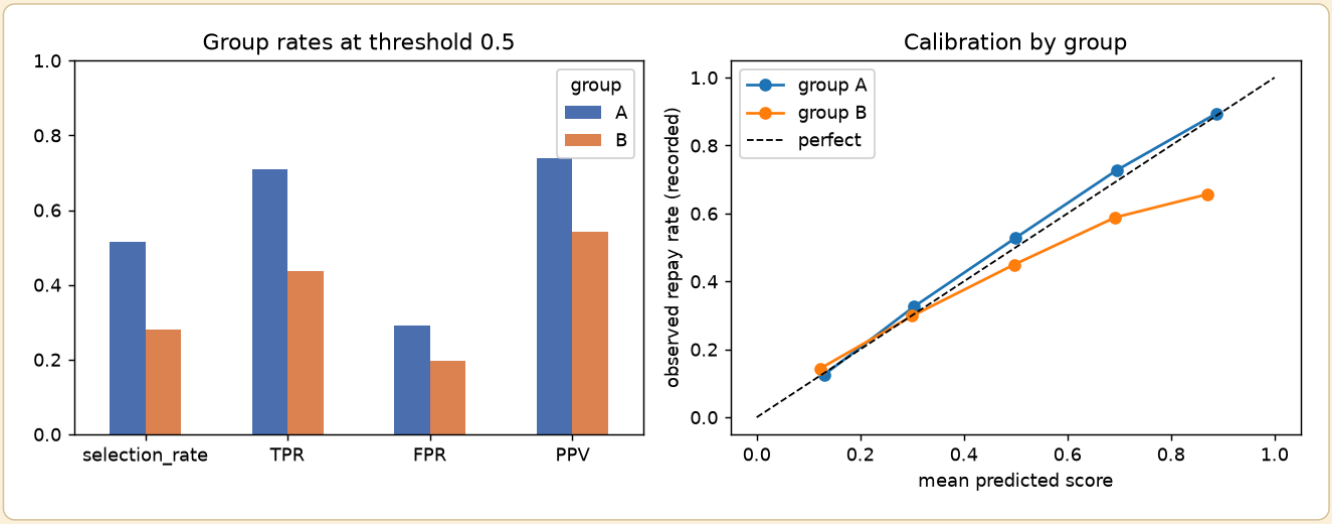


Figure — left: selection rate, TPR, FPR and PPV per group at threshold 0.5. Right: calibration curves; group B's curve sags below the diagonal at high scores.

What the numbers say:

1. **Every criterion fails.** Group B is approved 28% of the time vs 52% (ratio 0.54, far below 0.8); qualified B applicants are approved 44% of the time vs 71%.
2. **"Group-blind" didn't work.** The model never saw `group`, yet the other features predict it with AUC 0.886, mostly via `zip_region`. Dropping the sensitive column (**fairness through unawareness**) does not remove its influence and stops you measuring it.
3. **Label bias is invisible to the auditor.** On recorded labels B's base rate looks like 0.349; the truth is 0.435. Only knowing *how labels were produced* reveals that some B "defaults" are artefacts.
4. **Calibration differs.** A score of 0.89 comes true 89% of the time for A but only 66% for B (recorded labels), so a human reading the score is misled for one group.

The same audit with Fairlearn's `MetricFrame` **(not run — `fairlearn` not installed)**:

```
from fairlearn.metrics import MetricFrame, selection_rate, true_positive_rate, false_positive_rate
mf = MetricFrame(metrics={"sel": selection_rate, "tpr": true_positive_rate, "fpr": false_positive_rate},
                 y_true=y_test, y_pred=y_pred, sensitive_features=group_test)
print(mf.by_group); print(mf.difference())
```

Trade-offs

Choice	Pro	Con
Collect the sensitive attribute	Lets you measure and fix bias	Privacy risk; some jurisdictions restrict it; needs consent and access controls
Don't collect it	Simpler legally	Bias still happens through proxies — you just can't see it
Infer it (e.g. BISG surname + geography method used by US regulators)	Enables audits without collection	Error-prone; must be used only for auditing

Common mistakes

- Auditing on the training set instead of held-out data.
- Reporting gaps without group sizes and confidence intervals.
- Treating recorded labels as ground truth when the labelling process itself is suspect.
- Checking fairness once before launch and never again — drift changes gaps.

Real-world examples

Fairlearn (started at Microsoft) and IBM's AIF360 are open-source fairness toolkits; Google's What-If Tool visualises per-group metrics. US lenders run fair-lending analyses on underwriting models for regulatory compliance.

In the interview

30-second answer: "I slice held-out data by group, compute selection rate, TPR, FPR, PPV and calibration per group and the gaps between groups, and predict the sensitive attribute from the features to find proxies. I report group sizes and confidence intervals and keep tracking the metrics in production."

- *Why keep the sensitive attribute if the model doesn't use it?* → To measure and mitigate; you can store it separately with strict access controls.
- *How do you find proxies?* → Predict the group from features (AUC), check feature–group correlations, and inspect feature importances.

Self-check

1. Equalized-odds difference with TPR gap 0.27 and FPR gap 0.09? → 0.27 (the maximum).
2. What does an AUC of 0.886 for predicting group from features mean? → The features encode group membership strongly — proxies exist.
3. Why is dropping the sensitive column called "fairness through unawareness", and why is it weak? → The model is unaware of the column, but proxies carry the same information.

TL;DR

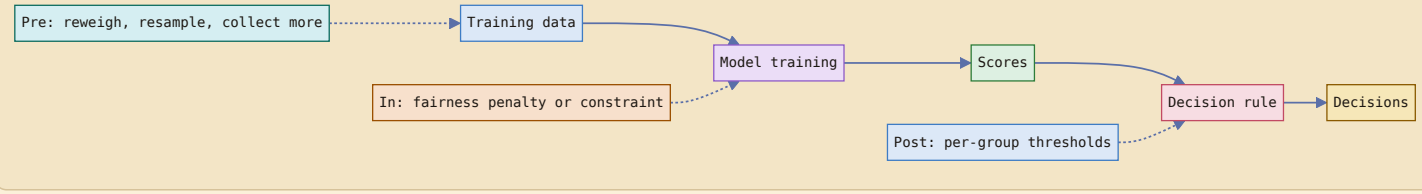
Slice metrics by group, compute the gaps and per-group calibration, test for proxies, and never trust "we removed the sensitive column".

19.4 Mitigating bias: before, during and after training

What it is

Bias mitigation changes the data, the training objective or the decision rule so that a chosen fairness criterion improves, at an acceptable cost in accuracy. The three intervention points:

Stage	Idea	Techniques
Pre-processing (fix the data)	Change what the model learns from	Collect more data for thin groups, re-sample, **reweighing** (Kamiran & Calders, 2012), relabel suspect labels, learn fair representations, synthetic augmentation for rare groups
In-processing (fix the training)	Add fairness to the objective	**Fairness penalties / constraints** (e.g. Zafar et al., 2017), reductions approach (Agarwal et al., 2018; Fairlearn's `ExponentiatedGradient`), adversarial debiasing (Zhang et al., 2018)
Post-processing (fix the decisions)	Keep the model, change how scores become decisions	**Group-specific thresholds** (Hardt, Price & Srebro, 2016), reject-option classification



Why it matters

Measurement without mitigation only documents harm. Post-processing needs no retraining (good for a vendor model); pre-processing leaves training code untouched; in-processing usually gives the best accuracy–fairness trade-off but needs custom training.

How it works

Reweighing. Give each (group, label) cell a weight so that, in the weighted data, label and group are statistically independent:

$$w(g, y) = P(A=g) \cdot P(Y=y) / P(A=g, Y=y)$$

Cells that are rarer than independence predicts (here: group B with a good outcome) get weights above 1. The model is then trained with `sample_weight=w`.

Fairness penalty. Add a term to the loss that grows with the gap you want to close. Here: logistic loss + $\lambda \cdot$(mean score of A − mean score of B)^2. Larger λ means more parity, less accuracy.

Threshold adjustment (post-processing). Keep the scores; choose a separate threshold per group, fitted on a *validation* set, so the chosen rate matches:

1. Fix group A's threshold (0.5).
2. Measure the target rate for A on validation data — its TPR (for equal opportunity) or selection rate (for demographic parity).
3. For group B pick the threshold at the matching quantile of its scores (among its true positives for TPR, among all its members for selection rate).
4. Apply both thresholds on the test set and re-audit.

Key code from `code/mitigation.py`:

```
def reweighing_weights(y, g):
    w = np.empty(len(y))
    for gv in (0, 1):
        for yv in (0, 1):
            m = (g == gv) & (y == yv)
            w[m] = (np.mean(g == gv) * np.mean(y == yv)) / np.mean(m)
    return w

def fair_logreg(X, y, g, lam, epochs=300, seed=0):
    # ... torch Linear layer + Adam
        p = torch.sigmoid(logits)
        gap = p[gA].mean() - p[gB].mean()
        loss = F.binary_cross_entropy_with_logits(logits, y_t) + lam * gap ** 2

def threshold_for_rate(score, target, positives_only_mask=None):
    s = score if positives_only_mask is None else score[positives_only_mask]
    return np.quantile(s, 1 - target)           # the top `target` fraction clears it

# equal opportunity: B's threshold makes TPR_B (validation) = TPR_A
tpr_A = np.mean(s_va[(gva == 0) & (yva == 1)] >= 0.5)
tB = threshold_for_rate(s_va[gva == 1], tpr_A, yva[gva == 1] == 1)
y_eo = np.where(gte == 0, s_te >= 0.5, s_te >= tB).astype(int)
```

Output (`acc` against recorded labels, `acc_true` against the hidden true labels; `EO_diff` = TPR gap):

```
reweighing weights  (A,y=0) (A,y=1) (B,y=0) (B,y=1): [1.123, 0.892, 0.791, 1.413]
equal-opportunity thresholds: A=0.500  B=0.356
demographic-parity thresholds: A=0.500  B=0.340
                          acc  acc_true  sel_A  sel_B  DP_diff  EO_diff  EO_diff_true
method
baseline (t=0.5)          0.704     0.701  0.532  0.269    0.263    0.290         0.291
pre: reweighing           0.703     0.704  0.501  0.349    0.153    0.155         0.159
in: DP penalty (lam=5)    0.690     0.695  0.472  0.420    0.053    0.055         0.059
post: equal opportunity   0.694     0.702  0.532  0.481    0.051    0.041         0.044
post: demographic parity  0.690     0.699  0.532  0.511    0.021    0.017         0.024
```

Reading it:

1. **Reweighing** roughly halves both gaps with no accuracy loss (group-B positives get weight 1.41).
2. **The penalty** closes the parity gap to 5 points for about 1.4 points of recorded accuracy.
3. **Threshold adjustment** closes the TPR gap from 29 to 4 points simply by lowering B's threshold to 0.356; accuracy on the *true* labels is unchanged (0.701 → 0.702), because part of the "unfairness" was the label bias.
4. The accuracy cost of fairness is often **smaller than expected**, but it is data-dependent: measure it.

Fairlearn equivalents **(not run — `fairlearn` not installed)**:

```python
from fairlearn.postprocessing import ThresholdOptimizer
from fairlearn.reductions import ExponentiatedGradient, EqualizedOdds
post = ThresholdOptimizer(estimator=base_model, constraints="true_positive_rate_parity",
                          prefit=True, predict_method="predict_proba")
post.fit(X_val, y_val, sensitive_features=g_val)
y_hat = post.predict(X_test, sensitive_features=g_test, random_state=0)

inproc = ExponentiatedGradient(LogisticRegression(), constraints=EqualizedOdds())
inproc.fit(X_train, y_train, sensitive_features=g_train)
```

Process controls matter as much as algorithms: audit who is in the data (per group and intersection) before training; collect more examples for thin groups, using synthetic data only as a supplement; give humans the authority to review and override high-stakes decisions; and document data choices so outsiders can audit them.

Trade-offs

Method	Pros	Cons
Reweighing / resampling	Simple, model-agnostic, no group needed at prediction time	Limited control over which criterion improves
Fairness penalty / constraints	Directly targets a criterion; good trade-offs	Custom training; hyperparameter λ; can be unstable
Group thresholds	No retraining; precise control; works on vendor models	Uses the group at decision time — **may be unlawful disparate treatment** in some settings (e.g. US employment law)
Collecting better data / labels	Fixes the cause	Slow and costly

When not to use it

Don't bolt on post-processing when the real issue is a broken label (fix the label), and don't deploy group thresholds without legal review.

Common mistakes

- Fitting thresholds on the test set (leaks information; always use a validation set).
- Tiny groups: thresholds become noisy — pool data or use Bayesian estimates.
- Declaring victory on one metric while another gets worse.

Real-world examples

LinkedIn has described re-ranking for gender-representative candidate search results in LinkedIn Recruiter. Fairlearn's `ThresholdOptimizer` implements Hardt et al.'s post-processing; AIF360 implements reweighing, adversarial debiasing and reject-option classification.

In the interview

30-second answer: "Pre-processing reweighs or improves the data; in-processing adds a fairness penalty or constraint to the loss; post-processing sets per-group thresholds on validation data. Post-processing is cheap and precise but uses the attribute at decision time, which has legal implications. Afterwards I re-audit every metric and the accuracy cost."

- *Which is best?* → Fix data and labels first; in-processing usually gives the best trade-off; post-processing when you can't retrain.
- *How big is the accuracy cost?* → Measure it; often a point or two, as here, especially when labels are biased.

Self-check

1. P(B)=0.3, P(Y=1)=0.5, P(B, Y=1)=0.1. Reweighing weight for (B, 1)? → 0.3·0.5/0.1 = 1.5.
2. Why fit thresholds on validation, not test? → To avoid overfitting the fairness adjustment to the evaluation data.
3. Which stage suits a model bought from a vendor? → Post-processing.

TL;DR

Pre-processing fixes data (reweighing), in-processing fixes the loss (fairness penalty), post-processing fixes the decision (group thresholds). Measure the accuracy cost and re-audit every criterion.

19.5 Explainability

What it is

Explainability (or **interpretability**) is the ability to say *why* a model made a prediction in terms a human can check. A **global** explanation describes the model's overall behaviour (which features matter, and how); a **local** one explains a single prediction (why *this* applicant was declined). An **intrinsically interpretable** model is readable directly (linear models, small trees, rule lists, GAMs, i.e. generalised additive models); a **post-hoc** method explains a black box from outside.

Analogy: a global explanation is a restaurant's recipe book; a local one is why tonight's dish tasted salty.

Why it matters

- **Debugging**: spot leakage, shortcuts and proxies.
- **Trust**: doctors, underwriters and judges need reasons.
- **Law**: US lenders must state the principal reasons for denying credit (ECOA/Regulation B adverse-action notices); GDPR requires meaningful information about the logic of solely automated significant decisions (§19.9).
- **Recourse**: telling a person what would change the decision.

How it works

1. Permutation importance (global).

1. Score the fitted model on held-out data (e.g. ROC-AUC).
2. Shuffle one column, breaking its link with the target; score again.
3. Importance = drop in score; repeat several shuffles and average.

Unlike tree impurity importance (Chapter 7), it is model-agnostic and measured on held-out data.

2. Partial dependence (PDP) and ICE (global/local). PD(x_s) = (1/n) Σᵢ f(x_s, x_c⁽ⁱ⁾): set feature s to a value for *every* row, average the predictions, repeat across a grid. **ICE** (individual conditional expectation) curves plot the same for each row separately; the PDP is their average. ICE reveals interactions that the average hides.

3. SHAP (local, adds up to global). Shapley values come from cooperative game theory (Shapley, 1953): the features are "players" sharing the "payout" f(x) − E[f]. Feature i's value is its average marginal contribution over all orders in which features could be added:

φᵢ = Σ over subsets S not containing i of |S|!(M−|S|−1)!/M! · [v(S ∪ {i}) − v(S)]

where v(S) is the expected prediction when only the features in S are known (the others drawn from background data). The weight |S|!(M−|S|−1)!/M! is just the fraction of all M! orderings of the features in which exactly the features of S come before i, so φᵢ is i's average extra contribution over every order of arrival. Properties: **efficiency** (φ's sum to f(x) − E[f]), **symmetry**, **dummy** (a feature that never changes the output gets 0), **additivity**. Exact computation needs 2^M subsets, so the SHAP library (Lundberg & Lee, 2017) uses approximations: KernelSHAP (any model, sampling) and TreeSHAP (exact and fast for tree ensembles).

4. LIME (local). Ribeiro et al., 2016: perturb the input around x, weight the black box's predictions by closeness to x, and fit a sparse linear model whose coefficients are the explanation. Fast, but results shift with the kernel width and seed.

5. Counterfactual explanations (local). "You would have been approved if your debt ratio were 0.25 instead of 0.40" — the smallest *actionable* change that flips the decision.

`code/explainability.py` trains a histogram gradient-boosting model on the credit data (without `group`):

```
pi = permutation_importance(model, Xte, yte, scoring="roc_auc", n_repeats=10, random_state=0)
PartialDependenceDisplay.from_estimator(model, Xte, [0, 1, 4], feature_names=FEATURES,
                                        kind="both", subsample=300, random_state=0)

def exact_shapley(f, x, background):
    M = len(x)
    def v(S):                                   # expected output with features S fixed to x
        Z = background.copy(); Z[:, list(S)] = x[list(S)]
        return f(Z).mean()
    phi = np.zeros(M)
    for i in range(M):
        others = [j for j in range(M) if j != i]
        for size in range(M):
            for S in combinations(others, size):
                w = factorial(size) * factorial(M - size - 1) / factorial(M)
                phi[i] += w * (v(S + (i,)) - v(S))
    return phi, v(())

def lime_like(f, x, X_ref, n=5000, width=0.75, seed=0):
    # ... Gaussian samples around x, proximity weights, weighted Ridge on standardised offsets
```

Output:

```
test accuracy: 0.705

Permutation importance (drop in test ROC-AUC when a column is shuffled):
  income_k        0.1472 +/- 0.0044
  debt_ratio      0.0596 +/- 0.0028
  prior_defaults  0.0573 +/- 0.0035
  years_employed  0.0132 +/- 0.0019
  zip_region      0.0019 +/- 0.0010

Applicant: {'income_k': 70.9, 'debt_ratio': 0.27, 'years_employed': 10.0, 'prior_defaults': 1.0, 'zip_region': 0.0}
model score f(x) = 0.816   baseline E[f] = 0.471
Shapley values (push towards approval if +):
  income_k        +0.178
  years_employed  +0.120
  debt_ratio      +0.111
  prior_defaults  -0.090
  zip_region      +0.027
check: baseline + sum(phi) = 0.816  (equals f(x))

LIME-style local slopes (score change per +1 sd):
  income_k        +0.195
  debt_ratio      -0.089
  prior_defaults  -0.069
  years_employed  +0.055
  zip_region      -0.016
```

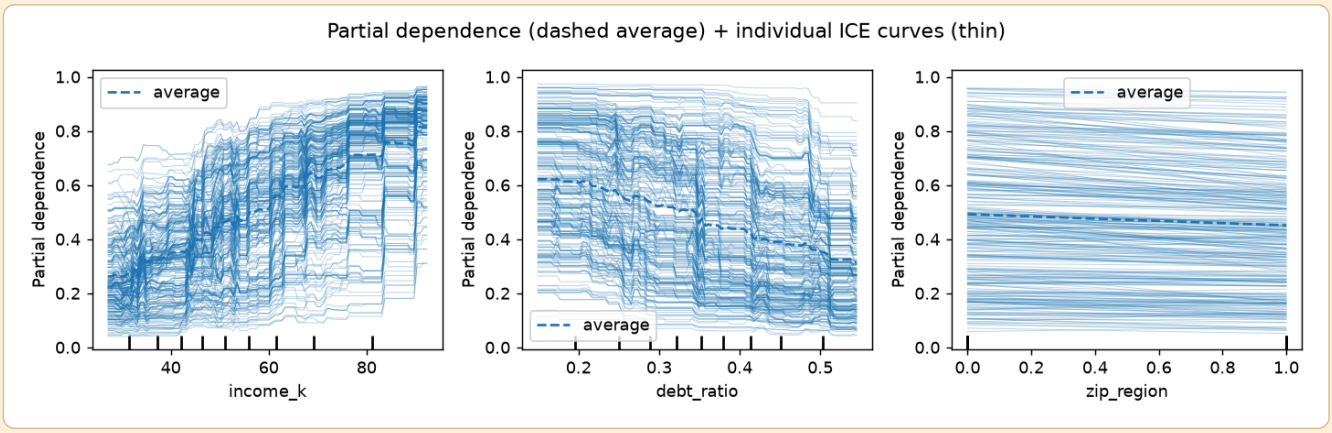


Figure — PDP (dashed) and ICE curves (thin): approval probability rises with income, falls with debt ratio, and dips slightly for zip region 1.

Reading it:

1. Income dominates globally. `zip_region` has small permutation importance — yet §19.3 showed it is a strong group proxy. **Low importance does not mean "fair"**; much of the group signal also flows through income.
2. For this applicant the Shapley values add up exactly: 0.471 + 0.178 + 0.120 + 0.111 − 0.090 + 0.027 = 0.816 (efficiency).
3. SHAP and LIME frame things differently but consistently: Shapley says "compared with an average applicant, living in zip 0 helped (+0.027)"; LIME's slope says "moving *towards* zip 1 would lower the score". (Our Gaussian perturbation of a 0/1 column is crude; the real LIME library samples categorical features properly.)

The SHAP library version **(not run — `shap` not installed)**:

```
import shap
explainer = shap.TreeExplainer(model)          # exact, fast for tree ensembles
sv = explainer(X_test)                         # Explanation object, one row per sample
shap.plots.waterfall(sv[3])                    # local: one applicant
shap.plots.beeswarm(sv)                        # global: distribution of impacts per feature
```

Trade-offs

Method	Scope	Pros	Cons
Interpretable model	Both	Faithful by construction	May lose accuracy on complex data
Permutation importance	Global	Model-agnostic, uses held-out data	Misleading with correlated features (shuffling makes unrealistic rows; importance splits between twins)
PDP / ICE	Global / local	Shows shape of effect	Assumes feature independence; averages hide interactions (use ICE)
SHAP	Local → global	Principled, additive, consistent	Costly for non-tree models; depends on background data; still correlational
LIME	Local	Fast, any model	Unstable; depends on kernel and sampling
Counterfactuals	Local	Actionable recourse	Many possible answers; must respect feasibility

When not to use it

- Don't present post-hoc explanations as *causal* — they explain the model, not the world.
- For high-stakes decisions consider an interpretable model first (Rudin, 2019, argues this strongly).
- Don't show raw SHAP plots to customers; translate into plain-language reasons.

Common mistakes

- Computing importance on training data (rewards overfitting).
- Ignoring correlated features — two twin features can each look unimportant.
- Treating one LIME run as the truth; check stability across seeds.
- Explaining the wrong output (log-odds vs probability) and confusing readers.

Real-world examples

Lenders produce "reason codes" for adverse-action notices; the major cloud ML platforms (Vertex AI, SageMaker Clarify, Azure ML) offer SHAP-style attributions; TreeSHAP is built into XGBoost and LightGBM (`pred_contribs=True` / `pred_contrib=True`).

In the interview

30-second answer: "Global explanations — permutation importance on held-out data, PDP and ICE — describe the whole model. Local ones explain a single prediction: SHAP splits it into additive feature contributions with game-theory guarantees; LIME fits a local linear surrogate. Both are correlational, so I use them for debugging and reason codes, not causal claims."

- *SHAP vs LIME?* → SHAP has axioms (efficiency, symmetry, dummy) and consistent results; LIME is faster but less stable. TreeSHAP is exact for trees.
- *How do you explain a decline to a customer?* → Top negative SHAP contributors mapped to plain-language reason codes, plus a counterfactual if possible.

Self-check

1. Baseline 0.40, SHAP values +0.10, −0.05, +0.20. Prediction? → 0.65.
2. Global or local: a PDP? a waterfall plot? → PDP global; waterfall local.
3. Why prefer held-out data for permutation importance? → Training-set importance rewards features the model overfit.

TL;DR

Global: permutation importance, PDP. Local: SHAP (adds up to f(x) − E[f]), LIME, counterfactuals. Explanations describe the model, not causality.

19.6 Privacy

What it is

Privacy in ML means controlling what personal information is collected, how it is used, and what the trained model can leak. **PII** (personally identifiable information) identifies a person directly (name, email, national ID) or in combination; **quasi-identifiers** such as ZIP code, birth date and sex are harmless alone but identifying together; **sensitive attributes** (health, finances, religion, biometrics) do great harm if revealed.

Why it matters

ML runs on data about people — photos, messages, purchases, scraped pages. Every team must answer:

1. **Consent and ownership.** Did people agree to their data training a model? If you send company data to a third-party AI service, is it used to train the vendor's models, and where can it flow next?
2. **Legal duties.** GDPR, California's CCPA/CPRA and similar laws require transparency, a lawful basis, and rights to access, delete or opt out.
3. **Harms.** Leaked data enables identity theft, harassment and discrimination; companies face fines and lost trust.
4. **Models memorise.** A trained model can reveal training records (§19.7), so "we only ship the model" is no privacy guarantee.

How it works

Anonymisation pitfalls. Removing names is *pseudonymisation*, not anonymisation; GDPR still treats pseudonymised data as personal data. Famous failures:

- Latanya Sweeney estimated that 87% of Americans are likely unique on 5-digit ZIP + birth date + sex (1990 census), and re-identified the Massachusetts governor's hospital record by linking "anonymised" health data with a public voter roll.
- AOL's "anonymised" 2006 search logs let journalists identify individual users; Netflix Prize ratings were re-identified by matching them with public IMDb reviews (Narayanan & Shmatikov, 2008).
- Strava's 2018 global heatmap exposed military bases from soldiers' runs: aggregates leak too.

k-anonymity (Sweeney, 2002): every combination of quasi-identifiers must be shared by at least k records, achieved by **generalisation** (ZIP 02138 → 021**, age 29 → "≤35") and **suppression** (dropping outliers). Its weaknesses led to **l-diversity** (each group must contain at least l distinct sensitive values) and **t-closeness** (each group's sensitive-value distribution must be close to the overall one).

Differential privacy (DP) (Dwork et al., 2006) is a guarantee about an *algorithm*: its output distribution barely changes whether or not any one person's record is in the data. Formally, M is ε-DP if for all datasets D, D' differing in one record and every set of outputs O:

$$P[M(D) \in O] \le e^{\varepsilon} \cdot P[M(D') \in O]$$

Small ε = strong privacy (ε = 1 means any outcome is at most e ≈ 2.72 times likelier with you in the data than without).

The **Laplace mechanism** achieves ε-DP for a numeric query f:

1. Compute the **sensitivity** Δf = the most one person can change f. A count has Δf = 1; a mean of values clipped to [L, U] over n people has Δf = (U − L)/n.
2. Add noise drawn from Laplace(0, b) with scale b = Δf/ε.
3. Release f(D) + noise. The expected absolute error is b; the standard deviation √2·b.

Useful DP properties: **composition** (answering k queries at ε each costs up to kε in total — the privacy budget), and **post-processing** (anything computed from a DP output stays DP).

`code/privacy_dp.py`:

```
def laplace_count(true_count, eps, rng):
    return true_count + rng.laplace(0.0, 1.0 / eps)           # sensitivity of a count = 1

def laplace_mean(x, lo, hi, eps, rng):
    x = np.clip(x, lo, hi)                                     # clipping bounds the sensitivity
    sens = (hi - lo) / len(x)
    return x.mean() + rng.laplace(0.0, sens / eps)

gen = df.assign(zip=df.zip.str[:3] + "**",
                age=pd.cut(df.age, [0, 35, 120], labels=["<=35", ">35"]).astype(str))
```

Output:

```
raw table: k = 1
generalised (zip prefix + age band): k = 4
zip    age   sex
021**  <=35  F                    [flu]
       >35   M      [cancer, flu, heart]

exact: count(all)=54  count(all except Bob)=53  -> Bob's status leaked: 1
DP (eps=0.5 each): 52.8 vs 48.8 -> difference +4.0 (noise sd ~ 2.8 per query)

true count(hiv) = 54,  true mean income = 61.69k
eps     noisy count (3 draws)        mean |err| count   noisy mean income
0.01       96.8,   122.2,  -155.1         96.54          62.01k
0.1        58.2,    57.6,    60.9          9.83          62.20k
1.0        54.4,    54.3,    54.6          1.06          61.56k
10.0       53.9,    54.0,    53.9          0.10          61.68k
```

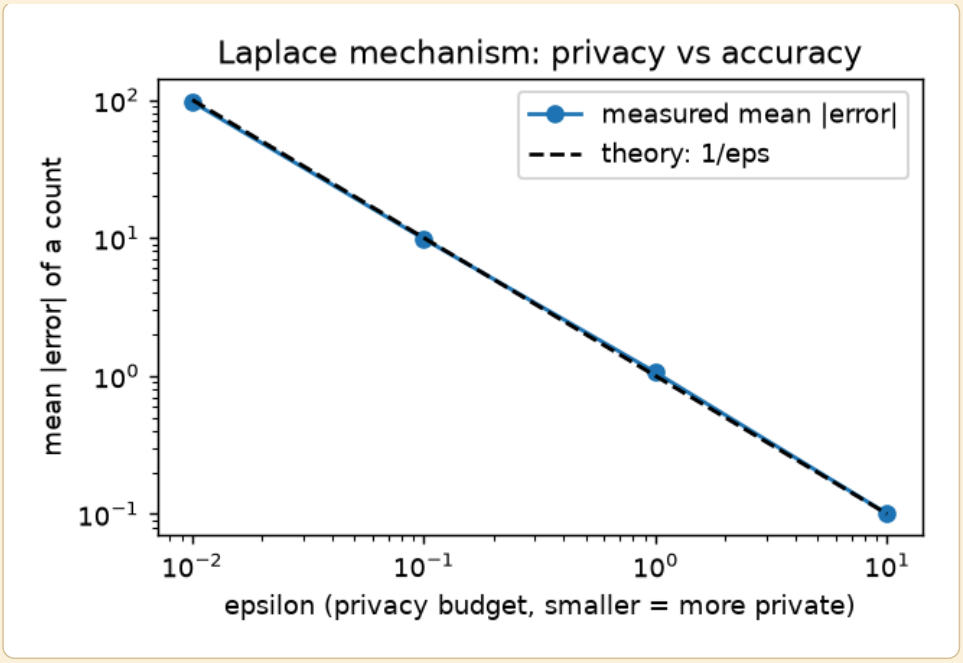


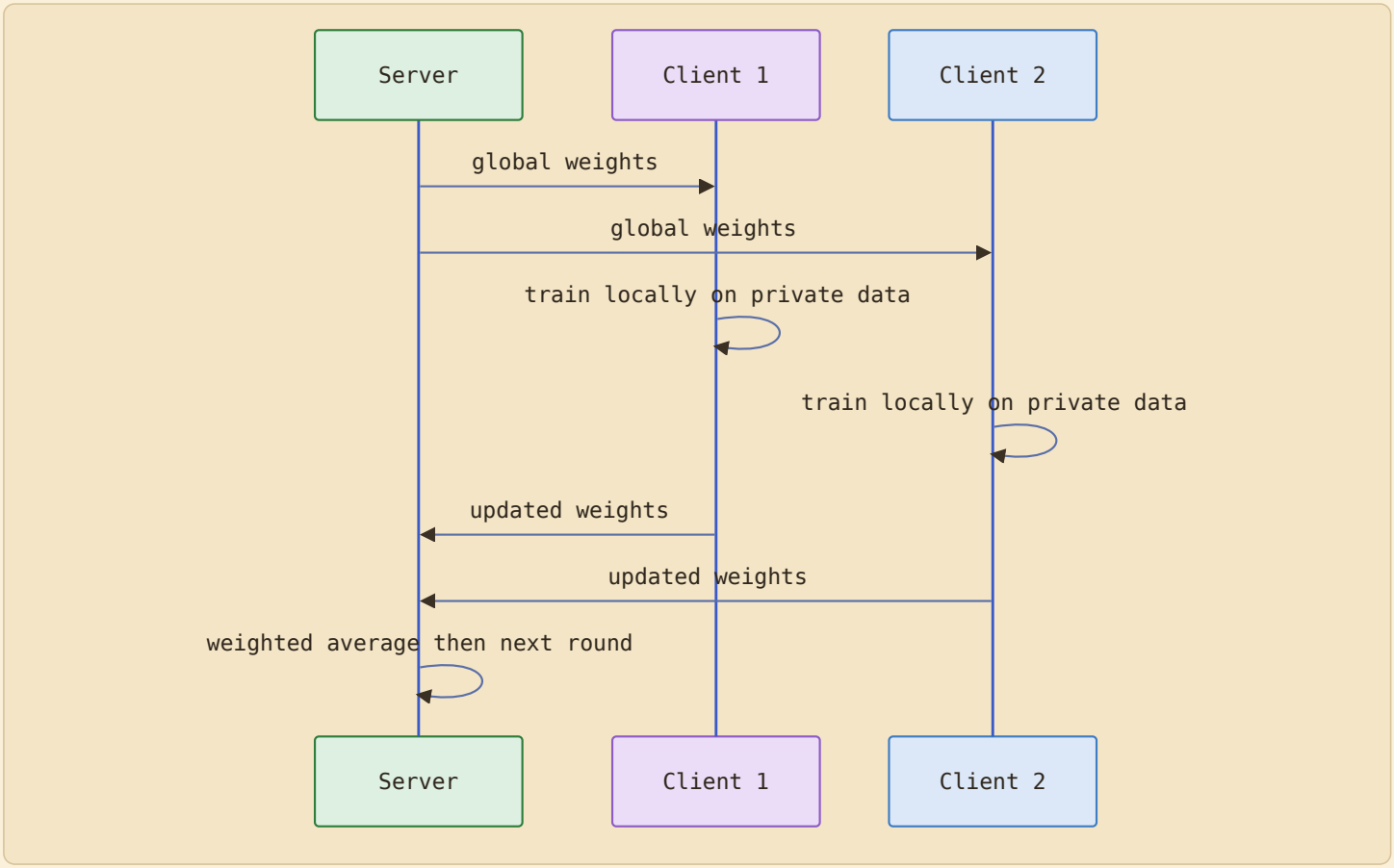
Figure — the Laplace mechanism's mean absolute error on a count follows 1/ε: ten times more privacy costs ten times more error.

Reading it:

1. Generalisation lifts k from 1 to 4, but everyone in the "021**, ≤35, F" group has flu, so knowing a woman aged ≤35 from that area is in the table reveals her diagnosis: a **homogeneity attack**, which l-diversity guards against.
2. Two harmless-looking exact counts ("all patients", "all except Bob") reveal Bob's HIV status: a **differencing attack**. With DP noise the difference (+4.0) says nothing reliable.
3. The mean-income query stays accurate even at ε = 0.01 because its sensitivity is tiny ((300 − 0)/10,000 = 0.03). **DP is cheap for large aggregates and expensive for small groups** — the central trade-off.

DP in model training. DP-SGD (Abadi et al., 2016) clips each example's gradient and adds Gaussian noise before each update; the PyTorch library Opacus and TensorFlow Privacy implement it. It costs accuracy, especially on small datasets and rare classes.

Federated learning (FL) (McMahan et al., 2017) keeps raw data on devices or at institutions and moves the *model* instead: the server sends the global model to clients, each trains a few epochs locally and returns only its updated weights, and the server averages them weighted by client data size (**FedAvg**) before the next round.



`code/federated_avg.py` (three "hospitals" with different sizes and non-identical data):

```
for rnd in range(1, 31):
    updates, sizes = [], []
    for Xc, yc in clients:                              # runs ON each client
        updates.append(local_train(w_global.copy(), Xc, yc))
        sizes.append(len(yc))
    w_global = np.average(updates, axis=0, weights=sizes)  # server: weighted average
```

Output:

```
round  1: global test accuracy = 0.786
round  5: global test accuracy = 0.815
round 10: global test accuracy = 0.823
round 30: global test accuracy = 0.826

centralised (all data pooled, 150 epochs): 0.824
client 0 alone (n=2500):            0.810
client 1 alone (n=1500):            0.767
client 2 alone (n=1000):            0.775
```

The federated model matches the pooled model (0.826 vs 0.824) and beats every client training alone — without any raw row leaving a client. Caveat: updates themselves can leak information (gradient inversion attacks), so production FL adds **secure aggregation** (the server sees only the sum of updates) and often DP noise.

Trade-offs

Technique	Protects against	Cost / limits
Data minimisation, access control, encryption	Breaches, misuse	Discipline; doesn't stop model leakage
Pseudonymisation	Casual identification	Easily reversed by linkage
k-anonymity (+ l-diversity, t-closeness)	Linkage on quasi-identifiers	Loses detail; weak against background knowledge; no formal guarantee
Differential privacy	Any attacker, any side information (formal bound)	Noise reduces accuracy; budget accounting; hard for small groups
Federated learning	Raw data centralisation	Communication cost, non-IID data, updates can leak — combine with secure aggregation/DP
Synthetic data	Sharing raw records	Can memorise real records unless generated with DP

When not to use it

- Don't use DP on tiny datasets where the noise swamps the signal — restrict access instead.
- Don't claim "anonymous" because names were removed.
- Don't adopt FL when the data can legally and safely be centralised — it adds real engineering complexity.

Common mistakes

- Forgetting composition: many queries at ε = 1 each add up to a weak guarantee.
- Not clipping inputs, so sensitivity is unbounded and the "DP" is fake.
- Logging raw PII in feature stores, debug logs or prompts.
- Keeping training data forever "just in case" (violates storage limitation under GDPR).

Real-world examples

The US Census Bureau protected 2020 census results with differential privacy (its "TopDown" algorithm). Apple uses local DP to learn popular emoji and new words from devices; Google's RAPPOR (2014) collected Chrome statistics with local DP. Google's Gboard keyboard trains next-word prediction with federated learning. Hospitals use FL consortia for medical imaging models.

In the interview

30-second answer: "Removing names isn't anonymisation — quasi-identifiers link back to people. k-anonymity resists linkage without a formal guarantee. Differential privacy bounds any one person's influence: Laplace noise of scale sensitivity/ε, with ε budgeted across queries; DP-SGD applies it to training. Federated learning keeps data local and averages updates, with secure aggregation and DP because updates can leak."

- *Sensitivity of a mean of n values in [0, 100]?* → 100/n.

Self-check

1. Count query, ε = 0.5. Laplace scale? → b = 1/0.5 = 2.
2. Ten queries at ε = 0.2 each. Total budget (basic composition)? → 2.0.

3. A 5-anonymous table where every record in a group has the same diagnosis — what's wrong? → No l-diversity: a homogeneity attack reveals the diagnosis.
4. What does the FedAvg server compute? → A data-size-weighted average of client model weights.

TL;DR

Pseudonymised ≠ anonymous. k-anonymity resists linkage but not background knowledge; differential privacy adds noise of scale Δf/ε for a provable bound; federated learning moves the model, not the data.

19.7 Security: attacks on ML models

What it is

Adversarial ML studies attackers who target the model itself rather than the servers around it. A **threat model** states what the attacker wants and can do: **white-box** (knows weights and code) or **black-box** (can only send queries and read answers); attacking at **training time** (the data) or **inference time** (the inputs).

Attack	When	Goal
Adversarial examples (evasion)	Inference	Make the model misclassify a crafted input (white-box: uses gradients; black-box: queries or a substitute model)
Data poisoning	Training	Degrade the model or plant a **backdoor** trigger
Model extraction (stealing)	Inference, via queries	Copy a paid or proprietary model
Model inversion	Inference, via queries	Reconstruct sensitive features or typical training inputs
Membership inference	Inference, via queries	Learn whether a record was in the training set
Training-data extraction	Inference, via queries	Make a generative model emit memorised training text or images

Why it matters

Models guard money and safety (fraud filters, malware detection, moderation, driving perception), so each attack maps to a real harm: fraud slipping through, a stolen model, or proof that a named person was in a cancer study.

How it works

1. Adversarial examples and FGSM. Szegedy et al. (2013) found that tiny, targeted perturbations flip neural-network predictions. Goodfellow, Shlens & Szegedy (2015) explained it by the models' near-linear behaviour in high dimensions and introduced the **Fast Gradient Sign Method**:

$$x_{adv} = \text{clip}(x + \varepsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)), 0, 1)$$

1. Compute the gradient of the loss with respect to the *input* pixels (not the weights).
2. Take its sign: +1 or −1 per pixel — the direction that increases the loss.
3. Move every pixel by ε in that direction (an L∞ budget of ε per pixel) and clip to the valid range.

Many small nudges, each pointing the "wrong" way, add up to a large change in the logit. Their paper's famous image: a panda classified with 57.7% confidence becomes a "gibbon" at 99.3% after an imperceptible perturbation. **PGD** (Madry et al., 2018) repeats FGSM-style steps and is the standard strong attack. The main defence is **adversarial training**: include adversarial examples in each training batch.

`code/fgsm_attack.py` (an MLP on sklearn's 8×8 digits, pixels scaled to [0, 1]):

```
def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    loss = nn.functional.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

# adversarial training: each batch = clean examples + their FGSM versions
if adversarial:
    xb = torch.cat([xb, fgsm(model, xb, yb, eps)]); yb = torch.cat([yb, yb])
```

Output:

```
eps    plain-model acc   adversarially-trained acc
0.0    0.976             0.989
0.05   0.796             0.946
0.1    0.365             0.826
0.2    0.061             0.298
0.3    0.017             0.019

example: true=1  clean pred=1 (1.00)  adversarial pred=8 (1.00)
```

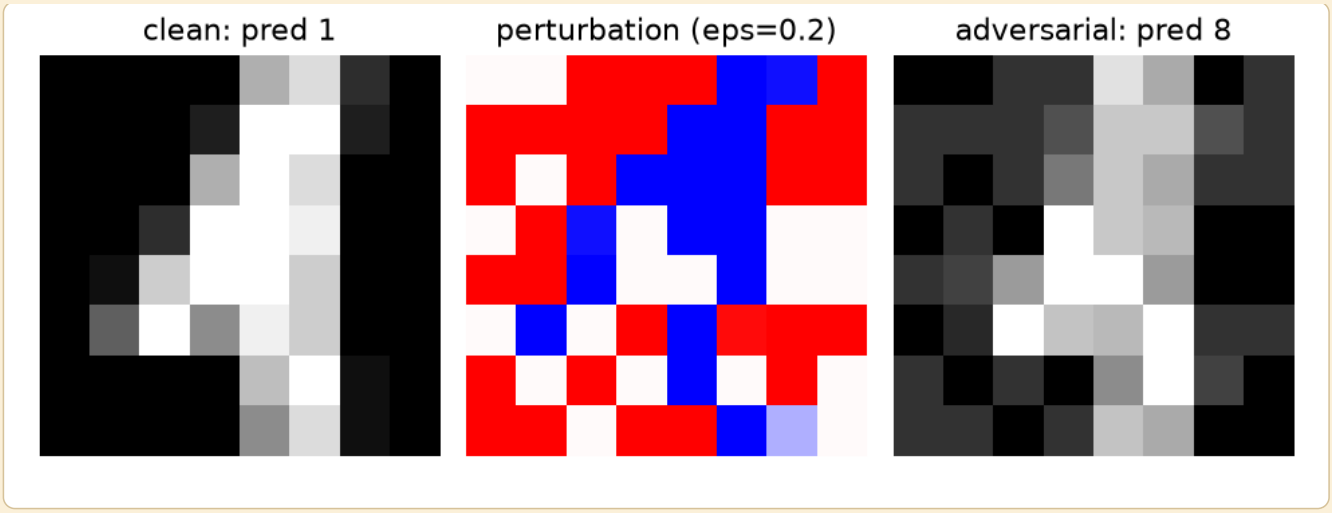


Figure — a "1" (left), the FGSM perturbation with ε = 0.2 (middle, red = brighter, blue = darker), and the result classified as "8" with full confidence (right).

Reading it: a per-pixel budget of 0.1 cuts the plain model from 97.6% to 36.5%. Adversarial training at ε = 0.1 keeps 82.6% — but gives little protection at larger budgets it never trained on. (On 8×8 images an ε of 0.2 is visible; on high-resolution photos far smaller budgets suffice and the change is invisible.)

Physical-world versions exist: Eykholt et al. (2018) made stop signs misread by adding sticker patterns; adversarial patches and glasses frames fool face recognition.

2. Data poisoning. The attacker corrupts training data. **Label flipping** changes labels; a **backdoor** (e.g. BadNets, Gu et al., 2017) adds a trigger pattern to some examples with a chosen label, so the model behaves normally until it sees the trigger. Risk grows when training on scraped web data, user feedback or crowd-sourced labels. Microsoft's Tay chatbot (2016) was withdrawn within a day after users deliberately taught it offensive content — a well-known case of manipulated learning from user input.

3. Model extraction. Tramèr et al. (2016) showed that prediction APIs can be copied by querying them and training a surrogate on the answers. The stolen copy also enables white-box attacks against the original.

4. Model inversion. Fredrikson et al. (2015) reconstructed recognisable face images of individuals from a face-recognition model using its confidence scores.

5. Membership inference. Shokri et al. (2017): models are more confident on examples they trained on, so high confidence on the true label suggests membership. Overfitting is the main driver. Carlini et al. (2021) extracted verbatim memorised training text — including personal contact details — from GPT-2.

`code/attacks_demo.py`:

```
# poisoning: random vs targeted (class 1 relabelled as 0)
y_tgt[rng.choice(ones, int(frac * len(ones)), replace=False)] = 0
# extraction: query the victim, train a copy on its answers
answers = victim.predict(Q); stolen = DecisionTreeClassifier(max_depth=10).fit(Q, answers)
# membership inference: confidence on the true label, members vs non-members
conf_in = model.predict_proba(X_in)[np.arange(len(y_in)), y_in]
auc = roc_auc_score(np.r_[np.ones(len(conf_in)), np.zeros(len(conf_out))], np.r_[conf_in, conf_out])
```

Output:

```
1) label-flip poisoning (logistic regression, test accuracy / recall of class 1)
   poisoned  0%:  random flips 0.843/0.838   targeted 1->0 0.843/0.838
   poisoned 10%:  random flips 0.846/0.835   targeted 1->0 0.841/0.778
   poisoned 20%:  random flips 0.843/0.854   targeted 1->0 0.812/0.689
   poisoned 30%:  random flips 0.840/0.835   targeted 1->0 0.769/0.574

2) model extraction - victim test accuracy 0.870
     100 queries: agreement with victim - random queries 0.659, look-alike queries 0.820
    1000 queries: agreement with victim - random queries 0.720, look-alike queries 0.892
    5000 queries: agreement with victim - random queries 0.845, look-alike queries 0.933

3) membership inference (attack = 'high confidence on the true label means member')
   overfit forest (no depth limit)  train acc 1.000  test acc 0.902  attack AUC 0.842
   regularised forest (depth 4)     train acc 0.896  test acc 0.851  attack AUC 0.556
```

Reading it: **random** label noise barely hurts, but **targeted** flipping of 10–30% of positives drags recall from 0.84 to 0.57, so a fraud filter can be taught to miss exactly the attacker's class. **Extraction** reaches 93% agreement with 5,000 realistic queries. **Membership inference** is strong (AUC 0.84) against an overfit model and near chance (0.56) after regularisation: generalisation is also a privacy defence.

6. Generative-AI misuse. Deepfakes (fake video, audio or images impersonating real people for fraud, blackmail or non-consensual imagery) and cheap **misinformation at scale** are harms of a different kind. Responses combine detection classifiers (an arms race), **provenance**

standards such as C2PA "Content Credentials", watermarking (e.g. Google DeepMind's SynthID), platform policies, labelling duties such as the EU AI Act's (§19.9), and media literacy.

Defences at a glance

Threat	Defences
Evasion	Adversarial training (PGD), input preprocessing (weak alone), certified robustness (randomised smoothing), ensembles, human review for high-stakes cases
Poisoning / backdoors	Data provenance and access control, outlier and label-noise detection, robust training, dataset versioning, backdoor scanning
Extraction	Rate limits, authentication, returning labels instead of full probabilities, query-pattern monitoring, watermarking the model
Inversion / membership	Regularisation, DP-SGD, limiting confidence outputs, deduplicating training data
Generative misuse	Provenance and watermarking, usage policies, abuse monitoring, red-teaming

Common mistakes

- Evaluating robustness only against weak attacks (e.g. FGSM) — test with PGD and adaptive attacks.
- Relying on "gradient masking" defences that only hide gradients; they are routinely broken.
- Returning full-precision probability vectors from public APIs when a label would do.
- Training on unvetted scraped or user-feedback data with no provenance.

Real-world examples

Spam and malware detection are adversarial by nature (attackers constantly probe filters). MITRE ATLAS catalogues real attacks on ML systems; the OWASP Top 10 for LLM Applications lists data/model poisoning and sensitive-information disclosure among its risks.

In the interview

> **30-second answer:** "Evasion uses adversarial examples — FGSM moves each input feature by ε along the sign of the loss gradient — and is countered by adversarial training. Poisoning corrupts training data or plants backdoors; defend with provenance and anomaly checks. Through a query API attackers can extract, invert or probe the model for membership; rate limits, label-only outputs, regularisation and DP help."

- *Why do adversarial examples exist?* → High-dimensional near-linear models: many tiny aligned changes add up to a large logit change.
- *Cost of adversarial training?* → Slower training (extra attack steps), sometimes lower clean accuracy, and robustness only within the trained budget.

Self-check

1. FGSM uses the gradient with respect to what? → The input, not the weights.
2. Which is more damaging: random 20% label noise or targeted 20% flipping of one class? → Targeted (recall fell to 0.69 vs ~0.85).
3. Two cheap defences against model extraction? → Rate limiting / authentication and returning only top labels.

TL;DR

Adversarial training against evasion; provenance against poisoning; rate limits and label-only outputs against extraction; regularisation and DP against membership inference; provenance and policy against generative misuse.

19.8 Documentation: model cards and datasheets

What it is

- A **model card** (Mitchell et al., 2019) is a short document shipped with a model describing what it is for, how it was evaluated — *broken down by group* — and its limits.
- A **datasheet for datasets** (Gebru et al., 2018; published in *Communications of the ACM* in 2021) documents how a dataset was created, what's in it and how it should (not) be used — modelled on electronics datasheets.

Why it matters

Most harms come from a model being used outside the conditions it was built for. Written, versioned documentation makes intended use, known gaps and evaluation results visible to downstream teams, auditors and regulators (the EU AI Act requires technical documentation for high-risk systems).

How it works

Model card sections (following Mitchell et al.):

Section	Our credit model
Model details, intended use	Logistic regression v1.3, owner, date; pre-screening loan applications with human review, *not* employment or insurance
Factors, metrics	Groups: sex, age band, region and intersections; AUC, per-group TPR/FPR, disparate impact, calibration
Training and evaluation data	Sources and time range, known label issues; held-out 2024 applications (n = 6,000)
Quantitative analyses	Per-group metrics with confidence intervals
Ethics, caveats	Proxy features (zip region), mitigation applied, residual gaps; re-audit quarterly, not for other populations

Datasheet questions are grouped as *motivation* (why, by whom, funded by whom), *composition* (contents, gaps, sensitive data, identifiability), *collection process* (how, consent), *preprocessing and labelling*, *uses*, *distribution* and *maintenance*.

Common mistakes

- Writing it once for launch and never updating it — keep it in the repo, generate the metric tables in the training pipeline, and version it with each release.
- Only aggregate metrics — the whole point is disaggregated (per-group) results.
- Omitting "out-of-scope uses".

Real-world examples

Hugging Face model cards (a README on every model page), Google's Model Card Toolkit, and system cards published by AI labs for large language models; Hugging Face also supports dataset cards.

In the interview

> **30-second answer:** "A model card documents intended use, out-of-scope uses, training and evaluation data, per-group metrics and limitations; a datasheet documents a dataset's motivation, composition, collection, labelling, uses and maintenance. I generate the metric sections automatically in the pipeline and version them with the model."

Self-check

1. What distinguishes a model card's metrics from an ordinary eval report? → They are disaggregated by group and conditions.
2. Name three datasheet sections. → Motivation, composition, collection process (also preprocessing, uses, distribution, maintenance).

TL;DR

Model cards and datasheets make intended use, data provenance and per-group performance explicit and versioned.

19.9 Regulation overview

This is an engineer's orientation, not legal advice. Laws change — check the official texts and your legal team.

What it is and why it matters

Laws govern both the **data** ML uses (privacy law) and the **systems** it produces (AI and anti-discrimination law); engineering artefacts — logs, human review, documentation, explanations — are often what compliance actually requires.

GDPR (EU General Data Protection Regulation, applies since 25 May 2018)

- Covers **personal data** of people in the EU, including processing by non-EU companies serving them. Truly anonymous data is out of scope; pseudonymised data is not.
- **Principles** include purpose limitation, **data minimisation**, accuracy, storage limitation and accountability; every use needs a **lawful basis** (consent, contract, legitimate interests…), with stricter rules for "special categories" such as health, biometrics and ethnicity.
- **Rights**: access, rectification, **erasure** ("right to be forgotten"), restriction, portability, objection.
- **Article 22**: people have the right not to be subject to decisions based *solely* on automated processing with legal or similarly significant effects; the exceptions (contract, law, explicit consent) require safeguards such as human intervention and the chance to give their view and contest. Articles 13–15 require "meaningful information about the logic involved"; whether that is a full "right to explanation" is debated.
- High-risk processing needs a **DPIA** (data protection impact assessment); fines reach €20 million or 4% of worldwide annual turnover, whichever is higher.

EU AI Act (Regulation (EU) 2024/1689)

Entered into force on 1 August 2024, with obligations phased in. It classifies AI systems by **risk**:

Tier	Examples	Obligations
Unacceptable (prohibited)	Social scoring; harmful manipulation or exploitation of vulnerabilities; untargeted scraping of face images; emotion recognition at work or school; biometric inference of sensitive traits; crime prediction purely from profiling; real-time remote biometric identification in public by police (narrow exceptions)	Banned (applied from 2 February 2025)
High risk	Safety components of regulated products (Annex I), and Annex III areas: biometrics, critical infrastructure, education, **employment**, access to essential services including **credit scoring**, law enforcement, migration, justice	Risk management, data governance (including bias checks), documentation, logging, **human oversight**, accuracy and robustness, conformity assessment, registration
Transparency (limited) risk	Chatbots, AI-generated or manipulated content (deepfakes), emotion recognition	Tell people they're interacting with AI; label synthetic content
Minimal risk	Spam filters, game AI, most business ML	No new obligations (voluntary codes)

Separately, providers of **general-purpose AI models** (e.g. large language models) have documentation and copyright-policy duties from 2 August 2025, with extra duties for models posing systemic risk. Top fines reach €35 million or 7% of worldwide turnover for prohibited practices.

Timeline update (2026). The EU's "Digital Omnibus on AI" (Regulation (EU) 2026/1744, in force since 27 July 2026) postponed the high-risk obligations: to **2 December 2027** for stand-alone Annex III systems and **2 August 2028** for AI in Annex I products. It also added prohibitions on AI systems that generate non-consensual intimate imagery or child sexual abuse material (applying from 2 December 2026), and gave generative systems placed on the market before 2 August 2026 until 2 December 2026 to meet the Article 50 duty to mark their output as AI-generated. Check the consolidated text for current dates.

Other rules worth knowing

- **CCPA/CPRA (California)**: rights to know, delete, correct and opt out of the sale or sharing of personal information.
- **US anti-discrimination law**: disparate *treatment* (using a protected attribute) and disparate *impact* (a neutral rule with unequal effect and no business necessity); under ECOA/Regulation B lenders must give specific denial reasons, and the CFPB says complex algorithms are no excuse (Circular 2022-03).
- **NYC Local Law 144** (enforced from July 2023): automated employment decision tools used for NYC jobs need an annual independent bias audit, published results, and candidate notice.
- **Sector rules** such as HIPAA (US health data) and model-risk-management guidance for banks (US SR 11-7).
- **Frameworks** (voluntary): NIST AI Risk Management Framework (2023), ISO/IEC 42001 (AI management systems, 2023).

In the interview

30-second answer: "GDPR governs personal data: lawful basis, minimisation, rights such as erasure, and Article 22 safeguards for solely automated significant decisions. The EU AI Act is risk-based: some uses are banned; high-risk systems such as credit scoring and hiring need risk management, data governance, documentation, logging and human oversight; chatbots and deepfakes carry transparency duties. In the US, fair-lending and employment law (adverse-action reasons, disparate impact, NYC's bias audits) shape design."

Self-check

1. Is a hiring-screening model high risk under the EU AI Act? → Yes, employment is an Annex III area.
2. Is pseudonymised data covered by GDPR? → Yes.
3. What does Article 22 give people? → Protection from solely automated significant decisions, with rights to human intervention and to contest.

TL;DR

GDPR governs personal data (lawful basis, minimisation, rights, Art. 22); the EU AI Act sorts systems into risk tiers, with credit and hiring high-risk; US fair-lending and employment law add their own duties.

19.10 Real-world incidents

COMPAS recidivism scores (2016)

- **What happened**: ProPublica analysed COMPAS risk scores (by Northpointe, now Equivant) for defendants in Broward County, Florida, checking who was re-arrested within two years. Among defendants who did **not** reoffend, Black defendants were labelled higher risk about twice as often as white defendants (44.9% vs 23.5%); among those who **did** reoffend, white defendants were more often labelled lower risk (47.7% vs 28.0%).
- **The response**: Northpointe argued the scores were equally predictive for both groups — a given score corresponded to similar reoffence rates (calibration/predictive parity).
- **Lesson**: *both sides were right about their own metric*. With different base rates, calibration and equal error rates cannot both hold — this debate triggered the impossibility proofs (§19.2). Also: "re-arrest" is a proxy label shaped by policing (measurement bias).

Amazon's experimental résumé-screening tool (reported 2018)

- **What happened**: Reuters reported that from 2014 Amazon built a tool to rate candidates, trained on ten years of résumés, mostly from men. It learned to penalise the word "women's" (as in "women's chess club captain") and downgraded graduates of two all-women's

colleges. Engineers removed those terms but could not rule out other proxies, and the team was disbanded; Amazon said recruiters never used the tool to evaluate candidates.
- **Lesson**: historical bias in labels, and proxies — removing explicit terms doesn't remove the pattern.

Apple Card credit limits (2019–2021)

- **What happened**: In November 2019 entrepreneur David Heinemeier Hansson complained publicly that his credit limit was 20 times his wife's despite shared finances; Steve Wozniak reported a similar experience. New York's Department of Financial Services (NYDFS) investigated Goldman Sachs, the issuer.
- **Outcome**: In March 2021 the NYDFS found **no violation of fair-lending law** (individual credit histories explained the differences) but criticised the lack of transparency and poor customer service.
- **Lesson**: even a lawful model loses trust if it cannot explain its decisions; spouses with shared finances can still have very different individual credit files.

Gender Shades and the Obermeyer health-risk algorithm (§19.1) complete the set of cases interviewers most often cite.

> **Self-check**

1. Which fairness criterion did ProPublica test, and which did Northpointe defend? → Equal error rates (FPR/FNR) vs calibration/predictive parity.
2. What did the NYDFS conclude about Apple Card? → No fair-lending violation, but transparency and service shortcomings.

> **TL;DR**

COMPAS = the impossibility result in real life; Amazon = historical bias and proxies; Apple Card = explainability and trust.

19.11 Responsible-ML checklist

Use this before launch and at every major retrain.

Problem framing

- [ ] Is ML appropriate? Who could be harmed? High-risk under the EU AI Act or similar law?
- [ ] Does the label measure what we care about? Which fairness criterion fits the harm (agreed with legal/domain experts)?

Data

- [ ] Datasheet: source, lawful basis, collection, labelling, known gaps; coverage audit per group and intersection.
- [ ] PII minimised and access-controlled; sensitive attributes stored separately for auditing; provenance and poisoning checks.

Model and evaluation

- [ ] Per-group metrics with confidence intervals; proxy check; mitigation chosen, its cost measured, all criteria re-audited.
- [ ] Explanations reviewed for leakage and proxies; reason codes where decisions affect people.
- [ ] Robustness tests where attackers exist; membership-inference risk; DP/FL where needed.

Deployment and monitoring

- [ ] Model card with intended and out-of-scope uses; human oversight and an appeal path.
- [ ] API hardening (authentication, rate limits, minimal outputs).
- [ ] Per-group monitoring, drift and feedback-loop checks; incident and rollback plan; scheduled re-audits.

19.12 Interview Q&A

The sections above each end with their own interview answers; these are the cross-cutting questions that combine them.

1. **Your model is 94% accurate. Is it fair?** → Unknown until you slice by group: selection rate, TPR, FPR, PPV and calibration per group, the gaps, and a proxy check. Accuracy alone hides group disparities.
2. **Explain SHAP to a product manager.** → It splits one prediction into per-feature contributions that add up from the average prediction to this one: "income added +0.18, a prior default took away −0.09".
3. **Is hashing user IDs anonymisation?** → No, it is pseudonymisation; linkage attacks re-identify people and GDPR still applies.
4. **Someone is scraping our pricing model's API. Risks and defences?** → Model extraction, which then enables white-box attacks. Rate limits, authentication, label-only responses, query-pattern monitoring, contractual terms.
5. **Design question: build a fair hiring screener.** → Question whether to automate at all; define the label carefully (job performance, not past hiring decisions); audit data coverage; choose a criterion (often selection-rate parity plus equal TPR) with legal input; mitigate and re-audit; give candidates reasons; keep human review; run bias audits (NYC LL 144 if applicable) and meet EU AI Act high-risk duties; monitor per-group outcomes after launch.

Chapter TL;DR

- **Bias** enters through history, sampling, measurement, labels, aggregation, evaluation, deployment and feedback loops; a biased label can't be fixed by a better model.
- **Fairness criteria** (parity, equal opportunity, equalized odds, predictive parity, calibration) conflict when base rates differ — choose by harm, audit per group, test for proxies.
- **Mitigate** before (reweighing, better data), during (penalties, constraints) or after training (group thresholds); measure the accuracy cost and re-audit.
- **Explain** globally (permutation importance, PDP/ICE) and locally (SHAP, LIME, counterfactuals) — never as causal claims.
- **Privacy**: pseudonymised ≠ anonymous; k-anonymity + l-diversity; differential privacy (noise $\Delta f/\varepsilon$, budgets compose), DP-SGD, federated averaging with secure aggregation.
- **Security**: evasion (FGSM/PGD, adversarial training), poisoning, extraction, inversion, membership inference, generative misuse.
- **Document and comply**: model cards, datasheets, GDPR Art. 22, EU AI Act risk tiers, US fair-lending and hiring-audit rules. Remember COMPAS, Amazon and Apple Card.

Chapter 20 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–19 into pages you can scan in a minute or two: one **cheat sheet** per topic, **decision tables** for "which algorithm / which metric for this problem?", the **50 interview questions** that come up most often with short answers, and an alphabetical **glossary** of 348 terms. Nothing here is new. Every line points back to where the idea is taught properly, written like **§6.3** (= Chapter 6, section 6.3); if a line doesn't make sense, follow the reference.

Every formula or number on a sheet that can be computed was recomputed by `code/cheatsheet_checks.py` (it also asserts the library defaults quoted here, for scikit-learn 1.9, PyTorch 2.x and NumPy 2.x):

```
OK  sheet_20_2_math
OK  sheet_20_3_api
OK  sheet_20_4_data_prep
OK  sheet_20_5_algorithms
OK  sheet_20_6_losses
OK  sheet_20_7_metrics
OK  sheet_20_8_validation
OK  sheet_20_9_deep_learning
OK  sheet_20_10_cnn_rnn_attention
OK  sheet_20_11_recommenders
OK  sheet_20_12_rl
OK  sheet_20_13_system_design
OK  sheet_20_14_production
OK  sheet_20_15_responsible
all 14 cheat-sheet checks passed
```

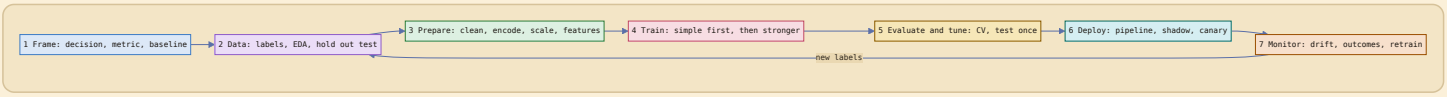
What's inside

#	Sheet	Built from
20.1	The ML workflow	Ch 1, 4, 6, 15, 16
20.2	Math essentials (key formulas)	Ch 2
20.3	NumPy, pandas and scikit-learn API	Ch 3, 4
20.4	Data preparation and feature engineering	Ch 4
20.5	Every algorithm in one table	Ch 5, 7–14
20.6	Loss functions	Ch 2, 5, 8, 10, 13, 14
20.7	Metrics: regression, classification, ranking and more	Ch 6, 9, 11, 13, 16
20.8	Validation and tuning	Ch 4, 6
20.9	Deep learning: layers, activations, optimizers, regularisation, debugging	Ch 10
20.10	CNN, RNN and attention: shapes and formulas	Ch 11, 12
20.11	Recommenders	Ch 13, 17
20.12	Reinforcement-learning update rules	Ch 14
20.13	ML system design: the 7-step framework and napkin math	Ch 15, 17, 18
20.14	Production and MLOps checklist	Ch 16
20.15	Responsible-ML checklist	Ch 19
20.16	Decision tables: which algorithm, which metric	all
20.17	Top 50 interview questions	all
20.18	Glossary (A–Z)	all

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19

Notation used throughout: x = feature vector, y = true label, ŷ = prediction, w = weights, b = bias, η = learning rate, σ = sigmoid, n = number of examples, d = number of features, k = number of neighbours / clusters / results (the context says which).

20.1 The ML workflow (§1.9, §4.10, §6.11, §15.3, §16.1)



Step	Do	Don't	Where
1. Frame	Name the decision the prediction drives, the task type (classification / regression / ranking / clustering), the metric tied to business value, and the **baseline** (current rule, human, "predict the mean")	Start by picking a model	§1.9, §15.3
2. Data	Find where labels come from and how late they arrive; set the test split aside **before** deep EDA; check legality and privacy	Explore the test set; assume labels "exist"	§4.1, §4.2
3. Prepare	Dedupe, impossible values → NaN, impute, encode, scale — all inside one `Pipeline`	Fit anything on the full dataset (leakage)	§4.3–4.6, §4.10
4. Train	Linear/logistic model first, then a random forest and gradient boosting; deep nets for images/text/audio	Jump to the fanciest model	§5.1, §8.4
5. Evaluate	Cross-validate on train, tune on validation folds, report the test score **once**, next to the baseline, per segment	Tune on the test set; quote accuracy on imbalanced data	§6.1, §6.6, §6.9
6. Deploy	Ship preprocessing + model as one artifact; offline gate → shadow → canary → ramp, with a fallback	Re-implement features in another language (skew)	§16.2, §16.6
7. Monitor	Health → inputs → predictions → outcomes; investigate alerts; retrain through the same gates	Auto-promote on a drift alert	§16.8, §16.10

Diagnosing a model from two numbers (§1.10, §6.7):

Train error	Validation error	Diagnosis	First fixes
High	High (close)	Underfitting (high bias)	More capacity, better features, less regularisation, train longer
Low	Much higher	Overfitting (high variance)	More data, more regularisation, simpler model, early stopping, bagging
Low	Low (close)	Good fit	Test once, ship
Suspiciously perfect	Suspiciously perfect	Probably **leakage**	Check feature timestamps, splits, duplicates, top importances (§4.8)

Supervision types (§1.4): supervised (labels) · unsupervised (no labels: clusters, dimensions, anomalies) · semi-supervised (few labels + many unlabelled) · self-supervised (labels made from the data, e.g. next token) · reinforcement (rewards from actions). **Classification** predicts a category, **regression** a number (§1.5). **Batch** learning retrains on everything periodically; **online** learning updates with `partial_fit` (§1.7). **Instance-based** (k-NN) keeps the data; **model-based** keeps parameters (§1.8).

20.2 Math essentials (Ch 2)

Linear algebra (§2.1–2.3)

Idea	Formula	Worked value
Dot product	$a \cdot b = \Sigma\, a_i b_i = \|a\|\|b\|\cos\theta$	$[1,2,3]\cdot[4,5,6] = \mathbf{32}$
L2 / L1 / L∞ norm	$\sqrt{\Sigma a_i^2}$ · $\Sigma\|a_i\|$ · $\max\|a_i\|$	$\|[1,2,3]\|_2 = \sqrt{14} \approx 3.74$
Cosine similarity	$a \cdot b \,/\, (\|a\|\|b\|)$	$32 / (3.74 \cdot 8.77) \approx \mathbf{0.975}$
Linear model	$\hat{y} = w \cdot x + b$; batch: $\hat{y} = Xw + b$	shapes $(n\times d)(d,) \to (n,)$
Matrix product	$(m\times n)(n\times p) \to m\times p$; entry = row·column; **AB ≠ BA**	$(100\times 20)(20\times 5) \to 100\times 5$
2×2 determinant / inverse	$\det = ad - bc$; $A^{-1} = (1/\det)[[d, -b], [-c, a]]$	$[[1,2],[3,4]]$: det −2, inverse $[[-2,1],[1.5,-0.5]]$
Solve Ax = c	`np.linalg.solve(A, c)` (never `inv`)	$x_1+2x_2=5$, $3x_1+4x_2=11 \to x = [1, 2]$
Eigenvector	$Av = \lambda v$	$[[2,1],[1,2]]$: $\lambda = 3$ on $[1,1]$, $\lambda = 1$ on $[1,-1]$
SVD	$M = U\Sigma V^T$ (rotate, stretch, rotate)	top-k singular values = best rank-k approximation
Normal equation	$\theta = (X^TX)^{-1}X^Ty$ (use `lstsq`)	Ridge: $(X^TX + \alpha I)^{-1}X^Ty$

Calculus and optimisation (§2.4–2.5)

Idea	Formula
Power / chain rule	$(x^n)' = nx^{n-1}$ · $(f(g(x)))' = f'(g(x))\cdot g'(x)$
Sigmoid and derivative	$\sigma(z) = 1/(1+e^{-z})$; $\sigma' = \sigma(1-\sigma)$; $\sigma(0) = 0.5$, $\sigma'(0) = 0.25$ (its maximum)
Gradient	∇f = vector of partial derivatives; points **uphill**
Gradient descent	$w \leftarrow w - \eta \nabla L(w)$. On $f = (w-3)^2$, $\eta = 0.1$: 0 → 0.6 → 1.08 (each step keeps 80% of the gap)
MSE gradient	$\nabla = (2/n)X^T(Xw - y)$
Log-loss gradient	$\nabla = (1/n)X^T(\sigma(Xw) - y)$ — "features × (prediction − truth)"
Numerical gradient check	$f'(x) \approx (f(x+h) - f(x-h)) / 2h$
Convergence (MSE)	GD converges if $\eta < 2/\lambda_{max}$ of the Hessian (§5.4)

Probability and statistics (§2.6–2.11)

Idea	Formula / rule	Worked value
Bernoulli / binomial / normal	mean p, var p(1−p) · mean np, var np(1−p) · mean μ, var σ^2	P(3 of 10, p = 0.3) = $C(10,3)\cdot 0.3^3\cdot 0.7^7 \approx$ **0.2668**
68–95–99.7 rule	±1σ, ±2σ, ±3σ of a normal	P(z > 2) ≈ 2.3%
Expectation rules	$E[aX+b] = aE[X]+b$; $Var[aX+b] = a^2Var[X]$; independent: $Var[X+Y] = Var\ X + Var\ Y$	$Var[3X] = 9\cdot Var[X]$
Conditional probability	$P(A\|B) = P(A\cap B)/P(B)$	
Bayes	$P(A\|B) = P(B\|A)P(A)/P(B)$ — posterior ∝ likelihood × prior	1% disease, 90% sensitivity, 5% FPR → P(sick|+) ≈ **0.154**; a 2nd positive → 0.766
z-score	$z = (x - \mu)/\sigma$	130 with μ 100, σ 15 → 2
Standard error (CLT)	$\sigma/\sqrt{n}$; 4× the data halves it	σ 20, n 100 → 2
Population vs sample std	NumPy `ddof=0` (÷n), pandas `ddof=1` (÷(n−1))	
Hypothesis test	p-value = P(data this extreme | H_0); reject if $p < \alpha$; Type I = α, power = 1 − β	p is **not** $P(H_0$ true)
Confidence interval	estimate ± critical value × SE (1.96 for 95%, large n)	bootstrap: resample, take 2.5th–97.5th percentiles
Multiple testing	Bonferroni: use α/m	20 tests at 0.05 → expect 1 false hit
Correlation	Pearson r (linear), Spearman (ranks, monotonic); r = 0 ≠ independent	$y = x^2$ can give r = 0

Information theory (§2.12)

Idea	Formula	Worked value (bits)
Entropy	$H(p) = -\Sigma\ p \log p$	fair coin 1; 90/10 coin 0.469; p = [0.7, 0.2, 0.1] → 1.157
Cross-entropy	$H(p, q) = -\Sigma\ p \log q \geq H(p)$	q = [0.5, 0.3, 0.2] → 1.280
KL divergence	$KL(p\|\|q) = H(p,q) - H(p) \geq 0$, **not symmetric**	0.123
Log loss (one-hot p)	$-(1/n)\ \Sigma$ [y ln p + (1−y) ln(1−p)]	labels [1,0,1,1], p [0.9,0.2,0.6,0.99] → **0.2123** (nats)

20.3 NumPy, pandas and scikit-learn API (Ch 3, §4.10)

NumPy (§3.2)

Task	Code	Remember
Create	`np.array`, `zeros`, `ones`, `eye`, `arange`, `linspace(0, 1, 5)`	one dtype per array; `float32` = 4 bytes, `float64` = 8
Inspect	`a.shape`, `a.dtype`, `a.ndim`, `a.size`, `a.nbytes`	shape `(n,)` ≠ `(n, 1)`
Reshape	`a.reshape(3, 4)`, `a.reshape(-1, 1)`, `a[:, None]`, `a.T`	scikit-learn wants 2-D `X`
Index	`M[0]`, `M[:2, 1:3]` → **view**; `M[M > 5]`, `M[[0, 2]]` → **copy**	`.copy()` when you need independence
Reduce	`M.sum(axis=0)` (one per column), `axis=1` (one per row), `keepdims=True`	the named axis disappears
Broadcast	align shapes from the right; equal or 1 → OK	(3,1)+(1,4) → (3,4); (3,2)+(3,) → error
Matrix product	`A @ B` (not `*`, which is element-wise)	
Standardise columns	`(X - X.mean(axis=0)) / X.std(axis=0)`	fit statistics on training data only
Random	`rng = np.random.default_rng(42)`; `rng.normal`, `rng.integers`, `rng.permutation`	seed once, draw everything from `rng`
Floats	`np.isclose(0.1 + 0.2, 0.3)`	`int8` 127 + 1 wraps to −128

pandas (§3.3)

Task	Code
Load / first look	`pd.read_csv(path)`; `df.head()`, `df.info()`, `df.dtypes`, `df.describe()`, `df.isnull().sum()`, `df["c"].value_counts()`
Fix types	`pd.to_numeric(s, errors="coerce")`, `pd.to_datetime(s)`, `s.astype("category")`
Select	`df["c"]` (Series), `df[["a", "b"]]` (DataFrame), `df.loc[rows, cols]` (labels, **end inclusive**), `df.iloc[0:2]` (positions, end exclusive)
Filter	`df[(df.a > 1) & (df.b == "x")]` — `&`, `|`, `~` with parentheses
Missing values	`dropna()`, `fillna(v)`, `df.groupby(g)["x"].transform("median")`, `df["x_missing"] = df.x.isna()`
Group	`df.groupby("g").agg(n=("x", "size"), mean_x=("x", "mean"))` (one row per group); `transform` keeps the original length
Join / stack	`a.merge(b, on="k", how="left", validate="many_to_one")`; `pd.concat([a, b])`
Vectorise	column arithmetic, `np.where`, `pd.cut`, `.str`, `.dt` — avoid row-wise `apply`
pandas 3	copy-on-write: assign (`df["c"] = df["c"].fillna(0)`), don't chain in-place calls

scikit-learn estimator API (§3.5, §4.10)

Convention	Meaning
`Model(hyper=...)`	hyperparameters go in the constructor
`fit(X, y)` → `self`	learns; learned attributes end in `_` (`coef_`, `mean_`, `classes_`)
`transform` / `fit_transform`	`fit_transform` on **train only**; `transform` on validation/test/production
`predict`, `predict_proba`, `decision_function`, `score`	`score` = accuracy (classifiers), R^2 (regressors)
`Pipeline` / `make_pipeline`	preprocessing + model = one estimator; refit per CV fold (no leakage), one artifact to save
`ColumnTransformer`	different preprocessing per column group; default `remainder="drop"`
`step__param`	address a parameter inside a pipeline, e.g. `prep__num__impute__strategy`, `model__C`
`set_output(transform="pandas")`	named DataFrame outputs for debugging
`joblib.dump(pipe, "m.joblib")`	save the whole pipeline; only load files you trust

```
pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler()), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
pipe = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=1000))])
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1, 10]}, cv=StratifiedKFold(5), scoring="average_precision")
search.fit(X_train, y_train)                # CV refits the preprocessing inside every fold
print(search.best_params_, search.score(X_test, y_test))   # test set used once
```

Defaults worth knowing (checked by the script): `LogisticRegression` C = 1.0 (L2), solver `lbfgs` · `RandomForestClassifier` `max_features="sqrt"` (regressor: 1.0) · `HistGradientBoosting*` learning_rate 0.1, `max_leaf_nodes` 31, `min_samples_leaf` 20 · `SVC` RBF kernel, C = 1, `gamma="scale"` · `KNeighbors*` k = 5, Minkowski p = 2 · `KMeans` `init="k-means++"`.

Module	Tools you'll use constantly
`model_selection`	`train_test_split`, `cross_val_score`, `cross_validate`, `KFold`, `StratifiedKFold`, `GroupKFold`, `TimeSeriesSplit`, `GridSearchCV`, `RandomizedSearchCV`
`preprocessing`	`StandardScaler`, `MinMaxScaler`, `RobustScaler`, `OneHotEncoder`, `OrdinalEncoder`, `TargetEncoder`, `PolynomialFeatures`, `KBinsDiscretizer`, `FunctionTransformer`
`impute`	`SimpleImputer`, `KNNImputer`, `IterativeImputer`
`metrics`	`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `average_precision_score`, `log_loss`, `brier_score_loss`, `mean_absolute_error`, `mean_squared_error`, `r2_score`, `ndcg_score`, `confusion_matrix`, `classification_report`
`calibration`	`CalibratedClassifierCV`, `calibration_curve`
`inspection`	`permutation_importance`, `PartialDependenceDisplay`

Frameworks (§3.7): scikit-learn (and XGBoost / LightGBM / CatBoost) for tables; PyTorch or TensorFlow/Keras for images, text, audio, GPUs and custom networks. Jupyter: seed first, Restart & Run All before sharing, move stable code into `.py` modules (§3.6).

20.4 Data preparation and feature engineering (Ch 4)

Labels (§4.1, §16.3)

Source	Speed	Weakness	Handle with
Natural outcomes (delivery time, chargeback)	Minutes → weeks	Late, incomplete, outcome ≠ truth	Train on mature rows; proxy labels for fresh data
Behaviour / implicit (clicks, watch time)	Seconds	Exposure and position bias	Log impressions and positions; correct bias (IPW)
Human annotation	Hours → days	Cost, reviewer quality	Versioned guidelines, Cohen's κ = (p_o − p_e)/(1 − p_e), gold questions, active learning
Weak supervision (labelling functions + label model)	Instant	Noisy, correlated rules	Snorkel-style label model; audit
LLM-assisted	Seconds	Correlated, systematic errors	Humans review samples, hard cases and per-subgroup slices

κ example: agreement 0.8, chance 0.5 → κ = **0.6**. Retrieval negatives: random (easy), in-batch (cheap, popularity-biased → logQ correction), hard (sharp, some false negatives).

EDA checklist (§4.2): shape & types → missingness (and is it related to the target?) → distributions (skew, sentinels like 999 / 9,999,999, cardinality) → target balance → feature–target and feature–feature relationships (**Spearman** for skewed data) → duplicates, impossible values, leaky IDs. Do detailed EDA on the training split.

Cleaning (§4.3)

Problem	Default fix
Exact duplicates	`drop_duplicates()`, then check the entity key is unique; dedupe **across** splits
Impossible values (age 999)	Set to NaN — they are errors, not outliers
Outliers	IQR fences (Q1 − 1.5·IQR, Q3 + 1.5·IQR), |z| > 3, or log first; cap at training-set percentiles; never blindly delete (fraud *is* the outlier)
Missing: MCAR / MAR / MNAR	Median (numbers) or `"missing"` (categories) **+ missing indicator**; KNN or iterative imputation when other columns predict it; HistGradientBoosting/XGBoost/LightGBM handle NaN natively
Judging an imputer	Hide known values, measure error, and compare downstream CV scores

Encoding (§4.4)

Encoder	Use for	Risk
One-hot (`handle_unknown="ignore"`)	Nominal, low–moderate cardinality	Width; dummy trap for unregularised OLS (`drop="first"`)
Ordinal	Ordered categories; fine for trees	Fake order for linear models / k-NN
Target encoding (`TargetEncoder`)	High cardinality	**Leakage** unless cross-fitted and smoothed
Hashing (`FeatureHasher`)	Unbounded IDs, streaming	Collisions, not reversible
Embeddings	IDs in deep models / recommenders	Needs data per ID; versioning

Scaling (§4.5)

Scaler	Formula	Use when
Standard	(x − mean)/std	Default for linear models, SVMs, neural nets, PCA, k-NN, k-means
Min-max	(x − min)/(max − min)	Bounded inputs (pixels ÷ 255), no big outliers
Robust	(x − median)/IQR	Outliers you keep: [10,11,11,12,12,13,100] → 100 ↦ (100 − 12)/1.5 = 58.67
Log / power	log1p(x), `PowerTransformer`	Heavy right tails (income, counts) — scaling alone never fixes skew
None	—	Trees and tree ensembles

Feature creation (§4.6)

Technique	Example
Interactions / polynomials	price × quantity; `PolynomialFeatures(2)` on 10 features → 65 columns
Binning	age buckets let a linear model fit a U-shape
Log transform	`log1p(income)`
Dates and cycles	day of week, is_weekend, "days since signup"; hour h → (sin 2πh/24, cos 2πh/24): hour 6 → (1, 0)
Text	Bag-of-words, TF-IDF: idf(t) = ln((1+n)/(1+df)) + 1, rows L2-normalised (n = 4, df = 2 → 1.511)
Aggregates and ratios	orders in last 30 min, tickets per month of tenure — must be **point-in-time correct**
Production groups	**entity** (batch), **behavioural aggregates** (streaming), **context** (request time), **cross** (pairs) — §4.6, §16.4

Splits (§4.7): random (i.i.d. data) · **stratified** (classification, keeps class ratios) · **time-based** (anything predicting the future) · **group** (all rows of one user/patient together, to generalise to new entities). Millions of rows → 98/1/1 is fine; a few thousand → cross-validation.

Leakage checklist (§4.8, §16.3): (1) would I know this value at prediction time? (2) split first, then fit everything inside a `Pipeline`; (3) time/group splits when deployment needs them; (4) dedupe across splits; (5) aggregates whose window includes the label moment; (6) label-adjacent fields (refund flag, resolution code); (7) distrust any single dominant feature or a score far above published baselines.

Imbalance (§4.9)

Technique	Effect on probabilities
Better metrics (precision, recall, PR-AUC)	—
Threshold moving	None — often all you need
Class weights (`class_weight="balanced"`)	Inflated → recalibrate
Negative downsampling (keep fraction w)	Inflated → **p_true = p·w / (p·w + 1 − p)**; w = 0.1, p = 0.5 → 0.091
Oversampling / SMOTE (a + u·(b − a) between minority neighbours)	Inflated; resample the **training** data only, after splitting

---

20.5 Every algorithm in one table (Ch 5, 7–14)

n = rows, d = features, k = neighbours/clusters, B = trees, M = boosting rounds, T = sequence length, h = hidden width. Complexities are rough orders of magnitude for intuition, not exact bounds.

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
Linear regression (OLS)	$\hat{y} = w \cdot x + b$ minimising MSE	Roughly linear trends, baselines, explaining effects	none (fit_intercept)	Fast, interpretable, extrapolates a trend	Misses curves; outliers; multicollinearity	$O(nd^2 + d^3)$ solver or $O(nd)$/epoch GD / $O(d)$	5.2–5.3
Ridge / Lasso / ElasticNet	OLS + L2 / L1 / both penalty	Many or correlated features; want sparsity (L1)	α (and `l1_ratio`)	Less overfitting; Lasso selects features	Must scale; α tuned by CV	As OLS / $O(d)$	5.7
Polynomial regression	Linear model on powers/products	Smooth curves, few features	degree	Same solvers	Feature explosion; wild at the edges	As OLS on expanded d	5.6
Logistic regression	$p = \sigma(w \cdot x + b)$, log-loss	Binary baseline, calibrated probabilities, sparse text	C (= 1/strength), class_weight	Convex, fast, odds ratios, roughly calibrated	Linear boundary	$O(nd)$ per iteration / $O(d)$	5.8
Softmax regression	K linear scores → softmax	Mutually exclusive classes	C	Coherent probabilities	Linear boundaries	$O(ndK)$ / $O(dK)$	5.9
k-NN	Vote/average of k nearest points	Small low-dimensional data, similarity search	k, metric, weights	No training, any boundary	Slow prediction, needs scaling, curse of dimensionality	$O(1)$ / $O(nd)$ brute force	8.1
Linear SVM	Max-margin hyperplane, hinge loss	High-dimensional sparse text	C	Fast, strong on text	No probabilities	$\sim O(nd)$ / $O(d)$	8.2
Kernel SVM (RBF)	Max margin in an implicit feature space	Small/medium data with curved boundaries	C, γ (tune jointly, log scale)	Convex, flexible, compact (support vectors)	Slow beyond tens of thousands of rows; scaling; no native probabilities	$\sim O(n^2 d)$ to $O(n^3 d)$ / $O(n_{SV} \cdot d)$	8.2
SVR	Fit inside an ε-tube	Small-data regression with noise	C, ε, γ	Ignores small errors	As kernel SVM	As SVM	8.2
Naive Bayes	Bayes + "features independent given class"	Text, tiny data, streaming baselines	α (smoothing), variant (Multinomial/Bernoulli/Gaussian)	Fastest to train, few data needed	Over-confident probabilities, no interactions	$O(nd)$ / $O(dK)$	8.3
Decision tree (CART)	Greedy binary splits maximising impurity decrease	Readable rules, quick look	max_depth, min_samples_leaf, ccp_alpha	Interpretable, no scaling, non-linear	High variance, axis-aligned, no extrapolation	$\sim O(nd \log n)$ / $O(\text{depth})$	7.1
Random forest	Bagged deep trees + random feature subset per split	Strong default with little tuning	n_estimators, max_features, max_depth / min_samples_leaf	Robust, parallel, free OOB score, more trees never hurt	Bigger/slower; less accurate than tuned boosting; no extrapolation	B × tree / $O(B \cdot \text{depth})$	7.3
AdaBoost	Re-weight misclassified samples; weighted vote of stumps	Clean data, simple baseline	n_estimators, learning_rate	Simple	Sensitive to noise/outliers (exponential loss)	M × stump / $O(M)$	7.4
Gradient boosting (sklearn HGB, XGBoost, LightGBM, CatBoost)	Sequential shallow trees fitted to the negative gradient	**Best default for tabular data**	learning_rate, n_estimators (+ early stopping), depth/leaves, min leaf size, subsample, L1/L2	Top tabular accuracy, native NaN/categoricals	Overfits with too many rounds; sequential	M × histogram tree ≈ $O(M \cdot nd)$ / $O(M \cdot \text{depth})$	7.4, 7.7
Voting / stacking	Combine different model families (stacking: meta-model on out-of-fold predictions)	Diverse models with different errors	base models, meta-model, cv	Reduces risk	More latency and upkeep; not guaranteed to win	Sum of models	7.5
k-means	Assign to nearest centroid, move	Round, similar-sized clusters	k, n_init (k-means++)	Fast, scales (MiniBatch)	Must pick k; spherical	$O(nkd)$ per iteration	9.1

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
	centroids, repeat				assumption; outliers		
Agglomerative	Merge closest clusters bottom-up (dendrogram)	Small data, nested structure	linkage, n_clusters or cut height	No k up front, interpretable tree	$O(n^2)$ memory	$\geq O(n^2)$	9.2
DBSCAN / HDBSCAN	Clusters = dense regions; noise = −1	Odd shapes, outliers, unknown k	eps, min_samples (HDBSCAN: min_cluster_size)	Any shape, labels noise	One global density (DBSCAN); eps hard in high-d	~O(n log n) with an index	9.3
Gaussian mixture (EM)	Soft assignment to k Gaussians	Elliptical/overlapping clusters, need probabilities	k (by BIC), covariance_type	Soft, elliptical	Local optima; k to choose	$O(nkd^2)$ per EM step (full cov.)	9.4
PCA	Project onto top eigenvectors of the covariance	Compression, denoising, pre-step, quick 2-D look	n_components (e.g. 0.95 variance)	Fast, deterministic, `transform`	Linear only; scale first	full SVD $\sim O(nd \cdot \min(n,d))$	9.5
t-SNE / UMAP	Non-linear 2-D maps preserving neighbours	Visualisation only	perplexity / n_neighbors, min_dist	Reveals clusters	Sizes and gaps meaningless; seed-dependent	t-SNE ~O(n log n) (Barnes-Hut)	9.6
Isolation Forest	Anomalies isolate in few random splits	Multivariate anomaly default	n_estimators, max_samples (256), contamination	Fast, no scaling	Misses anomalies visible only in combinations	~linear	9.7
LOF / One-Class SVM	Local density vs neighbours / boundary around normal data	Mixed densities / novelty on clean data	k / ν, γ	Local / flexible	$O(n^2)$-ish / slow on big n	—	9.7
Apriori / FP-Growth	Frequent itemsets → rules ranked by lift	Market-basket analysis	min_support, min_confidence	Readable rules	Many trivial rules	Grows with candidates	9.8
MLP (neural network)	Layers of $h = f(Wx + b)$, trained by backprop	Large data; building block of deep nets	depth, width, lr, batch size, dropout, weight decay	Learns features, very flexible	Data-hungry, many knobs; usually loses to GBDT on small tables	O(params) per example per step	10.1
CNN	Shared convolution kernels + pooling	Images, spatial grids	depth, channels, kernel size, augmentation	Few parameters, translation equivariance	Needs data or a pretrained backbone	$O(k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W)$ per layer	11.1–11.2
RNN / LSTM / GRU	Hidden state carried step by step	Streaming/small sequence models, time series	hidden size, layers, clipping	Constant memory per step	Sequential (no parallelism over time)	$O(T \cdot h^2)$	12.2–12.4
Transformer	Self-attention + feed-forward blocks	Text, speech, vision at scale	layers, d_model, heads, context length	Parallel, O(1) path between tokens	$O(T^2 \cdot d)$ attention cost	$O(T^2 d + Td^2)$ per layer	12.7
Matrix factorisation (SGD/ALS)	$\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$	Collaborative filtering baseline	k factors, λ, (α for implicit)	Strong, compact embeddings	Cold start; no context	SGD $O(\lvert R \rvert \cdot k)$/epoch	13.5
Two-tower	User and item encoders meeting at a dot product	Retrieval over millions of items	embedding dim, temperature, negatives	Items precomputed → ANN	Weak feature crosses; versioning	Per request: user tower + ANN	13.7
Multi-armed bandits (ε-greedy, UCB, Thompson)	Explore/exploit over k arms	Headlines, thumbnails, new-item traffic	ε, c, priors	Less regret than A/B	Non-stationarity handling	O(k) per step	14.4
Q-learning / SARSA	TD control on a Q-table	Small discrete MDPs	α, γ, ε schedule	Model-free, simple	Table must fit; sample-hungry	O(1) per update	14.7
DQN	Neural Q-function + replay + target network	Discrete actions, large state spaces	replay size, target sync C, ε	Scales to pixels	Unstable (deadly triad)	Network forward/backward	14.8
Policy gradient /	Gradient ascent on expected	Continuous actions, RLHF	lr, clip ε, entropy bonus, GAE λ	Stochastic and continuous	High variance, on-policy	Network forward/backward	14.9

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
actor-critic / PPO	return			policies			
ARIMA / ETS	AR + differencing + MA / smoothing	Single short time series	(p, d, q), seasonality	Built-in intervals, little data	One series at a time, few extra features	Small	12.10

20.6 Loss functions (Ch 2, 5, 8, 10, 13, 14)

Loss	Formula (one example)	Task / model	Notes	§
MSE (squared error)	$(\hat{y} - y)^2$	Regression, linear regression, neural nets	Punishes big errors; optimal constant = **mean**; MLE under Gaussian noise	5.2, 10.3
MAE (L1)	$\lvert\hat{y} - y\rvert$	Robust regression	Optimal constant = **median**	6.2, 10.3
Huber	$\frac{1}{2}e^2$ if $\lvert e\rvert \le \delta$, else $\delta(\lvert e\rvert - \frac{1}{2}\delta)$	Regression with outliers; DQN	Quadratic near 0, linear far away	10.3, 14.8
Quantile (pinball)	$\max(\tau \cdot e, (\tau - 1) \cdot e)$, $e = y - \hat{y}$	Prediction intervals, p90 forecasts	$\tau = 0.9$ → 90th percentile	12.10
Log loss / binary cross-entropy	$-[y \ln p + (1-y) \ln(1-p)]$	Logistic regression, sigmoid outputs, multi-label	Convex for LR; gradient at logit = $p - y$; PyTorch: `BCEWithLogitsLoss` (logits in)	5.8, 10.3
Categorical cross-entropy	$-\ln p_{true}$	Softmax classifiers, language models	Uniform guess = ln K (ln 10 = 2.303); gradient = p − one_hot; PyTorch `CrossEntropyLoss` takes logits + class indices	5.9, 10.3
Hinge	$\max(0, 1 - y \cdot f(x))$, $y \in \{-1, +1\}$	SVM	Zero for points beyond the margin	8.2
ε-insensitive	$\max(0, \lvert y - f(x)\rvert - \varepsilon)$	SVR	Errors inside the tube are free	8.2
Exponential	$e^{-y \cdot f(x)}$	AdaBoost	Explains its outlier sensitivity	7.4
Regularised squared error	$\Sigma(r - \hat{r})^2 + \lambda(\lVert p\rVert^2 + \lVert q\rVert^2 + b^2)$ over observed cells	Matrix factorisation	Implicit version weights every cell by confidence	13.5
Sampled softmax / InfoNCE	$-\ln [e^{s^+/\tau} / \Sigma e^{s/\tau}]$ over in-batch items (logits − log Q for popularity)	Two-tower retrieval, CLIP-style contrastive learning	Temperature $\tau \approx 0.05$–0.1	13.7, 18.6
Skip-gram negative sampling	$-\ln \sigma(v \cdot u^+) - \Sigma \ln \sigma(-v \cdot u^-)$	word2vec, item2vec	Negatives ∝ frequency^0.75	12.9, 13.6
Pairwise / listwise ranking	pairwise: which of two items is better (RankNet/LambdaMART); listwise: optimise NDCG-like list metrics	Learning-to-rank	Pointwise = predict each label independently	17.3
Regularisation terms	L2: $\lambda\Sigma w^2$ · L1: $\lambda\Sigma\lvert w\rvert$	Any	L1 → exact zeros; L2 → shrinkage; AdamW = decoupled L2	5.7, 10.7
TD / Q-learning target	$(r + \gamma \max_{a'} Q(s',a'; \theta^-) - Q(s,a; \theta))^2$ (Huber in DQN)	Value-based RL	θ^- = frozen target network	14.8
Policy gradient	$-\log \pi(a\lvert s) \cdot A_t$	REINFORCE, actor-critic	A = G − V(s) or the TD error	14.9
Reward model (preferences)	$-\ln \sigma(s_{preferred} - s_{rejected})$	RLHF	PPO then maximises reward − $\beta \cdot KL(\pi \,\|\, \pi_{SFT})$	14.10
Fairness penalty	loss + $\lambda \cdot$(mean score A − mean score B)^2	In-processing bias mitigation	Larger λ → more parity, less accuracy	19.4

Rule of thumb: pick the output layer and the loss together — identity + MSE/MAE/Huber (regression), sigmoid + BCE (binary, multi-label), softmax + cross-entropy (one label out of K) (§10.2). Never train on accuracy: its gradient is zero almost everywhere.

20.7 Metrics: formula and when to use (Ch 6, 9, 11, 13, 16)

Regression (§6.2, §16.7) — worked on y = (30, 25, 40, 35, 50), ŷ = (28, 27, 35, 35, 60):

Metric	Formula	Value	Use when
MAE	$(1/n)\Sigma\|e\|$	3.8	Cost grows linearly with error; label outliers shouldn't dominate
MSE / RMSE	$(1/n)\Sigma e^2$ / $\sqrt{\text{MSE}}$	26.6 / 5.16	Big misses are disproportionately bad (a 40-min miss adds 16× what a 10-min miss adds to MSE)
R^2	$1 - \Sigma e^2/\Sigma(y - \bar{y})^2$	0.641	Share of variance explained; 0 = predicting the mean, < 0 = worse
MAPE	$(100/n)\Sigma\|e/y\|$	—	Relative error; breaks near y = 0
p90 absolute error	90th percentile of $\|e\|$	—	Promises made on the tail (ETAs)
Per-cohort error	Any of the above per segment	—	Always: averages hide failing segments

Classification (§6.3, §6.5) — worked on 1,000 transactions, 100 fraud, 80 flagged, 60 correct (TP 60, FP 20, FN 40, TN 880):

Metric	Formula	Value	Use when
Accuracy	(TP + TN)/all	0.94 ("flag nothing" = 0.90)	Balanced classes, equal costs only
Precision (PPV)	TP/(TP + FP)	0.75	False alarms are expensive
Recall (sensitivity, TPR)	TP/(TP + FN)	0.60	Misses are expensive
F1 / Fβ	$2PR/(P + R)$ / $(1+\beta^2)PR/(\beta^2P + R)$	0.667	One number, imbalanced data; F2 favours recall
Specificity / FPR	TN/(TN + FP) / FP/(FP + TN)	0.978 / 0.022	Screening; ROC axes
ROC-AUC	P(random positive scored above random negative)	positives (0.9, 0.7, 0.4) vs negatives (0.8, 0.3, 0.2) → 7/9 = 0.778	Threshold-free ranking quality, moderate imbalance
PR-AUC (average precision)	Area under precision–recall curve; no-skill = base rate	—	**Rare positives** (fraud, ads, disease)
Log loss	$-\text{mean}[y \ln p + (1-y) \ln(1-p)]$	—	Probabilities feed decisions
Brier score	$\text{mean}((p - y)^2)$	y (1,0,1,1), p (0.9,0.2,0.6,0.3) → 0.175; p ≡ 0.5 → 0.25	Calibration quality (proper scoring rule)
ECE	Weighted gap between mean score and observed rate per bin	—	Calibration monitoring (ads)
Macro / weighted averages	Mean of per-class metrics (unweighted / by support)	—	Multiclass; macro exposes weak minority classes

Choosing a threshold (§6.3): with calibrated p, flag when **$p > C_{FP} / (C_{FP} + C_{FN})$** (review $5, missed fraud $100 → $p > 0.048$). Fraud and moderation use **two thresholds**: approve / review or step-up / block.

Ranking and retrieval (§6.4, §13.11, §17.3)

Metric	Formula	Use when
Precision@k	relevant in top k ÷ k	Fixed number of slots
Recall@k	relevant in top k ÷ all relevant	Candidate generation (did the good items survive?)
Hit rate@k	1 if any relevant item in top k	Leave-one-out recommender tests
AP / MAP	mean of precision@i at each relevant rank ÷ #relevant; MAP = mean over queries	Binary relevance, whole list
MRR	mean of 1/rank of the first relevant result	One good answer fast (Q&A, navigation)
NDCG@k	DCG/IDCG, $DCG = \Sigma\ rel_i / \log_2(i + 1)$ (exponential variant: (2^rel − 1))	Graded relevance, position matters
Coverage, diversity, novelty	share of catalogue recommended; 1 − mean pairwise similarity; mean popularity	Beyond-accuracy health of a recommender

Worked: relevances [3, 0, 2, 1, 0, 0], one relevant grade-2 document not retrieved → DCG@5 = 4.431, NDCG@5 = **0.778**; `sklearn.metrics.ndcg_score` builds the ideal only from items you pass it and reports 0.930 — always say which NDCG. With exponential gain, [3, 0, 2] → NDCG@3 = 0.956. Recs A–E with relevant {A, C, F}: P@5 0.4, R@5 0.667, AP@5 0.556, NDCG@5 0.704.

Other families

Area	Metrics	§
Clustering (no labels)	Silhouette s = (b − a)/max(a, b) (higher), Davies–Bouldin (lower), Calinski–Harabasz (higher), inertia/BIC (compare k only) — all favour convex clusters	9.1, 9.9
Clustering (some labels)	Adjusted Rand Index (0 random, 1 perfect), NMI	9.9
Dimensionality reduction	Explained variance, reconstruction error, trustworthiness	9.5, 9.9
Anomaly detection	Precision@k, average precision on a reviewed sample	9.7
Detection / segmentation	IoU = intersection/union (worked 0.5625); mAP over IoU thresholds (VOC 0.5; COCO 0.5–0.95); Dice for masks	11.7
Association rules	support, confidence = supp(A∪B)/supp(A), **lift** = confidence/supp(B) (> 1 positive)	9.8
Online / business	CTR, watch time, retention, revenue per search, resolution rate, promise-kept rate + guardrails	6.1, 16.9
Drift	PSI = Σ(cur − ref)·ln(cur/ref): < 0.1 stable, 0.1–0.25 watch, > 0.25 investigate; KS statistic D; null rates	16.8
Fairness	Selection rate, TPR, FPR, PPV and calibration per group; disparate-impact ratio (four-fifths rule)	19.2–19.3

20.8 Validation and tuning (Ch 4, 6)

Splitter	Folds	Use for
`train_test_split(stratify=y)`	One hold-out	Final test set
`KFold(5, shuffle=True)`	Random equal folds	Regression, i.i.d. rows
`StratifiedKFold`	Same class mix in every fold	Classification (default in `cross_val_score` for classifiers)
`GroupKFold` / `StratifiedGroupKFold`	One entity per fold	Users, patients, devices
`TimeSeriesSplit(gap=…)`	Train on past, validate on the next block	Anything time-ordered; gap mimics label delay
`RepeatedStratifiedKFold` / `LeaveOneOut`	Repeats / k = n	Small data / tiny data only
Nested CV	Outer loop around the whole tuning procedure	Unbiased estimate of "tune then train"

Report CV as **mean ± std**; differences under about one std are not clearly real (§6.6). Expected test error = **bias² + variance + irreducible noise**; averaging B models with variance σ^2 and correlation ρ gives variance $\rho\sigma^2 + (1-\rho)\sigma^2/B$ — bagging shrinks the second term, random feature subsets shrink ρ (§6.7, §7.2).

Tool	Answers	Read it
Learning curve (score vs training size)	Would more data help?	Curves converge low → more capacity; big persistent gap → more data / regularisation
Validation curve (score vs one hyperparameter)	Where is the sweet spot?	Pick the peak of the validation curve

Tuning strategies (§6.9)

Method	Budget control	Best for
Grid search	Product of grid sizes (4 × 5 values × 5 folds = 100 fits)	1–2 hyperparameters, cheap models
Random search	`n_iter`	Default for 3+ hyperparameters (explores each dimension more finely)
Successive halving / Hyperband	Resource per round	Many candidates, scalable resource (rows, epochs)
Bayesian optimisation (Optuna TPE, Gaussian process + expected improvement)	Trials	Expensive trials (boosting on big data, deep nets)

Recipe: defaults + baseline → tune the few that matter (boosting: learning rate + tree size; linear/SVM: C, γ; trees: depth/leaf size) → log-uniform ranges → widen a range if the best value sits on its edge → refit on all training data → test once. **Early stopping** (§6.10): monitor validation loss, stop after `patience` rounds without improvement, restore the best weights; never early-stop on the test set.

Probability calibration (§6.5): reliability diagram + Brier/log-loss on held-out data; fix with **Platt scaling** (sigmoid on the score, small data) or **isotonic regression** (step function, roughly ≥ 1,000 examples) via `CalibratedClassifierCV`; recheck when the base rate shifts. ROC-AUC is unchanged by any strictly increasing map.

20.9 Deep learning: layers, activations, optimizers, regularisation, debugging (Ch 10)

Building blocks (§10.1, §10.3, §10.4)

Piece	Formula / rule
Neuron	$z = w \cdot x + b$, $a = f(z)$
Dense layer	$h = f(Wx + b)$; batch: $H = f(XW^T + b)$; params = inputs × outputs + outputs (256 → 128: 32,896)
Why nonlinearity	Without f, $W_2(W_1x + b_1) + b_2$ is still one linear map; one perceptron can't learn XOR, one hidden layer can
Forward → loss → backward → update	$\theta \leftarrow \theta - \eta \cdot \partial L/\partial \theta$; backprop = chain rule from the loss backwards, ≈ 2–3× a forward pass
Output gradient	sigmoid + BCE and softmax + CE: $\partial L/\partial \text{logits} = p - y$
Weight gradient	$(\text{layer input})^T$ @ (gradient at the layer's pre-activation)
Gradient check	$(L(\theta + \varepsilon) - L(\theta - \varepsilon))/2\varepsilon \approx$ analytic gradient

Activations (§10.2)

Name	Formula	Range	Use
Sigmoid	$1/(1 + e^{-z})$; $\sigma' \le 0.25$	(0, 1)	Binary / multi-label output; LSTM gates
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$; max slope 1	(−1, 1)	RNN hidden states
ReLU	$\max(0, z)$	[0, ∞)	**Default hidden layer**; can "die"
Leaky ReLU / PReLU / ELU	z or αz ($\alpha \approx 0.01$) / α learned / $\alpha(e^z - 1)$	—	When many ReLUs die
GELU / SiLU (Swish)	$z \cdot \Phi(z)$ / $z \cdot \sigma(z)$	—	Transformers (BERT, GPT), EfficientNet, SwiGLU in LLMs
Softmax	$e^{z_i} / \Sigma e^{z_j}$ (subtract max first)	sums to 1	One label out of K; (2, 1, 0.1) → (0.659, 0.242, 0.099)
Identity	z	$\mathbb{R}$	Regression output

Optimizers (§10.5) — g = mini-batch gradient

Optimizer	Update	Default / note
SGD	$\theta \leftarrow \theta - \eta g$	Set lr explicitly
Momentum	$v \leftarrow \mu v + g$; $\theta \leftarrow \theta - \eta v$	$\mu = 0.9$ ≈ averaging ~10 steps; Nesterov = look-ahead
AdaGrad	$s \leftarrow s + g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$	Sparse features; step shrinks forever
RMSProp	$s \leftarrow \rho s + (1 - \rho)g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$	PyTorch lr 0.01, α (ρ) 0.99
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g$; $s \leftarrow \beta_2 s + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta\hat{m}/(\sqrt{\hat{s}} + \varepsilon)$	lr 1e-3, β = (0.9, 0.999), ε = 1e-8
AdamW	Adam + $\theta \leftarrow \theta - \eta\lambda\theta$ (decoupled)	weight_decay 0.01; the correct "Adam with weight decay"

Schedules: step decay (÷10 at milestones), exponential, reduce-on-plateau, **cosine annealing**, **linear warmup + cosine decay** (transformers/LLMs), one-cycle. LR finder: grow lr exponentially per batch, pick ≈ one-tenth of the lr at the loss minimum.

Initialisation and gradient flow (§10.6)

Scheme	Weight variance	For
Xavier / Glorot	$2/(\text{fan_in} + \text{fan_out})$	tanh, sigmoid, linear
He / Kaiming	$2/\text{fan_in}$ (fan_in 256 → 0.0078, std 0.088)	ReLU family
LeCun	$1/\text{fan_in}$	SELU, linear

Never initialise all weights equal (symmetry never breaks). Vanishing/exploding gradients (ten sigmoid layers scale gradients by $\le 0.25^{10} \approx 9.5 \times 10^{-7}$) → ReLU, He/Xavier init, BatchNorm/LayerNorm, residual connections, gradient clipping (`clip_grad_norm_(params, 1.0)`), gates (LSTM/GRU).

Regularisation (§10.7)

Technique	How	Watch out
Dropout (p)	Zero units at random in training; survivors × 1/(1 − p) (inverted dropout)	Call `model.eval()` for inference
Weight decay	λΣw² or AdamW's decoupled shrinkage (1e-4…1e-2 SGD; 0.01–0.1 AdamW)	Often skip biases and norm parameters
BatchNorm	x̂ = (x − μ_batch)/√(σ²_batch + ε), y = γx̂ + β; running stats at eval	Small batches; LayerNorm in transformers/RNNs, GroupNorm for small-batch vision
Early stopping	Stop after `patience` epochs without validation improvement; restore best	Report on a separate test set
Data augmentation	Label-preserving transforms (flips, crops, colour jitter, noise, Mixup/CutMix)	Training data only; don't flip a 6 into a 9
More data / smaller model	—	Often the most effective of all

Debugging checklist (§10.8)

1. Look at a few raw examples and labels **after** the data pipeline.
2. Initial loss ≈ chance: ln K for K classes (ln 10 = 2.303). Much higher → bad init/scale, wrong labels or loss.
3. **Overfit one mini-batch** to ~0 loss. Can't → a bug (labels, loss, no gradient flow, missing `optimizer.step()`).
4. LR range test; start with Adam 1e-3 (or SGD + momentum at the suggested lr).
5. Train with a validation set; plot both curves; normalise inputs with **training** statistics.
6. Only then add regularisation and tune; keep a simple baseline (logistic regression / GBDT) for "what good means".

Curve looks like	Likely cause	Try
Flat from step 0, near ln K	lr far too small; bug (frozen weights, shuffled labels, no step)	Overfit one batch; check the data pipeline
Rising, NaN, spikes	lr too high, exploding gradients, log(0)	Lower lr, clip gradients, logit-based losses
Train ↓, validation ↓ then ↑	Overfitting	Regularise, early stop, more data
Both high and close	Underfitting	Bigger model, train longer, higher lr, better features
Validation better than train	Dropout/augmentation only in training	Usually fine — check the split
Jagged	Small batch or high lr	Larger batch, lower lr

PyTorch loop (§10.9): `model.train()` → for each batch: `logits = model(xb)`; `loss = loss_fn(logits, yb)`; `opt.zero_grad()`; `loss.backward()`; `opt.step()` (then `scheduler.step()`). Evaluate under `model.eval()` + `torch.no_grad()`. Save `state_dict()`. `CrossEntropyLoss` wants `long` class indices; `BCEWithLogitsLoss` wants float targets shaped like the logits. Batch size 32–256 to start; imbalance → `weight=` / `pos_weight=` or `WeightedRandomSampler`.

20.10 CNN, RNN and attention: shapes and formulas (Ch 11, 12)

Convolutional networks (§11.1–11.2)

Quantity	Formula	Example
Output size	out = ⌊(n + 2p − k)/s⌋ + 1	n 5, k 3: s 1 p 0 → 3; s 2 → 2; p 1 → 5 ("same"); 224, k 7, s 2, p 3 → 112
Conv weights	k·k·C_in·C_out + C_out	`Conv2d(64, 128, 3)` → 73,856; `Conv2d(3, 16, 3)` → 448 vs 49,168 for a dense layer on 32×32×3
Tensor layout	PyTorch N×C×H×W; TensorFlow/NumPy images N×H×W×C	`permute(0, 3, 1, 2)`
Pooling	2×2 max, stride 2 halves H and W; no parameters; global average pooling before the head	
Receptive field	each k×k layer adds (k − 1) × product of earlier strides	stacked 3×3 (stride 1): 3, 5, 7 — three 3×3 = one 7×7 with 27C² vs 49C² weights
1×1 conv	mixes channels only	bottlenecks (Inception, ResNet)
Depthwise-separable	per-channel spatial conv + 1×1	≈ 8–9× less compute for 3×3 (MobileNet)
Residual block	y = ReLU(x + F(x))	identity gradient highway (ResNet)
ViT patches	(H/P)·(W/P) tokens	224×224, 16×16 → 196

Architecture timeline (§11.4): LeNet-5 (1998, conv-pool) → AlexNet (2012, ReLU + dropout + GPUs, top-5 15.3%) → VGG (2014, stacked 3×3) → Inception/GoogLeNet (2014, parallel branches, 1×1, GAP) → **ResNet** (2015, skip connections, 3.57% top-5 ensemble) → MobileNet (2017, depthwise-separable) → EfficientNet (2019, compound scaling of depth/width/resolution) → ViT (2020, patches + transformer) → ConvNeXt (2022).

Transfer learning (§11.5): small data similar to ImageNet → freeze backbone, train a new head; more or different data → fine-tune (all or top blocks) with a ~10× smaller lr; preprocess exactly like the pretraining data. **Augmentation (§11.6):** `RandomResizedCrop`, `RandomHorizontalFlip`, `ColorJitter`, `RandomErasing`, Mixup/CutMix, RandAugment — training only.

Vision tasks (§11.7): classification (label) · detection (box + class; IoU, **NMS**: keep the best box, drop overlaps above an IoU threshold, repeat; two-stage Faster R-CNN vs one-stage YOLO grid; mAP) · semantic segmentation (class per pixel; **U-Net** encoder–decoder with concatenated skip connections) · instance segmentation (mask per object; Mask R-CNN, SAM).

Recurrent networks (§12.2–12.5)

Model	Equations	Notes
RNN	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$	Shared weights over time; BPTT; truncated BPTT for long sequences
LSTM	$f_t, i_t, o_t = \sigma(W[h_{t-1}, x_t] + b)$; $\tilde{c}_t = \tanh(\dots)$; $\mathbf{c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t}$; $h_t = o_t \odot \tanh(c_t)$	Additive cell path fixes vanishing gradients; forget-gate bias 1; 4 weight blocks
GRU	$z_t, r_t = \sigma(\dots)$; $\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$; $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	3 weight blocks, similar accuracy
Seq2seq	Encoder → context → decoder; teacher forcing in training; greedy / **beam search** / sampling at inference	Fixed-size bottleneck → attention

Worked LSTM unit: c = 2.0, f 0.9, i 0.2, c̃ 0.5 → c_new = 1.9; o 0.7 → h = 0.7·tanh(1.9) = 0.669. PyTorch `nn.LSTM(input, hidden, num_layers, batch_first=True)` takes (batch, time, features) and returns (output, (h_n, c_n)). Parameter ratio RNN : GRU : LSTM = 1 : 3 : 4.

Attention and transformers (§12.6–12.7)

Idea	Formula / fact
Scaled dot-product attention	**$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})\cdot V$** — score, softmax, weighted sum
Why √d_k	Dot products of random unit-variance vectors have std √d_k; scaling keeps softmax out of saturation
Worked example	Q = K = [[1,0],[0,1],[1,1]], V = [[1,2],[3,4],[5,6]] → query 1 weights (0.401, 0.198, 0.401), output [3.0, 4.0]
Self- vs cross-attention	Q, K, V from the same sequence / Q from decoder, K and V from encoder
Causal mask	Upper triangle set to −∞ before softmax (decoders)
Multi-head	h heads of size d_model/h, concatenated and projected
Positional encoding	PE(pos, 2i) = sin(pos/10000^(2i/d)), PE(pos, 2i+1) = cos(…); or learned / RoPE
Block	Multi-head attention and a position-wise FFN (≈ 4× wider), each wrapped in a residual connection + LayerNorm; original: 6 + 6 layers, d_model 512, 8 heads
Families	Encoder-only (BERT), decoder-only (GPT, Claude, LLaMA), encoder–decoder (T5, Whisper)
Cost	Attention $O(T^2 \cdot d)$ vs RNN $O(T \cdot d^2)$; path length O(1) vs O(T); KV cache grows with context

Text (§12.8–12.9): normalise → tokenize → vocabulary (+ `<pad>`, `<unk>`, `<sos>`, `<eos>`) → ids → pad + mask. Subword tokenizers (BPE: merge the most frequent adjacent pair repeatedly; WordPiece; SentencePiece) have no OOV problem; ≈ 0.75 English words per token. word2vec: skip-gram (centre → context) or CBOW (context → centre) with negative sampling; GloVe, fastText (character n-grams), contextual embeddings (BERT).

Time series (§12.10): past-only features (lags, **shifted** rolling means, calendar); chronological splits (`TimeSeriesSplit`); beat naive and **seasonal-naive** baselines; ARIMA(p, d, q) = autoregression + differencing + moving average; trees can't extrapolate a trend → model differences.

20.11 Recommenders (Ch 13, 17.1)

Topic	Key facts
Framing	Fill a huge, sparse user–item matrix; optimise **top-K ranking** on an outcome closer to value than clicks (watch time, saves, purchases)
Feedback	Explicit (stars, real negatives, scarce) vs implicit (clicks/plays, abundant, unobserved ≠ negative → confidence c = 1 + α·count and sampled negatives)
Similarity	Cosine (direction), dot product (includes magnitude/popularity), adjusted cosine (subtract each user's mean), Jaccard $\|A∩B\|/\|A∪B\|$ for sets; for unit vectors $\|a-b\|^2 = 2 - 2\cos$
Content-based vs CF	Attributes (handles new items, over-specialises) vs other users' behaviour (quality and surprise; cold start, popularity bias); hybrids win
Item-based CF	$\hat{r}(u,i) = \text{mean}(u) + \Sigma_j S(i,j)(r(u,j) - \text{mean}(u)) / \Sigma_j\|S(i,j)\|$; precompute item–item neighbours (Amazon, 2003)
Matrix factorisation	$\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$; train on **observed** cells with L2; SGD: $err = r - \hat{r}$, $p_u \leftarrow p_u + \eta(err \cdot q_i - \lambda p_u)$; ALS: fix one side, solve ridge per user/item (parallel); implicit ALS covers all cells with confidence weights
Embeddings	Short learned vectors; routes: MF, two-tower, graph (PinSage), pretrained content encoders; item2vec = word2vec on sessions; "near" only means near for the training objective
Two-tower	User tower and item tower meet only at a dot product → item vectors precomputed into an ANN index, user tower per request; in-batch softmax with temperature, **logQ correction**, a few hard negatives; ship tower + index as **one version**
Funnel	Retrieval $10^8 \to 10^3$ (recall) → ranking $10^3 \to 10^2$ (precision/NDCG, rich crosses; GBDT, DLRM, Wide & Deep, DCN) → re-ranking → 10–50 (diversity, freshness, policy)
MMR	$\text{next} = \text{argmax}\ [\lambda \cdot \text{rel}(j) - (1-\lambda) \cdot \max_s \text{sim}(j, s)]$
ANN	IVF (k-means lists; `nprobe` ↑ = recall ↑), HNSW (layered graph; M, ef_search), PQ (compressed codes; IVF-PQ), trees (Annoy); always report recall vs exact search with latency
Cold start	New user: popularity, onboarding, context, fast session updates; new item: content features + controlled exploration (bandits)
Evaluation	Time-based split, full-catalogue ranking (sampled metrics mislead); P/R/hit rate@k, MAP, NDCG, coverage, diversity, novelty; **popularity baseline**; then A/B or interleaving
Bias	Popularity bias + feedback loops → measure (coverage, Gini of exposure), correct (logQ, IPS), explore, re-rank; log impressions and positions

20.12 Reinforcement-learning update rules (Ch 14)

Idea	Formula	Notes / worked value
MDP	$(S, A, P(s'\|s,a), R, \gamma)$; Markov: next state depends only on current state and action	§14.2
Return	$G_t = \Sigma_k\ \gamma^k\ r_{t+k+1} = r_{t+1} + \gamma G_{t+1}$	$\gamma = 0.9$: horizon $\approx 1/(1-\gamma) = 10$; $0.9^{10} \approx 0.35$
Value functions	$V^\pi(s) = E[G \mid s]$; $Q^\pi(s,a) = E[G \mid s, a]$; $\pi^*(s) = \text{argmax}_a\ Q^*(s,a)$	Q needs no model to act
Bellman expectation	$V^\pi(s) = \Sigma_a\ \pi(a\|s)\ \Sigma_{s'}\ P(s'\|s,a)[R + \gamma V^\pi(s')]$	Policy evaluation
Bellman optimality	$V^*(s) = \max_a\ \Sigma_{s'}\ P[R + \gamma V^*(s')]$; $Q^*(s,a) = \Sigma_{s'}\ P[R + \gamma\ \max_{a'}\ Q^*(s',a')]$	Treasure grid: 10, 8, 6.2, 4.58, …, −0.43 at the start
ε-greedy	random arm with probability ε, else best	Decay ε
UCB1	$\text{argmax}_a\ [Q(a) + c\sqrt{(\ln t\ /\ N_a)}]$, $c = \sqrt{2}$ in theory	Tune c for your horizon
Thompson sampling	Sample $\theta_a \sim \text{Beta}(\text{wins} + 1, \text{losses} + 1)$, pull argmax	Probability matching
Value iteration	$V(s) \leftarrow \max_a\ \Sigma\ P[R + \gamma V(s')]$ until stable	Needs the model (DP)
Policy iteration	Evaluate π, make π greedy on V, repeat	Few rounds
Monte Carlo	$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$	Unbiased, high variance, end of episode
TD(0)	$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$; **$\delta = r + \gamma V(s') - V(s)$**	Bootstraps, lower variance, every step
Q-learning (off-policy)	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma\ \max_{a'}\ Q(s',a') - Q(s,a)]$	Learns the greedy policy; cliff edge path
SARSA (on-policy)	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$, a' = next action taken	Learns what it does; safer path
DQN	Minimise $\text{Huber}(r + \gamma\ \max_{a'}\ Q(s',a';\ \theta^-) - Q(s,a;\ \theta))$ on replay mini-batches	Experience replay + target network; Double DQN: select with online, evaluate with target
REINFORCE	$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla \log \pi(a_t\|s_t)$	High variance → baseline: $A_t = G_t - V(s_t)$
Actor-critic	actor: $\theta \leftarrow \theta + \alpha \cdot \delta \cdot \nabla \log \pi$; critic: TD learning on V	PPO = clipped actor-critic update
RLHF	SFT → reward model $(-\ln \sigma(s_w - s_l))$ → PPO on reward $- \beta \cdot KL(\pi \parallel \pi_SFT)$	DPO skips the RL loop; GRPO uses group-mean baselines

Pitfalls (§14.12): reward hacking (a re-collectable +3 per step is worth $3/(1 - 0.9) = 30$), sample inefficiency, sparse rewards, credit assignment, seed instability, sim-to-real gap, unsafe exploration, the **deadly triad** (function approximation + bootstrapping + off-policy).

20.13 ML system design: the 7-step framework and napkin math (Ch 15, 17, 18)



Step	Say out loud	§
1. Frame	Family (predictive / generative / agentic — does it output a score, content or an action?); the known system it resembles; scope; ML task: input, output, category, **ML objective**	15.2, 15.3
2. Success	Online business metric + offline ML metric + guardrails; latency (p95/p99); scale (QPS, peak); cost per request or per completed task	15.3
3. Data	Sources; **label source and delay**; freshness (batch / streaming / request-time); feature groups; leakage and privacy	15.3, 16.3, 16.4
4. Model	Baseline (popularity, rules, LR) → GBDT → deep (two-tower, multi-task); name the trigger to upgrade	15.3
5. Serving	What's offline vs per request; caches (with permissions); cheaper paths; timeouts and fallbacks; funnel if the catalogue is big	15.3, 16.5, 16.6
6. Keep it working	Log inputs/outputs/versions → join outcomes → monitor 4 layers → gated retraining → shadow → canary → ramp	15.3, 16.8, 16.9
7. Failures and cost	Specific failure list with controls; cost = requests × share × calls × price, in the same unit as the benefit	15.3

Business goal → ML objective (§15.3): time spent → P(watch ≥ X s) or expected watch time (not raw clicks) · fraud losses → P(fraud) + cost-based thresholds · delivery promise → minutes to delivery + a product-owned buffer · search revenue → P(purchase | query, item) · churn → P(cancel within 30 days) (label is 30 days late).

Four assumptions that break with a model (§15.1): correctness is a spectrum (need quality metrics) · behaviour depends on data (drift) · compute is not cheap (model calls cost money) · failures are silent (a wrong answer is HTTP 200).

Napkin numbers (illustrative, from the chapters)

Quantity	Arithmetic	Result	§
QPS	requests/day ÷ 86,400; peak ≈ 2–3×	10 M/day → 116 avg, ~347 peak (3×)	15.3
Homepage recs	100 M DAU × 5 loads ÷ 86,400	≈ 5,800 avg, ~17 K peak (3×); × 500 candidates ≈ 8.7 M scores/s	17.1
Feed ranking	500 M DAU × 5 sessions × 3 calls ÷ 86,400	≈ 86.8 K/s; × 500 posts ≈ 43 M scores/s	17.2
Ads scoring	100 K requests/s × 300 candidates	30 M scores/s in a 50–100 ms auction	18.1
Vector memory	N × dims × 4 bytes (float32)	50 M × 128 → 25.6 GB; 100 M × 512 → 204.8 GB; PQ to 64 B → 6.4 GB	17.1, 18.6
Batch precompute	users × items per user	25 M × 500 = 12.5 B rows; active 5 M only → 2.5 B	16.5
LLM cost	10 M req/day × $0.01	$100 K/day ≈ $3 M/month	15.1
Support bot	500 K orders × 8% contacts × 10 calls × $0.01	40 K conversations, $4 K/day; compare **per resolved issue**	15.4
Moderation cascade	200 M uploads/day, 1% flagged to heavy models, 0.1% to humans	2 M heavy/day (≈ 23/s), 200 K human decisions/day (~670 reviewer-days)	18.3
A/B sample size	$n \propto 1/\delta^2$	5% CTR, +1% relative lift → ~3.0 M users per arm; half the lift → ~4× users	16.9
Latency budget (small model)	inference 1–10 ms, online-store lookup 2–10 ms, inline features 0–15 ms, routing/auth 1–5 ms, logging 1–2 ms	batch KV lookup ~1 ms vs online 10–100 ms	16.5, 16.6
Fraud	10 K tx/s × 0.2% fraud; ~50 ms authorisation, ~10 ms for the model	~20 fraud attempts/s	18.2
Video indexing	1 M uploads/day × 600 s ÷ 2 s per sampled frame	300 M frame encodings/day ≈ 3,500/s — indexing, not querying, dominates	17.4
Blurring batch	5 M panoramas × 24 tiles × 10 ms	1.2 M GPU-s/day ≈ 14 GPUs busy all day	18.7

Thirteen case studies in one line each (§17.1–18.9)

Design	Shape	Signature idea
Video recommendations	Multi-source retrieval → ranker → rules	Recall@K + watch time; cold start; position bias; judge sources by marginal recall
Feed ranking	Social fan-in → multi-task heads → value formula → integrity layer	Product-owned weights over calibrated heads; hide/report guardrails
Search ranking	Query understanding → BM25 + semantic retrieval → L1/L2 rankers	NDCG on judged sets; IPW for position-biased clicks; relevance floor for ads
Text-to-video search	Index title/ASR/OCR + CLIP-style frame vectors → fuse channels → ranker	Indexing dominates cost; learned fusion with channel-availability flags; segment vectors for moments
Ad click prediction	Eligibility → retrieval → pCTR/pCVR → auction → pacing	**Calibration** per segment (bid × pCTR); delayed conversions
Fraud detection	Rules + GBDT per transaction → two thresholds	Velocity/graph features; late and censored labels; PR-AUC and $ loss
Harmful content	Hash/rules → fast classifiers → heavy models → humans (cascade)	Prevalence of violating **views**; action ladder; flag rate = compute bill
People You May Know	FoF + affiliations + contacts + embeddings	Score = P(send) × P(accept | sent); Adamic-Adar; privacy fails closed
Music discovery	Weekly batch + online radio	Audio embeddings for new tracks; skip-driven re-ranking; diversity guardrail
Visual search	Image/text encoder → ANN → light ranker	Shared image–text space (CLIP-style); the index churns; re-embed on encoder upgrade
Privacy blurring	Tiled detector at a low threshold → context classifier → multi-view tracking → padded blur → audit	Recall per person across views and per pixel; fail closed; audits instead of A/B tests
Event recommendation	Geo + time filter + content two-tower → GBDT → re-rank → capacity check	Every item is cold: score content, place, time, friends; point-in-time features; organiser exposure
Similar listings	Per-market k-NN on session embeddings → availability filter → ranker	Skip-gram with booked-listing context and same-market negatives; averaged vectors for new listings

20.14 Production and MLOps checklist (Ch 16)

Architecture

- [ ] Five pieces named: training pipeline, feature layer, inference path, serving platform, monitoring loop; offline plane vs online plane (§16.1).
- [ ] Training pipeline: data preparation with **point-in-time-correct joins** → training → evaluation vs production → model registry with lineage (§16.2).
- [ ] One feature definition for training and serving; serving features logged and reused for training (Rules of ML #29, #32, #37) (§16.2).
- [ ] Feature store: offline history for training, online latest values for serving; batch vs streaming vs request-time features with freshness alerts (§16.4).
- [ ] Inference mode chosen by the deciding question "does it need request-time information?": batch, online, streaming, or precompute-then-rerank (§16.5).

Serving

- [ ] Stateless, versioned model service; itemised latency budget; batched feature reads; hot-entity caches (§16.6).
- [ ] Timeouts and an agreed **fallback** (previous model, cached score, heuristic); fail-open vs fail-closed decided per risk (§16.2, §16.6).
- [ ] CPU benchmark before GPUs; distillation / quantisation / pruning if needed (§16.6).
- [ ] Model version logged with every prediction (§16.6).

Evaluation and release

- [ ] Offline metric matches the problem (PR-AUC + threshold, recall@k + NDCG, MAE/RMSE + p90 + cohorts), calibration if scores are used as probabilities, naive and production baselines (§16.7).
- [ ] Release ladder: offline gate (overall **and per segment**) → shadow → sticky canary → ramp with a pre-agreed abort rule; rehearsed rollback of model + features + config (§16.6).
- [ ] A/B test: randomisation unit, one primary metric, guardrails, power (MDE, α, 1 − β), whole weeks, no peeking (or a sequential test), sample-ratio-mismatch check; interleaving / switchback / holdback when appropriate (§16.9).

Monitoring and retraining

- [ ] Four layers: service health → input distributions (PSI, KS, **null rates**) → prediction distribution → outcomes as labels arrive (§16.8).
- [ ] Data drift vs label shift vs concept drift vs plain bugs; alerts start investigations (§16.8).
- [ ] Retraining cadence set by measured decay and label maturity; triggered retrains go through the same gates; never auto-promote (§16.8).
- [ ] Feedback loops: log impressions and selection probabilities; small risk-approved exploration (§16.8).

MLOps (§16.10)

- [ ] Experiment tracking (code version, params, **data hash**, metrics, artifacts) — MLflow, W&B.
- [ ] Data and model versioning (DVC, lakeFS, Delta), data validation that fails the pipeline (TFDV, Great Expectations, Pandera).
- [ ] Registry with stages and owners; orchestration (Airflow, Kubeflow, Dagster); CI with feature unit tests and training–serving consistency tests; CD with shadow/canary/auto-rollback; continuous training (MLOps level 1 → 2).
- [ ] Test the infrastructure independently from the ML: push a trivial model through, confirm identical offline and online predictions (Rules of ML #5).
- [ ] Labels: source, delay, cost; downsample-then-recalibrate; temporal splits; PII minimised, deletion path defined (§16.3).

20.15 Responsible-ML checklist (Ch 19)

Problem framing

- [ ] Is ML appropriate at all? Who is harmed by a wrong output? Is it high-risk (credit, hiring, …) under the EU AI Act or similar law? (§19.9)
- [ ] Does the label measure what we care about (not arrests, past hires or cost as proxies)? (§19.1)
- [ ] Which fairness criterion fits the harm — demographic parity, equal opportunity, equalized odds, calibration — agreed with legal and domain experts? (§19.2)

Data

- [ ] Datasheet: motivation, composition, collection, labelling, uses, distribution, maintenance (§19.8).
- [ ] Coverage audit per group **and intersection**; lawful basis; PII minimised; sensitive attributes stored separately for auditing (§19.1, §19.6).
- [ ] Provenance and poisoning checks on scraped or user-generated data (§19.7).

Model and evaluation

- [ ] Per-group selection rate, TPR, FPR, PPV and calibration with confidence intervals; disparate-impact ratio (< 0.8 is a warning); proxy test (predict the group from the features) (§19.3).
- [ ] Mitigation chosen (reweighing w(g, y) = P(g)P(y)/P(g, y), fairness penalty, group thresholds on validation data), its accuracy cost measured, every criterion re-audited (§19.4).
- [ ] Explanations: permutation importance on held-out data, PDP/ICE, SHAP (values add up to f(x) − E[f]), counterfactuals; reason codes for adverse decisions; never causal claims (§19.5).
- [ ] Robustness (FGSM/PGD, adversarial training), membership-inference risk, DP-SGD / federated learning where needed (§19.6, §19.7).

Deployment and monitoring

- [ ] Model card with intended and out-of-scope uses and disaggregated metrics; human oversight and an appeal path (GDPR Art. 22) (§19.8, §19.9).
- [ ] API hardening: authentication, rate limits, label-only outputs (§19.7).
- [ ] Per-group monitoring, drift and feedback-loop checks, incident and rollback plan, scheduled re-audits (§19.11).

Key formulas: disparate impact = P(Ŷ=1 | B)/P(Ŷ=1 | A) (0.35/0.50 = 0.70) · Chouldechova: FPR = p/(1−p)·(1−PPV)/PPV·(1−FNR) · ε-DP: P[M(D) ∈ O] ≤ e^ε·P[M(D') ∈ O]; Laplace noise scale Δf/ε (count, ε 0.5 → 2); k queries at ε each cost kε · FGSM: x_adv = clip(x + ε·sign(∇ₓL), 0, 1) · FedAvg = data-size-weighted average of client weights.

20.16 Decision tables: which algorithm, which metric

Which algorithm for which problem

Problem	Start with (baseline)	Then try	Avoid / notes	§
Tabular classification or regression, < ~1 M rows	Logistic / linear regression; majority-class or mean baseline	Random forest → **gradient boosting** (LightGBM, XGBoost, CatBoost, `HistGradientBoosting`)	Deep nets usually lose on small tables; k-NN beyond ~20–50 features	7.6, 8.4
Many categorical features	CatBoost or `HistGradientBoosting` with native categoricals	Target encoding (cross-fitted) + GBDT	One-hot of 100 K levels for a linear model	4.4, 7.4
High-dimensional sparse text	TF-IDF + logistic regression / linear SVM; Naive Bayes for tiny data	Fine-tuned transformer if data and budget allow	RBF SVM on sparse text	4.6, 8.2, 8.3
Images	Fine-tune a pretrained CNN/ViT (ResNet-50, EfficientNet) with augmentation	Bigger backbone, more data	Training from scratch on a small dataset	11.5
Detection / segmentation	YOLO-style one-stage (real time) or Faster R-CNN; U-Net for masks	Transformer detectors (DETR)	Judging detectors by accuracy (use mAP)	11.7
Sequences / text generation	Transformer (pretrained)	LSTM/GRU for small, streaming or on-device models	Simple RNN for long dependencies	12.4, 12.7
Time-series forecast	Naive and seasonal-naive; ETS / ARIMA for one series	Global GBDT on lag features (model differences); deep models with lots of data	Random splits; trees on trending levels	12.10
Clustering, round groups	k-means (k-means++, scaled, silhouette/elbow for k)	GMM (ellipses, soft), agglomerative (hierarchy)	k-means on moons/rings	9.1–9.4
Clustering, odd shapes / noise	DBSCAN / HDBSCAN	—	One eps for mixed densities (use HDBSCAN)	9.3
Compression / pre-step	PCA (standardise first; ~95% variance)	Kernel PCA, TruncatedSVD for sparse	PCA for visualising non-linear structure	9.5
2-D visualisation	PCA first, then t-SNE / UMAP	UMAP for big data or new points	Reading cluster sizes/gaps in t-SNE	9.6
Anomaly detection, unlabelled	Robust z / IQR per feature; Isolation Forest multivariate	LOF (mixed densities), One-Class SVM (clean novelty data), autoencoders	Plain z-scores on tiny or skewed samples	9.7
Rare-event classification (fraud)	Rules + logistic regression → GBDT with velocity and graph features	Graph neural nets for rings, anomaly scores	Accuracy; single threshold	18.2
Recommendations	Popularity + item-based CF / co-occurrence	Implicit MF (ALS) → two-tower retrieval + GBDT/deep ranker	Pure CF for always-new catalogues	13.4–13.8
Search ranking	BM25 + popularity	Hybrid lexical + semantic retrieval, LambdaMART / deep L2, cross-encoder top-k	Training on raw position-biased clicks	17.3
Choosing among a few options with live feedback	A/B test	Bandits (Thompson, UCB, contextual LinUCB)	Fixed ε forever	14.4
Sequential decisions with a simulator	Value iteration (known model) / Q-learning (small)	DQN (discrete), PPO/SAC (continuous)	Full online RL on live money or people	14.5–14.9
Must explain every decision	Linear/logistic model, small tree, rule list, GAM	GBDT + SHAP reason codes	Deep black box without oversight	19.5

Which metric for which problem

Situation	Primary metric	Also report	Don't	§
Balanced classes, equal costs	Accuracy + confusion matrix	F1, log loss	—	6.3
False positives costly (spam hiding real mail)	Precision at a recall floor	PR curve	Quote precision without threshold and base rate	6.3
Misses costly (screening, fraud)	Recall at a precision floor	Expected cost	—	6.3
Rare positives (≤ ~1%)	PR-AUC / average precision + P/R at the operating threshold	Base rate, expected $ loss	Accuracy; ROC-AUC alone	6.3, 16.7
Comparing rankers, moderate imbalance	ROC-AUC	PR-AUC	—	6.3
Probabilities used in arithmetic (auctions, expected cost)	Log loss, Brier, ECE per segment	Reliability diagram	Trust AUC for calibration	6.5, 18.1
Multiclass with weak minority classes	Macro-F1 / per-class P, R	`classification_report`	Micro-average only	6.3
Regression, errors cost linearly	MAE	p90 error, per cohort	—	6.2
Regression, big misses very costly	RMSE	MAE, p90, per cohort	R^2 alone	6.2
Retrieval / candidate generation	Recall@k	Coverage	NDCG at this stage	6.4, 13.8
Ranked list with graded relevance	NDCG@k (state the gain and ideal)	MAP, MRR	Sampled-negative metrics	6.4, 17.3
One right answer fast	MRR	Hit rate@k	—	6.4
Recommender health	NDCG/MAP + coverage, diversity, novelty	Popularity baseline	Accuracy alone	13.11
Clustering without labels	Silhouette / DB / CH to compare settings	Stability, downstream value	Trust them on non-convex shapes	9.9
Object detection	mAP at IoU thresholds	Per-class AP	Accuracy	11.7
Moderation	Prevalence of violating views per category	Wrongful removals; share of appeals won	Items removed	18.3
Launch decision	Online business metric in an A/B test	Guardrails (latency, complaints, fairness)	Offline metric alone	16.9
Fairness	The criterion that matches the harm (equal opportunity, equalized odds, parity, calibration)	All others + group sizes and CIs	One unnamed "fairness" number	19.2

20.17 Top 50 interview questions

Crisp answers; each points to the full treatment.

Foundations

1. **What's the difference between AI, ML and deep learning?** → AI is the goal of intelligent behaviour (including rules and search); ML learns behaviour from data; deep learning is ML with many-layer neural networks that learn their own features. (§1.1)
2. **Supervised vs unsupervised vs self-supervised vs reinforcement?** → Labels / no labels / labels generated from the data (next token) / rewards from actions in an environment. (§1.4)
3. **Parameters vs hyperparameters?** → Parameters are learned in training (weights); hyperparameters are set before it (learning rate, depth, C) and tuned on validation data. (§1.6)
4. **Why three data splits?** → Train fits parameters, validation chooses hyperparameters and models, the test set is used once so its score stays unbiased. (§1.6, §4.7)
5. **What is data leakage? Give examples.** → Information unavailable at prediction time reaching training or evaluation: post-outcome features, scaling or feature selection fitted before splitting, random splits of time series, duplicates across splits. Prevent with point-in-time features, split-first pipelines and time/group splits. (§1.10, §4.8)
6. **Explain the bias–variance trade-off.** → Test error = bias² + variance + noise. Simple models underfit (high bias), flexible ones overfit (high variance); total error is U-shaped in complexity. (§6.7)
7. **How do you detect and fix overfitting?** → Train score much better than validation; fix with more data, regularisation, a simpler model, early stopping, bagging or augmentation. (§1.10, §6.7)
8. **Explain gradient descent and the learning rate.** → Repeat $w \leftarrow w - \eta \nabla L$. Too small is slow, too large diverges; mini-batches are the practical default; scale features for a rounder loss surface. (§2.5, §5.4)
9. **Explain Bayes' theorem with an example.** → Posterior ∝ likelihood × prior. A 90%-sensitive, 5%-false-positive test for a 1%-prevalence disease gives only ~15% P(sick | positive) — the base-rate effect behind low precision on rare events. (§2.7)

10. **What is a p-value — and what isn't it?** → P(data at least this extreme | H_0). It is not P(H_0 is true); significance ≠ importance; correct for multiple tests and don't peek. (§2.11)
11. **Correlation vs causation?** → Correlation can come from confounders, reverse causation or chance; causal claims need randomised experiments. Models find correlations — fine for prediction, risky for decisions. (§2.9)
12. **What is cross-entropy and why is it the classification loss?** → −ln of the probability given to the true class; it's the maximum-likelihood loss, convex for logistic regression, and punishes confident mistakes hardest, with gradient p − y at the logits. (§2.12, §10.3)

Data

13. **How do you handle missing values?** → Ask why they're missing (MCAR/MAR/MNAR); median or "missing" category plus a missing indicator as a default; KNN/iterative imputation when other columns predict it; fit imputers inside the pipeline; measure by masking known values. (§4.3)
14. **One-hot vs target encoding vs hashing?** → One-hot for small nominal sets; target encoding for high cardinality (cross-fitted and smoothed to avoid leakage); hashing for unbounded IDs (collisions); embeddings in deep models. (§4.4)
15. **Which models need feature scaling?** → Anything using distances, gradients or penalties: k-NN, k-means, SVM, linear/logistic with regularisation, neural nets, PCA. Trees don't. Fit the scaler on training data only. (§4.5)
16. **How do you deal with class imbalance?** → Change the metric (precision/recall, PR-AUC); move the threshold; class weights or downsampling (then recalibrate: $p \cdot w/(p \cdot w + 1 - p)$); SMOTE sometimes; resample training data only. (§4.9)
17. **Where do labels come from, and what goes wrong?** → Outcomes (late), behaviour (biased by exposure and position), human annotation (costly, needs agreement checks), weak supervision and LLMs (correlated errors). (§4.1, §16.3)

Classic models

18. **Linear vs logistic regression?** → Linear predicts a number by minimising MSE; logistic predicts a probability through a sigmoid by minimising log loss; its boundary is $w \cdot x + b = 0$ and e^w is an odds ratio. (§5.2, §5.8)
19. **Normal equation vs gradient descent?** → Exact $\theta = (X^TX)^{-1}X^Ty$, $O(d^3)$, no tuning — fine for small d; GD scales to huge n and d and any differentiable loss but needs a learning rate and scaled features. (§5.3)
20. **Ridge vs Lasso?** → L2 shrinks all weights and shares weight among correlated features; L1 sets some weights exactly to zero (feature selection). ElasticNet mixes both. More regularisation → more bias, less variance. (§5.7)
21. **How does a decision tree choose a split?** → It tries every feature and threshold and keeps the split with the largest impurity decrease (Gini or entropy; variance for regression); greedy, then prune by depth/leaf size or ccp_alpha. (§7.1)
22. **Bagging vs boosting?** → Bagging trains deep trees in parallel on bootstrap samples and averages (cuts variance); boosting adds shallow trees sequentially, each fitting the current errors (cuts bias). (§7.2–7.4)
23. **Random forest vs gradient boosting?** → RF: robust, little tuning, doesn't overfit by adding trees, free OOB score. GBDT: usually the most accurate on tables but needs a small learning rate and early stopping. (§7.3, §7.4)
24. **Why can feature importance mislead?** → Impurity importance is computed on training data, favours high-cardinality features and splits credit among correlated ones; use permutation importance on held-out data or SHAP. (§7.3, §19.5)
25. **Explain SVMs and the kernel trick.** → Maximise the margin $2/\|w\|$ (C trades width for violations); the dual needs only dot products, so a kernel $K(x, z) = \varphi(x) \cdot \varphi(z)$ gives non-linear boundaries without building φ. Tune C and γ together. (§8.2)
26. **Why is Naive Bayes "naive", and why does it still work?** → It assumes features are independent given the class; classification only needs the argmax right, so it works well for text, but its probabilities are over-confident. (§8.3)
27. **How does k-NN work and when does it fail?** → Vote/average of the k closest scaled points; prediction is O(nd) and distances concentrate in high dimensions (curse of dimensionality). (§8.1)
28. **How does k-means work and how do you choose k?** → Assign to nearest centroid, recompute means, repeat (local optimum; k-means++ and restarts). Choose k by silhouette, elbow and domain sense; it assumes round, similar clusters. (§9.1)
29. **What does PCA do?** → Rotates standardised data onto orthogonal directions of maximum variance (eigenvectors of the covariance / SVD); keep enough components for e.g. 95% variance. Linear and unsupervised. (§9.5)

Evaluation

30. **Precision vs recall — which do you optimise?** → Depends on error costs: precision when false alarms hurt, recall when misses hurt; set the threshold where $p > C_FP/(C_FP + C_FN)$ with calibrated scores, or use two thresholds with a review band. (§6.3)
31. **ROC-AUC vs PR-AUC?** → ROC-AUC = P(a positive outranks a negative), insensitive to base rate; with rare positives, FPR's huge denominator hides false alarms, so PR-AUC (no-skill = base rate) is the honest metric. (§6.3, §16.7)
32. **What is calibration and when does it matter?** → Predicted probabilities matching observed frequencies — separate from ranking. Needed when scores feed arithmetic (auctions, expected cost, human-read risk scores). Fix with Platt or isotonic on held-out data. (§6.5, §18.1)
33. **Explain k-fold cross-validation and its variants.** → k train/validate rounds averaged (mean ± std); stratified for classes, group folds for entities, forward-in-time splits with a gap for temporal data; nested CV for an unbiased tuned estimate. (§6.6)
34. **Grid vs random search vs Bayesian optimisation?** → Grid for 1–2 parameters; random search explores each dimension better on the same budget; Bayesian (Optuna) models score vs hyperparameters for expensive trials; log scales; test once. (§6.9)
35. **Explain NDCG.** → DCG sums graded relevance discounted by $\log_2(\text{rank} + 1)$; dividing by the ideal DCG normalises to [0, 1]. Rewards putting the most relevant items at the top. (§6.4, §17.3)

Deep learning

36. **How does backpropagation work?** → The chain rule applied from the loss backwards through the computational graph, reusing intermediate values, giving every gradient for about the cost of an extra forward pass; an optimizer then updates the weights. (§10.4)
37. **Why do we need non-linear activations, and which ones?** → Without them any stack is one linear map. ReLU for hidden layers (GELU/SiLU in transformers); sigmoid/softmax/identity at the output by task. (§10.2)
38. **Vanishing and exploding gradients — causes and fixes?** → Gradients are products of per-layer (or per-time-step) factors. Fix with ReLU, He/Xavier initialisation, normalisation, residual connections, gating (LSTM/GRU) and gradient clipping. (§10.6, §12.3)

39. **SGD vs momentum vs Adam vs AdamW?** → Momentum accumulates velocity; RMSProp/Adam scale each parameter's step by its recent gradient size (Adam adds bias correction); AdamW decouples weight decay. Adam(W) is the default for transformers; SGD + momentum stays strong for CNNs. (§10.5)
40. **What do dropout and batch normalisation do?** → Dropout zeroes random units in training so the net can't rely on co-adapted features (off at eval); BatchNorm normalises activations per mini-batch, allowing higher learning rates and adding mild regularisation (running stats at eval). (§10.7)
41. **Why CNNs for images?** → Weight sharing and local connectivity give far fewer parameters and translation equivariance; stacked 3×3 convs with downsampling grow the receptive field; ResNet skip connections make very deep nets trainable. (§11.1, §11.4)
42. **What is transfer learning?** → Start from a model pretrained on a big dataset, replace the head, freeze then fine-tune with a small learning rate; biggest gains when labels are scarce. (§11.5)
43. **How does an LSTM fix the RNN's problems?** → An additive cell state $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$ controlled by sigmoid gates lets information and gradients flow across many steps. (§12.4)
44. **Explain attention and the transformer.** → softmax($QK^T/\sqrt{d_k}$)V is a learned weighted lookup; transformers stack multi-head self-attention and feed-forward layers with residuals and LayerNorm plus positional encodings — parallel over the sequence, O(1) path between tokens, $O(n^2)$ cost. (§12.6, §12.7)

Recommenders, RL and systems

45. **Content-based vs collaborative filtering, and cold start?** → Attributes vs co-behaviour; CF finds quality and surprise but fails on new users/items. Cold start: popularity, onboarding, content features and controlled exploration. (§13.4, §13.10)
46. **How do you retrieve candidates from 100 M items in milliseconds?** → Two-tower model: item vectors precomputed into an ANN index (HNSW/IVF), user tower per request; in-batch negatives with logQ correction; tower and index shipped as one version; then a ranker. (§13.7–13.9)
47. **Q-learning vs SARSA? Exploration vs exploitation?** → Q-learning bootstraps from the max next action (off-policy, learns the optimal policy); SARSA from the action actually taken (on-policy, safer while exploring). Exploration: ε-greedy, UCB, Thompson sampling. (§14.4, §14.7)
48. **Walk me through designing an ML system.** → Frame → success metrics → data and labels → baseline → serving path → keep-it-working loop → failures and cost, translating the business goal into an observable ML objective and keeping product knobs (thresholds, buffers) outside the model. (§15.3)
49. **How do you monitor a model in production, and when do you retrain?** → Health → input drift (PSI/KS/nulls) → prediction drift → outcomes; alerts trigger investigation (often a broken pipeline); retrain on measured decay through offline, shadow and canary gates. (§16.8)
50. **Your model is 94% accurate — is it fair?** → Unknown until you slice by group: selection rate, TPR, FPR, PPV and calibration per group, proxy checks; criteria conflict when base rates differ, so choose by harm and mitigate (reweighing, penalties, thresholds). (§19.2–19.4, §19.12)

20.18 Glossary (A–Z)

352 terms. Each entry: **term** — plain meaning — the section that teaches it (the link opens that chapter).

Term	Meaning	Where
A/B test	Randomised experiment splitting units between control and treatment to measure a business metric	16.9
Accuracy	Share of predictions that are correct, (TP + TN)/all; misleading on imbalanced data	6.3
Action ladder	Graded moderation responses (remove, review, reduce reach/age-gate/warn, allow) set per category and region	18.3
Activation function	Non-linear function applied to a neuron's pre-activation (ReLU, sigmoid, tanh, GELU)	10.2
Active learning	Routing the examples with the most model uncertainty (or annotator disagreement) to human labellers first	4.1
Actor-critic	Policy-gradient method where a critic's TD error δ serves as the advantage for the actor's update	14.9
AdaBoost	Boosting by re-weighting misclassified samples; each stump gets a vote α = ½ ln((1 − ε)/ε)	7.4
Adam	Optimizer combining momentum and RMSProp-style per-parameter step sizes, with bias correction	10.5
Adamic-Adar	Link-prediction score summing 1/log(degree) over shared neighbours, so rare mutual friends count more	18.4
AdamW	Adam with decoupled weight decay applied directly to the weights	10.5
Adjusted cosine	Cosine similarity after subtracting each user's mean rating	13.3
Adjusted Rand Index (ARI)	Agreement between two clusterings corrected for chance: about 0 random, 1 perfect	9.9
Adversarial example	Input perturbed slightly, on purpose, so that a model misclassifies it	19.7
Adversarial training	Including adversarial examples in each training batch to gain robustness	19.7
Agentic AI	Systems that plan and take multi-step actions toward a goal using tools	1.1
Agglomerative clustering	Bottom-up clustering that repeatedly merges the two closest clusters	9.2
ALS (alternating least squares)	Matrix-factorisation solver that fixes one side and solves a ridge regression for the other, alternately	13.5
ANN (approximate nearest neighbour)	Search that returns almost the exact nearest vectors while scanning only a fraction of them	13.9
Anomaly detection	Finding rare points that differ markedly from the majority, usually without labels	9.7
Apriori	Frequent-itemset algorithm that prunes candidates because every subset of a frequent set must be frequent	9.8
ARIMA	Time-series model: autoregression on p lags, d differences, moving average on q past errors	12.10
Attention	Weighted average of value vectors with weights softmax(q·k/√d_k) from query–key similarity	12.6
AUC (ROC-AUC)	Probability a random positive is scored above a random negative; 0.5 random, 1 perfect	6.3
Autograd	Automatic differentiation that records tensor operations and runs backprop on `.backward()`	10.9
Automatic speech recognition (ASR)	Converting speech audio into a (timestamped) transcript, e.g. to make videos searchable	17.4
Average precision (AP)	Area under the precision–recall curve, the usual PR-AUC; no-skill value equals the base rate	6.3
Backpropagation	Chain rule applied from the loss backwards to get every parameter's gradient efficiently	10.4
Backpropagation through time (BPTT)	Backprop through an RNN unrolled over the sequence; gradients summed over time steps	12.2
Bagging	Training copies of a model on bootstrap samples and averaging them to cut variance	7.3
Bandit (multi-armed)	One-state RL problem of choosing among arms with unknown payoffs; isolates exploration vs exploitation	14.4
Baseline	Score of a trivial predictor (majority class, mean, last value, popularity) every model must beat	6.1
Base rate	How often the positive class occurs in the population; drives precision and PR-AUC	6.3
Batch inference	Predictions precomputed on a schedule and stored so a request becomes a lookup	16.5
Batch learning	Training on the full dataset at once and redeploying a frozen model periodically	1.7
Batch normalisation	Layer normalising activations over the mini-batch, then scaling and shifting with learned γ and β	10.7
Bayesian optimisation	Tuning with a surrogate model of score vs hyperparameters and an acquisition function	6.9
Bayes' theorem	P(A|B) = P(B|A)·P(A)/P(B): posterior ∝ likelihood × prior	2.7
Beam search	Decoding that keeps the k most probable partial sequences at each step	12.5
Bellman equation	Value now = expected reward now + γ × value of the next state	14.3
Bias (fairness)	Systematic, unjustified difference in how a model treats or performs for groups	19.1
Bias (statistical)	Error from wrong assumptions; distance of the average model from the truth	6.7
BIC	Bayesian Information Criterion: likelihood penalised by parameter count, lower is better (choose k in GMMs)	9.4
BM25	Lexical relevance score rewarding rare query terms, with diminishing returns and length normalisation	17.3
Boosting	Building an ensemble sequentially, each weak model correcting the current errors	7.4

Term	Meaning	Where
Bootstrap sample	n rows drawn with replacement from n rows; about 36.8% of rows are left out	7.3
BPE (byte-pair encoding)	Subword tokenizer that repeatedly merges the most frequent adjacent symbol pair	12.8
Brier score	Mean squared error of predicted probabilities; a proper scoring rule for calibration	6.5
Broadcasting	NumPy rule for combining arrays of different shapes by stretching size-1 dimensions	3.2
Calibration	Predicted probabilities matching observed frequencies; separate from ranking quality	6.5
Canary release	Giving a new model to a small slice of real traffic before ramping up	16.6
Candidate generation	First funnel stage that retrieves a few hundred plausible items from millions, judged by recall	13.8
Cascade	Cheap checks on everything, expensive checks only on what earlier stages flag	18.3
CatBoost	Gradient-boosting library with ordered boosting and strong native categorical handling	7.4
Censored label	Outcome never observed because your own action (e.g. blocking a payment) prevented it	18.2
Central limit theorem (CLT)	Sample means are approximately normal around the true mean with spread σ/√n	2.10
Chain rule	Derivative of nested functions = product of the local derivatives	2.4
Classification	Supervised task predicting a category	1.5
Class imbalance	One class much rarer than the other(s), as in fraud or disease	4.9
Class weights	Loss weights that make mistakes on the rare class cost more	4.9
CLIP	Image–text model trained contrastively so images and captions share one embedding space	18.6
Cohen's kappa	Chance-corrected agreement between two annotators, (p_o − p_e)/(1 − p_e)	4.1
Cold start	Recommending with little or no history for a new user, item or system	13.10
Collaborative filtering	Recommending from other users' behaviour: people who liked what you liked also liked X	13.4
ColumnTransformer	scikit-learn tool applying different preprocessing to different column groups	3.5
Concept drift	Change in the relationship between inputs and the label, P(Y|X)	16.8
Confidence interval	Range built by a procedure that captures the true value a stated share of the time	2.11
Confusion matrix	Table of TP, FP, FN and TN counts from which classification metrics are computed	6.3
Content-based filtering	Recommending items whose attributes resemble what the user liked	13.4
Contextual bandit	Bandit whose choice depends on context features (e.g. LinUCB)	14.4
Contrastive loss	Loss pulling matching pairs together and pushing non-matching pairs apart (e.g. InfoNCE)	18.6
Convolution	Sliding a small shared kernel over an input and summing element-wise products into a feature map	11.1
Correlation	Strength of co-movement; Pearson for linear, Spearman for monotonic (ranks)	2.9
Cosine similarity	Dot product divided by both lengths; compares direction only	2.1
Cost-complexity pruning	Removing tree branches that don't pay for their leaves, controlled by ccp_alpha	7.1
Counterfactual explanation	Smallest actionable input change that flips a decision	19.5
Covariance matrix	Matrix of how each pair of features varies together; PCA takes its eigenvectors	9.5
Cross-attention	Attention with queries from the decoder and keys/values from the encoder	12.7
Cross-entropy	Average surprise −Σ p log q when reality is p and the prediction is q; the classification loss	2.12
Cross-validation	Rotating the validation role across k folds and averaging the scores	6.6
Curse of dimensionality	In high dimensions space empties and distances concentrate, so "nearest" loses meaning	8.1
Cyclical encoding	Encoding periodic values as (sin, cos) so December sits next to January	4.6
Data augmentation	Creating extra training examples with label-preserving transformations	10.7
Data drift	Change in the input distribution P(X) (covariate shift)	16.8
Data leakage	Information unavailable at prediction time reaching training or evaluation	4.8
Data poisoning	Attack that corrupts training data, possibly planting a backdoor trigger	19.7
Datasheet for datasets	Document describing how a dataset was created, what it contains and how it should be used	19.8
DBSCAN	Density-based clustering: core points with ≥ min_samples neighbours within eps; others border or noise	9.3
Deadly triad	Function approximation + bootstrapping + off-policy learning, which can make RL diverge	14.8
Decision boundary	Where the classifier's decision changes; for logistic regression, w·x + b = 0	5.8
Decision tree	Model predicting by a sequence of feature-threshold questions ending in leaves	7.1
Deep learning	Machine learning with many-layer neural networks that learn their own features	1.1

Term	Meaning	Where
Delayed feedback	Labels (conversions, chargebacks) arriving long after the prediction	18.1
Demographic parity	Fairness criterion requiring equal approval (selection) rates across groups	19.2
Dendrogram	Tree of agglomerative merges; slicing it at a chosen height yields the clusters	9.2
Differential privacy	Guarantee that an algorithm's output barely changes whether any one person is in the data (parameter ε)	19.6
Dimensionality reduction	Describing data with fewer numbers (PCA, t-SNE, UMAP)	9.5
Discount factor (γ)	Weight on future rewards; horizon ≈ 1/(1 − γ)	14.2
Disparate impact ratio	Selection rate of one group divided by another's; below 0.8 fails the four-fifths rule of thumb	19.2
Distillation	Training a small student model to mimic a large teacher	16.6
DPO	Direct Preference Optimization: trains a policy on preference pairs without a separate reward model or RL loop	14.10
DQN	Deep Q-Network: neural Q-function trained with experience replay and a target network	14.8
Dropout	Randomly zeroing units during training so the network can't rely on co-adapted features	10.7
Dummy-variable trap	One-hot columns summing to 1 are collinear with the intercept; drop one for unregularised OLS	4.4
Dynamic programming (RL)	Computing optimal values from a known model by repeated Bellman backups	14.5
Early stopping	Halting training when validation loss stops improving for a set patience	6.10
ε-greedy	Explore a random action with probability ε, otherwise exploit the best known	14.4
Eigenvector	Direction a matrix only stretches, $Av = \lambda v$; λ is the eigenvalue	2.3
ElasticNet	Linear regression with a mix of L1 and L2 penalties	5.7
Embedding	Short dense learned vector representing a discrete thing (word, user, item)	13.6
Ensemble	Combination of several models' predictions	7.5
Entropy	Average uncertainty of a distribution, $-\Sigma\ p \log_2 p$ (bits)	2.12
Epoch	One full pass over the training data	2.5
Equalized odds	Fairness criterion requiring equal TPR and FPR across groups	19.2
Equal opportunity	Fairness criterion requiring equal true-positive rates across groups	19.2
EU AI Act	EU regulation sorting AI systems into risk tiers; credit scoring and hiring are high-risk	19.9
Experience replay	Storing transitions and training on random mini-batches of them to decorrelate updates	14.8
Explainability	Ability to say why a model made a prediction in human-checkable terms	19.5
Exploration–exploitation	Trade-off between trying options to learn and using the best-known option	14.4
Exposure bias	Logs only show outcomes for items the system chose to show	4.1
F1 score	Harmonic mean of precision and recall	6.3
Fail closed	On failure, block the action (don't publish, don't suggest) rather than proceed; used where errors harm people	18.7
Feature	Measurable input property used by a model	1.6
Feature engineering	Creating new inputs that make the pattern easier to learn	4.6
Feature hashing	Mapping categories to a fixed number of buckets with a hash function (hashing trick)	4.4
Feature importance	Score of how much each feature contributes (impurity-based, permutation, SHAP)	7.3
Feature map	Output grid of one convolution kernel applied across the input	11.1
Feature store	System serving consistent features: offline history for training, online latest values for serving	16.4
Federated learning	Training across devices or institutions by averaging locally trained model updates (FedAvg)	19.6
Feedback loop	A model's outputs shaping its own future training data	16.8
FGSM	Fast Gradient Sign Method: $x + \varepsilon \cdot \text{sign}(\nabla_x \text{ loss})$ creates an adversarial example	19.7
Fine-tuning	Continuing training of a pretrained model on your task with a small learning rate	11.5
Forget gate	LSTM gate deciding which cell-state entries to keep or erase	12.4
FP-Growth	Frequent-itemset mining without candidate generation, using a prefix tree	9.8
Funnel	Multi-stage pipeline: cheap high-recall retrieval, heavy ranking, rule-based re-ranking	13.8
GELU	Smooth ReLU-like activation $z \cdot \Phi(z)$ used in transformers	10.2
Generalisation	Performing well on new, unseen data	1.3

Term	Meaning	Where
Generative AI	Models that produce new content such as text, images, audio or code	1.1
Gini impurity	$1 - \Sigma\, p_k^2$, the chance of mislabelling a random sample from the node's class mix	7.1
Global average pooling	Averaging each feature map to one number before the classifier head	11.2
GMM (Gaussian mixture model)	Clustering as a mix of Gaussians fitted with EM, giving soft assignments	9.4
Gradient	Vector of partial derivatives; points in the direction of steepest increase	2.4
Gradient boosting	Boosting where each new tree fits the negative gradient of the loss (residuals for MSE)	7.4
Gradient clipping	Rescaling gradients whose norm exceeds a threshold to prevent explosions	10.6
Gradient descent	Minimising a loss by repeated steps $w \leftarrow w - \eta \nabla L$	2.5
Graph neural network	Neural network over a graph of entities, used e.g. to spot fraud rings	18.2
Grid search	Trying every combination of listed hyperparameter values with cross-validation	6.9
Group split	Keeping all rows of one entity in the same split	4.7
GRU	Gated recurrent unit with update and reset gates; lighter than an LSTM	12.4
Guardrail metric	Metric that must not get worse in an experiment, even if the primary improves	16.9
Hallucination	Generated content that is false or unsupported	15.2
He initialisation	Weight variance 2/fan_in, designed for ReLU layers	10.6
Hierarchical clustering	Clustering that builds a nested tree of merges (see agglomerative)	9.2
Hinge loss	$\max(0, 1 - y \cdot f(x))$; the SVM loss	8.2
HNSW	Hierarchical Navigable Small World graph index for approximate nearest-neighbour search	13.9
Holdback	Small slice kept on the old experience for months to measure long-term effects	16.9
Huber loss	Quadratic for small errors, linear for large; robust regression and DQN	10.3
Hyperparameter	Setting chosen before training, e.g. learning rate, depth or C	1.6
Hypothesis test	Checking whether an observed effect could plausibly be noise under a null hypothesis	2.11
Imputation	Filling missing values (mean, median, KNN, iterative)	4.3
In-batch negatives	Using the other items in a training batch as negatives for each user	13.7
Inference	Using a trained model to make predictions	1.3
Information gain	Entropy of the parent minus the weighted entropy of the children after a split	7.1
Instance-based learning	Predicting from stored similar examples rather than a fitted formula (e.g. k-NN)	1.8
Interleaving	Merging two rankers' results into one list and seeing whose items get clicked	16.9
Inverse propensity weighting (IPW)	Weighting each logged example by 1/P(it was exposed/examined) to remove selection or position bias	17.3
IoU (intersection over union)	Overlap of two boxes divided by their union; matches detections to ground truth	11.7
Isolation Forest	Anomaly detector: anomalies are isolated by random splits in fewer steps	9.7
Isotonic regression	Non-decreasing step-function calibration mapping; needs more data than Platt	6.5
Item2vec	word2vec's skip-gram applied to sessions or playlists to learn item embeddings	13.6
Jaccard similarity	$\lvert A \cap B \rvert \,/\, \lvert A \cup B \rvert$ for sets	13.3
k-anonymity	Every quasi-identifier combination shared by at least k records	19.6
Kernel trick	Replacing dot products with a kernel $K(x, z) = \varphi(x) \cdot \varphi(z)$ without computing φ	8.2
k-fold cross-validation	Cross-validation with k equal folds, each used once for validation	6.6
KL divergence	Extra surprise from predicting with q instead of p; non-negative and asymmetric	2.12
k-means	Clustering by alternately assigning points to the nearest centroid and recomputing centroids	9.1
k-means++	Initialisation picking centroids with probability $\propto$ squared distance to those already chosen	9.1
k-nearest neighbours (k-NN)	Predicting by majority vote or average of the k closest training points	8.1
KS test	Kolmogorov–Smirnov: maximum gap between two empirical CDFs, used for drift	16.8
L1 regularisation	Penalty $\lambda \Sigma \lvert w \rvert$ that drives some weights exactly to zero	5.7
L2 regularisation	Penalty $\lambda \Sigma w^2$ that shrinks all weights smoothly (weight decay)	5.7
Label	Target value to predict (y)	1.6
Label encoding	Mapping categories to integer codes; `LabelEncoder` is meant for targets	4.4

Term	Meaning	Where
Label shift	Change in the outcome distribution P(Y)	16.8
Laplace mechanism	Adding Laplace noise with scale sensitivity/ε to achieve ε-differential privacy	19.6
Laplace smoothing	Adding α (usually 1) to every count so unseen words don't get zero probability	8.3
Lasso	Linear regression with an L1 penalty; performs feature selection	5.7
Latency budget	Milliseconds the whole prediction path may take, itemised by stage	16.6
LayerNorm	Normalisation across the features of each example; standard in transformers	10.7
Leaky ReLU	ReLU variant with a small slope α for negative inputs	10.2
Learning curve	Training and validation score vs training-set size	6.8
Learning rate	Step size η of gradient descent	2.5
Learning-rate schedule	Changing the learning rate during training (step, cosine, warmup, one-cycle)	10.5
Learning to rank	Training models to order items (pointwise, pairwise, listwise losses)	17.3
Lift	Confidence(A → B) divided by support(B); above 1 means a positive association	9.8
LightGBM	Gradient-boosting library with leaf-wise histogram trees, GOSS and EFB	7.4
LIME	Local explanation fitting a sparse linear surrogate around one prediction	19.5
Linear regression	Predicting a number as w·x + b by minimising squared error	5.2
Local Outlier Factor (LOF)	Anomaly score comparing a point's local density with its neighbours'	9.7
Logistic regression	Linear score passed through a sigmoid to give a class probability	5.8
Logit	Raw pre-sigmoid or pre-softmax score; for logistic regression, the log-odds	5.8
Log loss	Negative log-likelihood of labels under predicted probabilities; binary cross-entropy	5.8
logQ correction	Subtracting log(sampling probability) from logits so popular items aren't over-penalised as negatives	13.7
LSTM	Recurrent cell with an additive cell state and forget, input and output gates	12.4
MAE	Mean absolute error; optimal constant prediction is the median	6.2
MAP	Mean average precision over queries or users	6.4
Matrix factorisation	Approximating the user–item matrix by user and item factor vectors whose dot product predicts affinity	13.5
Max pooling	Downsampling by keeping the largest value in each window	11.2
Membership inference	Attack deciding whether a record was in the training set	19.7
Mini-batch	Small subset of examples used for one gradient update	5.4
Minimum detectable effect (MDE)	Smallest lift an experiment is sized to detect; sample size ∝ 1/MDE²	16.9
MLOps	DevOps practices (versioning, testing, CI/CD, monitoring) applied to data and models	16.10
MLP (multi-layer perceptron)	Feed-forward network of fully connected layers with non-linear activations	10.1
MMR (maximal marginal relevance)	Greedy re-ranking trading relevance against similarity to items already chosen	13.8
Model card	Short document with a model's intended use, per-group evaluation and limits	19.8
Model extraction	Copying a model by querying its API and training a surrogate	19.7
Model registry	Store of model versions with metrics, lineage, stages and owners	16.10
Momentum	Optimizer that accumulates a velocity of past gradients	10.5
Monte Carlo (RL)	Updating value estimates toward the actual return at the end of an episode	14.6
MRR (mean reciprocal rank)	Mean of 1/rank of the first relevant result	6.4
MSE	Mean squared error; optimal constant prediction is the mean	6.2
Multicollinearity	Features that are near-copies of each other, making coefficients unstable	5.5
Multi-head attention	Several attention heads in parallel subspaces, concatenated and projected	12.7
Multi-task learning	One shared network body with a head per task or event	17.2
Naive Bayes	Bayes' theorem with features assumed independent given the class	8.3
NDCG	Normalised discounted cumulative gain: position-discounted graded relevance ÷ ideal	6.4
Negative sampling	Training against a few sampled negatives instead of a full softmax	12.9
Nested cross-validation	Outer CV loop around the whole tuning procedure for an unbiased estimate	6.6
NMS (non-maximum suppression)	Keeping the most confident detection and removing boxes that overlap it above an IoU threshold	11.7
Normal equation	Closed-form least-squares solution $\theta = (X^T X)^{-1} X^T y$	5.3

Term	Meaning	Where
Novelty effect	Temporary change in behaviour because something is new; fades over time	16.9
Odds ratio	e^w for a logistic-regression coefficient: factor by which the odds change per unit	5.8
Off-policy	Learning about one policy (e.g. greedy) while following another (e.g. ε-greedy)	14.7
One-hot encoding	One 0/1 column per category	4.4
One-vs-rest	K binary classifiers, one per class against the rest	5.9
Online learning	Updating a model continuously from small chunks of data (`partial_fit`)	1.7
Out-of-bag (OOB) score	Validation from the rows each bootstrap tree never saw	7.3
Overfitting	Fitting training noise; low training error, much higher validation error	1.10
Partial dependence (PDP)	Average prediction as one feature is varied over a grid	19.5
PCA	Projection onto orthogonal directions of maximum variance	9.5
Perceptron	Single neuron with a step activation; can't learn XOR	10.1
Permutation importance	Drop in held-out score when one feature's values are shuffled	7.3
Pipeline	scikit-learn object chaining preprocessing and a model into one estimator	3.5
Platt scaling	Calibration by fitting a logistic curve on the model's scores	6.5
Point-in-time join	Joining each training example to feature values as they were at its timestamp	4.8
Policy	Rule mapping states to actions (deterministic or stochastic)	14.3
Policy gradient	Gradient ascent on expected return, raising the log-probability of good actions	14.9
Pooling	Parameter-free downsampling of feature maps (max or average)	11.2
Popularity bias	Recommenders over-recommending already-popular items	13.12
Positional encoding	Position information added to token embeddings because attention ignores order	12.7
Position bias	Items shown higher get more clicks regardless of relevance	17.3
PPO	Proximal Policy Optimization: actor-critic that clips how far each update moves the policy	14.9
PR-AUC	Area under the precision–recall curve; the metric for rare positives	6.3
Precision	TP/(TP + FP): share of flagged items that are truly positive	6.3
Prevalence (of violating views)	Share of all views that land on violating content, estimated by sampling	18.3
Principal component	New axis in PCA; a linear combination of the original features	9.5
Product quantisation (PQ)	Compressing vectors into short codes for memory-efficient approximate search	13.9
Proxy feature	Feature that stands in for a sensitive attribute (e.g. ZIP region for group)	19.3
Pruning (trees)	Stopping growth early or cutting back branches to reduce overfitting	7.1
Pseudonymisation	Replacing identifiers (e.g. hashing IDs); still personal data, not anonymous	19.6
PSI (population stability index)	Σ(cur − ref)·ln(cur/ref) over bins; > 0.25 means investigate	16.8
p-value	Probability of data at least this extreme if the null hypothesis were true	2.11
Q-learning	Off-policy TD control updating Q toward r + γ·max Q(s', ·)	14.7
Quantisation	Storing weights/activations with fewer bits (e.g. int8) to shrink and speed up a model	16.6
R^2	Coefficient of determination: share of target variance explained, 1 − SSE/SST	6.2
Random forest	Bagged decision trees with a random subset of features considered at each split	7.3
Random search	Tuning by sampling hyperparameter combinations from distributions	6.9
Ranking	Ordering items for a user or query; also the precise funnel stage after retrieval	13.8
Recall	TP/(TP + FN): share of true positives that were caught (sensitivity)	6.3
Recall@k	Share of relevant items that appear in the top k	6.4
Receptive field	Region of the input that can influence a unit	11.2
Reciprocal rank fusion (RRF)	Merging ranked lists by summing 1/(k + rank) per list (k ≈ 60); needs no training or score calibration	17.4
Recommender system	System predicting which items a user will want from a large catalogue	13.1
Regression	Supervised task predicting a number	1.5
Regularisation	Anything that limits a model's freedom to fit noise (penalties, dropout, early stopping)	5.7
Reinforcement learning	Learning a policy by trial and error to maximise long-run reward	14.1

Term	Meaning	Where
Reliability diagram	Plot of mean predicted probability vs observed frequency per bin	6.5
ReLU	max(0, z); the default hidden-layer activation	10.2
Residual	Difference between a true value and the prediction	5.5
Residual connection	Skip path y = x + F(x) that gives gradients an identity highway	11.4
Retrieval	Candidate generation: narrowing millions of items to hundreds, graded on recall	17.1
Return (RL)	Discounted sum of future rewards $G_t = r_{t+1} + \gamma G_{t+1}$	14.2
Reward hacking	Agent maximising the reward as written rather than the intended goal	14.12
Reward model	Model trained on human preference comparisons to score responses in RLHF	14.10
Reweighing	Weighting (group, label) cells so group and label are independent in the weighted data	19.4
Ridge regression	Linear regression with an L2 penalty	5.7
RLHF	Reinforcement learning from human feedback: SFT, reward model, then PPO with a KL penalty	14.10
RMSE	Root mean squared error, in the target's units; weighs big misses more	6.2
RMSProp	Optimizer dividing each step by a running RMS of recent gradients	10.5
RNN (recurrent neural network)	Network that reads a sequence step by step, carrying a hidden state	12.2
SARSA	On-policy TD control updating Q toward r + γ·Q(s', a') for the action actually taken	14.7
Scaled dot-product attention	$\text{softmax}(QK^T/\sqrt{d_k})V$	12.6
Seasonal naive	Forecast repeating the value from one season ago (e.g. same weekday last week)	12.10
Self-attention	Attention where queries, keys and values all come from the same sequence	12.7
Self-supervised learning	Learning from labels generated from the data itself (e.g. next-token prediction)	1.4
Semi-supervised learning	Learning from a few labelled examples plus many unlabelled ones	1.4
Seq2seq	Encoder–decoder model mapping an input sequence to an output sequence	12.5
Session embeddings	Item vectors learned by skip-gram over browsing sessions, so items considered together are close	18.9
Shadow deployment	New model scores live traffic in parallel; outputs are logged but not used	16.6
SHAP	Shapley-value attributions splitting one prediction into additive feature contributions	19.5
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$, squashing a score into (0, 1)	2.4
Silhouette score	Average over points of (b − a)/max(a, b), where a = own-cluster distance and b = nearest-other-cluster distance	9.1
SMOTE	Oversampling by creating synthetic minority points between minority neighbours	4.9
Softmax	$e^{z_k}/\Sigma e^{z_j}$: turns K scores into probabilities that sum to 1	5.9
Sparsity (user–item matrix)	Most user–item cells are empty because each user touches few items	13.1
Stacking	Meta-model trained on out-of-fold predictions of diverse base models	7.5
Standard error	Spread of a sample statistic, σ/√n for a mean	2.10
Standardisation	Rescaling to mean 0, std 1: z = (x − μ)/σ	2.8
Stochastic gradient descent (SGD)	Gradient descent using one example or a mini-batch per step	2.5
Stratified split	Split keeping class proportions equal in every part	4.7
Stride	How many pixels a kernel moves between positions	11.1
Supervised learning	Learning from inputs paired with correct labels	1.4
Support (association rules)	Fraction of baskets containing an itemset	9.8
Support vector	Training point on or inside the margin that defines an SVM's boundary	8.2
SVM (support vector machine)	Classifier finding the maximum-margin hyperplane, optionally with kernels	8.2
Target encoding	Replacing a category with a smoothed, out-of-fold mean of the target	4.4
Target network	Frozen copy of the Q-network used to compute stable TD targets in DQN	14.8
Teacher forcing	Feeding the true previous token to the decoder during training	12.5
Temporal difference (TD) learning	Updating a value toward reward + γ·(estimate of next value) after every step	14.6
Test set	Data used once at the end to estimate real-world performance	1.6
TF-IDF	Term frequency × inverse document frequency: down-weights words common across documents	4.6
Thompson sampling	Sampling each arm's payoff from its posterior and pulling the best sample	14.4

Term	Meaning	Where
Threshold (decision)	Score cut-off turning probabilities into decisions; sets the precision–recall trade-off	6.3
Time-based split	Training on the past and testing on the future	4.7
Tokenization	Splitting text into tokens mapped to integer ids from a vocabulary	12.8
Training–serving skew	Features computed differently in production than in training	16.2
Transfer learning	Reusing a model pretrained on a large task as the starting point for yours	11.5
Transformer	Architecture of stacked self-attention and feed-forward blocks with residuals and LayerNorm	12.7
t-SNE	Non-linear 2-D visualisation preserving local neighbourhoods	9.6
Two-tower model	Separate user and item encoders whose embeddings meet at a dot product	13.7
UCB (upper confidence bound)	Bandit strategy adding an optimism bonus that shrinks as an arm is tried	14.4
UMAP	Non-linear dimensionality reduction for visualisation that can transform new points	9.6
Underfitting	Model too simple to capture the pattern; training and validation error both high	1.10
U-Net	Encoder–decoder segmentation network with concatenated skip connections	11.7
Universal approximation theorem	One hidden layer with enough units can approximate any continuous function on a bounded region	10.1
Unsupervised learning	Finding structure in data without labels	1.4
Validation curve	Training and validation score vs one hyperparameter	6.8
Validation set	Data used to compare models and tune hyperparameters	1.6
Value function	Expected return from a state, V(s), or from a state–action pair, Q(s, a)	14.3
Value iteration	Repeating the Bellman optimality update until values converge	14.5
Vanishing gradient	Gradients shrinking exponentially through many layers or time steps	10.6
Variance (model)	How much the model changes when trained on a different sample	6.7
Vectorisation	Replacing Python loops with whole-array operations	3.2
Version skew	A new user tower querying an index built by an old item tower; silently degrades quality	13.7
Vision Transformer (ViT)	Transformer applied to image patches treated as tokens	11.4
Weak learner	Model only slightly better than chance, e.g. a stump, used inside boosting	7.4
Weak supervision	Many noisy labelling functions combined by a label model into training labels	4.1
Weight decay	Shrinking weights each step, equivalent to an L2 penalty for SGD	10.7
Word2vec	Shallow model learning word embeddings from context (skip-gram or CBOW)	12.9
Xavier initialisation	Weight variance 2/(fan_in + fan_out), designed for tanh and sigmoid	10.6
XGBoost	Gradient-boosting library with a regularised second-order objective	7.4
YOLO	One-stage detector predicting boxes and classes over a grid in a single pass	11.7
z-score	(x − mean)/std: how many standard deviations from the mean	2.8

Chapter TL;DR

- **20.1–20.4**: the workflow (frame → data → prepare → train → evaluate → deploy → monitor), the math you need (dot products, gradients, Bayes, entropy), the NumPy/pandas/scikit-learn API and data preparation — split first, pipeline everything, fit on training data only.
- **20.5–20.8**: every algorithm with its knobs and costs, every loss, every metric with a worked number, and how to validate and tune honestly (CV, nested CV, calibration, early stopping).
- **20.9–20.12**: deep learning (activations, optimizers, init, regularisation, the debugging checklist), CNN/RNN/attention formulas, recommenders (MF, two-tower, funnel, ANN) and the RL update rules.
- **20.13–20.15**: the 7-step system-design framework with napkin numbers and the thirteen case studies, then the production/MLOps and responsible-ML checklists to run before every launch.
- **20.16–20.17**: decision tables for picking an algorithm and a metric, and 50 interview questions with crisp answers.
- **20.18**: the glossary — follow each **§** back to the chapter where the idea is explained properly.